PrimeTime Suite Variables and Attributes

Version V-2023.12-SP1, January 2024

SYNOPSYS®

Feedback

Contents

1. **PrimeTime Suite Variables** 22

a 22

arch 22
attributes 22
auto_fixable_violations 24
auto_link_disable 26
auto_wire_load_selection 27

b 28

boundary_ideal_network_violations 28
boundary_logic_value_violations 28
bus_naming_style 30

c 31

case_analysis_log_file 31
case_analysis_propagate_through_icg 32
case_analysis_sequential_propagation 32
clock_attributes_violations 33
clock_latency_violations 35
clock_mapping_violations 36
clock_relations_violations 39
clock_skew_with_uncertainty_violations 40
clock_uncertainty_violations 42
collection_result_display_limit 43
constraint_analysis_violation_limit 44
create_clock_no_input_delay 45

d 45

data_arrival_violations 45
dbr_ignore_external_links 47
default_oc_per_lib 48
delay_calc_enhanced_ccsn_waveform_analysis 48
delay_calc_waveform_analysis_constraint_arcs_compatibility 49
delay_calc_waveform_analysis_mode 50
disable_case_analysis 50

disable_case_analysis_ti_hi_lo 51
disable_missing_object_errors_in_golden_constraints_reader 52
distributed_cleanup_variables_for_swap 52
distributed_custom_protocol_error_detection_timeout 53
distributed_enable_analysis 53
distributed_farm_check_pending_workers_interval 54
distributed_farm_protocol_error_detection_timeout 55
distributed_farm_submit_timeout 56
distributed_flush_file_system_buffers 57
distributed_heartbeat_period 57
distributed_heartbeat_timeout 58
distributed_logging 58
distributed_sh_protocol_error_detection_timeout 60
distributed_show_task_timestamp 61
distributed_worker_exit_action 62
distributed_working_directory 63
e 63
eco_allow_filler_cells_as_open_sites 63
eco_allow_insert_buffer_always_on_cells 64
eco_allow_insert_buffer_mixed_ao_regular_nets 65
eco_allow_sizing_with_lib_cell_attributes 66
eco_alternative_area_ratio_threshold 67
eco_alternative_cell_attribute_restrictions 67
eco_alternative_cell_instance_based_restrictions 68
eco_enable_fixing_clock_used_as_data 70
eco_enable_graph_based_refinement 71
eco_enable_mim 73
eco_enable_overwrite_physical_design_path 73
eco_enable_report_unfixed_reason_stdout 74
eco_endpoint_merge_session_list 75
eco_estimation_output_columns 76
eco_insert_buffer_search_distance_in_site_rows 78
eco_instance_name_prefix 79
eco_leakage_exclude_unconstrained_cells 80
eco_mim_preserve 80
eco_net_name_prefix 81
eco_physical_disable_eco_on_fixed_cells 82
eco_physical_enable_route_preservation 83

eco_physical_match_site_row_names 84
eco_power_exclude_unconstrained_cells 86
eco_power_report_max_unusable_cells 86
eco_print_unfixed_reason_to_stdout_in_csv_or_txt_format 87
eco_report_unfixed_reason_max_endpoints 88
eco_save_session_data_type 88
eco_strict_lib_arc_equivalence 90
eco_strict_pin_name_equivalence 91
eco_update_path_timing 92
eco_voltage_slack_target 92
eco_write_changes_prepend_libfile_to_libcell 93
eco_write_changes_prepend_libname_to_libcell 95
enable_golden_constraints_reader 97
enable_golden_upf 97
enable_license_auto_reduction 97
enable_page_mode 98
enable_path_tagging 98
enable_rule_based_query 99
env_variables_violations 99
extract_model_capacitance_limit 100
extract_model_ccs_keep_voltage_corner 101
extract_model_clock_latency_arcs_include_all_registers 102
extract_model_clock_transition_limit 103
extract_model_create_variation_tables 103
extract_model_data_transition_limit 104
extract_model_db_naming_compatibility 105
extract_model_enable_report_delay_calculation 106
extract_model_exclude_output_transition_constraint 106
extract_model_gating_as_nochange 107
extract_model_include_clock_tree_pulse_width 107
extract_model_include_ideal_clock_network_latency 108
extract_model_include_mcp_in_arc_value 109
extract_model_include_noise_immunity_attributes 109
extract_model_include_power_ground_data 110
extract_model_include_transparent_latch_constraints 110
extract_model_include_upf_data 111
extract_model_keep_inferred_nochange_arcs 111
extract_model_lib_format_with_check_pins 112
extract_model_noise_iv_index_lower_factor 113

extract_model_noise_iv_index_upper_factor . . . 113
extract_model_noise_width_points . . . 114
extract_model_num_capacitance_points . . . 115
extract_model_num_clock_transition_points . . . 115
extract_model_num_data_transition_points . . . 116
extract_model_num_noise_iv_points . . . 117
extract_model_num_noise_width_points . . . 117
extract_model_pg_pin_direction_precedence . . . 118
extract_model_short_syntax_compatibility . . . 119
extract_model_single_pin_cap . . . 119
extract_model_single_pin_cap_max . . . 120
extract_model_split_partial_clock_gating_arcs . . . 121
extract_model_status_level . . . 122
extract_model_suppress_three_state . . . 123
extract_model_upf_supply_precedence . . . 123
extract_model_use_context_delta_delay . . . 124
extract_model_with_3d_arcs . . . 125
extract_model_with_ccs_timing . . . 126
extract_model_with_clock_latency_arcs . . . 127
extract_model_with_non_sequential_arcs . . . 127
extract_model_write_case_values_to_constraint_file . . . 128
extract_model_write_verilog_format_wrapper . . . 129

f . . . 130
filter_collection_extended_syntax . . . 130

g . . . 130
gca_setup_file . . . 130
global_timing_derate_violations . . . 131
golden_upf_report_missing_objects . . . 133
gui_build_query_data_table . . . 133
gui_custom_setup_files . . . 134
gui_default_window_type . . . 134
gui_disable_custom_setup . . . 135
gui_online_browser . . . 136
gui_selection_stack_depth . . . 136
gui_suppress_auto_layout . . . 137

h . . . 137
hier_block_interface_arc_trace_mode . . . 137
hier_block_interface_model_latch_level . . . 138

hier_characterize_context_mode 139
hier_clock_mapping_period_percent_tolerance 140
hier_clock_mapping_uncertainty_percent_tolerance 141
hier_constraint_output_compressed 142
hier_constraint_write_context 142
hier_constraints_include_cross_boundary_exceptions 143
hier_context_exclude_sub_block 144
hier_context_merge_clock_latency_mode 145
hier_context_merge_static_clock_latency_mode 145
hier_context_merge_strict_clock_equivalence 146
hier_create_template_scripts 147
hier_data_version 148
hier_disable_auto_clock_mapping 148
hier_disable_auto_multi_clock_mapping 149
hier_distributed_working_directory 150
hier_duplicate_internal_aggressor_timing_windows 151
hier_enable_analysis 151
hier_enable_ascii_context_writing_output_folder 152
hier_enable_detailed_side_input_path_timing 153
hier_enable_detailed_stub_path_timing 155
hier_enable_distributed_analysis 157
hier_enable_pba_clock_latency_context 158
hier_enable_release_resources_in_top_only_flow 159
hier_keep_required_time_mode 160
hier_merge_match_clock_with_constant 161
hier_model_enable_unconstrained_port_abstraction 162
hier_model_port_abstraction_ignore_register_count 163
hier_model_port_abstraction_ignore_register_percentage 164
hier_modeling_version 164
hier_override_clock_sense_from_context 165
hier_pp_report_limit 166
hierarchy_separator 166
i 167
ilm_ignore_percentage 167
imsa_save_eco_data 167
imsa_save_reporting_attributes 169
in_gui_session 170
input_slews_violations 170

interactive_multi_scenario_analysis_enabled 171
l 172
library_mapping_violations 172
library_pg_file_pattern 173
link_allow_design_mismatch 175
link_create_black_boxes 176
link_force_case 176
link_keep_cells_with_pg_only_connection 177
link_keep_pg_connectivity 178
link_keep_unconnected_cells 179
link_library 179
link_path 179
link_path_per_instance 180
link_use_only_master_cells_for_linking 181
lp_default_ground_pin_name 182
lp_default_power_pin_name 183
m 184
merge_model_allow_generated_clock_renaming 184
merge_model_ignore_pin_function_check 184
model_validation_capacitance_tolerance 185
model_validation_check_design 185
model_validation_ignore_pass 186
model_validation_output_file 186
model_validation_pba_clock_path 187
model_validation_percent_tolerance 188
model_validation_reanalyze_max_paths 188
model_validation_report_split 189
model_validation_save_session 190
model_validation_section 190
model_validation_significant_digits 191
model_validation_sort_by_worst 192
model_validation_timing_tolerance 192
model_validation_verbose 193
multi_core_allow_overthreading 197
multi_scenario_enable_analysis 198
multi_scenario_fault_handling 199
multi_scenario_license_mode 200
multi_scenario_merged_error_limit 201

multi_scenario_merged_error_log . . . 201
multi_scenario_message_verbosity_level . . . 202
multi_scenario_working_directory . . . 203
multi_user_access_group . . . 203
multi_user_logging_level . . . 204
mv_allow_pg_pin_reconnection . . . 205
mv_input_enforce_simple_names . . . 205
mv_upf_allow_negative_voltage . . . 206
mw_design_library . . . 206
mw_logic0_net . . . 207
mw_logic1_net . . . 207
n . . . 208
non_auto_fixable_violations . . . 208
o . . . 210
old_port_voltage_assignment . . . 210
operating_conditions_violations . . . 210
p . . . 211
parasitic_corner_name . . . 211
parasitic_explorer_enable_analysis . . . 213
parasitics_cap_warning_threshold . . . 214
parasitics_enable_multiple_physical_pins . . . 214
parasitics_enable_tail_annotation . . . 215
parasitics_log_file . . . 215
parasitics_rejection_net_size . . . 216
parasitics_res_warning_threshold . . . 217
parasitics_warning_net_size . . . 217
partition . . . 218
pba_derate_only_mode . . . 218
pba_enable_mis_delay_ocv_pessimism_reduction . . . 219
pba_enable_path_based_physical_exclusivity . . . 220
pba_enable_xtalk_delay_ocv_pessimism_reduction . . . 221
pba_exhaustive_any_slack_lesser_than . . . 222
pba_exhaustive_endpoint_path_limit . . . 222
pba_exhaustive_endpoint_time_limit . . . 224
pba_exhaustive_memory_limit . . . 225
pba_exhaustive_slack_tolerance_percentage . . . 225
pba_path_mode_enumerate_by_gba_slack . . . 226
pba_path_mode_sort_by_gba_slack . . . 226

pba_path_recalculation_limit_compatibility 227
pba_recalculate_full_path 228
pba_usage_context 229
pg_allow_pg_pin_reconnection 229
port_search_in_current_instance 230
power_activity_file_precedence_over_sca 230
power_activity_precedence_over_force_implied 231
power_analysis_mode 231
power_calc_use_ceff_for_internal_power 232
power_capp_edge_triggered_propagation 233
power_check_defaults 233
power_clock_network_include_clock_gating_network 234
power_clock_network_include_register_clock_pin_power 234
power_concurrent_event_analysis_work_dir 235
power_default_base_clock 235
power_default_static_probability 236
power_default_toggle_rate 237
power_default_toggle_rate_reference_clock 239
power_disable_cell_sdpd_activity_estimation 240
power_disable_exact_built_in_name_mapping 241
power_disable_exact_name_match_to_hier_pin 241
power_disable_exact_name_match_to_net 242
power_disable_sdpd_estimation 242
power_disable_switch_cell_internal_ground_pin_check 243
power_distributed_work_dir 243
power_dump_multi_rail_total_waveform 244
power_em_disable_extrapolation 244
power_enable_advanced_cycle_power_analysis 245
power_enable_advanced_fsdb_reader 245
power_enable_advanced_sv_name_mapping 246
power_enable_analysis 246
power_enable_annotation_precedence_over_implied 247
power_enable_background_when_tree_creation 247
power_enable_capp_backward_implication 247
power_enable_clock_cycle_based_glitch 248
power_enable_clock_domain_based_glitch 249
power_enable_clock_scaling 250
power_enable_concurrent_event_analysis 250
power_enable_concurrent_hybrid_analysis 251

power_enable_constant_init_on_rtl 251
power_enable_cycle_based_power_analysis 252
power_enable_delay_shifted_event_analysis 252
power_enable_distributed_analysis 253
power_enable_em_analysis 253
power_enable_exclude_output_port_capacitance 254
power_enable_feedthrough_in_empty_cell 254
power_enable_full_range_c1cn_wavg 254
power_enable_glitch_calculation 255
power_enable_glitch_detection_with_scaled_slew 255
power_enable_glitch_estimation 256
power_enable_ignore_create_generated_clock 256
power_enable_internal_pin_propagation 257
power_enable_low_effort_activity_propagation 258
power_enable_merged_fsdb 258
power_enable_mode_support 259
power_enable_mt_namemapping 259
power_enable_multi_rail_analysis 260
power_enable_multi_rtl_to_gate_mapping 260
power_enable_name_mapping_case_precedence 261
power_enable_new_rrm_view 261
power_enable_nldm_cap 262
power_enable_non_mission_mode_leakage 262
power_enable_parallel_sdpd_estimation 262
power_enable_parallel_when_tree_creation 263
power_enable_pin_internal_power 263
power_enable_pin_power_to_receiver_mode 264
power_enable_power_only_cap 264
power_enable_rtl_advanced_cycle_power_analysis 265
power_enable_rtl_analysis 265
power_enable_rtl_save_restore_variables 266
power_enable_saif_all_state_static_probability 266
power_enable_saif_estimated_toggle_count 267
power_enable_separate_glitch_report 268
power_enable_show_strip_path_warning 268
power_enable_signal_em_analysis 269
power_enable_time_based_ignore_huge_cell_delay 269
power_enable_timing_analysis 269
power_enable_write_clock_activity_in_power_waveform 270

power_estimate_power_for_unmatched_event 270
power_filter_glitch_pulse_width 271
power_full_transition_glitch_scaling 271
power_include_driverless_net_switching_power 272
power_include_initial_x_transitions 272
power_include_tiecell_power 273
power_keep_vcd_event_order 273
power_limit_extrapolation_range 273
power_match_state_for_logic_x 274
power_match_value_for_logic_x 274
power_order_clock_events 275
power_rail_output_file 275
power_read_activity_ignore_case 276
power_read_unique_net_activity 277
power_read_vcd_ignore_case 278
power_report_leakage_breakdowns 279
power_reset_negative_extrapolation_value 279
power_reset_negative_internal_power 280
power_scale_dynamic_power_at_power_off 280
power_scale_internal_arc 281
power_ssa_force_override_register_user_ssa 282
power_table_include_switching_power 282
power_use_c1cn_pin_capacitance 283
power_use_ccsp_pin_capacitance 283
power_use_file_simulation_time 284
power_x_transition_derate_factor 284
ps_cell_robustness_cap_filter_ratio 285
ps_cell_robustness_cell_loss_threshold 285
ps_cell_robustness_clock_period_ratio 286
ps_cell_robustness_skip_cell_pin_limit 286
ps_cell_robustness_tran_filter_ratio 287
ps_enable_analysis 288
ps_enable_save_timing_slack_data 288
ps_enable_timing_analysis 289
pt_ilm_dir 289
pt_model_dir 290
pt_shell_mode 290
pt_tmp_dir 291
ptxr_root 292

ptxr_setup_file . . . 293
q . . . 294
query_objects_format . . . 294
r . . . 295
rc_adjust_rd_when_less_than_rnet . . . 295
rc_always_use_max_pin_cap . . . 296
rc_cache_min_max_rise_fall_ceff . . . 297
rc_ccs_extrapolation_range_compatibility . . . 298
rc_ccsn_dc_current_limit_in_ma . . . 298
rc_ccsn_enable_library_error_reporting . . . 299
rc_ceff_use_delay_reference_at_cpin . . . 300
rc_degrade_min_slew_when_rd_less_than_rnet . . . 300
rc_degrade_wlm_net_slew_based_on_delay . . . 301
rc_driver_model_mode . . . 302
rc_fallback_coupling_cap_multiplier . . . 303
rc_filter_rd_less_than_rnet . . . 303
rc_rd_less_than_rnet_threshold . . . 305
rc_receiver_disable_cond_default_arcs . . . 305
rc_receiver_enable_library_error_reporting . . . 306
rc_receiver_model_mode . . . 307
rc_receiver_modeling_effort . . . 308
read_parasitics_load_locations . . . 309
report_capacitance_use_ccs_receiver_model . . . 310
report_default_significant_digits . . . 310
report_use_out_of_table_range . . . 311
s . . . 312
scaling_allow_pre_link . . . 312
scaling_calculation_compatibility . . . 313
scaling_disable_arc_consistency_check_for_exact_match . . . 313
scaling_enable_performance_mode . . . 314
sdc_name_map . . . 315
sdc_save_source_file_information . . . 315
sdc_version . . . 317
sdc_write_unambiguous_names . . . 318
sdf_align_multi_drive_cell_arcs . . . 319
sdf_align_multi_drive_cell_arcs_threshold . . . 320
sdf_always_include_preset_clear_arcs . . . 320
sdf_enable_cond_start_end . . . 321

sdf_enable_port_construct 321
sdf_enable_port_construct_threshold 322
search_path 322
selection_logging_no_core_action 323
selection_logging_too_many_objects_action 323
sh_allow_tcl_with_set_app_var 324
sh_allow_tcl_with_set_app_var_no_message_list 324
sh_arch 325
sh_command_abbrev_mode 325
sh_command_abbrev_options 326
sh_command_log_file 326
sh_continue_on_error 327
sh_deprecated_is_error 328
sh_dev_null 328
sh_disabled_is_error 329
sh_enable_constraint_analysis_mode 329
sh_enable_line_editing 330
sh_enable_page_mode 331
sh_enable_stdout_redirect 331
sh_enable_system_monitoring 332
sh_fast_analysis_mode_enabled 332
sh_flow_summary_file 333
sh_global_per_message_limit 334
sh_help_shows_group_overview 335
sh_high_capacity_effort 335
sh_high_capacity_enabled 336
sh_host_mode 337
sh_language 338
sh_launch_dir 338
sh_limited_messages 339
sh_line_editing_mode 340
sh_message_limit 341
sh_mm_resolve_clockdata 342
sh_new_variable_message 342
sh_new_variable_message_in_proc 343
sh_new_variable_message_in_script 344
sh_obsolete_is_error 346
sh_output_log_file 346
sh_product_version 346

sh_program_mode 347
sh_pt_tmp_dir_unique_mode 348
sh_script_stop_severity 349
sh_source_emits_line_numbers 350
sh_source_logging 351
sh_source_uses_search_path 352
sh_tcllib_app_dirname 352
sh_user_man_path 353
si_analysis_logical_correlation_mode 353
si_ccs_aggressor_alignment_mode 354
si_enable_analysis 354
si_enable_multi_input_switching_analysis 355
si_enable_multi_input_switching_timing_window_filter 356
si_filter_keep_all_port_aggressors 356
si_filter_per_aggr_noise_peak_ratio 357
si_ilm_keep_si_user_excluded_aggressors 358
si_multi_input_switching_analysis_mode 358
si_multi_input_switching_derate_scaling_for_library_cells_compatibility 360
si_multi_input_switching_precedence 361
si_multi_input_switching_tied_input_filter 362
si_noise_composite_aggr_mode 363
si_noise_endpoint_height_threshold_ratio 364
si_noise_immunity_default_height_ratio 365
si_noise_limit_propagation_ratio 365
si_noise_skip_update_for_report_attribute 366
si_noise_slack_skip_disabled_arcs 366
si_noise_update_status_level 367
si_use_driving_cell_derate_for_delta_delay 368
si_xtalk_composite_aggr_mode 369
si_xtalk_composite_aggr_noise_peak_ratio 370
si_xtalk_composite_aggr_quantile_high_pct 370
si_xtalk_delay_analysis_mode 371
si_xtalk_double_switching_mode 372
si_xtalk_exit_on_max_iteration_count 373
si_xtalk_exit_on_max_iteration_count_incr 374
si_xtalk_library_consistency_compatibility 374
si_xtalk_max_transition_mode 375
si_xtalk_use_physical_exclusivity_on_ignore_arrival_nets 375
simlink_enable_fast_monte_carlo 376

simlink_include_generated_clock_on_internal_pin 377
simlink_report_default_significant_digits 378
simlink_training_data_directory 378
svr_enable_ansi_style_port_declarations 379
svr_enable_vpp 380
svr_keep_unconnected_nets 380
synopsys_program_name 381
synopsys_root 381
t 382
tcl_interactive 382
tcl_library 382
tcl_pkgPath 382
timing_all_clocks_propagated 383
timing_allow_short_path_borrowing 383
timing_aocvm_analysis_mode 384
timing_aocvm_enable_analysis 385
timing_aocvm_enable_clock_network_only 386
timing_aocvm_enable_single_path_metrics 387
timing_aocvm_ocv_precedence_compatibility 388
timing_arrival_attribute_filter 388
timing_avoid_internal_pins_as_latch_loop_breaker 389
timing_bidirectional_pin_max_transition_checks 390
timing_calculation_across_broken_hierarchy_compatibility 390
timing_check_defaults 391
timing_check_include_constant_loop_compatibility 391
timing_clock_gating_check_fanout_compatibility 392
timing_clock_gating_propagate_enable 393
timing_clock_reconvergence_pessimism 393
timing_continue_on_scaling_error 394
timing_cross_voltage_domain_analysis_mode 395
timing_crpr_different_transition_derate 396
timing_crpr_different_transition_variation_derate 397
timing_crpr_remove_clock_to_data_crp 397
timing_crpr_remove_muxed_clock_crp 398
timing_crpr_threshold_ps 399
timing_disable_bus_contention_check 400
timing_disable_clock_gating_checks 401
timing_disable_cond_default_arcs 401

timing_disable_floating_bus_check 402
timing_disable_internal_inout_cell_paths 403
timing_disable_internal_inout_net_arcs 404
timing_disable_recovery_removal_checks 404
timing_early_launch_at_borrowing_latches 405
timing_enable_advanced_data_management 406
timing_enable_auto_mux_clock_exclusivity 407
timing_enable_clock_propagation_through_preset_clear 407
timing_enable_clock_propagation_through_three_state_enable_pins 408
timing_enable_constraint_variation 408
timing_enable_cross_voltage_domain_analysis 409
timing_enable_cumulative_incremental_derate 411
timing_enable_cumulative_incremental_path_margin 412
timing_enable_data_check_default_group 412
timing_enable_derate_multi_dimension_interpolation 413
timing_enable_derate_scaling_for_library_cells_compatibility 414
timing_enable_derate_scaling_interpolation_for_library_cells 414
timing_enable_dvd_analysis 416
timing_enable_from_clock_to_dangling_load_endpoints 416
timing_enable_graph_based_refinement 417
timing_enable_hier_boundary_checks 419
timing_enable_independent_crosstalk_in_data_network 420
timing_enable_library_max_cap_lookup_table 421
timing_enable_max_cap_precedence 422
timing_enable_max_capacitance_set_case_analysis 423
timing_enable_max_slew_precedence 423
timing_enable_max_transition_set_case_analysis 424
timing_enable_min_pulse_width_waveform_adjustment 424
timing_enable_native_vt_mistracking_analysis 425
timing_enable_normalized_slack 425
timing_enable_override_default_groups 426
timing_enable_preset_clear_arcs 427
timing_enable_pulse_clock_constraints 428
timing_enable_slew_variation 428
timing_enable_through_paths 430
timing_enable_transition_derate_exclude_mshift 431
timing_enable_unit_delay 431
timing_enable_via_variation 432
timing_enable_voltage_swing 432

timing_exception_preserve_user_object_order 433
timing_filter_arcs_for_crossing 434
timing_gclock_expand_odd_period_edge_list 435
timing_gclock_source_network_num_master_registers 435
timing_ideal_clock_zero_default_transition 436
timing_include_available_borrow_in_slack 436
timing_include_uncertainty_for_pulse_checks 437
timing_input_port_default_clock 438
timing_is_clock_pin_attribute_on_clock_source_compatibility 438
timing_keep_loop_breaking_disabled_arcs 439
timing_keep_waveform_on_points 440
timing_lib_cell_derived_clock_name_compatibility 441
timing_library_max_cap_from_lookup_table 441
timing_max_capacitance_derate 442
timing_max_capacitance_limit_from_library_only 443
timing_max_normalization_cycles 443
timing_max_transition_derate 444
timing_max_transition_limit_from_library_only 445
timing_min_pulse_width_high_pulse_threshold 446
timing_min_pulse_width_low_pulse_threshold 446
timing_ocvm_enable_distance_analysis 447
timing_ocvm_precedence_compatibility 448
timing_parallel_constraint_arcs_compatibility 449
timing_path_arrival_required_attribute_include_clock_edge 449
timing_path_signature_type 450
timing_pocvm_corner_sigma 451
timing_pocvm_enable_analysis 451
timing_pocvm_enable_delay_slew_correlation 452
timing_pocvm_enable_extended_moments 453
timing_pocvm_enable_extrapolation_warning 453
timing_pocvm_max_transition_clock_sigma 454
timing_pocvm_max_transition_sequential_sigma 455
timing_pocvm_max_transition_sigma 456
timing_pocvm_precedence 456
timing_pocvm_report_sigma 457
timing_point_arrival_attribute_compatibility 458
timing_prelayout_scaling 459
timing_propagate_interclock_uncertainty 459
timing_propagate_through_non_latch_d_pin_arcs 460

timing_reduce_multi_drive_net_arcs 461
timing_reduce_multi_drive_net_arcs_threshold 463
timing_refinement_max_slack_threshold 463
timing_refinement_maximum_critical_pin_percentage 464
timing_refinement_min_slack_threshold 465
timing_remove_clock_reconvergence_pessimism 466
timing_report_always_use_valid_start_end_points 468
timing_report_attribute_csv_delimiter 469
timing_report_attribute_skip_table_header 469
timing_report_fixed_width_columns_on_left 470
timing_report_hier_stub_pin_paths 472
timing_report_include_eco_attributes 474
timing_report_invert_clock_min_pulse_width_constraints 476
timing_report_min_pulse_width_show_clock_compatibility 476
timing_report_output_delay_on_clock_source 477
timing_report_pin_names_in_header 477
timing_report_recalculation_status 478
timing_report_skip_early_paths_at_intermediate_latches 479
timing_report_status_level 480
timing_report_trace_ideal_clocks 481
timing_report_unconstrained_paths 482
timing_report_unconstrained_paths_from_nontimed_startpoints 483
timing_report_union_tns 484
timing_report_use_worst_parallel_cell_arc 485
timing_save_block_level_reporting_data 485
timing_save_hier_context_data 486
timing_save_hier_model_data 487
timing_save_pin_arrival_and_required 488
timing_save_pin_arrival_and_slack 488
timing_separate_hier_side_inputs 489
timing_simultaneous_clock_data_port_compatibility 491
timing_single_edge_clock_gating_backward_compatibility 492
timing_through_path_max_segments 492
timing_update_effort 494
timing_update_include_graph_based_refinement 495
timing_update_status_level 496
timing_use_constraint_derates_for_pulse_checks 497
timing_use_link_library_on_best_match_failure 498
timing_use_slew_variation_in_constraint_arcs 499

timing_use_zero_slew_for_annotated_arcs . . . 500
timing_voltage_swing_high_pulse_threshold . . . 500
timing_voltage_swing_low_pulse_threshold . . . 501
timing_vt_mistracking_monte_carlo_sample_size . . . 502
training_data_directory . . . 502
u . . . 503
upf_allow_DD_primary_with_supply_sets . . . 503
upf_allow_net_voltage_override . . . 503
upf_create_implicit_supply_sets . . . 504
upf_enable_pst . . . 504
upf_name_map . . . 505
upf_power_switches_always_on . . . 505
upf_relax_association_checks . . . 506
upf_use_driver_receiver_for_io_voltages . . . 507
upf_wscript_retain_object_name_scope . . . 507
v . . . 508
var_enable_bounding_mixed_vt_derate . . . 508
variation_derived_scalar_attribute_mode . . . 508
variation_report_timing_increment_format . . . 509
vsa_non_dslg_cell_threshold . . . 510
vsa_voltage_slack_search_limit . . . 510
vt_mistracking_analysis_mode . . . 511
vt_mistracking_enable_bounding_analysis . . . 511
vt_mistracking_enable_path_analysis . . . 513
vt_mistracking_monte_carlo_sample_size . . . 514
vt_mistracking_partial_correlation_factor . . . 515
vt_skew_analysis_mode . . . 515
vt_skew_enable_bounding_analysis . . . 516
vt_skew_enable_path_analysis . . . 517
vt_skew_monte_carlo_sample_size . . . 518
vt_skew_partial_correlation_factor . . . 519
w . . . 520
wildcards . . . 520
write_script_include_library_constraints . . . 522
write_script_output_lumped_net_annotation . . . 522
y . . . 523
yield_enable_analysis . . . 523
yield_enable_timing_analysis . . . 524

2. PrimeTime Suite Attributes 525

c 525

category_node_attributes 525
category_tree_attributes 525
cell_attributes 526
clock_attributes 542
correlation_attributes 547
coupling_capacitor_attributes 548

d 550

design_attributes 550

g 567

ground_capacitor_attributes 567
gui_annotation_attributes 569
gui_object_attributes 571

l 571

lib_attributes 571
lib_cell_attributes 576
lib_pg_pin_info_attributes 581
lib_pin_attributes 582
lib_timing_arc_attributes 590

m 595

mode_attributes 595

n 596

net_attributes 596

p 611

path_group_attributes 611
pg_pin_info_attributes 612
pin_attributes 613
port_attributes 649

r 682

resistor_attributes 682

s 686

scenario_attributes 686

t 686

timing_arc_attributes 686
timing_path_attributes 691

timing_point_attributes 702
u 705
upf_attributes 705
upf_power_domain_attributes 706
upf_power_switch_attributes 707
upf_supply_net_attributes 709
upf_supply_port_attributes 710
upf_supply_set_attributes 711
v 712
variation_attributes 712

Feedback

1

PrimeTime Suite Variables

This document describes the variables supported by the PrimeTime Suite tool.

a

arch

This is a synonym for the read-only *sh_arch* variable.

See Also

- sh_arch

attributes

This man page provides general information about attributes.

Description

An attribute carries information about an object. For example, the *number_of_pins* attribute attached to a cell object indicates the number of pins in the cell.

You can use the following types of attributes:

- Application attributes - Predefined by the tool.
- User-defined attributes - Defined, set, and removed by the *define_user_attribute*, *set_user_attribute*, and *remove_user_attribute* commands, respectively.

To see the value of an attribute, use the *get_attribute* or *report_attribute* command. To list attributes, use the *get_defined_attributes*, *help_attributes*, or *list_attributes* command.

For details about the attributes of an object class, see the man page with "_attributes" appended to the object class name. For example, to see information about design attributes, use this command:

```
pt_shell> man design_attributes
```

Feedback

See Also

- define_user_attribute
- get_attribute
- get_defined_attributes
- help_attributes
- list_attributes
- remove_user_attribute
- set_user_attribute
- report_attribute
- category_node_attributes
- category_tree_attributes
- cell_attributes
- clock_attributes
- correlation_attributes
- design_attributes
- gui_object_attributes
- lib_attributes
- lib_cell_attributes
- lib_pg_pin_info_attributes
- lib_pin_attributes
- lib_timing_arc_attributes
- mode_attributes
- net_attributes
- path_group_attributes
- pg_pin_info_attributes
- pin_attributes
- port_attributes

Feedback

- scenario_attributes
- timing_arc_attributes
- timing_path_attributes
- timing_point_attributes
- upf_power_domain_attributes
- upf_power_switch_attributes
- upf_supply_net_attributes
- upf_supply_port_attributes
- upf_supply_set_attributes
- variation_attributes

---

auto_fixable_violations

This man page describes *auto_fixable* hierarchical boundary check violations shown by the *report_constraint* command in the HyperScale flow.

Description

The tool can automatically resolve *auto_fixable* boundary violations during subsequent iterations of block- and top-level HyperScale analysis. *auto_fixable* violations are for timing closure or signoff with the HyperScale flow. You should fix these violations after resolving the more critical *non_auto_fixable* violations.

The following violations are *auto_fixable*:

```
boundary_logic_value
clock_latency
clock_skew_with_uncertainty
data_arrival
input_slews
```

What Next

To resolve an *auto_fixable* violation, do one of the following actions:

- Rerun the analysis of the block where violations occur with the latest top-level context, and reoptimize the block if necessary.
- Optimize the top-level design to fit the block design scope.

Feedback

Examples

The following example shows a verbose report that shows only *auto_fixable* HyperScale boundary violations.

```
pt_shell> report_constraint -boundary_check_include {auto_fixable} \\
          -all_violators -verbose

****************************************
Report : constraint
         -all_violators
         -verbose
Design : top
 ...
****************************************

HyperScale constraints report

   Constraint: data_arrival

   Instance   Pin(Port name)        Window type   CLK           Block
 Top   Slack

 ------------------------------------------------------------------------
--------
   mtd_core   mtd_core/u0/A(clk1)   max_rise      top_CLK1(r)   N/A
 0.23  N/A
   mtd_core   mtd_core/u0/A(clk1)   min_rise      top_CLK1(r)   N/A
 0.23  N/A
   core3      core3/u0/A(clk1)      min_fall      top_CLK1(f)   N/A
 0.23  N/A

   Constraint: clock_latency

   Instance   Pin(Port name)        Window type   CLK           Block
 Top   Slack

 ------------------------------------------------------------------------
--------
   mtd_core   mtd_core/u0/A(clk1)   max_rise      top_CLK1(r)   0.00
 0.66  -0.66
   mtd_core   mtd_core/u0/A(clk1)   max_fall      top_CLK1(r)   N/A
 0.66  N/A
   core3      core3/u0/A(clk1)      min_fall      top_CLK1(r)   N/A
 0.66  N/A

   Constraint: input_slews

   Instance   Pin(rise/fall)        Window type   CLK           Block
 Top   Slack

 ------------------------------------------------------------------------
--------
```

```
   core        core/in1(rise)         max_rise      ---           N/A
 0.03  N/A
   core3       core3/in1(rise)        max_rise      ---           N/A
 0.03  N/A

1
```

See Also

- report_constraint
- boundary_logic_value_violations
- clock_latency_violations
- clock_skew_with_uncertainty_violations
- data_arrival_violations
- input_slews_violations
- non_auto_fixable_violations

---

auto_link_disable

Disables the autolink process.

Data Types

Boolean

Default false

Description

When this variable is set to *false* (the default), many PrimeTime commands attempt to automatically link the current design for you. For example, the *set_load* command invokes the linker if the current design is not linked. Automatic linking occurs only if the design is completely unlinked. If the current design is partially linked and has unresolved references, automatic linking does not occur. If the current design is totally linked, there is no need for an autolink, so it is not attempted.

Setting the *auto_link_disable* variable to *true* disables the autolink process. You can use this setting, along with the *link_design* command, to achieve the best possible performance when you have a large script that contains thousands of commands. Follow these steps:

1. Link the design manually with the *link_design* command.
2. Set the *auto_link_disable* variable to *true*.

Feedback

3. Source the script.
4. Reset the *auto_link_disable* variable to *false*.

Note that setting the *auto_link_disable* variable to *true* is intended to be used in conjunction with a manual link step.

See Also

- link_design

---

auto_wire_load_selection

Enables the automatic selection of wire load models.

Data Types

Boolean

Default true

Description

Setting this variable to *true* (the default) enables the automatic selection of wire load models, which are used to estimate net capacitances and resistances from the net fanout. When you set this variable to *false*, automatic selection of the wire load model is disabled.

The wire load models are described in the technology library. With the automatic selection of the wire load model, if the wire load mode is *segmented* or *enclosed*, the wire load model is chosen based on the area of the block containing the net either partially (for *segmented*) or fully (for *enclosed*). If the wire load mode is *top*, the wire load model is chosen based on the area of the top-level design for all nets in the design hierarchy.

When you manually select a wire load model for a block (using the *set_wire_load_model* command), automatic wire load selection for that block is disabled.

See Also

- report_wire_load
- set_wire_load_min_block_size
- set_wire_load_selection_group

Feedback

b

boundary_ideal_network_violations

This man page describes *boundary_ideal_network* hierarchical boundary check violations shown by the *report_constraint* command in the HyperScale flow.

Description

A *boundary_ideal_network* violation indicates a mismatch in the ideal networks between the top- and block-level runs at this object.

What Next

This is a *non_auto_fixable* violation. To fix a *boundary_ideal_network* violation, use the *set_ideal_network* or *remove_ideal_network* commands on the object at the top- or block-level runs.

See Also

- remove_ideal_network
- report_constraint
- set_ideal_network
- non_auto_fixable_violations

boundary_logic_value_violations

This man page describes *boundary_logic_value* hierarchical boundary check violations shown by the *report_constraint* command in the HyperScale flow.

Description

A *boundary_logic_value* violation indicates that the tool detected a conflict in the logic values set on or propagated to the specified hierarchical boundary pin or port.

For example, a *boundary_logic_value* violation occurs under the following conditions:

- HyperScale block-level analysis sets a SCANEN port with a case value of 0 and performs timing analysis.
- When the block is instantiated at the top level, a logic value not equal to 0 (either 1 or no case or constant values at all) propagates through the hierarchical pin.

Feedback

Note:

In the context of a *boundary_logic_value* HyperScale violation, the reported logic value might not always come from a user-defined *set_case_analysis*; it can also arise from a functional constant defined in the design, such as from Verilog, or from library cells with tie-high or tie-low connections.

The HyperScale analysis flow requires the constraints to be consistent between the block- and top-level runs. This include the same settings for case values and design constants on the block boundary. The HyperScale top flow captures the logic values for instance boundary and automatically uses the captured value to override the user-defined settings at block-level analysis to improve consistency of case value settings on ports. Therefore, this type of violation is considered auto-fixable by PrimeTime runs.

In multi-instance blocks where the HyperScale top-level run automatically merges into a common context for a single block-level analysis, the tool resolves conflicts in case analysis values on the same port across instances by setting no case values on the port for block-level analysis. In this situation, HyperScale does not report a *boundary_logic_value* violation for different instances because they are regarded as expected.

What Next

This is an *auto_fixable* violation. Confirm the case value difference, and either

- Manually align the constraints
- Apply the latest top context in the next block-level run to force it to align with the top-level context.

Examples

The following example shows a summary report for a *boundary_logic_value* violation.

```
pt_shell> report_constraint -boundary_check_include
 {boundary_logic_value} \\
          -all_violators

****************************************
Report : constraint
        -all_violators
        -boundary_check
 ...
****************************************

HyperScale constraints report

   Constraint: boundary_logic_value

   Instance        Num_vio        Reason
```

Feedback

```
   ------------------------------------------------
   block           1              top/block mismatch
```

The following example shows a verbose report:

```
pt_shell> report_constraint -boundary_check_include
 {boundary_logic_value} \\
          -all_violators -verbose

****************************************
Report : constraint
        -verbose
 ...
****************************************

HyperScale constraints report

   Constraint: boundary_logic_value

   Instance      Pin              Attributes       Block     Top
   ------------------------------------------------------------
   block         core/TSTEN       Case Value       1         X
```

See Also

- get_attribute
- report_case_analysis
- report_constraint
- auto_fixable_violations

---

bus_naming_style

Sets the naming format for a specific element of a bus.

Data Types

string

Default %s[%d]

Description

This variable is used by the native Verilog reader to set the naming format for a specific element of a bus. This is the way that the names of the individual bits of the bus appear in the application.

The default is "%s[%d]". For example, for bus A and index 12, the name would be A[12].

Feedback

See Also

- read_verilog

C

case_analysis_log_file

Specifies the name of the file into which the details of case analysis propagation are written.

Data Types

string

Default "" (empty)

Description

This variable specifies the name of a log file to be generated during propagation of constant values from case analysis or from nets tied to logic 0 or to logic 1.

The log file contains the list of all nets and pins that propagate constants. The constant propagation algorithm is an iterative process that propagates constants through nets and cells starting from a list of constant pins. The algorithm finishes when no more constants can be propagated. The format of the log file follows the constant propagation algorithm. For each iteration of the propagation process, the log file lists all nets and cells that propagate constants.

By default, this variable is set to an empty string, and no log file is generated during constant propagation.

If the file name is specified with the .gz extension, the output file is written in compressed (gzip) format.

See Also

- remove_case_analysis
- report_case_analysis
- report_disable_timing
- set_case_analysis
- disable_case_analysis

Feedback

case_analysis_propagate_through_icg

Specifies whether case analysis is propagated through integrated clock-gating cells.

Data Types

Boolean

Default false

Description

By default, this variable is set to *false*, which means that the tool does not propagate logic constants through integrated clock-gating cells. Regardless of whether the integrated clock-gating cell is enabled or disabled, no logic values propagate in the fanout of the cell.

If you set this variable to *true*, the tool propagates logic constants through integrated clock-gating cells. Set this variable before using the *update_timing* command.

An integrated clock-gating cell is enabled when its enable pin (or test enable pin) is set to logic 1. If the cell is disabled, the disable logic value for the cell is propagated in its fanout. For example, when the latch_posedge integrated clock gating is disabled, it propagates a logic 0 in its fanout.

Integrated clock-gating cells have the Library Compiler *clock_gating_integrated_cell* attribute. PrimeTime supports integrated clock-gating cells with latch memory elements. These cells are sequential in nature.

Integrated clock-gating cells that have no memory element are not sequential cells. Therefore, they are considered combinational and always propagate.

See Also

- remove_case_analysis
- set_case_analysis

case_analysis_sequential_propagation

Specifies whether case analysis is propagated across sequential cells.

Data Types

string

Default never

Feedback

Description

This variable specifies whether case analysis is propagated across sequential cells. Allowed values are *never* (the default) or *always*. When set to *never*, case analysis is not propagated across the sequential cells. When set to *always*, case analysis is propagated across the sequential cells.

The one exception to sequential propagation occurs when dealing with sequential integrated clock gating cells. When the *case_analysis_propagate_through_icg* variable is set to true, these types of integrated clock gating cells only propagate logic values.

See Also

- remove_case_sequential_propagation
- set_case_analysis
- set_case_sequential_propagation
- case_analysis_propagate_through_icg

---

clock_attributes_violations

This man page describes *clock_attributes* violations shown by the *report_constraint* command in the HyperScale flow.

Description

A *clock_attributes* violation indicates that a clock defined at the block level is successfully mapped to a clock at the top level, but certain elements of the clock definition do not match.

The following clock attributes are checked:

- *is_active* - true if the clock is active.
- *is_propagated* - true if the clock is propagated.
- *sources* - List of source pins or ports of the clock.
- *edges* - List of clock edges.
- *clock_sense* - Clock sense.

Clocks can be made active or inactive with the *set_active_clocks* command. The *is_propagated* attribute is *true* if the clock latency is determined by propagating delays from clock sources to register clock pins. Otherwise, ideal clocking is assumed. Set with *set_propagated_clock*.

Feedback

What Next

A *clock_attributes* violation is a *non_auto_fixable* violation. To resolve this violation, change the definition of clock in constraints based on the information given by the *report_constraint -verbose* command.

Examples

The following report summarizes clock attribute mismatch violations.

```
pt_shell> report_constraint -boundary_check_include {clock_attributes}
 -all_violators

****************************************
Report : constraint
        -all_violators
        -boundary_check
Design : wrapper
 ...
****************************************

HyperScale constraints report


   Constraint: clock_attributes

   Instance               Num_vio        Reason
   ------------------------------------------------------------------
   core1                  1              non-propagated block clock
   core3                  1              clock sources mismatch
```

The following example shows a verbose report. In this example, not propagated block-level clock CLK is mapped to a propagated top-level clock top_CLK. Block-level clock CLK has only one source in the instance core3, but the top-level clock top_CLK has two sources.

```
pt_shell> report_constraint -boundary_check_include {clock_attributes} \\
          -all_violators -verbose

****************************************
Report : constraint
        -all_violators
        -verbose
        -boundary_check
Design : wrapper
 ...
****************************************

HyperScale constraints report

   Constraint: clock_attributes
```

Feedback

```
   Instance    Attribute name      Block level (clock)    Top level
 (clock)

 ------------------------------------------------------------------------
-----
   core1       is_propagated       false (CLK)            true (top_CLK)
   core3       sources             {core3/clk1} (CLK)
 {core3/clk1,core3/clk2} (top_CLK)
```

See Also

- get_attribute
- report_clock
- report_constraint
- set_active_clocks
- non_auto_fixable_violations

---

clock_latency_violations

This man page describes *clock_latency* hierarchical boundary check violations shown by the *report_constraint* command in the HyperScale flow.

Description

A *clock_latency* violation indicates a mismatch of clock latency at a given pin between the top- and block-level analysis.

What Next

This is an *auto_fixable* violation. This violation can be automatically fixed in subsequent iterations, but the fix is not guaranteed because of interactions between blocks. Also, fix any clock existence violations first; otherwise, it is possible that arrival violations will not be fixed.

Examples

The following verbose report shows violations in clock latency settings applied at block-level analysis that are different to the top-level settings.

```
pt_shell> report_constraint -boundary_check_include {clock_latency} \\
          -all_violators -verbose

****************************************
Report : constraint
        -all_violators
        -verbose
        -boundary_check
```

Feedback

```
Design : top
 ...
****************************************

HyperScale constraints report

   Constraint: clock_latency

   Instance    Pin(Port name)      Window type    CLK           Block    Top
   Slack

 ---------------------------------------------------------------------------
-------
   core        core/u0/A(clk1A)    max_fall       top_CLK(f)    0.320    0.330
   -0.010
   core        core/u0/A(clk1A)    max_rise       top_CLK(r)    0.420    0.440
   -0.020
```

See Also

- report_constraint
- set_clock_latency
- auto_fixable_violations

---

clock_mapping_violations

This man page describes *clock_mapping* violations shown by the *report_constraint* command in the HyperScale flow.

Description

A *clock_mapping* violation indicates that PrimeTime cannot find a matching clock for the reported block or top-level clock with the given attributes.

A clean and complete mapping between top and block-level clocks must be established to perform valid timing analysis in HyperScale flow.

The tool automatically performs clock mapping according to the following characteristics captured for block and top-level clocks (in order of highest to lowest priority):

- Overlapping clock source objects on the same physical source network
- Matching clock periods
- Matching clock waveforms
- Matching clock names as prefix

Feedback

For HyperScale analysis, you must provide complete and consistent definitions for all clocks used in both block- and top-level analysis. At the top level, the full-chip flat constraints are often used with all clocks defined for the entire design, including HyperScale subblocks; this implies that the block constraints supplied for HyperScale block-level analysis must consistently define all clocks relevant to the given block. Clock mapping is the step to check and establishes this clock consistency between top- and block-level analysis. It is performed at both the block and top level. Any irregularity is reported as a *clock_mapping* violation.

Typical reasons for *clock_mapping* violations include:

- extra_clock - It means that an extra and redundant clock is defined at block level analysis with no corresponding clock at top level.
- missing_clock - It means that a top-level clock propagating to a port on the boundary of a HyperScale block, but the constraints used for the block level analysis for that block did not define a clock with matching characteristics.

What Next

This is a *non_auto_fixable* violation. To validate and fix the violation, edit block- or top-level timing constraints to ensure complete and consistent definitions of all clocks between top and block level analysis.

Examples

The following summary report shows clock mapping violations.

```
pt_shell> report_constraint -boundary_check_include {clock_mapping}
 -all_violators

****************************************
Report : constraint
        -all_violators
        -boundary_check
Design : wrapper
 ...
****************************************

HyperScale constraints report

   Constraint: clock_mapping

   Instance                Num_vio        Reason
   ------------------------------------------------------------
   core1                   1              missing_clock
   core2                   1              missing_clock
```

The following verbose report shows all clock mapping violations where clocks are defined at a block level, but no corresponding top-level clocks can be matched to these block-level clocks.

Feedback

```
pt_shell> report_constraint -boundary_check_include {clock_mapping} \\
          -all_violators -verbose

****************************************
Report : constraint
         -all_violators
         -verbose
         -boundary_check
Design : wrapper
 ...
****************************************

HyperScale constraints report

Constraint: clock_mapping

Instance   Block level clock   Top level Clock   Attributes
       Block clock ref_pin
--------------------------------------------------------------------------------
-------------------------
core1      CLK_extra1          ---               p=16.00,e={0.00
 8.00},a={p A}  {core1/clk1}
core2      CLK_extra1          ---               p=16.00,e={0.00
 8.00},a={p A}  {core2/clk1}
```

In the preceding example, the attribute column shows characteristics for the clock named in the column 'Block level'. These characteristics are the period (denoted by the character 'p'), clock edges ('e'), and other attributes ('a'). The possible values for other attributes are: "A" means that the clock is active, "p" means that the clock is propagated.

The following verbose report shows all clock mapping violations where a top level clock has propagated to boundary ports of block instances core1 and core2, but no block-level clock is using these ports as clock sources.

```
pt_shell> report_constraint -boundary_check_include {clock_mapping} \\
          -all_violators -verbose

****************************************
Report : constraint
         -all_violators
         -verbose
         -boundary_check
Design : wrapper
 ...
****************************************

HyperScale constraints report

Constraint: clock_mapping

Instance  Block level clock   Top level Clock   Attributes
  Block clock ref_pin
```

Feedback

```
------------------------------------------------------------------------
-------------------------
core1     ---                 CLKx              p=12.00,e={0 8},a={p A}
  {core1/clk2 core1/clk1}
core2     ---                 CLKx              p=12.00,e={0 8},a={p A}
  {core2/clk2 core2/clk1}
```

To fix violations shown in the preceding example, create block-level clocks using the list of source pins printed in the last column of the report.

See Also

- report_clock
- report_constraint
- non_auto_fixable_violations

---

clock_relations_violations

This man page describes *clock_relations* violations shown by the *report_constraint* command in the HyperScale flow.

Description

A *clock_relations* violation means that there is a mismatch of asynchronous or exclusive relation between clocks at top and block levels.

What Next

This is a *non_auto_fixable* violation. Use the *set_clock_groups* command to make relations between clocks the same at the top and block levels of the analysis.

Examples

The following example shows a summary report for clock_relations violations.

```
pt_shell> report_constraint -boundary_checks {clock_relations}
 -all_violators

****************************************
Report : constraint
        -all_violators
        -boundary_checks
Design : wrapper
 ...
****************************************

HyperScale constraints report

   Constraint: clock_relations
```

Feedback

```
   Instance             Num_vio        Reason
   ------------------------------------------------------------
   core2                1              clock async mismatch
   core                 2              clock async mismatch
```

The following example shows a verbose report for clock_relations.

```
pt_shell> report_constraint -boundary {clock_relations} -all_violators
 -verbose

****************************************
Report : constraint
        -all_violators
        -verbose
        -boundary_checks
Design : wrapper
 ...
****************************************

HyperScale constraints report

   Constraint: clock_relations

   Instance   Clock relation   Block level            Top level
   ------------------------------------------------------------------------
   core2      async            {int_CLK} <-> {top_CLK} {core2/int_CLK}
 <-> {}
   core       async            {int_CLK} <-> {top_CLK} {core/int_CLK} <->
 {}
```

See Also

- report_clock
- report_constraint
- set_clock_groups
- non_auto_fixable_violations

---

clock_skew_with_uncertainty_violations

This man page describes *clock_skew_with_uncertainty* violations shown by the *report_constraint* command in the HyperScale flow.

Description

A *clock_skew_with_uncertainty* violation indicates that a clock defined at block level is successfully mapped to a clock at the top level, but the interclock skew values associated with these two clocks are different.

Feedback

What Next

This is an *auto_fixable* violation. If a *clock_skew_with_uncertainty* violation is due to a mismatch of clock latency, it is automatically fixed in a subsequent iteration; however, the fix is not guaranteed because of interactions between blocks. Also, you must first fix any clock existence violations. If the violations are due to mismatch of uncertainty, change the clock uncertainty for the block- or top-level clocks with the *set_clock_uncertainty* command.

Examples

The following summary report shows *clock_skew_with_uncertainty* violations.

```
pt_shell> report_constraint -boundary_check_include
 {clock_skew_with_uncertainty} \\
          -all_violators

****************************************
Report : constraint
        -all_violators
        -boundary_check
Design : wrapper
 ...
****************************************

HyperScale constraints report

   Constraint: clock_skew_with_uncertainty

   Instance          Num_vio        Reason
   ------------------------------------------------
   core3             2              top/block mismatch
   core1             2              top/block mismatch
```

The following verbose report shows all *clock_skew_with_uncertainty* violations.

```
pt_shell> report_constraint -boundary_check_include
 {clock_skew_with_uncertainty} \\
          -all_violators -verbose

****************************************
Report : constraint
        -all_violators
        -verbose
        -boundary_check
Design : wrapper
 ...
****************************************

HyperScale constraints report

   Constraint: clock_skew_with_uncertainty
```

Feedback

```
   Instance    Type     From Clock     To Clock         Block    Top     Slack
   ------------------------------------------------------------------------
   core3       setup    CLK3 (fall)    CLK4_v (rise)    0.00     4.00    -4.00
   core3       hold     CLK1 (rise)    CLK2_v (fall)    0.00     1.00    -1.00
   core1       setup    CLK3 (fall)    CLK4_v (rise)    0.00     4.00    -4.00
   core1       hold     CLK1 (rise)    CLK2_v (fall)    0.00     1.00    -1.00
```

See Also

- report_clock
- report_constraint
- set_clock_latency
- set_clock_uncertainty
- auto_fixable_violations

---

clock_uncertainty_violations

This man page describes *clock_uncertainty* violations shown by the *report_constraint* command in the HyperScale flow.

Description

A *clock_uncertainty* violation indicates that a clock defined at block level is successfully mapped to a clock at top level, but the uncertainty (skew) values associated with these two clocks are different.

What Next

This is a *non_auto_fixable* violation. To resolve this violation, change the clock uncertainty for the block- or top-level clocks with the *set_clock_uncertainty* command.

Examples

The following summary report shows *clock_uncertainty* violations.

```
pt_shell> report_constraint -boundary_check_include {clock_uncertainty}
 -all_violators

****************************************
Report : constraint
        -all_violators
        -boundary_check
Design : wrapper
 ...
****************************************
```

Feedback

```
HyperScale constraints report

   Constraint: clock_uncertainty

   Instance          Num_vio        Reason
   ------------------------------------------------
   core3             2              top/block mismatch
   core1             2              top/block mismatch
```

The following verbose report shows all clock uncertainty mismatch violations.

```
pt_shell> report_constraint -boundary_check_include {clock_uncertainty}
 \\
          -all_violators -verbose

****************************************
Report : constraint
        -all_violators
        -verbose
        -boundary_check
Design : wrapper
 ...
****************************************

HyperScale constraints report

   Constraint: clock_uncertainty

   Instance   Type    From Clock    To Clock        Block   Top    Slack
   ------------------------------------------------------------------------
   core3      setup   CLK3 (fall)   CLK4_v (rise)   0.00    4.00   -4.00
   core3      hold    CLK1 (rise)   CLK2_v (fall)   0.00    1.00   -1.00
   core1      setup   CLK3 (fall)   CLK4_v (rise)   0.00    4.00   -4.00
   core1      hold    CLK1 (rise)   CLK2_v (fall)   0.00    1.00   -1.00
```

See Also

- report_clock
- report_constraint
- set_clock_uncertainty
- non_auto_fixable_violations

---

collection_result_display_limit

Sets the maximum number of objects that can be displayed by any command that displays a collection.

Feedback

Data Types

integer

Default 100

Description

Sets the maximum number of objects that can be displayed by any command that displays a collection. The default is *100*.

When a command, such as the *add_to_collection*) command, is issued at the command prompt, the result is implicitly queried, as though the *query_objects* command was called. To limit the number of objects displayed, set the *collection_result_display_limit* variable to an appropriate integer.

A value of -1 displays all objects; a value of 0 displays the collection handle ID instead of the object names in the collection.

See Also

- collections
- query_objects

---

constraint_analysis_violation_limit

Specifies the limit for the number of violations displayed during the *check_constraints* command for a check.

Data Types

integer

Default 10000

Description

This variable is used to set the limit on the number of violations displayed during the *check_constraints* command for a check. The default limit is set to 10000. To disable any limit including default on the number of violations, set the variable's value to -1.

See Also

- check_constraints

Feedback

create_clock_no_input_delay

Specifies delay propagation characteristics of clock sources created using the *create_clock* command.

Data Types

Boolean

Default false

Description

This variable affects the delay propagation characteristics of clock sources created using the *create_clock* command. When the variable is set to *false* (the default), the clock sources used in the data path are established as timing startpoints. The clock sources in the design propagates rising delays on every rising clock edge and propagates falling delays on every falling clock edge. To disable this behavior, set the *create_clock_no_input_delay* variable to *true*.

See Also

- create_clock

d

data_arrival_violations

This man page describes *data_arrival* violations shown by the *report_constraint* command in the HyperScale flow.

Description

A *data_arrival* violation reported in HyperScale top-level analysis means that the data arrival windows defined at the block-level run do not fully contain the actual arrival windows propagated at the same pin at the top level due to one of the following conditions:

- The minimum or early window analyzed at the block level is larger or later than the actual minimum arrivals reaching the specified pin or port.
- The maximum or late window at the block level is smaller or earlier than the actual maximum arrivals at the top level.

This HyperScale boundary constraint violation implies potential timing violations within the internal register-to-register paths of the block because those paths are not visible and therefore not reanalyzed in the HyperScale top-level run. This happens when there are wire couplings from the block interface paths to the internals of the block; the widening

Feedback

of the arrival window can result in different overlapping scenarios during the PrimeTime SI aggressor and victim analysis, result in additional crosstalk effects that are not fully accounted for during the block-level analysis of those internal victim paths.

Note: This HyperScale boundary constraint check for data_arrival is for the second order effects. The direct effect of a wider window reaching the block boundary is analyzed during top-level analysis because either the timing paths are already visible, or there are required times annotated at its fanout.

What Next

A *data_arrival* violation does not require your action to fix the design or constraint; this is an *auto_fixable* violation. If the effects of wider windows need to be validated, save the updated and current context captured for the block of interest, and rerun HyperScale block-level analysis using this new context. The actual windows (wider than the previous block-level run) are applied and analyzed automatically.

Examples

The following verbose report shows violations in pin arrival times at block-level analysis that is different from the actual arrival times computed at the top level.

During block-level analysis, an expected min/max arrival window of 2.0 ns to 4.0 ns is defined for the analysis, but top-level analysis actual propagated arrival window is 1.8247 ns to 4.1472 ns, which is wider than the block-level setting. Therefore, you need to save this computed window as the actual context for the block and apply it to the block in the next HyperScale run.

```
pt_shell> report_constraint -boundary_check_include {data_arrival} \\
          -all_violators -verbose

****************************************
Report : constraint
        -all_violators
        -verbose
        -boundary_check
Design : top
 ...
****************************************

HyperScale constraints report

   Constraint: data_arrival

   Instance  Pin(Port name)   Window type  CLK          Block   Top
 Slack

 ---------------------------------------------------------------------------
-----
   core      core/u0/A(clk1)  max_rise     top_CLK1(r)  4.0000  4.1472
 -0.1472
```

Feedback

```
   core       core/u0/A(clk1)   min_rise      top_CLK1(r)   2.0000   1.8247
-0.1753
```

See Also

- report_constraint
- auto_fixable_violations
- clock_latency_violations
- hier_enable_analysis
- input_slews_violations

---

dbr_ignore_external_links

Specifies whether to ignore external links when reading in database files.

Data Types

Boolean

Default false

Description

When this variable is set to its default of *false*, if the *read_db* command encounters an external link when reading a database (db) file, it extracts as much information as possible from the database (db) file so that the linker can restore the link. If you set this variable to *true*, the *read_db* command ignores external links and searches for an object by name only in the libraries set using the *link_path* variable.

External links are written by Design Compiler for objects (for example, wire load models and operating conditions), when there is a link from a design to another object in a library. The external link records information about the library to which the wire load was linked.

For example, if the TOP design has an external link for a wire load model named B100 in the nominal.db library, by default the linker attempts to load a nominal.db library. If it is not already loaded, the tool looks in that library for a wire load model named B100. To override this default behavior and use B100 instead from min.db, set the *dbr_ignore_external_links* variable to *true* and specify min.db in the *link_path* variable.

See Also

- link_design
- read_db

Feedback

- link_path
- search_path

default_oc_per_lib

Enables the use of a default operating condition per individual library.

Data Types

Boolean

Default true

Description

This variable enables the use of a default operating condition per individual library. By default, this variable is set to *true*, and each cell that does not have an explicitly-set operating condition (on the cell itself, on any of its parent cells, or on the design) is assigned the default operating condition of the library to which the cell belongs.

If you set this variable to *false*, all cells that do not have any explicitly-set operating condition are assigned the default operating condition of the main library (the first library set in the *link_path* variable).

The recommended flow is to explicitly set operating conditions on the design or on each hierarchical block that is powered by the same voltage (also called the voltage area).

This variable is mainly for obtaining backward compatibility for the corner case of using default conditions in PrimeTime version T-2002.09 and earlier releases.

See Also

- link_path
- set_operating_conditions

delay_calc_enhanced_ccsn_waveform_analysis

Enable CCS advanced waveform propagation enhancement for advanced nodes with CCS noise libraries.

Data Types

Boolean

Default true

Feedback

Description

When this variable is set to *true*, advanced waveform calculation results are closer to HSPICE simulation. For this setting to be effective, the CCS noise data must be well characterized and aligned with NLDM and CCS timing data.

It is recommended to run the *check_library* command in the Library Compiler tool to ensure the accuracy of CCS noise data for delay calculation.

This variable must be set before the *link_design* command is run.

See Also

- update_timing
- delay_calc_waveform_analysis_mode

---

delay_calc_waveform_analysis_constraint_arcs_compatibility

Specifies whether setup and hold constraint analysis uses nonlinear delay model (NLDM) lookup tables instead of waveform propagation.

Data Types

Boolean

Default true

Description

This variable specifies whether setup and hold constraint analysis uses nonlinear delay model (NLDM) lookup tables instead of waveform propagation, even when the *delay_calc_waveform_analysis_mode* variable is set to *full_design*.

Set the *delay_calc_waveform_analysis_constraint_arcs_compatibility* variable as follows:

- *true* (default) - Setup and hold constraint analysis uses nonlinear delay model (NLDM) lookup tables and does not consider the waveform disortion effect.
- *false* - If waveform propagation is enabled by the *delay_calc_waveform_analysis_mode*, setup and hold constraint analysis considers waveform distortion effects. This analysis requires CCS timing and noise data in the library.

See Also

- extract_model
- report_timing

Feedback

- update_timing
- delay_calc_waveform_analysis_mode

delay_calc_waveform_analysis_mode

Controls usage of CCS-based waveform analysis for uncoupled and signal integrity calculation.

Data Types

string

Default disabled

Description

You can set this variable to one of the following:

- *disabled* (default) - Disables CCS waveform analysis.
- *full_design* - Enables CCS waveform analysis on the entire design.

CCS waveform analysis requires libraries that contain CCS timing and CCS noise data. This feature, which includes uncoupled calculation, requires a PrimeTime SI license. However, you do not need to set the *si_enable_analysis* variable for uncoupled analysis.

See Also

- report_timing
- update_timing
- si_enable_analysis

disable_case_analysis

Specifies whether case analysis is disabled.

Data Types

Boolean

Default false

Description

By default, this variable is set to *false*, and constant propagation is performed in the design from pins either that are tied to a logic constant value or for those specified using the *set_case_analysis* command.

Feedback

For example, a typical design has several pins set to a constant logic value. By default, this constant value propagates through the logic to which it connects. When you set the *disable_case_analysis* variable to *true*, case analysis and constant propagation are not performed.

If you set the *disable_case_analysis_ti_hi_lo* variable to *true*, the tool does not propagate constants from pins that are tied to a logic constant value.

See Also

- remove_case_analysis
- report_case_analysis
- set_case_analysis
- disable_case_analysis_ti_hi_lo

---

disable_case_analysis_ti_hi_lo

Specifies whether logic constants are propagated from pins that are tied to a logic constant value.

Data Types

Boolean

Default false

Description

By default, this variable is set *false*, and constant propagation is performed from pins that are tied to a logic constant value. For example, a typical design has several pins set to a constant logic value. By default, this constant value propagates through the logic to which it connects. When you set this variable to *true*, constant propagation is not performed from these pins.

This current value of this variable does not alter the propagation of logic values from pins where the logic value has been set by the *set_case_analysis* command.

If you set the *disable_case_analysis* variable to *true*, all constant propagation is disabled regardless of the current value of the *disable_case_analysis_ti_hi_lo* variable.

See Also

- remove_case_analysis
- report_case_analysis

Feedback

- set_case_analysis
- disable_case_analysis

disable_missing_object_errors_in_golden_constraints_reader

Disable missing object errors in the golden constraint reader.

Data Types

Boolean

Default false

Description

When you set this variable to *true*, PrimeTime stops throwing missing object errors in golden constraint reading with load_constraint command. It is recommended that the variable should be used only for rule based search.

See Also

- load_constraints
- enable_golden_constraints_reader

distributed_cleanup_variables_for_swap

Controls cleaning up the state of variables when scenarios are swapped.

Data Types

string

Default user_and_application

Description

If in Distributed Multi-Scenario Analysis (DMSA) fewer host processes are available than there are scenarios, the hosts run in scenario swapping mode in which they rotate and swap scenarios one after the other. PrimeTime's internal and user-defined variables set at one scenario executing at a particular host will be removed or reset before loading of the image or the scriptware for another scenario when the scenario swap occurs.

To control the clean up of the state of PrimeTime's internal and user-defined variables during scenario image swapping, set the *distributed_cleanup_variables_for_swap* variable to one of these values:

none

Feedback

- PrimeTime's internal and user-defined variables will not be automatically removed or reset during scenario image creation or swapping.

user_and_application

- PrimeTime will clean up the state of internal and user variables in the scenario swapping. The cleaning up process means that the variables at a DMSA worker process will be reset to their state recorded after sourcing of .synopsys_pt_setup files and executing of "pt_shell -x" arguments prior to loading up scenario image or scripts.

user

- PrimeTime will only remove or reset user variables (and will not reset internal variables) when creating a new scenario image prior to the scenario swapping.

application

- PrimeTime will reset up the state of internally defined variables only when creating a new scenario image prior to the scenario swapping.

This variable can only take effect if set in the DMSA manager process.

---

distributed_custom_protocol_error_detection_timeout

Specifies the maximum time that the distributed manager waits for custom protocol worker processes to come online.

Data Types

integer

Default 86400

Description

This variable specifies the maximum time in seconds that the distributed manager waits for custom protocol worker processes that were defined by *set_host_options -protocol custom*.

The default is 86400 seconds (24 hours).

See Also

- set_host_options

---

distributed_enable_analysis

Enables the HyperGrid distributed analysis flow.

Feedback

Data Types

Boolean

Default false

Description

Set this variable to *true* to enable the HyperGrid distributed analysis flow in the PrimeTime tool.

In the distributed analysis flow, the tool analyzes the design and creates partitions with single run, improving the capacity and performance of timing analysis without impacting the quality of results.

Specify the computation resources by using the *set_host_options* command. By default, HyperGrid distributed analysis will use the specified num_processes with set_host_options as the number of partitions. To override the default partitioning of the design for analysis, use the *start_dsta -num_partitions* command option.

You must set the *distributed_enable_analysis* variable before start_dsta command in the wrapper script.

See Also

- start_dsta
- set_host_options
- start_hosts

---

distributed_farm_check_pending_workers_interval

Specifies how often the distributed manager polls the compute farm job manager to ensure that the job submissions are still queued and waiting.

Data Types

integer

Default 600

Description

When the *start_hosts* command is run, the distributed manager submits the worker farm jobs to the compute farm. These submitted jobs start out in a "queued/waiting" state. If the farm accepts and launches a job, that worker process registers itself with the distributed manager. However, if the farm rejects a job, there is no mechanism to actively inform the distributed manager of that failure.

Feedback

Instead, the distributed manager must periodically check the compute farm to confirm that the expected queued jobs are still queued. This variable controls how often (in seconds) this check is performed.

During this check, if the distributed manager finds jobs that (1) are no longer queued, and (2) have not successfully launched and registered themselves with the distributed manager, it marks these jobs as unavailable.

Furthermore, if the remaining pending jobs and running jobs are no longer sufficient to meet the minimum worker requirement of the *start_hosts* command, then the command fails with a CMCR-037 error. (For details, see the message man page.)

If this variable value is set too high (infrequent checks), it can take a long time for the distributed manager to notice rejected jobs.

If this variable value is set too low (frequent checks), the frequent checks might cause performance degradation of the compute farm job manager.

The default value provides a reasonable tradeoff for noticing rejected jobs without imposing a performing degradation on a busy compute farm.

The interval value must be set to 60 seconds or more. It can also be set to 0 to disable the check (although this is not recommended).

This check does not apply to worker processes that terminate *after* successfully launching on the compute farm and registering themselves with the distributed manager.

This check applies only to jobs that use job-submission farm types (such as *sge*, *lsf*, and *rtda*). It does not apply to jobs that use direct-run farm types (such as *rsh*, *sh*, and *ssh*).

See Also

- set_host_options
- start_hosts

---

distributed_farm_protocol_error_detection_timeout

Specifies the maximum time that the distributed manager waits for a worker processes launched on a compute farm to come online.

Data Types

integer

Default 0

Feedback

Description

This variable specifies the maximum time in seconds that the distributed manager waits for worker processes that were defined by *set_host_options -protocol* with the *lsf*, *sge*, *rtda*, or *pbs* option.

The default of 0 means that no timeout is used.

See Also

- set_host_options

---

distributed_farm_submit_timeout

Specifies the maximum time, in seconds, that the distributed manager waits for an expected response from a compute farm after a worker job is submitted.

Data Types

integer

Default 120

Description

This variable specifies the maximum time in seconds that the distributed manager waits for a response from a computer farm following a worker job's submission. After a job has been submitted to a compute farm, the manager process expects the submit command to print the text to standard output stream containing farm job ID and then exit.

In some cases, the farm submit command may not return the expected output within the given maximum time period. For example, when the LSF batch submit command cannot communicate with the mbatchd daemon, it may display a repeated error message until the connection is re-established:

```
$ bsub [options] my_batch_script.sh
Cannot connect to LSF. Please wait ...
Cannot connect to LSF. Please wait ...
Cannot connect to LSF. Please wait ...
Cannot connect to LSF. Please wait ...
Job <1234> submitted to queue <simulation>.
```

See Also

- set_host_options
- start_hosts

Feedback

distributed_flush_file_system_buffers

Controls when file system buffers are to be flushed to disk when scenarios are saved to, or restored from disks.

Data Types

string

Default always

Description

In a distributed context, when a worker is saving or restoring an image, it is possible that the image will not have been completely flushed to disk. In this case some or all of the image could be in file buffers and any attempt to read this image could lead to instability. To bypass this potential instability PrimeTime will force a flush to disk by calling the "sh sync" command. This variable adds more granularity to the user to control when this flush will be called. The granularity is set by setting the variable to one of the following values:

never

- PrimeTime worker will not attempt to flush file system buffers to disk.

after_save

- PrimeTime worker will flush file system buffers after saving a session.

before_restore

- PrimeTime worker will flush file system buffers before restoring a session.

always

- PrimeTime worker will flush file system buffers after saving a session and before restoring a session.

This variable can only take effect if set in the DMSA manager process. Setting the variable to any value other than "always" could result in issues with restoring a session on slow (or) problematic networks.

distributed_heartbeat_period

Specifies the interval at which the distributed workers send heartbeats to the manager process.

Data Types

integer

Feedback

Default 0

Description

This variable specifies the interval (in seconds) that the distributed workers send heartbeats to the distributed manager. This mechanism allows the distributed manager to know if the workers are still online. Set *distributed_logging* to highest to see the heartbeat messages in the system_log directory.

The default of 0 means that the heartbeat mechanism is disabled. Minimum allowed value is 6.

---

distributed_heartbeat_timeout

Specifies the heartbeat timeout (in missed heartbeats). If the manager misses these many heartbeats (consecutively) from a worker, then the worker is assumed to have failed.

Data Types

integer

Default 10

Description

The number of heartbeat intervals the manager will wait for a worker to send a heartbeat. If the manager does not get a heartbeat within [heartbeat_period*heartbeat_timeout] then the worker is assumed to have failed. The action taken upon this event depends on the *multi_scenario_fault_handling variable*. Set *distributed_logging* to highest to see the heartbeat messages in the system_log directory.

The default of 10 means that the manager waits for 10 missed heartbeats before considering a worker as failed. Minimum allowed value is 10.

---

distributed_logging

Specifies the verbosity of logging during distributed analysis.

Data Types

string

Default medium

Description

This variable controls the verbosity of the system logs written by the manager and worker processes during distributed analysis.

Feedback

After you start workers by using the *start_hosts* command, the manager and worker processes write the following log files in the system_log directory in the distributed working directory:

Manager log files

- /system_log/manager.*hostname*.*processId*.*uniqueId*.bcast

  Contains the broadcast information the manager shares with its workers in order for them to connect back to the manager.

- /system_log/manager.*hostname*.*processId*.err

  Contains the stderr output the manager captures when launching worker processes, typically it is empty.

- /system_log/manager.*hostname*.*processId*.log

  Contains logged information regarding manager startup, worker process launching, task processing, and connection status.

Worker log file

- /system_log/worker.W*n*.*hostname*.*processId*.log

  Contains logged information regarding worker startup, task processing, and connection status.

In the names of the preceding log files:

- *hostname* - Name of the host on which the process is running
- *processId* - ID of the process
- *uniqueId* - Session specific ID
- *n* - Unique worker ID

To specify the verbosity of data written in the log files, set the *distributed_logging* variable to one of these values:

low

- The manager log is compressed and contains minimal data relating to the manager startup, worker launches, and task processing. You can access the compressed log content only after running the *stop_hosts* command or after the manager PrimeTime session exits.
- No worker logs are generated.

medium

Feedback

- The manager log is uncompressed and contains minimal data relating to the manager startup, worker launches, and task processing.
- No worker logs are generated.

high

- The manager log is uncompressed and contains minimal data relating to the manager startup, worker launches, task processing, and connection status.
- Each worker generates an uncompressed worker log that contains detailed worker startup, task processing, and connection status.

highest

- The manager log is uncompressed and contains verbose data relating to the manager startup, worker launches, task processing, and connection status.
- Each worker generates an uncompressed worker log that contains detailed worker startup, task processing, and connection status.

You must set the *distributed_logging* variable before launching workers. After you run the *start_hosts* command, you cannot change the value of the variable.

See Also

- start_hosts

---

distributed_sh_protocol_error_detection_timeout

Specifies the maximum time that the distributed manager waits for sh, rsh, and ssh protocol worker processes to come online.

Data Types

integer

Default 300

Description

This variable specifies the maximum time in seconds that the distributed manager waits for worker processes that were defined by *set_host_options -protocol* with the *sh*, *rsh*, or *ssh* option.

See Also

- set_host_options

Feedback

distributed_show_task_timestamp

Print a timestamp with a distributed task status message.

Data Types

boolean

Default false

Description

When set to *true*, enables printing of a timestamp as part of the message that shows the status of a distributed task.

Examples

With distributed_show_task_timestamp set to the default value of *false*:

pt_shell> *remote_execute {update_timing}* remote_execute {update_timing}

Start of Manager/Worker Task Processing ---------- Started : Baseline image generation for scenario 'scen1' Sourcing : Scenario common data files to setup scenario 'scen1' Succeeded : Baseline image generation for scenario 'scen1' 'scen1' providing baseline image for scenario 'scen2' Started : Command execution in scenario 'scen1' Started : Command execution in scenario 'scen2' Sourcing : Scenario specific data files to setup scenario 'scen1' Restoring : Baseline image for scenario 'scen2' Succeeded : Command execution in scenario 'scen1' Sourcing : Scenario specific data files to setup scenario 'scen2' Succeeded : Command execution in scenario 'scen2' ---------- End of Manager/Worker Task Processing

With distributed_show_task_timestamp set to *true*:

pt_shell> *remote_execute {update_timing}* remote_execute {update_timing}

[Wed Sep 27 12:36:49 2023] Start of Manager/Worker Task Processing ---------- [Wed Sep 27 12:36:49 2023] Started : Baseline image generation for scenario 'scen1' [Wed Sep 27 12:36:49 2023] Sourcing : Scenario common data files to setup scenario 'scen1' [Wed Sep 27 12:36:52 2023] Succeeded : Baseline image generation for scenario 'scen1' 'scen1' providing baseline image for scenario 'scen2' [Wed Sep 27 12:36:52 2023] Started : Command execution in scenario 'scen1' [Wed Sep 27 12:36:52 2023] Started : Command execution in scenario 'scen2' [Wed Sep 27 12:36:52 2023] Sourcing : Scenario specific data files to setup scenario 'scen1' [Wed Sep 27 12:36:52 2023] Restoring : Baseline image for scenario 'scen2' [Wed Sep 27 12:36:52 2023] Succeeded : Command execution in scenario 'scen1' [Wed Sep 27 12:36:54 2023] Sourcing : Scenario specific data files to setup scenario 'scen2' [Wed Sep 27 12:36:54 2023] Succeeded : Command execution in scenario 'scen2' ---------- [Wed Sep 27 12:36:54 2023] End of Manager/Worker Task Processing

Feedback

See Also

- remote_execute

---

distributed_worker_exit_action

Controls the behavior of the exit command at distributed worker processes.

Data Types

string

Default disabled

Description

Controls the behavior of exit and quit commands for worker processes in distributed analysis flows in the PrimeTime tool. The variable can be set only at the manager process. Allowed values are as follows:

- *disabled* (the default): The exit command is disabled at worker processes.

  In the Distributed Multi-Scenario Analysis (DMSA) flow, an exit command at a worker causes an error message to be issued in the manager log indicating that the exit is disabled.

- *terminate_process*: Terminates the process in which the exit command is issued.

  In the DMSA flow, this also removes the scenario that issued the exit command from the current session.

- *terminate_process_if_idle*: Terminates the process in which the exit command is issued only after all tasks that are scheduled to run inside of this process are completed.

  In the DMSA flow, this removes the scenario that issued the exit command from the current session. The process running the scenario is terminated unless it is needed to host another scenario due to an insufficient number of processes in the ONLINE state to run all scenarios in the command focus in parallel (image swapping).

- *remove_scenario*: In the DMSA flow, only removes the scenario that issued the exit command from the current session, without terminating the process.

The value of this variable can be changed only in the distributed manager shell. The variable is read-only at worker processes, with its value synchronized from the manager process.

Feedback

See Also

- start_hosts
- report_host_usage

distributed_working_directory

Specifies the working directory location for HyperGrid distributed analysis.

Data Types

string

Default "./da_work_dir"

Description

This variable specifies the location of the working directory for HyperGrid distributed analysis. This directory is used to store intermediate runtime data as well as logs and reports.

If unspecified, the default is to use a da_work_dir/ directory in the current directory.

The specified directory must be network accessible for all the distributed farm hosts launched with the *set_host_options* command. The *distributed_working_directory* variable must be set before you use the *start_hosts* command.

See Also

- set_host_options
- start_hosts
- distributed_enable_analysis

e

eco_allow_filler_cells_as_open_sites

Specifies whether physically aware ECO commands treat filler cells as open sites.

Data Types

Boolean

Default true

Feedback

Description

The *eco_allow_filler_cells_as_open_sites* variable specifies whether physically aware ECO commands treat filler cells as open sites. The default is *true*, which allows ECO commands to insert and resize cells in sites where placed filler cells exist.

Filler cells are defined in a Library Exchange Format (LEF) file. In the LEF file, the CLASS construct specifies the type of macro used in the design. CORE macros are used to specify standard cells used in the core area of the design. A CORE macro can be any one of the following types:

```
FEEDTHRU
TIEHIGH
TIELOW
SPACER
ANTENNACELL
```

Physically aware ECO commands treats CORE macros of type SPACER and FEEDTHRU as filler cells.

You can set this variable to one of these values:

- *true* (the default) - Physically aware ECO commands treat all filler cells marked PLACED as open sites.
- *false* - Physically aware ECO commands treats all filler cells marked PLACED as occupied sites.

In a distributed multi-scenario analysis (DMSA) environment, set this variable in the *remote_execute* command string to apply the variable setting to the worker process.

See Also

- fix_eco_timing
- set_eco_options

---

eco_allow_insert_buffer_always_on_cells

Specifies whether UPF always-on buffer insertion is allowed during ECO optimization.

Data Types

boolean

Default false

Feedback

Description

This variable specifies whether always-on buffers can be inserted during ECO optimization by the *fix_eco_timing* and *fix_eco_drc* commands.

By default, the variable is *false*, which causes ECO commands to skip always-on buffer insertion on all nets. When this variable is set to *true*, the ECO commands consider always-on buffering during optimization.

Setting the variable to *true* enables usage of buffers with the UPF-related library cell attribute *always_on* set to *true*. These buffers can be inserted by the *fix_eco_drc* or *fix_eco_timing* command dedicated to always-on buffer insertion.

Always-on buffer insertion is only performed when the net has at least one always_on cell at the driver or load. Buffer insertion does not check MV rules such as multi_voltage, upf, or power_domain.

In a distributed multi-scenario analysis (DMSA) environment, set this variable in the *remote_execute* command string to apply the variable setting to the worker process.

For detailed information about UPF cells in ECOs, see the man pages of the *fix_eco_timing* and *fix_eco_drc* commands.

See Also

- fix_eco_timing
- fix_eco_drc

---

eco_allow_insert_buffer_mixed_ao_regular_nets

Enables regular buffer insertion on nets with a mix of always-on and regular loads.

Data Types

Boolean

Default true

Description

This variable applies to multiple fanout nets with always-on and regular loads. When the variable is true, regular buffer insertion is allowed on regular load pins of multiple fanout nets with always-on and regular loads. When the variable is false, regular buffer insertion is not allowed on nets with always-on load. When eco_allow_insert_buffer_always_on_cells is on, always-on buffer is inserted instead of regular buffer on both always-on and regular loads.

Feedback

See Also

- fix_eco_timing

---

eco_allow_sizing_with_lib_cell_attributes

Specifies a list of library cell UPF attributes; cells with or more of these attributes set to true are enabled for sizing during ECO optimization.

Data Types

list

Default "" (empty)

Description

This variable specifies a list of library cell attributes, consisting of zero or more of the following UPF-related attributes:

```
always_on
is_isolation
is_level_shifter
is_retention
```

Cells with one or more of these attribute set to true are enabled for sizing by the ECO commands (*fix_eco_timing*, *fix_eco_drc*, and *fix_eco_power*).

By default, the variable is set to an empty string, which disables sizing of UPF cells (always-on, isolation, level shifter, and retention cells).

To allow UPF cells to be sized, set the variable to the applicable UPF types. For example, to allow sizing of isolation and level shifter cells:

```
pt_shell> set_app_var eco_allow_sizing_with_lib_cell_attributes \\
  {is_isolation is_level_shifter}
```

In a distributed multi-scenario analysis (DMSA) environment, set this variable in the *remote_execute* command string to apply the variable setting to the worker process.

For detailed information about UPF cells in ECOs, see the man pages of the *fix_eco_timing*, *fix_eco_drc*, and *fix_eco_power* commands.

See Also

- fix_eco_timing
- fix_eco_drc

Feedback

- fix_eco_power
- list_attributes

---

eco_alternative_area_ratio_threshold

Specifies the maximum allowable area increase when resizing a cell during the ECO fixing process.

Data Types

float

Default 2.0

Description

This variable specifies the maximum allowable area increase when resizing a cell during the ECO fixing process, expressed as a multiple of the original area. The default is 2.0, which limits the new cell size to twice the area of the original cell.

To resize cells without any area limit, set the variable to 0.

Limiting the area increase can improve the timing predictability after physical implementation of an ECO, as greatly enlarged cells are more likely to disturb the layout during incremental placement and routing, resulting in poor timing correlation.

Distributed Multi-Scenario Analysis (DMSA)

In a distributed multi-scenario analysis (DMSA) environment, set this variable to the same value at both the manager level (before the *remote_execute* command) and at the worker level (inside the *remote_execute* command string). Note that you can push the current manager value to all scenarios by using the *set_distributed_variables* command.

See Also

- fix_eco_drc
- fix_eco_timing

---

eco_alternative_cell_attribute_restrictions

Specifies lib_cell attributes that restrict cell sizing, such that only a library cell with matching values for these attributes can be used.

Data Types

string

Feedback

Default ""

Description

This variable restricts the choice of alternative library cells for the purposes of cell sizing. The variable accepts a list of one or more application or user-defined attributes of the lib_cell objects. To resize a cell, the values of the attributes for the replacement cell must match those of the original cell. This is in addition to the requirement that the replacement cell be logically equivalent.

This variable affects all PrimeTime commands that perform cell sizing, including *size_cell*, *fix_eco_timing*, *fix_eco_drc*, *estimate_eco*, and *get_alternative_lib_cells*.

Distributed Multi-Scenario Analysis (DMSA)

In a distributed multi-scenario analysis (DMSA) environment, set this variable to the same value at both the manager level (before the *remote_execute* command) and at the worker level (inside the *remote_execute* command string). Note that you can push the current manager value to all scenarios by using the *set_distributed_variables* command.

Examples

The following example restricts replacement cells used for sizing to have the same *area* (an application attribute) and the same *family* (a user-defined attribute):

```
pt_shell> define_user_attribute -classes lib_cell -type string family
pt_shell> set eco_alternative_cell_attribute_restrictions "area family"
```

See Also

- define_user_attribute
- fix_eco_timing
- fix_eco_drc
- get_alternative_lib_cells
- report_alternative_lib_cells
- set_user_attribute
- size_cell

eco_alternative_cell_instance_based_restrictions

Specifies a user-defined attribute on hierarchical and leaf cells to use for instance-based sizing restrictions.

Feedback

Data Types

string

Default ""

Description

This variable allows you to specify the name of a user-defined attribute to use for instance-based sizing restrictions.

This variable affects all PrimeTime commands related to cell sizing, including *size_cell*, *fix_eco_timing*, *fix_eco_drc*, *fix_eco_power*, *estimate_eco*, *get_alternative_lib_cells* and *report_alternative_lib_cells*.

This variable is similar to the *eco_alternative_cell_attribute_restrictions* variable, except that you set it to the name of a user-defined attribute that can define restriction lists on individual cell objects, rather than defining the restriction list itself.

The restriction attribute can be defined on leaf cells and hierarchical cells. Lower-level specifications take precedence over higher-level specifications. Cell-level restrictions applied using this variable take precedence over global specifications applied using the *eco_alternative_cell_attribute_restrictions* variable.

An empty restriction list (*{}*) disables restrictions at that level.

Distributed Multi-Scenario Analysis (DMSA)

In a distributed multi-scenario analysis (DMSA) environment, set this variable to the same value at both the manager level (before the *remote_execute* command) and at the worker level (inside the *remote_execute* command string). The applied instance based restrictions should also be consistent across all the scenarios. Note that you can push the current manager value to all scenarios by using the *set_distributed_variables* command.

Examples

The following command creates a user-defined attribute for restriction lists:

```
pt_shell> define_user_attribute -class cell -type string
 my_sizing_restrictions
pt_shell> set_app_var eco_alternative_cell_instance_based_restrictions
 {my_sizing_restrictions}
```

The following commands restrict resizing operations in hierarchical cell u_dsma/ to replacement cells that have the same *area* (an application attribute) and the same *family* (a user-defined attribute on lib_cell objects), and to specific leaf cell buffers in the block to the same family only:

```
pt_shell> set_user_attribute [get_cells u_dsma]
 my_sizing_restrictions {area family}
```

Feedback

```
pt_shell> set_user_attribute [get_cells u_dsma/Uclkbuf*]
 my_sizing_restrictions {family}
```

See Also

- define_user_attribute
- set_user_attribute
- get_alternative_lib_cells
- report_alternative_lib_cells
- size_cell
- fix_eco_timing
- fix_eco_power
- fix_eco_drc
- eco_alternative_cell_attribute_restrictions

---

eco_enable_fixing_clock_used_as_data

Enables buffer insertion at clock-used-as-data pins to fix hold timing violations.

Data Types

Boolean

Default true

Description

When this variable is set to *true* (the default), the *fix_eco_timing -type hold* command considers buffer insertion to fix hold timing violations at pins with the attribute *is_clock_used_as_data* set to *true*.

When this variable is set to *false*, the *fix_eco_timing -type hold* command does not insert buffers at these pins.

In a distributed multi-scenario analysis (DMSA) environment, set this variable in the *remote_execute* command string to apply the variable setting to the worker process.

See Also

- fix_eco_timing

Feedback

eco_enable_graph_based_refinement

Enables HyperTrace graph-based refinement for ECO fixing.

Data Types

Boolean

Default false

Description

Setting this variable to *true* enables HyperTrace accelerated ECO fixing. This feature requires a PrimeECO license.

HyperTrace is a technology that accelerates path-based analysis by computing refined graph-based timing data, then using it to drive path searching and recalculation. This variable enables the use of graph-based refinement for the *-pba_mode* option of ECO fixing commands, namely *fix_eco_timing* and *fix_eco_power*.

HyperTrace reporting is controlled by a separate variable, *timing_enable_graph_based_refinement*. For details, see the man page.

By default, this variable is set to *false*, which disables graph-based refinement for ECO fixing.

The *fix_eco_timing* and *fix_eco_power* commands implement HyperTrace ECO fixing differently, as described in the following sections.

HyperTrace ECO Fixing With *fix_eco_timing*

The *fix_eco_timing* command supports HyperTrace ECO fixing for the following *-pba_mode* option values:

- *-pba_mode exhaustive*
- *-pba_mode ml_exhaustive*

The *fix_eco_timing* command internally calls HyperTrace reporting to accelerate the exhaustive PBA search. As a result, you must ensure that the refinement slack thresholds used by HyperTrace reporting bound the implicit or explicit *-slack_lesser_than* value of the ECO fixing command.

For example,

```
set_app_var eco_enable_graph_based_refinement true
set_app_var timing_refinement_max_slack_threshold 0.05
fix_eco_timing -type setup -slack_lesser_than 0.05
```

Feedback

If you are fixing hold violations, note that HyperTrace is disabled for min-delay paths by default because hold violations are typically short paths that do not benefit from HyperTrace acceleration.

If your design has complex violating min-delay logic cones (such as I/O or memory interfaces) that benefit from HyperTrace acceleration, set the min-delay refinement slack threshold for reporting to your target slack value:

```
set_app_var timing_refinement_min_slack_threshold 0
fix_eco_timing -type hold
```

The *fix_eco_timing* command prints the following message to indicate that HyperTrace ECO fixing is used:

```
Information: Enabling HyperTrace exhaustive path-based analysis reporting
 to accelerate exhaustive PBA-based ECO fixing
(PTECO-116)
```

HyperTrace ECO Fixing With *fix_eco_power*

The *fix_eco_power* command supports HyperTrace ECO fixing for the following *-pba_mode* option values:

- *-pba_mode path*
- *-pba_mode exhaustive*
- *-pba_mode ml_exhaustive*

The *fix_eco_power* command incorporates HyperTrace algorithms and data directly into its ECO fixing algorithms. As a result, there is no need to set the refinement slack thresholds used for reporting.

For example,

```
set_app_var eco_enable_graph_based_refinement true
fix_eco_power ...
```

The *fix_eco_power* command prints the following message to indicate that HyperTrace ECO fixing is used:

```
Information: Running HyperTrace-based ECO fixing (PTECO-115)
```

See Also

- fix_eco_power
- fix_eco_timing
- timing_refinement_max_slack_threshold
- timing_refinement_min_slack_threshold

Feedback

eco_enable_mim

Enables ECOs with multiply intantiated modules (MIMs).

Data Types

Boolean

Default false

Description

When this variable is set to *true*, an ECO command (*fix_eco_timing*, *fix_eco_drc*, or *fix_eco_power*) identifies MIMs in the design, makes the same changes across the MIM instances, and writes out a single change list file for each set of MIMs.

For the detailed information about ECOs with MIMs, see the man pages of the *fix_eco_timing*, *fix_eco_drc*, and *fix_eco_power* commands.

If this variable is set to *true*, a PrimeTime-ADV license is checked out when fixing is performed.

In a distributed multi-scenario analysis (DMSA) environment, set this variable manager level, before the *remote_execute* command.

See Also

- fix_eco_timing
- fix_eco_drc
- fix_eco_power
- set_eco_options
- write_changes

eco_enable_overwrite_physical_design_path

Allows *set_eco_options -physical_design_path* to overwrite previously defined physical data file specifications for the same block.

Data Types

Boolean

Default true

Feedback

Description

This variable controls whether new *set_eco_options -physical_design_path* specifications overwrite previous specifications for the same physical block.

When this variable is set to *true* (the default),

- The DEF file specified by the -physical_design_path option is checked for existence.
- The DEF file is applied to the physical block specified by the *DESIGN* statement inside the DEF file.
- A physical constraint file specified by the -physical_constraint_file option also applies to that physical block.
- If a block already has DEF or physical constraint files associated with it, the new specification overwrites the previous specification.

When this variable is set to *false*,

- The specified DEF file is not checked for existence.
- Multiple specifications applied to the same physical block will cause the *check_eco* command to fail.

Set this variable to *false* only when backward compatibility with older tool versions is needed.

In a distributed multi-scenario analysis (DMSA) environment, set this variable in the *remote_execute* command string to apply the variable setting to the worker processes.

See Also

- set_eco_options

---

eco_enable_report_unfixed_reason_stdout

Enables or disables the *fix_eco_timing, fix_eco_drc* command to report the reasons that timing violations are not fixed in the standard output.

Data Types

boolean

Default true

Feedback

Description

This variable controls whether to report unfixable reasons in the standard output. This variable is only valid and effective when the *-unfixable_reasons_format* option is used and *eco_report_unfixed_reason_max_endpoints* variable is set properly.

When this variable is set to *true* (the default), the unfixable reasons are reported in the CSV/text file and also on the standard output.

When this variable is set to *false*, the unfixable reasons are reported in the CSV/text file and not on the standard output.

See Also

- fix_eco_timing
- fix_eco_drc
- eco_report_unfixed_reason_max_endpoints

---

eco_endpoint_merge_session_list

Specifies the list of PrimeTime saved sessions that the *write_eco_design* command use to merge endpoints.

Data Types

string

Default "" (empty)

Description

When this variable specifies PrimeTime saved session directory paths, the *write_eco_design* command finds endpoint information from the sessions, merges them, and writes an ECO design that covers the merged endpoints.

Note that the PrimeTime saved sessions must be saved with the eco_save_session_data_type variable specified to store failing endpoints and other ECO information.

If the variable is not specified or an empty string, the *write_eco_design* command finds failing endpoints from its own session.

Suppose there are two independent STA scripts that save sessions. The following shows the two scripts that run in Host1 and Host2 independently and save two sessions. Note that the eco_save_sessions_data_type variables are specified to save failing timing endpoint data.

Feedback

```
# Script STA for FF running in Host1
read_verilog...
update_timing...
set eco_save_sessions_data_type timing
save_session $my_sessions/STA_SESSION_FF

# Script STA for SS running in Host2
read_verilog...
update_timing...
set eco_save_sessions_data_type timing
save_session $my_sessions/STA_SESSION_SS
```

Once the scripts complete and save PrimeTime sessions, the following two scripts write ECO designs after merging endpoints from the two saved sessions. In this way, DMSA is no longer required to run the write_eco_design command.

```
# Script to write ECO design for FF running in Host1
restore_session $my_sessions/STA_SESSIONS_FF
set eco_endpoint_merge_session_list \
"$my_sessions/STA_SESSION_FF $my_sessions/STA_SESSION_SS"
write_eco_design $rreco_db/FF


# Script to write ECO design for SS running in Host2
restore_session $my_sessions/STA_SESSIONS_SS
set eco_endpoint_merge_session_list \
"$my_sessions/STA_SESSION_FF $my_sessions/STA_SESSION_SS"
write_eco_design $rreco_db/SS
```

Once the ECO designs are written by the two scripts above, the rest of the flow is as same as the current flow; the following example shows that the *read_eco_design* command in DMSA manager detects two ECO designs in $rreco_db directory, launches two workers to read ECO designs, and runs fix_eco_timing or fix_eco_drc commands.

```
# DMSA script to read the ECO design for SS and FF scenarios
read_eco_design $rreco_db
```

See Also

- write_eco_design
- read_eco_design

---

eco_estimation_output_columns

Specifies the output columns to be displayed by the *estimate_eco* command for nonverbose reporting.

Data Types

string

Feedback

Default {area stage_delay arrival slack}

Description

This variable specifies the columns of data displayed by the *estimate_eco* command for nonverbose reporting. By default, columns labeled *area*, *stage_delay*, and *slack* are displayed.

Considering an estimated stage that consists of the estimated cell, its driver pin, and its load pin in the slack-critical path, the following keywords are available:

- *area* - Area of the specified library cell
- *area_slack* - The amount of open area available in the site row immediately surrounding the new cell after it replaces the existing one. An area slack of 0.0 means that the new cell abuts the two adjacent cells. A negative area slack means that the new cell overlaps the adjacent cells.
- *stage_delay* - Delay from the input pin of the current cell to the most slack-critical load pin
- *arrival* - Arrival time at the most slack-critical load pin
- *slack* - Timing slack at the most slack-critical load pin
- *transition* - Transition time at the most slack-critical load pin
- *min_transition* - Worst min_transition DRC value for any pin in the estimated stage
- *min_capacitance* - Worst min_capacitance DRC value for any pin in the estimated stage
- *max_transition* - Worst max_transition DRC value for any pin in the estimated stage
- *max_capacitance* - Worst max_capacitance DRC value for any pin in the estimated stage
- *max_fanout* - Worst max_fanout DRC value for any pin in the estimated stage
- *data_pin_slack* - Timing slack at the data pin of the current sequential cell

The DRC keywords return the worst design rule constraint value for all pins in the stage, including all load pins.

Distributed Multi-Scenario Analysis (DMSA)

In a distributed multi-scenario analysis (DMSA) environment, set this variable to the same value at both the manager level (before the *remote_execute* command) and at the worker level (inside the *remote_execute* command string). Note that you can push the current manager value to all scenarios by using the *set_distributed_variables* command.

Feedback

Examples

To display only area, slack, and maximum transition information,

```
pt_shell> set_app_var eco_estimation_output_columns {area slack
 max_transition}
```

See Also

- estimate_eco

---

eco_insert_buffer_search_distance_in_site_rows

Specifies the maximum distance from a violating pin that ECO commands search for open sites for buffer insertion, expressed as a multiple of the site row height.

Data Types

integer

Default 8

Description

This variable specifies the maximum distance away from a violating pin that ECO commands search for open sites for buffer insertion, expressed as a multiple of the site row height. By setting this variable, you can specify the search distance as a number of site rows without knowing the actual distance. This setting affects commands that perform buffer insertion at pins, such as *fix_eco_timing* and *fix_eco_drc*.

The default is 8, which restricts the search for open sites to no more than eight site rows above or below the violating pin, and within the same distance to the left and right. Thus, the search area for open sites is a square measuring 16 by 16 site row heights, centered on the violating pin.

Setting this variable to a larger number increases the chances of finding an open site when few are available. However, this may result in longer wire routes and more runtime to reach timing convergence. Setting the variable to a lower number ensures usage of closer sites and shorter routes, but less flexibility for finding open sites.

This variable only affects buffer insertion performed at violating pins in the physically aware ECO flow, in which the physical data has been loaded from LEF/DEF files or the IC Compiler II database as specified by the *set_eco_options* command. It does not affect on-route buffer insertion performed on long nets and high-fanout nets.

In a distributed multi-scenario analysis (DMSA) environment, set this variable in the *remote_execute* command string to apply the variable setting to the worker process.

Feedback

See Also

- set_eco_options
- fix_eco_drc
- fix_eco_timing

---

eco_instance_name_prefix

Specifies the prefix used for naming new cell instances created by the *insert_buffer* command.

Data Types

string

Default U

Description

By default, when creating new instances, the *insert_buffer* command uses the prefix letter "U" with sequential numbers for the instance names: U1, U2, U3, and so on.

To specify a different instance name prefix, set the *eco_instance_name_prefix* variable. The following rules apply:

- The first character must be A-Z or a-z.
- Subsequent characters must be A-Z, a-z, 0-9, or an underscore (_).

An instance number is appended to this prefix to form the new cell name. If the resulting cell name is already used, the instance number is incremented until an unused name is found.

In a distributed multi-scenario analysis (DMSA) environment, set this variable manager level, before the *remote_execute* command. The tool uses the variable setting in the manager regardless of the variable setting in the workers.

Examples

An engineering change order (ECO) is made during the second iteration of timing closure. You set this variable so that newly created buffer and inverter instances are named in an easily identifiable manner:

```
pt_shell> set eco_instance_name_prefix {Ueco2_}
Ueco2_
pt_shell> insert_buffer U2/Y slow/BUFX2
Information: Inserted 'Ueco2_1' at 'U2/Y'. (NED-046)
{"Ueco2_1"}
pt_shell> insert_buffer U2319/A slow/BUFX2
```

Feedback

```
Information: Inserted 'Ueco2_2' at 'U2/A'. (NED-046)
{"Ueco2_2"}
```

See Also

- insert_buffer
- eco_net_name_prefix

---

eco_leakage_exclude_unconstrained_cells

Specifies whether to exclude unconstrained cells during leakage recovery.

Data Types

Boolean

Default false

Description

By default, the *fix_eco_leakage* command swaps unconstrained cells with preferred library cells during leakage recovery to maximize leakage power reduction.

To prevent the *fix_eco_leakage* command from using unconstrained cells during leakage recover, set the *eco_leakage_exclude_unconstrained_cells* variable to *true*.

In the distributed multi-scenario analysis flow:

- A cell is considered to be unconstrained only if it is unconstrained across all scenarios. If the cell is constrained in at least one scenario, the cell is considered to be constrained.
- You must set this variable in the manager. The tool honors the variable setting only in the manager and ignores the variable setting in the workers.

See Also

- fix_eco_leakage

---

eco_mim_preserve

Preserves multiply instantiated module (MIM) grouping during buffer insertion and cell sizing.

Data Types

Boolean

Feedback

Default true

Description

When this variable is set to *true*, the *insert_buffer*, *remove_buffer*, and *size_cell* editing commands automatically apply the same changes to all instances in each MIM group, thereby preserving the grouping of each MIM group. It does not matter whether you use the *-all_mim_instances* option in the editing command.

When this variable is set to *false*, editing commands can modify some cells of a MIM group without affecting other others, causing the modified cells to be removed from the group. To apply the same changes to all instances in each MIM group, you can either use the *-all_mim_instances* option in the editing command or set the *eco_mim_preserve* variable to *true*.

To similarly preserve the MIM grouping during ECO fixing, set the *eco_enable_mim* variable to *true* before you use the *fix_eco_timing*, *fix_eco_drc*, or *fix_eco_power* command.

See Also

- fix_eco_drc
- fix_eco_timing
- fix_eco_power
- insert_buffer
- remove_buffer
- size_cell
- eco_enable_mim

---

eco_net_name_prefix

Specifies the prefix used for naming new nets created by the *insert_buffer* command.

Data Types

string

Default net

Description

By default, when creating new nets, the *insert_buffer* command uses the prefix string "net" with sequential numbers for the net names: net1, net2, net3, and so on.

Feedback

To specify a different net name prefix, set the *eco_net_name_prefix* variable. The following rules apply:

- The first character must be A-Z or a-z.
- Subsequent characters must be A-Z, a-z, 0-9, or an underscore (_).

A net number is appended to this prefix to form the new net name. If the resulting net name is already used, the net number is incremented until an unused name is found.

In a distributed multi-scenario analysis (DMSA) environment, set this variable manager level, before the *remote_execute* command. The tool uses the variable setting in the manager regardless of the variable setting in the workers.

Examples

An engineering change order (ECO) is made during the second iteration of timing closure. You set this variable so that newly created buffer and inverter nets are named in an easily identifiable manner:

```
pt_shell> set eco_net_name_prefix {NETeco2_}
NETeco2_
pt_shell> insert_buffer U2/Y slow/BUFX2
Information: Inserted 'U1' at 'U2/Y'. (NED-046)
{"U1"}
pt_shell> all_connected [get_pins U1/Z]
{"NETeco2_1"}
```

See Also

- insert_buffer
- eco_instance_name_prefix

---

eco_physical_disable_eco_on_fixed_cells

Disables sizing (as well as moving) of cells that have the FIXED attribute in DEF by physically aware ECO commands.

Data Types

Boolean

Default false

Description

The *eco_physical_disable_eco_on_fixed_cells* variable specifies whether physically aware ECO can size FIXED cells. The default is false, which allows ECO commands to size these cells.

Feedback

FIXED cells are defined in the Design Exchange Format (DEF) file, which defines the placement of the physical cells, as shown in the following example.

```
COMPONENTS 1024 ;
 - cell1 BUFX1 + FIXED ( 28160 136080 ) N ;
END COMPONENT
```

In the DEF file, a cell component can be specified with the following attribute:

```
FIXED
COVER
PLACED
UNPLACED
```

When a cell is defined with the FIXED attribute, its origin is fixed to specific coordinates on the site row. Physically aware ECO cannot move the cell from the coordinate of its fixed origin. However, the cell can be sized larger or smaller as long as cell origin does not change.

You enable or disable physically aware ECO sizing on FIXED cells by setting the variable to one of the following values:

- *true* - Physically aware ECO cannot size or move FIXED cells.
- *false* (default) - Physically aware ECO can size FIXED cells.

For physical ECO in a distributed multi-scenario analysis (DMSA) environment, set this variable inside the *remote_execute* command in order to apply this variable to the worker host processes.

See Also

- fix_eco_timing
- set_eco_options

---

eco_physical_enable_route_preservation

Enables routing guides output in ecotclout for PnR to preserve routing topologies of multi-fanout nets.

Data Types

boolean

Default false

Feedback

Description

Builds global routing connections (g-links) from original pin locations to the new locations after ECO operations such as cell sizing, move, and buffer insertion.

Passes the g-links to PnR tool using *create_shape* and *create_via* commands in the changelist script generated by *write_changes*. Currently supports ICC2 syntax only.

Must be set before any netlist editing operations (either manual or automatic).

In a distributed multi-scenario analysis (DMSA) environment, set this variable in the *remote_execute* command string to apply the variable setting to the worker process.

See Also

- write_changes
- fix_eco_timing

---

eco_physical_match_site_row_names

Enables site-aware physical ECO fixing, which restricts placement of library cells to specific site rows, as defined in the LEF and DEF files.

Data Types

Boolean

Default false

Description

When this variable is set to *true*, the ECO fixing commands (*fix_eco_timing*, *fix_eco_drc*, and *fix_eco_power*) restrict the placement of new library cells to specific site rows, as defined in LEF and DEF files invoked by the *set_eco_options* command. When the variable is set to *false* (the default), ECO fixing does not restrict the placement of cells to specific site rows.

To use site-aware physical ECO fixing, the LEF file must define the site name for the library cell, as shown in the following example.

```
MACRO BUFLVT1
  CLASS CORE ;
  ORIGIN 0 0 ;
  SIZE 1.632 BY 0.768 ;
  SYMMETRY X Y ;
  SITE 2Tunit ;
  ...
```

Feedback

Furthermore, the DEF file must identify the same site name in the row definition, as shown in the following example.

```
ROW _row_1 unit 1500 1488 FS DO 4080 BY 1 STEP 96 0 ;
ROW _row_2 unit 1500 1872 N DO 4080 BY 1 STEP 96 0 ;
 ...
ROW 2Xrow_1 2Tunit 1500 1680 N DO 4080 BY 1 STEP 96 0 ;
ROW 2Xrow_2 2Tunit 1500 2448 N DO 4080 BY 1 STEP 96 0 ;
ROW 2Xrow_3 2Tunit 1500 3216 N DO 4080 BY 1 STEP 96 0 ;
 ...
```

The following PrimeTime script performs ECO fixing in default mode using single-height cells followed by site-aware ECO fixing using double-height cells in 2Tunit rows.

```
set_eco_options \\
  -physical_tech_lib_path technology.lef.gz \\
  -physical_lib_path library.lef.gz \\
  -physical_design_path design.def.gz \\
  -physical_lib_constraint_file icc2_adv_tech_rule.gz \\
  -log_file lef_def.log

# Keep 2Tunit site name data when reading LEF/DEF physical data
set_eco_options -keep_site_names {2Tunit}

# Check and read in LEF/DEF physical data
report_eco_options
check_eco

# Run default ECO with single-height cells, $SINGLE_BUF buffer list
set_app_var eco_physical_match_site_row_names false
fix_eco_drc -type max_transition -physical_mode open_site \\
  -buffer_list $SINGLE_BUF -verbose
fix_eco_timing -type setup -physical_mode open_site -verbose
fix_eco_timing -type hold -physical_mode open_site \\
  -buffer_list $SINGLE_BUF -verbose

# Run site-aware ECO with double-height cells, $DOUBLE_BUF buffer list
set_app_var eco_physical_match_site_row_names true
fix_eco_drc -type max_transition -physical_mode open_site \\
  -buffer_list $DOUBLE_BUF -verbose
fix_eco_timing -type setup -physical_mode open_site -verbose
fix_eco_timing -type hold -physical_mode open_site \\
  -buffer_list $DOUBLE_BUF -verbose
```

For distributed multi-scenario analysis (DMSA), set the *eco_physical_match_site_row_names* variable in the worker process, not the manager.

For more information on preparing for site-aware physical ECO fixing, see the man page for the *set_eco_options* command.

Feedback

See Also

- fix_eco_timing
- fix_eco_drc
- fix_eco_power
- set_eco_options

---

eco_power_exclude_unconstrained_cells

Specifies whether to exclude unconstrained cells during power recovery.

Data Types

Boolean

Default false

Description

By default, the *fix_eco_power* command can resize or swap unconstrained cells with preferred library cells during power recovery to maximize power reduction.

To prevent the *fix_eco_power* command from using unconstrained cells during power recovery, set the *eco_power_exclude_unconstrained_cells* variable to *true*.

This variable has no effect on buffer removal by the *fix_eco_power* command. Only buffers that have both setup and hold constraints can be removed by the command.

In the distributed multi-scenario analysis (DMSA) flow:

- A cell is considered to be unconstrained only if it is unconstrained across all scenarios. If the cell is constrained in at least one scenario, the cell is considered to be constrained.
- You must set this variable in the manager. The tool honors the variable setting only in the manager and ignores the variable setting in the workers.

See Also

- fix_eco_power

---

eco_power_report_max_unusable_cells

Enables the *fix_eco_power* command to report the reasons that the power recovery process are not performed and specifies the maximum number of unusable cells listed in the report.

Feedback

Data Types

integer

Default 0

Description

This variable specifies the maximum number of unusable cells listed in the unusable cells report generated by the *fix_eco_power* command. By default, the variable is set to 0, which disables the unusable cells report. To enable the unusable cells reason report, set this variable to a positive number.

In a distributed multi-scenario analysis (DMSA) environment, set this variable manager level before running the *remote_execute* command.

See Also

- fix_eco_power

---

eco_print_unfixed_reason_to_stdout_in_csv_or_txt_format

Enables or disables the *fix_eco_timing, fix_eco_drc* command to report the reasons that timing violations are not fixed in the standard output.

Data Types

boolean

Default true

Description

This variable controls whether to report unfixable reasons in the standard output. This variable is only valid and effective when the *-unfixable_reasons_format* option is used and *eco_report_unfixed_reason_max_endpoints* variable is set properly.

When this variable is set to *true* (the default), the unfixable reasons are reported in the CSV/text file and also on the standard output.

When this variable is set to *false*, the unfixable reasons are reported in the CSV/text file and not on the standard output.

See Also

- fix_eco_timing
- fix_eco_drc
- eco_report_unfixed_reason_max_endpoints

Feedback

eco_report_unfixed_reason_max_endpoints

Enables the *fix_eco_timing* command to report the reasons that timing violations are not fixed and specifies the maximum number of violating endpoints listed in the report.

Data Types

integer

Default 0

Description

This variable specifies the maximum number of violating endpoints listed in the unfixed reason report generated by the *fix_eco_timing* command. By default, the variable is set to 0, which disables reporting of unfixed violation reasons. To enable the unfixed violation reason report, set this variable to a positive number.

In a distributed multi-scenario analysis (DMSA) environment, set this variable manager level, before the *remote_execute* command.

See Also

- fix_eco_timing

eco_save_session_data_type

Specifies violation data types to be saved and used by the *write_eco_design* command in the reduced-resource ECO DMSA flow.

Data Types

string

Default "" (empty)

Description

Set this variable to the names of one or more violation types. The *save_session* command saves additional ECO information related to these violation types for DMSA analysis. The *write_eco_design* command in DMSA mode creates an ECO design containing the additional information.

You can specify any one or more of the following keywords:

- *setup* - Setup timing violation
- *hold* - Hold timing violation
- *max_transition* - Maximum transition violation

Feedback

- *max_capacitance* - Maximum capacitance violation
- *max_fanout* - Maximum fanout violation
- *noise* - Noise violation

The following keywords are also accepted:

- *timing* - Same as *{setup hold}*
- *drc* - Same as *{max_transition max_capacitance max_fanout}*

Distributed Multi-Scenario Analysis (DMSA)

In a distributed multi-scenario analysis (DMSA) environment, set this variable to the same value at both the manager level (before the *remote_execute* command) and at the worker level (inside the *remote_execute* command string). Note that you can push the current manager value to all scenarios by using the *set_distributed_variables* command.

Examples

The following example saves both setup and hold violation ECO information.

```
pt_shell> set eco_save_session_data_type {setup hold}
pt_shell> save_session my_session
```

The following example saves max_transition violation ECO information.

```
pt_shell> set eco_save_session_data_type max_transition
pt_shell> save_session my_session
```

The following example saves setup, hold, and noise violation ECO information.

```
pt_shell> set eco_save_session_data_type {setup hold noise}
pt_shell> save_session my_session
```

The following example saves setup, hold, and noise violation ECO information using the *timing* keyword.

```
pt_shell> set eco_save_session_data_type {timing noise}
pt_shell> save_session my_session
```

The following example saves setup, hold, max_transition, max_capacitance, max_fanout, and noise violation ECO information.

```
pt_shell> set eco_save_session_data_type {timing drc noise}
pt_shell> save_session my_session
```

Feedback

See Also

- write_eco_design
- read_eco_design

---

eco_strict_lib_arc_equivalence

Specifies whether cell sizing operations require the replacement cell to have exactly the same timing arcs as the original cell.

Data Types

Boolean

Default true

Description

If this variable is set to *true* (the default), the tool can not size cells when the timing arcs do not match between the original and replacement cells. In that case, the tool considers a replacement cell to be sizeable only when its timing arcs exactly match those of the original cell.

If this is not the behavior you want, set this variable to *false*. The tool can size cells even when the timing arcs do not match between the original and replacement cells. The tool uses other library cell information to find the matching pin functionality between the original and replacement cells.

The following checks will be relaxed:

- Matching number of timing arcs
- Matching from/to pins of timing arcs
- Matching senses of timing arcs
- Matching 'when' conditions of timing arcs

This variable relaxes the timing arc requirements for manual ECO operations. It affects the following commands:

- *size_cell*
- *swap_cell* (allows better pin remapping between cells with different arcs)
- *get_alternative_lib_cells*
- *report_alternative_lib_cells*

This variable does not affect the *estimate_eco* or *fix_eco_** commands.

Feedback

Distributed Multi-Scenario Analysis (DMSA)

In a distributed multi-scenario analysis (DMSA) environment, set this variable to the same value at both the manager level (before the *remote_execute* command) and at the worker level (inside the *remote_execute* command string). Note that you can push the current manager value to all scenarios by using the *set_distributed_variables* command.

See Also

- get_alternative_lib_cells
- report_alternative_lib_cells
- size_cell

---

eco_strict_pin_name_equivalence

Specifies whether cell sizing operations require the replacement cell to have exactly the same pin names as the original cell.

Data Types

Boolean

Default false

Description

If this variable is set to *false* (the default), the tool can size cells even when the pin names do not match between the original and replacement cells. The tool uses other library cell information to find the matching pin functionality between the original and replacement cells.

If this is not the behavior you want, set this variable to *true*. In that case, the tool considers a replacement cell to be functionally equivalent only when its pin names exactly match those of the original cell.

Distributed Multi-Scenario Analysis (DMSA)

In a distributed multi-scenario analysis (DMSA) environment, set this variable to the same value at both the manager level (before the *remote_execute* command) and at the worker level (inside the *remote_execute* command string). Note that you can push the current manager value to all scenarios by using the *set_distributed_variables* command.

Feedback

See Also

- get_alternative_lib_cells
- report_alternative_lib_cells
- size_cell

---

eco_update_path_timing

Enables *estimate_eco* working in fast-estimation mode.

Data Types

Boolean

Default false

Description

In order to achieve better Qor quality, before *estimate_eco* engines starts working, timing should be updated for new netlist changes firstly. Normally, doing update-timing means more runtime penalty in estimate_eco engine.

When this variable is set to *true*, *estimate_eco* will do fast estimation without triggering timing update for latest netlist changes. As timing is not updated, we might see some accumulated Qor difference due to continuous netlist changes.

Commands like *fix_eco_timing*, *fix_eco_drc*, and *fix_eco_power* will not be impacted by this variable.

See Also

- estimate_eco

---

eco_voltage_slack_target

Specifies the target voltage slack in the slack shift calculation of voltage slack analysis.

Data Types

float

Default 0.0

Description

In voltage slack analysis, the slack shift will be calculated with a specific voltage slack if *eco_voltage_slack_target* is a non-zero value. The default value is 0.0, which means that

Feedback

the slack will not be recalculated. If *eco_voltage_slack_target* is a non-zero value, the slack shift will be calculated and stored in path attribute *slack_shift*.

See Also

- fix_eco_robustness

---

eco_write_changes_prepend_libfile_to_libcell

Prepends the link library file name to the library name and library cell references in the *write_changes* command change list.

Data Types

Boolean

Default false

Description

After you make changes to the design using ECO commands, the *write_changes* command writes out the commands that perform the changes. By default, the *insert_buffer* and *size_cell* commands written by the *write_changes* commands refer to each newly inserted or resized cell by the simple library cell name only, for example, "BUFFD2".

To change this behavior, set one or both of the following variables:

- *eco_write_changes_prepend_libname_to_libcell*: When true, prepends the library name to the library cell name, for example, "cb13_max/BUFFD2"
- *eco_write_changes_prepend_libfile_to_libcell*: When true, prepends the library file to the library name and library cell name, for example, "sc_max.db:cb13_max/BUFFD2"

If you set these variables, you must do so *before* you run the ECO fixing commands, as shown in the following examples.

By default, the *insert_buffer* and *size_cell* commands refer to the new cell by the simple library cell name:

```
pt_shell> fix_eco_timing -type setup
  ...
pt_shell> write_changes
############################################################
# Change list, formatted for PrimeTime
############################################################
current_instance
current_instance {I_ORCA_TOP/I_CONTEXT_MEM}
size_cell {out_bus_reg_8_675} {BUFFD2}
size_cell {out_bus_reg_8_674} {BUFBD2}
current_instance
```

Feedback

Set the *eco_write_changes_prepend_libname_to_libcell* variable to *true* to prepend the library name to the cell name:

```
pt_shell> set_app_var eco_write_changes_prepend_libname_to_libcell true
 ...
pt_shell> fix_eco_timing -type setup
 ...
pt_shell> write_changes
#########################################################
# Change list, formatted for PrimeTime
#########################################################
current_instance
current_instance {I_ORCA_TOP/I_CONTEXT_MEM}
size_cell {out_bus_reg_8_675} {cb13_max/BUFFD2}
size_cell {out_bus_reg_8_674} {cb13_max/BUFBD2}
current_instance
```

Also set the *eco_write_changes_prepend_libfile_to_libcell* variable to *true* to prepend the library file name to the library name and cell name. This fully specifies the library cell reference:

```
pt_shell> set_app_var eco_write_changes_prepend_libname_to_libcell true
 ...
pt_shell> set_app_var eco_write_changes_prepend_libfile_to_libcell true
 ...
pt_shell> fix_eco_timing -type setup
 ...
pt_shell> write_changes
#########################################################
# Change list, formatted for PrimeTime
#########################################################
current_instance
current_instance {I_ORCA_TOP/I_CONTEXT_MEM}
size_cell {out_bus_reg_8_675} {sc_max.db:cb13_max/BUFFD2}
size_cell {out_bus_reg_8_674} {sc_max.db:cb13_max/BUFBD2}
current_instance
```

Distributed Multi-Scenario Analysis (DMSA)

In a distributed multi-scenario analysis (DMSA) environment, set this variable to the same value at both the manager level (before the *remote_execute* command) and at the worker level (inside the *remote_execute* command string). Note that you can push the current manager value to all scenarios by using the *set_distributed_variables* command.

See Also

- create_cell
- insert_buffer

Feedback

- size_cell
- eco_write_changes_prepend_libname_to_libcell

---

eco_write_changes_prepend_libname_to_libcell

Prepends the link library name to library cell references in the *write_changes* change list.

Data Types

Boolean

Default false

Description

After you make changes to the design using ECO commands, the *write_changes* command writes out the commands that perform the changes. By default, the *insert_buffer* and *size_cell* commands written by the *write_changes* commands refer to each newly inserted or resized cell by the simple library cell name only, for example, "BUFFD2".

To change this behavior, set one or both of the following variables:

- *eco_write_changes_prepend_libname_to_libcell*: When true, prepends the library name to the library cell name, for example, "cb13_max/BUFFD2"
- *eco_write_changes_prepend_libfile_to_libcell*: When true, prepends the library file to the library name and library cell name, for example, "sc_max.db:cb13_max/BUFFD2"

If you set these variables, you must do so *before* you run the ECO fixing commands, as shown in the following examples.

By default, the *insert_buffer* and *size_cell* commands refer to the new cell by the simple library cell name:

```
pt_shell> fix_eco_timing -type setup
  ...
pt_shell> write_changes
############################################################
# Change list, formatted for PrimeTime
############################################################
current_instance
current_instance {I_ORCA_TOP/I_CONTEXT_MEM}
size_cell {out_bus_reg_8_675} {BUFFD2}
size_cell {out_bus_reg_8_674} {BUFBD2}
current_instance
```

Feedback

Set the *eco_write_changes_prepend_libname_to_libcell* variable to *true* to prepend the library name to the cell name:

```
pt_shell> set_app_var eco_write_changes_prepend_libname_to_libcell true
 ...
pt_shell> fix_eco_timing -type setup
 ...
pt_shell> write_changes
#########################################################
# Change list, formatted for PrimeTime
#########################################################
current_instance
current_instance {I_ORCA_TOP/I_CONTEXT_MEM}
size_cell {out_bus_reg_8_675} {cb13_max/BUFFD2}
size_cell {out_bus_reg_8_674} {cb13_max/BUFBD2}
current_instance
```

Also set the *eco_write_changes_prepend_libfile_to_libcell* variable to *true* to prepend the library file name to the library name and cell name. This fully specifies the library cell reference:

```
pt_shell> set_app_var eco_write_changes_prepend_libname_to_libcell true
 ...
pt_shell> set_app_var eco_write_changes_prepend_libfile_to_libcell true
 ...
pt_shell> fix_eco_timing -type setup
 ...
pt_shell> write_changes
#########################################################
# Change list, formatted for PrimeTime
#########################################################
current_instance
current_instance {I_ORCA_TOP/I_CONTEXT_MEM}
size_cell {out_bus_reg_8_675} {sc_max.db:cb13_max/BUFFD2}
size_cell {out_bus_reg_8_674} {sc_max.db:cb13_max/BUFBD2}
current_instance
```

Distributed Multi-Scenario Analysis (DMSA)

In a distributed multi-scenario analysis (DMSA) environment, set this variable to the same value at both the manager level (before the *remote_execute* command) and at the worker level (inside the *remote_execute* command string). Note that you can push the current manager value to all scenarios by using the *set_distributed_variables* command.

See Also

- create_cell
- insert_buffer

Feedback

- size_cell
- eco_write_changes_prepend_libfile_to_libcell

---

enable_golden_constraints_reader

Enables the golden constraint reader.

Data Types

Boolean

Default false

Description

When you set this variable to *true*, PrimeTime supports the golden constraint reading with load_constraint command.

See Also

- load_constraints
- sdc_name_map

---

enable_golden_upf

Enables the golden UPF flow.

Data Types

Boolean

Default false

Description

When you set this variable to *true*, PrimeTime supports the golden UPF flow.

See Also

- load_upf

---

enable_license_auto_reduction

Determines whether or not the manager returns licenses to the license server after a worker has finished using them.

Feedback

Data Types

Boolean

Default false

Description

When a worker finishes processing a task, it returns the licenses it used to the manager. When the *enable_license_auto_reduction* variable is set to its default of *false*, the manager keeps the licenses checked out for future worker usage. When you set this variable to *true*, the manager returns the licenses it receives from the workers back to the license server unless another worker has already requested usage of that license.

The *enable_license_auto_reduction* variable should be enabled with care. When the manager returns the licenses to the license server it might not be able to acquire them again. This reduces the number of workers that can execute concurrently leading to a degradation in performance during the subsequent analysis.

To activate automatic license reduction, set the variable to *true*.

```
pt_shell> set_app_var enable_license_auto_reduction true
```

---

enable_page_mode

This is a synonym for the *sh_enable_page_mode* variable.

Default false

See Also

- sh_enable_page_mode

---

enable_path_tagging

Enables path tagging by the *create_path_tag_set* command.

Data Types

Boolean

Default false

Description

By default, path tagging is disabled. To enable path tagging, set the *enable_path_tagging* variable to *true*.

Feedback

See Also

- create_path_tag_set
- get_timing_paths
- report_path_tag_set

---

enable_rule_based_query

Enables or disables rule-based matching.

Data Types

Boolean

Default false

Description

When this variable is set to *true*, rule-based matching is enabled. However, you must run the *set_query_rules* command first. If you set this variable to *true* without running the *set_query_rules* command first, the default query rules are used.

When this variable is set to *true*, the runtime for the query might be slower. Disable rule-based matching when it is no longer needed.

See Also

- set_query_rules

---

env_variables_violations

This man page describes *env_variables* hierarchical boundary check violations shown by the *report_constraint* command in the HyperScale flow.

Description

An *env_variables* violation indicates differences in the environment variables between block- and top-level analysis. The tool checks only the application-defined Tcl variables that affect the results (QoR) of the timing analysis.

What Next

This is a *non_auto_fixable* violation. You must resolve the difference in variable settings to ensure consistent results for the timing analysis.

Feedback

Examples

The following verbose report shows a simple difference of top level enabling CCS calculation while block level uses non-CCS.

```
pt_shell> report_constraint -boundary_check_include {env_variables}
 -all_violators -verbose

****************************************
Report : constraint
         -verbose
Design : wrapper
 ...
****************************************

HyperScale constraints report

   Constraint: env_variables

   Instance      Attribute              Block     Top
   -------------------------------------------------------------
   core2         rc_driver_model_mode   basic     advanced
```

See Also

- report_constraint
- non_auto_fixable_violations

---

extract_model_capacitance_limit

Defines the maximum bound on the capacitance value for output ports of the netlist.

Data Types

float

Default 64

Description

Specifies the maximum permissible capacitance (load) at an output port. The *extract_model* command characterizes circuit timing from zero to the value specified by this variable. Setting a tight capacitance bound through this variable improves the accuracy of the extracted delay tables for a range of anticipated capacitive load. If a gate in the design does not have a *max_capacitance* attribute in its library definition, the *extract_model* command uses the value of the *extract_model_capacitance_limit* variable when adding a *max_capacitance* design rule attribute to output pins of the model. If the design rule is already defined in library, the *extract_model_capacitance_limit* variable is used to establish a more constraining design rule in the resulting timing model.

Feedback

See Also

- extract_model
- extract_model_clock_transition_limit
- extract_model_data_transition_limit
- extract_model_num_capacitance_points

---

extract_model_ccs_keep_voltage_corner

set voltage corner for min, max, or nominal voltage

Data Types

string

Default min

Description

This variable is used to control how CCS ETM extraction manages on-chip-variation (OCV) min/max voltages.

When Library Compiler compiles CCS delay and noise data, it checks for full-rail DC swing responses against the supply voltage (defined by the *voltage_map* construct in the model), and it reports overshoot or undershoot responses beyond +/- 5% as violations.

This variable can be set to three values, which affect model extraction as follows:

- *min*
  - Sets *voltage_map* to the "*set_voltage -min*" value
  - Writes out only min-condition CCS noise data
- *nominal*
  - Sets *voltage_map* to the "*set_voltage -nominal*" value
  - Writes out min-condition CCS noise information
  - Writes out max-condition CCS noise information
- *max*
  - Sets *voltage_map* to the "*set_voltage*" value
  - Writes out only max-condition CCS noise data

Feedback

The default of this variable is *min* because (1) overshoot violations are more common than undershoot violations and (2) no nominal voltage is required to be set.

When the variable is set to *nominal*, the min-condition and max-condition voltages must both be within 5% of the nominal voltage. Otherwise, model extraction stops with a MEXT-102 error.

When a nominal voltage is set with the *set_voltage -nominal* command, it is used only when writing an ETM model; it is not used for design analysis.

This variable does not affect NLDM or CCS timing data.

See Also

- extract_model

---

extract_model_clock_latency_arcs_include_all_registers

Determines whether all clock paths are included in the computation when creating clock tree latency or clock insertion delay arcs in the extracted timing models (ETM).

Data Types

string

Default true

Description

If you set this variable to *false*, when extracting the clock tree latency, or clock insertion delay arcs, the PrimeTime model-generating commands traverse only the clock tree paths to the boundary registers. If this variable is set to *true* (the default), all clock tree paths, including those to the internal registers, are traversed.

This variable is effective only if the *extract_model_with_clock_latency_arc* variable is set to *true*.

The only modeling commands affected by this variable is the *extract_model* command. All formats of the extracted models are affected.

See Also

- extract_model
- report_clock_timing
- extract_model_with_clock_latency_arcs

Feedback

extract_model_clock_transition_limit

Defines the maximum bound on the transition time (slew) for ports that transitively drive the CLK pins of registers.

Data Types

float

Default 5

Description

Defines the maximum bound (in nanoseconds) on the transition time (slew) for ports that transitively drive the CLK pins of registers. Extracted timing tables are characterized for a transition range from zero to the specified bound. Specifying a tight bound for this variable improves the accuracy of extracted timing tables for a specified transition time range.

If a gate in the design whose input pin is connected to a clock port does not have a max_transition design rule in its library definition, the *extract_model* command uses the value of the *extract_model_data_transition_limit* variable when defining a max_transition design rule for the input port. If the design rule is already defined in library, this variable is used to establish a more constraining design rule in the resulting timing model.

See Also

- extract_model
- extract_model_capacitance_limit
- extract_model_data_transition_limit
- extract_model_num_capacitance_points
- extract_model_num_clock_transition_points
- extract_model_num_data_transition_points

extract_model_create_variation_tables

Decides whether the LVF tables are extracted into the ETM.

Data Types

string

Default auto

Feedback

Description

This variable controls what types of LVF tables will be created in the ETM when POCV is enabled.

When setting this variable to *auto* (the default), extract_model will generate delay sigma tables (ocv_sigma_cell_rise/fall). If slew variation is enabled, slew sigma tables will also be created. If variale timing_enable_constraint_variation is set to true, extract_model will also generate constraint sigma tables (ocv_sigma_rise/fall_constraint).

When setting this variable to *delay_only*, extract_model will only generate LVF tables for delay arcs but not for constraint arcs regardless whether constraint variation is enabled or not.

When setting this variable to *none*, extract_model will not generate any LVF tables. All the variation effects are embedded in the mean tables.

See Also

- extract_model
- timing_enable_constraint_variation

---

extract_model_data_transition_limit

Specifies an upper bound for transition time lookup table indexes in the extracted model.

Data Types

float

Default 5 in non-SI analysis, the maximum input port transition in SI analysis

Description

This variable controls how the upper bound for transition time lookup table indexes is computed in the extracted model.

The upper bound value is the most restrictive (lowest) of the following:

- The *max_transition* requirement of the relevant input port
  - This can come from library requirements or from *set_max_transition* specifications
  - This can come from input port requirements or from input pins of directly connected leaf cells
- The *extract_model_data_transition_limit* value

Feedback

- In non-SI analysis, the default is 5ns
  - In SI analysis, the default is maximum value of the transition arriving at the relevant input port, as set by either of:
    - *set_input_transition -max*
    - *set_driving_cell -max*

In the preceding rules, "relevant input port" refers to the port whose transition time is used for the table index lookup.

Specifying a lower bound for this variable can improve the accuracy of extracted timing tables for that specified transition time range.

For best accuracy in an SI analysis, the maximum transition value for each input port should be specified using either the *set_input_transition* or *set_driving_cell* command.

See Also

- extract_model
- extract_model_capacitance_limit
- extract_model_clock_transition_limit
- extract_model_num_capacitance_points
- extract_model_num_clock_transition_points
- extract_model_num_data_transition_points

---

extract_model_db_naming_compatibility

Determines whether the library name used in the .db format extracted model (ETM) should maintain the same naming style as in previous releases.

Data Types

string

Default true

Description

ETMs generated by PrimeTime can be written in either .lib (Liberty) or .db (Synopsys database) format. Historically, the naming of the library in these two formats output can be different. The .lib format uses the string specified for the *-output* option; while the library naming of the .db format has been using the design name of the block being extracted.

Feedback

By default, this variable is set to *true*, and the preceding behavior is unchanged for backward compatibility.

If you set this variable to *false*, the .db format naming follows the .lib format. For example, both formats use the value specified for the *-output* option. This might improve scripting convenience when both the .lib and .db formats ETM models are mixed in the flow.

See Also

- extract_model

---

extract_model_enable_report_delay_calculation

Determines report delay calculation to be performed on the timing arcs contained in models.

Data Types

string

Default true

Description

If you set this variable to *false*, the models generated by PrimeTime model extraction do not allow report delay calculation to be performed on the timing arcs contained in models.

This is enforceable only for the Synopsys database (.db) format output. For Liberty (.lib) format models, you can modify the files before compiling them to enable or disable the feature of the resulting library.

See Also

- extract_model
- report_delay_calculation

---

extract_model_exclude_output_transition_constraint

Controls whether to extract min/max transition constraints for output ports.

Data Types

boolean

Default true

Feedback

Description

This variable controls whether the *extract_model* command extracts min/max transition constraints for output ports.

When set to *true* (the default), min/max transition constraints *are not* extracted for output ports.

When set to *false*, min/max transition constraints *are* extracted for output ports.

See Also

- extract_model

---

extract_model_gating_as_nochange

Controls the conversion from clock-gating setup and hold arcs into no-change arcs in the extracted model.

Data Types

string

Default false

Description

When you set this variable to *true*, the clock gating setup and hold constraints are modeled as no-change arcs on the extracted model. When this variable is set to *false* (the default), clock gating checks are represented as separate setup and hold constraints.

This variable affects models in all output formats, such as the Synopsys database (.db) and Liberty (.lib) formats.

See Also

- extract_model
- extract_model_capacitance_limit

---

extract_model_include_clock_tree_pulse_width

Specifies whether the clock tree pulse width is included in the computation for minimum pulse width extraction or only the sequential clock pulse width is included.

Data Types

string

Feedback

Default false

Description

By default, the extract_model command includes only the sequential clock pulse width (the minimum pulse width on a clock pin) when extracting the minimum pulse width arcs.

To include the clock tree pulse width (the minimum pulse width on nonclock pins, typically for clock-gating cells), set the *extract_model_include_clock_tree_pulse_width* variable to *true*.

This variable affects only the *extract_model* command. All formats of the extracted models are affected.

See Also

- extract_model
- report_min_pulse_width

---

extract_model_include_ideal_clock_network_latency

Controls the behavior of accounting user-defined network latencies for ideal clocks in the extracted timing models (ETM).

Data Types

string

Default false

Description

By default, this variable is set *false*, and the extracted models treat ideal clock paths as zero delay in creating timing arcs, you are expected to reapply the same clock network latency values when the model is used.

If you set this variable to *true*, the PrimeTime model extraction uses the user-defined network latency for ideal clock paths leading to registers. This affects the delay tables created for constraint arcs such as setup, hold from the ideal clock port to data input ports, as well as sequential rising_edge and falling_edge delay arcs from ideal clock ports to output ports. It also affects the clock insertion delay arcs created in the model if extraction of such arcs are enabled by the *extract_model_with_clock_latency_arcs* variable.

This variable affects only the *extract_model* command and applies to all formats of the extracted models.

Feedback

See Also

- extract_model
- extract_model_with_clock_latency_arcs

---

extract_model_include_mcp_in_arc_value

Controls whether multicycle path timing constraints are extracted in ETM models.

Data Types

string

Default true

Description

If you set this variable to *true*, the models generated by PrimeTime's model extraction contain block-level multicycle paths effects incorporated directly into the ETM model's delay and constraint arcs.

Multicycle paths on interface timing paths are not recommended and can cause issues issue during top-level integration.

See Also

- extract_model
- set_multicycle_path

---

extract_model_include_noise_immunity_attributes

Controls whether to include noise immunity attributes in extracted model when the data is available.

Data Types

boolean

Default false

Description

This variable controls whether to extract noise immunity attributes in the extracted timing model when the data is available. This variable is only valid and effective when the *-noise* option of the *extract_model* command is used.

When this variable is set to *true*, the *extract_model* command extracts noise immunity attributes, such as *max_noise_immunity_high*, *max_noise_immunity_low*.

Feedback

When this variable is set to *false* (the default), the *extract_model* command will not include noise immunity attributes.

See Also

- extract_model
- update_noise

---

extract_model_include_power_ground_data

Controls whether the power and ground (PG) data is extracted or merged in the models.

Data Types

string

Default true

Description

When set to *true*, the models generated by the PrimeTime model extraction or merging contains the PG data, reflecting the Unified Power Format (UPF) data or PG Verilog existing in the original design or models.

See Also

- extract_model
- merge_models

---

extract_model_include_transparent_latch_constraints

includes context independent transperent latch constraints at interface in model extraction.

Data Types

boolean

Default true

Description

By setting this variable to *true*, the models generated by PrimeTime model extraction will consider all the setup constraints of the latches up to 2 levels at the interface of the design, and take the worst case between same data/clock with save edge.

Feedback

See Also

- extract_model
- timing_enable_through_paths

---

extract_model_include_upf_data

Controls whether the UPF data is extracted or merged in the models.

Data Types

string

Default true

Description

If you set this variable to *true*, the models generated by PrimeTime model extraction or merging contain UPF data reflecting the UPF data existing in the original design or models.

See Also

- extract_model
- merge_models

---

extract_model_keep_inferred_nochange_arcs

Controls whether to keep PrimeTime inferred nochange relationships as nochange timing arcs in the extracted model.

Data Types

string

Default false

Description

There are two ways for PrimeTime to perform a nochange constraint check between a data and a clock signal at a cell. The standard mechanism come explicitly from the library, when the timing arcs are defined as nochange timing types. The second mechanism is implicit. When interpreting cells based on its library arc types, PrimeTime can detect that between a data pin and a reference clock pin, there are pair-wise setup and hold arcs defined relative to the opposite edges of the clock signal, and the arcs do not form a standard edge-triggered regular flip-flop and also do not form a level-sensitive latch, PrimeTime can infer nochange relationship for the arc pair. For example,

Feedback

a *setup_clock_rise* and *hold_clock_fall* arc pair implies a *nochange_clock_high* check, meaning the data signal should be stable during the high pulse of the reference clock signal. This can be regarded as PrimeTime overrides the library defined arc types and treats the pair as forming a nochange type constraints instead of regular simple setup and hold.

In general, the extracted timing model (ETM) gives precedence to the library-defined timing types. This means, ETM always extracts those explicitly defined nochange arcs as nochange arcs in the model. For those implicitly inferred nochange arcs, by default, ETM extracts them as regular setup and hold arcs.

If you set this variable to *true*, model extraction aligns with PrimeTime timing analysis, detect those implicit nochange relationships, and overrides the setup/hold arc types as nochange the same way as timing analysis does. In certain special path or arc configuration, this alignment provides better match between timing analysis with the original netlists and with the extracted model during model validation.

This variable affects models in all output formats, such as Synopsys database (.db) and Liberty (.lib) format.

See Also

- extract_model

---

extract_model_lib_format_with_check_pins

Determines if PrimeTime model extraction should write the internal check pins created for Synopsys .db format models explicitly in the .lib format model files.

Data Types

string

Default false

Description

If you set this variable to *true*, the .lib format model files generated by PrimeTime model extraction has the same set of internal check pins that are created in the .db format models under certain conditions so that the PrimeTime timing engine correctly interprets the timing models.

By default, the variable is set to *false*. This might mean that the check pins need to be created by the down stream tools that interpret the .lib model files.

Feedback

See Also

- extract_model
- report_delay_calculation

---

extract_model_noise_iv_index_lower_factor

Controls the scale factor of the minimum index value used to create a steady-state current table, such as the I-V curve.

Data Types

float

Default -1

Description

This variable controls the scale factor of the minimum index value used to create a steady-state current table (for example, I-V curve). This variable is a scale factor that is multiplied by VDD (the power supply voltage). For example, if VDD is 1.8, the minimum voltage used is -1.0*1.8 = -1.8.

See Also

- extract_model
- report_noise
- extract_model_clock_transition_limit
- extract_model_data_transition_limit
- extract_model_noise_iv_index_upper_factor
- extract_model_num_clock_transition_points
- extract_model_num_data_transition_points
- extract_model_num_noise_width_points

---

extract_model_noise_iv_index_upper_factor

Controls the scale factor of the minimum index value used to create a steady-state current table, such as the I-V curve.

Data Types

float

Feedback

Default 2

Description

Controls the scale factor of the maximum index value used to create a steady-state current table, such as the I-V curve. This variable is a scale factor that is multiplied by VDD (the power supply voltage). For example, if VDD is 1.8, the maximum voltage used is 2.0*1.8 = 3.6.

See Also

- extract_model
- report_noise
- extract_model_clock_transition_limit
- extract_model_data_transition_limit
- extract_model_noise_iv_index_lower_factor
- extract_model_num_clock_transition_points
- extract_model_num_data_transition_points
- extract_model_num_noise_width_points

---

extract_model_noise_width_points

Selects the exact noise width points of extracted noise immunity tables.

Data Types

string

Default "" (empty string)

Description

Selects the exact noise width points of extracted noise immunity tables. The tool uses this variable only when you use the *extract_model -noise* command, and PrimeTime SI selects the library's noise immunity table for detecting noise on the input ports.

The value of this variable is a string of spaced floating values. For example,

```
set extract_model_noise_width_points "0.1 0.2 0.5 1.0"
```

Feedback

See Also

- extract_model
- report_noise
- extract_model_clock_transition_limit
- extract_model_data_transition_limit
- extract_model_num_clock_transition_points
- extract_model_num_data_transition_points
- extract_model_num_noise_width_points

---

extract_model_num_capacitance_points

Controls the size of extracted timing tables by defining the number of capacitance (load) points in these tables.

Data Types

integer

Default 5

Description

Controls the size of extracted timing tables by defining the number of capacitance (load) points in these tables.

See Also

- extract_model
- extract_model_capacitance_limit
- extract_model_clock_transition_limit
- extract_model_data_transition_limit
- extract_model_num_clock_transition_points
- extract_model_num_data_transition_points

---

extract_model_num_clock_transition_points

Controls the size of extracted timing tables by defining the number of clock transition time (slew) points in these tables.

Feedback

Data Types

integer

Default 5

Description

Controls the size of extracted timing tables by defining the number of clock transition time (slew) points in these tables.

See Also

- extract_model
- extract_model_capacitance_limit
- extract_model_clock_transition_limit
- extract_model_data_transition_limit
- extract_model_num_data_transition_points
- extract_model_num_capacitance_points

extract_model_num_data_transition_points

Controls the size of extracted timing tables by defining the number of data transition time (slew) points in these tables.

Data Types

integer

Default 5

Description

Controls the size of extracted timing tables by defining the number of data transition time (slew) points in these tables.

See Also

- extract_model
- extract_model_capacitance_limit
- extract_model_clock_transition_limit
- extract_model_data_transition_limit

Feedback

- extract_model_num_clock_transition_points
- extract_model_num_capacitance_points

extract_model_num_noise_iv_points

Controls the size of extracted noise steady-state current tables, for example I-V curve, by defining the number of index, such as voltage, points in these tables.

Data Types

integer

Default 10

Description

Controls the size of extracted noise steady-state current tables, such as I-V curve, by defining the number of index (for example, voltage) points in these tables.

The tool uses this variable only when you use the *extract_model -noise* command, and PrimeTime SI selects the library's steady-state current tables for detecting noise on the output and inout ports.

See Also

- extract_model
- report_noise
- extract_model_clock_transition_limit
- extract_model_data_transition_limit
- extract_model_noise_iv_index_lower_factor
- extract_model_noise_iv_index_upper_factor
- extract_model_num_clock_transition_points
- extract_model_num_data_transition_points
- extract_model_num_noise_width_points

extract_model_num_noise_width_points

Controls the size of extracted noise immunity tables by defining the number of noise width points in these tables.

Feedback

Data Types

int

Default 5

Description

Controls the size of extracted noise immunity tables by defining the number of noise width points in these tables.

The tool uses this variable only when you use the *extract_model -noise* command, and PrimeTime SI selects the library's noise immunity tables for detecting noise on the input ports.

See Also

- extract_model
- report_noise
- extract_model_clock_transition_limit
- extract_model_data_transition_limit
- extract_model_noise_width_points
- extract_model_num_clock_transition_points
- extract_model_num_data_transition_points

---

extract_model_pg_pin_direction_precedence

Controls the precedence that will determine the pg_type attribute of a pg_pin in an ETM.

Data Types

string

Default none

Description

This variable controls how the extract_model command determines the values of the pg_type attribute of pg_pins in extracted timing models.

Leaving this variable set to none (the default) specifies that pg_type of supply_net be derived from the type of supply_net connected to the pg_pin. Setting the variable to ‘driver’ specifies that pg_type of the driver pg_pin of the connected supply_net is used. e.g. if the driver of the supply_net is of type internal_power, setting this var to *driver* will set the pg_type of ETM’s pg_pin to internal_power

Feedback

See Also

- extract_model
- set_port_attributes
- set_related_supply_net

extract_model_short_syntax_compatibility

Controls whether the short groups are generated in extracted timing model for those feedthrough wires.

Data Types

string

Default true

Description

When this variable is set to true, extract_model will write short syntax for the feedthrough wires. When extract_model_upf_supply_precedence is set to netlist, extract_model will not assign related_power/ground_pin or related_bias_pin for the shorted ports. The timing arcs are not affected by this variable.

This variable affects models in all output formats, such as the Synopsys database (.db) and Liberty (.lib) formats.

See Also

- extract_model
- extract_model_upf_supply_precedence

extract_model_single_pin_cap

Provides backward compatibility with the introduction of the minimum, maximum, rise, and fall specific pin capacitances to account for Miller effect.

Data Types

string

Default true

Description

This variable is needed to deal with libraries accounting for Miller Effect by having different minimum, maximum, rise, and fall capacitance values for pins of library cells. The actual

Feedback

timing arcs extracted is not affected by this variable and still accounts for the Miller effect if it is available in the library and applicable in the analysis condition. For minimum and maximum paths and rise and fall transitions, the pin capacitances used to calculate the path delay are different if they are different in the library and the analysis type is not "single" operating condition.

You can set this variable to one of these values:

- *true* - The models extracted keep only a single capacitance value for a pin. The capacitance written is the maximum of all the minimum, maximum, rise, and fall values.
- *false* - If you are using the .lib and .db file formats, different minimum, maximum, rise, and fall pin capacitance values are extracted, if available and applicable. These values are written as per the .lib and .db syntax.

This variable affects only the *extract_model* command.

See Also

- extract_model

---

extract_model_single_pin_cap_max

Provides a method to write only minimum capacitance values in a model that is in single capacitance mode.

Data Types

string

Default true

Description

This variable is needed to deal with libraries accounting for Miller Effect by having different minimum, maximum, rise, and fall capacitances for pins of library cells. The actual timing arcs extracted is not affected by this variable and still accounts for the Miller effect, if available in the library and applicable in the analysis condition. For minimum and maximum paths and rise and fall transitions, the pin capacitances used to calculate the path delay are different if they are different in the library and the analysis type is not a "single" operating condition.

The *extract_model_single_pin_cap_max* variable takes effect only if the *extract_model_single_pin_cap* variable is set to *true*. You can set *extract_model_single_pin_cap_max* to one of these values:

- *true* - Writes the maximum capacitance to the model.
- *false* - Writes the minimum capacitance to the model.

Feedback

This variable affects only the *extract_model* command.

See Also

- extract_model
- extract_model_single_pin_cap

---

extract_model_split_partial_clock_gating_arcs

Controls whether to split the incomplete clock gating check setup and hold arcs in the extracted timing model (ETM).

Data Types

string

Default false

Description

You can set this variable to one of these values:

- *false* - Merges all clock gating checks and attaches them to the same pin.
- *true* - Detects clock gating setup and hold constraints that cannot be paired together; creates a separate internal check pin to avoid difference in timing analysis with the model in PrimeTime.

This variable affects models in all output formats, such as the Synopsys database (.db) and Liberty (.lib) formats.

Standard clock gating constraint is essentially a no-change check to ensure that the active clock pulses are not clipped by the gating control signal. Therefore, setup and hold arcs need to be paired to check against opposite edges of the active clock pulse. For example, a setup on clock rise arc needs to be paired with a hold on clock fall edge to ensure no clipping of the positive clock pulse by the gating signal. However, there is no explicit syntax support in the Library to define such arc pairing. Therefore, the pairing must occur automatically by PrimeTime when library cell timing arcs are analyzed. Consequently, this pair-wise relationship between setup and hold also determines the clock edges/cycles to be used when performing the checks.

In particular, when setup check present, the hold check is performed on the edge whose opposite edge has been prechosen as the most constraining or setup analysis. In cases where a proper pairing relationship cannot be established among the arcs, PrimeTime treats the missing part as not being constrained. If the setup check is missing, hold is used as the primary constraint in choosing the most constraining edge.

Feedback

As a compact timing model, ETM retains the most constraining relationship exists between an input port and its related clock port. There are many paths and endpoints that contribute to the constraints between them. ETM must evaluate the worst-case and lump all the originally separate clock gating checks existed in pair but at different cells in the netlist into simple setup/hold arcs all between the same input and clock of the macro library cell. In doing so, the details of the original arc and path pairing relationship is lost, resulting in potentially different arc pairing when the ETM is used versus during netlist timing. Consequently, model validation can show a mismatch due to different clock edges are used for the originally mispairing check. The difference arises most commonly when:

- The original setup and hold arcs in the library cells in the netlist are not standard clock-gating checks. For example, they are defined to be the same clock edge.
- There are non-unate clock paths reaching the same cell, or a mix of inverting and noninverting clock paths reaching different gating check cells.

To retain the original arc relationships without the standard syntax support in Liberty to define arc pairing, internal pins need to be introduced to force split of different classes of pairing on to different pins. This arc separation controls the automatic pairing process when ETM is used in PrimeTime, thus reproducing the original timing behavior existed in the netlist.

See Also

- extract_model
- extract_model_capacitance_limit

---

extract_model_status_level

Controls the message displaying for progress of the model extraction process.

Data Types

string

Default none

Description

This variable controls the number of progress messages shown during the timing model extraction process.

You can set this variable to one of these values:

- *none* - Does not show progress messages.
- *low* - Shows messages at the beginning of major phases of the *extract_model* command.

Feedback

- *medium* - Shows all messages for *low* and messages at the beginning of extraction subphases that might incur significant processing time.
- *high* - Shows all messages for *medium* and percentage complete messages for long running subphases.

See Also

- extract_model

---

extract_model_suppress_three_state

Determines report delay calculation to be performed on the timing arcs contained in models.

Data Types

string

Default false

Description

When you set this variable to *true*, the generated models do not contain the *is_three_state* pin attribute.

The *extract_model* command adds the *is_three_state* pin attribute to the .lib models. The attribute is added for pins that are driven by three state logic. This allows for the extracted models to be used along with other three state drivers. The tool assumes that only one of the three state drivers is driving at a time.

See Also

- extract_model

---

extract_model_upf_supply_precedence

Controls the precedence of *set_port_attributes*, *set_related_supply_net*, netlist logic, and primary supply net in determining the *related_power_pin* and *related_ground_pin* attributes of a pin in an ETM.

Data Types

string

Default netlist

Feedback

Description

This variable controls how the *extract_model* command determines the values of the *related_power_pin* and *related_ground_pin* attributes of pins in extracted timing models.

Leaving this variable set to *netlist* (the default) specifies that only the netlist is considered. The *set_port_attributes*, *set_related_supply_net*, and primary supply net of the power domain are not used.

Setting this variable to *internal* specifies the following precedence, from highest to lowest:

1. *set_port_attributes -receiver_supply* for input and *set_port_attribute -driver_supply* for output
2. Netlist logic
3. Primary supply net of the power domain

Setting this variable to *external* specifies the following precedence, from highest to lowest:

1. *set_port_attributes -driver_supply* for input and *set_port_attributes -receiver_supply* for output
2. *set_related_supply_net*
3. Netlist logic
4. Primary supply net of the power domain

The *related_power_pin* and *related_ground_pin* attributes are not set for the following ports, regardless of the value of this variable:

- For unconnected ports
- For wire-only port-to-port feedthroughs with no leaf cells connected (represented in the ETM by the Liberty *short()* construct)

See Also

- extract_model
- set_port_attributes
- set_related_supply_net

---

extract_model_use_context_delta_delay

Disable cross-talk re-calculation on port nets in ETM.

Feedback

Data Types

boolean

Default true

Description

If set this variable to *false*, the models generated by PrimeTime model extraction will re-calculate corss-talk on each port net for every input transiont and/or output load.

If set this variable to *true*, the models generated by PrimeTime model extraction will re-use corss-talk on each port net calculated during update_timin.

See Also

- extract_model
- si_enable_analysis

---

extract_model_with_3d_arcs

Enables or disables creating models contain arcs with three-dimensional delay and transition tables.

Data Types

string

Default true

Description

When this variable is set to its default of *true*, the PrimeTime model-generating commands attempt to merge certain output-to-output delays, clock-to-output, or both arcs into three-dimensional arcs with related_output_load as the third variable in the delay table, transition table, or both tables.

When you set this variable to *false*, the model keeps all output-to-output, clock-to-output, or both constraint arcs as they are, which is the default behavior of model extraction for PrimeTime version T-2002.09 and earlier releases.

The only modeling command affected by this variable is the *extract_model* command. All formats of the extracted models are affected.

If your downstream tools do not know how to handle timing arcs with three-dimensional delay tables, set this variable to *false* before extracting the timing model.

Feedback

See Also

- extract_model

---

extract_model_with_ccs_timing

Controls whether extracted timing models (ETMs) include CCS timing and CCS noise data.

Data Types

string

Default false

Description

This variable controls whether the tool creates extracted timing models (ETMs) with CCS timing and CCS noise data.

When this variable is set to *false* (the default), the *extract_model* command creates an ETM that includes only NLDM table data.

To create an ETM that contains CCS timing and CCS noise data,

- This variable must be set to *true*.
- Advanced waveform propagation must be enabled by setting *delay_calc_waveform_analysis_mode* to *true*.
- The technology libraries used by the current design must contain CCS timing and CCS noise data.

With these settings, the resulting ETM contains both CCS timing and CCS noise modeling data, which is needed to use advanced waveform propagation analysis at the higher level of hierarchy where the model is used.

This variable affects models in all output formats, such as the Synopsys database (.db) and Liberty (.lib) formats.

See Also

- extract_model
- delay_calc_waveform_analysis_mode

Feedback

extract_model_with_clock_latency_arcs

Enables or disables creating clock tree latency, or clock insertion delay arcs in extracted timing models (ETM).

Data Types

string

Default false

Description

When you set this variable to *true*, the PrimeTime model-generating commands traverse all the clock tree paths, compute the insertion delay of the paths, and create clock latency arcs in the model. Note clock insertion delay is the path delay measured between clock source and the destination registers, constraints from such as clock-gating cells are not considered.

By default, this variable is set to *false*, and the extracted models do not have clock insertion delay arcs in them.

This variable affects only the *extract_model* command; it applies to all formats of the extracted models.

If you extract .lib format models with clock latency arcs, you must compile the model with Library Compiler version V-2003.12 or later.

The clock latency arcs compensate and balance clock tree skews at chip level. Those clock latency arcs in the models are also considered by the *report_clock_timing* PrimeTime command when reporting the clock tree latency and skews. Commands such as *report_delay_calculation*, *set_timing_derate*, *set_annotated_delay*, and *get_timing_arcs* also recognize the new insertion delay arcs.

See Also

- extract_model
- report_clock_timing
- extract_model_clock_latency_arcs_include_all_registers

extract_model_with_non_sequential_arcs

Controls whether the data checks are extracted in the models.

Data Types

string

Feedback

Default false

Description

If you set this variable to *true*, the models generated by PrimeTime model extraction contain non_sequential setup/hold arcs that represent the data checks in the original design. Only data checks that are on the interface logic will be included in the ETM.

See Also

- extract_model

---

extract_model_write_case_values_to_constraint_file

Controls whether to write logic values separately into the extracted timing model (ETM) constraint file due to the user-defined case analysis value propagation.

Data Types

string

Default false

Description

If this variable is set to its default of *false*, all logic constant values reaching block I/O ports are written into the .lib and .db files as a function attribute for the pin. This occurs regardless of whether the logic value is due to propagate circuit intrinsic functional constants or user-defined case analysis values. This is the behavior of the ETM.

If you set this variable to *true* and model extraction detects any logic constant set or propagated to the I/O boundary of the block as a result of user-defined analysis settings, the logic value is written into the ETM constraint file with the proper *set_case_analysis* command.

This variable affects models in all output formats, such as Synopsys database (.db) and Liberty (.lib) formats.

An ETM generated by PrimeTime, captures I/O timing behavior for the purpose of static timing analysis. By design, the primary flow intention is to improve the capacity and performance of downstream consumer tools in their timing driven implementation, optimization, and analysis steps. By design, the ETM does not retain any functional information about the original netlist and is essentially a functional black-box. Lacking functional definition, an ETM by itself might not be well suited for design steps where the cell function is of primary concern. However, because logic values and their propagation impact timing of both the block and higher level netlist where the model becomes instances, ETMs have to keep the logic values from the block that propagated to the output ports, so that their effects to timing analysis can be carried consistently to higher

Feedback

level. These logic values can come from inherent netlist logic constants or from user-defined case analysis values. PrimeTime represents the logic values of the macro block with the simple function attribute on pins. This ensures consistency from a timing analysis perspective. For certain design flows where some tools consuming ETMs in certain steps need to be able to differentiate logic values resulted from functional constant versus case analysis propagation. Optionally, it is possible to write logic values resulting from case analysis to a separate constraint file, and write only the functional constant with function attribute.

It is worth noting that the constraint file written along with the .lib, .db model, or both files should be used together with the ETM to completely reproduce the timing behavior of the original netlist with the model.

See Also

- extract_model
- set_case_analysis

---

extract_model_write_verilog_format_wrapper

Writes Verilog format wrapper for wrapper plus core style ETM.

Data Types

Boolean

Default false

Description

You can set this variable to one of these values:

- *false* - The *extract_model* command creates a database format wrapper for wrapper plus core style extracted timing model (ETM).
- *true* - The *extract_model* command creates Verilog format wrapper is created instead.

See Also

- extract_model

Feedback

f

filter_collection_extended_syntax

Data Types

boolean

Default true

Description

This variable is obsolete and setting it has no effect. Extended features are always enabled.

See Also

- filter_collection

g

gca_setup_file

Specifies the name of the file that is read by the In-Design PrimeTime GCA flow before constraint analysis.

Data Types

string

Default "" (empty)

Description

This variable specifies the name of a Tcl setup file to be read by the In-Design PrimeTime GCA flow before constraint analysis. You can specify user-defined constraint checking rules and violation waivers for PrimeTime GCA in this setup file. PrimeTime GCA sources this file after loading the design and constraints.

Note:

This setup file is read by PrimeTime GCA at a different time than the .synopsys_gca.setup initialization file, which is loaded during the startup of PrimeTime GCA.

By default, this variable is set to an empty string, and no setup file is read after design loading during the In-Design PrimeTime GCA flow.

Feedback

See Also

- check_constraints

---

global_timing_derate_violations

This man page describes *global_timing_derate* hierarchical boundary check violations shown by the *report_constraint* command in the HyperScale flow.

Description

A *global_timing_derate* violation indicates differences in the timing derating applied between block- and top-level analysis.

During block-level analysis, the global timing derating values are captured and saved, this includes both design-level timing derating values and library cell derating values, but does not include instance specific derating settings. These saved values are used to check against top-level analysis settings applicable to the block instances.

Exact matching is not required. If block-level derating results in more conservative timing analysis, the tools reports no violations. Instead, the tools reports a violation if top-level analysis requires more margin than block-level analysis.

What Next

This is a *non_auto_fixable* violation. To resolve the differences in timing deratings, either apply the same derating as top-level for block-level analysis, or use more conservative settings at the block level than at the top level.

Also note that violations reported for library cell derating values can also arise from the libraries being different. For more information about library mismatches between block- and top-level analysis, use the *library_mapping* command. For those cases, aligning the libraries might automatically resolve these library cell-related timing derating mismatches.

Examples

The following verbose report shows global design timing deratings and library-cell specific deratings (FD2 in the lsi_10k library). Deratings applied at block-level analysis show smaller margins than those applied at top-level analysis.

```
pt_shell> report_constraint -boundary_check_include
 {global_timing_derate} \\
          -all_violators -verbose

****************************************
Report : constraint
        -all_violators
        -verbose
        -boundary_check
Design : top
```

Feedback

```
 ...
****************************************

HyperScale constraints report

   Constraint: global_timing_derate

   Instance      Derate type            Window type    Block    Top
 Slack

 ------------------------------------------------------------------------
---
   md            static_data_net        min_rise       0.9      0.8
 -0.1
   md            static_data_net        min_fall       0.9      0.8
 -0.1
   md            static_clock_net       min_rise       0.9      0.8
 -0.1
   md            static_clock_net       min_fall       0.9      0.8
 -0.1
   md            dynamic_data_net       max_rise       1        1.1
 -0.1
   md            dynamic_data_net       max_fall       1        1.1
 -0.1
   md            dynamic_data_net       min_rise       0.95     0.9
 -0.05
   md            dynamic_data_net       min_fall       0.95     0.9
 -0.05
   md            dynamic_clock_net      max_rise       1        1.1
 -0.1
   md            dynamic_clock_net      max_fall       1        1.1
 -0.1
   md            dynamic_clock_net      min_rise       0.95     0.9
 -0.05
   md            dynamic_clock_net      min_fall       0.95     0.9
 -0.05
   md            cell_check             max_rise       1.34     1.38
 -0.04
   md            cell_check             max_fall       1.34     1.38
 -0.04
   lsi_10k:FD2   library_cell_data      min_rise       0.81     0.79
 -0.02
   lsi_10k:FD2   library_cell_data      min_fall       0.81     0.79
 -0.02
   lsi_10k:FD2   library_cell_clock     min_rise       0.81     0.79
 -0.02
   lsi_10k:FD2   library_cell_clock     min_fall       0.81     0.79
 -0.02
   lsi_10k:FD2   library_cell_check     max_rise       1.44     1.46
 -0.02
   lsi_10k:FD2   library_cell_check     max_fall       1.44     1.46
 -0.02
```

Feedback

See Also

- report_constraint
- report_timing_derate
- set_timing_derate
- non_auto_fixable_violations

---

golden_upf_report_missing_objects

Specifies whether to report missing object errors in the golden UPF flow.

Data Types

Boolean

Default false

Description

All UPF tools are expected to suppress missing object errors during golden UPF reapplication with a few exceptions. If you set this variable to *true*, PrimeTime displays information messages for missing objects.

See Also

- load_upf
- enable_golden_upf

---

gui_build_query_data_table

Controls whether the tool initializes data for fast data query after the GUI starts.

Data Types

Boolean

Default The default value for *gui_build_query_data_table* is true.

Description

This variable controls whether IC Compiler initializes query data during GUI startup. By default, the *gui_build_query_data_table* variable is true and IC Compiler initializes query data, which might require some time for a large design. You can set the variable to false to prevent the tool from initializing query data.

Feedback

Examples

The following example prevents the tool from initializing query data.

```
prompt> set gui_build_query_data_table false
```

See Also

- printvar

---

gui_custom_setup_files

Variable for specifying GUI customization files to be loaded during GUI startup.

Data Types

Tcl list of fully-qualified file names

Default $synopsys/admin/setup/.synopsys_<app>_gui.tcl

Description

This variable specifies a set of files that should be sourced when the GUI starts up. You can add this variable setting to the application setup file to provide GUI customizations to individuals, or share customizations among a group of users. A GUI customization file typically contains commands to specify hotkeys, menus, or toolbars which implement specific functions to support a custom environment or flow.

The GUI initializes and searches the list of files in order. Each file in the list is sourced. Finally, your .synopsys_<app>_gui.tcl file is loaded to complete the customizations.

You can disable the loading of the customizations by setting the *gui_disable_custom_setup* variable.

See Also

- printvar
- gui_disable_custom_setup

---

gui_default_window_type

,IP *gui_default_window_type* Read-only variable specifying the default window type for GUI customization commands.

Data Types

Boolean

Feedback

Default false

Description

This read-only variable contains the name of the window type that is used as the default when a menu, hotkey, or toolbar customization is specified without specifying a window type explicitly.

See Also

- gui_create_menu
- gui_create_toolbar
- gui_create_toolbar_item
- gui_delete_menu
- gui_delete_toolbar
- gui_delete_toolbar_item
- gui_report_hotkeys
- gui_set_hotkey
- printvar

---

gui_disable_custom_setup

Variable for disabling gui customizations

Data Types

Boolean

Default false

Description

This variable specifies whether GUI customization loading is disabled during GUI startup. Set the variable to true to prevent GUI customizations from loading. This includes customizations specified in the *gui_custom_setup_files* variable as well as your .synopsys_<app>_gui.tcl file.

See Also

- printvar
- gui_custom_setup_files

Feedback

gui_online_browser

Specifies the name of the browser used to invoke the online help system from the help menu of the product.

Data Types

string

Default netscape

Group

gui

Description

This string variable holds the value of the default browser which is used to invoke online help system from Help menu of the application. The value the variable should be either *netscape*, *mozilla* or *firefox*.

If you specify a browser other than those listed above, the application defaults to the netscape browser.

Use the following command to determine the current value of the variable:

```
prompt> printvar gui_online_browser
```

See Also

- gui_custom_setup_files

gui_selection_stack_depth

Specifies the maximum depth of the selection stack.

Data Types

Integer

Default 10

Group

gui

Description

When the user pushes a selection onto the selection stack the max depth is checked and any extra items, beyond the stack depth, are discarded.

Feedback

This check it too enforce a limit of memory and resources needed to maintain the selection stack where each selection can contain a large number of object references.

If the value is set to a value less than 1 then the maximum depth is ignored so items are never discarded.

```
prompt> printvar gui_selection_stack_depth
```

See Also

- gui_selection_stack

---

gui_suppress_auto_layout

Variable for disabling automatic opening of the Layout window.

Data Types

Boolean

Default false

Description

Specifies whether the Layout window is prevented from opening when loading a new design.

See Also

- printvar

---

h

---

hier_block_interface_arc_trace_mode

Specifies the types of combinational arcs to trace and preserve in the block interface during HyperScale model generation.

Data Types

string

Default enabled

Feedback

Description

This variable specifies the types of combinational arcs to trace and preserve in the block interface during HyperScale model generation:

- *timing* - Allows tracing of valid timing arcs only; prevents tracing of either disabled timing arcs or case analysis set by the *set_case_analysis* command.
- *enabled* - Allows the tracing of all enabled timing arcs. Case analysis settings are ignored. This is the default.
- *all* - Allows the tracing of all combinational arcs, even if disabled timing arcs or arcs with case value set by *set_case_analysis* command.

Usually the *timing* setting results in more compact model and potentially better runtime and memory for top-level analysis, so it is recommended when block-level constraints are of high quality for the block interface.

For a more flexible model, which can work with potentially different disabled arcs and *set_case_analysis* settings on the block interface path from the top-level constraints during top-level analysis, use the *all* option.

The default, *enabled*, traces through pins with case values and therefore can work for potentially different *set_case_analysis* settings at the top level, but stops at disabled arcs because the model already has disabled timing arcs built-in when performing top-level analysis.

See Also

- set_case_analysis
- set_disable_timing
- write_hier_data

---

hier_block_interface_model_latch_level

Specifies the depth of latch while doing Hyperscale extraction.

Data Types

int

Default 2

Description

Specifies the maximum number of levels of level sensitive latches considered as block interface model for top level analysis. For a transparent latch based design, this helps reduce the block size so top level Hyperscale model based timing analysis can see

Feedback

faster runtime and smaller memory. During Hyperscale model generation, the tool traces through transparent latches and stops at a register(regardless of a transparent latch, a non transparent latch or a flip flop), up to the number of levels specified by this option. For example, if you specify the number as 3, the tool will traces through the first 3 transparent latches and mark them as interface of the design and will stop at the 4th register(regardless of a transparent latch, a non transparent latch or a flip flop). This variable controls both the input interface from the input data ports, as well as the output interface launched from latches to output ports. This variable has a default value as 2. The maximum value of this variable depends on the the value set by *timing_through_path_max_segments*. If the value is larger than the maximum value define by *timing_through_path_max_segments*, Hyperscale will set it as the same value as defined by *timing_through_path_max_segments*. If the variable is set as any number smaller than 0, Hyperscale will use 0 instead.

---

hier_characterize_context_mode

Sets the context characterization mode to perform either full context characterization or only extraction of constraints without timing analysis.

Data Types

string

Default full_context

Description

This variable specifies the mode of characterization driven by the *characterize_context* and *write_context* commands. The variable setting also affects the behavior of other commands such as *update_timing* and *read_parasitics*.

The variable can be set to either *full_context* or *constraints_only*.

full_context (the default)

The *full_context* mode is the full-accuracy context characterization mode. In this mode, there are no impacts to regular PrimeTime data reading and timing analysis. Use the *characterize_context* command to select the block instances for which the tool performs additional steps to capture or preserve the full timing context computed during timing analysis. You can write out the context data with the *write_context* in several formats.

constraints_only

This *contraints_only* setting selects the constraint extraction mode in which the tool perform minimal analysis during *update_timing* and automatically skips unnecessary data loading steps (parasitics, AOCV/POCV tables, annotated delays, and so on) and disables unnecessary features (SI, POCV, CRPR, and so on). Use the *characterize_context* command to specify the block instances from which to extract a set of full block-level

Feedback

constraints. Then use the *write_context* command to write out the constraints in script format, either constraints.pt and variables.pt or constraints.sdc and variables.sdc.

The *constraints_only* mode limits the *update_timing* command to only the steps needed to extract the block constraints, such as the propagation of clocks, constants, and timing exceptions. This mode is recommended only for writing out block constraints, and not for any detailed timing analysis or optimization.

This mode is incompatible with HyperScale analysis and cannot be used in any session in which HyperScale analysis is performed.

Use a separate PrimeTime session for *constraints_only* constraint extraction, as in the following script:

```
set_app_var hier_characterize_context_mode constraints_only
read_verilog Counter.v
read_verilog Top.v
link_design
 ...
characterize_context -block Counter
 ...
update_timing   # Constraint extraction only,
 ...            # no timing information generated
write_context -format ptsh Counter -output $cnsDir
```

Note: This variable affects the behavior of the *link_design* command, so you need to set it before linking.

See Also

- characterize_context
- link_design
- write_context

---

hier_clock_mapping_period_percent_tolerance

Specifies a tolerance for automatic clock mapping that allows clocks with different frequencies or duty cycles to be mapped as equivalent in HyperScale hierarchical analysis.

Data Types

float

Default 0

Feedback

Description

By default, HyperScale hierarchical analysis automatically maps top-level and block-level clocks as equivalent when the clock characteristics match exactly. To allow automatic mapping between clocks that have slightly different periods or duty cycles, set this variable to the desired tolerance.

Specify the tolerance as a decimal fraction of the clock period and duty cycle. For example, to allow a difference of 2 percent, enter 0.02 as the tolerance value. In that case, the smaller period must be within 2 percent of the larger period, and the smaller duty cycle must be within 2 percent of the larger duty cycle, for the clocks to be considered equivalent for mapping purposes.

If all other conditions for mapping clocks are met, and the period and duty cycle are within the threshold difference, the clocks are automatically mapped.

See Also

- report_clock
- set_clock_map
- set_hier_config
- hier_disable_auto_clock_mapping
- timing_save_hier_context_data

---

hier_clock_mapping_uncertainty_percent_tolerance

Specifies a tolerance for automatic clock mapping that allows clocks with different uncertainties to be mapped as equivalent in HyperScale hierarchical analysis.

Data Types

float

Default 0

Description

By default, HyperScale hierarchical analysis automatically maps top-level and block-level clocks as equivalent when the clock characteristics match exactly. To allow automatic mapping between clocks that have slightly different uncertainties, set this variable to the desired tolerance.

Specify the tolerance as a decimal fraction of the clock uncertainty. For example, to allow a difference of 2 percent, enter 0.02 as the tolerance value. In that case, the

Feedback

smaller uncertainty must be within 2 percent of the larger uncertainty, for the clocks to be considered equivalent for mapping purposes.

If all other conditions for mapping clocks are met, and the uncertainty is within the threshold difference, the clocks are automatically mapped.

See Also

- report_clock
- set_clock_map
- set_hier_config
- hier_disable_auto_clock_mapping
- timing_save_hier_context_data

hier_constraint_output_compressed

Specifies whether to compress the HyperScale ASCII boundary I/O context output.

Data Types

Boolean

Default true

Description

By default, the HyperScale ASCII boundary I/O output is compressed in the gzip format.

To manually uncompress the gzip output file, run the following command:

```
% gunzip -S ".pt" constraints.pt
```

To disable the compression of the output, set the *hier_constraint_output_compressed* variable to *false*.

See Also

- write_hier_data

hier_constraint_write_context

Specifies whether the HyperScale top-level flow writes the ASCII boundary I/O context.

Data Types

Boolean

Feedback

Default false

Description

This variable controls whether the tool writes ASCII I/O context in the HyperScale top-level flow. By default, the tool writes out binary context. If you set this variable to *true*, the HyperScale top-level flow also writes out the ASCII I/O context. This variable must be set before update_timing. Otherwise a full update_timing must be executed after it is set to *true*.

See Also

- save_session
- set_hier_config
- hier_enable_analysis
- hier_constraint_output_compressed

---

hier_constraints_include_cross_boundary_exceptions

Specifies whether the ASCII format constranits written by write_context command to include cross boundary exceptions.

Data Types

Boolean

Default false

Description

This variable controls whether the tool writes the cross boundary exceptions in the ASCII format (ptsh/sdc) constraints written by write_context command. By default, the tool writes out the cross boundary exceptions only in gbc format. By setting the variable to *true*, the constraints written in ptsh/sdc formats will also include cross boundary exceptions. The variable works for both characterize_context "full_context" and "constraints_only" mode and the cross boundary exceptions written into the constraints are expressed with additional virtual clocks to refer to the exception objects that are outside of the block boundary.

See Also

- write_context
- characterize_context
- hier_constraint_output_compressed

Feedback

hier_context_exclude_sub_block

Enables the Excluded flow for constraint extraction of HyperScale hierarchical analysis.

Data Types

Boolean

Default false

Description

Set this variable to *true* to enable the multi-level excluded flow for the ASCII format (ptsh/sdc) constraints written by write_context command. By default, the tool writes out whole block context information for every characterized instances. In excluded flow, the constraints from inside characterized sub-instances is excluded. In order to analyze the block level, the reverse delay information for hierarchical pins of excluded instances is also written out.

Examples

The following script excludes M1/B1 from the context description of M1. Because M1/B1 is the sub-instance of M1 and M1/B1 is characterized as well. In excluded flow, the *write_context* doesn't write out the constraints of M1/B1.

```
set hier_context_exclude_sub_block true
   ...
  characterize_context -block MID -instances {M1} -name MID
  characterize_context -block BLK -instances {M1/B1} -name BLK
  update_timing -full
  write_context {M1} -format ptsh MID_dir
```

And the tool writes out the boundary input and output timing information for excluded M1/B1. The output delay is annotated to the hierarchical input pins, and the input delay is annotated to the hierarchical output pins. For example, the context description of previous script:

```
set_input_delay 0.5 [get_pins {B1/out}] -clock [get_clocks {CLK}]
  set_output_delay 0.3 [get_pins {B1/in}] -clock [get_clocks {CLK}]
```

See Also

- hier_enable_analysis
- hier_characterize_context_mode
- characterize_context

Feedback

hier_context_merge_clock_latency_mode

Specifies the types of total clock latency merge to keep total clock latency for MIM merging during block-level analysis.

Data Types

string

Default min_max_value

Description

This variable specifies the types of total clock latency merge to keep total clock latency for MIM merging during-block level analysis:

- *min_max_value* - Use the min of min, max of max value as total latency.
- *largest_window* - Use the maximum difference of max - min value as total latency.

The total latency merging with min of min, max of max using *min_max_value* gets the large setup CRPR at the block level. Most of time, the large CRPR can just compensate the merged enlarged pessimistic total latency window without optimism. When *largest_window* mode is set, and the noted clock is asynchronous to other clocks of the block, MIM context merging is using the largest window which has the miximum difference of max - min value to minimize clock window and pessimism in SI analysis.

See Also

- report_timing
- report_crpr
- hier_context_merge_static_clock_latency_mode

hier_context_merge_static_clock_latency_mode

Specifies the types of static latency merge to keep static latency for MIM merging during block-level analysis.

Data Types

string

Default automatic

Feedback

Description

This variable specifies the types of static latency merge to keep static latency for MIM merging during-block level analysis:

- *automatic* - Guarantee no optimism when there is large CRPR using the *min_max_value* method.
- *min_max_value* - Use the min of min, max of max value as static latency.
- *smallest_window* - Use the minimum difference of max - min value as static latency.
- *match_total_latency* - After getting the total min/max latencies, fetch the corresponding static latency of that path.

The static latency merging with min of min, max of max using *min_max_value* gets the large setup CRPR at the block level. Most of time, the large CRPR can just compensate the merged enlarged pessimistic total latency window without optimism. This is where delta static latency merging is needed to guarantee no optimism using *automatic* as the default value.

For some cases, there is no physical meaning for the *automatic* method. That is why *match_total_latency* is needed. When the min/max total latency of one path is fetched, the corresponding static latency of that path is also fetched, avoiding the meaningless static latency after MIM merging.

See Also

- report_timing
- report_crpr

---

hier_context_merge_strict_clock_equivalence

Controls HyperScale context merging behavior for MIM instances that are not fully mergeable.

Data Types

Boolean

Default true

Description

By default, HyperScale top-level analysis enforces exact matching of clocks across all multiply instantiated modules (MIMs) being merged for hierarchical analysis and constraint extraction. If there is any mismatch for a clock on a certain instance, that instance is completely excluded from context merging for all types of context data.

Feedback

The *hier_context_merge_strict_clock_equivalence* variable lets you modify this behavior. It is set to *true* by default. Set the variable to *false* to relax the strict requirements and allow a module with mismatching context to be merged in certain cases.

The PrimeTime tool automatically performs context data merging during HyperScale analysis when you specify more than one instance of a given block with the same configuration. By default, constraint and context data merging can be performed only when the clocks are exactly matched across the instances.

To decide whether two instances are mergeable, all boundary and internal real clocks are checked and matched between the instances. Two instances are considered unmergeable if no match can be found between one or more clocks. This checking is done across all MIM instances, and the mergeable instances are clustered as a group. By default, if no MIM reference is specified (the *-mim_reference* option is not used in the *set_hier_config* command), the cluster with the largest number of instances is used for context merging, and the instances not in this cluster are excluded from merging.

When you set this variable to *false*, instead of completely dropping the remaining instances outside of the cluster unconditionally, the tool considers merging them. If the context difference is not clock-related, the block is merged. If the context difference is clock-related, the block is merged if a matching clock can be found from the reference instance set; or if no matching clock can be found, the block is still considered unmergeable and the relevant context from the instance is dropped.

See Also

- save_session
- set_hier_config
- update_timing
- hier_merge_match_clock_with_constant

---

hier_create_template_scripts

Enables flat to hyperscale scripts generation.

Data Types

Boolean

Default false

Description

When you set this variable to *true*, flat to hyperscale scripts generation flow is enabled within PrimeTime. In this flow, PrimeTime reads in the flat script and generates hyperscale

Feedback

run scripts. Besides this, this flow also performs constraint extraction so that user can directly run the hyperscale analysis flow without doing constraint extraction. Note users need to call *write_hier_data* to trigger the flow. Hyperscale scripts and constraints will be generated in the directory specified by *write_hier_data*.

Enabling *hier_create_template_scripts* affects the *link_design*, *update_timing* and *save_session* commands.

Note: Because the *link_design* command is controlled by this variable, you must set the *hier_create_template_scripts* variable before linking. After the design is linked, further changes to this variable results in error, and it has no effect.

See Also

- link_design
- save_session
- set_hier_config
- update_timing
- write_hier_data

hier_data_version

Indicates the PrimeTime HyperScale database version that the application currently runs.

Data Types

string

Description

This read-only variable is set to the HyperScale database version of the application currently running.

See Also

- save_session

hier_disable_auto_clock_mapping

Disables automatic clock mapping in the HyperScale hierarchical analysis flow.

Data Types

Boolean

Feedback

Default false

Description

By default, HyperScale hierarchical analysis performs automatic mapping of clocks between the top level and block level by looking for matching clock characteristics.

To disable automatic clock mapping, set the *hier_disable_auto_clock_mapping* variable to *true*. This is useful when you want to perform interactive or fully manual clock mapping. With this setting, you need to map clocks by using the *set_clock_map* command. If you have a HyperScale constraint extractor run, you can do this by sourcing the clock_map.pt file. Unspecified clock mappings remain unmapped.

See Also

- report_clock
- set_clock_map
- set_hier_config
- hier_clock_mapping_period_percent_tolerance
- timing_save_hier_context_data

---

hier_disable_auto_multi_clock_mapping

Disables automatic top N to block M clock mapping in the HyperScale hierarchical analysis block level flow.

Data Types

Boolean

Default false

Description

By default, HyperScale hierarchical analysis performs automatic mapping of clocks between the top level and block level by looking for matching clock characteristics when top level has more than one clock entering the same port or block level has more than one clock defined at the same port.

To disable automatic multiple clock mapping at block level, set the *hier_disable_auto_multi_clock_mapping* variable to *true*. This is useful when you want to perform interactive or fully manual clock mapping. With this setting, you need to map clocks by using the *set_clock_map* command. If you have a HyperScale constraint extractor run, you can do this by sourcing the clock_map.pt file. Unspecified clock mappings remain unmapped.

Feedback

See Also

- report_clock
- set_clock_map
- set_hier_config
- hier_clock_mapping_period_percent_tolerance
- hier_disable_auto_clock_mapping
- timing_save_hier_context_data

---

hier_distributed_working_directory

Specifies the working directory for HyperScale distributed analysis. The directory must be accessible to the distributed farm hosts for read and write operations during timing analysis.

Data Types

string

Default ""

Description

You need to set this variable to perform distributed HyperScale analysis or use the flat reporting flow. The specified directory is used to store the intermediate runtime data as well as logs and reports.

The specified directory must be network accessible for all the distributed farm hosts launched with the *set_host_options* command. The *hier_distributed_working_directory* variable must be set before you use the *start_hosts* command.

See Also

- save_session
- set_hier_config
- set_host_options
- start_hosts
- update_timing
- hier_enable_analysis

Feedback

hier_duplicate_internal_aggressor_timing_windows

Specifies whether top-level HyperScale analysis duplicates the SI arrival windows for extra clocks propagated into the block that were not defined during block-level analysis when the internal aggressors are captured in the model data.

Data Types

Boolean

Default false

Description

In the HyperScale analysis flow, an internal aggressor is a crosstalk aggressor net in a block-level register-to-register timing path that acts on a victim net in the block-to-top interface. The block abstraction process removes all register-to-register paths but retains internal aggressors to model their SI effects on the interface. The internal aggressor model includes the driver, load, and transition arrival windows of the aggressor net.

When this variable is set to *false* (the default), top-level analysis considers the clocks that are defined and properly mapped to block-level clocks when using the models captured for internal aggressors for SI alignment analysis.

When this variable is set to *true*, in addition to the above for mapped clocks, top-level analysis also duplicates the SI windows of internal aggressors for the extra clocks that exist at the top level but were not defined for the block level during HyperScale model creation. It calculates the windows for the extra clocks by considering all the windows of the block clocks that existed during model extraction and have been mapped to the top clock.

See Also

- set_hier_config
- write_hier_data
- hier_enable_analysis

hier_enable_analysis

Enables the HyperScale hierarchical analysis flow.

Data Types

Boolean

Default false

Feedback

Description

Set this variable to *true* to enable the HyperScale hierarchical analysis flow in the PrimeTime tool.

In the HyperScale flow, the tool analyzes the block-level and top-level portions of the design using separate runs and accurately handles the timing interfaces across hierarchical boundaries, significantly improving the capacity and performance of hierarchical timing analysis without impacting the quality of results.

Enabling HyperScale analysis affects the behavior of the *link_design*, *update_timing*, *save_session*, and *report_constraints* commands.

Because the behavior of the *link_design* command depends on whether the HyperScale flow is enabled, you must set the *hier_enable_analysis* variable before you link the design.

When HyperScale analysis is enabled, use the *set_hier_config* command to specify the HyperScale analysis parameters.

See Also

- link_design
- report_constraint
- save_session
- set_dont_override
- set_hier_config
- update_timing

---

hier_enable_ascii_context_writing_output_folder

Specifies whether ASCII output from *write_context* should be written as a directory or as a single file.

Data Types

boolean

Default true

Description

This variable controls how the ASCII formats of the *write_context* command (*ptsh*, *dctcl*, *sdc*) are written.

Feedback

When set to *true* (the default), the context information is written as a directory, named per the *-output* option, containing context files:

```
pt_shell> write_context wishbone -output context_dir
pt_shell> sh find context_dir -type f
context_dir/CONSTRAINT/eth_wishbone/wishbone/constraints.pt
context_dir/CONSTRAINT/clock_map.pt
```

When set to *false*, the context information is written to a single file, named per the *-output* option:

```
pt_shell> write_context wishbone -output context_file.pt
pt_shell> sh ls -l context_file.pt
-rw-r--r-- 1 user group 1218401 Apr  6 13:13 context_file.pt
```

This variable can be used in a HyperScale top-level analysis, and also in non-HyperScale runs where the *characterize_context* command is used.

See Also

- write_context
- characterize_context

---

hier_enable_detailed_side_input_path_timing

Specifies if PrimeTime HyperScale analysis should capture the full details of paths starting from side inputs (internal startpoint).

Data Types

Boolean

Default false

Description

Side input path is a HyperScale specific behavior. In the HyperScale analysis flow, there are block-level and top-level analyses. Side input paths at block level are internal register to register paths, and they converge with interface paths by sharing some common logic through some multiple-input cells, For example, an AND gate whose A pin is on an interface path starting from a block port and leading to some register, while its B pin is on a path starting from a register cell. During HyperScale block data reduction, paths through the A pin are retained while the fanin of B pin is removed. This pin is the side input pin of the interface path. To have the same accuracy of flat analysis at the HyperScale top level for the paths leading to the common register of the AND gate, the tool captures and annotates all the timing data propagated at B pin in binary format. At the top level, the paths appear to be starting from these dangling side input pins.

Feedback

When this variable is set to *false* (the default), the paths starting from these side inputs maintain their effects to the interface paths, such as worst slew and SI crosstalk windows, but themselves are false paths at top level.

When this variable is set to *true*, detailed arrival times are captured in the model to reproduce the slacks of block internal timing paths end at interface registers. This detailed model has runtime and memory impact to both block and top level analysis. When this variable is set to *true*, please also refer to variable *timing_separate_hier_side_inputs* for further controls on reporting.

Examples

The following example shows the path reported as HyperScale side input group when both variables are set to *true*.

```
report_timing -from $side_input
****************************************
Report : timing
        -path_type full
        -delay_type max
        -max_paths 1
 ...
****************************************

  Startpoint: blk/u3/B (internal path startpoint clocked by top_CLK)
  Endpoint: blk/ff1 (rising edge-triggered flip-flop clocked by top_CLK)
  Path Group: **HyperScale_side_input**
  Path Type: max

  Point                                    Incr       Path
  -----------------------------------------------------------
  clock top_CLK (rise edge)                0.00       0.00
  clock network delay (propagated)         0.02       0.02
  input external delay                     4.68       4.70 f
  blk/u3/B (ND2)                           0.00       4.70 f
  blk/u3/Z (ND2)                           0.64 &     5.34 r
  blk/ff1/D (FD2)                          0.00 &     5.34 r
  data arrival time                                   5.34

  clock top_CLK (rise edge)               10.00      10.00
  clock network delay (propagated)         1.16      11.16
  blk/ff1/CP (FD2)                                   11.16 r
  library setup time                      -0.85      10.31
  data required time                                 10.31
  -----------------------------------------------------------
  data required time                                 10.31
  data arrival time                                  -5.34
  -----------------------------------------------------------
  slack (MET)                                         4.97
1

report_path_group
```

Feedback

```
****************************************
Report : path_group
 ...
****************************************

Path_Group     Weight    From      Through        To
--------------------------------------------------------------------------------
-------
**HyperScale_side_input**
                  1.00   { blk/u3/B ... }
                                   *              *
**async_default**
                  1.00   -         -              -
**clock_gating_default**
                  1.00   -         -              -
**default**       1.00   -         -              -
top_CLK           1.00   *         *              top_CLK
top_CLK2          1.00   *         *              top_CLK2
```

See Also

- hier_enable_analysis
- hier_enable_detailed_stub_path_timing
- report_timing

---

hier_enable_detailed_stub_path_timing

Specifies whether to capture full details of required timing for paths ending at HyperScale stub pins (internal endpoints).

Data Types

Boolean

Default false

Description

In the HyperScale flow, there are block-level and top-level analyses. Stub pin paths exist at the top-level analysis and are internal endpoints within a HyperScale block. Stub pins exist because block-internal register-to-register paths are not fully retained at the top level. A stub pin is created when a starting register has paths leading to block output ports -- that is, starting from an interface register -- while the starting register also has timing paths leading to other registers inside the block, which makes the path shared with block-internal paths. The points where a path branches to interface paths and internal paths are designated as stub pins.

Feedback

When a pin becomes a stub, its entire transitive fanout, including all the logic on the paths from the stub pin to the capture registers in the block, plus the clock paths of these internal capture register, are not retained in the block abstraction and therefore invisible at the top-level analysis. A stub pin is often one of the load pins of a multiload net. Topologically speaking, a stub pin is symmetric to a side input, which happens on a block input interface due to multifanin cells.

When this variable is set to *false* (the default), more compact timing data is captured for the stub pin for the best runtime and memory of both block-level and top-level timing analysis.

When this variable is set to *true*, PrimeTime performs detailed backward required time propagation calculation in block-level analysis and captures all the details of required time at the stub pin. Then these required times are annotated on the stub pins at the top level.

The reporting of the paths that ends at stub pins (half of the block-internal register-to-register paths) is controlled by another variable, *timing_report_hier_stub_pin_paths*.

Examples

The following example shows the path reported in the **HyperScale_stub_default** intrinsic path group when both variables are set to *true*.

```
prompt> report_timing -path_type full_clock_expanded -from block/ffa2/Q
 ...
  Startpoint: block/ffa2 (rising edge-triggered flip-flop clocked by
 SYSCLK)
  Endpoint: block/ffa4/D
               (internal path endpoint clocked by SYSCLK)
  Path Group: **HyperScale_stub_default**
  Path Type: max

  Point                                        Incr       Path
  -----------------------------------------------------------------
  clock SYSCLK (rise edge)                    0.000      0.000
  clock source latency                        0.700      0.700
  CLK (in)                                    0.000      0.700 r
  block/clk (BLOCK)                           0.000      0.700 r
  block/cbufa1/Y (CLKBUFX2)                   0.385      1.085 r
  block/ffa2/CK (DFFX2)                       0.000      1.085 r
  block/ffa2/Q (DFFX2) <-                     0.460      1.545 f
  block/ffa4/D (DFFX2)                        0.000      1.545 f
  data arrival time                                      1.545

  clock SYSCLK (rise edge)                    4.000      4.000
  clock source latency                        0.400      4.400
  CLK (in)                                    0.000      4.400 r
  block/clk (BLOCK)                           0.000      4.400 r
  block/cbufa1/A (CLKBUFX2)                   0.000      4.400 r
  output external delay                      -0.004      4.396
  data required time                                     4.396
```

```
  ------------------------------------------------------------
  data required time                                   4.396
  data arrival time                                   -1.545
  ------------------------------------------------------------
  slack (MET)                                          2.850


1

prompt> report_path_group
 ...
Path_Group     Weight   From      Through        To
------------------------------------------------------------
**HyperScale_stub_default**
                1.00   -         -              -
**async_default**
                1.00   -         -              -
**clock_gating_default**
                1.00   -         -              -
**default**     1.00   -         -              -
SYSCLK          1.00   *         *              SYSCLK

1
```

See Also

- hier_enable_analysis
- hier_enable_detailed_side_input_path_timing
- timing_report_hier_stub_pin_paths
- get_timing_paths
- report_timing

hier_enable_distributed_analysis

Enables the HyperScale distributed analysis flow.

Data Types

Boolean

Default false

Description

Set this variable to *true* to enable the HyperScale distributed analysis flow in the PrimeTime tool.

Feedback

In the distributed analysis flow, the tool analyzes the design hierarchically with single run and accurately handles the timing interfaces across hierarchical boundaries, improving the capacity and performance of hierarchical timing analysis without impacting the quality of results.

Specify the computation resources by using the *set_host_options* command. By default, HyperScale distributed analysis analyzes each block at the top level in a separate host process. To override the default partitioning of the design for analysis, use the *set_hier_config* command.

Enabling HyperScale distributed analysis affects the behavior of the *link_design*, *update_timing*, *save_session*, and reporting commands.

Because the behavior of the *link_design* command depends on whether HyperScale analysis is enabled, you must set the *hier_enable_distributed_analysis* variable before you link the design.

If the variable *hier_characterize_context_mode* is set to *constraints_only*, you cannot set the *hier_enable_distributed_analysis* variable to *true*.

You cannot set the *hier_enable_distributed_analysis* variable when the *hier_create_template_scripts* variable is set to *true*.

See Also

- set_hier_config
- set_host_options
- start_hosts
- hier_characterize_context_mode
- hier_create_template_scripts

hier_enable_pba_clock_latency_context

Specifies whether block context to capture path based latency for clock path entering the block, in addition to graph based latency.

Data Types

Boolean

Default false

Description

This variable is only effective for AOCVM analysis, which is enabled by setting *timing_aocvm_enable_analysis* to *true*.

Feedback

This variable applies to both the HyperScale flow and the binary context characterization flow. In HyperScale top level flow, by default, context for all the HyperScale sub-blocks are automatically captured; in flat flow, *characterize_context* and *write_context* maybe used to capture the context of target block(s).

When this variable is set to *false* (the default), the propagated clock latencies entering the block under characterization with be graph based analysis (GBA), which is accurate for block level GBA and conservative for block level PBA with the context.

When this variable is set to *true*, the propagated clock latencies is computed by performing full clock expansion back to the source and using path based analysis (PBA) to recalculate the latency. Using this context at block level PBA will further improve accuracy.

See Also

- hier_enable_analysis
- hier_characterize_context_mode
- set_hier_config
- characterize_context
- report_timing

---

hier_enable_release_resources_in_top_only_flow

Enables release of block compute resources after *update_timing* in the top-only distributed analysis flow.

Data Types

Boolean

Default true

Description

When this variable is set to true (the default), compute resources for block partitions are released after *update_timing* in the top-only distributed analysis flow. You invoke this flow by using the command *set_hier_config -enable_top_only_flow* ...

When you set this variable to *false*, compute resources for block partitions are not released after *update_timing* in the top-only distributed analysis flow, allowing multiple timing updates in the flow.

Feedback

See Also

- set_hier_config
- update_timing

---

hier_keep_required_time_mode

Specifies what type of required timing for stub pins are kept at the HyperScale block and loaded into HyperScale top.

Data Types

string

Default high_fanout_stub

Description

The *hier_keep_required_time_mode* variable controls what type of required timing for stub pins will be kept at HyperScale block and load into HyperScale top. It can be set separately at HyperScale block and top.

- *all_stub* - Set variable to *all_stub* at block means all required timing from stub pins will be kept at block and pass to top. Set variable to *all_stub* at top means all required timing for stub pins from block will be loaded at top.
- *high_fanout_stub* - Set variable to *high_fanout_stub* at block means only required timing from high fanout stub pins will be kept at block and pass to top. Set variable to *high_fanout_stub* at top means only required timing for high fanout stub pins from block will be loaded at top.
- *internal_path_stub* - Set variable to *internal_path_stub* at block means only required timing from internal path stub pins will be kept at block and pass to top. Set variable to *internal_path_stub* at top means only required timing for internal path stub pins from block will be loaded at top.
- *no_stub* - Set variable to *no_stub* at block means no required timing from stub pins will be kept at block and pass to top. Set variable to *no_stub* at top means no timing for stub pins from block will be loaded at top.

The following table shows with the specific setting of *hier_keep_required_time_mode* variable at block and top, which set of required timing for stub pins will be available at HyperScale top:

block | all_stub | high_fanout_stub | internal_path_stub | no_stub | top | | | | | ----------+----------+----------+----------+---------+ all_stub | all stub | high fanout stub | internal path stub | no stub | ----------+---------- +----------+----------+---------+ high_fanout_stub | high fanout stub | high fanout

Feedback

stub | no stub | no stub | ----------+----------+----------+----------+---------+ internal_path_stub |internal path stub| no stub | internal path stub | no stub | ----------+----------+----------+----------+---------+ no_stub | no stub | no stub | no stub | ----------+----------+----------+----------+---------+

See Also

- save_session
- set_hier_config

---

hier_merge_match_clock_with_constant

Enables clock and constant matching in HyperScale multi-instance data merge.

Data Types

Boolean

Default false

Description

The default value of the *hier_merge_match_clock_with_constant* variable is *false*, which means for multi-instances blocks, PrimeTime enforces the exact matching rule for clocks across those instances in the top-level HyperScale analysis and in constraint extraction.

PrimeTime automatically performs the context data merging during HyperScale analysis when you specify more than one instance of a given block with the same configuration. However, constraint and context data merging can be performed only when the clocks are exactly matched across the instances.

Exact matching does not require that the same clocks propagate to the same block port at the top level, or the same clocks be defined on the same pins inside the different instances of the same block. Instead, it requires that those clocks physically relevant for the instances match in fundamental attributes such as clock periods and waveforms.

However, a reused block is often designed with some redundancy to be instantiated for different situations in a chip. For example, some clock network is pushed into a block as feedthrough paths for more efficient clock delivery at chip. When such a block is multiply-instantiated, certain clock ports can be intentionally tied to constants when they are not driven by global clock signals for specific modes of the instance context. At these ports, some instances have clocks while some do not; by default, this violates the clock matching rule and results in unmerged contexts or constraints across these instances.

Feedback

When you set this variable to *true*, such mismatches between clock and constant across instances of the same block are relaxed, and the merged result treats the port as having a clock because it represents a more conservative timing analysis at the block level.

Note: This variable results in merging of context and constants with a union of clocks across instances. PrimeTime does not automatically create new constraints to group the clocks by instances even if they might be truly physically exclusive across instances. This implies potentially more pessimistic block-level analysis with the merged context when comparing to the full-chip flat analysis.

See Also

- save_session
- set_hier_config
- update_timing

---

hier_model_enable_unconstrained_port_abstraction

Specifies whether unconstrained ports should be automatically abstracted during HyperScale block model creation.

Data Types

Boolean

Default true

Description

In a HyperScale block-level analysis, interface logic is retained in the block-level models created for use in top-level timing analysis.

To improve the runtime and memory of top-level analysis, there are two ways that PrimeTime automatically prunes block interface logic:

- Pruning the fanout of ports that have only unconstrained logic
- Pruning the fanout of high-fanout ports that reach many endpoints in the block

This variable controls whether the unconstrained logic pruning is applied. When set to *true* (the default), input ports with no constrained paths in the fanout are pruned during block model creation.

For details on high-fanout pruning, see the variables mentioned in the SEE ALSO section.

Feedback

See Also

- hier_model_port_abstraction_ignore_register_count
- hier_model_port_abstraction_ignore_register_percentage

---

hier_model_port_abstraction_ignore_register_count

Specifies the register fanout threshold, as an absolute count, an input port must exceed to be automatically abstracted during HyperScale block model creation.

Data Types

integer

Default 256

Description

In a HyperScale block-level analysis, interface logic is retained in the block-level models created for use in top-level timing analysis.

To improve the runtime and memory of top-level analysis, there are two ways that PrimeTime automatically prunes block interface logic:

- Pruning the fanout of ports that have only unconstrained logic
- Pruning the fanout of high-fanout ports that reach many endpoints in the block

This variable is part of the high-fanout port pruning. A port's interface logic is removed if both of the following are true:

- The fanout register count of a port exceeds the threshold value of this variable
- The number of fanout registers of a port, as a percentage of all registers in the block, exceeds the threshold value of the *hier_model_port_abstraction_ignore_register_percentage* variable

For details on unconstrained port pruning, see the unconstrained port variable mentioned in the SEE ALSO section.

See Also

- hier_model_enable_unconstrained_port_abstraction
- hier_model_port_abstraction_ignore_register_percentage

Feedback

hier_model_port_abstraction_ignore_register_percentage

Specifies the register fanout threshold, as a percentage of all registers, an input port must exceed to be automatically abstracted during HyperScale block model creation.

Data Types

float

Default 1 (percent)

Description

In a HyperScale block-level analysis, interface logic is retained in the block-level models created for use in top-level timing analysis.

To improve the runtime and memory of top-level analysis, there are two ways that PrimeTime automatically prunes block interface logic:

- Pruning the fanout of ports that have only unconstrained logic
- Pruning the fanout of high-fanout ports that reach many endpoints in the block

This variable is part of the high-fanout port pruning. A port's interface logic is removed if both of the following are true:

- The number of fanout registers of a port, as a percentage of all registers in the block, exceeds the threshold value of this variable
- The fanout register count of a port exceeds the threshold value of the *hier_model_port_abstraction_ignore_register_count* variable

For details on unconstrained port pruning, see the unconstrained port variable mentioned in the SEE ALSO section.

See Also

- hier_model_enable_unconstrained_port_abstraction
- hier_model_port_abstraction_ignore_register_count

hier_modeling_version

Specifies the current modeling technique version.

Data Types

float

Default 1.0

Feedback

Description

This variable specifies the current modeling technique version. In the default version of 1.0, the verification script duplicates constraints in the instance script. There is no good way to test the instance script. When an ETM is instantiated at the top level, you must change the names of generated clocks in the constraint file so they match the names of clocks. For ILM, when a model is instantiated more than one time at the top level, you must also change the constraint file such that the names of clocks are uniquified.

If you set this variable to 2.0, the *-validate* option of the *create_ilm* and *extract_model* commands is enabled, and the generated models are automatically validated after they are created. In addition, all interface logic model (ILM) files generated are added to the pt_model_dir/ILM/design_name directory, and all extracted timing model (ETM) files are added to the pt_model_dir/ETM/output directory. These names are consistent for ILM and ETM.

In version 2.0, a test design is always created for ETM and ILM. This netlist information of the test design is in the mod_verif.v file. The constraints in the mod_verif.pt.gz file are those constraints that define the outside context (for example, the *set_input_delay* command), and the constraints in the mod_inst.pt.gz file are those constraints that are brought to the top level by the model. The mod_verif.pt.gz file sources the mod_inst.pt.gz file directly, and there are no duplicated constraints in these two files.

In version 2.0, all constraints in the mod_inst.pt.gz file are in a Tcl procedure. This procedure has two arguments: inst_name and clock_name_prefix. The constraint file needs to be sourced only one time although the model might be instantiated more than one time. The names of clocks whose sources are inside the block and all generated clocks are uniquified. You do not need to change the constraint script to uniquify clocks. You do not need to use the *current_instance* command before sourcing this constraint file.

In version 2.0, internal nongenerated clocks are created in the mod_inst.pt.gz file for ETM. The uncertainty information of all generated clocks and internal nongenerated clocks are also added to the mod_inst.pt.gz file.

See Also

- create_ilm
- extract_model
- pt_model_dir

---

hier_override_clock_sense_from_context

Override the clock source sense if mismatch between context and constraints has been found at the pin.

Feedback

Data Types

Boolean

Default true

Description

By default, HyperScale hierarchical analysis applies the clock source sense from context.

To force the clock source sense from constraints, set the *hier_override_clock_sense_from_context* variable to *false*. This is useful when you want to forcibly apply the constrained clock sense at the pin.

---

hier_pp_report_limit

The point to point exceptions in report_context content limitation.

Data Types

integer

Default 500

Description

When we do report_context, the point to point exceptions are also reported, but we limit the number of exceptions to 500 by default, if you want to report more than that, you can manually increase that number by "set hier_pp_report_limit $number"

---

hierarchy_separator

Determines how hierarchical elements of the netlist are delimited in reports and how they are searched for in selections and other commands.

Data Types

string

Default / (forward slash)

Description

This command determines how hierarchical elements of the netlist are delimited in reports and how they are searched for in selections and other commands. The choice of a separator is limited to these characters: bar (|), caret (^), at (@), dot (.), and the default of a forward slash (/).

Normally, you should accept the default, forward slash (/). However, in some cases where the hierarchy character is embedded in some names, the search engine might produce

Feedback

results that are not intended. Using the *hierarchy_separator* variable is a convenient method for dealing with this situation. For example, consider a design that contains a hierarchical cell A, which contains hierarchical cells B and B/C; B/C contains D; B contains C. Searching for A/B/C/D is ambiguous and might not match what you intended. However, if you set the *hierarchy_separator* variable to the vertical bar |, searching for A|B/C|D is explicit, as is A|B|C|D.

i

ilm_ignore_percentage

Specifies a threshold for the percentage of total registers in the transitive fanout of an input port, beyond which the port is to be ignored when identifying interface logic.

Data Types

float

Default 25

Description

Specifies a minimum threshold for the percentage of the total registers in the transitive fanout of an input port, beyond which the port is to be ignored when identifying interface logic.

This variable affects the *-auto_ignore* option of the *identify_interface_logic* command. The *-auto_ignore* option automatically determines those ports (for example, scan enable and reset ports) that should be ignored when the *identify_interface_logic* command places the *is_interface_logic_pin* attribute on objects to identify them as part of the interface logic model (ILM) for the design. Ports are automatically ignored if they fan out to a percentage of total registers in the design greater than the value specified by this variable.

For more information about creating ILMs and associated commands, see the *identify_interface_logic* man page.

See Also

- identify_interface_logic

imsa_save_eco_data

Specifies an additional set of attributes to be saved and restored in IMSA sessions to support physically aware ECO operations using the PrimeTime GUI.

Feedback

Data Types

Boolean

Default false

Description

By default, the *save_session -only_timing_paths* command saves only the paths in the specified timing path collection, the attributes of those timing paths and their timing points, and the attributes of the associated clocks. No other attributes are saved. The *restore_session* command restores the paths and attributes to an interactive multi-scenario analysis (IMSA) session.

To enable saving and restoring a predefined additional set of attributes to support physically aware ECO operations using the PrimeTime GUI, set the *imsa_save_eco_data* variable to *true*. Then the *save_session -only_timing_paths* command saves the following additional attributes:

```
Class      Attribute Name
-------------------------
pin        max_rise_slack
pin        min_rise_slack
pin        max_fall_slack
pin        min_fall_slack
pin        max_capacitance
pin        max_transition
pin        max_fanout
```

You can also expand the set of saved and restored attributes by using *imsa_save_reporting_attributes* variable, and add or remove specific attributes by using the *set_imsa_attributes* command.

You can report the list of attributes enabled for saving by using the *set_imsa_attributes -report* command.

In DMSA mode, the value of this variable is synchronized from the manager process to worker processes.

See Also

- get_attribute
- get_timing_paths
- restore_session
- save_session

Feedback

- set_imsa_attributes
- imsa_save_reporting_attributes

---

imsa_save_reporting_attributes

Specifies an additional set of attributes to be saved and restored in IMSA sessions to support expanded reporting capabilities.

Data Types

Boolean

Default false

Description

By default, the *save_session -only_timing_paths* command saves only the paths in the specified timing path collection, the attributes of those timing paths and their timing points, and the attributes of the associated clocks. No other attributes are saved. The *restore_session* command restores the paths and attributes to an interactive multi-scenario analysis (IMSA) session.

To enable saving and restoring a predefined additional set of attributes to support expanded reporting capabilities in the IMSA session, set the *imsa_save_reporting_attributes* variable to *true*. Then the *save_session -only_timing_paths* command saves the following additional attributes:

```
Class      Attribute Name
-------------------------------------------
pin        max_rise_slack
pin        min_rise_slack
pin        max_fall_slack
pin        min_fall_slack
pin        max_capacitance
pin        max_transition
pin        actual_fall_transition_max
pin        actual_rise_transition_max
pin        annotated_rise_transition_delta_max
pin        annotated_fall_transition_delta_max
pin        constraining_max_transition
design     max_transition
design     max_capacitance
```

You can also expand the set of saved and restored attributes by using *imsa_save_eco_data* variable, and add or remove specific attributes by using the *set_imsa_attributes* command.

You can report the list of attributes enabled for saving by using the *set_imsa_attributes -report* command.

Feedback

In DMSA mode, the value of this variable is synchronized from the manager process to worker processes.

See Also

- get_attribute
- get_timing_paths
- restore_session
- save_session
- set_imsa_attributes
- imsa_save_eco_data

in_gui_session

This read-only variable is *true* when the graphical user interface (GUI) is active and *false*, the default, when the GUI is inactive.

Data Types

Boolean

Default false

Description

You can use this variable when writing Tcl code that depends on the presence of the GUI. If you have invoked the *gui_start* command and the GUI is active, the read-only variable is *true*. Otherwise, the variable is *false* and the GUI is inactive.

See Also

- gui_start
- gui_stop

input_slews_violations

This man page describes *input_slews* violations shown by the *report_constraint* command in the HyperScale flow.

Description

An *input_slews* violation indicates a mismatch of input slews at a pin.

Feedback

What Next

This is an *auto_fixable* violation that HyperScale automatically resolves in subsequent iterations.

See Also

- report_constraint
- auto_fixable_violations

---

interactive_multi_scenario_analysis_enabled

Indicates whether the Interactive Multi-Scenario Analysis (IMSA) mode is enabled.

Data Types

Boolean

Default false

Description

This read-only variable indicates whether the Interactive Multi-Scenario Analysis (IMSA) mode is enabled. IMSA is enabled by restoring a PrimeTime session saved using the *save_session* command with the *-only_timing_paths* option.

While in IMSA mode, multiple sessions saved with the *-only_timing_paths* option can be restored provided that the netlists are substantially equivalent. This allows for the analysis of timing paths from different corners of a design without requiring all of the scenarios to be online such as in Distributed Multi-Scenario Analysis (DMSA).

Since *save_session* with the *-only_timing_paths* option ignores most timing data to prevent excessive memory consumption, there are restrictions on PrimeTime commands which require timing data.

SESSION COMPATIBILITY

There are two primary restrictions on the compatibility of sessions. • Variables which modify the behaviour of timing paths must be set identically on all scenarios. • Designs must be substantially equivalent. The design layout must be identical with two exceptions. Firstly, the pin ordering may diverge, provided the lib pin names remain identical. Secondly, ETM internal pins may or may not exist. If the first session restored does not have an internal pin for ETM models, then internal pins in all subsequent restores of timing paths will be virtualized. That is, they will be present in the timing path but not exist in the netlist.

Feedback

See Also

- restore_session

l

library_mapping_violations

This man page describes *library_mapping* violations shown by the *report_constraint* command in the HyperScale flow.

Description

A *library_mapping* violation indicates differences in the libraries being used. The use of libraries includes linking the design, selecting operating condition, and defining the scaling library group. A mismatch in libraries used between block- and top-level analysis can cause different timing results and therefore need to be fixed.

Timing constraint commands referring to libraries include (and not limited to) *set_min_library*, *define_scaling_lib_group*, *set_operating_condition*, *set_wire_load_model*, also the variables such as *link_path*.

Note that *library_mapping* violations might also result in other violations such as operating conditions and timing derates, because they can refer to libraries that are different between block and top, and result in violations reported for those other checks also.

What Next

This is a *non_auto_fixable* violation. You must resolve the difference in libraries. Consistent usage and settings of library related constraints are required to ensure consistent results for the timing analysis.

Consistent settings and usage of libraries between the block and top levels can also resolve other types of HyperScale boundary constraint violations, such as difference in operating conditions and library-cell derating.

Examples

The following verbose report shows a simple difference of the top level enabling CCS calculation while the block level does not use CCS.

```
pt_shell> report_constraint -boundary_check_include {library_mapping} \\
          -all_violators -verbose

****************************************
Report : constraint
        -verbose
 ...
```

Feedback

```
****************************************

HyperScale constraints report

Constraint: library_mapping

Instance   Type            Block                                  Top
-------------------------------------------------------------------------------
--------------------------
blk1       Main Library    /my/libraries/A_lib.db:nldm_lib
  /remote/libraries/ccs_1.1.db:ccs_lib
```

See Also

- define_scaling_lib_group
- report_constraint
- set_min_library
- set_operating_conditions
- set_wire_load_model
- link_path
- non_auto_fixable_violations
- search_path

---

library_pg_file_pattern

Specifies the file name pattern for the power and ground (PG) Tcl side files for library PG conversion and update.

Data Types

string

Default ""

Description

Use this variable to specify the file name pattern for the PG Tcl side files for library PG conversion and update. By setting the variable to a valid file name pattern, the tool finds the associated PG Tcl side file for the logic libraries provided. At library loading time, the tool performs on-the-fly PG updates on the in-memory databases. With this on-the-fly library PG conversion and update capability, the tool relaxes the requirement of complete PG pin library for UPF specifications.

Feedback

As default, the variable is set to "", which means that there is no PG Tcl side file, unless specified.

For advanced users, string substitution can be used for finding PG Tcl side file (limit one occurrence per pattern):

- Use pattern "__DIR__" for path to .db dir
- Use pattern "__FILE__" for leaf file name for .db

There can be one Tcl for all databases, one Tcl per database, or one Tcl per a group of databases. For example:

1. One Tcl for all databases:

- At the current directory

```
set library_pg_file_pattern "libpg_sidefile.pg"
```

- At different location

```
set library_pg_file_pattern "/my_dir/libpg_sidefile.pg"
```

1. One Tcl per database: • At the same location as the original database files

```
set library_pg_file_pattern "%d/%f.pg"
```

- At a directory called "pg_sidefiles" under the same directory as original the original database

```
set library_pg_file_pattern "%d/pg_sidefiles/%f.pg"
```

- At a different location called "/my_dir"

```
set library_pg_file_pattern "/my_dir/%d/%f.pg"
```

1. One Tcl for a group of databases at the same location as the database files

```
set library_pg_file_pattern "%d/libpg_sidefile.pg"
```

Note: You must set this variable before loading the libraries.

Feedback

This on-the-fly database updates is disabled in the power domain backward compatibility mode (*set power_domains_compatibility true*).

---

link_allow_design_mismatch

Controls the behavior of the link design when pin mismatch between instance and reference occur.

Data Types

Boolean

Default false

Description

This variable controls whether design linking succeeds when pin mismatches between instance and reference occur. By default, linking fails if there are pin mismatches between the instance and reference. For example, when a pin exists in the instance but does not exist in the library, link issues an error and fails.

If you set this variable to *true*, the extra pin is ignored, and design linking continues. This allows you to gather useful information even if part of a design is missing.

Common causes of mismatches include:

1. A pin has different directions in instance and reference.
2. A pin of instance does not exist in reference.
3. A bus has different widths in instance and reference.

When a mismatch occurs, the reference always take precedence.

- In case 1, the direction of the pin in the linked design is from the reference.
- In case 2, the pin is ignored and does not exist in the reference.
- In case 3, all extra bits in the instance are ignored, and the bus width in the linked design is the same as the bus width from the reference.

Note that for bus width mismatches, the least significant bit of instance is mapped to least significant bit of the reference; all extra bits of the instance are ignored, and all extra bits of the reference are dangling.

To report mismatches found during link, use the *report_design_mismatch* command.

Feedback

See Also

- link_design
- report_design_mismatch

---

link_create_black_boxes

Enables design linking to automatically convert unresolved references into black boxes.

Data Types

Boolean

Default true

Description

By default, this variable is set to *true*, and design linking automatically converts each unresolved reference into a black box, an empty cell with no timing arcs. The result is a completely linked design on which you can analysis.

If you set this variable to *false*, unresolved references remain unresolved, and most analysis commands cannot work.

See Also

- link_design
- search_path

---

link_force_case

Controls the case sensitivity behavior of the link design command.

Data Types

string

Default check_reference

Description

This variable controls the case-sensitive or insensitive behavior of the *link_design* command. Allowed values are the default of *check_reference*, *case_sensitive*, or *case_insensitive*. The *check_reference* option means that the case sensitivity of the link is determined only by the case sensitivity of the input format that created that reference. For example, a VHDL reference is linked case-insensitively, whereas a Verilog reference is linked case-sensitively.

Feedback

Some caveats apply to this variable, as follows:

1. Do not set the *link_force_case* variable to *case_insensitive* if you are reading in source files from case-sensitive formats (for example, Verilog). Doing so could cause inconsistent, unexpected, and undesirable results. For example, you might have an instance u1 of design 'inter', but might have loaded a design 'Inter'. If you do a case-insensitive link, you get design 'Inter'. The side effect is that the relationship between u1 and 'inter' is gone; it has been replaced by a relationship between u1 and 'Inter'. Changing the *link_force_case* variable back to *check_reference* or *case_sensitive* does not restore the original relationship. You would have to remove the top design, reload it, and relink. Note: Design Compiler has the same restriction.
2. Do not change the value of this variable within a session. Doing so could cause numerous error and warning messages that can cause confusion.
3. Setting the *link_force_case* variable to *case_insensitive* can decrease the performance of the *link_design* command.

See Also

- link_design
- link_create_black_boxes

---

link_keep_cells_with_pg_only_connection

Keeps cells that are connected to only power or ground (PG) nets after design linking.

Data Types

Boolean

Default false

Description

This variable controls whether to keep cells that are connected only to power or ground (PG) nets after design linking. This variable is ignored if the *link_keep_unconnected_cells* variable is set to *true*.

- *false* (the default) - Deletes cells that are connected only to PG nets.
- *true* - Keeps cells that are connected only to PG nets.

You must set this variable before you load designs and libraries into PrimeTime.

Feedback

See Also

- link_design
- link_keep_unconnected_cells

---

link_keep_pg_connectivity

Enables the Verilog PG flow, which reads in Verilog PG netlist and derives the rail connectivity data from that netlist.

Data Types

Boolean

Default false

Description

By default, the PrimeTime tool uses UPF-specified power intent and ignores PG connectivity information in Verilog netlists. However, you can optionally have the tool to derive PG connectivity from a PG Verilog netlist, in which case there is no need to load UPF files.

To do this, set the *link_keep_pg_connectivity* variable to *true* before you read in any design information. When you read in the netlist, the tool derives the UPF power intent from the PG netlist data and implicitly updates the design database as if it were running UPF commands like *create_power_domain*, *create_supply_port*, *create_supply_net*, and *connect_supply_net*. After you link the design, you can set the supply voltages by using the *set_voltage* command, just like the UPF flow.

This PG netlist flow works with design data generated in either the UPF-prime flow or the golden UPF flow. It supports PrimeTime features such as timing analysis, voltage scaling, power switches, and ECOs. Note that the PG netlist flow requires all the netlists to have PG information; it does not accept mixture of PG and non-PG netlists.

Loading the UPF file might be necessary for using the *extract_model* command in combination with the *set_port_attributes* and *set_related_supply_net* commands.

To enable reconnection of supply nets to PG pins that already have connections in the PG connectivity from the netlist, set the *pg_allow_pg_pin_reconnection* variable to *true*.

See Also

- connect_supply_net
- create_power_domain
- create_power_switch

Feedback

- create_supply_port
- pg_allow_pg_pin_reconnection

link_keep_unconnected_cells

Keeps unconnected cells after design linking.

Data Types

Boolean

Default false

Description

This variable controls whether to keep unconnected cells after design linking.

- *false* (the default) - Discards unconnected cells. By default, the tool also discards cells that are connected to only power/ground (PG) nets; to keep cells that are connected to only PG nets, set the *link_keep_cells_with_pg_only_connection* to *true*.
- *true* - Keeps unconnected cells. With this setting, the tool also keeps cells that are connected to PG nets, regardless of the *link_keep_cells_with_pg_only_connection* variable setting.

You must set this variable before you load designs and libraries into PrimeTime.

See Also

- link_design
- link_keep_cells_with_pg_only_connection

link_library

This is a synonym for the *link_path* variable.

See Also

- link_path

link_path

Specifies a list of libraries, design files, and library files used during linking.

Feedback

Data Types

list

Default *

Description

This variable specifies a list of libraries, design files, and library files used during linking. The *link_design* command looks at those files and tries to resolve references in the order that you specify.

The *link_path* variable can contain three types of elements: *, a library name, or a file name.

The "*" entry in the value of this variable indicates that the *link_design* command should search all the designs loaded in the pt_shell while trying to resolve references. Designs are searched in the order in which they were read.

For elements other than "*", PrimeTime searches for a library that has already been loaded. If that search fails, PrimeTime searches for a file name using the *search_path* variable.

The libraries that are specified by the *link_path* variable are loaded in parallel with Verilog files during the *read_verilog* command. For best runtime performance, set the *search_path* and *link_path* variables before you run the *read_verilog* command.

See Also

- link_design
- link_path_per_instance
- search_path

---

link_path_per_instance

Overrides the default link path for selected leaf cell or hierarchical cell instances.

Data Types

list

Default (empty)

Description

This variable, which takes effect only if set before linking the current design, overrides the default *link_path* variable for selected leaf cell or hierarchical cell instances. The format is

Feedback

a list of lists. Each sublist consists of a pair of elements: a set of instances, and a *link_path* specification that should be used for and within these instances. For example,

```
        set link_path {* lib1.db}
set link_path_per_instance [list
   [list {ucore} {* lib2.db}]
   [list {ucore/usubblk} {* lib3.db}]]
```

Entries are used to link the specified level and below. If a given block matches multiple entries in the per-instance list, the more specific entry overrides the more general entry. In the preceding example:

1. lib3.db would be used to link blocks 'ucore/usubblk' and below.
2. lib2.db would be used to link 'ucore' and below (except within 'ucore/subblk').
3. lib1.db would be used for the remainder of the design (everything except within 'ucore').

The default value of the *link_path_per_instance* variable is an empty list, meaning that the feature is disabled.

See Also

- link_design
- link_path
- link_path_per_instance

---

link_use_only_master_cells_for_linking

Applies a temporary workaround for supporting designs that have instances of physical variant cells.

Data Types

Boolean

Default false

Description

Physical variant cells are library cells that have multiple physical views, but only a single Liberty timing library definition. In Liberty, they are identified by the *physical_variant* attribute being defined on a "master" cell.

To link designs that have instances of such physical variant library cells, set this variable to *true*. In this case, the tool links all such instances to the master library cell of the physical

Feedback

variant cell. With this setting, you will not be able to refer to physical variant library cells in your scripts. Any cell-based derates must be applied to the master cells only.

See Also

- link_design

---

lp_default_ground_pin_name

Specifies the default ground pin name for creating default PG pins for the library cells in non-PG pin libraries.

Data Types

string

Default ""

Description

Use this variable to specify the default ground pin name for default PG pin creation. This variable is also used for ground pin name for database conversion for legacy rail_connection libraries. One default power and one default ground PG pins are created for the library cells in non-PG pin libraries, which are performed on the in-memory databases by the tool at library loading time. With this capability, the tool relaxes the requirement of PG pin library for UPF specifications.

The name is used for pg_pin of library cells and voltage_map defined at library level. For library cells from PG pin libraries, the variable have no impact. The default PG pin creation isl only triggered when the PG Tcl side file is not provided (specified by the *library_pg_file_pattern* variable). UPF explicit connections (using the *connect_supply_net* command) are not allowed on default PG pins created based on the default PG pin name variables.

The default of this variable is empty "", which means that there is no default PG pin added unless specified. To enable default PG pin creation, set the *lp_default_power_pin_name* variable to a valid name.

Note: You must set this variable before loading libraries.

Database updates are disabled in power domain backward compatibility mode (when you set the *power_domains_compatibility* variable to *true*).

Feedback

See Also

- connect_supply_net
- library_pg_file_pattern
- lp_default_power_pin_name

---

lp_default_power_pin_name

Specifies the default power pin name for creating default PG pins for the library cells in non-PG pin libraries.

Data Types

string

Default ""

Description

Use this variable to specify the default power pin name for default PG pin creation. One default power and one default ground PG pins is created for the library cells in non-PG pin libraries, which is performed on the in-memory databases by the tool at the time the library is loaded. With this capability, the tool relaxes the requirement of PG pin library for UPF specifications.

The name is used for pg_pin of library cells and voltage_map defined at library level. For library cells from PG pin libraries, the variable have no impact. The default PG pin creation is triggered only when the PG Tcl side file is not provided (specified by the *library_pg_file_pattern* variable). UPF explicit connections (using the *connect_supply_net*) is not allowed on default PG pins created based on the default PG pin name variables.

The default of this variable is empty "", which means that there is no default PG pin added unless specified. To enable default PG pin creation, you must set the *lp_default_ground_pin_name* variable to a valid name.

Note: You must set this variable before loading the library.

The database updates are disabled in power domain backward compatibility mode (Use the *set power_domains_compatibility TRUE* command).

See Also

- connect_supply_net
- library_pg_file_pattern
- lp_default_ground_pin_name

Feedback

m

merge_model_allow_generated_clock_renaming

Enables or disables forcing merging ETM models by resolving conflicting generated clocks.

Data Types

Boolean

Default false

Description

When you merge an extracted timing model (ETM), generated clocks in each ETM model with the same name should have the same definition. By default, if you use the *merge_models* command to merge ETMs with generated clocks of the same name but different definitions, the tool issues an error message, and the model merging fails.

When you set this variable to *true*, this issue is resolved by renaming the generated clock to include the corresponding mode name as suffixes.

See Also

- merge_model_ignore_pin_function_check

merge_model_ignore_pin_function_check

Enables or disables forcing merging ETM models to ignore function attributes.

Data Types

Boolean

Default false

Description

When you merge ETM models, the tool expects either a *true* or a *false* in the *function* attribute. If you use the *merge_models* command to merge ETM models with non-constant Boolean functions, the tool issues an error, and model merging fails. If you set this variable to *true*, the models are forced to merge, ignoring the *function* attribute.

Feedback

See Also

- merge_models
- merge_model_allow_generated_clock_renaming

---

model_validation_capacitance_tolerance

Specifies the absolute error tolerance for capacitance data.

Data Types

list

Default {0.001 0.001}

Description

Specifies the absolute error tolerance for capacitance data. If this is used in conjunction with the *-percent_tolerance* option, both tolerances must be exceeded to cause a FAIL. For information about other absolute tolerances, see the *model_validation_timing_tolerance* man page.

This variable is effective only in the automatic model validation flow, such as when the output from either the *create_ilm* or *extract_model* command is used with the *-validate* option. The output of the *compare_interface_timing* command is not affected by this variable.

See Also

- create_ilm
- extract_model
- hier_modeling_version
- model_validation_timing_tolerance

---

model_validation_check_design

Checks the design constraints and settings when auto model validation fails

Data Types

Boolean

Default true

Feedback

Description

If you set this variable to *true*, the tool tries to find design constraints and settings that are not recommended for creating and validating models.

If you set this variable to *false*, no checking is performed.

This variable is effective only in the automatic model validation flow.

See Also

- create_ilm
- extract_model
- hier_modeling_version

---

model_validation_ignore_pass

Excludes all passing comparisons; therefore, fails are only displayed if this variable is set to its default of *true*.

Data Types

Boolean

Default true

Description

Excludes all passing comparisons; therefore, fails are only displayed if this variable is set to its default of *true*.

This variable is effective only in the automatic model validation flow, such as when the output from either the *create_ilm* or *extract_model* command is used with the *-validate* option. The output of the *compare_interface_timing* command is not affected by this variable.

See Also

- create_ilm
- extract_model
- hier_modeling_version

---

model_validation_output_file

Specifies the name of the file that stores the model validation results.

Feedback

Data Types

string

Default verif.out

Description

Specifies where the model validation results should be written. If this variable is empty, the results are printed at the standard output. If no path is given in this name, this file is created in the model_validation subdirectory under the directory specified by the *pt_model_dir* variable.

This variable is effective only in the automatic model validation flow, such as when the output from either the *create_ilm* or *extract_model* command is used with the *-validate* option. The output of the *compare_interface_timing* command is not affected by this variable.

See Also

- create_ilm
- extract_model
- hier_modeling_version
- pt_model_dir

---

model_validation_pba_clock_path

Enables path-based analysis on clock portion of mismatched paths.

Data Types

Boolean

Default false

Description

By default, path-based analysis in model validation is only performed on the data portion of mismatched paths. When you set this variable to *true*, path-based analysis is performed on the clock portion too. This can help resolving mismatched paths.

This variable is effective only in the automatic model validation flow, such as when the output from either the *create_ilm* or *extract_model* command is used with the *-validate* option. The output of the *compare_interface_timing* command is not affected by this variable.

Feedback

See Also

- create_ilm
- extract_model
- hier_modeling_version

---

model_validation_percent_tolerance

Specifies the absolute error tolerance for time data during model validation.

Data Types

list

Default {0.00 0.00}

Description

This option is similar to the *model_validation_timing_tolerance* variable except that it uses the percent relative error instead of an absolute error. The exception is that slack data ignores this variable and uses only the *model_validation_timing_tolerance* variable. If this variable is specified in conjunction with either the *model_validation_timing_tolerance* or *model_validation_capacitance_tolerance* variable, both tolerances must be exceeded to cause a FAIL.

This variable is effective only in the automatic model validation flow, such as when the output from either the *create_ilm* or *extract_model* command is used with the *-validate* option. The output of the *compare_interface_timing* command is not affected by this variable.

See Also

- create_ilm
- extract_model
- hier_modeling_version
- model_validation_capacitance_tolerance
- model_validation_timing_tolerance

---

model_validation_reanalyze_max_paths

Specifies the maximum number of mismatched paths that are analyzed again.

Feedback

Data Types

integer

Default 1000

Description

During automatic model validation, some mismatched paths after graph-based analysis are reselected to do path-based analysis. This variable specifies the maximum number of selected paths. The smaller the number, the faster the validation occurs; however, more unresolved mismatches might occur.

This variable is effective only in the automatic model validation flow, such as when the output from either the *create_ilm* or *extract_model* command is used with the *-validate* option. The output of the *compare_interface_timing* command is not affected by this variable.

See Also

- create_ilm
- extract_model
- hier_modeling_version

---

model_validation_report_split

Enables line-splitting in the validation report.

Data Types

Boolean

Default true

Description

Allows line-splitting in a report. This is most useful for performing diffs on previous scripts or for postprocessing the script.

This variable is effective only in the automatic model validation flow, such as when the output from either the *create_ilm* or *extract_model* command is used with the *-validate* option. The output of the *compare_interface_timing* command is not affected by this variable.

Feedback

See Also

- create_ilm
- extract_model
- hier_modeling_version

---

model_validation_save_session

Saves a session during model validation that constrains the testing design of the model.

Data Types

Boolean

Default true

Description

When the *model_validation_save_session* variable is set to *true* (the default), a session is saved that constrains the testing design of the model.

If you set this variable to *false*, no session is saved.

This variable is effective only in the automatic model validation flow.

See Also

- create_ilm
- extract_model
- hier_modeling_version

---

model_validation_section

Specifies a list of data sections to be included in the comparison.

Data Types

string

Default "" (empty)

Description

Specifies a list of data sections to be included in the validation; only those sections are validated. By default, all sections are validated. If it is not empty, defined sections are

Feedback

validated. The following list describes the keywords that are accepted in the section_list as valid:

- *slack* - Specifies that only worst-case slacks are to be validated.
- *transition_time* - Specifies that only actual transition times on ports are to be validated.
- *capacitance* - Specifies that only actual capacitances on ports are to be validated.
- *design_rules* - Specifies that only design rules on ports are to be validated.
- *missing_arcs* - Specifies that only arcs in the reference timing file that are missing from the compare timing file are to be included in the report.

This variable is effective only in the automatic model validation flow, such as when the output from either the *create_ilm* or *extract_model* command is used with the *-validate* option. The output of the *compare_interface_timing* command is not affected by this variable.

See Also

- create_ilm
- extract_model
- hier_modeling_version

---

model_validation_significant_digits

Specifies the number of digits to the right of the decimal point that are to be reported in the validation results.

Data Types

integer

Default 3

Description

Allowed values are 0-13. The default value is 3.

This variable is effective only in the automatic model validation flow, such as when the output from either the *create_ilm* or *extract_model* command is used with the *-validate* option. The output of the *compare_interface_timing* command is not affected by this variable.

Feedback

See Also

- create_ilm
- extract_model
- hier_modeling_version

---

model_validation_sort_by_worst

Specifies that the output is to be sorted from worst to best.

Data Types

Boolean

Default true

Description

Specifies that the output is to be sorted from negative to positive, with the worst failure first. After the FAIL section, all PASS lines are sorted alphabetically within each path attribute.

This variable is effective only in the automatic model validation flow, such as when the output from either the *create_ilm* or *extract_model* command is used with the *-validate* option. The output of the *compare_interface_timing* command is not affected by this variable.

See Also

- create_ilm
- extract_model
- hier_modeling_version

---

model_validation_timing_tolerance

Specifies the absolute error tolerance for time data during model validation.

Data Types

list

Default {0.01 0.025}

Feedback

Description

Specifies the absolute error tolerance for time data during model validation. Each tolerance is a list of either one or two floating point numbers. All tolerances are inclusive, and the sign of the values has no effect. The first value is the optimistic tolerance and the second value is the pessimistic tolerance. If there is only one number it serves to specify both the pessimistic and optimistic tolerances. If this is used in conjunction with the *model_validation_percent_tolerance* variable, both tolerances must be exceeded to cause a FAIL. For other absolute tolerances, see the *model_validation_capacitance_tolerance* man page.

This variable is effective only in the automatic model validation flow, such as when the output from either the *create_ilm* or *extract_model* command is used with the *-validate* option. The output of the *compare_interface_timing* command is not affected by this variable.

See Also

- create_ilm
- extract_model
- hier_modeling_version
- model_validation_percent_tolerance

---

model_validation_verbose

Specifies that detailed debugging of mismatched paths is performed.

Data Types

Boolean

Default false

Description

Specifies that a detailed debugging of mismatched paths is performed. If you set this variable to *true*, a detailed debugging of mismatched paths is performed, which generates a report that shows why those paths are mismatched. The name of the report is the name of the output file specified by the *model_validation_output_file* variable with the .detail extension.

For ILMs, the tolerance of delay and slews is one-tenth of the tolerance specified by the *model_validation_timing_tolerance* variable, and the tolerance of capacitance is the same with the *model_validation_capacitance_tolerance* variable. If either of these tolerances are

Feedback

violated for a stage, the detailed information of this stage is printed; however, only the first stage violating the tolerances is printed.

For ETMs, since paths from the model are generally simple, the *report_etm_arc* style report is printed for each mismatched arc and stage information is not compared.

If it is found that the topologies of paths from the netlist and the model are different with each other, another detailed debugging is performed: the exactly same path from the model is selected from the netlist and compared against with the original path selected from the netlist. This debugging can help you understand why the *create_ilm* or *extract_model* command picks a different critical path.

This variable is effective only in the automatic model validation flow, such as when the output from either the *create_ilm* or *extract_model* command is used with the *-validate* option. The output of the *compare_interface_timing* command is not affected by this variable.

Examples

The following examples shows that the mismatch is because an aggressor that is not screened in the block gets screened when the interface logic model (ILM) is used.

```
Netlist path:

  Startpoint: IN (input port clocked by CLK)
  Endpoint: ff1 (rising edge-triggered flip-flop clocked by CLK')
  Path Group: CLK
  Path Type: max

  Point                        Fanout    Cap      Trans       Incr
 Path
-------------------------------------------------------------------------
------
  clock CLK (rise edge)                                      0.00
 0.00
  clock network delay (propagated)                           0.00
 0.00
  input external delay                                       0.20
 0.20 f
  IN (in)                                          0.00      0.00
 0.20 f
  IN (net)                      1     1.00
  U1/A (ND2)                                       0.00      0.00
 0.20 f
  U1/Z (ND2)                                       0.17      0.67 &
 0.87 r
  n1 (net)                      1     1.23
  ff1/D (FD1)                                      0.18      0.01 &
 0.88 r
  data arrival time
 0.88
```

Feedback

```
  clock CLK' (rise edge)                                          0.60
 0.60
  clock source latency                                            0.00
 0.60
  CLK (in)                                          0.00          0.00
 0.60 f
  CLK (net)                       3     3.00
  U2/A (IV)                                         0.00          0.00
 0.60 f
  U2/Z (IV)                                         0.14          0.52
 1.13 r
  n2 (net)                        1     1.00
  ff1/CP (FD1)                                      0.14          0.00
 1.13 r
  library setup time                                             -0.80
 0.33
  data required time
 0.33
--------------------------------------------------------------------------
------
  data required time
 0.33
  data arrival time
 -0.88
--------------------------------------------------------------------------
------
  slack (VIOLATED)
 -0.55


The dominant exceptions are:
        None

Model path:

  Startpoint: IN (input port clocked by CLK)
  Endpoint: ff1 (rising edge-triggered flip-flop clocked by CLK')
  Path Group: CLK
  Path Type: max
  Scenario: model_validation

  Point                         Fanout    Cap      Trans       Incr
 Path
--------------------------------------------------------------------------
------
  clock CLK (rise edge)                                           0.00
 0.00
  clock network delay (propagated)                                0.00
 0.00
  input external delay                                            0.20
 0.20 f
```

Feedback

```
  IN (in)                                          0.00       0.00
 0.20 f
  IN (net)                        1      1.00
  U1/A (ND2)                                       0.00       0.00
 0.20 f
  U1/Z (ND2)                                       0.17       0.67 &
 0.87 r
  n1 (net)                        1      1.23
  ff1/D (FD1)                                      0.17       0.00 &
 0.87 r
  data arrival time
 0.87

  clock CLK' (rise edge)                                      0.60
 0.60
  clock source latency                                        0.00
 0.60
  CLK (in)                                         0.00       0.00
 0.60 f
  CLK (net)                       3      3.00
  U2/A (IV)                                        0.00       0.00
 0.60 f
  U2/Z (IV)                                        0.14       0.52
 1.13 r
  n2 (net)                        1      1.00
  ff1/CP (FD1)                                     0.14       0.00
 1.13 r
  library setup time                                         -0.80
 0.33
  data required time
 0.33
--------------------------------------------------------------------------
------
  data required time
 0.33
  data arrival time
 -0.87
--------------------------------------------------------------------------
------
  slack (VIOLATED)
 -0.54


The dominant exceptions are:
         None


Stage 0 with difference :
  Point                       Fanout     Cap       Trans       Incr
 Path

 ---------------------------------------------------------------------------
-----
```

Feedback

```
  U1/Z (ND2)                                          0.17       0.67 &
 0.87 r
  U1/Z (ND2)                                          0.17       0.67 &
 0.87 r
  ff1/D (FD1)                                         0.18       0.01 &
 0.88 r
  ff1/D (FD1)                                         0.17       0.00 &
 0.87 r

 ------------------------------------------------------------------------
-----
  Path    Aggressor                  Screen                   Switching
 Bump
          Net                                                (ratio of
 VDD)
  -----  ----------------------     ----------------
 ---------------
   0     n5                         active                    0.03
   1     n5                         screened_by_macro_model
                                                              0.00
------------------------------------------------------------------------
------
```

See Also

- create_ilm
- extract_model
- hier_modeling_version

---

multi_core_allow_overthreading

Controls overthreading with more than one thread per CPU core.

Data Types

Boolean

Default true

Description

Given a maximum limit on the CPU cores that the current PrimeTime process can use, the count of simultaneously active threads can exceed that limit. This is determined differently for different threaded algorithms and commands. The degree of overthreading is set up with the guarantee that most of the time the cores usage limit that you set is honored. However, it is possible that for very short intervals, the usage might exceed that limit.

Feedback

If this is absolutely undesirable, set the *multi_core_allow_overthreading* variable to *false* to guarantee that the process cores utilization never exceeds the user maximum cores limit. This would result in reduced multicore analysis performance.

Note that modifying this variable only affects algorithms invoked within Tcl commands launched after you set the variable.

See Also

- set_host_options

---

multi_scenario_enable_analysis

Enables Distributed Multi-Scenario Analysis (DMSA) mode.

Data Types

Boolean

Default false

Description

Set this variable to *true* to enable the Distributed Multi-Scenario Analysis (DMSA) flow in the PrimeTime tool.

Verifying a chip design generally requires multiple PrimeTime runs to check correct operation under different operating conditions and different operating modes. A specific combination of operating conditions and operating modes for a given chip design is called a scenario.

The PrimeTime tool can analyze several scenarios in parallel using DMSA. Instead of analyzing each scenario in sequence, DMSA uses a manager PrimeTime process that sets up and controls multiple worker processes to execute the timing analysis for each scenario. The processing of scenarios can be distributed onto different hosts running in parallel, thus reducing the overall turnaround time.

After you enable DMSA mode, you cannot disable it for the rest of the PrimeTime session.

The variable can only be set to *true* if these conditions are met:

- No design has been loaded into PrimeTime
- No other distributed mode has been activated
- The GUI has not been started

Changing the *multi_scenario_enable_analysis* variable to *true* also sets the variable *sh_host_mode* to *manager*.

Feedback

See Also

- sh_host_mode
- create_scenario
- current_session
- start_hosts

---

multi_scenario_fault_handling

Controls how fatal errors are handled in remote processes.

Data Types

string

Default ignore

Description

Controls how fatal errors are handled in remote processes. The following values are allowed:

- *exit* - If the manager detects fatal errors in the remote processes, it shuts down the entire analysis after all the executing tasks have been processed. Before exiting, the manager reports which scenarios have caused a fatal error.
- *ignore* (default) - If the manager detects fatal errors in the remote processes, it removes the scenarios that caused a fatal error from the current session when all the executing tasks have been processed. It then reruns the command encountering the fatal error on the remaining scenarios in command focus. It repeats this process until the command either succeeds or the command focus is depleted of all scenarios. If there are no scenarios left in command focus, the command does not run again. If all the scenarios in the session are removed, the session is removed. If the resources required to sustain a session are no longer available, the current session is removed. For the resource requirements needed to sustain a session, see the *current_session* man page.

See Also

- current_scenario
- current_session

Feedback

multi_scenario_license_mode

Specifies the DMSA licensing mode, either scenario-based or core-based.

Data Types

string

Default scenario

Description

Set this variable at the DMSA manager to either "*scenario*" (the default) or "*core*" to specify either scenario-based or core-based licensing:

- In scenario-based licensing, one license is required for each running scenario, without considering the number of cores being used.
- In core-based licensing, one license is required for running each 32 cores, without considering the number of scenarios being run.

Here are the specific actions carried out in each licensing mode:

- *scenario* (default) - For every license feature required by the design in a scenario, a license key for each feature is checked out from the manager by each concurrently running scenario in order to perform the analysis.
- *core* - When you set the variable to *core*, the manager attempts to check out one PrimeTime-ADV feature license. If the license cannot be checked out, the variable is automatically set back to *scenario*. If the license can be checked out, the variable remains set to *core* and the manager allows up to 32 cores of concurrent worker execution without additional licenses. For example, 4 workers configured using *set_host_options -max_cores 8* can all use PrimeTime-ADV licensed capabilities.

  For every additional 32 cores of concurrent execution (or part thereof), an additional PrimeTime-ADV feature license is checked out by the manager. For example, 5 workers configured using *set_host_options -max_cores 8* would require the manager to check out a total of two PrimeTime-ADV feature licenses. If the manager cannot acquire additional PrimeTime-ADV feature licenses, the workers requiring the additional licenses are queued for later execution.

  After the variable is set to *core*, one PrimeTime-ADV license must remain checked out at the manager at all times. Attempting to remove all PrimeTime-ADV feature licenses removes all but one license key.

HyperGrid distributed analysis always uses the *core* licensing mode; it is not affected by this variable.

Feedback

See Also

- current_scenario
- current_session
- set_host_options

---

multi_scenario_merged_error_limit

Defines the maximum number of errors or warnings of a particular type to be considered for merging in the merged error log on a per task basis.

Data Types

integer

Default 100

Description

In multi-scenario analysis, errors, warnings, and informational messages are produced by both the manager and worker processes. For the data produced by the workers, full (compressed) error, warning, and informational messages are stored in the file specified by the *multi_scenario_merged_error_log* variable. To avoid excessive memory usage and network traffic, the number of errors on a per type basis is limited to the number set using the *multi_scenario_merged_error_limit* variable. For example, if the *multi_scenario_merged_error_limit* variable is set to 10, a maximum of 10 errors of type CMD-003 is written to the log specified using the *multi_scenario_merged_error_log* variable. The default value of the *multi_scenario_merged_error_limit* variable is 100.

See Also

- multi_scenario_merged_error_log

---

multi_scenario_merged_error_log

Specifies a file location where full (compressed) error, warning, and informational messages are stored for data produced by the workers.

Data Types

string

Default none

Feedback

Description

In multi-scenario analysis, errors, warnings, and informational messages are produced by both the manager and worker processes. For the data produced by the workers, full (compressed) error, warning, and informational messages are stored in the file specified by the *multi_scenario_merged_error_log* variable. All errors, warnings, and informational messages are displayed as in regular PrimeTime, with the addition of the name of the scenario where the message occurred. If all scenarios produce the same message, only one message is stored with all used instead of listing all scenario names. This describes the term compressed used above. Along with the multi-scenario merged error log, worker errors are displayed in full at the manager while worker warnings are summarized at the manager (worker informational messages are not displayed at the manager). At each worker, the errors and warnings are written in full to the worker log file.

See Also

- multi_scenario_merged_error_limit

---

multi_scenario_message_verbosity_level

Specifies the verbosity level of messages printed at the manager during worker processing.

Data Types

string

Default default

Description

During worker processing, the manager reports back several different types of messages. You can use the *multi_scenario_message_verbosity_level* variable to control the types of messages reported at the manager.

You can set this variable to one of these values:

- *low* - The manager prints messages for errors, fatal errors, and task failures.
- *default* - The manager prints messages for licensing, errors, warnings, fatal errors, task status, and task failures.

See Also

- multi_scenario_merged_error_log

Feedback

multi_scenario_working_directory

Specifies the root working directory for all multi-scenario analysis data, including log files.

Data Types

string

Default (current working directory)

Description

In multi-scenario analysis, the working directory can be explicitly specified using the *multi_scenario_working_directory* variable. This defines the root working directory for all multi-scenario analysis data, including log files. The working directory must be visible to the manager and all the workers. You must have write permissions in the specified directory. If no value is specified for the *multi_scenario_working_directory* variable, the working directory defaults to the directory from which the analysis was invoked.

See Also

- multi_scenario_working_directory

multi_user_access_group

Name of UNIX group used to create the output files written by the multi-user server session.

Data Types

string

Default ""

Description

This variable specifies the name of the UNIX group used to create the output files written by the multi-user server session.

In a multi-user session, client sessions run as subprocesses of the server session. The files written by these clients are written under the server user's ownership, but they will have read/write permissions enabled for all members of this group, ensuring that the client users can access and delete their files as needed.

This variable specifies the default name of the UNIX group. If the *-access_group* option of the *start_multi_user_server* command is used, it takes precedence. The UNIX group name is required to start the multi-user server.

Feedback

If this variable is used to specify the group name, it must be set before the server is started, and it cannot be changed after starting the session.

See Also

- start_multi_user_server
- report_multi_user_server

---

multi_user_logging_level

Specifies the verbosity of logging during multi-user analysis.

Data Types

string

Default low

Description

This variable controls the verbosity of the output log written out by the multi-user server session.

A multi-user server session is started by running the *start_multi_user_server* command. If the *-log* option of the *start_multi_user_server* command is used, then the log file is written to the specified path. Otherwise, the log file is written to a ${server_name}_multi_user.log file in the server's current directory.

To specify the verbosity of data written in the log files, set this variable to one of these values:

- *none*

  This value disables multi-user logging. If the *-log* option is specified, it is ignored.

- *low*

  The session log contains information about the server and client configurations, and timestamps for client connections and disconnections.

- *high*

  The session log contains all the information described above. In addition, it includes all commands executed by the server and client sessions, along with the timestamps of command execution.

The *multi_user_logging_level* variable must be set before starting the multi-user server session.

Feedback

See Also

- start_multi_user_server
- report_multi_user_server

---

mv_allow_pg_pin_reconnection

Enables reconnection of a supply net to a PG pin.

Data Types

Boolean

Default false

Description

Set this variable to *true* to enable reconnection of a supply net to a PG pin that already has a connection.

This variable allows reconnections on only PG pins. Reconnection of a supply net to a port is not allowed even with the variable is set to *true*.

See Also

- connect_supply_net

---

mv_input_enforce_simple_names

Enforces usage of simple names for restricted commands as per the IEEE 1801 Unified Power Format (UPF) standard.

Data Types

Boolean

Default false

Description

This variable controls whether hierarchical names are accepted when you specify an argument that requires simple names, according to the UPF standard.

By default, the tool accepts hierarchical names even if the UPF standard requires them to be simple.

If you set this variable to *true*, the restricted command causes the tool to error out when you use hierarchical names. Note that this variable does not affect the *create_supply_net* and *create_supply_set* commands, which always require simple names.

Feedback

See Also

- connect_supply_net
- create_power_domain
- create_power_switch
- create_supply_port

---

mv_upf_allow_negative_voltage

Causes the tool to check for negative power and ground supply voltages set for logic pins of macros and interface timing models, and disables timing analysis for these pins.

Data Types

boolean

Default false

Description

You can set this variable to the following values:

- *false* (the default) -- The tool does not perform any check for negative power or ground supply voltages associated with logic pins.
- *true* -- The tool checks for negative power and ground supply voltages for the logic pins of macros and interface timing model cells. A negative supply voltage can be specified in the library cell definition or UPF supply net. If it detects a negative voltage, it applies the *set_disable_timing* command to that pin, which disables all timing paths through the pin. The *set_disable_timing* command is written by the *write_script* and *write_sdc* commands, and appears in the HyperScale model for the block.

See Also

- report_disable_timing
- set_disable_timing

---

mw_design_library

Specifies the name of the design library for the *read_milkyway* command.

Data Types

string

Feedback

Default "" (empty)

Description

This variable specifies the name of the design library for the *read_milkyway* command. If you run the *read_milkyway* command without specifying the design library name, the command reads this variable to get the library name. If you run the *read_milkyway* command with the design library name, this variable is set to that name.

See Also

- read_milkyway

---

mw_logic0_net

Specifies the name of constant zero net for the *read_milkyway* command.

Data Types

string

Default VSS

Description

This variable specifies the name of the net is the logic low net for the *read_milkyway* command. When the *read_milkyway* command is reading a design and sees a net by this name, the command treats the net as if it were tied to logic 0.

See Also

- read_milkyway
- mw_logic1_net

---

mw_logic1_net

Specifies the name of the constant one net for the *read_milkyway* command.

Data Types

string

Default VDD

Feedback

Description

This variable specifies the name of the net is the logic high net for the *read_milkyway* command. When the *read_milkyway* command is reading a design and sees a net by this name, the command treats the net as if it were tied to logic 1.

See Also

- read_milkyway
- mw_logic0_net

n

non_auto_fixable_violations

This man page describes *non_auto_fixable* hierarchical boundary check violations shown by the *report_constraint* command in the HyperScale flow.

Description

The following violations are *non_auto_fixable*:

```
boundary_ideal_network
clock_attributes
clock_mapping
clock_relations
clock_uncertainty
env_variables
global_timing_derate
library_mapping
operating_conditions
```

The *non_auto_fixable* violations invalidate the HyperScale flow. To use HyperScale, you must fix *non_auto_fixable* violations in the beginning or middle of the design flow.

What Next

To resolve *non_auto_fixable* violations,

- Focus on aligning block-level constraints to top-level constraints, such as environment and clocks.
- If necessary, improve the specification of top-level constraints for compatibility with hierarchical analysis.

Examples

The following verbose report shows *non_auto_fixable* HyperScale boundary violations.

Feedback

```
pt_shell> report_constraint -all_violators -verbose
 -boundary_check_include {non_auto_fixable}

****************************************
Report : constraint
        -all_violators
        -verbose
Design : top
 ...
****************************************

HyperScale constraints report

   Constraint: clock_attributes

   Instance      Attribute name    Block level (clock)     Top level
 (clock)

 ------------------------------------------------------------------------
-------
   core2         is_propagated     false (CLK)             true (top_CLK)
   core3         sources           {core3/clk1} (CLK)      {core3/clk2}
 (top_CLK)

   Constraint: clock_relations

   Instance      Clock relation    Block level             Top level

 ------------------------------------------------------------------------
-------
   core2         async             {int_CLK} <-> {top_CLK}
 {core2/int_CLK} <-> {}
   core2         async             {vCLK} <-> {top_CLK}    {top_vCLK} <->
 {}
```

See Also

- report_constraint
- auto_fixable_violations
- boundary_ideal_network_violations
- clock_attributes_violations
- clock_mapping_violations
- clock_relations_violations
- clock_uncertainty_violations
- env_variables_violations

Feedback

- global_timing_derate_violations
- library_mapping_violations
- operating_conditions_violations

O

old_port_voltage_assignment

Reverts to a port's previous voltage assignment.

Data Types

Boolean

Default false

Description

By default, the tool infers a port voltage from its load or driver pins. If you set this variable is set to *true*, the tool reverts to the previous voltage assignment mode for ports and no longer infers port voltages from load or driver pins.

operating_conditions_violations

This man page describes *operating_conditions* hierarchical boundary check violations shown by the *report_constraint* command in the HyperScale flow.

Description

An *operating_conditions* violation indicates differences in the operating condition related settings.

Timing constraint commands referring to libraries include (and not limited to) *set_operating_condition*, *set_temperature set_voltage*.

Note: Do not save and check the operating condition settings applied to specific design objects such as pins, cells. This check only covers design-wide operating conditions, or settings on block boundary ports.

What Next

This is a *non_auto_fixable* violation. You must resolve the difference in operating condition settings. Consistent usage and settings of operating condition is required to ensure consistent results from the timing analysis. The difference can be resolved by the

Feedback

commands listed previously. If the violation is a side effect of library mismatch, resolving library_mapping violation is required.

Examples

The following verbose report shows mismatches in the operating condition between the top and block.

```
pt_shell> report_constraint -boundary_check_include
 {operating_conditions} \\
          -all_violators -verbose

****************************************
Report : constraint
        -verbose
 ...
****************************************

HyperScale constraints report

   Constraint: operating_conditions

   Instance   Setting           Block         Top
   ------------------------------------------------------------------
   blk        Max Library       /my/libraries/pt_lib.db:pt_lib

 /remote/libraries/pt_lib_max.db:pt_lib_max
   blk        Max Condition     WCCOM         nom_pvt
   blk        Analysis Type     bc_wc         Single
   blk        Min Temperature   125           70
```

See Also

- report_constraint
- set_operating_conditions
- set_temperature
- set_voltage
- non_auto_fixable_violations

p

parasitic_corner_name

Specifies the parasitic corner to be loaded by the *read_parasitics* command.

Feedback

Data Types

string

Default ""

Description

This variable specifies the parasitic corner to be loaded by the *read_parasitics* command. Setting this variable is optional. The tool behavior in relation to parasitic corners is as follows:

- If you set the variable to a specified corner:
  1. If the tool reads SPEF files with multiple corners, the *read_parasitics* command annotates the nets with only the data of the specified corner.
  2. If the tool reads GPD files, with a single or multiple corners, the *read_parasitics* command annotates the nets with the specified corner, or all the corners if *parasitic_explorer_enable_analysis* is set to true. In the case of multiple annotated corners, commands other than *read_parasitics* will only affect the specified corner. The log of *read_parasitics* will show the list of annotated corners with the specified one appearing first (the order of the following ones, if any, is not relevant).
  3. If the specified corner does not exist in the SPEF files, the tool issues an error.
- If the variable is not set:
  1. If the tool reads SPEF files with multiple corners, the tool issues an error.
  2. If, the tool reads SPEF files with a single corner, it annotates the associated parasitic data.
  3. If the tool reads GPD files, with a single or multiple corners, the *read_parasitics* command annotates the nets with the first corner by default, or all the corners if *parasitic_explorer_enable_analysis* is set to true. In the case of multiple annotated corners, commands other than *read_parasitics* will only affect the default corner. The log of *read_parasitics* will show the list of annotated corners with the default one appearing first (the order of the following ones, if any, is not relevant).
- You can set this variable at any time before reading the parasitic data.
- After reading parasitic data, you cannot create a new or destroy an existing parasitic corner. This means that you can set the variable only to an existing parasitic corner.

See Also

- read_parasitics
- parasitic_explorer_enable_analysis

Feedback

parasitic_explorer_enable_analysis

Enables the Parasitic Explorer option, which supports analysis of GPD parasitic data.

Data Types

Boolean

Default false

Description

The Parasitic Explorer lets you query parasitic resistors and capacitors that have been back-annotated on the design by the *read_parasitics* command in Galaxy Parasitic Database (GPD) format and to report the properties of parasitic data stored in GPD directories. To enable the Parasitic Explorer, set this variable to *true*. A PrimeTime-ADV-PLUS license is required.

The Parasitic Explorer feature lets you do the following:

- Create collections of parasitic components with the *get_resistors*, *get_ground_capacitors*, and *get_coupling_capacitors* commands
- Query parasitic element attributes such as resistance, capacitance, subnodename, layer name, layer number, and physical location
- Query the layers, parasitic corners, and other information in GPD directories with the *report_gpd_properties* command
- Control the reading and annotation of parasitic data in the PrimeTime tool with the *set_gpd_config* command

See Also

- read_parasitics
- get_ground_capacitors
- get_coupling_capacitors
- get_gpd_corners
- get_gpd_layers
- get_resistors
- report_gpd_properties
- report_gpd_config

Feedback

- reset_gpd_config
- set_gpd_config

parasitics_cap_warning_threshold

Specifies a capacitance threshold beyond which a warning message is issued during the reading of a parasitics file.

Data Types

float

Default 0.0

Description

If this variable is set with a value greater than 0.0, the *read_parasitics* command issues a PARA-014 warning if it finds in the parasitics file a capacitance value, in picofarads, greater than this threshold. The default is 0.0, in which case no checking is done.

Use this variable to detect large, unexpected capacitance values written to parasitics files by other applications. The capacitor is still used, but you can quickly find it in the parasitics file.

To define an analogous resistance threshold, set the *parasitics_res_warning_threshold* variable.

See Also

- read_parasitics
- parasitics_res_warning_threshold

parasitics_enable_multiple_physical_pins

Enables multiple physical pins support in the GPD and SPEF parasitics formats.

Data Types

Boolean

Default false

Description

When this variable is set to *true*, the SPEF and GPD parasitics readers support detailed parasitics with multiple physical pins information.

Feedback

In some cases, a single logical port might be associated with multiple physical shapes. For example, for a port-connected net, the wire might reach the block boundary at multiple physical locations. Such ports are described as having multiple physical pins in the detailed parasitics file.

In the detailed parasitics information, equivalent physical pins use a special SNPS_EEQ_ ("electrically equivalent") prefix in their port (*P) definitions.

When this feature is enabled and such parasitics are read in, the tool recognizes the equivalent physical pins and maps them to the same logical netlist port. When multiple parasitics files are read in and stitched together, the equivalent physical pins are connected correctly.

This feature is not supported in the incremental SPEF reading flow with the *read_parasitics -eco* command.

The default is *false*.

See Also

- read_parasitics

---

parasitics_enable_tail_annotation

Enables reading of tail annotation data from parasitic files to support via variation analysis.

Data Types

Boolean

Default false

Description

Setting this variable to *true* enables reading of tail annotation information from parasitic data files. This includes information related to metal layers, resistor parameters, via area, and so on needed for via variation analysis.

See Also

- read_ivm
- timing_enable_via_variation

---

parasitics_log_file

Specifies the location of the output of parasitic commands when run in a background process.

Feedback

Data Types

string

Default parasitics_command.log

Description

The tool might launch a side process to perform parasitic operations in parallel with portions of the main run. This variable determines where the output data of the side process should go. By default, it goes to a file named parasitics_command.log in the current working directory. To modify the file name, set this variable.

The *parasitics_log_file* variable works only when multiple CPU cores are used. If only one CPU core is specified or available, this variable setting has no effect and the tool writes out parasitic information to the default log file.

If the *parasitics_log_file* variable is set to empty string "", then the output parasitic info is displayed in the default log file.

See Also

- read_parasitics

---

parasitics_rejection_net_size

Defines a threshold number of nodes in an annotated parasitic network beyond which the detailed network is automatically replaced by a lumped capacitance.

Data Types

integer

Default 20000

Description

Defines a threshold number of nodes in an annotated parasitic network beyond which the detailed network is automatically replaced by a lumped capacitance. The default is *20000*.

This variable is one of a pair of variables, *parasitics_warning_net_size* and *parasitics_rejection_net_size*, that help you prevent unacceptable or unexpected runtimes caused by large numbers of nodes in an annotated parasitic network. If the *read_parasitics* command finds a number of nodes that exceeds the value of the *parasitics_warning_net_size* variable (default 10000), the PARA-003 message warns you that extended runtime could occur. If the *read_parasitics* command finds a number of nodes that exceeds the value of the *parasitics_rejection_net_size* variable (default 20000), the network is rejected and automatically replaced by a lumped capacitance, and you receive a PARA-004 message warning.

Feedback

The value of the *parasitics_warning_net_size* variable is ignored if it is greater than or equal to the value of the *parasitics_rejection_net_size* variable.

See Also

- read_parasitics
- parasitics_warning_net_size

parasitics_res_warning_threshold

Specifies a resistance threshold beyond which a warning message is issued during the reading of a parasitics file.

Data Types

float

Default 0.0

Description

When this variable is set with a value greater than 0.0, the *read_parasitics* command issues a PARA-014 warning if it finds in the parasitics file a resistance value, in ohms, greater than this threshold. The default is 0.0, in which case no checking is done.

Use this variable to detect large, unexpected resistance values written to parasitics files by other applications. The resistor is still used, but you can quickly find it in the parasitics file.

To define an analogous capacitance threshold, set the *parasitics_cap_warning_threshold* variable.

See Also

- read_parasitics
- parasitics_cap_warning_threshold

parasitics_warning_net_size

Defines a threshold number of nodes in an annotated parasitic network beyond which a message is issued that warns of a potential extended runtime.

Data Types

int

Default 10000

Feedback

Description

Defines a threshold number of nodes in an annotated parasitic network beyond which a message is issued that warns of a potential extended runtime. The default is 10000.

This variable is one of a pair of variables, *parasitics_warning_net_size* and *parasitics_rejection_net_size*, that help you prevent unacceptable or unexpected runtimes caused by large numbers of nodes in an annotated parasitic network. If the *read_parasitics* command finds a number of nodes that exceeds the value of the *parasitics_warning_net_size* variable (default 10000), you receive a PARA-003 message warning you that extended runtime could occur. If the *read_parasitics* command finds a number of nodes that exceeds the value of the *parasitics_rejection_net_size* variable (default 20000), the network is rejected and automatically replaced by a lumped capacitance. You receive a PARA-004 message warning you of that action.

The value of the *parasitics_warning_net_size* variable is ignored if it is greater than or equal to the value of the *parasitics_rejection_net_size* variable.

See Also

- read_parasitics
- parasitics_rejection_net_size

---

partition

Specifies the partition name in context of the current task in execution.

Data Types

string

Default (none)

See Also

- remote_execute

---

pba_derate_only_mode

Specifies that only the path derates are reevaluated during path-based analysis.

Data Types

Boolean

Default false

Feedback

Description

This variable applies to the path-based analysis performed during the *get_timing_paths* and *report_timing* commands when you specify the *-pba_mode* option. This variable controls whether regular path-based analysis (path-specific slew propagation) effects are performed during a path-based analysis in addition to adjusting the deratings on the path according to the path-based conditions.

If the variable is set to *false* (the default), the tool performs both regular path-based analysis and derating adjustment during a path-based analysis. This option removes the maximum amount of pessimism from the design, but the runtime can be long.

If you set the variable to *true*, the tool only adjusts the derating according to the path-based conditions during the path-based analysis.

This variable is useful in the advanced OCV flow, parametric OCV flow, and simultaneous multivoltage analysis (SMVA) used with a derating method and is recommended in these flows to achieve the fastest runtime.

Note that advanced or parametric OCV path-based analysis is applied only if user-specified advanced or parametric OCV information exists.

Also note that in parametric OCV analysis, the random variation pessimism of graph-based analysis over path-based analysis is zero. Therefore, running path-based analysis with this variable set to *true* in parametric OCV only removes pessimism resulting from systematic variation, that is, distance-based derating.

See Also

- get_timing_paths
- read_aocvm
- report_timing

---

pba_enable_mis_delay_ocv_pessimism_reduction

Reduces the pessimism during path-based window alignment and overlap computation due to clock on-chip variation (OCV) for multi-input switching (MIS) analysis.

Data Types

Boolean

Default false

Feedback

Description

This variable controls whether PrimeTime reduces the pessimism due to clock on-chip variation (OCV) when collecting the arrival windows of the input pins for overlap computation during path-based MIS analysis for deriving the actual coefficient factor. You can set this variable to one of these values:

- *false* (the default) - Does not reduce the pessimism due to clock on-chip-variation (OCV) when collecting the arrival windows of the input pins in path-based MIS analysis.
- *true* - Reduces the pessimism due to clock on-chip-variation (OCV) when collecting the arrival windows of the input pins in path-based MIS analysis. This pessimism reduction is computationally intensive and should be used only when the clock path is long, and the on-chip variation is large.

To use this feature, you must enable clock reconvergence pessimism removal (CRPR) by setting the *timing_remove_clock_reconvergence_pessimism* variable to *true*.

See Also

- get_timing_paths
- report_timing
- timing_remove_clock_reconvergence_pessimism

---

pba_enable_path_based_physical_exclusivity

Specifies whether a path-based or stage-based physical exclusivity crosstalk computation is used during path-based analysis of paths involving physically exclusive clocks.

Data Types

integer

Default false

Description

If this variable is set to *false* (the default), each delay calculation stage is evaluated independently of the other stages in the path. For instance, consider two clocks that are physically exclusive, CLK1 and CLK2. For one stage in the path, an aggressor clocked by CLK1 might result in the worst delta delay. For the next stage, an aggressor clocked by CLK2 might result in the worst delta delay. In a stage-based approach, these deltas are both used for the corresponding stages in the path. This approach is runtime efficient but can possibly result in some pessimism.

If you set this variable to *true*, the path is recalculated multiple times to consider each possible victim and aggressor combination across the physically-exclusive clocks. In this

Feedback

case, aggressors from CLK1 and CLK2 cannot simultaneously attack different stages across the path. This removes the pessimism of the stage-based approach, but at the cost of additional runtime.

It is recommended to use the default setting of *false* for most analysis, but set the variable to *true* for the final signoff run if additional pessimism removal is needed during path-based analysis.

See Also

- get_timing_paths
- report_timing

---

pba_enable_xtalk_delay_ocv_pessimism_reduction

Reduces the pessimism during path-based crosstalk delay analysis due to clock on-chip variation (OCV).

Data Types

Boolean

Default false

Description

This variable controls whether PrimeTime SI reduces the pessimism due to clock on-chip variation (OCV) on the victim and aggressor arrivals during path-based crosstalk delay analysis. You can set this variable to one of these values:

- *false* (the default) - Does not reduce the pessimism due to clock on-chip-variation (OCV) on the victim and aggressor arrivals.
- *true* - Reduces the pessimism due to clock on-chip-variation (OCV) on the victim and aggressor arrivals. This pessimism reduction is computationally intensive and should be used only when the clock path is long, and the on-chip variation is large.

To use this feature, you must enable clock reconvergence pessimism removal (CRPR) by setting the *timing_remove_clock_reconvergence_pessimism* variable to *true*.

See Also

- get_timing_paths
- report_timing
- timing_remove_clock_reconvergence_pessimism

Feedback

pba_exhaustive_any_slack_lesser_than

Any pba_exhaustive path with slack less than the value specified by the variable may be returned by exhaustive PBA path gathering.

Data Types

string (float, or string "disabled")

Default disabled

Description

If this variable is set to a float value, exhaustive PBA path gathering commands may return any violating path(s) satisfying the reporting criteria with slack less than the value of the variable, not necessarily the most violating one(s).

The list of commands affected is

```
report_timing -pba_mode exhaustive ...
get_timing_paths -pba_mode exhaustive ...
report_constraint -pba_mode exhaustive ...
report_global_timing -pba_mode exhaustive ...
report_qor -pba_mode exhaustive ...
```

See Also

- get_timing_paths
- report_timing

pba_exhaustive_endpoint_path_limit

Defines a limit for exhaustive path-based analysis.

Data Types

string (Range: 1 to large number, or string "infinity")

Default infinity

Description

This variable applies to the exhaustive path-based analysis performed by the following commands:

```
report_timing -pba_mode exhaustive ...
get_timing_paths -pba_mode exhaustive ...
report_constraint -pba_mode exhaustive ...
report_global_timing -pba_mode exhaustive ...
report_qor -pba_mode exhaustive ...
```

Feedback

Exhaustive analysis is computationally intensive, and it is intended to be used only when the design is approaching signoff.

In certain topologically complex designs, you get full path coverage with faster runtime when you leave the variable set to the default, the string "infinity". This selects the "all paths" mode.

Alternatively, for less complex designs, you can get faster runtime by setting a positive integer value for this variable, such as 25000. This selects the "path enumeration" mode. In this mode, to specify whether graph-based analysis paths are included in the results from exhaustive path-based analysis after the endpoint path limit is reached, set the *pba_path_recalculation_limit_compatibility* variable.

For more information, see SolvNet article 2694662, "All-Paths Exhaustive Path-Based Analysis Mode (pba_exhaustive_endpoint_path_limit = infinity)."

There are several other measures that improve the runtime of the analysis.

- Uniquify paths through parallel arcs
- Use conservative values for the *-nworst* and *-max_paths* options
- Set a realistic threshold for the *-slack_lesser_than* option

When an advanced on-chip variation (OCV) analysis is being performed, there are several additional variable settings that improve the runtime of the analysis.

- Enable CRPR
- Enable graph-based advanced OCV
- Enable path-based advanced OCV only mode

See Also

- get_timing_paths
- report_timing
- pba_derate_only_mode
- pba_path_recalculation_limit_compatibility
- timing_aocvm_enable_analysis
- timing_remove_clock_reconvergence_pessimism
- timing_report_use_worst_parallel_cell_arc

Feedback

pba_exhaustive_endpoint_time_limit

Specifies a computation time limit, in milliseconds, per endpoint for exhaustive path-based analysis.

Data Types

string (Range: 1 to large number, or string "infinity")

Default infinity

Description

This variable specifies a computation time limit, in milliseconds, per endpoint for exhaustive path-based analysis, which is enabled when you invoke exhaustive path-based analysis with one of the following commands:

```
report_timing -pba_mode exhaustive ...
get_timing_paths -pba_mode exhaustive ...
report_constraint -pba_mode exhaustive ...
report_global_timing -pba_mode exhaustive ...
report_qor -pba_mode exhaustive ...
```

These commands perform exhaustive path-based analysis on a set of endpoints. The time spent analyzing an endpoint depends on the topological complexity of the paths leading to that endpoint.

If this variable is set to a number, each endpoint is subject to the specified time limit. If time limit is exceeded for any endpoint, analysis of that endpoint analysis ends with a UITE-680 warning message, and analysis continues with the next endpoint. By default, the variable is set to the string "infinity", which means no time limit.

All the endpoints for which UITE-680 was issued due to exceeding the time limit are added as collection elements to the design attribute named *pba_exhaustive_endpoint_limit_exceeded_pins*.

Due to the usage of heavily threaded algorithms, the number of UITE-680 messages issued and the contents of the *pba_exhaustive_endpoint_limit_exceeded_pins* design attribute can differ from run to run, even for the same design.

See Also

- get_timing_paths
- report_timing
- pba_exhaustive_endpoint_path_limit

Feedback

pba_exhaustive_memory_limit

Limits the amount of memory exhaustive path-based analysis is allowed to use.

Data Types

string

Default high

Description

This variable controls the amount of memory which can be used during path-based recalculation for reporting commands with the *-pba_mode exhaustive* option.

- *high* (the default) - High memory use which can result in better runtime.
- *medium* - Reduced memory usage which can result in longer runtime than the high setting.

pba_exhaustive_slack_tolerance_percentage

Defines a tolerance percentage such that the slack of the most violating path returned for a given endpoint is within the specified percentage of the real worst path slack to that endpoint.

Data Types

float (Range: 0 to 100)

Default 0

Description

This variable applies to the exhaustive path-based analysis performed by the following commands:

```
report_timing -pba_mode exhaustive ...
get_timing_paths -pba_mode exhaustive ...
report_constraint -pba_mode exhaustive ...
report_global_timing -pba_mode exhaustive ...
report_qor -pba_mode exhaustive ...
```

The slack of the most violating path returned for a given endpoint is within the specified percentage of the real worst path slack to that endpoint. For example, endpoint $A has paths with pba exhaustive slack -11ps and -10ps. With pba_exhaustive_slack_tolerance_percentage set to 10, either the -11ps or the -10ps path may be returned by *report_timing -pba_mode exhaustive -to $A -nworst 1*.

Feedback

See Also

- get_timing_paths
- report_timing

---

pba_path_mode_enumerate_by_gba_slack

Specifies how paths are enumerated when you use the *-pba_mode path* option.

Data Types

Boolean

Default true

Description

This variable applies to path-based analysis when you use the *report_timing* or *get_timing_paths* command with the *-pba_mode path* and *max_paths* options.

- *true* (the default) - Enumerates paths based on the graph-based analysis slack.

  For example, if you use the *report_timing -pba_mode path -max_paths 5 -slack_lesser_than 0* command, the tool recalculates the worst graph-based analysis path at the 5 worst violating graph-based analysis endpoints.

- *false* - Enumerates paths based on the path-based analysis slack. The tool continues to recalculate paths until the *-max_paths* option criteria is satisfied. This setting leads to a longer runtime but potentially returns a better set of relevant paths.

  For example, if you use the *report_timing -pba_mode path -max_paths 5 -slack_lesser_than 0* command, the tool recalculates the worst graph-based analysis path at the worst graph-based analysis endpoints until it finds 5 violating path-based analysis paths.

See Also

- get_timing_paths
- report_timing

---

pba_path_mode_sort_by_gba_slack

Specifies how paths are sorted when you use the *-pba_mode path* option.

Data Types

Boolean

Feedback

Default false

Description

This variable applies to path-based analysis when you use the *report_timing* or *get_timing_paths* command with the *-pba_mode path* option.

When the variable is set to *false* (the default), the tool sorts and filters paths based on the recalculated slack. For example, if you use the *report_timing -slack_lesser_than 0 -pba_mode path* command, and the worst path has a graph-based slack of -1 and a recalculated slack of 1, this path does not appear in the final report.

When you set the variable to *true*, the tool sorts and filters paths based on the graph-based slack calculated during the *update_timing* command. With this setting, the *-pba_mode path* option does not change the order of the paths generated by the *report_timing* or *get_timing_paths* command.

See Also

- get_timing_paths
- report_timing

---

pba_path_recalculation_limit_compatibility

Controls whether graph-based analysis paths are included in results from path-based analysis recalculation when the path recalculation limit is reached.

Data Types

Boolean

Default true

Description

If this variable is set to *true* (the default), PrimeTime only returns path-based analysis paths from the *get_timing_paths* command if the *-pba_mode* option is not set to *none*. Also, when this variable is *true*, the *report_timing* command with the *-pba_mode* option set to anything other than *none* reports only recalculated paths.

If this variable is set to *false*, PrimeTime can report both recalculated and non-recalculated paths.

This variable has an effect only when the *pba_exhaustive_endpoint_path_limit* variable is set to a numeric value, which selects the "path enumeration" exhaustive path-based analysis mode and specifies the per-endpoint limit. It has no effect when the *pba_exhaustive_endpoint_path_limit* variable is set to the string "infinity", which selects the "all paths" mode, the default.

Feedback

See Also

- get_timing_paths
- report_timing
- pba_exhaustive_endpoint_path_limit

---

pba_recalculate_full_path

Allows path-based analysis to recalculate full clock paths, borrowing path segments, and data check reference paths.

Data Types

integer

Default false

Description

When this variable is set to *true*, PrimeTime allows all path-based analysis commands (*report_timing* and *get_timing_paths*) to recalculate full clock paths, borrowing paths, and data check reference paths in addition to the data paths.

When this variable is set to its default of *false*, PrimeTime blocks recalculation of clock paths, borrowing path portions, and data check reference paths and the original timing is retained. This allows paths obtained with the *-path full_clock*, *-path full_clock_expanded*, and *-trace_latch_borrow* options to be fully reported, while avoiding recalculation on the borrowing, data check references, and clock portions of the path. The reason this might be desirable is that with certain circuit topologies, a conservative path-based recalculation of the clock, data check reference, or borrowing path might not be guaranteed. This can happen when there are multiple clock, data check reference, or borrowing paths which can apply to the path. Only the worst pre-recalculation clock, data check reference, or borrowing path is included for recalculation. This might not be the worst path after recalculation.

In advanced on-chip variation (AOCV) mode, the depth and distance metrics are always recomputed by path recalculation regardless of the value of this variable. In advanced OCV mode, this variable only has an effect when path slew recomputation is enabled when the *pba_derate_only_mode* variable is set to its default of *false*. When this variable is set to *true*, path-specific delay calculation is performed along the clock and data paths. When this variable is set to *false*, path-specific delay calculation is only performed along the data path.

Feedback

See Also

- get_timing_paths
- report_timing
- pba_derate_only_mode

---

pba_usage_context

Specifies whether pba should err on the side of pessimism ("signoff") or optimism ("convergence") in the presence of early termination, e.g. if *pba_exhaustive_endpoint_time_limit* is exceeded.

Data Types

string

Default signoff

Description

This variable specifies whether exhaustive PBA reporting, when it encounters early termination criteria, return potentially pessimistic, GBA-derived paths ("signoff") or potentially optimistic, PBA-derived paths ("convergence").

The early termination criteria covered are:

```
pba_exhaustive_endpoint_path_limit
pba_exhaustive_slack_tolerance_percentage
```

See Also

- get_timing_paths
- report_timing
- pba_exhaustive_endpoint_path_limit
- pba_exhaustive_slack_tolerance_percentage

---

pg_allow_pg_pin_reconnection

Enables reconnection of a supply net to a PG pin in the PG connectivity from netlist.

Data Types

Boolean

Default false

Feedback

Description

Set this variable to *true* to enable reconnection of a supply net to a PG pin that already has a connection in the PG connectivity from netlist.

This variable allows reconnections on only PG pins. Reconnection of a supply net to a port is not allowed even with the variable set to *true*.

See Also

- connect_supply_net
- link_keep_pg_connectivity

---

port_search_in_current_instance

Controls whether the *get_ports* command can get ports of the current instance.

Data Types

Boolean

Default false

Description

If this variable is set to *false* (the default), the *get_ports* command gets ports of the current design. If this variable is set to *true*, the *get_ports* command gets ports of the current instance.

See Also

- get_ports

---

power_activity_file_precedence_over_sca

Enables or disables activity annotation to take precedence over constants (logic constants or case constants) in PrimePower.

Data Types

Boolean

Default true

Feedback

Description

When set to *false*, constants take precedence over annotated activity. This variable works in both average and time-based power analysis mode. This variable applies only to *set_switching_activity* command and not for *set_switching_activity -force* command.

See Also

- set_switching_activity
- set_case_analysis

---

power_activity_precedence_over_force_implied

Enables or disables activity annotation to take precendence over "set_switching_activity -force" propagated nets in PrimePower. The activity type for "set_switching_activity -force" propagated nets is "force_implied". By default, file and SSA annotation have higher precedence than "force_implied". This variable works in both average and time based analysis mode.

Data Types

Boolean

Default true

Description

When set to *false* , "force_implied" take precedence over file and set_switching_activity.

---

power_analysis_mode

Sets the power analysis mode.

Data Types

string

Default averaged

Description

This variable explicitly selects the analysis mode for power calculation. PrimePower provides three different analysis modes: *averaged*, *time_based*. Set this variable before the first power command, otherwise, the default mode is assumed. For a particular

Feedback

analysis mode, you must provide the appropriate activity information. The allowed values are as follows:

- *averaged* (the default): PrimePower calculates power based on toggle-rate and state-probability. Only averaged power results are calculated. In this mode, it can take the VCD file and SAIF file as activity input files. Use the *set_switching_activity* and *set_case_analysis* commands to set the statistical switching activity on top of the default switching activity. For more information, see the *update_power* man page.
- *time_based*: PrimePower calculates power based on the events from VCD. Averaged power results are calculated, along with calculations for peak powers and time-based power waveforms. You must provide the VCD file in this mode. Both gate-level VCD and RTL VCD can be specified for this mode. For more information, see the *update_power* man page.

  The *power_analysis_mode* variable can change in one run. However, if you change the setting, all the activity and power data is removed internally. You must provide activity information before the *update_power* command.

  In addition, you can use the *set_power_analysis_options* to specify the options for power analysis.

See Also

- read_saif
- read_vcd
- report_power
- set_case_analysis
- set_power_analysis_options
- set_switching_activity
- update_power
- power_enable_analysis

---

power_calc_use_ceff_for_internal_power

Specifies whether to use effective C for internal power calculation.

Data Types

Boolean

Default false

Feedback

Description

This variable controls whether to use effective capacitance in the sense of timing as the output net capacitance parameter when looking up internal power tables during the power calculation stage. If the variable is *true*, use effective capacitance; otherwise use the total net capacitance.

---

power_capp_edge_triggered_propagation

Controls to perform event propagation through edge triggered sequential cells in CAPP flow.

Data Types

Boolean

Default true

Description

In CAPP flow, all sequential outputs are expected to be annotated, which doesn't happen always in practice. In such cases, PrimePower supports propagation through sequential elements, however, propagation through edge triggered sequential elements sometimes lead to overestimation of power. This variable controls enables/disables propagation through edge triggered sequential elements in CAPP flow. By default, propagation through edge triggered elements is enabled in PrimePower.

---

power_check_defaults

Defines the default checks for the *check_power* command.

Data Types

list

Default out_of_table_range missing_table missing_function

Description

Defines the default checks performed when the *check_power* command is executed without any options. The same default checks are also performed if the *check_power* command is used with the *-include* or *-exclude* options. The default check list defined by this variable can be overridden by either redefining it before the *check_power* command is executed or using the *-override_defaults* option of the *check_power* command.

If an undefined check is specified while redefining this variable, a warning is issued by the next execution of the *check_power* command.

Feedback

See Also

- check_power

---

power_clock_network_include_clock_gating_network

Indicates that clock gating networks are included in the clock network.

Data Types

Boolean

Default false

Description

This variable affects the predefined *clock_network* and *register* power groups. When the variable is set to *true*, discrete logic structure that functions as clock gating belongs to the *clock_network* power group. Only the typical clock gating logic is considered a qualified clock gating network included in the clock network. The typical clock gating logic starts from the output of a level-sensitive latch driven by the specified clock. The clock gating logic possibly goes through some buffers, and returns to the specified clock network through one of the input pins of an AND or OR gate. The input pin must be a PrimeTime clock check enabled pin, either inferred or manually set. Therefore, the results can be affected by clock gating check related commands or variables. When the clock gating network is included in the clock network, the latch is regarded as part of the *clock_network* power group, but not the *register* power group.

See Also

- create_power_group
- report_power

---

power_clock_network_include_register_clock_pin_power

Indicates whether the register clock pin power is included when reporting *clock_network* power.

Data Types

Boolean

Default true

Feedback

Description

This variable affects the power report for the predefined *clock_network* and *register* power groups. When set to its default of *true*, the internal power of registers caused by the toggling of register clock pin when output pin does not toggle is included as *clock_network* power and excluded from *register* power. When set to *false*, the power is included as *register* power and excluded from *clock_network* power.

See Also

- create_power_group
- report_power

---

power_concurrent_event_analysis_work_dir

Specifies the directory for the generation of all output files from the child processes in concurrent analysis for PrimePower in time-based mode.

Data Types

string

Default work_pp

Description

When specified, all output files as well as power waveforms and worker logs from child processes are generated in the specified directory for distributed analysis in time-based mode.

See Also

- set_concurrent_event_analysis_options
- power_enable_concurrent_event_analysis
- power_enable_distributed_analysis

---

power_default_base_clock

Specifies the default power_base_clock to be be used by PrimePower.

Data Types

String

Default fastest

Feedback

Description

This variable allows PrimePower to use the specified clock as the default power_base_clock. For unconstrained pins/nets, usually the fastest clock in the design is selected as the power_base_clock. This variable can be used to overrirde this default behaviour.

The dominant_clock mode of power_default_base_clock selects the clock in the design with the most number of instances. If there are multiple clocks with same maximum number of instances, then the fastest clock is selected as the power_base_clock.

This should be specified after design contraints (SDC) are read and prior to set_switching_activity/update_power command.

See Also

- set_power_base_clock

---

power_default_static_probability

Specifies the default static probability value.

Data Types

float

Default 0.5

Description

The *power_default_static_probability* variable (along with the *power_default_toggle_rate* and *power_default_toggle_rate_reference_clock* variables) determines the switching activity of unannotated nets that are driven by primary inputs or black box cells.
The *power_default_toggle_rate* variable specifies the default toggle rate value. The *power_default_toggle_rate_reference_clock* variable specifies how the related clock for default toggle rate is determined.

For other nonannotated nets, PrimePower propagates the switching activities of the driving cell inputs based on the cell functionality to derive the switching activity required for power calculations. This mechanism cannot be used for primary inputs and black box outputs. Instead, the following values are used for these type of nets:

- Annotated values are used.
- In some cases, unannotated switching activity values can still be accurately derived. For example, if the net drives a buffer cell and the output of this cell is annotated, your annotated values are used as the default values. Also, if the input is a clock the clock period and waveform are used to derive the switching activity values.

Feedback

- If the static probability is not annotated, the value of the *power_default_static_probability* variable is used for the static probability value.
- If the toggle rate is not annotated by you, whether the static probability is set or not, the following is used for the toggle rate value:

  ```
  dtr * fclk
  ```

  where fclk is the frequency of the related clock, and the dtr is the value of the *power_default_toggle_rate* variable.

The *power_default_toggle_rate_reference_clock* variable determines the related clock. You can set this variable to these settings:

- *fastest* - The related clock is the fastest clock in the design.
- *related* - The related clock depends on the clock domain that the net belongs to.

The value of *power_default_static_probability* should be between 0.0 and 1.0, both inclusive. The value of *power_default_toggle_rate* should be greater or equal to 0.0. Also, if the value of *power_default_static_probability* is 0.0 or 1.0, the value of *power_default_toggle_rate* should be 0.0. If the value of *power_default_toggle_rate* is 0.0, the value of the *power_default_static_probability* should be either 0.0 or 1.0.

The default of *power_default_static_probability* is 0.5 and the default of *power_default_toggle_rate* is 0.1.

See Also

- power_default_toggle_rate
- power_default_toggle_rate_reference_clock
- set_switching_activity

---

power_default_toggle_rate

Specifies the default toggle rate value.

Data Types

float

Default 0.1

Description

The *power_default_toggle*, *power_default_toggle_rate_reference_clock*, and *power_default_static_probability* variables are used to determine the switching activity of non-user-annotated nets that are driven by primary inputs or black-box cells.

Feedback

For other nonannotated nets, PrimePower propagates the switching activities of the driving cell inputs based on the cell functionality to derive the switching activity required for power calculations. This mechanism cannot be used for primary inputs and black-box outputs. Instead the following values are used for these type of nets:

- User-annotated values are used.
- In some cases, unannotated switching activity values may still be accurately derived, for example, if the net drives a buffer cell and the output of this cell is user annotated, then the user annotated values are used as the default values. Also, if the input is a clock then the clock period and waveform are used to derive the switching activity values.
- If the static probability is not annotated then the value of the *power_default_static_probability* variable is used for the static probability value.
- If the toggle rate is not user annotated, no mater the static probability is set or not, the following is used for the toggle rate value:

  dtr * fclk

  where fclk is the frequency of the related clock, and dtr is the value of the *power_default_toggle_rate* variable.

The related clock is determined by the value of *power_default_toggle_rate_reference_clock*. Two values (fastest, related) are allowed for the value of *power_default_toggle_rate_reference_clock* variable. If the value fastest is specified, the related clock would be simply the fastest clock in the design. If the value related is given, the related clock would depend on which clock domain the net belongs to.

The value of *power_default_static_probability* should be between 0.0 and 1.0, both inclusive. The value of *power_default_toggle_rate* should be greater or equal to 0.0. Also, if the value of *power_default_static_probability* is 0.0 or 1.0, then the value of *power_default_toggle_rate* should be 0.0. If the value of *power_default_toggle_rate* is 0.0, then the value of *power_default_static_probability* should be either 0.0 or 1.0.

The default of *power_default_static_probability* is 0.5 and the default of *power_default_toggle_rate* is 0.1.

See Also

- set_switching_activity
- power_default_static_probability
- power_default_toggle_rate_reference_clock

Feedback

power_default_toggle_rate_reference_clock

Specifies how the related clock for default toggle rate is determined.

Data Types

string

Default related

Description

The *power_default_toggle_rate_reference_clock*, *power_default_toggle_rate*, and *power_default_static_probability* variables are used to determine the switching activity of non-user-annotated nets that are driven by primary inputs or black-box cells. The *power_default_toggle_rate* is used to specify the default toggle rate value, and the *power_default_toggle_rate_reference_clock* variable is used to specify how the related clock for default toggle rate is determined.

For other nonannotated nets, PrimePower propagates the switching activities of the driving cell inputs based on the cell functionality to derive the switching activity required for power calculations. This mechanism cannot be used for primary inputs and black-box outputs. Instead the following values are used for these type of nets:

- User-annotated values are used.
- In some cases, unannotated switching activity values may still be accurately derived, for example, if the net drives a buffer cell and the output of this cell is user annotated, then the user annotated values are used as the default values. Also, if the input is a clock then the clock period and waveform are used to derive the switching activity values.
- If the static probability is not annotated then the value of the *power_default_static_probability* variable is used for the static probability value.
- If the toggle rate is not user annotated, the following is used for the toggle rate value regardless of whether the static probability is set or not:

  dtr * fclk

  where fclk is the frequency of the related clock, and the dtr is the value of the *power_default_toggle_rate* variable.

The related clock is determined by the value of *power_default_toggle_rate_reference_clock*. Two values (*fastest*, *related*) are allowed for the value of *power_default_toggle_rate_reference_clock* variable. If the value fastest is specified, the related clock would be simply the fastest clock in the design. If the value related is given, the related clock would depend on which clock domain the net belongs to.

Feedback

The value of *power_default_static_probability* should be between 0.0 and 1.0, both inclusive. The value of *power_default_toggle_rate* should be greater or equal to 0.0. Also, if the value of *power_default_static_probability* is 0.0 or 1.0, then the value of *power_default_toggle_rate* should be 0.0. If the value of *power_default_toggle_rate* is 0.0, then the value of *power_default_static_probability* should be either 0.0 or 1.0.

The default of both variables is 0.5.

See Also

- set_switching_activity
- power_default_static_probability
- power_default_toggle_rate

---

power_disable_cell_sdpd_activity_estimation

Disables state-dependent and path-dependent (SDPD) switching activity estimation for instances on SDPD tracking list, in hybrid power analysis mode. This variable has been deprecated and replaced by a new variable *power_disable_sdpd_estimation*. Use the new variable, the current variable is discontinued in the newer versions of PrimePower.

Data Types

Boolean

Default false

Description

This variable has been deprecated and replaced by a new variable *power_disable_sdpd_estimation*. Use the new variable, the current variable is discontinued in the newer versions of PrimePower.

In averaged power analysis mode, when SDPD tracking is enabled (hybrid power analysis), for instances in the SDPD tracking list, the SDPD activity is calculated only for the state or path conditions from VCD or FSDB file which are matched with the library SDPD power arcs. By default, for unmatched state or path conditions from VCD or FSDB, no SDPD activity is calculated or estimated.

If the *power_disable_cell_sdpd_activity_estimation* variable is set to *false*, SDPD activity estimation gets enabled for instances on the SDPD tracking list, the SDPD activity is estimated from the unmatched state or path conditions from VCD or FSDB, and is assigned to all library power arcs not matching state or path conditions from VCD or FSDB file including default arc.

Feedback

This variable should be used together with the *set_power_analysis_options* command with the *-sdpd_tracking enabled* option to enable activity estimation for instances in the SDPD tracking list, even if you specify an input activity file. This distributes the activities over all the state conditions (including default state) defined in the library.

The default is *false*. If you want to use only matching state or path conditions from VCD or FSDB file, then set the variable to *true*.

See Also

- set_power_analysis_options
- power_disable_sdpd_estimation

---

power_disable_exact_built_in_name_mapping

Enables or disables exact and built-in name matching of VCD or SAIF objects to the gate-level object when annotating activity for the power analysis.

Data Types

Boolean

Default false

Description

When set to *true*, the tool does not perform exact and built-in name matching of the VCD or SAIF objects to the gate-level objects. The tool honors only objects that are explicitly mapped with the *set_rtl_to_gate_name* command.

See Also

- set_rtl_to_gate_name

---

power_disable_exact_name_match_to_hier_pin

Enables or disables exact matching of VCD or SAIF objects to hierarchical pin names in the gate-level netlist when annotating activity for PrimePower power analysis.

Data Types

Boolean

Default false

Feedback

Description

If you set this variable to *true*, the tool does not perform exact name matching of VCD or SAIF objects to gate-level hierarchical pins. The tool honors only hierarchical pins that are explicitly mapped with the *set_rtl_to_gate_name* command

See Also

- set_rtl_to_gate_name

---

power_disable_exact_name_match_to_net

Disables exact name matching of the RTL VCD or SAIF objects to the net names in the gate-level netlist when annotating switching activity during power analysis.

Data Types

Boolean

Default false

Description

If you set the *power_disable_exact_name_match_to_net* variable to *true*, the exact name matching of the RTL VCD or SAIF objects to gate-level nets stops. Only the nets which are explicitly mapped using the *set_rtl_to_gate_name* command will be honored.

See Also

- set_rtl_to_gate_name

---

power_disable_sdpd_estimation

Disables state-dependent and path-dependent (SDPD) switching activity estimation for instances on SDPD tracking list, in hybrid power analysis mode.

Data Types

Boolean

Default false

Description

In average mode power analysis, when SDPD tracking is enabled (hybrid analysis), for instances in SDPD tracking list, the SDPD activity is calculated only for the state or path conditions from VCD or FSDB file which are matched with library SDPD power arcs. By

Feedback

default, for unmatched state or path conditions from VCD or FSDB, no SDPD activity is calculated or estimated.

If the *power_disable_sdpd_estimation* variable is set to *false*, SDPD activity estimation gets enabled for instances on the SDPD tracking list, the SDPD activity is estimated for the unmatched state or path conditions from VCD or FSDB, and is assigned to all library power arcs not matching state or path conditions from VCD or FSDB file including default arc.

This variable should be used together with the *set_power_analysis_options* command with the *-sdpd_tracking enabled* option to enable activity estimation for the list of instances specified through the option *-sdpd_tracking_cell* in the SDPD tracking list, even if you specify an input activity file. This distributes the activities over all the state conditions (including default state) defined in the library.

The default is *false*. If you want to use only matching state or path conditions from VCD or FSDB file, then set the variable to *true*.

See Also

- set_power_analysis_options

---

power_disable_switch_cell_internal_ground_pin_check

Disables on-state check of ground PG pins during power analysis.

Data Types

Boolean

Default false

Description

When set to *true*, disables on-state check of ground PG pins during power analysis. If the ground PG pin is in off-state, the cell is considered to be in shutdown mode and no power is reported. By default, PrimePower performs on-state check of ground PG pins during power analysis.

See Also

- set_disable_pg_pins

---

power_distributed_work_dir

Specifies the directory for the generation of all output files from the child processes in distributed power analysis for PrimePower in time-based mode.

Feedback

Data Types

string

Default work_pp

Description

When specified, all output files as well as power waveforms and worker logs from child processes are generated in the specified directory for distributed analysis in time-based mode.

See Also

- set_concurrent_event_analysis_options
- power_enable_concurrent_event_analysis
- power_enable_distributed_analysis

power_dump_multi_rail_total_waveform

Enables generation of total power waveform and rail-specific power waveform, for the design simultaneously.

Data Types

Boolean

Default false

Description

When set to *true*, PrimePower generates the following waveform for the design: 1. Total power 2. Combined design rails 3. Per rail for the specified rails

This variable should be used only when the multi rail analysis and generation of power waveform per rail are enabled during time-based power analysis.

See Also

- set_power_analysis_options
- update_power

power_em_disable_extrapolation

Enables or disables extrapolation for PrimePower electromigration analysis.

Feedback

Data Types

Boolean

Default false

Description

By default, PrimePower will extrapolate one extra grid point outside EM table. If you set this variable to *true*, the tool will stop extrapolating outside table range and will cap the value at boundary points.

---

power_enable_advanced_cycle_power_analysis

Enables Cycle Power, a fast cycle accurate time based power analysis

Data Types

Boolean

Default false

Description

When set to *true*, enables Cycle Power a fast cycle accurate time based power analysis feature in PrimePower. PP-Elite license is needed to run this feature. Without this variable set to *true*, PrimePower runs regular time based power analysis. By default, Cycle Power is disabled and this variable is set to *false*.

If you set this variable to *true* and enable Cycle Power, you must obtain a PP-Elite license in addition to PrimePower license. You cannot use Cycle Power without the licenses.

See Also

- power_enable_analysis
- power_analysis_mode
- update_power
- report_power

---

power_enable_advanced_fsdb_reader

Enables or disables advanced fsdb reader for PrimePower.

Data Types

Boolean

Feedback

Default false

Description

When set to *true*, enables advanced fsdb reader for PrimePower to support regular fsdb file reading as well as virtual fsdb file reading. Without this variable set to *true*, virtual fsdb file reading is not supported. By default, *power_enable_advanced_fsdb_reader* is disabled for PrimePower.

---

power_enable_advanced_sv_name_mapping

Enables or disables advanced name mapping.

Data Types

Boolean

Default false

Description

When set to *true*, it enables the advanced SystemVerilog aware name mapping. This can significantly improve the signal annotation rate with RTL FSDB or VCD input files, with SystemVerilog constructs. File "check_name_mapping.txt" is generated with the details of name mapping. By default, *power_enable_advanced_sv_name_mapping* is disabled for PrimePower.

See Also

- read_fsdb
- read_vcd

---

power_enable_analysis

Enables or disables PrimePower, which provides power analysis.

Data Types

Boolean

Default false

Description

When set to *true*, enables PrimePower, so that you can perform power analysis. Without this variable set to *true*, you cannot see power related data. By default, PrimePower is disabled; this variable is set to *false*.

Feedback

If you set this variable to *true* and enable PrimePower, you must obtain a PrimePower license. You cannot use PrimePower without a license.

See Also

- report_power

---

power_enable_annotation_precedence_over_implied

Enables precedence of "annotated" values over "implied" values in PrimePower.

Data Types

Boolean

Default true

Description

By default this variable is set to *true*, which means toggle rate (TR) or static probability (SP) values annotated from FSDB or VCD takes precedence over implied values.

See Also

- power_activity_precedence_over_force_implied

---

power_enable_background_when_tree_creation

Enables background creation of the library when tree expression.

Data Types

Boolean

Default false

Description

When set to *true*, enables background creation of the library when tree expression. This action improves the overall power data initialization runtime. Set this variable before the first power command.

---

power_enable_capp_backward_implication

Enables backward implication on buffer or inverters in cycle-accurate-peak-power (CAPP) flow.

Feedback

Data Types

Boolean

Default false

Description

When set to *true*, the backward implication is enabled in time-based mode. By default, this variable is set to *false* and the backward implication is disabled in time-based mode.

---

power_enable_clock_cycle_based_glitch

Enables clock-cycle-based glitch detection. This variable should not be set when *power_enable_clock_domain_based_glitch* is also enabled.

Data Types

Boolean

Default false

Description

When set to *true*, this variable enables PrimePower to do clock-cycle-based glitch detection in time-based analysis. This is allowed for regular time-based flow (VCD or FSDB with delays) or delay shifted flow. When there are more than two transitions occurring on a pin or net within reference clock period, PrimePower considers these transitions as glitch transitions. If there are odd numbers of transitions in the reference clock period, PrimePower considers the last transition as a regular transition and the rest as glitch transitions. The power associated with glitch transitions is categorized as glitch power.

The fastest clock period in the design is considered as reference clock period.

Depending on the pulse width, a glitch transition is categorized as an inertial glitch (IG) or transport glitch (TG) and the detailed power calculation report generated by the *report_power_calculation* command includes transport glitch and inertial glitch details.

When this variable is enabled, more glitch power is likely to be reported compared to when the variable is disabled.

When *power_enable_clock_domain_based_glitch* is also enabled, then clock-cycle-based glitch analysis is disabled, and instead, clock-domain-based glitch analysis is done.

Feedback

See Also

- power_enable_clock_domain_based_glitch
- power_enable_glitch_calculation
- power_filter_glitch_pulse_width

---

power_enable_clock_domain_based_glitch

Enables clock-domain-based glitch detection and categorization. Both RTL and gate-level FSDB flow are supported. This variable automatically overrides *power_enable_clock_cycle_based_glitch* variable when both are set to *true*.

Data Types

Boolean

Default false

Description

When set to *true*, this variable enables PrimePower to do clock-domain-based glitch detection in time-based power analysis. Instead of using the fastest clock period in the design, the *power_base_clock* attribute of each pin or net is used for glitch analysis and categorization. This allows glitch categorization accuracy to improve.

This is allowed for regular time-based flow (VCD or FSDB with delays) or delay shifted flow. When there are more than two transitions occurring on a pin or net within reference clock period (clock period of the related base clock), PrimePower considers these transitions as glitch transitions. If there are odd numbers of transitions in the reference clock period, PrimePower considers the last transition as a regular transition and the rest as glitch transitions. The power associated with glitch transitions is categorized as glitch power.

Depending on the pulse width, a glitch transition is categorized as an inertial glitch (IG) or transport glitch (TG) and the detailed power calculation report generated by the *report_power_calculation* command includes transport glitch and inertial glitch details.

When this variable is enabled, more glitch power is likely to be reported compared to when the variable is disabled.

When *power_enable_clock_domain_based_glitch* is enabled, then the clock-cycle-based glitch analysis is disabled, and instead, clock-domain-based glitch analysis is done.

Feedback

See Also

- power_enable_clock_cycle_based_glitch
- power_enable_glitch_calculation

power_enable_clock_scaling

Enables or disables clock scaling for power analysis in PrimePower.

Data Types

Boolean

Default false

Description

When set to *true*, this variable enables PrimePower to scale average power number according to clock frequencies specified in SDC and the *set_power_clock_scaling* command.

See Also

- set_power_clock_scaling

power_enable_concurrent_event_analysis

Enables or disables concurrent event analysis in PrimePower.

Data Types

Boolean

Default false

Description

When set to *true*, enables concurrent event analysis in average and time-based modes in PrimePower for speeding up power analysis runtime with a large FSDB file. It is recommended to use this variable with the *read_fsdb* command on a gate-level FSDB. With concurrent event analysis enabled, the tool runs power analysis through multiple processes over different time windows and the power results are then merged. The time window is split by the PrimePower based on the time interval specified by the *read_fsdb* command. If no time interval is specified, the total simulation window in the FSDB file is used. PrimePower checks if the FSDB data in the given time window is large enough to perform concurrent event analysis.

Feedback

See Also

- power_enable_concurrent_hybrid_analysis
- read_fsdb
- report_power

---

power_enable_concurrent_hybrid_analysis

Enables or disables concurrent hybrid analysis in PrimePower.

Data Types

Boolean

Default false

Description

When set to *true*, enables concurrent hybrid analysis in average mode in PrimePower. This can be useful for speeding up the power analysis runtime with a large FSDB file. It is recommended to use this variable with 'read_fsdb' command on a gate-level FSDB. With concurrent hybrid analysis enabled , the analysis is run with multiple processes over different time windows and the power results are then merged. The time window is split by PrimePower based on the user specified time interval in 'read_fsdb' command. If no time interval is specified , then the total simulation window in the FSDB is used . PrimePower will check if the FSDB data in the given time window is large enough to perform concurrent hybrid analysis.

See Also

- report_power

---

power_enable_constant_init_on_rtl

Enables constant initialization on unannotated RTL points.

Data Types

Boolean

Default false

Description

When set to *true*, it initializes to zero, all the unannotated RTL points and the activity source is set to default. If the variable is set to *false*, it remains uninitialized, and can

Feedback

cause unwanted X propagation in the downstream logic. By default, this variable is set to *false*.

The feature only works in time-based power analysis mode.

See Also

- get_switching_activity
- power_x_transition_derate_factor

power_enable_cycle_based_power_analysis

This is a synonym for the *power_enable_advanced_cycle_power_analysis* variable.

Default false

See Also

- power_enable_advanced_cycle_power_analysis

power_enable_delay_shifted_event_analysis

Enables or disables delay shifted event analysis in PrimePower.

Data Types

Boolean

Default false

Description

When set to *true*, enables delay aware peak power analysis in PrimePower using RTL or Zero-delay VCD/FSDB . With the feature enabled , users can get realistic power waveforms with RTL or Zero-delay FSDB/VCD using the delays obtained from PrimeTime. This is expected to give an accuracy comparable to peak power analysis done using SDF annotated Gate-level FSDB/VCD. Appropriate delays are added to each events in the activity file and power is computed using these delay shifted events. For CAPP flow , delays are added to propagated events as well. For this feature to work , the power analysis mode must be set to time_based and the *"rtl"* or *"zero_delay"* option (as appropriate) must be provided when reading the VCD or FSDB file. Peak power is calculated using a sampling interval of 1ps. The "-waveform_interval" or "-cycle_accurate_clock" or "-cycle_accurate_cycle_count" options in set_power_analysis_options command are ignored in this flow.

Feedback

See Also

- set_power_delay_shifted_event_analysis_options

---

power_enable_distributed_analysis

Enables or disables distributed power analysis.

Data Types

Boolean

Default false

Description

When set to *true*, it enables distributed power analysis flow in PrimePower and performs power analysis in parallel using the multiple machine distribution technology. By default, *power_enable_distributed_analysis* is disabled for PrimePower.

See Also

- set_concurrent_event_analysis_options
- set_distributed_power_analysis_options
- set_host_options

---

power_enable_em_analysis

Enables or disables electromigration analysis for PrimePower.

Data Types

Boolean

Default false

Description

When set to *true*, enables electromigration analysis for PrimePower. Without this variable set to *true*, you cannot run electromigration related commands. By default, *power_enable_em_analysis* is disabled for PrimePower.

See Also

- power_enable_signal_em_analysis

Feedback

power_enable_exclude_output_port_capacitance

Enables or disables *output port capacitance* from total power capacitance calculation.

Data Types

Boolean

Default true

Description

When set to *false*, *total power capacitance* includes output port capacitance. With this variable set to *true*, you can subtract output port capacitance from total power capacitance. By default, *power_enable_exclude_output_port_capacitance* is enabled for PrimePower. Supported in both averaged and time_based mode.

See Also

- report_power

power_enable_feedthrough_in_empty_cell

Enables PrimePower to check for presence of feedthrough nets in empty hierarchical cells.

Data Types

Boolean

Default true

Description

By default, this variable is set to *true* which means PrimePower checks the presence of feedthrough nets in empty hierarchical cells, and if such net is present, the cell is no longer considered as a black box cell. If the variable is set to *false*, it identifies a hierarchical cell, which does not contain any other cell inside it, as a black box cell.

See Also

- get_cells

power_enable_full_range_c1cn_wavg

Enables or disables weighted averaging of C1CN capacitance.

Feedback

Data Types

Boolean

Default true

Description

When set to *true*, it enables the weighted averaging of C1CN capacitance segments over full capacitance range. By default, this variable is set to *true*. This variable works when C1CN capacitance mode is enabled in PrimePower.

See Also

- power_use_c1cn_pin_capacitance

---

power_enable_glitch_calculation

Enables glitch power analysis in time-based power analysis mode using the gate-level vector, or RTL or zero-delay FSDB with delay shifting.

Data Types

Boolean

Default true

Description

This variable enables glitch power analysis in time-based power analysis mode with gate-level vector or, RTL or zero-delay FSDB with delay shifting. The default is *true*.

See Also

- power_enable_clock_cycle_based_glitch
- power_enable_clock_domain_based_glitch
- power_enable_separate_glitch_report

---

power_enable_glitch_detection_with_scaled_slew

Enables or disables glitch detection with scaled slew for PrimePower.

Data Types

Boolean

Default true

Feedback

Description

When set to *true*, PrimePower uses unscaled full transition to detect glitch pulses as the primary criterion. If the criterion fails, then scaled full transition is used to detect glitch pulses.

This variable works only when the variable *power_full_transition_glitch_scaling* is enabled.

When set to *false*, PrimePower uses scaled full transition to detect glitch pulses.

See Also

- power_full_transition_glitch_scaling

---

power_enable_glitch_estimation

Enables glitch power estimation in cycle-accurate-peak-power (CAPP) analysis mode with RTL FSDB/VCD as input.

Data Types

Boolean

Default true

Description

This variable enables glitch power estimation in CAPP analysis mode with RTL VCD/FSDB as input. The estimated glitch power is reported in power analysis report.

See Also

- report_power

---

power_enable_ignore_create_generated_clock

Ignores the generated clock activity set by the *create_generated_clock* command. Instead, the propagated activity from the upstream master clock is used.

Data Types

Boolean

Default false

Description

When set to *true*, the generated clock activity set by the *create_generated_clock* command is ignored. Instead, the propagated activity from the upstream master clock after

Feedback

getting propagated through the netlist is used. The generated clock activity is overridden by the propagated clock activity.

If the generated Clock is defined at the output of the black box cell, by default the generated clock activity is used and the propagated activity is ignored.

PSW-214 warning is issued if the generated clock activity is overridden by the propagated clock.

---

power_enable_internal_pin_propagation

Enables or disables activity propagation and power calculation through internal pins of multi-stage sequential cells, like Multibit FFs with internal gating control, retention registers, etc. Also, allows propagation through cells which are missing BST/EBST in library.

Data Types

Boolean

Default false

Description

This enhancement allows activity propagation and power calculation through internal pins of multi-stage sequential cells, like Multibit FFs with internal gating control, Retention registers, etc. Logic function for internal pin should be described in the library, similar to output pins.

When set to *true*, propagation happens from input through internal pins of sequential cells to output pin. Switching activity of internal pin and associated net is computed during power analysis.

If library contains internal/leakage power arcs with internal pin in SDF when condition, then internal and leakage power will be calculated using internal pin state.

This feature is available in both averaged and Time based mode power analysis. In average mode, switching activity is propagated and used for power calculation. Similarly, in Time based mode, vectors are propagated through internal pin in CAPP mode.

When this variable is set to *false*, internal pin is assumed to be in 'X' state and static probability of internal pin is set to 0.5 for evaluation of internal/leakage power arcs.

This feature also allows propagation through instances which are missing BST (Boolean State Table) in the library. For example, sometimes library developers may generate ICG cells (with or without internal pins) without adding state table in library cell. These cells still have "function_id" or "test_function_id" defined on the internal and output pins. PrimePower uses "function_id" in order to propagate through these cells.

Feedback

This is the list of all types of cells which are currently supported for special propagation when variable is turned ON: 1.Multibit FF • Is not black box, is sequential and internal pin count >=1 and driver pin count >=2 . 2. Multistage Retention cell • Is not blackbox, is sequential, is multi-stage retention cell, and internal pin count >=3 . 3. Has function id, missing BST • If blackbox, is sequential, NO state_table in libcell, uses function_id and test_function_id for logic propagation. 4. ICG without BST on output pins • For ICG cell attributes (is_integrated_clock_gating_cell is TRUE and is_clock_gating_check is FALSE), 5. Multi-stage Synchronizer cells • Is not black box, is sequential, internal pin count >= 1, and no internal pin BST .

power_enable_low_effort_activity_propagation

Enables low effort activity propagation.

Data Types

Boolean

Default false

Description

When set to *true*, it enables the low effort switching activity propagation in PrimePower averaged power analysis. This can reduce runtime significantly for switching activity propagation in the averaged power analysis, without disturbing QoR for designs with 90 percent or higher annotation on Synthesis invariant points. By default, *power_enable_low_effort_activity_propagation* is disabled for PrimePower.

power_enable_merged_fsdb

Resolves conflict activity on the nets in the merged FSDB file. A conflict activity for a given net is defined as activities with different transitions in same time stamp. The activity for different net segments of a net can be different across hiearchical levels. This variable works in both average and time based analysis mode.

Data Types

Boolean

Default false

Description

When set to *true* , resolves conflicting activity on the net in the merged FSDB file.

Feedback

power_enable_mode_support

Enables mode based power analysis in PrimePower.

Data Types

Boolean

Default false

Description

When set to *true*, the cell mode set by the *set_mode* command for the cell is used for the internal and leakage power lookup from the library. Power value would be looked up using the matching cell mode value from the library. If the cell mode is not set for the cell, then the internal and leakage power lookup happens using the usual state-dependent-path-dependent and state-dependent information respectively.

If set to *false*, the cell mode is ignored during the power analysis even if the *set_mode* command is used.

The feature only works in averaged power analysis mode.

See Also

- report_power

power_enable_mt_namemapping

Enables multi threaded name mapping PrimePower.

Data Types

Boolean

Default false

Description

When set to true, name mapping is done through threaded infrastructure. This feature is set to be true with read_saif -parallel option or where parallel name mapping is needed.

See Also

- report_power

Feedback

power_enable_multi_rail_analysis

Enables or disables PrimePower concurrent multirail power analysis.

Data Types

Boolean

Default false

Description

When this variable is set to *true*, PrimePower starts concurrent multirail power analysis power updates. Under concurrent multirail power analysis mode, power data for each rails or supply nets is maintained and processed individually and concurrently. Power reports of different combinations of rail and supply net specifications can be generated without the need to update power again.

The *-rails* option of the *report_power* command requires that this variable is set to *true*.

See Also

- report_power

power_enable_multi_rtl_to_gate_mapping

Enables or disables mapping of multiple RTL objects to a gate-level object.

Data Types

Boolean

Default false

Description

When set to *true*, it allows multiple RTL objects to be mapped to the same gate-level object as specified in the name mapping file, and PrimePower keeps track of all the RTL objects during name mapping while reading SAIF or FSDB input file. By default, PrimePower registers only the first RTL object to gate-level object mapping and ignores the rest in the name mapping file.

See Also

- set_rtl_to_gate_name
- report_name_mapping

Feedback

power_enable_name_mapping_case_precedence

Enables case sensitive name mapping.

Data Types

Boolean

Default false

Description

When set to true, during activity annotation from activity file, case sensitive rtl name gets precedence over non-case sensitive rtl name.

See Also

- report_power
- report_power_calculation
- read_vcd
- read_fsdb
- set_rtl_to_gate_name

power_enable_new_rrm_view

Enables the new hierarchical reporting view for report_rtl_metrics -view hier command

Data Types

Boolean

Default false

Description

When set to *true*, report_rtl_metrics -view hier command will report the power data in a format which adds new colums to the report and matches report_power . The leaf cell classification into catagories like register, clock network etc uses the PTPX approach and application variables for report_power are considered during power data calculations.

See Also

- report_rtl_metrics
- report_power

Feedback

power_enable_nldm_cap

Enables NLDM pin capacitance in power analysis

Data Types

Boolean

Default false

Description

By default, PrimePower uses CCS receiver capacitance model for pin capacitance in power analysis if library contains both CCS receiver capacitance and NLDM (scalar) pin capacitance. With power_enable_nldm_cap enabled, PrimePower uses NLDM(scalar) capacitance for power analysis and ignores CCS receiver capacitance model.

Default value for the application variable is false. *set power_enable_nldm_cap true*.

power_enable_non_mission_mode_leakage

Enables or disables non mission leakage model support.

Data Types

Boolean

Default false

Description

When set to *true*, enables non mission leakage model support, to perform leakage power analysis with library cells having the new constructs. Without this variable set to *true*, only the older (default) library model is supported. The new model is supported in both averaged and time-based power analysis modes. By default, the variable is set to *false*.

See Also

- report_power

power_enable_parallel_sdpd_estimation

Enables or disables parallel state-dependent and path-dependent estimation in PrimePower averaged power analysis flow.

Data Types

Boolean

Feedback

Default true

Description

When this variable is *true* (the default), the tool performs parallel state-dependent and path-dependent estimation. If you set this variable to *false*, the tool stops parallel state-dependent and path-dependent estimation.

See Also

- set_power_analysis_options

---

power_enable_parallel_when_tree_creation

Enables multithreading in building of the library when tree expression.

Data Types

Boolean

Default false

Description

When set to *true*, it enables library when tree expression building in the multithreaded mode. This improves the overall power data initialization runtime. The variable should be set before running any power analysis command. By default, *power_enable_parallel_when_tree_creation* is disabled for PrimePower.

---

power_enable_pin_internal_power

Enables or disables reporting of internal_power attribute on a pin.

Data Types

Boolean

Default false

Description

When set to *true*, internal_power attribute can be queried and reported on pin. Without this variable set to *true*, you cannot query or report internal_power attribute on pin. By default, *power_enable_pin_internal_power* is disabled for PrimePower. You have to set *power_enable_pin_internal_power* after *power analysis mode and before any power command.*

Feedback

See Also

- report_attribute
- get_attribute

---

power_enable_pin_power_to_receiver_mode

Enables or disables *switching_power* attribute on the pin of the cell.

Data Types

Boolean

Default false

Description

When set to *true*, *switching_power* can be queried on the pin and reported. Without this variable set to *true*, you cannot query *switching_power* on the pin. By default, *power_enable_pin_power_to_receiver_mode* is disabled for PrimePower.

See Also

- report_power
- list_attributes

---

power_enable_power_only_cap

Enables power specific capacitance.

Data Types

Boolean

Default false

Description

When set to *true*, the tool reads new power specific capacitance values from the library and uses it for accurate power calculation. When enabled, it has the highest priority over all the other settings.

See Also

- update_power

Feedback

power_enable_rtl_advanced_cycle_power_analysis

Enables Cycle Power, a fast cycle accurate time based power analysis in RTL power analysis.

Data Types

Boolean

Default false

Description

When setting to *true*, enables Cycle Power, a fast cycle accurate time based power analysis feature in PrimePower RTL. PP-RTL-Elite license is needed to run this feature. Without this variable setting to *true*, PrimePower RTL runs regular time based power analysis. By default, Cycle Power is disabled and this variable is set to *false*.

If you set this variable to *true* and enable Cycle Power, you must obtain a PP-RTL-Elite license in addition to PrimePower-RTL license. You cannot use Cycle Power without the licenses.

See Also

- power_enable_analysis
- power_analysis_mode
- update_power
- report_power

power_enable_rtl_analysis

Enables RTL power analysis.

Data Types

Boolean

Default false

Description

When set to *true*, enables PrimePower RTL, to perform RTL power analysis. Without this variable set to *true*, you cannot see RTL power related data. By default, PrimePower RTL is disabled and set to *false*.

When this variable is set to *true*, PrimePower-RTL license is checked out.

Feedback

See Also

- compute_metrics
- report_rtl_metrics

---

power_enable_rtl_save_restore_variables

Enables PP-RTL to automatically save-restore variables in re-use flow.

Data Types

Boolean

Default false

Description

When set to *true*, enables PrimePower RTL, to automatically save and restore variables in re-use flow. Without this variable set to *true*, you cannot see the variables set to original values.

See Also

- compute_metrics
- report_rtl_metrics

---

power_enable_saif_all_state_static_probability

Enables or disables computation of static probability including all possible logic states from SAIF.

Data Types

Boolean

Default false

Description

When set to *true*, PrimePower computes the static probability taking into account all possible logic states included in SAIF - '1', '0', 'X', 'Z', and 'B' using the following equation:

SP = (T1 + 0.5*(TX+TZ+TB))/(T1+T0+TX+TZ+TB)

By default, PrimePower only considers logic state '1' and '0' from SAIF for computation of static probability using following equation SP = T1/(T1+T0)

Feedback

power_enable_saif_estimated_toggle_count

Calculates the total toggle count (TC) for a pin by aggregating its state-dependent and path-dependent TC, if the total TC information for an instance pin is missing in the SAIF.

Data Types

Boolean

Default false

Description

In averaged power analysis, when the total TC is missing for an instance pin but state-dependent and path-dependent TCs are present in the SAIF, set this variable to *true* to sum up all the state-dependent and path-dependent TCs to calculate the total TC for that pin.

If set to *false*, then the toggle rate for such a pin is not annotated from the SAIF file. In such a case, observe the propagated activity on that pin if annotation for the connected net and receiver pins are also missing in the SAIF.

Example:

(INSTANCE reg_out2_reg\[31\]

(PORT

(Q

(T0 264.78577) (T1 235.21423)

(COND (D * !CP) (RISE)

(IOPATH CDN (TC 145) (TG 0) (IG 0))

COND (D * !CP) (FALL)

(IOPATH CDN (TC 63) (TG 0) (IG 0))

)

)

In the above SAIF example, T0 and T1 are present for the Q pin but total TC is missing. When you enable this variable, PrimePower computes a total TC = (145 + 63) = 208 and compute the pin toggle rate accordingly.

See Also

- get_switching_activity

Feedback

power_enable_separate_glitch_report

Enables or disables reporting separate inertial and transport glitch report of glitch rate values on net and glitch power values on cell.

Data Types

Boolean

Default false

Description

When set to *true*, *inertial_glitch_rate* and *transport_glitch_rate* attributes can be queried and reported on net, and *inertial_glitch_power* and *transport_glitch_power* attributes can be queried and reported on cell.

By default, *power_enable_separate_glitch_report* is disabled for PrimePower. You have to set *power_enable_separate_glitch_report* after power analysis mode and before any power command.

See Also

- report_attribute
- get_attribute
- list_attributes

power_enable_show_strip_path_warning

Enables or disables PWR-1007 strip_path error messages.

Data Types

Boolean

Default false

Description

When this variable is set to *true*, it enables the PWR-1007 error messages for reporting the signals which are not present in the fsdb (strip_path + RTL name).

The *power_enable_advanced_sv_name_mapping* variable must also be set to *true* in order to use the feature.

Feedback

power_enable_signal_em_analysis

Enables or disables PrimePower signal electromigration analysis.

Data Types

Boolean

Default false

Description

This is a mega variable to turn on required variables and options to retrieve specific data for signal electromigration analysis. Note that *power_enable_analysis* must be enabled before enabling this variable.

See Also

- power_enable_analysis
- check_signal_em

power_enable_time_based_ignore_huge_cell_delay

Enables ignoring huge arc delays beyond a threshold delay during time-based power analysis.

Data Types

Boolean

Default true

Description

Enables ignoring huge arc delays beyond a threshold delay during time-based power analysis, and take default arc delay value. This influences power arc selection by matching input and output transition. With the variable set to false, PrimePower considers the actual cell arc delays.

power_enable_timing_analysis

Enables or disables PrimeTime timing analysis in PrimePower.

Data Types

Boolean

Default false

Feedback

Description

When set to *true*, enables PrimeTime in PrimePower shell , so that you can perform timing analysis. Without this variable set to *true*, you cannot see timing related data. By default, PrimeTime is disabled; this variable is set to *false* in PrimePower.

If you set this variable to *true* and enable PrimeTime, you must obtain a PrimeTime license. You cannot use PrimeTime without a license.

See Also

- update_timing

---

power_enable_write_clock_activity_in_power_waveform

Enables or disables writing fastest clock activity waveform in output power waveform.

Data Types

Boolean

Default false

Description

When this variable is set to *true*, PrimePower writes out the fastest clock activity waveform in the output power waveform. The fastest clock would be added as another signal at the top level module in the output FSDB.

This var has to be set to true before triggering The *update_power* command to enable the feature.

See Also

- update_power

---

power_estimate_power_for_unmatched_event

Controls to estimate power when no table is found in the library to match a certain event.

Data Types

Boolean

Default true

Feedback

Description

Sometimes when an output pin toggles, PrimePower cannot find a matching table in the library based on the current state of the cell. This variable controls whether PrimePower should skip this event without power contribution or try to estimate a power number for it according to all the tables in the library relating to this output pin.

---

power_filter_glitch_pulse_width

Enables filtering of glitch pulses based on their pulse width in the time-based mode.

Data Types

float

Default -1.0

Description

This variable allows the filtering of glitch pulses based on their pulse widths that are less than the specified threshold value, so no toggle or power is calculated for these events. The specified value is a time in ns. This feature works in time-based flow and also in delay shifted flow with the zero-delay gate-level VCD/FSDB having delay information.

This featured is enabled with the following variable setting:

set power_filter_glitch_pulse_width *value*

The value can be any real number >= 0. The default is -1.0 which means the feature is disabled.

See Also

- power_enable_clock_cycle_based_glitch

---

power_full_transition_glitch_scaling

Enables the scaling of transition times used to identify and calculate the power of glitches

Data Types

float

Default 1.0

Description

To prevent over-estimation of glitch power, PrimePower enables scaling of the transition times used to identify and calculate the power of glitches, such that they represent the

Feedback

times required to reach full logic levels; instead of the times required to transition from the slew trip-points defined in the library. In the time-based analysis mode, PrimePower relies on the following formulas to identify glitches and to scale their power: Glitch Identification: If (trise + tfall) > 2 * pulse_width then pulse = glitch Glitch Power Scaling Ratio: ((2 * pulse_width)/(trise + tfall))2 The transition times derived from timing analysis represent the times to reach the trip-points (eg 20%-80%); which can cause would-be-glitches to be seen as full logic level transitions. In the K-1512 release, users can specify a scaling ratio, by which to multiply the transition times; identify more glitch transitions; and to scale their power accordingly.

This featured is enabled with the following variable setting: set power_full_transition_glitch_scaling <value> The value can be any real number between 1.0 and 10.0. The default value is 1.0

power_include_driverless_net_switching_power

Computes switching power for nets without a leaf level driver instance and not connected to primary input port.

Data Types

Boolean

Default false

Description

By default, PrimePower does not compute switching power for driverless nets. This app var enables switching power calculation for driverless nets in design and reports the switching power contribution for such nets in report_power output. Switching power attribute value is set on driverless nets as well.

power_include_initial_x_transitions

Controls to count x power in the power up initialization stage.

Data Types

Boolean

Default false

Description

Initially, if you do not set a logic value to a certain net, PrimePower assumes the logic value is X. In the later stage, the value becomes *0* or *1*. This variable controls whether PrimePower should count the power caused by the X->0 or X->1 toggle.

Feedback

power_include_tiecell_power

Analyzes and reports power of the tie cells.

Data Types

Boolean

Default false

Description

When set to *true*, enables power analysis of the tie cells in averaged power analysis mode. By default, this variable is set to *false* and the power of tie cells is not reported.

power_keep_vcd_event_order

Enables PrimePower to preserve the order of events read from VCD/FSDB in cycle-accurate-peak-power (CAPP) flow.

Data Types

Boolean

Default false

Description

By default, PrimePower reorders the events in CAPP flow so that output events occur after the input events within a time-stamp. When set to *true*, PrimePower preserves the order of events read from VCD/FSDB.

This variable works in both time-based and averaged power analysis mode with *-sdpd_tracking* enabled, for the cells whose state-dependent and path-dependent activity is being tracked.

See Also

- report_power

power_limit_extrapolation_range

Limits extrapolation to a certain range for internal power calculations.

Data Types

Boolean

Default false

Feedback

Description

By default, PrimePower extrapolates without limits when performing internal power table lookups for out-of-range data points.

To stop extrapolation at one additional index grid when performing lookups in NLPM (nonlinear power model) internal power tables, set this variable to *true*.

---

power_match_state_for_logic_x

Specifies logic x interpretation.

Data Types

string

Default 0

Description

Specifies how PrimePower interprets logic x in the Boolean function of the *when* state of a power table. This variable uses the following settings:

- 0: regards x as zero (0)
- 1: regards x as one (1)
- x/X: regards x as neither 0 nor 1. For example, when there is any x logic, the Boolean function is evaluated as false.

See Also

- report_power

---

power_match_value_for_logic_x

Specifies logic x interpretation.

Data Types

string

Default x

Feedback

Description

Specifies how PrimePower interprets logic x in the annotation from FSDB. This variable uses the following settings:

- 0: regards x as zero (0)
- 1: regards x as one (1)

See Also

- report_power

---

power_order_clock_events

Enables PrimePower to process clock pin events before other data pin events, read from the VCD or FSDB in cycle-accurate-peak-power (CAPP) flow.

Data Types

Boolean

Default false

Description

When set to *true*, for all the events occurring within a given time-stamp, the PrimePower reorders the clock pin events before other data pin events for a given sequential cell. This is useful when reading FSDB or VCD generated without sequence information.

This variable works in both time-based and averaged power analysis mode with *-sdpd_tracking* enabled, for the cells whose state-dependent and path-dependent activity is being tracked.

See Also

- report_power

---

power_rail_output_file

Specifies the output file name for writing out power information for PrimeRail.

Data Types

string

Default "" (empty)

Feedback

Description

The *power_rail_output_file* variable is provided for PrimeRail. If the file name is provided with this variable, during power calculation relevant power information is written into the file provided. Note that information is written in a binary format. This file is then read by PrimeRail.

Starting with the B-2008.12 release, the *update_power* command can perform both average and peak power analysis. The *power_analysis_mode* variable can be used to specify the power analysis mode to perform. PrimePower can perform different types of power analysis based on the mode setting. The default power analysis mode is *averaged*. For more information, see the *power_analysis_mode* man page.

The *set_power_analysis_options* command can be used to specify the options for power analysis. It must be set before the *update_power* command to take effect in power analysis. Issuing the *set_power_analysis_options* command with no option can reset all the power analysis options to default. Use the *report_power_analysis_options* command to get the current analysis option settings. See the *set_power_analysis_options* man page for more information.

PrimePower can consume either time-based (for example, a VCD file) or statistical switching activity information (for example, a SAIF file) for power calculation. For a particular analysis mode, appropriate activity information must be provided. For example, in time-based power analysis mode, a VCD file must be provided. In averaged power analysis mode, any form of switching activity information is accepted. You can specify a VCD file by using the *read_vcd* command. The statistical switching activity can be specified by using the *read_saif* or *set_switching_activity* commands.

See Also

- report_power_analysis_options
- set_power_analysis_options
- update_power
- power_analysis_mode

---

power_read_activity_ignore_case

Use the *power_read_activity_ignore_case* variable instead of the *power_read_vcd_ignore_case* variable to control ignore case when reading activity files.

Data Types

Boolean

Default true

Feedback

Description

The *power_read_vcd_ignore_case* variable has been replaced with the *power_read_activity_ignore_case* variable. For backward compatibility, the *power_read_vcd_ignore_case* variable is an alias for the *power_read_activity_ignore_case* variable.

The *power_read_activity_ignore_case* variable controls match pin, net and cell names from VCD or SAIF file and those from the design case sensitively if the value of this variable is *false* or design case insensitively if the value is *true*. This variable also affects the *set_rtl_to_gate_name* command.

See Also

- read_saif
- read_vcd
- set_rtl_to_gate_name

---

power_read_unique_net_activity

Reads unique net activities from VCD/FSDB.

Data Types

Boolean

Default true

Description

When set to *true*, this variable enables the tool to read and annotate unique net activities from VCD/FSDB. Typically a VCD/FSDB activity file has annotations for different net segments and pins of the same logical net. This can lead to annotation conflicts and spurious toggles or glitches on the given net. With this variable enabled, the tool assigns a unique literal or var id to each gate-level net in the design corresponding to a VCD/FSDB object name.

The value changes for these literal or var id are read and annotated on the respective nets. The mapping of a VCD/FSDB object name to a given gate-level net is done using the following order of precedence.

1. Match with the net driver which is a leaf-level output pin or a primary input port of the design.
2. Match with the top-level segment of the given net.

Feedback

3. Match with any other logical segment of the given net.
4. Match with either of the net load which is a leaf-level input or inout pin or primary output port of the design.

This variable works for both average and time-based modes.

This variable has no effect if the variable *power_enable_merged_fsdb* is set to *true*.

See Also

- power_enable_merged_fsdb
- power_disable_exact_name_match_to_net
- power_disable_exact_name_match_to_hier_pin
- read_fsdb

---

power_read_vcd_ignore_case

Ignore cases when reading activity files.

Data Types

Boolean

Default true

Description

The *power_read_vcd_ignore_case* variable has been replaced with the *power_read_activity_ignore_case* variable. For backward compatibility, the *power_read_vcd_ignore_case* variable is an alias for the *power_read_activity_ignore_case* variable.

The *power_read_activity_ignore_case* variable controls match pin, net and cell names from VCD or SAIF file and those from the design case sensitively if the value of this variable is *false* or design case insensitively if the value is *true*. This variable also affects the *set_rtl_to_gate_name* command.

See Also

- read_saif
- read_vcd
- set_rtl_to_gate_name

Feedback

power_report_leakage_breakdowns

Controls to report leakage components.

Data Types

Boolean

Default false

Description

To specify whether the *report_power* command prints out leakage power components, set this variable to one of the following values:

- *false* (the default) - Does not report leakage power components. For example, only total leakage is reported.
- *true* - Reports intrinsic leakage and gate leakage in addition to the total leakage in the summary report and the cell-based power report.

See Also

- report_power

power_reset_negative_extrapolation_value

Resets the negative extrapolated energy value from library to zero.

Data Types

Boolean

Default false

Description

The *power_reset_negative_extrapolation_value* variable is used to reset the negative extrapolated energy value from library to zero, if the energy value for the boundary points is positive. If this variable is set to *true*, the negative extrapolated energy value is reset to zero.

In some cases, the values of variables (mostly output capacitance and input slew), which is used for extracting the energy number from library energy tables is out of range. For example, the values can be greater or smaller than the boundary points. In this scenario, extrapolation techniques are used for deriving the energy number from library energy tables. If the variable values are too small or too big, the derived energy number can come out negative. However, the energy number corresponding to the boundary points

Feedback

can be positive. Using the negative energy number given the fact that the energy number corresponding to boundary points is positive, results in incorrect energy numbers.

To resolve this problem, use the *power_reset_negative_extrapolation_value* variable, which if set to *true*, resets the negative extrapolated energy numbers to zero, if the energy number for boundary points is positive.

---

power_reset_negative_internal_power

Resets the negative internal power to zero.

Data Types

Boolean

Default false

Description

This variable resets the negative internal power to zero. You can use this variable if you are confident about the accuracy of the power tables. Furthermore, if the values of capacitance and input slew values are out of range from the range specified in the power tables, the internal energy number can be negative due to extrapolation or interpolation. This variable gives you the choice to reset the negative internal energy numbers to zero.

See Also

- report_power
- update_power

---

power_scale_dynamic_power_at_power_off

Indicates if the dynamic power is scaled according to the static probability of the corresponding power supply net. When this variable is set to *false*, only leakage power is scaled by the power-on probability.

Data Types

Boolean

Default false

Description

The statistical activity information can be input from the SAIF file, the *set_switching_activity* and *set_case_analysis* commands, or from default settings and propagated through the whole design. PrimePower is built with power management

Feedback

awareness and can reflect power saving technology used in the design in the calculated power results. If the power supply net is switched off during simulation, PrimePower can report the correct dynamic and static (leakage) power based on the statistical activity information.

If the given statistical activity information does not contain any toggle happened at the power-off state, only leakage power is scaled by the power-on probability. This is the default behavior. However, if the input activity information includes toggles happened at the power-off state, both dynamic and leakage power is scaled. Under such situation, you need to set the *power_scale_dynamic_power_at_power_off* variable to *true*, so that PrimePower applies the scaling to dynamic power as well.

This variable has no effect if there is no power switch Boolean function defined. Also it only applies to averaged power analysis mode. It has no effect for time-based power analysis mode. As in time-based mode, PrimePower monitors the status of the power supply net and determines the power consumption at event basis.

See Also

- power_analysis_mode
- read_saif
- set_switching_activity
- set_case_analysis

---

power_scale_internal_arc

Enables scaling of state probabilities of internal power arcs.

Data Types

Boolean

Default false

Description

By default, this variable is set to *false*, which disables scaling. If you set this variable to *true*, the sum of the probabilities of internal arc matches with the toggle rate of the pin, even if there is no default arc for the pin in the library.

See Also

- update_power

Feedback

power_ssa_force_override_register_user_ssa

Enables or disables special activity propagation of sequential cells (DFFs, Latches) when CLK pin is set to a constant value (0/1) using "set_switching_activity -force" in PrimePower. In case of sequential cells, switching activity (TR/SP/GR) of clock pin of is copied to Q pin for DFFs. For latches, clock pin activity is copied to Q pin depending on static probability (SP) value of clock pin. If SP(CLK)==1, TR/GR of CLK pin is copied to Q pin, else Q pin retains previous activity value.

Data Types

Boolean

Default false

Description

When set to *true*, applies special propagation for sequential cells when CLK pin is set to a constant value (0/1) using "set_switching_activity -force". Allows user to do what-if power analysis.

power_table_include_switching_power

Indicates whether power tables in technology library include switch power.

Data Types

Boolean

Default true

Description

Indicates whether the nonlinear power model (NLPM) in the technology library includes switch power components. There can be 2 types of NLPM internal power tables from characterization. One type contains pure internal energy. Another type contains (total energy - 0.5 * switching energy). PrimePower supports the second format by default. For average power analysis, the difference of these two types of characterization does not have significant impact on the results. The effects of added switching energy canceled out by events of opposite edges. For power waveform generation, the added 0.5 switching energy makes the difference, especially for single cell or very small designs. For correlation and characterization verification purposes, small designs or single cell designs might be used and the differences can be significant.

When the variable is set to *false*, it means the power tables in the technology library are of the first type. For example, the values in the tables are pure internal energy. When it is set to *true*, the values in power tables are of the second type. PrimePower chooses the proper power calculation approach based on this variable.

Feedback

power_use_c1cn_pin_capacitance

Specifies whether to use the C1CN capacitance model when deriving capacitance values for the power analysis.

Data Types

Boolean

Default false

Description

When set to *true*, PrimePower takes slew and voltage dependent C1CN capacitance values into account when calculating the input capacitance. For a given rise or fall slew, the library with C1CN pin information provides an array of rise or fall capacitance values corresponding to the array of different voltages during the rise or fall transition. Using this model to derive pin capacitance leads to accurate power calculation.

See Also

- update_power

power_use_ccsp_pin_capacitance

Specifies whether to include the additional capacitance contribution of the Miller Effect when deriving capacitance values for power analysis.

Data Types

Boolean

Default false

Description

If you set this variable to *true*, PrimePower takes into account the waveform distortion due to the Miller Effect, when calculating the input capacitance. This setting is recommended when analyzing power for deep submicron, low voltage technologies where the gate-to-source and gate-to-drain capacitance contributions to the Miller Effect are more pronounced.

See Also

- update_power

Feedback

power_use_file_simulation_time

Takes full simulation time of the FSDB file, even if it has no events at end of the file.

Data Types

Boolean

Default false

Description

When set to *true*, simulation time is set to the end of the FSDB file, even if it has no events at the end of the file, in both averaged and time-based flow. By default, empty events at the end of the file are ignored, so the simulation time becomes smaller than the actual file simulation time window.

The feature only works for the *read_fsdb* command in averaged power analysis and time-based power analysis modes.

See Also

- report_power
- read_fsdb

power_x_transition_derate_factor

Sets the scale factor for X-transition power.

Data Types

float

Default 0.5

Description

Controls a scale factor for X-transition power.

See Also

- update_power

Feedback

ps_cell_robustness_cap_filter_ratio

Sets the ratio which determines which cells are filtered out based on the *drc_max_capacitance_slack* criterion for PrimeShield load_cap and wire_cap cell robustness analysis.

Data Types

float

Default 0.0

Description

This variable only applies to PrimeShield load_cap and wire_cap cell robustness analyses. It can take a value between *0.0* and *1.0*.

Use this variable to control which cells are included in the PrimeShield load_cap and wire_cap robustness analyses. If the ratio of driver pin cap to *drc_max_capacitance_slack* limit for a pin is below the value of this variable, then the PrimeShield tool does not include the cell for robustness analysis.

See Also

- ps_enable_analysis
- report_cell_robustness
- report_voltage_robustness

ps_cell_robustness_cell_loss_threshold

Sets the threshold which determines which cells get filtered out in PrimeShield variation cell robustness analysis.

Data Types

float

Default 1e-6

Description

This variable applies to PrimeShield cell robustness analysis for *-type variation* . It controls which cells are filtered out and not included in the list of violator cells that get reported. If the High Sigma Failure Rate (HSFR) of a cell is less than this threshold, then it will not be included in the list of violator cells which get reported. The value of the variable should be positive.

Feedback

See Also

- ps_enable_analysis
- report_cell_robustness
- report_voltage_robustness

---

ps_cell_robustness_clock_period_ratio

Sets the ratio which determines the *slack_lesser_than* threshold that is used in PrimeShield cell robustness analysis.

Data Types

float

Default 0.0

Description

This variable applies to PrimeShield cell robustness analysis. It can take a value between *0.0* and *1.0*. It only applies when Hypertrace (for the case of *-pba* option in *report_cell_robustness*), is used to obtain PBA pin slacks for cell robustness analysis.

The numerical value obtained by multiplying fastest active clock period by this ratio is added to to the value set by *slack_lesser_than* and the resulting numerical value is used to set the refinement threshold for Hypertrace pin slack computation.

In Hypertrace, the refinement threshold determines the timing critical region of the timing graph used for computing pin slacks.

See Also

- ps_enable_analysis
- timing_refinement_max_slack_threshold
- report_cell_robustness
- report_voltage_robustness

---

ps_cell_robustness_skip_cell_pin_limit

Sets the number which determines which cells are filtered out based on the number of pins in the cell during the PrimeShield cell robustness analysis.

Feedback

Data Types

integer

Default 20

Description

This variable only applies to PrimeShield cell robustness analyses. It can take a value *0* and higher.

Use this variable to control which cells are excluded from cell robustness analysis based on number of pins on the cell.

See Also

- report_cell_robustness
- report_voltage_robustness

---

ps_cell_robustness_tran_filter_ratio

Sets the ratio which determines thet cells that are filtered out based on the *drc_max_transition_slack* criterion for PrimeShield transition cell robustness analysis.

Data Types

float

Default 0.0

Description

This variable only applies to PrimeShield transition cell robustness analysis. It can take a value between *0.0* and *1.0*. If the ratio of input pin transition to *drc_max_transition_slack* limit for that pin on any of the cell input pins is below the value of this variable, then that cell is not included in robustness analysis.

Use this variable to control which cells are included in the transition robustness analysis.

See Also

- ps_enable_analysis
- report_cell_robustness
- report_voltage_robustness

Feedback

ps_enable_analysis

Enables or disables PrimeShield, which provides design variation analysis.

Data Types

Boolean

Default false

Description

When set to *true*, this variable enables PrimeShield, which enables you to perform design variation analysis. The default is *false*, which disables PrimeShield variation analysis.

Setting this variable to *true* requires a PrimeShield license.

See Also

- sim_analyze_path
- get_design_variation
- report_design_variation
- report_variation_bottleneck
- report_cell_robustness
- report_voltage_robustness

ps_enable_save_timing_slack_data

Enables or disables efficient storage of timing slack data for PrimeShield cell robustness analysis.

Data Types

Boolean

Default false

Description

This variable is needed to perform PrimeShield cell robustness analysis.

When set to *true*, it enables *timing_save_pin_arrival_and_slack* and also enables efficient storage of timing slack data.

Setting it to *false* disables *timing_save_pin_arrival_and_slack* and disables efficient storage of timing slack data.

Feedback

See Also

- ps_enable_analysis
- timing_save_pin_arrival_and_slack
- report_cell_robustness
- report_voltage_robustness

---

ps_enable_timing_analysis

Enables or disables PrimeTime timing analysis in PrimeShield.

Data Types

Boolean

Default false

Description

When set to *true*, this variable enables PrimeTime functionality within the PrimeShield shell, which enables you to perform timing analysis. The default is *false*, which disables PrimeTime timing analysis in the PrimeShield shell.

Setting this variable to *true* requires a PrimeTime license.

See Also

- update_timing
- ps_enable_analysis

---

pt_ilm_dir

Specifies a directory for PrimeTime to create ILM related files.

Data Types

string

Default (current directory)

Description

Specifies a directory for PrimeTime to create ILM related files. By default, the value is ".", the current directory. You can set this variable to any directory, such as /dir1/dir2/ilm.

Feedback

If *hier_modeling_version* is set to 2.0, the directory to create ILM related files is specified by *pt_model_dir* instead of this variable.

See Also

- create_ilm
- create_si_context
- write_arrival_annotations
- hier_modeling_version
- pt_model_dir

pt_model_dir

Specifies a directory in which to create model related files.

Data Types

string

Default "." (current directory)

Description

Specifies a directory in which to create model related files. By default, the value is ".", the current directory. You can set this variable to any directory, such as /dir1/dir2/model.

This variable is only effective when variable *hier_modeling_version* is 2.0. All the ILM and ETM related files are in the ilm/etm subdirectory under *pt_model_dir*.

See Also

- create_ilm
- create_si_context
- extract_model
- write_arrival_annotations
- hier_modeling_version
- pt_model_dir

pt_shell_mode

Specifies the mode of operation of the current shell (deprecated).

Feedback

Data Types

string

Default none

Description

This variable is deprecated; use the *sh_host_mode* variable instead. This variable will be obsoleted and removed from the tool in the U-2022.12 release.

This read-only variable specifies the mode of operation of the current shell:

- *primetime* - Current PrimeTime shell was launched in non-multi-scenario mode.
- *primetime_manager* - Current shell was launched with the *-multi_scenario* option.
- *primetime_slave* - Current shell was launched by the *start_hosts* command in multi-scenario mode.

The *pt_shell_mode* variable is useful in scripts or setup files that are sourced by both the manager and the worker.

See Also

- pt_shell

---

pt_tmp_dir

Specifies a directory for PrimeTime to use as temporary storage.

Data Types

string

Default /tmp

Description

This variable specifies a directory for PrimeTime to use as temporary storage. By default, the value is "/tmp". The uniquification behavior of *pt_tmp_dir* is controlled by variable *sh_pt_tmp_dir_unique_mode*. You can set this variable to any directory with proper read/write permissions, such as ".", the current directory.

A very important use of the disk storage specified by *pt_tmp_dir* is for the high capacity mode (for more details about high capacity mode, see *set_program_options*). From a storage point of view, there are temporary files created during the run of PrimeTime to store some data at runtime, the files stored in pt_tmp_dir are automatically removed either during the run or at the exit of the program. To monitor and identify the subdirectories and files that are still in active use inside *pt_tmp_dir*, the tool writes special empty files with

Feedback

names in the form of LOCK#hostname#pid to indicate the host and pid of the PrimeTime process that is actively using the files in the directory.

See Also

- sh_pt_tmp_dir_unique_mode
- set_program_options

---

ptxr_root

Specifies an alternative installation root path for PrimeTime to look for the executables required by the PrimeTime External Reader (ptxr).

Data Types

string

Default By default, this variable is the same as the root path where PrimeTime is installed.

Description

When set to a different path from the default PrimeTime installation root, this variable contains a path name to the executables specific to PrimeTime external reader (ptxr). Instead of using the reader programs installed within the PrimeTime root path, this provides user with the flexibility of supplying an alternative program that is equivalent to the natively installed executable to achieve reading of file formats that are only supported by ptxr.

Because this alternative root path is outside of PrimeTime, the availability and completeness of that installation is not guaranteed by PrimeTime. When the expected ptxr executables cannot be located within the user specified root path, PrimeTime will fall back and proceed with the natively installed program.

This variable is intended for use only when the natively installed ptxr programs do not work with certain files of supported formats. Most often it happens when trying to read files generated by a newer version of synopsys tool such as DesignCompiler or PhysicalCompiler.

The following example uses the *ptxr_root* variable to specify an alternative ptxr program.

```
pt_shell> set ptxr_root /tools/DC2004.12-SP1/snps/synopsys
/tools/DC2004.12-SP1/snps/synopsys
pt_shell> read_ddc new_design.ddc
Information: Using user set ptxr_root
 '/tools/DC2004.12-SP1/snps/synopsys'.
Beginning read_ddc...
 ...
```

Feedback

Note: When this variable is set, all downstream file reading that requires ptxr will use the reader from the specified path unless you explicitly set it to the default.

See Also

- read_ddc
- read_lib
- ptxr_setup_file

---

ptxr_setup_file

Specifies the path to a setup file to be read by the PrimeTime External Reader (ptxr) instead of home and local .synopsys_dc.setup files.

Data Types

string

Default By default, this variable does not exist.

Description

Set this variable to a setup file for the PrimeTime external reader (ptxr). By default, this variable does not exist until you set it with a value.

This variable is used only when you read a netlist with the PrimeTime external reader by running one of the following commands:

- *read_verilog -hdl_compiler*
- *read_vhdl*
- *read_ddc*

The PrimeTime external reader is the same as the reader used by Design Compiler. When you execute one of the preceding commands, the PrimeTime external reader by default reads your .synopsys_dc.setup files (not .synopsys_pt.setup), including the system, home, and local setup files, to access needed variables. You cannot disable reading of system .synopsys.dc.setup files, but you can substitute a ptxr-specific setup file for the home and local setup files by setting the *ptxr_setup_file* variable.

You write the ptxr setup file in Tcl. The file can contain a very limited set of commands: comments, blank lines, and variable assignments. For example:

```
# My ptxr_setup_file
set bus_naming_style "%s(%d)"
set bus_extraction_style "%s[%d:%d]"
```

Feedback

The following example shows how to use the ptxr setup file when reading a Verilog netlist with ptxr:

```
pt_shell> set ptxr_setup_file my_ptxr.setup
my_ptxr.setup
pt_shell> read_verilog -hdl_compiler module1.v
 ...
```

To discontinue using the ptxr setup file, unset the *ptxr_setup_file* variable:

```
pt_shell> unset ptxr_setup_file
```

The ptxr_setup_file in PrimteTime also supports Library Compiler i.e. using this interface, users will be able to set LC specific settings in a file, pass it to this variable in PrimeTime and then file will used by Library Compiler called in PrimeTime session.

The following example describes how user can provide the settings :

```
lc_user.txt file :
set lc_enable_pad_pin_in_macro_cell true

In Primetime tcl:

set ptxr_setup_file lc_user.txt
read_lib ...
```

Here read_lib will use lc variables when it calls lc_shell based on lc_user.txt file. Moreover, this will not affect PrimeTime run.

See Also

- read_ddc
- read_verilog

q

query_objects_format

Data Types

string

Default application specific

Description

This variable sets the format that the *query_objects* command uses to print its result. There are two supported formats: Legacy and Tcl.

Feedback

The Legacy format looks like this:

```
{"or1", "or2", "or3"}
```

The Tcl format looks like this:

```
{or1 or2 or3}
```

Please see the man page for *query_objects* for complete details.

See Also

- query_objects

r

rc_adjust_rd_when_less_than_rnet

Enables or disables overriding of library-derived drive resistance when it is much less than the dynamic RC network impedance to ground.

Data Types

Boolean

Default true

Description

When this variable is set to *true* (the default), the tool checks the library-derived drive resistance, and if it is less than the dynamic RC network impedance to ground by an amount equal to or greater than the threshold value contained in the *rc_rd_less_than_rnet_threshold* variable (default 0.45), PrimeTime adjusts the drive resistance using an empirical formula. To disable the checking and adjustment, set the *rc_adjust_rd_when_less_than_rnet* variable to *false*.

When the library-derived drive resistance is much less than the dynamic RC network impedance to ground, the behavior of the resistor-based driver model can deviate from that of transistors. In this case, the tool replaces the drive resistance with a value obtained by using an empirical formula to improve accuracy, and issues the RC-009 message. This entire process of checking, detection, replacement, and issuing of the message is referred to as the "RC-009 condition".

Feedback

This variable is one of a set of four variables relevant to the RC-009 condition. The other three are effective only when *rc_adjust_rd_when_less_than_rnet* is true, and are as follows:

- *rc_degrade_min_slew_when_rd_less_than_rnet* determines whether or not slew degradation is used in min analysis mode when the RC-009 condition occurs. The default is false. To use slew degradation during the RC-009 condition, set this variable to true.
- *rc_filter_rd_less_than_rnet* determines whether the RC-009 message is issued only when a network delay is greater than the corresponding driver transition time. The default is true. To receive RC-009 messages every time PrimeTime overrides the drive resistance, set this variable to false.
- *rc_rd_less_than_rnet_threshold* specifies the threshold beyond which PrimeTime overrides the library-derived drive resistance with an empirical formula. The default is 0.45. If there is a reason why the default is not appropriate in your situation, you can set this variable to another value.

See Also

- rc_degrade_min_slew_when_rd_less_than_rnet
- rc_filter_rd_less_than_rnet
- rc_rd_less_than_rnet_threshold
- RC-009

---

rc_always_use_max_pin_cap

Specifies whether to use the pin capacitances from the minimum or maximum library during minimum RC delay calculation.

Data Types

Boolean

Default false

Description

To specify which pin capacitances are used during minimum RC delay calculation, set the *rc_always_use_max_pin_cap* variable to one of these values:

- *false* (the default) - Uses the pin capacitances from the minimum library. • *true* - Uses the pin capacitances from the maximum library.

Feedback

See Also

- set_min_library

---

rc_cache_min_max_rise_fall_ceff

Specifies whether to cache min/max rise/fall values of effective capacitance computed during RC delay calculation.

Data Types

Boolean

Default false

Description

Specifies whether to cache min/max rise/fall values of effective capacitance computed during RC delay calculation. These cached values can be queried via attributes on driver pins and ports with driving cells.

If you set the *rc_cache_min_max_rise_fall_ceff* variable to *true* before a timing update, the tool caches the effective capacitance (Ceff) associated with propagated driver behavior according to min/max and rise/fall characteristics.

To return the cached values, query the following driver pin or port attributes:

- *cached_ceff_max_rise*
- *cached_ceff_min_rise*
- *cached_ceff_max_fall*
- *cached_ceff_min_fall*

These cached attributes are useful for obtaining the worst-case effective capacitance for every driver in the design. The values of the cached attributes depend on the selected slew-propagation mode.

Notes:

- If the *rc_cache_min_max_rise_fall_ceff* variable is *false* during the most recent timing update, the cached values are not created or updated. This means that the attribute values might be nonexistent or invalid.
- Other effective capacitance attributes -- that is, *effective_capacitance_min*, *effective_capacitance_max*, *ceff_params_min*, and *ceff_params_max*) -- are computed at query time; therefore returning those values takes considerably more runtime.

Feedback

The following example shows how to use the attribute query results only when the attributes exist.

```
        set ceff \\
[get_attribute -quiet $obj ceff_min_rise]
if {[string length $ceff] != 0} {
  ...
}
```

The cached values are removed only when you execute the *remove_annotated_parasitics* command or when a netlist editing command has similar reason to remove annotated parasitics.

See Also

- remove_annotated_parasitics

---

rc_ccs_extrapolation_range_compatibility

Enables enhanced CCS extrapolation.

Data Types

Boolean

Default true

Description

Set this variable to one of these values:

- *true* (the default) - Disables the enhanced CCS extrapolation feature.
- *false* - Enables enhanced extrapolation for CCS library data when any extrapolation occurs.

See Also

- timing_max_capacitance_derate
- timing_max_transition_derate
- RC-011

---

rc_ccsn_dc_current_limit_in_ma

Sets the maximum DC current threshold for the CCS noise model limit check.

Feedback

Data Types

float

Default 10000

Description

This variable sets the maximum DC current threshold (in mA) for the CCS noise model limit check.

Unrealistically large current values can cause inaccurate delay calculation. When a CCS noise model fails this current limit check, it is invalidated and ignored in delay calculation.

This check is performed during library loading, but only when the *rc_ccsn_enable_library_error_reporting* variable is set to *true*.

See Also

- PTLIB-009
- rc_ccsn_enable_library_error_reporting

---

rc_ccsn_enable_library_error_reporting

Enable CCS noise model checking during library loading.

Data Types

Boolean

Default false

Description

This variable enables CCS noise model checking during library loading.

The default is *false* (checks are disabled).

When set to *true*, the tool performs CCS noise model checks (such as NLDM consistency and model integrity checks) as libraries are loaded.

If a model fails the check, it is invalidated and not used in delay calculation. Such issues are reported with PTLIB-009 error messages.

If library scaling groups are being used, invalidated CCS noise models can cause scaling group inconsistencies, resulting in SLG-403 errors.

Feedback

See Also

- PTLIB-009
- SLG-403

---

rc_ceff_use_delay_reference_at_cpin

Specifies whether to compute C-effective using a driver delay relative to that for the output pin capacitance.

Data Types

boolean

Default false

Description

PrimeTime adjusts C-effective to match library and RC-network delays to within a small percentage. When the library delay is very large, as would be the case for a driver with many internal stages (such as an extracted timing model), this criterion can be met without a sufficient number of C-effective iterations.

If you set *rc_ceff_use_delay_reference_at_cpin* to *true*, then PrimeTime excludes the portion of the library delay due to output pin capacitance from the C-effective convergence criterion. This allows a sufficient number of iterations to occur.

Note: if *rc_ceff_use_delay_reference_at_cpin* is *true*, then the library data must be characterized all the way down to the output pin capacitance.

---

rc_degrade_min_slew_when_rd_less_than_rnet

Enables or disables the use of slew degradation in min analysis mode during the RC-009 condition.

Data Types

Boolean

Default false

Description

When this variable is *false* (the default), the tool does not use slew degradation through RC networks in min analysis mode during the RC-009 condition. When *true*, the tool uses slew degradation during the RC-009 condition. This variable is effective only if the *rc_adjust_rd_when_less_than_rnet* variable is *true*.

Feedback

The "RC-009 condition" means a condition in which the tool checks the library-derived drive resistance, and if it is less than the dynamic RC network impedance to ground by an amount equal to or greater than the value of the *rc_rd_less_than_rnet_threshold* variable, the tool adjusts the drive resistance using an empirical formula to improve accuracy, and issues the RC-009 message. In case this improved accuracy is not sufficient, the tool provides extra pessimism by not using slew degradation in min analysis mode; however, superfluous min delay violations could occur as a side effect. You can keep slew degradation on in min analysis mode after you have qualified the RC-009 methodology for your accuracy requirements, by setting this variable to true.

rc_degrade_min_slew_when_rd_less_than_rnet is one of a set of four variables relevant to the RC-009 condition. The other three are as follows:

- *rc_adjust_rd_when_less_than_rnet* enables or disables the RC-009 condition; the default is true. When this variable is set to false, PrimeTime does not check the drive resistance, and the values of the other related variables do not matter.
- *rc_filter_rd_less_than_rnet* determines whether the RC-009 message is issued only when a network delay is greater than the corresponding driver transition time. The default is true. To receive RC-009 messages every time PrimeTime overrides the drive resistance, set this variable to false. This variable has no effect if *rc_adjust_rd_when_less_than_rnet* is false.
- *rc_rd_less_than_rnet_threshold* specifies the threshold beyond which PrimeTime overrides the library-derived drive resistance with an empirical formula. The default is 0.45 ohms. You can override this default by setting the variable to another value. This variable has no effect if *rc_adjust_rd_when_less_than_rnet* is false.

Note: If *rc_degrade_slew_when_rd_less_than_rnet* is false while *rc_filter_rd_less_than_rnet* is true, the RC-009 message is not issued.

See Also

- rc_adjust_rd_when_less_than_rnet
- rc_filter_rd_less_than_rnet
- rc_rd_less_than_rnet_threshold
- RC-009

---

rc_degrade_wlm_net_slew_based_on_delay

Calculates the net slew degradation for wire load model based on net delay if the library does not have slew degradation data.

Feedback

Data Types

Boolean

Default false

Description

By default, PrimeTime does not compute the net slew degradation for wire load model if the library does not have slew degradation data.

If you set this variable to *true*, PrimeTime calculates the net slew degradation for wire load model based on net delay if library does not have slew degradation data.

See Also

- report_delay_calculation
- report_timing
- update_timing

---

rc_driver_model_mode

Specifies the driver model type for RC delay calculation.

Data Types

string

Default advanced

Description

To specify the driver model type for RC delay calculation, set the *rc_driver_model_mode* variable to one of these values:

- *basic* - Uses the driver models derived from the conventional delay and slew schema present in design libraries.
- *advanced* - Uses the advanced driver model if Composite Current Source (CCS) data is available. The *report_delay_calculation* command used on a cell arc shows the "Advanced driver modeling" message.

If the *rc_driver_model_mode* variable is set to *basic*, and the variable *rc_receiver_model_mode* is set to *advanced*, the tool uses the advanced voltage-dependent capacitance models to derive an equivalent single capacitance dependent only on the rise, fall, minimum, or maximum arc condition. These equivalent capacitances are used in analysis instead of the pin capacitances from the library.

Feedback

See Also

- report_delay_calculation
- rc_receiver_model_mode

---

rc_fallback_coupling_cap_multiplier

Set the multiplier for coupling capacitance in case of detailed delay calculation failure and fallback to NLDM delay lookup with lumped cap for max analysis.

Data Types

float

Default 1

Description

If detailed CCS delay calculation fails, such as due to a voltage mismatch between driver and load, the tool might revert to NLDM delay calculation with a lumped capacitance value.

When this failure occurs, crosstalk delay calculation is not performed.

For min-delay analysis, a load capacitance or zero is used, which is highly conservative and bounds the possible crosstalk delta delay.

For max-delay analysis, the total capacitance is used. This total includes the coupling capacitance multiplied by the factor specified by this variable. The default multiplier is 1.0, which is also conservative, but the calculation can be made more conservative by increasing the value of the multiplier. The theoretical upper bound for crosstalk delta delay is achieved with a multiplier of 3, but it could result in highly pessimistic results for max-delay analysis.

This variable has effect only when SI analysis is enabled by setting the *si_enable_analysis* variable to *true*.

See Also

- RC-104
- RC-004

---

rc_filter_rd_less_than_rnet

Enables or disables the display of RC-009 messages when the network delay is less than the corresponding driver transition time.

Feedback

Data Types

Boolean

Default true

Description

To control the display of RC-009 messages, set the *rc_filter_rd_less_than_rnet* variable to one of these values:

- *true* (the default) - Displays the RC-009 message only when a network delay is greater than the corresponding driver transition time.
- *false* - Displays the RC-009 message whenever it overrides the library-derived drive resistance during the RC-009 condition.

This variable is effective only if the *rc_adjust_rd_when_less_than_rnet* variable is *true*.

The "RC-009 condition" means a condition in which PrimeTime checks the library-derived drive resistance. If the drive resistance is less than the dynamic RC network impedance to ground by an amount equal to or greater than the value of the *rc_rd_less_than_rnet_threshold* variable, the tool replaces the drive resistance with a value obtained by using an empirical formula to improve accuracy and issues the RC-009 message. The filtering provided by *rc_filter_rd_less_than_rnet* isolates those timing calculations known to be most sensitive to drive resistance. The network delay is not compared with the slew itself, but with the time the driver reaches its later slew trip point.

In addition to the *rc_filter_rd_less_than_rnet* variable, the following variables also affect the RC-009 condition:

- *rc_adjust_rd_when_less_than_rnet*
- *rc_degrade_min_slew_when_rd_less_than_rnet*
- *rc_rd_less_than_rnet_threshold*

See Also

- rc_adjust_rd_when_less_than_rnet
- rc_degrade_min_slew_when_rd_less_than_rnet
- rc_rd_less_than_rnet_threshold
- RC-009

Feedback

rc_rd_less_than_rnet_threshold

Specifies the RC-009 threshold, beyond which the tool overrides the library-derived drive resistance with an empirical formula, to improve accuracy.

Data Types

float

Default 0.45

Description

This variable specifies a metric threshold to be used by PrimeTime to determine whether to adjust the library-derived drive resistance using an empirical formula; see the RC-009 man page for more information about this condition. This variable is effective only if the *rc_adjust_rd_when_less_than_rnet* variable is *true*.

To determine the value appropriate for a given technology, look at PrimeTime accuracy versus drive strength for cells connected to very resistive networks, such as top-level routes, with *rc_adjust_rd_when_less_than_rnet* set false. At a particular drive strength, the accuracy suffers due to the limitation in the delay/slew schema that RC-009 addresses. Then set *rc_adjust_rd_when_less_than_rnet* true and use a value of *rc_rd_less_than_rnet_threshold* high-enough to be used for all the drivers (such as, 1.0). Next, gradually decrease *rc_rd_less_than_rnet_threshold* until RC-009 switches off at the specified drive strength.

As technology shrinks, so do drive resistances, causing an increased occurrence of RC-009. In that case, you can decrease the *rc_rd_less_than_rnet_threshold* variable or switch to using Composite Current Source (CCS) data for delay calculation.

See Also

- rc_degrade_min_slew_when_rd_less_than_rnet
- rc_filter_rd_less_than_rnet
- rc_rd_less_than_rnet_threshold
- RC-009

rc_receiver_disable_cond_default_arcs

Disables default (unconditional) timing arcs that exist alongside conditional arcs from receiver model use.

Data Types

boolean

Feedback

Default false

Description

This variable pertains to default (unconditional) arcs between any pair of pins that also has at least one conditional arc.

When set to *true*, this variable disables these default arcs from receiver model use; only the conditional-arc receiver models are considered.

When set to *false* (the default), receiver models from all of the arcs - default and conditional - are considered, which is conservative.

If the related *timing_disable_cond_default_arcs* variable is set to *true* (its default is *false*), this variable is ignored and the default-arc disabling behavior is applied to both timing arcs and arc-based receiver models.

Some libraries have a pessimistic default arc alongside conditional arcs. If the conditional arc *when* conditions cover all possible state-dependent delays for all cells, set this variable to *true* for improved accuracy.

For example, consider a 2-input XOR gate with inputs A and B and output Z. If the delays between A and Z are specified with two arcs with respective conditions 'B' and 'B~', a default arc between A and Z is not needed and should be disabled.

See Also

- report_delay_calculation
- rc_receiver_model_mode
- timing_disable_cond_default_arcs

---

rc_receiver_enable_library_error_reporting

Enables CCS C1CN receiver model checks.

Data Types

Boolean

Default false

Description

This variable enables CCS C1CN receiver model checks for issues, such as for negative capacitance segment values and glitches.

Feedback

The checks are disabled by default, but they can be enabled by setting this variable to *true*. When the checks are enabled, issues are reported with PTLIB-010 messages as the libraries are loaded into memory.

See Also

- PTLIB-010

---

rc_receiver_model_mode

Specifies the receiver model type for RC delay calculation.

Data Types

string

Default advanced

Description

To specify the receiver model type for RC delay calculation, set the *rc_receiver_model_mode* variable to one of these values:

- *basic* - Uses the pin capacitances specified in the design libraries. The basic model is a single capacitance dependent only on the rise, fall, minimum, or maximum arc condition.
- *advanced* - Uses the advanced receiver model if data for it is present, and if the network is driven by the advanced driver model. The advanced model is a voltage-dependent capacitance additionally dependent on input-slew and output capacitance. The advanced receiver model is part of the Synopsys Composite Current-Source (CCS) model. The *report_delay_calculation* command used on a network arc shows the "Advanced receiver modeling" message.

The advanced model has many advantages, such as the improvement accuracy of delays and slews. Another advantage is that nonlinearities, such as the Miller effect, are addressed.

When the *rc_receiver_model_mode* variable is set to *advanced*, and the network is not driven by the advanced driver model -- for example, the variable *rc_driver_model_mode* is set to *basic* or lumped load is used -- the tool uses the advanced voltage-dependent capacitance models to derive an equivalent single capacitance dependent only on the rise, fall, minimum, or maximum arc condition. These equivalent capacitances are used in analysis instead of the pin capacitances from the library. The *report_delay_calculation* command used on a network arc does not show the "Advanced receiver modeling" message for these calculations, since only an equivalent single capacitance is used.

Feedback

See Also

- report_delay_calculation
- rc_driver_model_mode

---

rc_receiver_modeling_effort

Controls the computational effort for receiver modeling algorithm in PrimeTime.

Data Types

string

Default high

Description

When the *delay_calc_enhanced_ccsn_waveform_analysis* variable is set to *true* to enable enhanced waveform propagation analysis, PrimeTime performs adaptive receiver modeling to integrate CCS noise models with library receiver models (C1CN).

This variable controls how much effort is used for receiver modeling. Available options for this variable are *high* (the default) and *low*.

The default value of *high* provides the best accuracy. Normally, receiver modeling takes only a small fraction of the timing update runtime and the computational effort is minimal.

However, for large scaling library groups, the runtime for receiver modeling can be higher. To reduce the runtime in this case, you can set this variable to *low*.

This variable has an effect only when enhanced waveform propagation is enabled:

```
# enable advanced waveform propagation (default is disabled)
set_app_var delay_calc_waveform_analysis_mode full_design

# enable enhanced advanced waveform propagation (default is true)
set_app_var delay_calc_enhanced_ccsn_waveform_analysis true
```

See Also

- update_timing
- delay_calc_enhanced_ccsn_waveform_analysis
- delay_calc_waveform_analysis_mode

Feedback

read_parasitics_load_locations

Specifies that *read_parasitics* should load location information during the reading of a parasitics file.

Data Types

Boolean

Default false

Description

When this variable is set to *true*, *read_parasitics* loads the locations of various nodes of nets, pins, and ports that are present in the parasitic file. The default is *false*, in which case location data is not be loaded into PrimeTime.

The location data is stored using attributes. The attributes are set to the coordinate value directly from the parasitics files, and no interpretation or unit conversion is performed. The following attributes are available on pin and port objects:

- x_coordinate (float)
- y_coordinate (float)

These attributes define a single (x, y) point. The following attributes are available on cell and net objects.

- x_coordinate_min (float)
- x_coordinate_max (float)
- y_coordinate_min (float)
- y_coordinate_max (float)

These attributes define a bounding box around the cell or net. For cells, the bounding box is computed using all pins of the cell. For nets, the bounding box is computed using all net terminals (port and pins). If the parasitics file includes coordinates for intermediate modes, these are also considered for the net's bounding box.

If location data has been loaded, the data is included in any parasitics files written out by PrimeTime. If you remove the parasitics from a net (using the *remove_annotated_parasitics* command, for example), PrimeTime also deletes the location data.

See Also

- read_parasitics

Feedback

report_capacitance_use_ccs_receiver_model

Specifies whether the basic or advanced receiver model is used to report receiver pin capacitance.

Data Types

Boolean

Default true

Description

When this variable is set to *true*, the advanced CCS receiver model is used to report receiver pin capacitance by the *report_net*, *report_attribute*, *report_delay_calculation*, and *report_timing* commands. When it is set to *false*, the basic library-derived lumped pin capacitance is used. The variable setting only affects pin capacitance reporting, not delay calculation.

See Also

- report_timing
- report_net
- report_delay_calculation

report_default_significant_digits

Specifies the default number of significant digits used to display values in reports.

Data Types

integer

Default 2

Description

Sets the default number of significant digits for many reports. Allowed values are 0-13; the default is 2. Some report commands (for example, the *report_timing* command) have a *-significant_digits* option that overrides the value of this variable.

Not all reports respond to this variable. Check the man page of individual reports to determine whether they support this feature.

See Also

- report_timing

Feedback

report_use_out_of_table_range

Controls whether the *report_constraint* command uses library table bounds for transition and capacitance limit checking.

Data Types

Boolean

Default false

Description

This variable controls whether the *report_constraint* command uses library table bounds for the following DRC checks:

```
max_capacitance
max_transition
min_capacitance
min_transition
```

When this variable is set to *false* (the default), the *report_constraint* command performs the *max_capacitance*, *max_transition*, *min_capacitance*, and *min_transition* DRC checks using requirements from the following sources:

- Library-defined DRC requirements
- User-defined DRC requirements
  - Applied to library pins
  - Applied to pins and ports
  - Applied to clock objects
  - Applied to the current design

When multiple requirements apply, the most restrictive requirement is used. This is the default tool operation.

When this variable is set to *true*, the DRC requirements from the sources above are ignored. Instead, the DRC check requirements are derived entirely from the library lookup table bounds:

- *max_capacitance* - upper bound of capacitance-indexed tables
- *max_transition* - upper bound of transition-indexed tables
- *min_capacitance* - lower bound of capacitance-indexed tables
- *min_transition* - lower bound of transition-indexed tables

Feedback

This behavior allows the *report_constraint* command to be used to efficiently identify and report extrapolations of library data that could affect delay calculation accuracy.

Note that user-defined requirements applied with the *-force* option of the *set_max_transition* or *set_max_capacitance* command are always considered, regardless of the value of this variable.

See Also

- report_constraint
- set_max_capacitance
- set_max_transition
- set_min_capacitance

S

scaling_allow_pre_link

Enables the define_scaling_lib_group command to be used before link.

Data Types

Boolean

Default true

Description

When this variable is *true* (the default), the *define_scaling_lib_group* command can be run before the design is linked if an exact-match scaling group is being defined. For example,

```
# define link library
set_app_var link_path {* lib_1.05V.db}

# define scaling library group
define_scaling_lib_group \
  -exact_match \
  {lib_0.9V.db lib_1.05V.db lib_1.3V.db}

# read and link design
read_verilog TOP.v
link_design TOP
```

This command ordering allows the allows the design link process to be aware of any extra arcs in the scaling libraries that are not in the link library, so that they are handled correctly during analysis.

Feedback

Mismatched timing arc support is limited to exact-match scaling flows (voltage conditions exactly match library conditions). True scaling flows (with interpolation between libraries) still require consistent libraries within each group, with the scaling group defined after design link.

When this variable is *false*, if the *define_scaling_lib_group* command is run before the design is linked, the command fails with an SLG-302 error. This matches the behavior of PrimeTime releases prior to the T-2022.03 release.

See Also

- define_scaling_lib_group

---

scaling_calculation_compatibility

Enables the usage of 2016.06 and earlier scaling algorithms which is less accurate.

Data Types

Boolean

Default false

Description

Set this variable to one of these values:

- *true* - Disables the more accurate new library scaling, use the old scaling method of 2016.06 and earlier releases.
- *false* (the default) - Enables new nonlinear scaling optimization to achieve better accuracy, particularly for low voltage regions. Also enables a better handling of the mismatched slew index range across different libraries, this could reduce the number of RC-011s due to mismatched slew index ranges.

See Also

- define_scaling_lib_group
- RC-011

---

scaling_disable_arc_consistency_check_for_exact_match

Controls whether arc consistency checks are disabled or enabled for exact-match scaling library groups.

Feedback

Data Types

Boolean

Default false

Description

This variable controls whether library arc consistency checks are applied between libraries in exact-match scaling groups (defined using the *-exact_match_only* option of the *define_scaling_lib_group* command).

When set to *false* (the default), arc consistency checks are applied (not disabled) in exact-match scaling library groups.

When set to *true*, arc consistency checks are disabled in exact-match scaling library groups.

See Also

- define_scaling_lib_group

---

scaling_enable_performance_mode

Specifies the different modes for improving scaling runtime.

Data Types

Boolean

Default true

Description

This variable needs to be set before the define_scaling_lib_group commands or linking. When define_scaling_lib_group commands are used, this variable changes flow/algorithms to improve runtime performace of true scaling but has no impact on exact_match. Especially, this variable can have big runtime improvement for the scaling group with large scaling library number. It can have a little scaling QoR impact but it won't degrate scaling accruacy.

Set this variable to one of these values:

- *false* - Use the default flow and no Algorithm/flow changes.
- *true* (the default) - Enables the flow, data, and algorithm changes to speed up scaling calculation runtime, particularly for update_timing.

Feedback

See Also

- define_scaling_lib_group
- scaling_calculation_compatibility

---

sdc_name_map

Specifies a list of {design_name name_map_file} to be used during golden constraint reading.

Data Types

list

Default NULL

Description

This variable specifies a list in the format of {*design_name name_map_file*} to be used when golden constraint is reapplied to a post synthesis netlist. The *name_map_file* is the name map file for the *design_name* design. The specified *name_map_file* guides the golden constraint reader on the *design_name* design.

See Also

- load_constraints
- enable_golden_constraints_reader

---

sdc_save_source_file_information

Enables or disables storage of file name and line number information for SDC commands read in from script files.

Data Types

Boolean

Default false

Feedback

Description

Setting this variable to *true* enables PrimeTime to preserve source location information, namely the source file name and line number. The source location tracking applies to all SDC commands defined per the SDC standard, such as:

- *set_false_path*
- *set_multicycle_path*
- *set_max_delay*
- *set_min_delay*

You can change the setting of this variable only if no exceptions have been applied. If at least one exception command has already been successfully applied, attempting to set this variable results in a CMD-013 error message, and the variable value remains unchanged.

Source information is not available for any commands that were not input using the *source* or *read_sdc* command. (Tcl files sourced using the *-f* command line option are internally processed through the *source* command, so they are supported.) Commands entered interactively at the shell prompt do not store source location data. Also, commands applied inside control structures, such as "if" statements, loops, or procedure calls, are not accurately tracked.

Note that this information is not saved by the *save_session* command.

This location data per exception command can be viewed using either the *report_exceptions* or *report_timing -exceptions* command.

Examples

Here is an example of the timing exceptions report using the *report_exceptions* command.

```
pt_shell> report_exceptions

****************************************
Report : exceptions
 ...
****************************************

Reasons :   f  invalid start points
            t  invalid end points
            p  non-existent paths
            o  overridden paths

From                To                    Setup          Hold
------------------------------------------------------------------------------
arbiter/lat_reg/CK  { arbiter/state_reg_3_/D arbiter/state_reg_2_/D }
                                          cycles=2       *
         [ location = /path/constraints.tcl:197 ]
```

Here is an example of the timing exceptions report using the *report_timing -exceptions* command.

```
pt_shell> report_timing -exceptions all

****************************************
Report : timing
        -path_type full
        -delay_type max
        -max_paths 1
        -exceptions all
 ...
****************************************

[ Timing report omitted. ]

The dominant exceptions are:
From          To             Setup                    Hold
-------------------------------------------------------------------------------

arbiter/lat_reg/CK
              { arbiter/state_reg_3_/D arbiter/state_reg_2_/D }
                             cycles=2                 *
         [ location = /path/constraints.tcl:197 ]

The overridden exceptions are:
        None
```

Here is an example of the SDC file written by the *write_sdc out.sdc -include {sourcefile_refs}* command.

```
pt_shell> write_sdc out.sdc -include {sourcefile_refs}

Information: Performing logical update. (UITE-499)
Information: Building multi voltage information for entire design.
 (MV-022)
Information: SDC supports a subset of the constraints supported by
 PrimeTime. Some constraints cannot be preserved by the write_sdc
 command. (WSCR-002)
```

See Also

- report_exceptions
- report_timing
- write_sdc

---

sdc_version

Specifies the Synopsys Design Constraints (SDC) version that was written.

Feedback

Data Types

string

Default 2.0

Description

The *sdc_version* variable is meaningful only within the context of reading an SDC file. Setting it outside an SDC file has no impact, other than to produce an informational message.

The *write_sdc* command writes a command to the SDC file to set the *sdc_version* variable to the version that was written. There is no user control over the version of SDC that is written. The most current version is written. When the *read_sdc* command reads the SDC file, it validates the version specified in the file (if present) with the version requested by the command.

See Also

- read_sdc
- write_sdc

---

sdc_write_unambiguous_names

Specifies whether ambiguous hierarchical names are made unambiguous when they are written to SDC files.

Data Types

Boolean

Default true

Description

If this variable is set to *true* (the default), the application ensures that cell, net, pin, lib_cell, and lib_pin names written to the Synopsys Design Constraints (SDC) file are not ambiguous. When hierarchy has been partially flattened, embedded hierarchy separators can make names ambiguous, so that it is unclear which hierarchy separator characters are part of the name, and which are real separators.

Beginning with SDC Version 1.2, hierarchical names can be made unambiguous using the *set_hierarchy_separator* SDC command and the *-hsc* option of the following SDC object access commands:

```
get_cells
get_lib_cells
get_lib_pins
```

Feedback

get_nets
get_pins

By default, PrimeTime and Design Compiler writes an SDC file using these features to create unambiguous names.

The recommended practice is to accept the default behavior and allow the application to write SDC files that do not contain ambiguous names. However, if you are using a third-party application that does not support the unambiguous hierarchical names feature of SDC (in SDC Versions 1.2 and later), you can suppress this feature by setting the *sdc_write_unambiguous_names* variable to *false*. The *write_sdc* command issues a warning if you set this variable to *false*.

See Also

- write_sdc

---

sdf_align_multi_drive_cell_arcs

Specifies whether PrimeTime unifies the small timing differences in driver cell outputs of a parallel network when writing out SDF delay values.

Data Types

Boolean

Default false

Description

Small timing differences in the timed switching characteristics of the mesh arcs can cause the simulation to fail. If you set the *sdf_align_multi_drive_cell_arcs* variable to *true*, PrimeTime attempts to align the delays between the driver pins of the parallel network and the load pins of the network. The cell and net delay arcs written to the SDF file is adjusted to make this happen. The net arcs are only altered if you set the *sdf_enable_port_construct* variable to *true*. Therefore, to eliminate the small timing differences, you must set both the *sdf_align_multi_drive_cell_arcs* and *sdf_enable_port_construct* variables to *true*, and the following criteria must be true:

1. All cells have to be nonsequential, nonhierarchical cells.
2. All cells must be single input, single output devices.
3. None of the drivers of the parallel network are three-state buffers.

Feedback

See Also

- sdf_align_multi_drive_cell_arcs_threshold
- sdf_enable_port_construct
- sdf_enable_port_construct_threshold

---

sdf_align_multi_drive_cell_arcs_threshold

Specifies the threshold below which multidrive cell arcs are aligned during the *write_sdf* command.

Data Types

float

Default 1

Description

Small timing differences in the timed switching characteristics of the mesh arcs can cause the simulation to fail. If you set the *sdf_align_multi_drive_cell_arcs* variable to *true*, PrimeTime attempts to unify the delays between the driver pins of the parallel network and the load pins of the network. The cell delay arcs written to the Standard Delay Format (SDF) file is adjusted to make this happen. The criteria for this to occur is that the delay values of the parallel cells are within a threshold of each other, where the threshold is specified by the *sdf_align_multi_drive_cell_arcs_threshold* variable. The threshold value is specified in pico seconds (ps), where the default value is 1 ps.

See Also

- sdf_align_multi_drive_cell_arcs_threshold
- sdf_enable_port_construct
- sdf_enable_port_construct_threshold

---

sdf_always_include_preset_clear_arcs

Enables preset and clear delay calculation in write_sdf.

Data Types

Boolean

Default false

Feedback

Description

If you set this variable to *true* then *write_sdf* will calculate the delay values of preset and clear timing arcs, even when the value of variable *timing_enable_preset_clear_arcs* is *false*.

See Also

- write_sdf
- timing_enable_preset_clear_arcs

---

sdf_enable_cond_start_end

Enables or disables support for the *sdf_cond_start* and *sdf_cond_end* attributes.

Data Types

Boolean

Default false

Description

If you set this variable to *true*, PrimeTime supports the *sdf_cond_start* and *sdf_cond_end* attributes on timing arcs. These attributes impact the way the *read_sdf* and *write_sdf* commands deal with timing arcs.

See Also

- read_sdf

---

sdf_enable_port_construct

Enables or disables support for port construct usage during the *write_sdf* command.

Data Types

Boolean

Default false

Description

For designs with high-fanin, high-fanout mesh clock networks, large Standard Delay Format (SDF) files are produced. If you set the *sdf_enable_port_construct* variable to *true*, PrimeTime attempts to reduce the size of the produced SDF file. PrimeTime uses the port construct instead of the interconnect construct when executing the *write_sdf* command.

Feedback

Using the port construct is restricted by the *sdf_enable_port_construct_threshold* variable. Any group of parallel nets in the design that are not driven or driving three-state buffers and that have a delay value within the threshold as defined by the *sdf_enable_port_construct_threshold* variable is written out using a port construct. Otherwise, the interconnect construct is used. The port construct is not used on nets outside the clock network. It must be noted that the produced SDF file can contain both port and interconnect statements for a given load pin. In this case, the port statement is written out first, followed by interconnect statements.

See Also

- sdf_enable_port_construct_threshold

---

sdf_enable_port_construct_threshold

Sets the threshold value for the parallel net arcs delay variance below which parallel nets are written out using the port construct during the *write_sdf* command.

Data Types

float

Default 1

Description

For designs with high-fanin, high-fanout mesh clock networks, large Standard Delay Format (SDF) files are produced. If you set the *sdf_enable_port_construct* variable to *true*, PrimeTime attempts to reduce the size of the produced SDF file. The *sdf_enable_port_construct_threshold* variable provides a maximum value for the parallel net arcs delay variance below which parallel nets are written out using the port construct.

The threshold value is specified in picoseconds (ps).

See Also

- sdf_enable_port_construct

---

search_path

Specifies a list of directories that contain design files, library files, and scripts.

Data Types

list

Default "" (empty)

Feedback

Description

This variable specifies a list of directories where PrimeTime searches for design files, library files, and scripts. Normally, the *search_path* variable is set to a central library directory.

The default of this variable is the empty string, "".

The *read_db* and *link_design* commands particularly depend on the *search_path* variable.

The *source* command searches for scripts using the *search_path* variable if you set the *sh_source_uses_search_path* variable to *true*.

See Also

- link_design
- read_db
- source
- sh_source_uses_search_path

---

selection_logging_no_core_action

This variable determines how the *change_selection_no_core* command behaves.

Data Types

String

Default ""

Description

This variable determines how the Tcl command *change_selection_no_core* behaves.

See Also

- change_selection_no_core

---

selection_logging_too_many_objects_action

This variable determines how the *change_selection_too_many_objects* command behaves.

Data Types

String

Feedback

Default ""

Description

This variable determines how the *change_selection_too_many_objects* Tcl command behaves.

See Also

- change_selection_too_many_objects

---

sh_allow_tcl_with_set_app_var

Allows the *set_app_var* and *get_app_var* commands to work with application variables.

Data Types

string

Default application specific

Description

Normally the *get_app_var* and *set_app_var* commands only work for variables that have been registered as application variables. Setting this variable to *true* allows these commands to set a Tcl global variable instead.

These commands issue a CMD-104 error message for the Tcl global variable, unless the variable name is included in the list specified by the *sh_allow_tcl_with_set_app_var_no_message_list* variable.

See Also

- get_app_var
- set_app_var

---

sh_allow_tcl_with_set_app_var_no_message_list

Suppresses CMD-104 messages for variables in this list.

Data Types

string

Default application specific

Feedback

Description

This variable is consulted before printing the CMD-104 error message, if the *sh_allow_tcl_with_set_app_var* variable is set to *true*. All variables in this Tcl list receive no message.

See Also

- get_app_var
- set_app_var

---

sh_arch

Indicates the system architecture of your machine.

Data Types

string

Default platform-dependent

Description

The *sh_arch* variable is set by the application to indicate the system architecture of your machine. Examples of machines being used are sparcOS5, amd64, and so on. This variable is read-only.

---

sh_command_abbrev_mode

Sets the command abbreviation mode for interactive convenience.

Data Types

string

Default application specific

Description

This variable sets the command abbreviation mode as an interactive convenience. Script files should not use any command or option abbreviation, because these files are then susceptible to command changes in subsequent versions of the application.

Although the default value is *Anywhere*, it is recommended that the site startup file for the application set this variable to *Command-Line-Only*. It is also possible to set the value to *None*, which disables abbreviations altogether.

Feedback

To determine the current value of this variable, use the *get_app_var sh_command_abbrev_mode* command.

See Also

- sh_command_abbrev_options
- get_app_var
- set_app_var

---

sh_command_abbrev_options

Turns off abbreviation of command dash option names when false.

Data Types

boolean

Default application specific

Description

When command abbreviation is currently off (see sh_command_abbrev_mode) then setting this variable to false will also not allow abbreviation of command dash options. This variable also impacts abbreviation of the values specified to command options that expect values to be one of an allowed list of values.

This variable exists to be backward compatible with previous tool releases which always allowed abbreviation of command dash options and option values regardless of the command abbreviation mode.

It is recommended to set the value of this variable to false.

To determine the current value of this variable, use the *get_app_var sh_command_abbrev_options* command.

See Also

- sh_command_abbrev_mode
- get_app_var
- set_app_var

---

sh_command_log_file

Specifies the name of the file to which the application logs the commands you executed during the session.

Feedback

Data Types

string

Default empty string

Description

This variable specifies the name of the file to which the application logs the commands you run during the session. By default, the variable is set to an empty string, indicating that the application's default command log file name is to be be used. If a file named by the default command log file name cannot be opened (for example, if it has been set to read only access), then no logging occurs during the session.

This variable can be set at any time. If the value for the log file name is invalid, the variable is not set, and the current log file persists.

To determine the current value of this variable, use the *get_app_var sh_command_log_file* command.

See Also

- get_app_var
- set_app_var

---

sh_continue_on_error

Allows processing to continue when errors occur during script execution with the *source* command.

Data Types

Boolean

Default application specific

Description

It is recommended to use the *-continue_on_error* option to the *source* command instead of this variable because that option only applies to a single script, and not the entire application session.

When set to *true*, the *sh_continue_on_error* variable allows processing to continue when errors occur. Under normal circumstances, when executing a script with the *source* command, Tcl errors (syntax and semantic) cause the execution of the script to terminate.

When *sh_continue_on_error* is set to *false*, script execution can also terminate due to new error and warning messages based on the value of the *sh_script_stop_severity* variable.

Feedback

To determine the current value of the *sh_continue_on_error* variable, use the *get_app_var sh_continue_on_error* command.

See Also

- get_app_var
- set_app_var
- source
- sh_script_stop_severity

sh_deprecated_is_error

Raise a Tcl error when a deprecated command is executed.

Data Types

Boolean

Default application specific

Description

When set this variable causes a Tcl error to be raised when an deprecated command is executed. Normally only a warning message is issued.

See Also

- get_app_var
- set_app_var

sh_dev_null

Indicates the current null device.

Data Types

string

Default platform dependent

Description

This variable is set by the application to indicate the current null device. For example, on UNIX machines, the variable is set to */dev/null*. This variable is read-only.

Feedback

See Also

- get_app_var

---

sh_disabled_is_error

Raise a Tcl error when a disabled command is executed.

Data Types

Boolean

Default application specific

Description

When set this variable causes a Tcl error to be raised when a disabled command is executed. When false, only a warning message is issued.

Disabled commands have no effect.

See Also

- get_app_var
- set_app_var

---

sh_enable_constraint_analysis_mode

Controls whether the internal checker is used for constraint analysis instead of PTC.

Data Types

boolean

Default false

Description

When this variable is set to *true*, PrimeTime uses the internal checker for constraint analysis. In this case, the *check_constraints* command performs rule checking natively within PrimeTime to find potential problems in the design and its constraints.

When this variable is set to *false* (the default), PrimeTime calls PTC to perform the analysis.

See Also

- check_constraints

Feedback

sh_enable_line_editing

Enables the command line editing capabilities in PrimeTime.

Data Types

Boolean

Default true

Description

If this variable set to its default of *true*, advanced UNIX-like shell capabilities are enabled.

This variable needs to be set in the .synopsys_pt.setup file to take effect.

Key Bindings

The *list_key_bindings* command displays current key bindings and the edit mode. To change the edit mode, the *sh_line_editing_mode* variable can be set in either the .synopsys_pt.setup file or directly in the shell.

Command Completion

The editor can complete commands, options, variables, and files given a unique abbreviation. You must type part of a word and press the Tab key to get the complete command, variable, or file. For command options, type -, and press the Tab key to get the options list.

If no match is found, the terminal bell rings. If the word is already complete, a space is added to the end if it isn't already there, to speed typing and provide a visual indicator of successful completion. Completed text pushes the rest of the line to the right. If there are multiple matches, all the matching commands, options, files, or variables are automatically listed.

Completion works in following context sensitive ways:

- The first token of a command line : completes commands
- Token that begins with "-" after a command : completes command arguments
- After a ">", "|" or a "sh" command : completes file names
- After a set, unset or printvar command : completes variables
- After '$' symbol : completes variables
- After the help command : completes command

Feedback

- After the man command : completes commands or variables
- Any token which is not the first token and does not match any of the preceding rules : completes file names

See Also

- list_key_bindings
- sh_line_editing_mode

---

sh_enable_page_mode

Displays long reports one page at a time (similar to the UNIX *more* command.

Data Types

Boolean

Default application specific

Description

This variable displays long reports one page at a time (similar to the UNIX *more* command), when set to *true*. Consult the man pages for the commands that generate reports to see if they are affected by this variable.

To determine the current value of this variable, use the *get_app_var sh_enable_page_mode* command.

See Also

- get_app_var
- set_app_var

---

sh_enable_stdout_redirect

Allows the redirect command to capture output to the Tcl stdout channel.

Data Types

Boolean

Default application specific

Feedback

Description

When set to *true*, this variable allows the redirect command to capture output sent to the Tcl stdout channel. By default, the Tcl *puts* command sends its output to the stdout channel.

See Also

- get_app_var
- set_app_var

---

sh_enable_system_monitoring

Specifies whether critical system resources monitoring is enabled or disabled.

Data Types

Boolean

Default true

Description

If this variable is *true*, critical system resources are monitored throughout the run. When critical events occur, PrimeTime notifies you by issuing appropriate warning messages explaining the event that has arisen.

If this variable is *false*, critical system resources are not monitored.

---

sh_fast_analysis_mode_enabled

Specifies whether fast analysis mode is enabled.

Data Types

Boolean

Default false

Description

This read-only variable indicates if fast analysis mode is enabled. In fast analysis mode, there is a trade-off between accuracy and better performance.

To change the value of this variable, use the *set_program_options* command.

Feedback

See Also

- set_program_options

---

sh_flow_summary_file

Enable Flow Summary report and define the file name to write flow summary output to.

Data Types

string

Default "" (empty string)

Description

This variable enables flow summary reporting and defines the output file for the report. This variable could be set to *stdout* to show output on terminal in an interactive session or to a file name to write output in a file. The default value *""* means that the output channel is not defined and the flow summary utility is turned off.

The utility *Flow Summary* reports an structured summary of scripts and PrimeTime command execution throughout the analysis flow. This utility outputs the execution flow in form of indented and nested summary including the runtime consumed by notable PrimeTime commands and events. This utility also highlights some major tool events responsible for large runtime consumption.

This utility shows indented summary of PT commands, uncaptured runtime, custom markers by user (using command *define_flow_summary_segment*) showing new level of source with individual command runtime and cumulative source runtime. It packages multiple invocations of same commands within a scope to show their cumulative runtime. It creates indented and nested output for all commands/scripts called within a sourced script and custom nesting for block of script using command *define_flow_summary_segment*.

Major events/special commands which could be performance intensive are highlighted in the report. These highlighted events show the command invoking the event and location of the command in the script. Some examples of highlighted events are:

- forcing full timing update, in-validating timing update, invoking logical_update_timing, halting background parasitics read
- some other special commands like link_design, update_noise, read_db etc

Sample highlighted event:

<<< Event: Invoking logical_update_timing; File: multicycles.pt; Line: 3; "all_fanout -flat -endpoints_only -from _sel17"

Feedback

The flow summary could be turned ON/OFF multiple times during the flow. If the specified file already exists, subsequent flow summary information is appended to the end.

Examples

```
pt_shell>set sh_flow_summary_file ./summary.out
```

Output of flow summary report:

```
START  {
| source read_design.pt  {
| | <<< Event: Invoking link_design; File: read_design.pt; Line: 3;
 "link"
| } 0.6s
| source constraints.pt  {
| | | <<< Event: Invoking logical_update_timing; File: multicycles.pt;
 Line: 3;          "all_fanout -flat -endpoints_only -from _sel17"
| | source multicycles.pt  {
| | | <<< Event: Invoking logical_update_timing; File: multicycles.pt;
 Line: 3;          "all_fanout -flat -endpoints_only -from _sel17"
| | } 0.1s
| } 0.5s
| <<< Event: Invoking update_timing; File: run.pt; Line: 8;
 "update_timing"
| update_timing: 14.9s
| uncaptured: 0.2s
} 16.3s
```

See Also

- define_flow_summary_segment

---

sh_global_per_message_limit

Specifies the limit applied to all messages, other than those whose limit is set explicitly by the *set_message_info* command.

Data Types

integer

Default 10000

Description

This variable defines the limit for all messages, except those with a limit set using the *set_message_info* command.

This variable is distinct from the *sh_message_limit* variable. Unlike that variable, *sh_global_per_message_limit* applies to all messages, and is not limited in scope to any particular command or commands. Messages defined by the *sh_limited_messages*

Feedback

variable are bound by the *sh_message_limit* limit during the scope of relevant commands (such as *update_timing* and *read_parasitics*), but are additionally bound by the *sh_global_per_message_limit* limit over the course of a PrimeTime session.

To remove this limit, set the *sh_global_per_message_limit* variable to 0.

See Also

- get_message_info
- print_message_info
- set_message_info
- sh_limited_messages
- sh_message_limit

sh_help_shows_group_overview

Changes the behavior of the "help" command.

Data Types

string

Default application specific

Description

This variable changes the behavior of the *help* command when no arguments are specified to help. Normally when no arguments are specified an informational message with a list of available command groups is displayed.

When this variable is set to false the command groups and the commands in each group is printed instead. This variable exists for backward compatibility.

See Also

- help
- set_app_var

sh_high_capacity_effort

Specifies the level of capacity effort for the timing analysis of PrimeTime and PrimeTime SI. It only provides simple heuristics for trade-off between capacity and performance of the program. It does not have any impact on the analysis results.

Feedback

Data Types

string

Default default

Description

Specifies the effort level for capacity improved mode of the program. Allowed values are *default*, *low*, *medium*, and *high*. The *default* value corresponds to the *medium* setting.

When effort level increases, the peak memory required by the tool is expected to reduce, with potentially slightly longer runtime. It should be clarified that this variable only provides simple heuristic control on the trade-off between capacity and performance. And most importantly, regardless of the value, this variable alone does not change the results of the analysis.

This variable is only effective when the program is running in high capacity mode by using the *set_program_options -enable_high_capacity* command.

If the program is already in high capacity mode, further change of this variable do not have any effect until the next time the above command is issued.

See Also

- set_program_options

---

sh_high_capacity_enabled

Specifies whether high capacity mode is enabled.

Data Types

Boolean

Default true

Description

This read-only variable specifies the mode of analysis. The value of this variable changes after you run the *set_program_options* command.

This variable and the *sh_high_capacity_effort* variable are saved and restored in the session data. For more information, see the *set_program_options* man page.

See Also

- set_program_options
- sh_high_capacity_effort

Feedback

sh_host_mode

Specifies the mode of operation of the current shell.

Data Types

string

Default scalar

Description

This read-only variable specifies the mode of operation of the current application shell.

- *scalar* - The current shell was launched by the user in a non-distributed mode.
- *manager* - The current shell was launched by the user to be the manager process of a distributed analysis.
- *worker* - The current shell was launched by a manager process to be a worker process in a distributed analysis.

In a flat (scalar) analysis, the user launches and interacts with a single application process.

In a distributed analysis, the user launches a manager process and configures details of the distributed analysis (such as the number of worker processes). The manager process then launches remote worker processes when the *start_hosts* command is run.

The *sh_host_mode* variable is useful in scripts or setup files that can be sourced by multiple process types (such as both manager and worker processes).

Examples

The following example shows how the variable changes value when a newly launched PrimeTime session is switched from flat (non-distributed) analysis to distributed multi-scenario analysis (DMSA):

```
pt_shell> printvar synopsys_program_name
synopsys_program_name = "pt_shell"
pt_shell> printvar sh_host_mode
sh_host_mode          = "scalar"
pt_shell> set_app_var multi_scenario_enable_analysis true
Information: Switching into Distributed Multi Scenario Analysis mode.
 (MS-001)
true
pt_shell> printvar sh_host_mode
sh_host_mode          = "manager"
```

Note that the related variable *synopsys_program_name* indicates which tool's shell was launched. For more details on this variable, see the man page.

Feedback

See Also

- pt_shell
- synopsys_program_name

---

sh_language

Controls the tool language used by the application interactive console.

Data Types

string

Default tcl

Description

This variable controls the language used by the application interactive console. Some applications support languages in addition to Tcl as an extension language.

To determine the current value of this variable, use the *get_app_var sh_language* command.

See Also

- get_app_var
- set_app_var

---

sh_launch_dir

Specifies the launch directory of the current PrimeTime shell.

Data Types

string

Description

This read-only variable defines the launch directory of the current PrimeTime shell. In multi-scenario analysis, all workers are launched from the same directory as the manager. However during the course of analysis, the worker changes its current working directory multiple times but the *sh_launch_dir* variable remains constant across all workers and the manager.

Feedback

sh_limited_messages

Specifies the set of message types that have a limit by default when the *read_parasitics*, *report_annotated_parasitics* with the *-check* option, *read_sdf*, or *update_timing* command is invoked. This limit is defined by the *sh_message_limit* variable.

Data Types

string

Default "DES-002 RC-002 RC-004 RC-005 RC-006 RC-009 RC-011 RC-012 RC-013 RC-104 RC-201 RC-202 RC-203 RC-204 SLG-214 SLG-216 SLG-217 SLG-222 SLG-227 SLG-228 SLG-229 PTE-014 PTE-060 PTE-070 PYSR-001 PYSR-002 PYSR-003 PYSR-004 PYSR-005 PTE-114 SDF-036 SDF-046 UITE-494 UITE-474 LNK-039 LNK-038 LNK-043 LNK-044 UPF-757 HS-015 HS-023 HS-031 HS-037 PTE-101 PTIO-5 UITE-504 UITE-519 UITE-529 UITE-581 DES-028 DES-023 DES-024 ENV-003 XTALK-307 XTALK-308 PARA-001 PARA-003 PARA-004 PARA-005 PARA-006 PARA-007 PARA-010 PARA-011 PARA-020 PARA-040 PARA-041 PARA-043 PARA-044 PARA-045 PARA-046 PARA-047 PARA-050 PARA-051 PARA-052 PARA-053 PARA-060 PARA-061 PARA-063 PARA-064 PARA-065 PARA-066 PARA-067 PARA-068 PARA-071 PARA-072 PARA-073 PARA-074 PARA-075 PARA-078 PARA-079 PARA-081 PARA-082 PARA-083 PARA-084 PARA-085 PARA-086 PARA-087 PARA-088 PARA-089 PARA-090 PARA-092 PARA-093 PARA-094 PARA-095 PARA-096 PARA-098 PARA-100 PARA-104 PARA-106 PARA-007 PARA-110 PARA-111 PARA-112 PARA-113 PARA-114 PARA-115 PARA-118 PARA-119 PARA-120 PARA-121 PARA-122 PARA-123 PARA-124 PARA-140 PARA-141 PARA-142 PARA-143 PARA-144 PARA-145 PARA-146 PARA-159 PARA-162 PARA-183 PARA-184 PARA-185 SPFP-001 SPFP-002 SPFP-010 SPFP-011 SPFP-012 SPFP-013 SPFP-014 SPFP-015 SPFP-100 SPFP-101 SPFP-102 SPFP-103 SPFP-104 SPFP-105 SPFP-106 SPFP-107 SPFP-108 SPFP-109 SPFP-110 SPFP-111 SPFP-112 SPFP-113 SPFP-114 SPFP-115 SPFP-116 SPFP-118 SPFP-119 SPFP-120 SPFP-121 SPFP-122 SPFP-123 NED-075"

Description

This variable defines the set of messages that have a limit by default when the *read_parasitics*, *report_annotated_parasitics* with the *-check* option, *read_sdf*, *report_constraint*, or *update_timing* command is executed. This limit has no impact on messages that are emitted from other commands.

This limit is refreshed for each invocation of the *read_parasitics*, *report_annotated_parasitics* with the *-check* option, *read_sdf*, *report_constraint*, or implicit or explicit *update_timing* command. Each time one of these commands is invoked, the *sh_message_limit* command allows the specified number of messages to be issued for each message type specified with the *sh_limited_messages* variable. If the limit is exceeded for one type of message, a warning message appears showing the type of message that is suppressed.

Feedback

The setting of this variable has lower priority than the *set_message_info* command. If the *set_message_info* command is used to set the limit for a message type, this variable has no impact for that message type.

This variable is distinct from the *sh_global_per_message_limit* variable, which applies to all message types, and is not limited in scope to any particular command or commands. Any message defined by the *sh_limited_messages* variable is also bound by the *sh_global_per_message_limit* limit over the course of a PrimeTime session.

To remove this default limit, either set the *sh_limited_messages* variable to *{}* (an empty string) or set the *sh_message_limit* variable to 0.

See Also

- get_message_info
- print_message_info
- set_message_info
- sh_message_limit
- sh_global_per_message_limit

---

sh_line_editing_mode

Enables vi or Emacs editing mode in the PrimeTime shell.

Data Types

string

Default emacs

Description

This variable sets the command line editor mode to *emacs* (the default) or *vi*.

Use the *list_key_bindings* command to display the current key bindings and edit mode.

You can set this variable in the .synopsys_pt.setup file or directly in the shell. The *sh_enable_line_editing* variable must be set to its default of *true*.

See Also

- list_key_bindings
- sh_enable_line_editing

Feedback

sh_message_limit

Specifies the default limit of messages defined by the *sh_limited_messages* variable during the *read_parasitics*, *report_annotated_parasitics -check*, *read_sdf*, and *update_timing* commands.

Data Types

integer

Default 100

Description

This variable defines the default limit for messages in the *sh_limited_messages* variable printed from several commands, including *link_design*, *read_parasitics*, *report_annotated_parasitics* (with the *-check* option), *read_sdf*, and *update_timing*. This limit is not used for messages issued by other commands.

This limit is refreshed when you use the *read_parasitics*, *report_annotated_parasitics -check*, *read_sdf*, or implicit or explicit *update_timing* command. Each time you invoke one of these commands, the *sh_message_limit* variable displays the number of messages that show up for each message type set using the *sh_limited_messages* variable. If the limit is exceeded for one type of message, a warning message explains that this type of message is being suppressed.

This variable setting has lower priority than the *set_message_info* command. If you use the *set_message_info* command to specify the limit for a message type, the default limit on that message type is not effective.

This variable is distinct from the *sh_global_per_message_limit* variable, which applies to all messages, and is not limited in scope to any particular command or commands. Any message which is bound by the *sh_message_limit* limit is also bound by the *sh_global_per_message_limit* limit over the course of a PrimeTime session.

To remove this default limit, set the *sh_limited_messages* variable to "" (no value), or set the *sh_message_limit* variable to 0.

See Also

- get_message_info
- print_message_info
- set_message_info
- sh_limited_messages
- sh_global_per_message_limit

Feedback

sh_mm_resolve_clockdata

Controls whether clock-data conflict will be resolved during mode merging.

Data Types

boolean

Default true

Description

When this variable is set to *true*, clock-data conflicts will be resolved during mode merging

When this variable is set to *false*, clock-data conflicts will not be resolved during mode merging

See Also

- create_merged_modes

sh_new_variable_message

Controls a debugging feature for tracing the creation of new variables.

Data Types

Boolean

Default application specific

Description

The *sh_new_variable_message* variable controls a debugging feature for tracing the creation of new variables. Its primary debugging purpose is to catch the misspelling of an application-owned global variable. When set to *true*, an informational message (CMD-041) is displayed when a variable is defined for the first time at the command line. When set to *false*, no message is displayed.

Note that this debugging feature is superseded by the new *set_app_var* command. This command allows setting only application-owned variables. See the *set_app_var command man page for details.*

Other variables, in combination with *sh_new_variable_message*, enable tracing of new variables in scripts and Tcl procedures.

Warning: This feature has a significant negative impact on CPU performance when used with scripts and Tcl procedures. This feature should be used only when developing scripts

Feedback

or in interactive use. When you turn on the feature for scripts or Tcl procedures, the application issues a message (CMD-042) to warn you about the use of this feature.

To determine the current value of this variable, use the *get_app_var sh_new_variable_message* command.

See Also

- get_app_var
- set_app_var
- sh_new_variable_message_in_proc
- sh_new_variable_message_in_script

---

sh_new_variable_message_in_proc

Controls a debugging feature for tracing the creation of new variables in a Tcl procedure.

Data Types

Boolean

Default false

Description

The *sh_new_variable_message_in_proc* variable controls a debugging feature for tracing the creation of new variables in a Tcl procedure. Its primary debugging purpose is to catch the misspelling of an application-owned global variable.

Note that this debugging feature is superseded by the new *set_app_var* command. This command allows setting only application-owned variables. Please see the *set_app_var* command man page for details.

Note that the *sh_new_variable_message* variable must be set to *true* for this variable to have any effect. Both variables must be set to *true* for the feature to be enabled. Enabling the feature simply enables the *print_proc_new_vars* command. In order to trace the creation of variables in a procedure, this command must be inserted into the procedure, typically as the last statement. When all of these steps have been taken, an informational message (CMD-041) is generated for new variables defined within the procedure, up to the point that the *print_proc_new_vars* commands is executed.

Warning: This feature has a significant negative impact on CPU performance. This should be used only when developing scripts or in interactive use. When you turn on the feature, the application issues a message (CMD-042) to warn you about the use of this feature.

Feedback

To determine the current value of this variable, use the *get_app_var sh_new_variable_message_in_proc* command.

See Also

- get_app_var
- print_proc_new_vars
- set_app_var
- sh_new_variable_message
- sh_new_variable_message_in_script

---

sh_new_variable_message_in_script

Controls a debugging feature for tracing the creation of new variables within a sourced script.

Data Types

Boolean

Default false

Description

The *sh_new_variable_message_in_script* variable controls a debugging feature for tracing the creation of new variables within a sourced script. Its primary debugging purpose is to catch the misspelling of an application-owned global variable.

Note that this debugging feature is superseded by the new *set_app_var* command. This command allows setting only application-owned variables. See the *set_app_var* command man page for details.

Note that the *sh_new_variable_message* variable must be set to *true* for this variable to have any effect. Both variables must be set to *true* for the feature to be enabled. In that case, an informational message (CMD-041) is displayed when a variable is defined for the first time. When *sh_new_variable_message_in_script* is set to *false* (the default), no message is displayed at the time that the variable is created. When the *source* command completes, however, you see messages for any new variables that were created in the script. This is because the state of the variables is sampled before and after the *source* command. It is not because of inter-command sampling within the script. So, this is actually a more efficient method to see if new variables were created in the script.

For example, given the following script a.tcl:

```
echo "Entering script"
set a 23
```

Feedback

```
echo a = $a
set b 24
echo b = $b
echo "Exiting script"
```

When *sh_new_variable_message_in_script* is *false* (the default), you see the following when you source the script:

```
prompt> source a.tcl
Entering script
a = 23
b = 24
Exiting script
Information: Defining new variable 'a'. (CMD-041)
Information: Defining new variable 'b'. (CMD-041)
prompt>
```

Alternatively, when *sh_new_variable_message_in_script* is *true*, at much greater cost, you see the following when you source the script:

```
prompt> set sh_new_variable_message_in_script true
Warning: Enabled new variable message tracing -
         Tcl scripting optimization disabled. (CMD-042)
true
prompt> source a.tcl
Entering script
Information: Defining new variable 'a'. (CMD-041)
a = 23
Information: Defining new variable 'b'. (CMD-041)
b = 24
Exiting script
prompt>
```

Warning: This feature has a significant negative impact on CPU performance. This should be used only when developing scripts or in interactive use. When you turn on the feature, the application issues a message (CMD-042) to warn you about the use of this feature.

To determine the current value of this variable, use the *get_app_var sh_new_variable_message_in_script* command.

See Also

- get_app_var
- set_app_var
- sh_new_variable_message
- sh_new_variable_message_in_proc

Feedback

sh_obsolete_is_error

Raise a Tcl error when an obsolete command is executed.

Data Types

Boolean

Default application specific

Description

When set this variable causes a Tcl error to be raised when an obsolete command is executed. Normally only a warning message is issued.

Obsolete commands have no effect.

See Also

- get_app_var
- set_app_var

sh_output_log_file

Specifies the name of the file to which all application output is logged.

Data Types

string

Default "" (empty)

Description

This variable specifies the name of the file to which the application logs all output information during a session. By default, this variable is set to an empty string, indicating that the application's output is not logged.

This variable can be set only in a setup file. After setup files have been read, the variable becomes read-only.

See Also

- sh_command_log_file

sh_product_version

Indicates the version of the application currently running.

Feedback

Data Types

string

Description

This variable is set to the version of the application currently running. The variable is read only.

To determine the current value of this variable, use the *get_app_var sh_product_version* command.

See Also

- get_app_var

---

sh_program_mode

Specifies the mode of running the tool.

Data Types

string

Default default

Description

Specifies the mode of program execution.

Allowed values are *default* and *file_identification*.

The *default* mode is the regular mode of operation used for design analysis.

In the *file_identification* mode, most commands are disabled, while commands that read files are replaced by lightweight alternatives that open the required files for reading and then immediately close them. Most other tool commands finish immediately without running. In the file identification mode, PrimeTime does not load any design or timing data and does not perform any timing analysis.

Commands that create collections return a string token "=mock_collection=" instead of a collection. Commands that manipulate collections (such as *index_collection*) check for this value, and if found, return the same token. If the token is not found, the collection command executes normally.

The *file_identification* mode is designed to be used with the Synopsys Testcase Packager, which records all file accesses during the execution of an application. The intent of the mode is to emulate only the file accesses which would have been performed if PrimeTime were to run a testcase.

Feedback

See Also

- set_program_options

---

sh_pt_tmp_dir_unique_mode

Enable uniquification mode for pt_tmp_dir.

Data Types

boolean

Default false

Description

This variable controls the uniquification for pt_tmp_dir. It is disabled by default.

When enabled, pt_tmp_dir will be changed to a subdirectory to the path specified by user, in format of .PTXXXXXX, where 'X' is random generated alphanumeric character. If a relative path is provided, pt_tmp_dir will be converted to absoluted path.

When a session saved with *sh_pt_tmp_dir_unique_mode* enabled is to be restored by *restore_session*, unless user has set *pt_tmp_dir* specifically, an unique subdirectory will be created in the base directory in saved session and used as *pt_tmp_dir*.

When disabled, pt_tmp_dir retains the path specified by user.

Set pt_tmp_dir to an invalid path or unwritable path will produce *PTIO-1* message, along with current pt_tmp_dir value.

Changing *sh_pt_tmp_dir_unique_mode* from false to true, will uniquify current *pt_tmp_dir* if it is not uniquified already. Changing *sh_pt_tmp_dir_unique_mode* from true to false does not affect *pt_tmp_dir*.

Any temporary directories created with *sh_pt_tmp_dir_unique_mode* enabled will be removed at exit.

Examples

Uniquification mode enabled

```
pt_shell> pwd
/dira/sub_dir1

pt_shell> set pt_tmp_dir /tmp
/tmp/.PTBvRm9t

# Set pt_tmp_dir to same base path, will NOT create another unique
 subdirectory
pt_shell> set pt_tmp_dir /tmp
```

Feedback

```
/tmp/.PTBvRm9t

# Set pt_tmp_dir to same path, will NOT create another unique
 subdirectory
pt_shell> set pt_tmp_dir /tmp/.PTBvRm9t
/tmp/.PTBvRm9t

pt_shell> set pt_tmp_dir .
/dira/sub_dir1/.PTgy5ZMC
```

Uniquification mode disabled

```
pt_shell> set sh_pt_tmp_dir_unique_mode false
false

pt_shell> pwd
/dira/sub_dir1

pt_shell> set pt_tmp_dir /tmp
/tmp

pt_shell> set pt_tmp_dir .
 .
```

See Also

- pt_tmp_dir

---

sh_script_stop_severity

Indicates the error message severity level that would cause a script to stop running before it completes.

Data Types

string

Default application specific

Description

When a script is run with the *source* command, there are several ways to get it to stop running before it completes. One is to use the *sh_script_stop_severity* variable. This variable can be set to *none*, *W*, or *E*.

Feedback

- When set to *E*, the generation of one or more error messages by a command causes a script to stop.
- When set to *W*, the generation of one or more warning or error messages causes a script to stop.
- When set to *none*, the generation messages does not cause the script to stop.

Note that *sh_script_stop_severity* is ignored if *sh_continue_on_error* is set to *true*.

To determine the current value of this variable, use the *get_app_var sh_script_stop_severity* command.

See Also

- get_app_var
- set_app_var
- source
- sh_continue_on_error

---

sh_source_emits_line_numbers

Indicates the error message severity level that causes an informational message to be issued, listing the script name and line number where that message occurred.

Data Types

string

Default application specific

Description

When a script is executed with the *source* command, error and warning messages can be emitted from any command within the script. Using the *sh_source_emits_line_numbers* variable, you can help isolate where errors and warnings are occurring.

This variable can be set to *none*, *W*, or *E*.

- When set to *E*, the generation of one or more error messages by a command causes a CMD-082 informational message to be issued when the command completes, giving the name of the script and the line number of the command.
- When set to *W*, the generation of one or more warning or error messages causes a the CMD-082 message.

Feedback

The setting of *sh_script_stop_severity* affects the output of the CMD-082 message. If the setting of *sh_script_stop_severity* causes a CMD-081 message, then it takes precedence over CMD-082.

To determine the current value of this variable, use the *get_app_var sh_source_emits_line_numbers* command.

See Also

- get_app_var
- set_app_var
- source
- sh_continue_on_error
- sh_script_stop_severity
- CMD-081
- CMD-082

---

sh_source_logging

Indicates if individual commands from a sourced script should be logged to the command log file.

Data Types

Boolean

Default application specific

Description

When you source a script, the *source* command is echoed to the command log file. By default, each command in the script is logged to the command log file as a comment. You can disable this logging by setting *sh_source_logging* to *false*.

To determine the current value of this variable, use the *get_app_var sh_source_logging* command.

See Also

- get_app_var
- set_app_var
- source

Feedback

sh_source_uses_search_path

Indicates if the *source* command uses the *search_path* variable to search for files.

Data Types

Boolean

Default application specific

Description

When this variable is set to rue the *source* command uses the *search_path* variable to search for files. When set to *false*, the *source* command considers its file argument literally.

To determine the current value of this variable, use the *get_app_var sh_source_uses_search_path* command.

See Also

- get_app_var
- set_app_var
- source
- search_path

sh_tcllib_app_dirname

Indicates the name of a directory where application-specific Tcl files are found.

Data Types

string

Description

The *sh_tcllib_app_dirname* variable is set by the application to indicate the directory where application-specific Tcl files and packages are found. This is a read-only variable.

See Also

- get_app_var

Feedback

sh_user_man_path

Indicates a directory root where you can store man pages for display with the *man* command.

Data Types

list

Default empty list

Description

The *sh_user_man_path* variable is used to indicate a directory root where you can store man pages for display with the *man* command. The directory structure must start with a directory named *man*. Below *man* are directories named *cat1*, *cat2*, *cat3*, and so on. The *man* command will look in these directories for files named *file.1*, *file.2*, and *file.3*, respectively. These are pre-formatted files. It is up to you to format the files. The *man* command effectively just types the file.

These man pages could be for your Tcl procedures. The combination of defining help for your Tcl procedures with the *define_proc_attributes* command, and keeping a manual page for the same procedures allows you to fully document your application extensions.

The *man* command will look in *sh_user_man_path* after first looking in application-defined paths. The user-defined paths are consulted only if no matches are found in the application-defined paths.

To determine the current value of this variable, use the *get_app_var sh_user_man_path* command.

See Also

- define_proc_attributes
- get_app_var
- man
- set_app_var

si_analysis_logical_correlation_mode

Enables or disables logical correlation analysis during PrimeTime SI delay or noise calculation.

Data Types

Boolean

Default true

Description

If this variable is set to its default of *true*, PrimeTime SI enables logical correlation analysis while performing crosstalk delay or crosstalk noise analysis. In logical correlation analysis, PrimeTime SI considers the logical relationships between multiple aggressor nets where buffers and inverters are used, so that the analysis is less pessimistic.

If you set this variable to *false*, PrimeTime SI assumes that the aggressor nets switch together in the direction that causes worst-case crosstalk delay or worst-case crosstalk noise bump on a victim net. When logical correlation analysis is turned off, PrimeTime SI results are expected to be slightly more pessimistic; however, the PrimeTime SI runtime is faster.

See Also

- si_enable_analysis

---

si_ccs_aggressor_alignment_mode

Specifies aggressor alignment mode used in the CCS-based gate-level simulation engine.

Data Types

string

Default lookahead

Description

To specify the aggressor alignment mode used in the CCS-based gate-level simulation engine, set this variable to one of these values:

- *lookahead* (default) - Enables the lookahead alignment feature for the CCS-based gate-level simulation engine so that PrimeTime SI finds the alignment that results in worst-case receiver output delay. Note that such an alignment might not correspond to the worst-case stage delay.
- *stage* - Disables the lookahead alignment feature.

See Also

- si_enable_analysis

---

si_enable_analysis

Enables or disables PrimeTime SI, which performs crosstalk analysis.

Feedback

Data Types

Boolean

Default false

Description

By default, this variable is set to *false*, which disables PrimeTime SI.

To enable PrimeTime SI, set this variable to *true*, so that the *update_timing* and *report_timing* commands perform crosstalk-aware timing calculations. If you set this variable to *true*, you must also do the following:

1. Obtain a PrimeTime SI license. You cannot use PrimeTime SI without a license.
2. Use the *read_parasitics -keep_capacitive_coupling* command to read in the coupling parasitics for your design. PrimeTime SI is useful only if the design has coupling parasitics data.

See Also

- read_parasitics
- report_timing
- update_timing

---

si_enable_multi_input_switching_analysis

Enables or disables multi-input switching (MIS) analysis.

Data Types

Boolean

Default false

Description

Set this variable to one of these values:

- *false* (default) - Disables MIS analysis.
- *true* - Enables MIS analysis, which uses the delay and slew coefficients specified by the *set_multi_input_switching_coefficient* command.

  When you set this variable to *true*, the tool checks out a PrimeTime-ADV license.

Feedback

See Also

- report_multi_input_switching_coefficient
- reset_multi_input_switching_coefficient
- set_multi_input_switching_coefficient
- si_enable_multi_input_switching_timing_window_filter

si_enable_multi_input_switching_timing_window_filter

Controls whether input pin arrival windows are considered by multi-input switching (MIS) analysis.

Data Types

Boolean

Default true

Description

This variable controls whether input pin arrival windows are considered by multi-input switching (MIS) analysis.

The valid values are:

- *true* (default) - Enables window alignment and overlap checking during MIS analysis. The tool collects the arrival windows of input pins, performs overlap checking, and computes the MIS effects.
- *false* - Ignores all arrival window information and applies whatever MIS effects are possible for qualifying cells.

This variable applies to all three MIS analysis modes: *lib_cell*, *lib_arc*, and *advanced*.

See Also

- report_multi_input_switching_coefficient
- set_multi_input_switching_coefficient
- si_enable_multi_input_switching_analysis

si_filter_keep_all_port_aggressors

Specifies whether to filter the aggressors for a victim net that is connected to a port.

Feedback

Data Types

Boolean

Default false

Description

When this variable is set to *false* (the default), the normal filtering algorithm is applied. When you set this variable to *true*, all aggressors for a port net are kept. It is recommended to set this variable to *true* before using the *extract_model* command.

The tool filters aggressor nets, regardless of the variable setting, when the net

- Does not have valid parasitics or driver
- Is constant logic or the cell is a single pin cell
- Is excluded by user-specified settings

See Also

- si_enable_analysis

---

si_filter_per_aggr_noise_peak_ratio

Specifies the threshold for the voltage bump introduced by an aggressor at a victim node, divided by VCC, below which the aggressor net can be filtered out during electrical filtering.

Data Types

float

Default 0.01

Description

Specifies the threshold for the voltage bump introduced by an aggressor at a victim node; the default is 0.01. PrimeTime SI uses the *si_filter_per_aggr_noise_peak_ratio* variable during the electrical filtering phase to determine whether an aggressor net can be filtered.

An aggressor net, along with its coupling capacitors, is filtered when the peak voltage of the voltage bump induced on the victim net divided by VCC is less than the value specified with this variable.

See Also

- si_enable_analysis

Feedback

si_ilm_keep_si_user_excluded_aggressors

Specifies whether to include user-excluded PrimeTime SI aggressors in the interface logic model (ILM).

Data Types

Boolean

Default false

Description

Set this variable to one of these values:

- *false* (default) - Excludes user-excluded PrimeTime SI aggressors from the ILM. You exclude aggressors by setting nets to be constant or by using the *set_si_delay_analysis* or *set_si_noise_analysis* command.
- *true* - Includes user-excluded PrimeTime SI aggressors in the ILM.

See Also

- create_ilm
- set_si_delay_analysis
- set_si_noise_analysis

si_multi_input_switching_analysis_mode

Controls the multi-input switching (MIS) analysis mode.

Data Types

string

Default lib_cell

Description

Multi-input switching (MIS) allows PrimeTime to model multi-input switching effects, which occur when multiple inputs transition simultaneously (as compared to the one-input-at-a-time stimulus used during library cell characterization).

Three analysis modes are available. This variable specifies both the list of modes to use, and in which order of precedence (highest first).

Feedback

- *lib_cell* (default) - Uses library cell coefficients to perform multi-input switching analysis. In this mode, you must supply pre-characterized delay coefficients with the *set_multi_input_switching_coefficient* command.
- *lib_arc* - Uses library timing arc coefficients. In this mode, the tool takes user-supplied library timing arc coefficients to model the multi-input switching speedup of each cell timing arc.

  A PrimeTime-ADV-PLUS license is checked out when this value is set.
- *advanced* - Enables advanced MIS analysis. In this mode, the tool calculates the multi-input switching speedup of a cell using its input slew/waveform and output load without the need for user-supplied annotated coefficients. See below for the supported library cell types and prerequisites.

  A PrimeTime-ADV-PLUS license is checked out when this value is set.

This variable takes effect only when the *si_enable_multi_input_switching_analysis* is set to *true*.

When the user specifies a single mode or multiple modes by the *si_multi_input_switching_analysis_mode* variable, the MIS analysis modes to use and their precedence are determined based on this variable.

On successful update of MIS precedence, an *MIS-014* message is thrown with a MIS mode or a list of MIS modes in their order of precedence (highest first). If multiple computation criteria can apply to a qualifying library cell the right analysis mode is selected as per the order of precedence.

Advanced MIS Analysis Prerequisites

Advanced MIS analysis requires the following library data:

- CCS timing models
- CCS noise models

In addition, the advanced waveform propagation mode must also be enabled using *delay_calc_waveform_analysis_mode* variable.

Advanced MIS analysis supports the following library cell types:

```
AND2   OR2   NAND2   NOR2   AO211   AOI211   OA211   OAI211
AND3   OR3   NAND3   NOR3   AO21    AOI21    OA21    OAI21
AND4   OR4   NAND4   NOR4   AO22    AOI22    OA22    OAI22
                            AO31    AOI31    OA31    OAI31
```

To see the list of library cells that qualify for advanced MIS calculation, use the *report_multi_input_switching_lib_cells* command.

Feedback

Library timing arc based MIS Analysis Prerequisites

Library timing arc based MIS analysis requires the following library data for each arc:

- A float factor characterizing the rise MIS speedup
- A string denoting the set of pins characterizing the rise MIS speedup
- A float factor characterizing the fall MIS speedup
- A string denoting the set of pins characterizing the fall MIS speedup

The data can be either imported from the library or applied by script using the *define_user_attribute* command. For details of how to specify this information, see the user guide documentation.

See Also

- si_enable_multi_input_switching_analysis
- report_multi_input_switching_lib_cells
- delay_calc_waveform_analysis_mode

---

si_multi_input_switching_derate_scaling_for_library_cells_compatibility

Enables or disables user-defined coefficients on library cells in scaling group libraries for multi-input switching (MIS) analysis.

Data Types

Boolean

Default true

Description

User-defined multi-input switching (MIS) coefficients are applied with the *set_multi_input_switching_coefficient* command. This variable controls how coefficients are considered in scaling library groups.

When set to *true* (the default), only MIS coefficients applied to library cells in the link library are considered. MIS coefficients applied to library cells in other libraries in the scaling group have no effect.

Feedback

When set to *false*, MIS coefficients applied to library cells in scaling libraries are considered as follows:

- If a cell's voltage configuration exactly matches that of a scaling library, that library's library cell MIS coefficient is used.
- If a cell's voltage configuration does not exactly match a scaling library, the link library cell coefficient is used.

This variable must be set before any libraries are loaded into memory.

In a future release of PrimeTime, it is planned that the default of this variable will change to *false*. This behavior change could affect user scripts.

See Also

- define_scaling_lib_group
- set_multi_input_switching_coefficient

---

si_multi_input_switching_precedence

Controls the precedence of multi-input switching (MIS) analysis modes.

Data Types

string

Default lib_cell advanced lib_arc

Description

Note: This variable is deprecated and will be obsoleted in a future release of PrimeTime. Use the *multi_input_switching_analysis_mode* variable to specify both the analysis modes to use, and in which order of precedence (highest first).

This variable controls the precedence of calculation of MIS speedup for a cell when different modes of MIS analysis apply to the same cell.

The different modes of MIS analysis are: *lib_cell*, *lib_arc*, and *advanced* analysis. The MIS precedence is an ordered list of modes that determines the precedence. You can change the default precedence by providing a different order of modes. In case all tokens are not

Feedback

specified, the default precedence string is used to complete the specification. The tokens in the string can be one of:

- *lib_cell* (default) - User set *set_multi_input_switching_coefficient* takes precedence over advanced MIS calculated or library arc coefficients.
- *advanced* - PT computed advanced MIS calculation takes precedence over user set *set_multi_input_switching_coefficient* or library arc coefficients.
- *lib_arc* - Library arc based coefficients take precedence over user set *set_multi_input_switching_coefficient* or advanced MIS calculated coefficients.

Examples

The following example provides a full specification of MIS precedence:

```
pt_shell> set_app_var si_multi_input_switching_precedence {lib_arc
 advanced
lib_cell"
```

The above implies that library arc based coefficients take the highest precedence followed by advanced MIS calculated coefficients followed by user specified library cell coefficients.

The following example provides an incomplete specification of MIS precedence:

```
pt_shell> set si_multi_input_switching_precedence "advanced"
```

PT completes the above specification using the default precedence as *"advanced lib_cell lib_arc"*, implying that advanced MIS calculated coefficients take the highest precedence followed by user specified library cell coefficients followed by library arc based coefficients.

See Also

- si_multi_input_switching_analysis_mode

---

si_multi_input_switching_tied_input_filter

Specifies whether a tied-input filter is used to limit where MIS effects are applied.

Data Types

string

Default none

Description

This variable controls whether MIS effects are filtered (limited) to only to cells that have tied inputs.

Feedback

In this case, "tied inputs" refers to input pins that are driven by the same physical net. Pins with logically equivalent drivers through buffers or inverters do not count.

The valid values for this variable are as follows, in order of most conservative to least conservative:

- *none* (default) - MIS effects are not limited to tied-input cells; this filter is disabled.
- *any_inputs* - MIS effects are limited to only cells that have at least two tied input pins.
- *all_inputs* - MIS effects are limited to only cells that have all input pins tied together.

For *none* and *any_inputs* tied-input cell filtering, untied input pins are analyzed for their contribution using arrival windows or infinite windows, as determined by the *si_enable_multi_input_switching_timing_window_filter* variable.

This variable applies to all three MIS analysis modes: *lib_cell*, *lib_arc*, and *advanced*.

See Also

- si_enable_multi_input_switching_analysis
- si_enable_multi_input_switching_timing_window_filter
- si_multi_input_switching_analysis_mode

---

si_noise_composite_aggr_mode

Specifies the composite aggressor mode for noise analysis.

Data Types

string

Default disabled

Description

This variable specifies which composite aggressor mode is used in PrimeTime SI noise analysis. The following values are allowed for the variable:

- *disabled* (default) - Disables the composite aggressor feature.

  In *disabled* composite aggressor mode, PrimeTime SI uses its original flow with composite aggressor completely off to analyze the noise.

Feedback

- *statistical* - Causes PrimeTime SI to calculate noise by using the statistical composite aggressor flow.

  PrimeTime SI aggregates the effect of multiple small aggressors into a single composite aggressor, thereby reducing the computational complexity and improving the performance.

See Also

- report_noise_calculation

---

si_noise_endpoint_height_threshold_ratio

Specifies a value that defines the threshold where noise propagation stops. The ratio is between 0.0 and 1.0 of VDD.

Data Types

float

Default 0.75

Description

This variable sets a threshold voltage for an endpoint. When the propagated noise reaches this threshold voltage, noise propagation stops, and the load pin of the net is recorded as an endpoint. This variable only affects the *report_at_endpoint* analysis mode of the noise update.

This variable applies only to combinational circuit pins because sequential cell pins are automatically (noise) endpoints.

Examples

Suppose VDD is 1.0 V, and the variable is set to 0.75. In addition, suppose net N1 has a noise bump with the height of 0.8 V. Since the height of the noise bump is greater than 0.75 V, net N1 is recorded as an endpoint. Since N1 is an endpoint, there is no propagated noise at the next stage of net N1.

See Also

- report_noise
- report_noise_calculation
- set_noise_parameters

Feedback

si_noise_immunity_default_height_ratio

Specifies the noise immunity default height ratio.

Data Types

float

Default 0.4

Description

This variable sets a noise immunity default if a noise pin is not constrained. If the noise pin has no data to compute noise immunity value, the tool calculates a default noise immunity height by multiplying this variable and VDD of the pin.

Set this variable to a value between 0.0 and 1.0.

If this variable is set to 1.0, the pin is not constrained anymore, and no noise slack is calculated. The pins with no constraints are reported as "none" by the *check_noise* command.

See Also

- check_noise
- report_noise_calculation

si_noise_limit_propagation_ratio

Specifies the maximum amount of propagated noise if the noise height passes noise immunity.

Data Types

float

Default 0.75

Description

During a noise update, if a noise passes the immunity criteria, then the propagated height is reduced to a specified ratio of the noise immunity value. This ratio is specified by the *si_noise_limit_propagation_ratio* variable. This variable only affects the *report_at_source* analysis mode of the noise update.

Set this variable to a value between 0.0 and 1.0.

Feedback

See Also

- set_noise_parameters
- update_noise

---

si_noise_skip_update_for_report_attribute

Controls whether to skip the noise update during the *report_attribute* command.

Data Types

Boolean

Default false

Description

This variable controls whether to perform implicit noise update when reporting all attributes with the *report_attribute* command.

When this variable is set to *true*, attributes that require a noise update are reported by the *report_attribute* command only if the noise information is up-to-date.

If the variable is set to *false* (the default), reporting of noise-related attribute values in *report_attribute* triggers an implicit noise update.

See Also

- report_attribute
- update_noise

---

si_noise_slack_skip_disabled_arcs

Skips disabled timing arcs for noise slack calculation.

Data Types

Boolean

Default false

Feedback

Description

This variable applies to noise slack computation on a pin that has all its forward timing arcs disabled. For such pins, to specify whether its disabled timing arcs are considered for noise slack calculation, set this variable to one of these values:

- *false* (default) - If the pin has all forward timing arcs disabled, this setting calculates noise slacks for all timing arcs.
- *true* - If the pin has all forward timing arcs disabled, this setting does not calculate noise slack for any timing arcs.

If a pin has at least one enabled forward arc, this variable does not affect noise slack calculation. In such a case, only the enabled arcs are used to perform noise slack calculation.

Examples

On a pin with one forward arc, if the arc is disabled by the *set_case_analysis* or *set_disable_timing* command, no noise slack is calculated for it if the *si_noise_slack_skip_disabled_arcs* variable is set to *true*. However, if the variable is set to *false*, noise slack is calculated for this arc.

See Also

- get_timing_arcs
- set_case_analysis
- set_disable_timing

---

si_noise_update_status_level

Controls the number of progress messages displayed during the update of noise analysis.

Data Types

string

Default none

Description

To control the number of progress messages displayed during the noise update process, set this variable to one of these values:

- *none* (default) - Displays no messages.
- *low* - Displays messages only at the beginning and end of the update.

Feedback

- *high* - Displays messages at the beginning and end of the update, and messages for the noise calculation step show the completion percentage in steps of 10 percents.

  When you set this variable to *low* or *high*, the progress of the noise update is reported for an explicit update (using the *update_noise* command) or for an implicit update invoked by another command (for example, the *report_noise* command) that forces a noise update.

See Also

- report_noise
- update_noise

---

si_use_driving_cell_derate_for_delta_delay

Allows crosstalk delta delay for one net to be derated using the relevant derate factor for the cell driving that net.

Data Types

Boolean

Default false

Description

If you set this variable to *true*, the crosstalk delta delays for each net is derated using the derate factors from the cell driving that net.

The relevant derating factor to be applied adheres to the same precedence rules as the driving cell itself. For example, if no instance-specific derate factor was set on the driving cell then the hierarchical cell, the library cell, and finally the global derate factors are checked for a relevant derate factor.

To see what derating factors are to be applied to the net in question, first obtain the driving cell ($driving_cell), and use the following command:

```
pt_shell> report_timing_derate [get_cells $driving_cell]
```

If you use the *report_timing* command with the *-derate* option, the tool reports the nonderated crosstalk delta delay as before. In addition, the derate column reports the net derating factor used to derate the delta-free net delay.

Feedback

See Also

- report_timing
- report_timing_derate
- set_timing_derate

---

si_xtalk_composite_aggr_mode

Specifies the composite aggressor mode for crosstalk delay.

Data Types

string

Default disabled

Description

To specify the composite aggressor mode for PrimeTime SI crosstalk delay analysis, set this variable to one of these values:

- *disabled* (default) - Disables the composite aggressor feature.

  In the *disabled* mode, PrimeTime SI uses its original flow with composite aggressor completely off to calculate the crosstalk delay.

- *statistical* - Causes PrimeTime SI to calculate crosstalk by using the statistical composite aggressor flow.

  PrimeTime SI aggregates the effect of some small aggressors (including filtered ones) into a single composite aggressor, reducing the computational complexity and improving the performance.

  Due to runtime consideration, SI logical correlation analysis, case analysis, timing window overlap analysis, etc are only performed for effective aggressors. They are not being performed for small filtered aggressors that are included in the composite aggressor.

See Also

- remove_si_delay_disable_statistical
- report_delay_calculation
- report_si_delay_analysis
- set_si_delay_disable_statistical

Feedback

- si_xtalk_composite_aggr_noise_peak_ratio
- si_xtalk_composite_aggr_quantile_high_pct

---

si_xtalk_composite_aggr_noise_peak_ratio

Controls the composite aggressor selection for crosstalk analysis.

Data Types

float

Default 0.01

Description

This variable specifies the threshold value as a ratio of the crosstalk bump to VDD. Below this threshold, aggressors are selected into the composite aggressor group. The default is 0.01, which means that all aggressor nets with crosstalk-bump-to-VDD ratio less than 0.01 is selected into the composite aggressor group.

This variable works together with other filtering thresholds, *si_filter_per_aggr_noise_peak_ratio*, to determine which aggressors are selected into the composite aggressor group.

See Also

- remove_si_delay_disable_statistical
- report_si_delay_analysis
- set_si_delay_disable_statistical
- si_filter_per_aggr_noise_peak_ratio
- si_xtalk_composite_aggr_mode
- si_xtalk_composite_aggr_quantile_high_pct

---

si_xtalk_composite_aggr_quantile_high_pct

Controls the composite aggressor creation for statistical analysis.

Data Types

float

Default 99.73

Feedback

Description

This variable specifies the probability in percentage format that any given real combined bump height is less than or equal to the computed composite aggressor bump height. Given the specified probability, the resulting quantile value for the composite aggressor bump height is calculated.

The default of this variable is 99.73, which corresponds to a 3-sigma probability that the real bump height from any randomly-chosen combination of aggressors is covered by the composite aggressor bump height.

See Also

- remove_si_delay_disable_statistical
- report_si_delay_analysis
- set_si_delay_disable_statistical
- si_xtalk_composite_aggr_mode
- si_xtalk_composite_aggr_noise_peak_ratio

---

si_xtalk_delay_analysis_mode

Specifies the arrival window alignment mode used for crosstalk delta delay.

Data Types

string

Default all_paths

Description

This variable specifies how the alignment between victim and aggressor nets is performed to compute crosstalk delta delay in PrimeTime SI. The allowed values are *all_paths* and *all_path_edges*.

In the *all_paths* (default) alignment mode, the tool considers the full range of all possible victim net arrival times, from the earliest minimum-delay arrival to the latest maximum-delay arrival, among all paths that pass through the victim net. Any overlap between the aggressor arrival window and the range between these two extreme victim net arrival times triggers a delta delay adjustment to the victim arrival transition.

Feedback

The *all_paths* window overlap mode is simple and fast, but it is also conservative because the actual victim transitions might not overlap the aggressor window. For example:

- The aggressor window might overlap a region inside the victim window where there are no actual victim transitions; the victim transitions occur before and after (but not during) the aggressor window.
- The aggressor window might overlap the early, minimum-delay victim transitions but not the late, maximum-delay victim transitions; the tool still applies delta delays to the maximum-delay transitions, even though the aggressor window does not overlap them.
- The aggressor window might overlap the late, maximum-delay victim transitions but not the early, minimum-delay victim transitions; the tool still applies delta delays to the minimum-delay transitions, even though the aggressor window does not overlap them.

To get a more accurate analysis at the cost of more runtime, set the *si_xtalk_delay_analysis_mode* to *all_path_edges*. The *all_path_edges* mode considers the actual early or late arrival edge times, without combining them into a continuous window. For maximum-delay analysis, it considers only latest-arrival edges of paths that pass through the victim net. Similarly, for minimum-delay analysis, it considers only earliest-arrival edges. Delta delay adjustments to victim arrival times are performed only when the aggressor arrival window overlaps one or more arrival edges of the applicable type (either latest or earliest arrivals). Thus, late-arrival overlaps with the aggressor window do not affect early-arrival delay calculations, and vice versa.

The *all_path_edges* mode is more accurate (less pessimistic) than the *all_paths* mode because it can reduce the number of cases where delta delays are applied, yet it is safe for signoff because it finds all situations where the aggressor transitions can worsen the arrival time of the transition in the victim net.

See Also

- si_enable_analysis

---

si_xtalk_double_switching_mode

Controls the double switching detection during the PrimeTime SI timing analysis.

Data Types

string

Default disabled

Description

When this variable is set to *disabled*, double switching detection is disabled.

Feedback

When you set this variable to one of the following values, PrimeTime SI checks whether crosstalk bump on the switching victim could cause the output to switch twice (and cause a pulse) instead of the desired single signal propagation:

- *clock_network* - Detects the potential double switching in the clock network, which could cause the double clocking (where the clock could switch twice on a the sensitive edge) or false clocking (where the switching bump on the nonsensitive edge could actually latch the state).
- *full_design* - Detects the potential double switching in the data path as well as clock path. Double switching on a data path is less severe then double switching on the clock network.

The double switching detection needs CCSN library information on the victim load cell.

After the update_timing command, you can access this information by using the *report_si_double_switching* command or by the net attributes by using the *si_has_double_switching* and *si_double_switching_slack* commands.

The *si_has_double_switching* victim net attribute is true whenever there is a potential double switching on any of the load pins.

The victim net attribute, *si_double_switching_slack*, has the bump slack, reducing the switching bump by that much amount could remove the double switching. If the victim net does not cause double switching the *si_double_switching_slack* attribute is "POSITIVE". If the victim net load pins does not have CCS noise model information, the attribute is reported as "INFINITY".

The victim nets having the double switching is automatically reselected to higher iteration so that they could be reanalyzed with more accurate analysis.

The double switching happens when, the switching bump and transition time are large and fed into drivers that are strong enough to amplify this. To avoid double switching, either of them can be reduced.

See Also

- report_si_double_switching
- si_enable_analysis

---

si_xtalk_exit_on_max_iteration_count

Specifies the maximum number of incremental timing iterations, after which PrimeTime SI exits the analysis loop.

Data Types

integer

Feedback

Default 2

Description

This variable specifies the maximum number of incremental timing iterations. PrimeTime SI exits the analysis loop after performing up to this number of iterations.

The default of this variable is 2, meaning that PrimeTime SI exits the analysis loop after performing two iterations. To override this default, set the variable to another integer; the minimum allowed value is 1.

See Also

- si_enable_analysis

---

si_xtalk_exit_on_max_iteration_count_incr

Specifies a maximum number of timing iterations following what-if change (such as size_cell) to the design, after which PrimeTime SI exits the analysis loop.

Data Types

integer

Default 2

Description

A timing update for signal integrity analysis is done in an iterative way. The number of iterations is controlled by the *si_xtalk_exit_on_max_iteration_count* variable. The *si_xtalk_exit_on_max_iteration_count_incr* variable has the same function but is used when update_timing can be done incrementally. Incremental signal integrity timing is only done after minor changes, such as *size_cell*, *insert_buffer*, *set_coupling_separation*. Large number of changes or any other change result in a full *update_timing*.

This variable is deprecated and will be obsolete in a future releases of PrimeTime.

See Also

- si_xtalk_exit_on_max_iteration_count

---

si_xtalk_library_consistency_compatibility

Specifies the compatibility mode for SI delay calculation.

Data Types

Boolean

Feedback

Default false

Description

In SI delay calculation, PrimeTime checks the CCS noise v.s. NLDM library data consistency for victim driver cells. When the library data are inconsistent, CCS noise library data will not be used in crosstalk delay delay calculation. Please use Library Complier check_library command to check CCS noise v.s. NLDM consistency.

The default value of this variable is *false*, which is suitable for libraries with good CCS noise v.s. NLDM consistency.

When the variable is set to *true*, it allows CCS noise based crosstalk delay calculation with imperfect library.

See Also

- si_enable_analysis

---

si_xtalk_max_transition_mode

Specifies whether PrimeTime SI uses uncoupled or coupled values for maximum and minimum transition constraint checks.

Data Types

string

Default uncoupled

Description

This variable specifies how PrimeTime SI computes the transition for crosstalk delay analysis. The allowed values are

- *uncoupled* (the default) - Uses uncoupled transition values.
- *reliability* - Uses coupled transition values based on the delta slew.

See Also

- report_constraint

---

si_xtalk_use_physical_exclusivity_on_ignore_arrival_nets

Performs or discards physical exclusivity analysis during PrimeTime SI delay analysis.

Feedback

Data Types

Boolean

Default false

Description

If this variable is set to its default of *false*, PrimeTime SI discards physical exclusivity analysis while performing crosstalk delay for nets with ignore_arrival marking based on set_si_delay_analysis. For instance, consider two clocks that are physically exclusive, CLK1 and CLK2. For a victim stage, an aggressor (name 'A') clocked by CLK1 and another aggressor (name 'B') clocked by CLK2. And these aggressor nets have ignore_arrival marking based on set_si_delay_analysis. Aggressor 'A' and 'B' can simulaneously attack the victim stage. This setting is runtime efficient but can possibly result in some pessimism.

If you set this variable to *true*, the victim stage is analyzed multiple times to consider each possible victim and aggressor combination across the physically-exclusive clocks. In this case, aggressors 'A' and 'B' cannot simultaneously attack the victim stage. This removes the pessimism when nets are marked for ignore_arrival using set_si_delay_analysis.

---

simlink_enable_fast_monte_carlo

Enables fast Monte Carlo simulation in Simlink POCV simulation.

Data Types

boolean

Default false

Description

This variable controls whether Simlink POCV simulation uses full Monte Carlo or fast Monte Carlo simulation.

When this variable is set to *false* (the default), Simlink POCV simulation uses full Monte Carlo simulation, which can be used as a reference as needed.

When this variable is set to *true*, Simlink POCV simulation uses fast Monte Carlo simulation, which is an accelerated simulation mode with similar accuracy to full Monte Carlo.

To use fast Monte Carlo simulation, HSPICE version O-2018.09-SP2-1 or later is recommended.

Feedback

Fast Monte Carlo simulation also requires that the *sim_setup_simulator* command be run prior to *sim_setup_library*, so that the required library-level learning can be performed. See the EXAMPLES section for an example.

Examples

The following example performs path-based analysis, sets a variable to a collection containing the worst path, and analyzes timing path by performing fast Monte Carlo SPICE simulation.

```
pt_shell> set_app_var ps_enable_analysis true
pt_shell> set_app_var simlink_enable_fast_monte_carlo true
pt_shell> sim_setup_simulator -simulator /global/apps/hspice/bin/hspice
pt_shell> sim_setup_library -lib [get_libs *] -sub_circuit sub_path
 -header header_path
pt_shell> set pba_path [get_timing_paths -pba_mode path -path_type
 full_clock_expanded]
pt_shell> sim_analyze_path $pba_path -variation
```

The following example performs path-based analysis, sets a variable to a collection containing the worst path, and compares PrimeTime path arrival nominal and corner value with fast Monte Carlo SPICE simulation results.

```
pt_shell> set_app_var ps_enable_analysis true
pt_shell> set_app_var simlink_enable_fast_monte_carlo true
pt_shell> sim_setup_simulator -simulator /global/apps/hspice/bin/hspice
pt_shell> sim_setup_library -lib [get_libs *] -sub_circuit sub_path
 -header header_path
pt_shell> set pba_path [get_timing_paths -pba_mode path -path_type
 full_clock_expanded]
pt_shell> sim_validate_path $pba_path -variation
```

See Also

- sim_analyze_path
- sim_setup_library
- sim_setup_simulator
- sim_validate_path

---

simlink_include_generated_clock_on_internal_pin

Provides SimLink support for generated clocks defined on cell internal pins.

Data Types

Boolean

Feedback

Default false

Description

This variable provides SimLink support for generated clocks defined on cell internal pins. When this variable is set to *true*, the measurements of timing arcs associated with internal pins are written out in the output SPICE deck correctly.

```
sim_analyze_path(2)
sim_setup_library(2)
sim_setup_simulator(2)
sim_validate_path(2)
```

simlink_report_default_significant_digits

Specifies the default number of significant digits used to display values in simlink reports.

Data Types

integer

Default 4

Description

Sets the default number of significant digits for simlink reports. Allowed values are 0-13; the default is 4.

The simlink commands (for example, *sim_validate_path*) have a *-significant_digits* option that overrides the value of this variable. Please check the man page of simlink commands for more information.

See Also

- sim_validate_path
- sim_validate_setup
- sim_validate_stage
- sim_validate_noise

simlink_training_data_directory

Specifies the directory for storing simlink training data.

Data Types

string

Feedback

Default ""

Description

To enable machine learning for faster Monte Carlo simulation in Simlink simulation, set the *simlink_enable_fast_monte_carlo* variable to *true*. By default, learning data is saved in working directory defined by *sim_setup_simulator -work_dir*. By settting the *simlink_training_data_directory* variable to the name of a directory, *sim_setup_library* command uses this directory to store training data.

If the training data directory is shared by many users, please make sure write permission is opened in the directory and its sub-directory. Or users without write permission will see warning in *sim_setup_library* and default directory is used to save learning data.

See Also

- sim_analyze_path
- sim_setup_simulator
- sim_setup_library
- sim_validate_path
- simlink_enable_fast_monte_carlo

---

svr_enable_ansi_style_port_declarations

Enables ANSI style port declarations in verilog

Data Types

Boolean

Default false

Description

This variable is used to enable ANSI Style port declarations in verilog. Disabled by default. You can set this variable to one of these values:

- *false* (the default) - Disables ANSI style port declarations.
- *true* - Enables ANSI style port declarations.

See Also

- read_verilog

Feedback

svr_enable_vpp

Enables preprocessing of Verilog files.

Data Types

Boolean

Default false

Description

This variable enables or disables the preprocessing of Verilog files.

By default, this variable is set to *false*, which disables the preprocessing of Verilog files.

If you set this variable to *true* before you run the *read_verilog* command, the Verilog preprocessor detects and expands the following Verilog directives:

- `define
- `else
- `endif
- `ifdef
- `include
- `undef

The preprocessor creates intermediate files in the directory that is specified by the *pt_tmp_dir* variable. Also, the `include directive uses the *search_path* variable to find files.

Very few structural Verilog files use preprocessor directives. Set this variable to *true* only if your Verilog file contains these directives.

See Also

- read_verilog
- pt_tmp_dir
- search_path

svr_keep_unconnected_nets

Specifies whether to preserve or discard unconnected nets.

Feedback

Data Types

Boolean

Default true

Description

This variable is used only by the native Verilog reader. You can set this variable to one of these values:

- *true* (the default) - Preservies unconnected nets.
- *false* - Discards unconnected nets.

See Also

- read_verilog

---

synopsys_program_name

Indicates the name of the program currently running.

Data Types

string

Description

This variable is read only, and is set by the application to indicate the name of the program you are running. This is useful when writing scripts that are mostly common between some applications, but contain some differences based on the application.

To determine the current value of this variable, use *get_app_var synopsys_program_name*.

See Also

- get_app_var

---

synopsys_root

Indicates the root directory from which the application was run.

Data Types

string

Feedback

Description

This variable is read only, and is set by the application to indicate the root directory from which the application was run.

To determine the current value of this variable, use *get_app_var synopsys_root*.

See Also

- get_app_var

t

tcl_interactive

Indicates whether the Tcl interpreter is operating in interactive mode.

Data Types

string

Default 0

Description

This global variable indicates whether the Tcl interpreter is operating in interactive mode. If the variable is set to 1 , then the interpreter operates as an interactive interpreter.

You cannot change the setting of this read-only Tcl variable.

tcl_library

Specifies the path to the Tcl library.

Data Types

string

Default The default path is determined by your tool installation.

Description

You cannot change the setting of this read-only Tcl variable.

tcl_pkgPath

Specifies the path to the Tcl library.

Feedback

Data Types

string

Default The default path is determined by your tool installation.

Description

You cannot change the setting of this read-only Tcl variable.

---

timing_all_clocks_propagated

Determines whether or not all clocks are created as propagated clocks.

Data Types

Boolean

Default false

Description

If this variable is set to *false* (the default), clocks are created as nonpropagated clocks. If you set this variable to *true*, all clocks subsequently created by the *create_clock* or *create_generated_clock* command are created as propagated clocks.

By default, the *create_clock* and *create_generated_clock* commands create only nonpropagated clocks. You can subsequently define some or all clocks to be propagated clocks using the *set_propagated_clock* command. However, if you set the *timing_all_clocks_propagated* variable to true, the *create_clock* and *create_generated_clock* commands subsequently create only propagated clocks.

Setting this variable to *true* or *false* affects only clocks created after the setting is changed. Clocks created before the setting is changed retain their original condition (propagated or nonpropagated).

See Also

- create_clock
- create_generated_clock
- set_propagated_clock

---

timing_allow_short_path_borrowing

Enables time borrowing through level sensitive latches for hold time checks.

Feedback

Data Types

Boolean

Default false

Description

This variable affects time borrowing for short paths (used for hold checks) at a level-sensitive latch. By default, PrimeTime performs time borrowing only for long paths (used for setup checks).

The default model is a conservative model for short paths. It is valid even during power-up transient state.

An aggressive model is to allow borrowing for short paths. It is valid only during steady state.

To enable borrowing for short paths, set the *timing_allow_short_path_borrowing* variable to *true*.

See Also

- report_timing
- set_max_time_borrow

---

timing_aocvm_analysis_mode

Configures the depth calculation of an advanced on-chip variation (AOCV) analysis.

Data Types

string

Default separate_launch_capture_depth

Description

The *timing_aocvm_analysis_mode* variable specifies one of three methods to calculate path metrics during AOCV analysis. The path metrics consist of depth and distance values:

- Depth is used to index the random component of variation in an AOCV derating table. Depth is defined as the number of cell (or net) delay timing arcs in a path from the path common point. Random coefficients affect the depth calculation. For more information, see the *set_aocvm_coefficient* man page.
- Distance is used to index the systematic component of variation in an AOCV derating table. Distance is defined as the length of the diagonal of the bounding box enclosing

Feedback

the cell (or net) delay timing arcs in a path from the path common point. The cell at the path endpoint is included in the cell bounding box. Only nodes and terminals of the network along the net arc are included in the net bounding box.

To specify the analysis mode, set the *timing_aocvm_analysis_mode* variable to one of these values:

- *separate_launch_capture_depth* - Calculates separate depth values for launch and capture paths. The launch clock and data paths are considered together, and the capture clock path is considered separately.
- *combined_launch_capture_depth* - Considers launch and capture depths together and calculates a combined depth for the entire path.
- *separate_data_and_clock_metrics* - Calculates separate depths for the clock and data paths and uses the appropriate AOCV delay deratings based on the separate depths.

The AOCV path metric calculation is further influenced by the *timing_aocvm_enable_clock_network_only*, *timing_aocvm_enable_single_path_metrics*, and *timing_ocvm_enable_distance_analysis* variables.

See Also

- get_timing_paths
- read_aocvm
- remove_aocvm
- report_aocvm
- report_timing
- set_aocvm_coefficient
- timing_ocvm_enable_distance_analysis
- timing_aocvm_enable_clock_network_only
- timing_aocvm_enable_single_path_metrics

timing_aocvm_enable_analysis

Enables graph-based advanced on-chip variation (AOCV) analysis.

Data Types

Boolean

Default false

Feedback

Description

When this variable is set to *false* (the default), the tool does not perform graph-based advanced on-chip variation (AOCV) timing updates. A path-based advanced OCV analysis can be performed in this mode using the *-pba_mode* option of the *report_timing* and *get_timing_paths* commands. In this mode, constant timing derates specified using the *set_timing_derate* command are required to pessimistically bound the analysis. You should specify constant derates that do not clip the range of the path-based advanced OCV derates to avoid optimism.

When you set this variable to *true*, the graph-based advanced OCV timing update is performed as part of the *update_timing* command. A path-based advanced OCV analysis can also be performed in this mode. In this mode, constant timing derates are not required and, in fact, constant derates for *static delays* are ignored. Graph-based advanced OCV derates computed during the *update_timing* command tightly bound the path-based advanced OCV derates without clipping their range. Note that setting this variable to *true* automatically switch the design into *on_chip_variation* analysis mode using the *set_operating_conditions* command.

See Also

- get_timing_paths
- read_aocvm
- report_aocvm
- report_timing
- set_operating_conditions

---

timing_aocvm_enable_clock_network_only

Configures the advanced on-chip variation (AOCV) analysis to be applied in the clock network only.

Data Types

Boolean

Default false

Description

This variable specifies whether AOCV analysis is performed on all timing arcs of the chip or only for timing arcs in the clock network.

Feedback

- *true* - Applies AOCV derating to arc delays in the clock network only; calculates clock network AOCV depth and distance metrics based on clock network topology only. The data network receives constant (OCV) derating, if constant derates have been annotated for data network objects; otherwise, they are not derated.
- *false* (the default) - Applies AOCV derating to all arc delays and calculates AOCV depth and distance metrics by considering both clock and data network topology.

See Also

- timing_aocvm_analysis_mode
- timing_aocvm_enable_single_path_metrics
- timing_ocvm_enable_distance_analysis

---

timing_aocvm_enable_single_path_metrics

Prevents the use of separate depth and distance values for nets and cells during advanced on-chip variation (AOCV) analysis.

Data Types

Boolean

Default false

Description

This variable determines whether AOCV analysis considers depth and distance metrics for cell arcs and net arcs separately.

- *true* - Calculates and uses only cell arc metrics to look up both cell and net deratings. This behavior is backward compatible with the legacy Tcl-based "LOCV" solution.
- *false* (the default) - Uses separate depth and distance values for nets and cells during AOCV analysis. The tool uses cell metrics when looking up cell deratings and net metrics when looking up net deratings.

See Also

- timing_aocvm_analysis_mode
- timing_aocvm_enable_clock_network_only
- timing_ocvm_enable_distance_analysis

Feedback

timing_aocvm_ocv_precedence_compatibility

Controls the fallback to on-chip variation (OCV) deratings when advanced on-chip variation (AOCV) is enabled.

Data Types

Boolean

Default false

Description

This variable is obsolete and provided for backward compatibility only. Use the *timing_ocvm_precedence_compatibility* variable instead.

See Also

- timing_ocvm_precedence_compatibility

timing_arrival_attribute_filter

Controls what type of arrival events are considered by pin/port arrival attributes.

Data Types

string

Default exclude_ignore_clock_latency

Description

When startpoint arrivals are launched by *-ignore_clock_latency* exceptions or by path segmentation, they have a zero value at the startpoint. They can be smaller (sometimes much smaller) than arrivals that include clock latency, which can introduce unexpected results in scripts that do not expect this difference. As a result, these arrivals are excluded by default.

This variable controls what type of arrival events considered by the following attributes:

```
max_arrival
max_fall_arrival
max_rise_arrival
min_arrival
min_fall_arrival
min_rise_arrival
```

Feedback

Valid values are:

- *exclude_ignore_clock_latency* (default)
  - Includes clocked startpoint arrivals
  - Excludes startpoint arrivals launched by *-ignore_clock_latency* exceptions
  - Excludes arrivals launched by path segmentation
- *ignore_clock_latency_only*
  - Excludes clocked startpoint arrivals
  - Includes startpoint arrivals launched by *-ignore_clock_latency* exceptions
  - Excludes arrivals launched by path segmentation
- *none*
  - Includes clocked startpoint arrivals
  - Includes startpoint arrivals launched by *-ignore_clock_latency* exceptions
  - Includes arrivals launched by path segmentation

This variable does not affect the *arrival_window* attribute.

---

timing_avoid_internal_pins_as_latch_loop_breaker

Guides the tool in selecting latch D pins as loop breaker D pins to avoid internal pins of cells.

Data Types

Boolean

Default false

Description

This variable pertains to transparent latch D pins that are internal pins of cells, when *timing_enable_through_paths* is enabled.

When the *timing_enable_through_paths* variable is enabled, some latch D pins in the design are selected to act as loop breaker D pins. On these pins, borrowing still occurs, but paths through the pins are not shown, and instead are broken into (1) a path to the D pin and (2) a borrow path from the D pin.

When the *timing_avoid_internal_pins_as_latch_loop_breaker* variable is set to *true*, the tool tries to avoid selecting internal pins of cells as loop breaker D pins.

Feedback

When set to *false* (the default), internal D pins of cells do not have special precedence relative to other D pins for loop breaker selection.

Some cells, such as primary-secondary flip flops, are modeled as two transparent latches in a single cell. It can be useful to avoid selecting the secondary latch in such cells as loop breaker latches, so that paths are not broken inside a cell.

For example,

```
set_app_var timing_avoid_internal_pins_as_latch_loop_breaker true
```

See Also

- set_latch_loop_breaker
- timing_enable_through_paths

---

timing_bidirectional_pin_max_transition_checks

Specifies the extent of max transition design rule checks on bidirectional pins.

Data Types

string

Default both

Description

This variable specifies the extent of a max transition design rule checks for bidirectional pins. Set the variable to one of these values:

- *both* (the default) - Checks the driver and load.
- *driver* - Checks the driver.
- *load* - Checks the load.

---

timing_calculation_across_broken_hierarchy_compatibility

Specifies whether delay calculation ignores hierarchical pins where clock constraints are specified when performing detailed RC calculation.

Data Types

Boolean

Default false

Feedback

Description

Set this variable to one of the following values:

- *false* (the default) - Computes the interconnect delay from leaf pin to leaf pin; ignores intervening hierarchical pins where clock constraints are defined.
- *true* - Attempts to use the hierarchical pin during calculation.

See Also

- create_clock
- create_generated_clock
- report_delay_calculation

---

timing_check_defaults

Defines the default checks for the *check_timing* command.

Data Types

list

Default generated_clocks generic latch_fanout loops no_clock no_input_delay partial_input_delay unconstrained_endpoints unexpandable_clocks no_driving_cell pulse_clock_non_pulse_clock_merge pll_configuration voltage_level

Description

This variable defines the default checks to be performed when you run the *check_timing* command without any options, or when you modify the default list using the *-include* or *-exclude* option.

You can change this variable setting before you run the *check_timing* command. Alternatively, you can override the default checks by running the *check_timing* command with the *-override_defaults* option.

See Also

- check_timing

---

timing_check_include_constant_loop_compatibility

Controls the constant loops check compatibility for the *check_timing* command.

Feedback

Data Types

Boolean

Default false

Description

You can set this variable to the following values:

- *false* (the default) - The tool will print non-constant loops under the *loops* check and constant loops under the *constant_loops* check during *check_timing*.
- *true* - The tool will print both non-constant loops and constant loops under the *loops* check. The *constant_loops* check value will be ignored.

See Also

- check_timing

---

timing_clock_gating_check_fanout_compatibility

Controls whether the effects of the *set_clock_gating_check* command propagates through logic, or applies only to the specified design object.

Data Types

Boolean

Default false

Description

This variable controls the behavior of the *set_clock_gating_check* command when it is applied to design netlist objects (ports, pins or cells).

If this variable is set to *false*, PrimeTime uses the current behavior where the effects of the *set_clock_gating_check* command apply only to the specified design objects, and do not propagate through the transitive fanout. When this behavior is enabled, specifying the command on a port has no effect. This behavior is consistent with Design Compiler and IC Compiler.

To apply the clock gating settings to an entire clock domain without enumerating all gating cells, clock objects can be provided to the *set_clock_gating_check* command. When clock objects are provided, the clock gating settings apply to all instances of clock gating for those clocks.

If this variable is set to *true*, PrimeTime uses the behavior from older versions where the *set_clock_gating_check command* also applies to the transitive fanout of the specified

Feedback

design objects. There is no way to configure only the specified design objects without also propagating the clock gating settings to downstream logic.

See Also

- set_clock_gating_check

---

timing_clock_gating_propagate_enable

Allows the gating enable signal delay to propagate through the gating cell.

Data Types

integer

Default true

Description

You can set this variable to one of these values:

- *true* (the default) - Allows the delay and slew from the data path of the gating check to propagate. If the output goes to a data pin, setting this variable to *true* produces the most desirable behavior.
- *false* - Prevents the delay and slew from the data path of the gating check from propagating. Only the delay and slew from the clock path is propagated. If the output goes to a clock pin of a latch, setting this variable to *false* produces the most desirable behavior.

---

timing_clock_reconvergence_pessimism

Selects signal transition sense matching for computing clock reconvergence pessimism removal.

Data Types

string

Default normal

Description

Determines how the value of the clock reconvergence pessimism removal (CRPR) is computed with respect to transition sense.

Feedback

Set this variable to one of the following values:

- *normal* - The CRPR value is computed even if the clock transitions to the source and destination latches are in different directions on the common clock path. It is computed separately for rise and fall transitions and the value with smaller absolute value is used.
- *same_transition* - The CRPR value is computed only when the clock transition to the source and destination latches have a common path and the transition is in the same direction on each pin of the common path. If the source and destination latches are triggered by different edge types, CRPR is computed at the last common pin at which the launch and capture edges match.

  If the variable is set to *same_transition*,the CRPR for min pulse width checks will be computed at the last common pin where the launch and capture edges match. The result will be zero if a common pin can not be found.

See Also

- get_timing_paths
- report_timing
- timing_remove_clock_reconvergence_pessimism

---

timing_continue_on_scaling_error

Allows timing analysis continue when a scaling library group error condition occurs, instead of exiting the session.

Data Types

Boolean

Default false

Description

Set this variable to specify whether a scaling library group failure (SLG-001 error condition) causes an exit from the tool:

- *false* (the default) - The tool reports an SLG-001 error message and ends the tool session when a scaling library group error condition occurs.
- *true* - The tool reports an SLG-418 warning message and shows a list of SLG-XXX error conditions, but allows timing analysis to continue when a scaling library group error condition occurs. If scaling information is missing for one or more operating condition parameters, scaling will not be performed for those parameters.

Feedback

See Also

- define_scaling_lib_group
- SLG-001
- SLG-418

---

timing_cross_voltage_domain_analysis_mode

Specifies how simultaneous multivoltage analysis (SMVA) should analyze timing paths.

Data Types

string

Default full

Description

This variable specifies how simultaneous multivoltage analysis (SMVA) should analyze timing paths. SMVA analysis is enabled by the *timing_enable_cross_voltage_domain_analysis* variable.

The allowed values are:

- *full* - All paths are concurently analyzed at all relevant voltage configurations, including both cross-domain paths and within-domain paths.
- *only_crossing* - Only cross-domain paths are analyzed at their relevant voltage configurations. Within-domain paths are propagated to accurately model signal integrity effects, but their slacks are not calculated.
- *exclude_crossing* - Only within-domain paths are analyzed at their relevant voltage configurations. Cross-domain paths are propagated to accurately model signal integrity effects, but their slacks are not calculated.
- *capture_reduced_model* - Only cross-domain paths are analyzed at their worst and best voltage configurations inferred from all voltage levels of the supply nets as if running worst and best corner flow despite enabling SMVA. This is a special mode to enable reduced cross-domain SMVA run and requires that the slack and arrival times at the pins be saved. So the variable to control saving pin arrival and slacks (timing_save_pin_arrival_and_slack) is also locked in this mode.

See Also

- timing_enable_cross_voltage_domain_analysis

Feedback

timing_crpr_different_transition_derate

Specifies the fraction of the common clock reconvergence pessimism (CRP) that is removed if the transitions on the launching and capturing clock paths are different.

Data Types

float

Default 1.0

Description

When computing the common clock pessimism, the path to the common point is assumed to be correlated.

If the "normal" setting of the variable *timing_clock_recovergence_pessimism* is used, a pessimism value is computed for both the rising path and falling path to the common point, and the minimum is removed. This assumes the launch and capture paths, despite different transitions, are fully correlated.

However, the paths are actually only partially correlated. The gates and wires are the same, but the paths through the transistors of the gates may be different.

To reduce the amount of pessimism removed, set this variable to a factor less than 1.0. For example, setting it to 0.90 causes 90 percent of the calculated pessimism to be removed, resulting less reported slack and a more conservative analysis than removing 100 percent of the calculated pessimism.

Note that setting a factor of 0.0 results in no pessimism being removed for different transitions. This is not equivalent to using the "same_transition" setting for the *timing_clock_recovergence_pessimism* variable. In that setting, the tool continues to search for a common point with matching transitions. With the "normal" setting, the topological common point is always selected, irrespective of the whether the transitions match.

In parametric on-chip variation analysis, this variable derates just the nominal value of the common clock pessimism. To derate the variation, use the variable *timing_crpr_different_transition_variation_derate*.

See Also

- timing_remove_clock_reconvergence_pessimism
- timing_clock_reconvergence_pessimism
- timing_crpr_different_transition_variation_derate

Feedback

timing_crpr_different_transition_variation_derate

Specifies the fraction of the variation of the common clock reconvergence pessimism (CRP) that is removed if the transitions on the launching and capturing clock paths are different.

Data Types

float

Default 0.0

Description

Parametric on-chip variation (POCV) analysis internally computes arrival, required, and slack values based on statistical distributions. The common clock pessimism removes both nominal and variation pessimism from the slack.

When computing the common clock pessimism, if the launch and capture transition (rise/fall) at the common point are the same, the variation at the common point is removed from the slack.

If the "normal" setting of the variable *timing_clock_recovergence_pessimism* is used, and the launch and capture transition (rise/fall) at the common point are different, the variation will not be removed. This is because the rising path and the falling path are not fully correlated.

If this is too pessimistic, a percentage of the variation can be removed by setting the variable *timing_crpr_different_transition_variation_derate* to a non-zero value. i.e. to remove 10% of the variation, use the value 0.1. To allow the full credit of the variation, use 1.0.

See Also

- timing_pocvm_enable_analysis
- timing_remove_clock_reconvergence_pessimism
- timing_clock_reconvergence_pessimism
- timing_crpr_different_transition_derate

timing_crpr_remove_clock_to_data_crp

Allows the removal of clock reconvergence pessimism (CRP) from paths that fan out directly from clock source to the data pins of sequential devices.

Feedback

Data Types

Boolean

Default false

Description

If this variable is set to *true*, CRP is removed for all paths that fan out directly from clock source pins to the data pins of sequential devices.

If this variable is set to *false*, only the CRP up to the clock source pin which fans out to the data pin of the sequential device is removed. This is because the path up to a clock source pin is considered to be a clock path.

Consider the following example, where GCLK1 as a generated clock with CLK as its master clock. In this case, the CRP between pins A and B would removed irrespective of the value of the variable. However, when the variable is set to *true*, additional CRP between pins B and C would also be removed.

```
+-- buff3 (D) --- FF1/D
                                                  |
A (CLK) --- buff1 (B GCLK1) --- buff2 (C) --- |-- buff4 (E) --- FF1/CP
```

Note:

When this variable is set to *true*, all sequential devices that reside in the fanout of clock source pins must be handled separately in the subsequent timing update. This might cause severe performance degradation to the timing update.

See Also

- timing_remove_clock_reconvergence_pessimism

---

timing_crpr_remove_muxed_clock_crp

Controls whether clock reconvergence pessimism removal (CRPR) considers common path reconvergence between related clocks.

Data Types

Boolean

Default true

Description

This variable controls the CRPR in cases where two related clocks reconverge in the logic. Two clocks are related if one is a generated clock and the other is its parent, or both are generated clocks of the same parent clock. Although this variable name refers specifically

Feedback

to multiplexers, the variable applies to any situation where two related clocks reconverge within combinational logic.

If this variable is set to *true*, the separate clock paths up to the multiplexer are treated as reconvergent, and the CRP includes the reconvergence point as well as any downstream common logic. If this variable is set to *false*, the common pin is the last point where the clocks diverged to become related clocks.

If the design contains related clocks which switch dynamically (a timing path launches from one related clock and the clock steering logic switches dynamically so the path captures on the other related clock), then this variable should be set to false so the CRP is not removed.

The default is *true*, which removes the additional CRP.

See Also

- timing_remove_clock_reconvergence_pessimism

---

timing_crpr_threshold_ps

Specifies amount of pessimism that clock reconvergence pessimism removal (CRPR) is allowed to leave in the report.

Data Types

float

Default 5

Description

This variable specifies the amount of pessimism that clock reconvergence pessimism removal (CRPR) is allowed to leave in the report. The unit is in picoseconds (ps), regardless of the units of the main library. The minimum allowed value is 1 picosecond.

The threshold is per reported slack; setting this variable to the *TH1* value means that reported slack is no worse than *S* - *TH1*, where *S* is the reported slack when *timing_crpr_threshold_ps* is set close to zero.

The variable has no effect if CRPR is not active (*timing_remove_clock_reconvergence_pessimism* is *false*). The larger the value of *timing_crpr_threshold_ps*, the faster the runtime when CRPR is active. The recommended setting is about one half of the stage (gate plus net) delay of a typical stage in the clock network.

In most cases, the default setting provides a reasonable tradeoff between accuracy and runtime. You can use different settings throughout the design cycle: larger during the

Feedback

design phase, smaller for signoff. You might need to experiment and set a different value when moving to a different technology.

See Also

- timing_remove_clock_reconvergence_pessimism

---

timing_disable_bus_contention_check

Disables checking for timing violations resulting from transient contention on design buses.

Data Types

Boolean

Default false

Description

This variable applies only to bus designs that have multiple three-state drivers.

When set to *false* (the default), PrimeTime reports these timing violations. When set to *true*, PrimeTime ignores timing setup and hold (max and min) violations that occur as a result of transient bus contention.

Bus contention occurs when more than one driver is enabled at the same time. By default, PrimeTime treats the bus as if it is in an unknown state during this region of contention, and reports a timing violation if the setup and hold regions extend into the contention region. Note that checking is done only for timing violations and not for logical and excessive power dissipation violations, which are outside the scope of static timing analysis tools.

Set this variable to *true* only if you are certain that transient bus contention regions never occur. By setting the value to *true*, you guarantee that on a multidriven three-state bus, the drivers in the previous clock cycle are disabled before the drivers in the current clock cycle are enabled. If you set this variable to *true*, you must ensure that the *timing_disable_bus_contention_check* variable is *false*. The *timing_disable_bus_contention_check* and *timing_disable_floating_bus_check* variables cannot both be *true* at the same time.

During the switching between the high-impedance (Z) state and the high/low state, the timing behavior (for example, intrinsic delay) of three-state buffers is captured in the Synopsys library using the three_state_disable and three_state_enable timing arc types. These timing arcs connect the enable pin to the output pin of the three-state buffers. For more information, see the Library Compiler documentation.

Feedback

See Also

- timing_disable_floating_bus_check

timing_disable_clock_gating_checks

Disables clock gating checks, including both automatically inferred and library-defined clock gating checks.

Data Types

Boolean

Default false

Description

Set this variable to *true* to disable all clock gating checks, including both automatically inferred and library-defined clock gating checks, as well as inferred checks modified by the *set_clock_gating_check* command.

When this variable is set to *false* (the default), clock gating checks are enabled.

For more information, see SolvNet article 015769, "How Are Clock Gating Checks Inferred?"

See Also

- report_constraint
- set_clock_gating_check

timing_disable_cond_default_arcs

Disables default (unconditional) timing arcs that exist alongside conditional arcs.

Data Types

boolean

Default false

Description

This variable pertains to default (unconditional) arcs between any pair of pins that also has at least one conditional arc.

When set to *true*, this variable disables these default arcs; only the conditional arcs are considered.

Feedback

When set to *false* (the default), all of the arcs - default and conditional - are considered, which is safe (but potentially pessimistic).

Some libraries have a pessimistic default arc alongside conditional arcs. If the conditional arc *when* conditions cover all possible state-dependent delays for all cells, set this variable to *true* for improved accuracy.

For example, consider a 2-input XOR gate with inputs A and B and output Z. If the delays between A and Z are specified with two arcs with respective conditions 'B' and 'B~', a default arc between A and Z is not needed and should be disabled.

By default, this variable applies the default-arc disabling behavior to both delay calculation and arc-based receiver models.

However, if this variable is set to *false*, you can still enable the default-arc disabling behavior for arc-based receiver models by setting the following related variable:

```
set_app_var rc_receiver_disable_cond_default_arcs true
```

See Also

- report_delay_calculation
- report_disable_timing
- rc_receiver_disable_cond_default_arcs

---

timing_disable_floating_bus_check

Disables checking for timing violations resulting from transient floating design buses.

Data Types

Boolean

Default false

Description

This variable applies only to bus designs that have multiple three-state drivers. When set to *true*, PrimeTime ignores timing setup and hold (max and min) violations that occur as a result of transient floating buses. When set to *false* (the default), PrimeTime reports these timing violations.

The floating bus condition occurs when no driver controls the bus at a given time. By default, PrimeTime treats the bus as if it is in an unknown state during this region of contention, and reports a timing violation if the setup and hold regions extend into the floating region. Note that checking is done only for timing violations, and not for logical violations, which are outside the scope of static timing analysis tools.

Feedback

Set this value to *true* only if you are certain that transient floating bus regions never occur. By setting the value to *true*, you guarantee that on a multidriven three-state bus, the drivers in the previous clock cycle are disabled before the new drivers in the current clock cycle are enabled. If you set this variable to *true*, you must ensure that the *timing_disable_bus_contention_check* variable is *false*. The *timing_disable_floating_bus_check* and *timing_disable_bus_contention_check* variables cannot both be *true* at the same time.

During the switching between the high-impedance (Z) state and the high/low state, the timing behavior (for example, intrinsic delay) of three-state buffers is captured in the Synopsys library using the three_state_disable and three_state_enable timing arc types. These timing arcs connect the enable pin to the output pin of the three-state buffers. For more information, see the Library Compiler documentation.

See Also

- timing_disable_bus_contention_check

---

timing_disable_internal_inout_cell_paths

Enables bidirectional feedback paths within a cell.

Data Types

Boolean

Default true

Description

If this variable is set to *true* (the default), PrimeTime automatically disables bidirectional feedback paths in a cell. If you set this variable to *false*, bidirectional feedback paths in cells are enabled.

This variable has no effect on timing of bidirectional feedback paths that involve more than one cell (that is, if nets are involved); these feedback paths are controlled by the *timing_disable_internal_inout_net_arcs* variable.

See Also

- remove_disable_timing
- report_timing
- set_disable_timing
- timing_disable_internal_inout_net_arcs

Feedback

timing_disable_internal_inout_net_arcs

Controls whether bidirectional feedback paths across nets are disabled or not.

Data Types

Boolean

Default true

Description

If this variable is set to *true* (the default), PrimeTime automatically disables bidirectional feedback paths that involve more than one cell; no path segmentation is required. Note that only the feedback net arc between non-bidirectional driver and load is disabled.

If you set this variable to *false*, these bidirectional feedback paths are enabled.

This variable has no effect on timing of bidirectional feedback paths that are completely contained in one cell (that is, if nets are not involved); these feedback paths are controlled by the *timing_disable_internal_inout_cell_paths* variable.

See Also

- remove_disable_timing
- report_timing
- set_disable_timing
- timing_disable_internal_inout_cell_paths

timing_disable_recovery_removal_checks

Disables the timing analysis of recovery and removal checks in the design.

Data Types

Boolean

Default false

Description

If this variable is set to *false* (the default), PrimeTime performs recovery and removal checks. For a description of these checks, see the man page for the *report_constraint* command.

To disable recovery and removal timing analysis, set this variable to *true*.

Feedback

See Also

- report_constraint

---

timing_early_launch_at_borrowing_latches

Removes clock latency pessimism from the launch times for paths which begin at the data pins of transparent latches.

Data Types

Boolean

Default true

Description

The following description assumes that the data paths of interest are setup paths, because it refers specifically to time borrowing scenarios. However, if the *timing_allow_short_path_borrowing* variable is *true*, the same description applies to borrowing hold paths too.

When a latch is in its transparent phase, data arriving at the D-pin passes through the element as though it were combinational. To model this scenario, whenever PrimeTime determines that time borrowing occurs at such a D-pin, paths that originate at the D-pin are created.

Sometimes there is a difference between the launching and capturing latch latencies, due to either reconvergent paths in the clock network or different minimum and maximum delays of cells in the clock network. For setup paths, PrimeTime uses the late value to launch and the early value to capture. This achieves the tightest constraint and avoids optimism. However, for paths starting from latch D-pins, this is pessimistic since data simply passes through and therefore does not even "see" the clock edge at the latch.

When the *timing_early_launch_at_borrowing_latches* variable is *true* (the default), the tools eliminates this pessimism by using the early latch latency to launch such paths. Note that only paths that originate from a latch D-pin are affected. When the variable is set to *false*, the tools uses late clock latency to launch all setup paths in the design.

When the *timing_early_launch_at_borrowing_latches* and *timing_remove_clock_reconvergence_pessimism* variables are both set to *true*, the tool applies CRPR to paths that do not start from transparent latch D-pins, while the paths that start from transparent latch D-pins have early launch using the minimum path. It is not possible to apply both pessimism removal techniques on the same timing path.

Feedback

The following recommended settings depend on the characteristics of your design:

- On designs with long clock paths, consider setting *timing_early_launch_at_borrowing_latches* to *false*. This allows CRPR to be applied on paths that start from transparent latch D-pins. When clock paths are long, CRPR can be a more powerful pessimism reduction technique.
- On designs with shorter common clock paths, or where critical paths traverse several latches, consider setting *timing_early_launch_at_borrowing_latches* to *true*. This results in more pessimism removal, even though paths that start from transparent D-pins do not get any CRPR credit.

See Also

- report_timing
- timing_allow_short_path_borrowing
- timing_remove_clock_reconvergence_pessimism

---

timing_enable_advanced_data_management

Enables advanced data management.

Data Types

Boolean

Default false

Description

Setting this variable to *true* enables the feature advanced data management. When the feature is enabled, the tool will use a new method of storing design data that reduces in-memory consumption and allows larger designs to be analyzed. The feature is recommended for designs with over 100 million instances, particularly those using parametric on-chip variation (POCV) analysis with moment-based modeling. There is a small increase in runtime and disk space usage when this feature is enabled. As a convenience, when the feature is enabled, the tool also sets the sh_high_capacity_effort variable to high. (The default high capacity effort level is medium).

By default, the variable is set to *false*.

See Also

- sh_high_capacity_effort
- set_program_options

Feedback

timing_enable_auto_mux_clock_exclusivity

Enables automatic inference of MUX cells for clock exclusivity.

Data Types

Boolean

Default false

Description

If you set this variable to *true*, the tool automatically identifies the MUX cells on the clock network and forces the clocks that go through different data lines of the MUX cells to be exclusive.

By default, the variable is set to *false* and the tool does not automatically infer the exclusivity of clocks from MUX cells in the clock network. Please note that if there is a generated clock defined after the exclusivity point, the exclusivity states will be lost. This is similar to the existing behavior of the generated clocks. When a generated clock is created at a pin, all other clocks arriving at that pin are blocked unless they also have generated clock versions created at that pin.

Please note that this feature is not currently supported in *extract_model* and *write_eco_design* commands. PrimeTime will honor exclusivity settings in Hyperscale. Timing context with exclusivity will be captured by *characterize_context* command, and written out by *write_context* command. Please note that cross-boundary exclusivity will only be written in binary GBC format. It is not included in ASCII PTSH constraint format.

See Also

- set_clock_exclusivity
- set_disable_auto_mux_clock_exclusivity
- report_clock

timing_enable_clock_propagation_through_preset_clear

Enables propagation of clock signals through preset and clear pins.

Data Types

Boolean

Default false

Feedback

Description

If you set this variable to *true*, clock signals propagate through the preset and clear pins of a sequential device. This occurs only if clock signals are incident on such pins.

If CRPR is enabled, the tool considers any sequential devices in the fanout of such pins for analysis.

---

timing_enable_clock_propagation_through_three_state_enable_pins

Allows the clocks to propagate through the enable pin of a three-state cell.

Data Types

int

Default false

Description

If this variable is set to *false* (the default), PrimeTime does not propagate clocks between a pair of pins if there is at least one timing arc with a disable sense between those pins.

To allow the clocks to propagate through the enable pins of three-state cells, set this variable to *true*.

---

timing_enable_constraint_variation

Enables constraint variation in parametric on-chip variation (POCV) analysis.

Data Types

Boolean

Default false

Description

If you set this variable to *true* and set the *timing_pocvm_enable_analysis* variable to *true*, constraint variation is considered in parametric on-chip variation (POCV) analysis.

See Also

- get_timing_paths
- read_aocvm
- report_aocvm

Feedback

- report_timing
- set_operating_conditions
- timing_pocvm_enable_analysis

---

timing_enable_cross_voltage_domain_analysis

Enables Simultaneous Multi Voltage Analysis

Data Types

Boolean

Default false

Description

Set this variable to true to enable Simultaneous Multi Voltage Analysis of timing paths. The analysis flow type (or mode) can be selected using the related variable *iming_cross_voltage_domain_analysis_mode*. Graph-based Simultaneous Multi Voltage Analysis is performned in each one of the following analysis modes: *full*, *only_crossing* and *exclude_crossing*. In these analysis modes, PrimeTime automatically identifies and analyzes the design at all the relevant voltage combinations according to the user specification. If the *legacy* analysis mode is selected, PrimeTime performs a legacy flow which identifies and analyzes only the cross-domain paths using path-based analysis and re-calculation. The first section below describes in more detail the first three analysis modes, and the second section details the legacy analysis mode.

- *Graph-based Simultaneous Multi Voltage Analysis modes full*, *only_crossing*, *exclude_crossing* Set any of these analysis modes to enable efficient concurent analysis of the design paths at all the relevant voltage configurations, with support for any number of voltage levels per voltage domain. PrimeTime automatically identifies all the relvant voltage configurations according to the user specification, accurately analyzing the design paths. These are the flow steps and the corresponding data necessary for running Graph-based Simultaneous Multi Voltage Analysis:
  - Enabling cross-domain analysis and defining the analysis type using *timing_enable_cross_voltage_domain_analysis* and *iming_cross_voltage_domain_analysis_mode*
  - Defining scaling library groups Libraries characterized at the desired voltage configurations are required to run Graph-based Simultaneous Multi Voltage Analysis. Scaling library groups of the relevant libraries are specified using the *define_scaling_lib_group* command. Both *-exact_match_only* and scaling flows are supported.
  - Reading the *UPF* specification or using *PG Verilog*

Feedback

    - Specifying Voltage Domains and Reference Voltage Levels using the commands *get_supply_group* and *set_voltage_levels*, respectively.
    - Providing voltage values with *set_voltage* commands.
    - [Optional] Configuring DVFS scenarios for selective SMVA analysis using the *configure_dvfs_scenarios* command
    - [Optional] Defining DVFS scenarios and DVFS scenario dependent design constraints using the *get_dvfs_scenarios* command and the *-dvfs_scenarios* command options of the relevant design constraints commands. This is useful for associating different constraints with different voltage configurations. For example, clock waveforms can be specified to take different values at different voltage configurations.

- *legacy Path-based Simultaneous Multi Voltage Analysis mode* Set this analysis mode to enable reduced-pessimism analysis of the timing paths that cross multiple voltage domains. Standard on-chip variation analysis leads to pessimistic results for cross-domain paths because it can consider the behavior of different cells operating at both the high voltage and low voltage within a given domain at the same time. After single-domain path violations are found and fixed using standard on-chip variation analysis, you can analyze just the cross-domain paths and reduce the pessimism of the results by setting this analysis mode.

  When this analysis mode is used, the *update_timing* command identifies paths that traverse multiple voltage domains and subsequently limits the reporting to only those paths. If a path-based analysis is performed by using the *-pba_mode path* option of the *report_timing* command, PrimeTime performs an efficient path-based recalculation of the cross-domain paths. This analysis finds the worst-case voltage for each domain crossed by each path, but eliminates the pessimism that occurs in standard on-chip variation analysis.

  If you do not have any reliable characterization information with respect to voltage variation, you can use derating to analyze the impact of different supply voltages. The *set_cross_voltage_domain_analysis_guardband* command sets derating factors that apply whenever the *timing_enable_cross_voltage_domain_analysis* variable is set to true. Note: the legacy analysis mode is limited to two voltage levels per voltage domain.

See Also

- timing_cross_voltage_domain_analysis_mode
- report_timing
- set_cross_voltage_domain_analysis_guardband
- update_timing

Feedback

- set_voltage_levels
- set_voltage
- define_scaling_lib_group

timing_enable_cumulative_incremental_derate

Enables or disables the accumulation of incremental derate values.

Data Types

Boolean

Default false

Description

This variable controls whether the *set_timing_derate -increment* command replaces or adds to the existing incremental value stored at the specified object scope.

When set to *false* (the default), the *set_timing_derate -increment* command completely replaces any existing incremental derate setting stored at that object scope.

When set to *true*, the *set_timing_derate -increment* command numerically adds the specified incremental derate value to any existing incremental derate value stored at that object scope.

This variable does not cause incremental derate settings across different object scopes (leaf cell/net, library cell, hierarchical cell, global) to be added together; such settings still follow the usual object scope precedence rules.

For example, given the following commands:

```
set_app_var timing_enable_cumulative_incremental_derate true
set_timing_derate -late -cell_delay -increment 0.01 [get_cells core/U123]
set_timing_derate -late -cell_delay -increment 0.01 [get_cells core/U123]
set_timing_derate -late -cell_delay -increment 0.04 [get_cells core]
set_timing_derate -late -cell_delay -increment 0.04 [get_cells core]
```

then the leaf cell incremental derate of 0.02 is used for core/U123. If the leaf cell incremental derate is subsequently removed:

```
reset_timing_derate -increment [get_cells core/U123]
```

then the hierarchical cell incremental derate of 0.08 is used for core/U123.

See Also

- set_timing_derate

Feedback

timing_enable_cumulative_incremental_path_margin

Controls the accumulation of *set_path_margin -increment* values.

Data Types

Boolean

Default false

Description

If you set this variable to *true*, the value of the margin for the path is the sum of all the incremental margins.

If you set this variable to *false*, the value of the margin for the path is the last applied incremental margin. Last specified path margin with *-increment* option, overrides all the old increment values.

See Also

- set_path_margin

timing_enable_data_check_default_group

Controls whether data-to-data check paths are included in a separate ***data_check_default*** path group.

Data Types

boolean

Default false

Description

This variable controls how data-to-data check timing paths are allocated to path groups.

When set to *false* (the default), data check paths are placed in the path group of their endpoint clock.

When set to *true*, all data check paths are included in a dedicated default path group named ***data_check_default***.

See Also

- set_data_check
- report_path_group

Feedback

- remove_path_group
- timing_enable_override_default_groups

---

timing_enable_derate_multi_dimension_interpolation

Enables or disables the multi-dimensional interpolation of scaled derate values on library cells.

Data Types

Boolean

Default true

Description

This variable controls how multiple-rail library cell timing derates are computed in voltage scaling flows.

When set to *true* (the default),

- If a multiple-rail cell's voltage configuration does not exactly match a scaling library, a multi-dimensionally scaled (interpolated) derate computed from nearby scaling libraries is used.
- If a multiple-rail cell's voltage configuration exactly matches that of a scaling library, that library's derate data is used.

When set to *false*,

- If a multiple-rail cell's voltage configuration does not exactly match a scaling library, the worst-case derate from nearby scaling libraries is used.
- If a multiple-rail cell's voltage configuration exactly matches that of a scaling library, that library's derate data is used.

This variable must be set before the design is linked. It also requires that regular scaling interpolation be enabled by setting the *timing_enable_derate_scaling_interpolation_for_library_cells* variable to *true* (the default is *false*).

Liberty Variation Format (LVF) variation tables stored in scaling group libraries are always scaled by the delay calculation engine; this variable does not apply.

See Also

- set_timing_derate
- read_ocvm

Feedback

- timing_enable_derate_scaling_for_library_cells_compatibility
- timing_enable_derate_scaling_interpolation_for_library_cells

timing_enable_derate_scaling_for_library_cells_compatibility

Enables or disables derate scaling on library cells in scaling group libaries.

Data Types

Boolean

Default true

Description

When set to *true* (the default),

- Only timing derates applied to library cells in link libraries are considered.
- Timing derates applied to library cells in scaling libraries (other than the link library) are ignored.

When set to *false*,

- Timing derates applied to library cells across all scaling group libraries are considered.
- The derate is computed according to the setting of the *timing_enable_derate_scaling_interpolation_for_library_cells* variable.

This variable must be set before the design is linked.

In a future release of PrimeTime, the default of this variable could change to *false*, which could affect the behavior of timing derates in scaling flows.

Liberty Variation Format (LVF) variation tables stored in scaling group libraries are always scaled by the delay calculation engine; this variable does not apply.

See Also

- define_scaling_lib_group
- set_timing_derate
- timing_enable_derate_scaling_interpolation_for_library_cells

timing_enable_derate_scaling_interpolation_for_library_cells

Enables or disables interpolation based derate scaling on library cells in scaling group libaries.

Feedback

Data Types

Boolean

Default false

Description

This variable controls how library cell timing derates are computed in voltage scaling flows.

When set to *false* (the default),

- Only derates applied by the *set_timing_derate* command are considered across scaling group libraries.
- If a cell's voltage configuration exactly matches that of a scaling library, that library's derate data is used.
- If a cell's voltage configuration does not exactly match a scaling library, the worst-case derate from nearby scaling libraries is used.
- For table-based timing derates read in by the *read_ocvm* command, only the link library tables are used.

When set to *true*,

- Scaling is performed for both *set_timing_derate* derates and table-based timing derates read in by the *read_ocvm* command.
- If a cell's voltage configuration exactly matches that of a scaling library, that library's derate data is used.
- If a cell's voltage configuration does not exactly match a scaling library, a fully scaled (interpolated) derate computed from nearby scaling libraries is used.
- If a cell has multiple rails, then the behavior depends on the *timing_enable_derate_multi_dimension_interpolation* variable value.

This variable must be set before link.

All derate tables will be purged when the state of this variable changes. This variable has no effect when timing derate scaling is disabled because the *timing_enable_derate_scaling_for_library_cells_compatibility* variable is set to its default of *true*.

Liberty Variation Format (LVF) variation tables stored in scaling group libraries are always scaled by the delay calculation engine; this variable does not apply.

Feedback

See Also

- define_scaling_lib_group
- set_timing_derate
- read_ocvm
- timing_enable_derate_scaling_for_library_cells_compatibility

---

timing_enable_dvd_analysis

Enables or disables PrimeTime DvD, which performs dynamic voltage drop analysis.

Data Types

Boolean

Default false

Description

By default, this variable is set to *false*, which disables PrimeTime DvD.

To enable PrimeTime DvD, set this variable to *true*, so that the *report_timing* and *report_voltage_robustness* commands can perform DvD-aware timing calculations. If you set this variable to *true*, you must also do the following:

1. Obtain a PrimeTime DvD license. You cannot use PrimeTime DvD without a license.
2. Use the *read_dvd* command to read in the dynamic voltage drop information for your design.

See Also

- read_dvd
- remove_dvd
- report_timing
- report_voltage_robustness

---

timing_enable_from_clock_to_dangling_load_endpoints

Allows *set_max_delay* and *set_min_delay* specifications with *-from* to apply to diode/antenna cells as endpoints.

Feedback

Data Types

Boolean

Default false

Description

This variable controls whether *set_max_delay* and *set_min_delay* specifications can apply to diode/antenna cells as endpoints.

The default is *false*, which does not apply the specifications to diode/antenna cells.

When this variable is set to *true*, *set_max_delay* and *set_min_delay* specifications will apply to diode/antenna cells (and other similar cells with no forward arcs) as endpoints when all of the following are true:

- The specification uses the *-from*, *-rise_from*, or *-fall_from* option.
- The "from" *object_list* consists only of clock objects.
- No other path selection objects (such as *-through* or *-to*) are specified.

Additional runtime can result when using this feature.

Regular logic cells (with timing arcs) that do not drive any nets are unaffected by this feature. Only special cells with no forward arcs are affected.

See Also

- set_max_delay
- set_min_delay

---

timing_enable_graph_based_refinement

Enables HyperTrace graph-based refinement for faster exhaustive path-based analysis reporting.

Data Types

Boolean

Default false

Description

Setting this variable to *true* enables HyperTrace accelerated exhaustive path-based analysis (PBA) for reporting. This feature requires a PrimeTime-ADV-PLUS license.

Feedback

HyperTrace is a technology that accelerates the exhaustive PBA search by computing refined graph-based timing data, then using it to drive path searching and recalculation. This variable enables the use of graph-based refinement for the *-pba_mode exhaustive* option of various reporting commands (*report_timing*, *report_constraint*, and so on).

HyperTrace ECO fixing is controlled by a separate variable, *eco_enable_graph_based_refinement*. For details, see the man page.

By default, this variable is set to *false*, which disables graph-based refinement for reporting.

With graph-based refinement enabled, when exhaustive path-based analysis is invoked for the first time, the tool first refines the timing on the critical (violating) portion of the design found by graph-based analysis. This accelerates all subsequent exhaustive path-based analyses.

Graph-based refinement is recommended when:

- The design is near the signoff stage
- Exhaustive path-based analysis is taking more than 50 percent of the flow runtime
- Fewer than 10 percent of the pins in the design have timing violations

Following these guidelines helps ensure that the acceleration of exhaustive path-based analysis outweighs the refinement analysis overhead.

By default, the refinement analysis includes all pins that belong to paths with negative slack. This is appropriate for the default *-slack_lesser_than* setting of 0.0 in signoff analysis. If you set the *-slack_lesser_than* option of the reporting command to a value other than 0.0, whether negative or positive, you need to ensure that the slack threshold for graph-based refinement is set to the same value or a higher (more positive) value.

These variables control the refinement slack thresholds for reporting:

- *timing_refinement_max_slack_threshold* for maximum delay (setup) analysis
- *timing_refinement_min_slack_threshold* for minimum delay (hold) analysis

Setting a refinement slack threshold to a lower (more negative) value causes fewer pins to be considered for refinement, saving runtime and memory. However, if the threshold is set too low, graph-based refinement provides no benefit to the exhaustive path-based analysis. The threshold is too low when it is lower than the *-slack_lesser_than* setting of the reporting command.

The optimum refinement threshold value is equal to the *-slack_lesser_than* setting of the reporting command. If you are running multiple exhaustive path-based analyses using different *-slack_lesser_than* settings, set the refinement threshold equal to the largest (most positive) of those settings.

Feedback

By default, the refinement slack thresholds are set as follows:

```
timing_refinement_max_slack_threshold 0.0
timing_refinement_min_slack_threshold disabled
```

You can set each threshold to any value (positive, negative, or zero) or to the string "*disabled*" to disable graph-based refinement. For minimum delay (hold) analysis, refinement is disabled by default because minimum-delay paths are typically short, exhaustive analysis is typically very fast, and graph-based refinement does not provide a benefit.

See Also

- report_constraint
- report_timing
- timing_refinement_max_slack_threshold
- timing_refinement_min_slack_threshold

---

timing_enable_hier_boundary_checks

Specifies whether to skip scope checking for HyperScale blocks.

Data Types

Boolean

Default false

Description

In some HyperScale flows, including some bottom-up flows, you do not need to perform hierarchical scope checking for some blocks. In that case, to save runtime and disk space, set *timing_enable_hier_boundary_checks* to false. Then the *write_hier_data* command will skip scope checking, except a small subset needed for nonfixable scope violations. Nonfixable violations are still checked and reported by the *report_constraint* command.

See Also

- report_constraint
- set_hier_config
- write_hier_data
- timing_save_hier_model_data
- timing_save_hier_context_data

Feedback

timing_enable_independent_crosstalk_in_data_network

Controls the computation of crosstalk delta delay for a victim net with edges launched by multiple clocks.

Data Types

string

Default none

Description

This variable controls the timing window calculation of delta delay for a victim net in the presence of multiple clocks. Modern designs often combine multiple modes, referred to as a merged mode, into a single timing analysis run, resulting in crosstalk pessimism.

When this variable is set to its default of *none*, the tool computes a single worst-case delta delay for a data network victim net based on the timing windows of all clocked edges arriving at the victim. This can result in pessimism because different clocks might have different arrival characteristics and thus different delta delays.

For example, a particular design with functional and test modes might have a larger delta delay in the test mode than in the functional mode. Without this feature, when performing a single signoff run using a merged mode with both the functional and test clocks, the tool computes a single worst-case delta delay for the victim net, across both the test and functional modes. The delta delay of the victim nets for paths in the functional mode are therefore dependent on the delta delay of the paths in the test mode, and vice versa. This can creates pessimism between the timing of the merged mode analysis, as compared to individual functional and test mode analysis. Note that for data nets, this pessimism occurs even if the functional and test clocks are made physically exclusive with respect to each other.

To reduce the pessimism of worst-case delta delays, this variable can be set to the following values:

- *all_clocks* - In this mode, the tool computes and stores a separate delta delay for each individual clock arriving at the victim. This mode can provide greater pessisim reduction, but it can come at the cost of increased analysis runtime if there are many coincident clocks in the design.
- *physically_exclusive_clocks* - In this mode, the tool computes a separate delta delay for each group of victim net clock domains that can physically coexist at the same time. This setting can still provide some pessimism reduction, but with less memory and runtime than the *all_clocks* setting. If the design has no physically exclusive clock constraints, the results are equivalent to a setting of *none*.

Feedback

This mode considers the following:

- Clocks specified as globally physically exclusive by the *set_clock_groups -physically_exclusive* command
- Clocks inferred as locally physically exclusive by the *set_clock_exclusivity* command or the *timing_enable_auto_mux_clock_exclusivity* variable

In both modes, only different clocks can result in separate delta delay calculations; different-arrival versions of the same clock do not result in separate delta delay calculations.

If this feature was enabled in the preceding merged-mode example, the path driven by the functional clock would have a smaller delta delay compared to the path driven by the test clock, thus reducing pessimism.

Although the variable can be used as a general pessimism reduction feature, it is intended for use in merged-mode design analysis to narrow the gap between the merged mode results and the individual mode results.

This variable affects only data net delta delays. Clock nets always use the equivalent of *all_clocks* (per-clock) delta delay computation.

See Also

- si_enable_analysis
- set_clock_groups

---

timing_enable_library_max_cap_lookup_table

Specifies whether the frequency-indexed lookup table values for max_capacitance, if they exist, are to be used.

Data Types

Boolean

Default true

Description

If this variable is set to *true* (the default), and a max_capacitance lookup table is defined on the library pin, the lookup table is used to determine the max_capacitance value applied at the pin. The lookup table is indexed by clock frequency, the inverse of the clock period. The actual max_capacitance applied is the most restrictive max_capacitance

Feedback

constraint for the specified pin. Specifically, the smallest max_capacitance value is taken among:

- The lookup table-derived value
- The library-cell max_capacitance value
- All user-specified max_capacitance values (at the pin, port, clock, or design level) at the pin

To ignore the lookup table-derived value, set this variable to *false*.

See Also

- report_constraint
- set_max_capacitance
- timing_enable_max_cap_precedence
- timing_library_max_cap_from_lookup_table

---

timing_enable_max_cap_precedence

Enables precedence rules for max_capacitance constraints.

Data Types

Boolean

Default false

Description

If the *timing_enable_max_cap_precedence* variable is set to *false* (the default), the most restrictive (smallest) max_capacitance pertaining to a pin or port is applied.

If you set the *timing_enable_max_cap_precedence* variable to *true*, the pin- or port-level max_capacitance value takes precedence over the library-derived max_capacitance value, which in turn take precedence over design-level max capacitance value. A max_capacitance limit set on a design or on a clock is considered to be a design-level limit.

See Also

- report_constraint
- set_max_capacitance

Feedback

- timing_library_max_cap_from_lookup_table
- timing_enable_max_slew_precedence

---

timing_enable_max_capacitance_set_case_analysis

Enables the checking of the max capacitance constraint on constant pins.

Data Types

Boolean

Default false

Description

To check the max capacitance constraint for constant pins, set this variable to *true*.

See Also

- report_constraint
- timing_enable_max_transition_set_case_analysis

---

timing_enable_max_slew_precedence

Enables precedence rules for max_transition constraints.

Data Types

Boolean

Default false

Description

If the *timing_enable_max_slew_precedence* variable is set to *false* (the default), the most restrictive (smallest) max_transition pertaining to a pin or port is applied.

If the *timing_enable_max_slew_precedence* variable is set to *true*, the pin- or port-level max_transition value takes precedence over the library-derived max_transition value, which in turn take precedence over design-level max_transition value. A max_transition limit set on a design or on a clock is considered to be a design-level limit.

Feedback

See Also

- report_constraint
- set_max_transition
- timing_enable_max_cap_precedence

timing_enable_max_transition_set_case_analysis

Enables the checking of the max transition constraint on set case constant pins.

Data Types

Boolean

Default false

Description

This variable is used to enable the checking of transition on case constant pin if there is transition exist. If the variable is set as false, all transitions on case constant pin will be ignored. The variable will not re-enable the case disabled arcs. To check the max transition constraint for set case constant pins, set this variable to *true*.

See Also

- report_constraint
- timing_enable_max_capacitance_set_case_analysis

timing_enable_min_pulse_width_waveform_adjustment

Enables advanced waveform adjustments during minimum pulse width checking.

Data Types

Boolean

Default false

Description

Setting this variable to *true* enables advanced waveform adjustments in the clock network during minimum pulse width checking.

By default, the variable is set to *false* and the tool measures the width of a pulse where the clock waveform crosses the voltage threshold at 50 percent of the supply voltage.

Feedback

When the variable is set to *true*, the measurement threshold can be set to any fraction of the supply voltage between 0.0 and 1.0. The measurement adjustments occur only when advanced waveform propagation is enabled by setting the *delay_calc_waveform_analysis_mode* variable. Set the thresholds using the *timing_min_pulse_width_high_pulse_threshold* and *timing_min_pulse_width_low_pulse_threshold* variables.

To see the minimum pulse with check with advanced waveform adjustments, use the *report_min_pulse_width* command.

See Also

- report_min_pulse_width
- set_min_pulse_width
- delay_calc_waveform_analysis_mode
- timing_min_pulse_width_high_pulse_threshold
- timing_min_pulse_width_low_pulse_threshold

timing_enable_native_vt_mistracking_analysis

This is a synonym for the *vt_mistracking_enable_path_analysis* variable.

Data Types

Boolean

Default false

See Also

- set_vt_mistracking_derate
- update_timing
- get_timing_paths
- vt_mistracking_enable_bounding_analysis
- vt_mistracking_analysis_mode

timing_enable_normalized_slack

Enables or disables normalized slack analysis during timing updates.

Feedback

Data Types

Boolean

Default false

Description

If this variable is set to *true*, the tool performs normalized slack analysis during timing updates. If set to *false* (the default), the tool does not perform normalized slack analysis and reporting.

Normalized slack analysis is an additional optional analysis that computes a normalized slack for each timing path. The normalized slack is the slack divided by the idealized allowed propagation delay. For paths in a single clock domain, the allowed propagation delay is the time between two different edges of the clock. For 50% duty cycle clocks, the allowed propagation delay is usually an integer number of half periods.

Normalized slack can be used to determine the paths which limit the clock frequency. Normalized slack prioritizes violating paths allowed few clock cycles. Fixing these paths first allows the most improvement in the clock period.

If normalized slack analysis is enabled during update timing, paths can be gathered and reported using normalized slack, using the *report_timing -normalized_slack* and *get_timing_paths -normalized_slack* commands.

See Also

- report_timing

---

timing_enable_override_default_groups

Specifies whether to group recovery/removal, clock-gating setup/hold, and data check paths to user created path groups if they satisfy user path group specifications, instead of grouping them to corresponding default groups, i.e. ***async_default***, ***clock_gating_default*** and ***data_check_default***.

Data Types

Boolean

Default true

Description

When this variable is set to *true* (the default), any recovery/removal and clock-gating setup/hold path will be put into user created path group if they satisfy the specifications provided to *group_path* command.

Feedback

Consequently this impacts all commands report path, such as *report_constraint*. For example, when *report_constraint* is called with *-recovery_group* option, it checks only the default ***async_default*** path group, if any recovery path is moved to a user create path group, it will not be checked or reported by *report_constraint -recvoery_group*, but instead it will be checked and reported by *report_constraint -max_delay_groups*.

When this variable is set to *false*, then any recovery/removal and clock_gating setup/hold path will stay in corresponding default path groups, i.e. ***async_default*** and ***clock_gating_default***.

Data check timing path allocation to path group is controlled by *timing_enable_data_check_default_group* and *timing_enable_override_default_groups* only impacts the grouping of data check timing paths when *timing_enable_data_check_default_group* is true.

This variable affects all commands report path, such as *report_timing* and *report_constraint* command, as impacted checks may be moved to a different path group.

See Also

- report_constraint
- report_path_group
- group_path
- timing_enable_data_check_default_group

---

timing_enable_preset_clear_arcs

Enables preset and clear arcs.

Data Types

Boolean

Default false

Description

When this variable is set to *false* (the default), PrimeTime disables all preset and clear timing arcs.

To enable asynchronous preset and clear timing arcs for use during timing path analysis, set this variable to *true*.

The tool performs minimum pulse width checks defined on asynchronous preset and clear pins, regardless of this variable setting.

Feedback

See Also

- report_timing

---

timing_enable_pulse_clock_constraints

Enables the checking of pulse clock constraints.

Data Types

Boolean

Default true

Description

This variable determines if pulse clock constraints are checked or not.

When this variable is *true*, the tool checks the pulse clock constraints set by the *set_pulse_clock_min_width*, *set_pulse_clock_max_width*, *set_pulse_clock_min_transition*, and *set_pulse_clock_max_transition* commands.

When this variable is *true*, the min pulse width constraints set by the *set_min_pulse_width* command do not apply to pulse clock networks and more specific pulse clock constraints checked.

See Also

- report_constraint
- report_pulse_clock_max_transition
- report_pulse_clock_max_width
- report_pulse_clock_min_transition
- report_pulse_clock_min_width
- set_pulse_clock_max_transition
- set_pulse_clock_max_width
- set_pulse_clock_min_transition
- set_pulse_clock_min_width

---

timing_enable_slew_variation

Enables transition variation in parametric on-chip variation (POCV) analysis.

Feedback

Data Types

Boolean

Default true

Description

When set to *true*, this variable enables transition variation in parametric on-chip variation (POCV) analysis.

The corner used for timing analysis is specified by the *timing_pocvm_corner_sigma* variable.

If the *timing_enable_slew_variation* variable is set to *true*, variation reporting is enabled for *report_constraint -max_transition -verbose*. The corner used for reporting is specified by the *timing_pocvm_max_transition_sigma* variable.

This variable has an effect only when POCV itself is enabled by setting the *timing_pocvm_enable_analysis* variable to *true*.

Examples

```
pt_shell> report_constraint -max_tran -verbose -sig 4 XV0_0/a
    Sigma: 3.0
    Pin: XV0_0/a
                                 Mean    Sensit     Value
    max_transition             0.3726             0.3726
  - Transition Time            0.5000    0.0000    0.5000
  -------------------------------------------------------
    Slack                     -0.1274    0.0000   -0.1274   (VIOLATED)
```

See Also

- get_timing_paths
- read_aocvm
- report_aocvm
- report_constraint
- report_timing
- set_operating_conditions
- timing_pocvm_corner_sigma
- timing_pocvm_enable_analysis
- timing_pocvm_max_transition_sigma

Feedback

timing_enable_through_paths

Enables or disables advanced analysis and reporting through transparent latches.

Data Types

Boolean

Default false

Description

When set to *true*, this variable enables advanced analysis through transparent latches during timing updates and reporting. When set to *false* (the default), the tool disables advanced analysis and reporting through transparent latches.

By default, the tool analyzes and reports paths through transparent latches as a series of path segments between latches. These segments can be reported together using the *report_timing* option *-trace_latch_borrow*. Max pin slacks (*max_rise_slack*, *max_fall_slack*) for a pin in the design can be affected by borrowing latches in the fanin of the pin, but are not affected by timing calculations in parts of the design past the first level of latches in the fanout of the pin.

If you enable advanced analysis through transparent latches, you can report paths through latches as a single timing path. Pin slacks can be affected by timing calculations past the first level of latches in the fanout. In addition, you can report specific paths through latches by using the *-from*, *-through*, and *-to* options of *report_timing*, where the options specify objects that are separated by one or more transparent latches.

The advanced analysis is limited when there are latch loops in the design. The tool chooses specific latch data pins in the loops to act as loop breaker latches. For these latch data pins, the behavior is the same as if the *timing_enable_through_paths* variable was set to *false*. Reporting through these special latch data pins is not supported. The tool automatically selects which latch data pins to act as loop breaker latches. You can guide the selection using the *set_latch_loop_breaker* command. Because of the runtime associated with the advanced analysis, by default the tool also selects some latch data pins outside loops to have the same behavior as if *timing_enable_through_paths* was *false*. You can use the *timing_through_path_max_segments* variable to control the selection of these pins.

See Also

- report_timing
- set_latch_loop_breaker
- timing_through_path_max_segments

Feedback

timing_enable_transition_derate_exclude_mshift

Enables the transition meanshift quantity to be derated.

Data Types

Boolean

Default true

Description

When this variable is set to *true* (the default), PrimeTime applies the nominal transition derate against the transition meanshift quantity. When set to alse *(non default), then the annotated transition derate is applied only to the nominal transition.*

timing_enable_unit_delay

Enables the unit delay calculation mode of timing analysis.

Data Types

Boolean

Default false

Description

Setting this variable to *true* enables the unit delay calculation mode. In this mode, the tool computes a 1.0 unit delay value (in library units) for every cell, net, and coupled-net delta delay, and for pin transitions and coupled-net delta transitions.

This mode of analysis can be used for flow testing and constraint validation wherein the numerical timing results can be ignored. This feature improves the runtime, although it might not reduce memory consumption.

For best runtime, you should also disable the following features -- CRPR, SI, AOCVM, POCVM, and advanced latch analysis -- if you are using them:

```
set_app_var timing_remove_clock_reconvergence_pessimism false
set_app_var si_enable_analysis false
set_app_var timing_aocvm_enable_analysis false
set_app_var timing_pocvm_enable_analysis false
set_app_var timing_enable_through_paths false
```

See Also

- si_enable_analysis
- timing_enable_through_paths

Feedback

- timing_pocvm_enable_analysis
- timing_remove_clock_reconvergence_pessimism

---

timing_enable_via_variation

Enables via variation analysis.

Data Types

Boolean

Default false

Description

Setting this variable to *true* enables the via variation analysis. Using this data in POCV timing analysis produces more accurate results by taking into account the effects of interconnect via variation.

To use this feature, POCV analysis must be enabled (*timing_pocvm_enable_analysis* variable set to *true*) and you must specify the variation parameters in a text file, and read in that data with the *read_ivm* command.

See Also

- read_ivm
- report_ivm
- timing_pocvm_enable_analysis

---

timing_enable_voltage_swing

Enables voltage swing checking in advanced waveform propagation mode.

Data Types

Boolean

Default false

Description

Setting this variable to *true* enables voltage swing checking, which determines whether a high pulse gets close enough to the power rail voltage, and whether a low pulse gets close enough to the ground rail voltage. By default, the variable is set to *false* and no voltage swing checking is performed.

Feedback

Voltage swing checking, when enabled, operates only on clock network pins, and only when advanced waveform propagation is enabled by setting the *delay_calc_waveform_analysis_mode* variable to *full_design*.

To perform voltage swing checking, use the *report_min_pulse_width* command with the *-voltage_swing* option.

To set the threshold voltages used for checking the pulse heights, set the *timing_voltage_swing_low_pulse_threshold* and *timing_voltage_swing_high_pulse_threshold* variables.

See Also

- report_min_pulse_width
- set_min_pulse_width
- delay_calc_waveform_analysis_mode
- timing_voltage_swing_high_pulse_threshold
- timing_voltage_swing_low_pulse_threshold

timing_exception_preserve_user_object_order

Specifies whether PrimeTime tries to preserve the original object order within exceptions for printing.

Data Types

Boolean

Default false

Description

If this variable is set to *false* (the default), the tool saves memory and does not preserve the original object order within timing exceptions. for the purposes of *write_script*, *report_exceptions*, and *write_sdc*.

If you set this variable to *true*, the tool tries to preserve the original object order within exceptions. There are some situations where object order will not be preserved. If exceptions are partly overwritten by later exceptions, object or may not be preserved. If exception specification uses mixed object types (clock/pin/net/cell) in the same -from/-through/-to specifier, object order may not be preserved. Object order is not preserved if the exception is part of a Hyperscale model or block context.

This variable has no effect on timing results. It only affects printing of timing exceptions.

Feedback

Setting this variable to *true* can cause large increases in memory usage if many timing exceptions exist on the design, or the timing exceptions contain many objects.

Timing exceptions affected include *set_multicycle_path*, *set_false_path*, *set_max_delay*, *set_min_delay*, and *set_path_margin*.

The variable must be on when the exception is specified in order to take effect. If the variable is on when some exceptions are specified, and off when others are specified, then some exceptions may have their object order preserved for printing.

See Also

- write_script
- write_sdc

---

timing_filter_arcs_for_crossing

Specifies what arcs to exclude when inferring domain crossing in simultaneous multivoltage analysis (SMVA) analysis.

Data Types

string

Default none

Description

This variable specifies which arc types to exclude from checking when simultaneous multivoltage analysis (SMVA) is inferring domain crossing paths.

\timing_filter_arcs_for_crossing variable.

The allowed values are:

- *none* - All arcs are checked.
- *cell_arcs* - Cell arcs are excluded from checking for domain crossing.
- *net_arcs* - Net arcs are excluded from checking for domain crossing.
- *only_level_shifter* - Only cell arcs of cells with is_level_shifter attribute.

See Also

- timing_cross_voltage_domain_analysis_mode

Feedback

timing_gclock_expand_odd_period_edge_list

Controls the odd period edge list expansion of the generated clock to one more cycle.

Data Types

Boolean

Default true

Description

If you set this variable to *true*, the generated clocks with odd period edge list will consider all transition possibilities by expanding to one more cycle.

If you set this variable to *false*, the generated clocks with odd period edge list will not consider all transition possibilities by expanding to one more cycle. It considers only one cycle.

See Also

- set_clock_jitter

timing_gclock_source_network_num_master_registers

Specifies the maximum number of register clock pins clocked by the master clock allowed in generated clock source latency paths.

Data Types

integer

Default 10000000

Description

This variable specifies the maximum number of register clock pins clocked by the master clock allowed in generated clock source latency paths. The variable does not effect the number of register traversed in a single path that do not have a clock assigned or are clocked by another generated clock that has the same primary master as the generated clock in question.

Register clock pins or transparent-D pins of registers clocked by unrelated clocks are not traversed in determining generated clock source latency paths. An unrelated clock is any clock that the primary master clock differs from the generated clock when source latency paths are being computed.

Feedback

timing_ideal_clock_zero_default_transition

Specifies whether or not a zero transition value is assumed for sequential devices clocked by ideal clocks.

Data Types

Boolean

Default true

Description

This variable specifies a transition value to use at clock pins of a flip-flop. If you run the *set_clock_transition* command, and the clock is ideal, the transition value is used, and this variable has no effect. If the clock transition is not set by the *set_clock_transition* command, and the clock is ideal, then this variable has the following effect:

- *true* (the default) - Uses a zero transition value for ideal clocks.
- *false* - Uses a propagated transition value.

If the clock is ideal, the *set_clock_transition* command overrides the effect of this variable.

See Also

- report_delay_calculation
- set_clock_transition

timing_include_available_borrow_in_slack

Specifies whether PrimeTime includes available borrow time in slack.

Data Types

Boolean

Default false

Description

If this variable is set to *false* (the default), the slack of a signal arriving before the latch opening edge is measured relative to the open edge and does not include available borrow time. A signal arriving during the transparent interval is considered to have a slack of zero. Violations are measured with respect to the closing latch edge.

If you set this variable to *true*, any path terminating at the data pin of a transparent latch has positive or negative slack measured with respect to the closing transition at the latch. That is, available borrow time is considered a component of slack. Available borrow time is

Feedback

typically the duration of the active clock region minus the setup time required. A maximum time borrow set on a latch could decrease this available borrow time.

If this variable is set to *true*, slack results can be misleadingly optimistic in the sense that a pin can appear to have positive slack, but increasing delay at the pin can lead to additional borrowing and timing violations downstream of the latch. This is a serious problem for ECO flows. Do not enable this variable when running ECO flows, or PrimeTime ECO in particular. If the variable is enabled during PrimeTime ECO fixing, additional setup violations can occur.

For most situations, rather than using *timing_include_available_borrow_in_slack*, it is better to use advanced latch analysis, by enabling *timing_enable_through_paths*. The advanced latch analysis provides slack calculations through most latches, without optimism for ECO flows.

See Also

- report_timing
- set_max_time_borrow

---

timing_include_uncertainty_for_pulse_checks

Specifies how the clock uncertainty is applied to minimum period and minimum pulse width checks.

Data Types

string

Default setup_hold

Description

To specify how the clock uncertainty is applied for minimum period and minimum pulse width checks, set this variable to one of the following values:

- *setup_hold* - Applies the worst setup or hold uncertainty.
- *setup_only* - Applies only the setup uncertainty.
- *hold_only* - Applies only the hold uncertainty.
- *none* - Applies neither the setup nor hold uncertainty.

Use the *set_clock_uncertainty* command to specify the uncertainty for a specified clock. If you use the command with neither the *-setup* nor *-hold* option, then the clock uncertainty is applied to both setup and hold checks.

Feedback

See Also

- report_constraint
- report_min_period
- report_min_pulse_width
- set_clock_uncertainty

---

timing_input_port_default_clock

Specifies whether a default clock is assumed at input ports for which you have not defined a clock with the *set_input_delay* command.

Data Types

Boolean

Default false

Description

This variable affects the behavior of PrimeTime when you set an input delay without a clock on an input port.

If you set this variable to *true*, the input delay on the port is set with respect to one imaginary clock so that the inputs are constrained. This also causes the clocks along the paths driven by these input ports to become related. Also, the period of this clock is equal to the base period of all these related clocks. PrimeTime SI excludes victim and aggressor arrival windows associated with this imaginary clock for crosstalk analysis.

If this variable is set to *false*, no such imaginary clock is assumed.

See Also

- set_input_delay

---

timing_is_clock_pin_attribute_on_clock_source_compatibility

When set to *true*, the value of pin attribute "is_clock_pin" will be *true* for clock source pins/ports, otherwise, the value of the attribute will be *false*

Data Types

Boolean

Default false

Feedback

Description

By definition, for a design instance pin object, the *"is_clock_pin"* attribute is *true* if the pin is a valid and active clock pin in the design; that is, it is reached by a clock signal, and the sequential cell instance containing this pin is not disabled by disabled timing arcs or by case analysis. When clock source is defined on cell instance pin that is not clock pin, such as the Q output, PrimeTime no longer marks the pin with *"is_clock_pin"* attribute, and is now consistent with IC Compiler II.

Please note that the variable only affects the value of the attribute *"is_clock_pin"*, and therefore, can be set after update_timing.

See Also

- get_attribute
- report_attribute

---

timing_keep_loop_breaking_disabled_arcs

Specifies whether to keep .db inherited disabled timing arcs for static loop breaking.

Data Types

Boolean

Default false

Description

When *true*, enables inheriting of .db disabled timing arcs for loop breaking. When *false* (the default), do not accept .db disabled timing arcs for loop breaking.

If the .db inherited disabled timing arcs do not break all of the loops, the default static loop breaking technique breaks the loops unless the dynamic loop breaking technique is enabled.

The .db inherited disabled timing arcs can be removed individually without affecting the other .db inherited disabled timing arcs.

There is a difference between Design Compiler and PrimeTime where additional *set_case_analysis* or *set_disable_timing* commands do not remove .db inherited disabled timing arcs.

For this variable to take effect, you must set it before link is performed. If you set this variable after linking, it has no effect.

To remove .db inherited arcs after they are accepted, use the *remove_disable_timing* command because they are user-defined.

Feedback

The *is_db_inherited_disabled* timing_arc attribute indicates whether an arc is a .db inherited disabled arc.

To remove all .db inherited disable timing arcs for loop breaking, use this command:

```
pt_shell> remove_disable_timing [get_timing_arcs -of [get_cell *] \\
          -filter "is_db_inherited_disabled == true"]
```

See Also

- remove_disable_timing
- report_disable_timing

---

timing_keep_waveform_on_points

Keeps the waveform data from advanced waveform propagation on timing points when timing paths are created by the *get_timing_paths* command.

Data Types

Boolean

Default false

Description

By default, the tool does not keep the waveform data from advanced waveform propagation on timing points because most timing analysis flows do not use this data. This default behavior reduces the peak memory usage.

To override the default behavior and keep the waveform data on timing points during the *get_timing_paths* command, set the *timing_keep_waveform_on_points* variable to *true*. Use this variable setting when

- You use the *write_spice_deck* command. This keeps the waveform data on timing points so that the corresponding waveform can be written to the SPICE deck.
- You want to see the exact waveform in the PrimeTime GUI.

Note: This variable does not affect the advanced waveform propagation mode. This variable is only for timing paths created by the *get_timing_paths* command after a full timing update. After you change this variable setting, a full timing update is not required.

See Also

- get_timing_paths
- write_spice_deck

Feedback

timing_lib_cell_derived_clock_name_compatibility

Specifies how generated clock names are derived from library files.

Data Types

Boolean

Default true

Description

This variable specifies how generated clock names are derived from library files; you must set this variable before linking the design:

- *true* - Derives the name of the generated clock from the name of its source or the first source name in case of multisource generated clocks.
- *false* - Takes the generated clock name directly taken from the explicit specification in the library.

See Also

- link_design
- report_clock

timing_library_max_cap_from_lookup_table

Specifies whether the frequency-indexed lookup table values for max_capacitance, if they exist, override other library-derived max_capacitance constraints.

Data Types

Boolean

Default false

Description

If this variable is set to *false* (the default), the *report_constraint* command uses the most restrictive max_capacitance constraint for the specified pin. Specifically, the smallest max_capacitance value is taken among:

- The lookup table-derived value
- The library-cell max_capacitance value
- All user-specified max_capacitance values (at the pin, port, clock, or design level) at the pin

Feedback

If you set this variable to *true*, and a max_capacitance lookup table is defined on the library pin, and if a valid data signal reaches the pin (that is, if there is at least one clock launching a signal that arrives at the pin), all other library-cell max_capacitance values are ignored at the pin. This variable gives precedence to the frequency-based max_capacitance constraint even when this constraint is more lenient than other library-derived constraints. The tool applies the user-specified max_capacitance constraints as normal.

See Also

- report_constraint
- set_max_capacitance
- timing_enable_library_max_cap_lookup_table
- timing_enable_max_cap_precedence

timing_max_capacitance_derate

Specifies a scaling factor for design rule constraint max_capacitance violation, ranging from 1.0 to 10.0.

Data Types

float

Default 1.0

Description

This variable relaxes the max_capacitance globally in the entire design. The max_capacitance allowed, which is the worst of library limit and user-specified values, is multiplied by this factor.

If this variable is set to 1.0 (the default), the max_capacitance violation threshold does not change.

Set *rc_ccs_extrapolation_range_compatibility* to *false* before you set this variable to a value greater than 1.0.

Setting this variable checks out a PrimeTime ADV license.

Feedback

See Also

- report_constraint
- rc_ccs_extrapolation_range_compatibility
- timing_max_transition_derate

timing_max_capacitance_limit_from_library_only

Specifies whether the max_capacitance limit is to be taken from the library, overriding limits from other sources.

Data Types

Boolean

Default false

Description

If this variable is set to *false* (the default), the *report_constraint* command and the drc_constraining_max_capacitance attribute use the most restrictive max_capacitance constraint for the specified pin. If this variable is set to *true* the library derived value will be used, overriding limits from other sources (user-set on design or pin).

If this variable and the variable *timing_enable_max_cap_precedence* are both set to *true*, this variable will take priority, and the library-derived limit will be applied.

If this variable is set to *true*, and a pin has a max_capacitance limit set with *set_max_capacitance -force*, the latter will take priority and the user-set limit will be applied.

See Also

- report_constraint
- set_max_capacitance
- timing_enable_max_cap_precedence

timing_max_normalization_cycles

Sets the upper limit for the denominator when calculating normalized slack.

Data Types

integer

Feedback

Default 4

Description

Normalized slack analysis is an additional optional analysis that computes a normalized slack for each timing path. The normalized slack is the slack divided by the idealized allowed propagation delay. For paths in a single clock domain, the allowed propagation delay is the time between two different edges of the clock. For 50% duty cycle clocks, the allowed propagation delay is usually an integer number of half periods.

The denominator in the normalization is limited by the value of the *timing_max_normalization_cycles* variable setting multiplied by the period of the launch clock of a path. Beyond this limit, the denominator takes the value of the limit.

Limiting the denominator saves runtime during analysis. A larger value of the limit might increase runtime.

See Also

- report_timing
- timing_enable_normalized_slack

---

timing_max_transition_derate

Specifies a scaling factor for the design rule constraint max_transition violation, ranging from 1.0 to 1.5.

Data Types

float

Default 1.0

Description

This variable relaxes the max_transition globally in the entire design. The max_transition allowed, which is the worst of library limit and user-specified values on the pin, is multiplied by this factor.

If this variable is set to 1.0 (the default), the max_transition violation threshold does not change.

Set *rc_ccs_extrapolation_range_compatibility* to *false* before you set this variable to a value greater than 1.0.

Setting this variable checks out a PrimeTime ADV license.

Feedback

See Also

- report_constraint
- rc_ccs_extrapolation_range_compatibility
- timing_max_capacitance_derate

---

timing_max_transition_limit_from_library_only

Specifies whether library-defined max_transition requirements take precedence over user-defined requirements.

Data Types

Boolean

Default false

Description

This variable controls whether library-defined *max_transition* requirements take precedence over user-defined requirements.

If this variable is set to *false* (the default), then maximum transition checking applies the most restrictive *max_transition* requirement from all sources (library-defined and user-defined).

If this variable is set to *true*, then

- Library-defined *max_transition* requirements take precedence over user-defined *set_max_transition* specifications, regardless of value.
- The *timing_enable_max_slew_precedence* variable is ignored.

Note that even when this variable is set to *true*, user-defined requirements specified with the *-force* option of *set_max_transition* take precedence over library requirements.

See Also

- report_constraint
- set_max_transition
- timing_enable_max_slew_precedence

Feedback

timing_min_pulse_width_high_pulse_threshold

Specifies the threshold voltage for measuring high pulse widths during minimum pulse width checking with advanced waveform propagation.

Data Types

float

Default 0.5

Description

This variable sets the threshold voltage for minimum pulse width checking for high pulses (low-high-low) in the clock network by the *report_min_pulse_width* command. The width of the high pulse is measured at the specified fraction of the rail voltage. For a more conservative check, set the threshold to a value greater than the default (0.5) but below 1.0.

This feature works only when waveform propagation is enabled by setting the *delay_calc_waveform_analysis_mode* variable to *full_design* and waveform adjustment is enabled by setting the *timing_enable_min_pulse_width_waveform_adjustment* variable to *true*.

See Also

- report_min_pulse_width
- set_min_pulse_width
- delay_calc_waveform_analysis_mode
- timing_enable_min_pulse_width_waveform_adjustment
- timing_min_pulse_width_low_pulse_threshold

timing_min_pulse_width_low_pulse_threshold

Specifies the threshold voltage for measuring low pulse widths during minimum pulse width checking with advanced waveform propagation.

Data Types

float

Default 0.5

Feedback

Description

This variable sets the threshold voltage for minimum pulse width checking for low pulses (high-low-high) in the clock network by the *report_min_pulse_width* command. The width of the low pulse is measured at the specified fraction of the rail voltage. For a more conservative check, set the threshold to a value less than the default (0.5) but above 0.0.

This feature works only when waveform propagation is enabled by setting the *delay_calc_waveform_analysis_mode* variable to *full_design* and waveform adjustment is enabled by setting the *timing_enable_min_pulse_width_waveform_adjustment* variable to *true*.

See Also

- report_min_pulse_width
- set_min_pulse_width
- delay_calc_waveform_analysis_mode
- timing_enable_min_pulse_width_waveform_adjustment
- timing_min_pulse_width_high_pulse_threshold

---

timing_ocvm_enable_distance_analysis

Enables distance-based analysis during advanced on-chip variation (AOCV) or parametric on-chip variation (POCV) analysis.

Data Types

Boolean

Default true

Description

To control whether distance-based analysis is performed during AOCV or POCV analysis, set this variable to one of these values:

- *true* (the default) - Performs distance-based analysis if you loaded location information with the *read_parasitics* command. This information is then used to index the systematic component of variation in the derating tables.
- *false* - Does not perform distance-based analysis even if all prerequisite information is read into the current PrimeTime session.

Feedback

See Also

- read_parasitics
- read_parasitics_load_locations
- timing_aocvm_enable_analysis
- timing_pocvm_enable_analysis

---

timing_ocvm_precedence_compatibility

Controls the fallback to on-chip variation deratings when advanced on-chip variation (AOCV) or parametric on-chip variation (POCV) is enabled.

Data Types

Boolean

Default false

Description

Set this variable to one of these values:

- *false* (the default) - Considers both OCV and AOCV or POCV deratings for a given object. The tool chooses the derating for graph- and path-based analysis by the following order of precedence (from highest to lowest priority):
- OCV leaf cell derating
- AOCV or POCV library cell derating
- OCV library cell derating
- AOCV or POCV hierarchical cell derating
- OCV hierarchical cell derating
- AOCV or POCV design derating
- OCV design derating
- *true* - Ignores the OCV deratings for AOCV and POCV analysis.

This variable controls the deratings for both cell delays and net delays. Path-based OCV deratings set using *pba_derate_list* is not supported for AOCV or POCV analysis. The AOCV or POCV guard band derating is used only if the AOCV or POCV derating factor is used for an object.

Feedback

This variable is effective only when the *timing_aocvm_enable_analysis* or *timing_pocvm_enable_analysis* variable is set to *true*.

See Also

- set_timing_derate
- timing_aocvm_enable_analysis
- timing_pocvm_enable_analysis

timing_parallel_constraint_arcs_compatibility

Controls how parallel constraint arc values are processed and stored.

Data Types

Boolean

Default false

Description

This variable controls how parallel constraint arcs are processed and stored during analysis. Valid values are:

- *false* (default) - Parallel constraint arc values can be merged (depending on their value) for more efficient analysis. When values are merged, the results are always bounding.
- *true* - All parallel constraint arc values are kept; no merging is done. This value matches the default behavior of older PrimeTime releases.

This variable must be set before the *link_design* command is run.

In *write_sdf* flows, you might need to set this variable to *true* to get the expected SDF output.

See Also

- link_design

timing_path_arrival_required_attribute_include_clock_edge

Includes or excludes the clock edge time (the *endpoint_clock_close_edge_value* and *startpoint_clock_close_edge_value* attributes) in the *arrival* and *required* timing path attributes.

Feedback

Data Types

Boolean

Default false

Description

Set this variable to one of the following values:

- *false* (the default) - Excludes the clock edge time of the launching or capturing clocks of the path from the calculation of the *arrival* or *required* timing path attribute, respectively.
- *true* - Includes the clock edge time of the launching or capturing clocks of the path in the calculation of the *arrival* or *required* timing path attribute, respectively. When you use this setting, the *arrival* and *required* attribute values match the arrival and required times reported by the *report_timing* command.

See Also

- report_timing
- timing_path_attributes

---

timing_path_signature_type

Specifies the type of timing path signature reported by the *signature* attribute. The *signature* attribute is available from path collections created with get_timing_paths or from path collections which have been restored in Interactive Multi-Scenario Analysis Mode (IMSA).

Data Types

String

Default "default"

Description

default

Type: string

The default signature type which incorporates all available information to provide a unique path including data and clock path information as well as.

data

Type: string

The data signature type only incorporates static data path information.

Feedback

data_and_clock

Type: string

The data and clock signature type incorporates static data and clock path information.

See Also

- enable_path_tagging
- get_timing_paths
- interactive_multi_scenario_analysis_enabled
- save_session
- restore_session

timing_pocvm_corner_sigma

Selects the standard deviation to be used for parametric on-chip variation analysis in PrimeTime when calculating corner values from statistical quantities.

Data Types

float

Default 3.0

Description

Parametric on-chip variation analysis internally computes arrival, required and slack values based on statistical distributions, When performing comparisons between these statistical quantities, PrimeTime needs to know at what corner these statistical values are evaluated for reporting to guarantee a pessimistic analysis.

The *timing_pocvm_corner_sigma* sets this corner to be used during *update_timing*.

See Also

- timing_pocvm_corner_sigma
- timing_pocvm_enable_analysis

timing_pocvm_enable_analysis

Enables graph-based parametric on-chip variation (POCV) analysis.

Feedback

Data Types

Boolean

Default false

Description

Setting this variable to *true* enables parametric on-chip variation (POCV) timing analysis mode; that is, all operations in PrimeTime are performed in POCV mode. Unlike in advanced on-chip variation (AOCV) mode, there is no mode where POCV can be used in PBA only.

Note: Setting this variable to *true* automatically switches the design into *on_chip_variation* analysis mode using the *set_operating_conditions* command.

See Also

- get_timing_paths
- read_aocvm
- report_aocvm
- report_timing
- set_operating_conditions

---

timing_pocvm_enable_delay_slew_correlation

Enables enhanced delay/slew correlation in moment-based parametric on-chip variation (POCV) analysis.

Data Types

Boolean

Default false

Description

In a basic POCV analysis, each cell instance is considered to be statistically independent. However, when slew variation is enabled, the output slew from one cell induces a correlated delay component in subsequent cells.

When this variable is set to *false* (the default), the tool performs a basic analysis of this effect.

When this variable is set to *true*, the tool performs a more advanced analysis of this effect that can improve accuracy in a moment-based (non-Gaussian) analysis.

Feedback

This variable has an effect only when both of the following conditions are true:

- The *timing_pocvm_enable_extended_moments* variable is set to *true* to enable moment-based POCV analysis.
- The *timing_enable_slew_variation* variable is set to *true* to consider slew variation.

See Also

- timing_enable_slew_variation
- timing_pocvm_enable_analysis
- timing_pocvm_enable_extended_moments

timing_pocvm_enable_extended_moments

Enables extended statistical moments LVF support for parametric on-chip variation (POCV) analysis.

Data Types

Boolean

Default false

Description

Setting this variable to *true* enables extended statistical moments to be computed and propagated for timing quantities during parametric on-chip variation (POCV) timing analysis mode. That is, all operations in PrimeTime are performed in POCV mode using extended moments as described in Liberty LVF extension. The extended moments are mean_shift, std_dev and skewness as defined in Liberty LVF extension in order to model non Gaussian distribution.

Note: Setting this variable to *true* has any effect only if *timing_pocvm_enable_analysis* is also set to *true* (i.e. PrimeTime POCV analysis has been enabled as well).

See Also

- timing_pocvm_enable_analysis

timing_pocvm_enable_extrapolation_warning

Enables the printing of the warning message UITE-581 to indicate POCV sigma lookup has encountered extrapolation greater than 10 percent.

Feedback

Data Types

Boolean

Default false

Description

Setting this variable to *true* enables the printing of a warning message if POCV sigma lookup encounters extrapolation greater than 10 percent.

See Also

- timing_pocvm_enable_analysis

---

timing_pocvm_max_transition_clock_sigma

Specifies the POCV standard deviation used for reporting maximum transition violations by the *report_constraint -max_transition* command for pins in the clock network.

Data Types

float or "disabled"

Default disabled

Description

When this variable is set to its default value of "disabled" the *report_constraint -max_transition* command reports transition times with the sigma value of *timing_pocvm_max_transition_sigma* variable for all pins.

If it is set to a float value, the sigma applied to clock network pins is *timing_pocvm_max_transition_clock_sigma* (it is assumed that *timing_pocvm_max_transition_clock_sigma* is larger than *timing_pocvm_max_transition_sigma*. Since all clock network pins also carry data signals, *timing_pocvm_max_transition_sigma* is applied to all pins if it is larger than *timing_pocvm_max_transition_clock_sigma*.

Note that the value of this variable also impacts the *drc_actual_max_transition* attribute of pin and port objects. As with *timing_pocvm_max_transition_sigma*, you can change this variable without triggering a timing update because it only affects maximum transition time reporting, not setup/hold timing analysis.

See Also

- timing_pocvm_max_transition_sigma
- timing_pocvm_corner_sigma

Feedback

- timing_pocvm_report_sigma
- timing_enable_slew_variation
- timing_pocvm_enable_analysis

timing_pocvm_max_transition_sequential_sigma

Specifies the POCV standard deviation used for reporting maximum transition violations by the *report_constraint -max_transition* command for sequential clock and data pins.

Data Types

float or "disabled"

Default disabled

Description

When this variable is set to its default value of "disabled" the *report_constraint -max_transition* command reports transition times with the sigma value of *timing_pocvm_max_transition_sigma* variable for all pins.

If it is set to a float value, the sigma applied to sequential clock and data pins is *timing_pocvm_max_transition_sequential_sigma* (it is assumed that *timing_pocvm_max_transition_sequential_sigma* is larger than *timing_pocvm_max_transition_sigma*. *timing_pocvm_max_transition_sigma* is applied to sequential clock and data pins if it is larger than *timing_pocvm_max_transition_sequential_sigma*. For sequential pins in the clock network, the largest of *timing_pocvm_max_transition_sigma*, *timing_pocvm_max_transition_clock_sigma* and *timing_pocvm_max_transition_sequential_sigma* is applied.

Note that the value of this variable also impacts the *drc_actual_max_transition* attribute of pin and port objects. As with *timing_pocvm_max_transition_sigma*, you can change this variable without triggering a timing update because it only affects maximum transition time reporting, not setup/hold timing analysis.

See Also

- timing_pocvm_max_transition_sigma
- timing_pocvm_max_transition_clock_sigma
- timing_pocvm_corner_sigma
- timing_pocvm_report_sigma

Feedback

- timing_enable_slew_variation
- timing_pocvm_enable_analysis

timing_pocvm_max_transition_sigma

Specifies the POCV standard deviation used for reporting maximum transition violations by the *report_constraint -max_transition* command.

Data Types

float

Default 0.0

Description

Parametric on-chip variation (POCV) analysis internally computes timing quantities (such as arrival, required, and transition times) based on statistical distributions. Transition time variation analysis is enabled when the *timing_pocvm_enable_analysis* and *timing_enable_slew_variation* variables are both set to *true*.

When performing comparisons between statistical transition values, by default the tool considers the values at 3.0 standard deviations away from the mean as the worst case. You can modify this parameter by setting the *timing_pocvm_corner_sigma* variable (default 3.0).

By default, the *report_constraint -max_transition* command reports nominal transition times (zero sigma). To report sigma corner values instead, set the *timing_pocvm_max_transition_sigma* variable to desired sigma value, such as 3.0. Note that the value of this variable also impacts the *drc_actual_max_transition* attribute of pin and port objects. You can change this variable without triggering a timing update because it only affects maximum transition time reporting, not setup/hold timing analysis.

See Also

- timing_pocvm_corner_sigma
- timing_pocvm_report_sigma
- timing_enable_slew_variation
- timing_pocvm_enable_analysis

timing_pocvm_precedence

Controls the precedence of POCV single-parameter variation applied from side files and LVF libraries.

Feedback

Data Types

string

Default file

Description

In a POCV analysis, single-parameter variation data for timing distribution behaviors can be provided as a side file or as Liberty Variation Format (LVF) constructs in a library.

This variable controls the precedence used between side files and LVF libraries:

- *file* (the default) - side files take precedence over LVF libraries.
- *library* - LVF libraries take precedence over side files.
- *lib_cell_in_file* - the tool uses the following priority, in decreasing order of precedence:
- Library cell table
- Hierarchical cell table
- Design table

This variable is effective only when the *timing_pocvm_enable_analysis* variable is set to *true*.

See Also

- read_ocvm
- timing_pocvm_enable_analysis

---

timing_pocvm_report_sigma

Specifies the standard deviation to be used for parametric on-chip variation analysis in PrimeTime when reporting corner values from statistical quantities.

Data Types

float

Default 3.0

Description

Parametric on-chip variation analysis internally computes arrival, required and slack values based on statistical distributions.

Feedback

When performing comparisons between these statistical quantities, PrimeTime needs to know at what corner these statistical values are evaluated for reporting to guarantee a pessimistic analysis this corner is set by the *timing_pocvm_corner_sigma* variable.

During reporting, a less pessimistic corner can be evaluated without performing a full timing update by selecting a corner value for reporting only. This reporting only corner value can be set using the *timing_pocvm_report_sigma* variable. The result is guaranteed to be bounding the result one would get by setting the *timing_pocvm_corner_sigma* to the new specified corner followed by a full update_timing, but can be more pessimistic.

The value specified for *timing_pocvm_report_sigma* must be smaller than the value of the *timing_pocvm_corner_sigma* variable, or else it is ignored.

See Also

- timing_pocvm_corner_sigma
- timing_pocvm_enable_analysis

---

timing_point_arrival_attribute_compatibility

Includes or excludes the arrival to startpoint in the *arrival* attribute for timing points of a timing path.

Data Types

Boolean

Default true

Description

This variable controls whether the *arrival* attribute of *timing_point* objects is relative to the current path segment only, or is cumulative across all segments of the path.

When this variable is *true* (the default), arrival values start at zero in each path segment along the path.

When this variable is *false*, arrival values are cumulative across path segments and include the following:

- Input external delay (*startpoint_input_delay_value*)
- Source/propagagated clock latency (*startpoint_clock_latency*)
- Time given to startpoint (*time_lent_to_startpoint*)
- The delay of previous path segments

Note that the cumulative arrival values do not include the clock edge time.

Feedback

When this variable is *false*, *arrival* attribute values match the arrival values reported by the *report_timing* command (except for the clock edge time, if it is nonzero).

See Also

- report_timing
- timing_path_attributes
- timing_point_attributes

timing_prelayout_scaling

Enables scaling of delay and transition times in pre-layout flow to approximate effects of mismatching driver and load signal levels.

Data Types

Boolean

Default true

Description

Enables scaling of delay and transition times in pre-layout flow to approximate effects of mismatching driver and load signal levels. When this variable is set to *true* (the default), then in pre-layout flow (without detailed parasitics) delay and transition times along net arcs are scaled to describe the same physical waveform using local trip points and voltage level on the load cell.

No scaling is done for a post-layout flow because PrimeTime measures delays on analog waveforms.

See Also

- report_delay_calculation

timing_propagate_interclock_uncertainty

Enables or disables the propagation of interclock uncertainty through transparent latches in PrimeTime.

Data Types

Boolean

Default false

Feedback

Description

When *false* (the default), the interclock uncertainty is calculated for each latch-to-latch path independently, from the clock at the launch latch to the clock at the capture latch, even when latches operate in transparent mode.

When *true*, clock uncertainty information is propagated through each latch operating in transparent mode, as though it were a combinational element. This allows an entire sequence of latch-to-latch stages to be considered a single path for interclock uncertainty calculation, provided that time borrowing occurs at the endpoint of each intermediate stage.

Operating with this variable set to true can lead to more accurate results for designs containing transparent latches, at the cost of some CPU time and memory resources.

For example, consider a pipeline containing latches A, B, and C, clocked by clocks 1, 2, and 3, respectively. The tool treats the paths between A and B and between B and C as distinct. In reality, however, if latch B is in transparent mode, data passes through it as though it were a combinational element. Regardless of whether interclock uncertainty has been applied between clocks 1 and 3, the default behavior is to apply the uncertainty between clocks 2 and 3 when calculating slack at latch C. It is more accurate, however, to apply the uncertainty between the clock at the path startpoint (clock 1, latch A) and the clock at the path endpoint (clock 3, latch C), if defined.

With *timing_propagate_interclock_uncertainty* set to *true*, the correct interclock uncertainty is applied, as though the path from latch A to latch C through the transparent latch B were a single, extended path. That is, the uncertainty is propagated through the transparent latch. To find out the startpoint of this extended path, use *report_timing -trace_latch_borrow*.

See Also

- report_timing
- set_clock_uncertainty

---

timing_propagate_through_non_latch_d_pin_arcs

Propagates cell arcs from data pins for edge-triggered devices.

Data Types

Boolean

Default false

Feedback

Description

By default, under certain conditions, PrimeTime does not allow propagation through the cell arcs from data pins of edge-triggered devices.

To allow propagation through the cell arcs from data pins for edge-triggered devices, set this variable to *true*. Changing the setting of this variable triggers a full update timing subsequently.

See Also

- update_timing

---

timing_reduce_multi_drive_net_arcs

Enables or disables the collapsing of parallel timing arcs to improve PrimeTime performance and memory utilization.

Data Types

Boolean

Default false

Description

Designs with high-fanin, high-fanout mesh clock networks can cause significant performance degradation and explosion in memory requirements. Evidently, detailed parasitics would further exaggerate these problems. The suggested flow is to Spice the clock network and annotate clock mesh cell and net delays and transition times at the driver and load pins.

To improve performance and memory requirements for clock network analysis, PrimeTime has to reduce the number of timing arcs it must consider. To do this, set the *timing_reduce_multi_drive_net_arcs* variable to *true*.

The reduction operation is performed during design linking; therefore, the variable has to be set prior to that. After the design is linked, modifying the value of the *timing_reduce_multi_drive_net_arcs* variable does not cause parallel timing arcs to be collapsed or restored.

Potential instances of parallel drivers are detected at design nets based on the product of the size of a net's global drivers and loads. If this product were greater than the value of

Feedback

the *timing_reduce_multi_drive_net_arcs_threshold* variable, the net would be considered for reduction. In order to reduce the fanin of such nets, the following criteria must be true:

1. All drivers should have at most one output pin driving the mesh.
2. All driver cells must be the same type (lib_cell).
3. All driver cells relevant inputs must correspondingly connect to the same nets.

For every successfully reduced net, a *PTE-046* message is issued, specifying the reduced net and the corresponding driver after the reduction. For unsuccessful attempts, a *PTE-047* message is issued to explain the reason the net drivers cannot be reduced.

The *PTE-046* message would specify the "selected" driver. Alternatively, obtain the selected driver by computing the back arcs to any load pin of the mesh using get_timing_arcs with the -to option.

You need to annotate accurate delays to selected mesh driver pin back arcs as well as delays from the selected driver to all the mesh load pins using *set_annotated_delay*, in addition to transition times at the selected drivers and all the load pins using *set_annotated_transition*. The *check_timing* command verifies that the reduction affected multidriven nets in the clock network and that the necessary annotated delays and transitions do exist as indicated above.

Note that the reduced cells are not physically removed from the netlist, but that no timing arcs exist from their output pins. Therefore, flows using the *write_changes* command are not be affected. However, not having the timing arcs out of the collapsed cells implies that the *report_timing* command through these cells or the setting of point-to-point exceptions are completely ignored.

SDF annotations to or from collapsed cells issue the *PTE-048* informational message noting that a particular delay annotation is ignored. Note that the remaining cell post-collapse is arbitrarily selected; therefore, no assertion can be made as to its annotated delay. The same applies to Reduced Standard Parasitic Format (RSPF) annotations. Flows using the *write_sdf* command must account for the reduced timing arcs.

See Also

- check_timing
- report_delay_calculation
- report_timing
- set_annotated_delay
- set_annotated_transition
- write_changes

Feedback

- write_sdf
- timing_reduce_multi_drive_net_arcs_threshold

timing_reduce_multi_drive_net_arcs_threshold

Provides a threshold for the product of some net's fanin and fanout beyond which a parallel timing arc in the net's fanin might be reduced.

Data Types

integer

Default 10000

Description

For a net, the number of timing arcs through the net is equal to the product of the net's drivers and loads. For designs with high-fanin, high-fanout mesh clock networks, significant performance degradation and explosion in memory requirements can occur.

Setting the *timing_reduce_multi_drive_net_arcs* variable improves parallel drivers reduction.

The *timing_reduce_multi_drive_net_arcs_threshold* variable provides a minimum value for the driver-load product, below which the net is not considered for reduction.

See Also

- timing_reduce_multi_drive_net_arcs

timing_refinement_max_slack_threshold

Defines the maximum delay (setup) slack threshold that determines which pins are included in graph-based refinement analysis.

Data Types

float (range: -large number to +large number, or string "disabled")

Default 0.0

Description

HyperTrace is a technology that accelerates the exhaustive PBA search by computing refined graph-based timing data, then using it to drive path searching and recalculation. This feature is enabled by setting the *timing_enable_graph_based_refinement* variable to *true*.

Feedback

When enabled, HyperTrace graph-based refinement considers pins to be critical for maximum delay (setup) timing when they belong to a timing path with setup slack less than the threshold specified by the *timing_refinement_max_slack_threshold* variable. The tool performs refinement on pins with slack worse than this threshold.

You can set this variable to any value (positive, negative, or zero) or to the string "*disabled*" to entirely disable graph-based refinement for maximum delay reporting. The *timing_refinement_max_slack_threshold* variable is set to 0.0 by default, which is appropriate for the default *-slack_lesser_than* setting of 0.0 for the reporting command.

If you set this variable to a number, the optimum value is equal to the *-slack_lesser_than* setting of the reporting command for maximum delay reporting. If you are running multiple exhaustive path-based analyses using different *-slack_lesser_than* settings for maximum delay reporting, set the refinement threshold equal to the largest (most positive) of those settings.

The refinement analysis is intended for usage when the design timing is close to signoff and the number of pins considered for refinement with the specified threshold is reasonably limited. Otherwise, the runtime and memory required for refinement could exceed the benefit to exhaustive path-based analysis. For more information, see the man page for the *timing_enable_graph_based_refinement* variable.

See Also

- timing_enable_graph_based_refinement
- timing_refinement_min_slack_threshold

---

timing_refinement_maximum_critical_pin_percentage

Defines the maximum crticial region size for which graph-based refinement will be performed.

Data Types

float

Default 100.0

Description

This variable specifies in percentage format the maximum size of the critical region for which graph-based refinement will be performed.

During a non-incremental graph-based refinement analysis the size of the critical region (as defined by *timing_refinement_max_slack_threshold* and *timing_refinement_min_slack_threshold*) will be determined and if it exceeds the value

Feedback

of this variable then graph-based refinement analysis will not be performed. Subsequent commands will behave as if graph-based refinement were disabled.

By default, graph-based refinement applies to the critical region regardless of its size. However, early in the design flow, graph-based refinement may be prohibitively expensive in terms of runtime and memory overhead. This variable allows the graph-based refinement to be blocked until the design is in a state where the percentage of critical pins is below a given threshold (e.g. 10 percent) to permit the refinement to be conducted only where it is beneficial for a given design.

See Also

- timing_enable_graph_based_refinement
- timing_refinement_max_slack_threshold
- timing_refinement_min_slack_threshold

---

timing_refinement_min_slack_threshold

Defines the minimum delay (hold) slack threshold that determines which pins are included in graph-based refinement analysis.

Data Types

string (range: -large number to +large number, or string "disabled")

Default disabled

Description

HyperTrace is a technology that accelerates the exhaustive PBA search by computing refined graph-based timing data, then using it to drive path searching and recalculation. This feature is enabled by setting the *timing_enable_graph_based_refinement* variable to *true*.

When enabled, HyperTrace graph-based refinement considers pins to be critical for minimum delay (hold) timing when they belong to a timing path with hold slack less than the threshold specified by the *timing_refinement_max_slack_threshold* variable. The tool performs refinement on pins with slack worse than this threshold.

You can set this variable to any value (positive, negative, or zero) or to the string *disabled* to entirely disable graph-based refinement for minimum delay reporting. The *timing_refinement_min_slack_threshold* variable is set to "*disabled*" by default because minimum-delay exhaustive analysis is typically very fast and does not gain much benefit from graph-based refinement.

Feedback

If you set this variable to a number, the optimum value is equal to the *-slack_lesser_than* setting of the reporting command for minimum delay reporting. If you are running multiple exhaustive path-based analyses using different *-slack_lesser_than* settings for minimum delay reporting, set the refinement threshold equal to the largest (most positive) of those settings.

The refinement analysis is intended for usage when the design timing is close to signoff and the number of pins considered for refinement with the specified threshold is reasonably limited. Otherwise, the runtime and memory required for refinement could exceed the benefit to exhaustive path-based analysis. For more information, see the man page for the *timing_enable_graph_based_refinement* variable.

See Also

- timing_enable_graph_based_refinement
- timing_refinement_max_slack_threshold

---

timing_remove_clock_reconvergence_pessimism

Enables clock reconvergence pessimism removal (CRPR).

Data Types

Boolean

Default true

Description

When this variable is *true* (the default), the tool removes clock reconvergence pessimism from slack calculation and minimum pulse width checks.

Clock reconvergence pessimism (CRP) is a difference in delay along the common part of the launching and capturing clock paths. The most common causes of CRP are reconvergent paths in the clock network, and different min and max delay of cells in the clock network.

CRP is independently calculated for rise and fall clock paths. You can use the *timing_clock_reconvergence_pessimism* variable to control CRP calculation with respect to transition sense. In the case of the capturing device being a level-sensitive latch two CRP values are calculated:

- crp_open, which is the CRP corresponding to the opening edge of the latch
- crp_close, which is the CRP corresponding to the closing edge of the latch

Feedback

The required time at the latch is increased by the value of crp_open and therefore reduce the amount of borrowing (if any) at the latch. Meanwhile, the maximum time borrow allowed at the latch is affected by shifting the closing edge by crp_close.

CRP is calculated differently for minimum pulse-width checks. It is given as the minimum of (maximum rise arrival time - minimum rise arrival time) and (maximum fall arrival time - minimum fall arrival time) at the pin where the check is being made.

If the *si_enable analysis* variable is set to *true*, delays in the clock network could also include delta delays resulting from crosstalk interaction. Such delays are dynamic in nature, that is, they can vary from one clock cycle to the next, causing different delay variations (either speed-up or slow-down) on the same network, but during different clock cycles.

PrimeTime SI considers delta delays as part of the CRP calculation only if the type of timing check deployed derives its data from the same clock cycle.

Similarly, if dynamic annotations have been set on the design, the clock delays computed using these annotations are only used to calculate CRP if type of timing check deployed derives its data from the same clock cycle. Such dynamic annotations include dynamic clock latency, which can be specified with the *set_clock_latency* command, or dynamic rail voltage, which can be specified with the *set_rail_voltage* command.

In transparent-latch based designs, you should set the *timing_early_launch_at_borrowing_latches* variable to *false* when clock reconvergence pessimism removal (CRPR) is enabled. In this case, CRPR applies even to paths whose startpoints are borrowing, leading to better pessimism reduction overall.

Any effective change in the setting of the *timing_remove_clock_reconvergence_pessimism* variable causes full *update_timing*. You cannot perform one *report_timing* operation that considers CRP and one that does not without full *update_timing* in between.

Limitations: CRPR does not support paths that fan out directly from clock source pins to the data pins of sequential devices. To enable support for such paths, set the *timing_crpr_remove_clock_to_data_crp* variable to *true*.

To turn CRPR on:

```
pt_shell> set_app_var timing_remove_clock_reconvergence_pessimism true
true
pt_shell> report_timing
```

See Also

- get_timing_paths
- report_analysis_coverage

Feedback

- report_bottleneck
- report_constraint
- report_crpr
- report_min_pulse_width
- report_timing
- set_clock_latency
- set_rail_voltage
- si_enable_analysis
- timing_clock_reconvergence_pessimism
- timing_crpr_remove_clock_to_data_crp
- timing_crpr_threshold_ps
- timing_early_launch_at_borrowing_latches

timing_report_always_use_valid_start_end_points

Requires the *-from*, *-rise_from*, and *-fall_from* options to specify valid timing startpoints and the *-to*, *-rise_to*, and *-fall_to* options to specify valid timing endpoints.

Data Types

Boolean

Default false

Description

This variable affects the way the *report_timing*, *report_bottleneck*, and *get_timing_path* commands interpret the *-from*, *-rise_from*, *-fall_from*, *-to*, *-rise_to*, and *-fall_to* options.

When set to *false* (the default), the *from_list* objects are interpreted as all pins found by given objects. Objects specified with an asterisk (*) wildcard or cell name might return invalid startpoints or endpoints. For example, *-from [get_pins FF/*]* includes input, output, and asynchronous pins. The tool considers these invalid startpoints or endpoints to be throughpoints and continues the path searching. Although it is convenient to use, it is not suggested because of longer runtimes.

To report only valid startpoints and endpoints, set this variable to *true*. It is always suggested to use input ports or register clock pins for the *from_list* objects and output ports or register data pins for the *to_list* objects.

Feedback

See Also

- get_timing_paths
- report_bottleneck
- report_timing

---

timing_report_attribute_csv_delimiter

Specifies the delimiter for csv reports generated by the *report_attribute* command.

Data Types

String

Default ","

Description

This variable will be used as the delimiter for *report_attribute*. when in CSV output mode.

See Also

- report_attribute
- timing_report_attribute_skip_table_header

---

timing_report_attribute_skip_table_header

Specifies whether to omit the table header from reports generated by the *report_attribute* command.

Data Types

Boolean

Default false

Description

When this variable is set to *true*, reports generated by the *report_attribute* command do not have a table header; only the table contents are shown in the report. Otherwise, if the variable is set to *false* (the default), the table header is generated.

See Also

- report_attribute
- timing_report_attribute_csv_delimiter

Feedback

timing_report_fixed_width_columns_on_left

Controls the position where the instance/net/pin name details are displayed during the timing report process.

Data Types

BOOLEAN

Default FALSE

Description

Controls the position where the pin name details are displayed when you use the *report_timing*, *report_min_period*, and *report_min_pulse_width* commands. Valid values are *false* (the default), and *true*.

For *report_min_period*, and *report_min_pulse_width*, the feature works with the *full_clock* and *full_clock_expanded* path types but not the *summary* and *short* path types.

Full hierarchical pin names are becoming very long thereby affecting the readability of the generated timing reports. Columns can become misaligned. To help remedy this situation, this variable displays the instance/net/pin name (ie the Point column in the timing report) at the end of the report. This variable will make sure that all the columns are intact.

In the distributed multi-scenario analysis (DMSA) flow, to enable this feature,set this variable to TRUE in the manager. The tool uses the variable setting in the manager regardless of the variable setting in the workers.

Examples

The following example shows a conventional report and a report with the numeric, fixed-width columns on the left.

```
pt_shell> report_timing
  Startpoint: SET (input port clocked by CLK1)
  Endpoint: n02_reg (rising edge-triggered flip-flop clocked by CLK1)
  Path Group: CLK1
  Path Type: max

  Point                                    Incr       Path
  -----------------------------------------------------------
  clock CLK1 (rise edge)                   0.00       0.00
  clock network delay (propagated)         0.00       0.00
  input external delay                     0.50       0.50 f
  SET (in)                                 0.00       0.50 f
  n01_reg/Q (FDES)                         0.60 H     1.10 r
  OR2S_cell/Y (OR2S)                       0.00       1.10 r
  n02_reg/D (FDES) <-                      0.00       1.10 r
  data arrival time                                   1.10
```

```
  clock CLK1 (rise edge)                    10.00      10.00
  clock network delay (propagated)           0.00      10.00
  clock reconvergence pessimism              0.00      10.00
  n02_reg/T (FDES)                                     10.00 r
  library setup time                        -0.35       9.65
  data required time                                    9.65
  ------------------------------------------------------------
  data required time                                    9.65
  data arrival time                                    -1.10
  ------------------------------------------------------------
  slack (MET)                                           8.55

pt_shell> set_app_var timing_report_fixed_width_columns_on_left true
true
pt_shell> report_timing
  Startpoint: SET (input port clocked by CLK1)
  Endpoint: n02_reg (rising edge-triggered flip-flop clocked by CLK1)
  Path Group: CLK1
  Path Type: max

      Incr       Path    Point
  -----------------------------------------------------------
      0.00       0.00    clock CLK1 (rise edge)
      0.00       0.00    clock network delay (propagated)
      0.50       0.50 f  input external delay
      0.00       0.50 f  SET (in)
      0.60 H     1.10 r  n01_reg/Q (FDES)
      0.00       1.10 r  OR2S_cell/Y (OR2S)
      0.00       1.10 r  n02_reg/D (FDES) <-
                 1.10    data arrival time

     10.00      10.00    clock CLK1 (rise edge)
      0.00      10.00    clock network delay (propagated)
      0.00      10.00    clock reconvergence pessimism
                10.00 r  n02_reg/T (FDES)
     -0.35       9.65    library setup time
                 9.65    data required time
  -----------------------------------------------------------
                 9.65    data required time
                -1.10    data arrival time
  -----------------------------------------------------------
                 8.55    slack (MET)
```

See Also

- report_timing
- report_min_period
- report_min_pulse_width

Feedback

timing_report_hier_stub_pin_paths

Specifies whether the *report_timing* command reports paths ending at HyperScale stub pins (internal endpoints) in the internal separate path group.

Data Types

Boolean

Default true

Description

A timing path ending at a stub pin is a HyperScale-specific behavior. In the HyperScale flow, there are block-level and top-level analyses. Stub pin paths exist at the top-level analysis and are internal endpoints within a HyperScale block. Stub pins exist because block internal register-to-register paths are not fully retained at the top level. A stub pin is created when a starting register has paths leading to block output ports -- that is, starting from an interface register -- while the starting register also has timing paths leading to other registers inside the block, which makes the path shared with block internal paths. The points where a path branches to interface paths and internal paths are designated as stub pins. When a pin becomes a stub, its entire transitive fanout, including all the logic on the paths from the stub pin to the capture registers in the block, plus the clock paths of these internal capture register, are not retained in the block abstraction and therefore invisible at the top-level analysis. A stub pin is often one of the load pins of a multiload net. Topologically speaking, a stub pin is symmetric to a side input, which happens on a block input interface due to multifanin cells.

To reproduce the worst slack on the interface register in the transitive fanin of the stub pins, HyperScale can automatically capture the required times during block-level analysis and annotate on stub pins during top-level analysis when block abstraction is reused. These required times do not change the timing analysis on the fanin, but produced pin slack as if the fanout still presents, which is very important for ECO.

Set the *timing_report_hier_stub_pin_paths* variable to one of these values:

- *true* - Reports the timing paths ending at stub pins in the **HyperScale_stub_default** intrinsic path group. Note that these stub pins are usually not timing path endpoints in block-level or top-level flat analysis.
- *false* - Omits reporting the paths ending at stub pins.

Note:

This variable does not affect timing analysis; it only affects how the *report_timing* and *get_timing_paths* commands report timing paths.

Feedback

Examples

The following example shows the path reported in the **HyperScale_stub_default** intrinsic path group when the *timing_report_hier_stub_pin_paths* variable is set to *true*.

```
report_timing -path_type full_clock_expanded -from block/ffa2/Q
*************************************
Report : timing
        -path_type full_clock_expanded
        -delay_type max
        -max_paths 1
        -sort_by group
Design : SIMPLE
 ...
*************************************

  Startpoint: block/ffa2 (rising edge-triggered flip-flop clocked by
 SYSCLK)
  Endpoint: block/ffa4/D
               (internal path endpoint clocked by SYSCLK)
  Path Group: **HyperScale_stub_default**
  Path Type: max

  Point                                        Incr       Path
  -----------------------------------------------------------------
  clock SYSCLK (rise edge)                    0.000      0.000
  clock source latency                        0.700      0.700
  CLK (in)                                    0.000      0.700 r
  block/clk (BLOCK)                           0.000      0.700 r
  block/cbufa1/Y (CLKBUFX2)                   0.385      1.085 r
  block/ffa2/CK (DFFX2)                       0.000      1.085 r
  block/ffa2/Q (DFFX2) <-                     0.460      1.545 f
  block/ffa4/D (DFFX2)                        0.000      1.545 f
  data arrival time                                      1.545

  clock SYSCLK (rise edge)                    4.000      4.000
  clock source latency                        0.400      4.400
  CLK (in)                                    0.000      4.400 r
  block/clk (BLOCK)                           0.000      4.400 r
  block/cbufa1/A (CLKBUFX2)                   0.000      4.400 r
  output external delay                      -0.004      4.396
  data required time                                     4.396
  -----------------------------------------------------------------
  data required time                                     4.396
  data arrival time                                     -1.545
  -----------------------------------------------------------------
  slack (MET)                                            2.850


1

report_path_group
*************************************
```

Feedback

```
Report : path_group
Design : SIMPLE
 ...
*************************************

Path_Group     Weight    From      Through        To
-------------------------------------------------------------
**HyperScale_stub_default**
                1.00    -         -              -
**async_default**
                1.00    -         -              -
**clock_gating_default**
                1.00    -         -              -
**default**     1.00    -         -              -
SYSCLK          1.00    *         *              SYSCLK

1
```

See Also

- get_timing_paths
- report_timing
- hier_enable_analysis

---

timing_report_include_eco_attributes

Includes cell and net attribute information in reports generated by the *report_timing* command.

Data Types

Boolean

Default false

Description

When this variable is set to true, a column labeled "Attributes" is added in the output of the *report_timing* command to show information about cells and nets in the timing report:

- *dont_touch* (for cell or net)
- *dont_use* (for cell)
- *ideal_net* (for net)
- *via_ladder* (for pin)

Feedback

These letters d, i, and u appear in the Attributes column to indicate where the attribute is true for a cell or net in the timing point list:

```
pt_shell> set_app_var timing_report_include_eco_attributes true
 ...
pt_shell> report_timing ...
 ...
Attributes:
    d - dont_touch
    u - dont_use
    i - ideal_net
    V - via_ladder

Startpoint: A1/B1/reg_out
               (rising edge-triggered flip-flop clocked by clk)
  Endpoint: reg_out[11]
               (output port clocked by clk)
  Path Group: in2out
  Path Type: max

  Point                                    Incr       Path     Attributes
  ---------------------------------------------------------------------------
  clock clk (rise edge)                    0.00       0.00
  clock network delay (propagated)         3.30       3.30
  A1/B1/reg_out_reg_11_/CP (dfn)           0.00       3.30 r         V
  A1/B1/reg_out_reg_11_/Q (dfn)            0.87       4.18 f   i     V
  icc_route_opt6/ZN (inv7)                 0.00       4.18 r   i d u
  icc_route_opt8/ZN (inv7)                 0.07       4.25 f     d u
  reg_out[11] (out)                        0.01       4.25 f
  data arrival time                                   4.25

  clock clk (rise edge)                    1.67       1.67
  clock network delay (propagated)         0.00       1.67
  clock reconvergence pessimism            0.00       1.67
  output external delay                   -0.17       1.50
  data required time                                  1.50
  ---------------------------------------------------------------------------
  data required time                                  1.50
  data arrival time                                  -4.25
  ---------------------------------------------------------------------------
  slack (VIOLATED)                                   -2.75
```

When the variable is set to false (the default), the Attributes column is not displayed in timing reports.

See Also

- report_timing

Feedback

timing_report_invert_clock_min_pulse_width_constraints

Determines how the high/low min pulse width constraints set on a clock are applied at a pin.

Data Types

Boolean

Default false

Description

This variable determines how the high/low min pulse width constraints set on a clock are applied at a pin.

When this variable is *false*, the high/low min pulse width constraint, set by *set_min_pulse_width* on a clock, to be applied at a given pin in the clock network is selected independently of the sense of the clock network from clock source to the pin.

When this variable is *true*, the high/low min pulse width constraint, set by *set_min_pulse_width* on a clock, to be applied at a given pin in the clock network is selected based on the sense of the clock network from clock source to the pin. This means that if the clock waveform is inverted at pin relative to the clock source then the min pulse width constraints are also inverted. For non unate networks, the most restrictive of high/low min pulse width constraints is used.

Note that this variable only impacts min pulse width constraints set on clocks. Constraints set on design/pin with *set_min_pulse_width* or inferred from the library are applied directly as specified.

See Also

- report_constraint
- report_min_pulse_width
- set_min_pulse_width

timing_report_min_pulse_width_show_clock_compatibility

Backward compatibility to disable printing the clock name in the summary report of *report_min_pusle_width* -path_type summary and *report_constraint* -min_pulse_width

Data Types

Boolean

Default false

Feedback

Description

Setting this variable to *true* disables printing the clock name in the summary report of min_pule_width (clock name is printed by default).

See Also

- report_min_pulse_width

---

timing_report_output_delay_on_clock_source

Recognizes output delays specified on clock sources defined on output ports.

Data Types

Boolean

Default false

Description

If you set this variable to *true*, the tool will recognize the output delays specified on clock sources defined on output ports. As a result, the tool will be able to set output path delays and therefore, the output paths will be reported.

By default, the variable is set to *false* and the tool discards the output delays specified on clock source pins defined on output ports.

See Also

- set_output_delay

---

timing_report_pin_names_in_header

Causes the startpoint and endpoint lines of a timing report header to display specific pins instead of only the cells.

Data Types

Boolean

Default false

Feedback

Description

This variable affects how the *report_timing* command displays the startpoint and endpoint at the beginning of the path report when the object is a cell input or output. When the variable is set to *false* (the default), the cell instance names are displayed:

```
Startpoint: ORCA/BLENDER/s3_reg[18]
               (rising edge-triggered flip-flop clocked by SCLK)
  Endpoint: ORCA/BLENDER/s4_reg[31]
               (rising edge-triggered flip-flop clocked by SCLK)
```

When the variable is set to *true*, the specific startpoint and endpoint pins are displayed:

```
Startpoint: ORCA/BLENDER/s3_reg[18]/CP
               (rising edge-triggered flip-flop clocked by SCLK)
  Endpoint: ORCA/BLENDER/s4_reg[31]/D
               (rising edge-triggered flip-flop clocked by SCLK)
```

This variable also affects reports generated by other commands that display timing paths, such as the verbose modes of *report_constraint* and *report_clock_timing*.

See Also

- report_timing
- report_clock_timing
- report_constraint

---

timing_report_recalculation_status

Displays progress messages during an exhaustive path-based analysis.

Data Types

string

Default low

Description

This variable controls the reporting of information messages during path-based recalculation for the *report_timing* and *get_timing_paths* commands with the *-pba_mode exhaustive* option.

The number of messages varies based on the setting of the variable, as follows:

- *none* - Displays no messages.
- *low* - Displays a message only at the end of the recalculation for each clock group.

Feedback

- *medium* - Displays messages only at the beginning and end of the recalculation for each clock group.
- *high* - Displays all messages for *medium*; in addition, the tool displays the total number of endpoints to search and the completion percentage for searching these endpoints.

The following example shows information messages when the variable is set to *high*:

```
Information(nworst 5, max_paths 20): recalculating group
 **async_default**...
Information: No paths to recalculate.
Information(nworst 5, max_paths 20): recalculating group
 **clock_gating_default**...
Information: No paths to recalculate.
Information(nworst 5, max_paths 20): recalculating group **default**...
Information: No paths to recalculate.
Information(nworst 5, max_paths 20): recalculating group clk...
Information: recalculated_paths 10
Information: recalculated_paths 20
Information: recalculated_paths 30
Information: finished_endpoints 3, total_endpoints 5, recalculated_paths
 30
Information: finished_endpoints 5, total_endpoints 5, recalculated_paths
 30
Information: Recalculated 30 paths and returned 20 paths for group 'clk'.
  The maximum number of paths recalculated at an endpoint was 10.
```

See Also

- get_timing_paths
- report_timing

---

timing_report_skip_early_paths_at_intermediate_latches

Disables the reporting of timing paths with early arrival at intermediate latches.

Data Types

Boolean

Default false

Description

A path is said to arrive early at an intermediate latch if it arrives before the clock causes the latch to open.

Feedback

To control the reporting of timing paths that arrive early at intermediate latches, set this variable to one of these values:

- *false* (the default) - The *report_timing* command reports paths that arrive early at intermediate latches. These paths are indicated as "(early)" in the report at the individual transparency window where the arrival arrived early. The tool tries to identify which window is early on the path. Due to numerical rounding on paths that are only slightly early, it is possible for some early paths to be filtered out.
- *true* - The *report_timing* command skips paths that arrive early at intermediate latches.

This variable is effective only if you enable advanced analysis through transparent latches by setting the *timing_enable_through_paths* variable to *true*). With advanced analysis through transparent latches, you can report paths through latches as a single timing path that is composed of a sequence of paths segments. Each segment terminates at an intermediate latch.

The behavior invoked by setting this variable to true is incompatible with the *-cover_design*, *-cover_through* and *-start_end_pair* options to *report_timing* and *get_timing_paths*.

When the *-pba_mode path* and *-pba_mode exhaustive* options are used to *report_timing* and *get_timing_paths*, the paths which are "early" at intermediate latches before path-based analysis is performed on the paths, will be skipped, but the paths which are "early" at intermediate latches after path-based abalysis is performed on the paths, will not be skipped.

See Also

- report_timing
- timing_enable_through_paths

---

timing_report_status_level

Controls the number of progress messages displayed during the timing report process.

Data Types

string

Default none

Description

Controls the number of progress messages displayed when you use the *report_timing* and *report_constraint* commands. Valid values are *none* (the default), *low*, *medium*, and *high*.

Feedback

The number of messages varies based on the setting of the variable, as follows:

- *none* - Displays no messages.
- *low* - Displays messages only at the beginning and end of the timing report.
- *medium* - Displays all messages for *low*; in addition, displays messages at the beginning of searching for each clock group.
- *high* - Displays all messages for *medium* when you use the *report_timing* command; in addition, displays the total number of endpoints to search and the completion percentage for searching these endpoints.

The report_constraint command displays status only when the *timing_report_status_level* variable is set to *high*, and the *-verbose* option is used. The *report_constraint* command displays only progress status related to timing paths search.

This variable controls the display only for the timing report. Sometimes, the timing report might trigger a timing update. If you want to see the progress status for timing update, set the *timing_update_status_level* variable.

See Also

- report_constraint
- report_timing
- timing_update_status_level

---

timing_report_trace_ideal_clocks

Causes clock paths to be shown for ideal clock networks in *report_timing -path_type full_clock_expanded* and associated commands that display clock paths.

Data Types

Boolean

Default false

Description

This variable enables the display of ideal clock network paths by *report_timing -path_type full_clock_expanded* and similar reporting commands (*report_clock_timing -verbose* and *report_min_pulse_width -path_type full_clock_expanded*).

When the variable is set to *false* (the default), the clock path through an ideal clock network is not displayed. Instead, a single line summarizes the ideal clock latency. The topological path may nonetheless be of interest. For example, you might want to determine which ideal clock path is reaching a register where you did not expect it.

Feedback

When the variable is set to *true*, the paths through ideal clock networks are displayed by *report_timing -path_type full_clock_expanded*, even though the delays along such paths do not impact the timing of associated registers. Timing calculations remain governed by the ideal settings.

Note that the path increments shown in the ideal network might not match the increments that appear when the ideal network is changed to propagated by the *set_propagated_clock* command.

When the variable is set to *true*, a timing path gathered using the *get_timing_paths* command supports the query of *capture_clock_paths* and *launch_clock_paths* attributes even when the path is clocked by ideal clocks.

See Also

- report_timing
- get_timing_paths
- report_clock_timing
- report_ideal_network
- set_ideal_latency
- set_ideal_network
- set_propagated_clock

---

timing_report_unconstrained_paths

Specifies whether the tool searches for unconstrained paths when you use the *report_timing* or *get_timing_paths* command.

Data Types

Boolean

Default false

Description

When this variable is set to *false* (the default), the *report_timing* and *get_timing_paths* commands search only for constrained paths. If the design has many unconstrained paths, this setting uses less runtime because the tool does not spend time analyzing the unconstrained paths.

To search for and report unconstrained paths as well as constrained paths, set this variable to *true*. Unconstrained paths have a startpoint or endpoint without a defined clock. For example, if no clocks, input delays, or output delays are defined, all paths are

Feedback

unconstrained. The tool first searches for constrained paths that meet the search criteria; if there are none, it then searches for unconstrained paths.

When searching for unconstrained paths, instead of finding the paths with the least slack, the *get_timing_paths* and *report_timing* commands look for and report paths with the longest delay by default, or for paths with the shortest delay if the *-delay_type* option is set to *min*. Searching for unconstrained paths can cause unexpectedly long runtimes, as reported by the UITE-413 warning message.

When the *timing_report_unconstrained_paths* variable is set to true, the *report_timing* command reports unconstrained paths. Starting with the Q-2019.12 release, if the design has no constrained paths at all of the specified type (max or min), then at least one topological option such as *-from*, *-through*, or *-to* must be used in the *report_timing* command.

See Also

- get_timing_paths
- report_timing

---

timing_report_unconstrained_paths_from_nontimed_startpoints

This variable sets whether to include non-timed startpoints for unconstrained paths reporting.

Data Types

Boolean

Default false

Description

If *timing_report_unconstrained_paths_from_nontimed_startpoints* is set to false, then startpoints which do not launch timing signals are ignored during unconstrained reporting. This variable has no impact if unconstrained reporting is disabled (i.e. when *timing_report_unconstrained_paths* is *false*).

When *timing_report_unconstrained_paths_from_nontimed_startpoints* is set to *false* (the default), unconstrained reporting by *report_timing* and *get_timing_paths* ignores paths whose startpoints are inactive from a timing perspective.

Examples of such non-timed startpoints include (a) pins whose cells or nets are structurally disconnected from the circuit; (b) pins without backward arcs or whose backward arcs are disabled.

When this variable is set to *true* then unconstrained reporting will allow paths whose startpoints are inactive from a timing perspective, using a launch time of 0.0 at the

Feedback

startpoint. This was the default behavior of releases prior the 2019.03 release. The runtime and memory overhead of unconstrained reporting may increase in this mode.

The variable may be toggled before calls to *get_timing_paths* / *report_timing* and does not require an additional timing update.

This variable is not to be confused with *timing_report_always_use_valid_start_end_points* which determines the handling of user-specified from-pins which are not startpoints at all in a topological sense, specifying whether they should be treated as through-pins or simply rejected.

See Also

- timing_report_unconstrained_paths
- timing_report_always_use_valid_start_end_points
- report_timing
- get_timing_paths

---

timing_report_union_tns

Specifies whether to use the union method for reporting total negative slack and number of violating endpoints.

Data Types

Boolean

Default true

Description

If this variable is set to *false*, *report_qor -summary* computes the design's TNS as the sum of TNS for all path groups in that design. In multi-scenario analysis, the design's TNS is computed as the sum of TNS for all path groups in all scenarios in focus.

If the variable is set to *true* (the default), *report_qor -summary* counts each violating endpoint only once when computing the total negative slack (TNS) and the number of violating endpoints for the design. In multi-scenario analysis, the design's TNS is computed as the sum of worst negative violations per endpoint across all scenarios and path groups.

See Also

- report_qor

Feedback

timing_report_use_worst_parallel_cell_arc

Enables uniquification of paths through parallel cell arcs.

Data Types

Boolean

Default true

Description

Multiple cell arcs of the same sense can exist between the same pair of pins. In designs with large numbers of such parallel cell arcs, there can often be an explosion of seemingly identical paths reported. Use this variable to specify whether to report every path through parallel cell arcs.

When you set this variable to *false*, PrimeTime reports all paths through parallel cell arcs.

When you set this variable to *true*, PrimeTime reports only the worst path through a set of parallel cell arcs. PrimeTime chooses the arc with the worst delay to determine the worst path. This variable setting has no effect in designs with no parallel cell arcs.

See Also

- get_timing_paths
- report_timing

timing_save_block_level_reporting_data

Forces PrimeTime HyperScale to save all report data at the block level.

Data Types

Boolean

Default false

Description

This variable controls how block-level data is generated in the HyperScale flow for reporting commands. By default, the reporting command has to be called at the block level in order for the data to be available at the top level. If this variable is set to *true*, the generation of that data is done automatically at the block level. There is no need for an explicit command call at the block level to get block data at the top.

Feedback

See Also

- report_analysis_coverage
- report_global_timing
- report_qor

---

timing_save_hier_context_data

Specifies whether the *write_hier_data* command writes out the hierarchical context data for HyperScale blocks.

Data Types

Boolean

Default true

Description

In the HyperScale analysis flow, the *set_hier_config* command specifies which blocks in the design are HyperScale blocks. By default, when the use the *set_hier_config* command, the design is considered a top-level design, and the *write_hier_data* command writes out the hierarchical context data for HyperScale blocks in the design.

In some flows, including some bottom-up flows, you do not need to write out the block context for the HyperScale blocks in the design and you do not need to perform hierarchical scope checking for those blocks. In that case, to save runtime and disk space, set the *timing_save_hier_context_data* variable to *false*. Then the *write_hier_data* command skips all context capture and scope checking, except a small subset needed for nonfixable scope violations. Nonfixable violations are still checked and reported by the *report_constraint* command.

See Also

- report_constraint
- set_hier_config
- write_hier_data
- timing_save_hier_model_data

Feedback

timing_save_hier_model_data

Specifies explicitly that the current design is a HyperScale block-level design, which ensures that the *write_hier_data* command writes out a HyperScale block model for the design.

Data Types

Boolean

Default false

Description

In the HyperScale analysis flow, the current design is treated as a block-level design under any one or more of the following conditions:

- The design does not contain HyperScale blocks (the *set_hier_config* command is not used in the session).
- The *read_context* command is used in the session.
- The *timing_save_hier_model_data* variable is set to *true*.

In any of these cases, the *write_hier_data* command writes out a HyperScale block model for the current design. This model can be used for analysis at the next higher level of hierarchy.

In most block-level designs, it is not necessary to set the *timing_save_hier_context_data* variable because the lack of lower-level HyperScale blocks, or using the *read_context* command, already implies that it is a block-level design.

However, in the unusual case of a mid-level HyperScale block containing lower-level HyperScale blocks, when you do not use the *read_context* command, set the *timing_save_hier_context_data* variable to *true* to ensure that the *write_hier_data* command writes out a HyperScale block model for the design.

See Also

- read_context
- set_hier_config
- write_hier_data
- timing_save_hier_context_data

Feedback

timing_save_pin_arrival_and_required

Specifies whether the arrival and required times of all pins are kept in memory.

Data Types

Boolean

Default false

Description

When set to *true*, the arrival and required times of all pins of the design are kept in memory. When set to *false* (the default), arrival and required times are stored on an as-needed basis for the analysis you are performing.

This variable is very similar in effect to the *timing_save_pin_arrival_and_slack* variable, enabling the same features (particularly slack and arrival window attribute query). To query slack attributes or arrival window attributes on pins that are not endpoints of the design, set one or both of these variables to *true*.

You should also set this variable to *true* in the specific case where the *write_sdf_constraints* command forms part of your flow, as this command requires additional information be stored at all pins. If the *write_sdf_constraints* command is used while this variable is set to *false*, it is set to *true* automatically and an informational message issued.

See Also

- report_timing
- timing_save_pin_arrival_and_slack

timing_save_pin_arrival_and_slack

Specifies whether the slacks of all pins are kept in memory.

Data Types

Boolean

Default false

Description

When set to *true*, the arrival times and slacks of all pins of the design are kept in memory. When set to *false* (the default), arrival times and slacks are preserved only for endpoints of the design.

Feedback

This variable is very similar in effect to the *timing_save_pin_arrival_and_required* variable, enabling the same features (particularly slack and arrival window attribute query). To query slack attributes or arrival window attributes on pins that are not endpoints of the design, set one or both of these variables to *true*.

See Also

- report_timing
- timing_save_pin_arrival_and_required

---

timing_separate_hier_side_inputs

Specifies if PrimeTime HyperScale analysis should group paths starting from side inputs (internal startpoint) into its own and separate path groups.

Data Types

Boolean

Default false

Description

Side input path is a HyperScale specific behavior. In the HyperScale analysis flow, there are block-level and top-level analyses. Side input paths at block level are internal register to register paths, and they converge with interface paths by sharing some common logic through some multiple-input cells, For example, an AND gate whose A pin is on an interface path starting from a block port and leading to some register, while its B pin is on a path starting from a register cell. During HyperScale block data reduction, paths through the A pin are retained while the fanin of B pin is removed. This pin is the side input pin of the interface path. To have the same accuracy of flat analysis at the HyperScale top level for the paths leading to the common register of the AND gate, the tool captures and annotates all the timing data propagated at B pin in binary format. At the top level, the paths appear to be starting from these dangling side input pins.

When this variable is set to *false* (the default), the paths starting from these side inputs maintain their original path group (capture clock by default). *report_timing* and *get_timing_paths* commands report them as critical paths in the same path group as the interface paths converging at the same block interface registers.

When this variable is set to *true*. An internal path group "**HyperScale_side_inputs**" is automatically created and all the paths starting from the block side inputs at HyperScale top level analysis are analyzed and reported in this group. Note this does not impact the actual analysis accuracy or QoR, affects only the path reporting.

Feedback

Examples

The following example shows the path reported in HyperScale side input when the variable is set to *true*.

```
report_timing -from $side_input
****************************************
Report : timing
        -path_type full
        -delay_type max
        -max_paths 1
 ...
****************************************

  Startpoint: blk/u3/B (internal path startpoint clocked by top_CLK)
  Endpoint: blk/ff1 (rising edge-triggered flip-flop clocked by top_CLK)
  Path Group: **HyperScale_side_input**
  Path Type: max

  Point                                        Incr       Path
  -----------------------------------------------------------------
  clock top_CLK (rise edge)                    0.00       0.00
  clock network delay (propagated)             0.02       0.02
  input external delay                         4.68       4.70 f
  blk/u3/B (ND2)                               0.00       4.70 f
  blk/u3/Z (ND2)                               0.64 &     5.34 r
  blk/ff1/D (FD2)                              0.00 &     5.34 r
  data arrival time                                       5.34

  clock top_CLK (rise edge)                   10.00      10.00
  clock network delay (propagated)             1.16      11.16
  blk/ff1/CP (FD2)                                       11.16 r
  library setup time                          -0.85      10.31
  data required time                                     10.31
  -----------------------------------------------------------------
  data required time                                     10.31
  data arrival time                                      -5.34
  -----------------------------------------------------------------
  slack (MET)                                             4.97
1

report_path_group
****************************************
Report : path_group
 ...
****************************************

Path_Group     Weight   From      Through        To
--------------------------------------------------------------------------------
-------
**HyperScale_side_input**
                 1.00   { blk/u3/B ... }
                                 *              *
```

Feedback

```
**async_default**
                  1.00    -          -               -
**clock_gating_default**
                  1.00    -          -               -
**default**       1.00    -          -               -
top_CLK           1.00    *          *               top_CLK
top_CLK2          1.00    *          *               top_CLK2
```

See Also

- get_timing_paths
- hier_enable_analysis
- report_timing

---

timing_simultaneous_clock_data_port_compatibility

Enables or disables the simultaneous behavior of input port as a clock and data port.

Data Types

Boolean

Default false

Description

When the *timing_simultaneous_clock_data_port_compatibility* variable is set to *false* (the default), an input port can behave simultaneously as a clock and data port, and you can use the *set_input_delay* command to define the timing requirements for input ports relative to a clock. In this situation, the following applies:

- If you specify the *set_input_delay* command relative to a clock defined at the same port and the port has data sinks, the command is ignored and an error message is issued. There is only one signal coming to port, and it cannot be at the same time data relative to a clock and the clock signal itself.
- If you specify the *set_input_delay* command relative to a clock defined at a different port and the port has data sinks, the input delay is set and controls data edges launched from the port relative to the clock.
- Regardless of the location of the data port, if the clock port does not fanout to data sinks, the input delay on the clock port is ignored and you receive an error message.

When you set the *timing_simultaneous_clock_data_port_compatibility* variable to *true*, the simultaneous behavior is disabled, and the *set_input_delay* command defines the arrival time relative to a clock. In this situation, when an input port has a clock defined on it, the tool considers the port exclusively as a clock port and imposes restriction on the data edges that are launched. Also, you cannot set input delays relative to another clock.

Feedback

See Also

- set_clock_latency
- set_input_delay

---

timing_single_edge_clock_gating_backward_compatibility

Enables clock gating checks when only one edge of a clock arrives at the clock pin of a clock gate.

Data Types

Boolean

Default false

Description

When this variable is set to *false* (the default), the tool skips clock gating checks for the missing edge. The clock gating check is performed for only the edge that is not missing. The PTE-109 warning message indicates which edge is missing and which check is not performed.

When this variable is set to *true*, the tool skips clock gating checks if only one edge of the clock (rise or fall) arrives at the clock gate. When this happens, the PTE-074 warning message is issued.

See Also

- PTE-074
- PTE-109

---

timing_through_path_max_segments

Specifies how often loop-breaker latches should be distributed along transparent latch sequences.

Data Types

integer

Default 5

Feedback

Description

When advanced latch analysis is enabled (by setting the *timing_enable_through_paths* variable to *true*), transparent latches can be throughpaths, which allows timing reports to traverse through them to include multiple transparent latch path segments.

However, to bound analysis complexity and avoid issues with loops, the tool converts some of these latches to "loop-breaker" latches, causing them to become level-sensitive path startpoints and endpoints (the behavior used when advanced latch analysis is disabled).

The procedure for selecting these loop-breaker latches is as follows:

1. Extract the latch graph (defined as the design logic in the fanin/fanout of all transparent latches, bounded by non-transparent startpoints/endpoints).
2. Select explicitly-specified loop-breaker latches resulting from *set_latch_loop_breaker* specifications.
3. Select loop-breaker latches to break loops that remain through transparent-latch paths. (This step tries to honor *set_latch_loop_breaker -avoid* specifications, but loop-breaking takes priority.)
4. Starting at latch graph endpoints, proceed backward and convert every *N*th transparent latch to a "path-breaker" latch, which is a loop-breaker latch used to break up long sequences of transparent latches. (If the *N*th latch has a *set_latch_loop_breaker -avoid* specification, the trace is allowed to proceed one additional latch level backward.)

This variable sets the value *N* in step 4. Smaller values cause more path-breaker latches to be inserted, reducing the maximum number of transparent latch path segments that can be included a single timing path report.

A special value of *0* disables step 4 completely, maximizing the number of transparent latch path segments that can be reported. This configuration is allowed, but analysis might be slower.

(Note that loop-breaker latches selected in step 3 become endpoints for step 4.)

To report the loop-breaker and path-breaker latches in your design, use the *report_latch_loop_groups* command. (Note that this report can refer to path-breaker latches within loop logic as "loop-breaker" latches, even though they were inserted during path-breaker insertion according to this variable.)

This variable has no effect when the *timing_enable_through_paths* variable is set to its default of *false*.

Feedback

See Also

- report_timing
- set_latch_loop_breaker
- timing_enable_through_paths

---

timing_update_effort

Controls the computational effort (in CPU time) and memory usage for the fast timing update algorithm in PrimeTime.

Data Types

string

Default medium

Description

Controls the computational effort (in CPU time) and memory usage for the fast timing update algorithm in PrimeTime. Allowed values are as follows:

- *low*: The computational effort is low (that is, the *update_timing* command is fast); however, the memory usage is not bounded and can increase significantly if the number of changes is very large.
- *medium* (the default): The computational effort is low (that is, the *update_timing* command is fast); however, the memory usage is bounded by 10% over the memory usage for initial timing. If this bound is not sufficient to accommodate all of your changes, PrimeTime issues an informational message and automatically switches to a less efficient algorithm that is more conservative in the memory usage. In this case, you might need to change to the *low* value. However, even with this small 10% bound, PrimeTime can accommodate a relatively large number of changes. Therefore, it is unlikely that you need to change this default setting.
- *high*: The computational effort is high (that is, the *update_timing* command becomes slow); however, there is no increase in the memory used for the initial timing of the design.

If a design is timed again after a change, the algorithm reuses a portion of the computation done for the initial timing. For example, if a design was loaded and timed using the *update_timing* command, and the capacitance on a port was changed using the *set_load* command, the effort spent in the execution of a subsequently issued the *update_timing* command is smaller than that for the first issued the *update_timing* command.

Feedback

See Also

- set_load
- update_timing

---

timing_update_include_graph_based_refinement

Controls how HyperTrace updates are performed during timing updates.

Data Types

string

Default default

Description

HyperTrace accelerated exhaustive PBA is enabled when the *timing_enable_graph_based_refinement* variable is set to *true*.

The first HyperTrace update invoked is always a full update of the critical-region design data. This variable controls how HyperTrace updates behave in subsequent incremental timing updates, provided that:

- A full (non-incremental) HyperTrace update has already been performed.
- No unsupported ECO change commands have been issued.

Allowed values for this variable are:

default

In this mode,

- Full timing updates never perform HyperTrace updates.
- The first exhaustive PBA command performs a full HyperTrace update.
- Subsequent incremental timing updates (including those implicitly performed by exhaustive PBA reporting after design changes) perform incremental HyperTrace updates.

Feedback

always

In this mode,

- Full timing updates always perform full HyperTrace updates.
- Subsequent incremental timing updates (including those implicitly performed by exhaustive PBA reporting after supported design changes) perform incremental HyperTrace updates.
- Exhaustive PBA commands themselves never require a HyperTrace update (full or incremental).

If using *parallel_execute* to perform exhaustive PBA reporting, use this value to precompute the HyperTrace graph-based refinement before parallel command execution is run. (Refinement cannot be computed within a *parallel_execute* command.)

never

In this mode,

- Timing updates (full or incremental) never perform HyperTrace updates.
- The first exhaustive PBA command performs a full HyperTrace update.
- Subsequent exhaustive PBA commands after supported design changes also perform full (not incremental) HyperTrace updates.

If you are running a script that performs manual ECO operations (*size_cell*, *insert_buffer*, and so on) that are not intermixed with exhaustive PBA reporting, consider temporarily setting this variable to *never* to defer HyperTrace updates until the script is complete.

Note that with this value, HyperTrace updates are always full updates. After the ECO operations are complete, be sure to set the this variable back to *default* to allow incremental HyperTrace updates.

See Also

- timing_enable_graph_based_refinement
- timing_refinement_max_slack_threshold
- timing_refinement_min_slack_threshold

---

timing_update_status_level

Controls the number of progress messages displayed during the timing update process.

Feedback

Data Types

string

Default none

Description

This variable controls the number of progress messages displayed during the timing update process. The following settings are allowed:

- *none* - Displays no messages.
- *low* - Displays messages only at the beginning and the end of the update.
- *medium* - Displays all messages for the *low* setting, with the addition of messages for intermediate timing update steps, constant propagation, delay calculation, and slack computation.
- *high* - Displays all messages for the *medium* setting, with the addition of messages for the delay calculation and arrival calculation steps, the completion percentage in large designs, and groups for which slack computation is performed.

See Also

- report_timing
- update_timing

---

timing_use_constraint_derates_for_pulse_checks

Enables or disables using timing constraint derates for min_pulse_width and min_period constraints.

Data Types

Boolean

Default false

Feedback

Description

If you set this variable to *true*, PrimeTime uses a legacy behavior in which constraint arc derates defined by the *set_timing_derate -cell_check -late* command are used to derate the *min_pulse_width* and *min_period* checks as follows:

- *set_timing_derate -cell_check -late -fall* applies to *min_pulse_width* checks for low pulses.
- *set_timing_derate -cell_check -late -rise* applies to *min_pulse_width* checks for high pulses.
- *set_timing_derate -cell_check -late* (largest across *-rise* and *-fall*) applies to *min_period* checks.
- Newer-style derates applied with the *-min_period* and *-min_pulse_width* options of *set_timing_derate* are ignored.

Note that this behavior does not apply to the derates set using the *-pocvm_subtract_sigma_factor_from_nominal* option.

See Also

- report_min_pulse_width
- set_timing_derate

---

timing_use_link_library_on_best_match_failure

For library scaling groups, specifies how a best-match failure should be handled.

Data Types

Boolean

Default true

Description

For library scaling groups, the best-match feature allows interpolation between libraries to be disabled, so that delay calculation is performed only at the available library conditions, directly using library data.

During delay calculation, the tool attempts to find the library matching the needed PVT conditions (within the specified tolerance).

Feedback

If a best-match library is not found, this variable specifies what fallback library should be used:

- When set to *true* (the default), the link library data is used, and an SLG-333 warning is issued.
- When set to *false*, the scaling group library with the closest PVT conditions is used, and an SLG-332 warning is issued.

See Also

- define_scaling_lib_group

---

timing_use_slew_variation_in_constraint_arcs

Enables transition variation impact in constraint variation in parametric on-chip variation (POCV) analysis.

Data Types

string

Default none

Description

You can set this variable to one of the following:

- *none* (default) - Do not consider transition variation impacts on constraint variation for setup or hold arcs.
- *setup* - Consider transition variation impact for setup arcs only.
- *hold* - Consider transition variation impact for hold arcs only.
- *setup_hold* - Consider transition variation impact for both setup and hold arcs.

This variable has an effect only when the following variables are also set:

```
pt_shell> set_app_var timing_pocvm_enable_analysis true
pt_shell> set_app_var timing_enable_slew_variation true
pt_shell> set_app_var timing_enable_constraint_variation true
```

See Also

- timing_pocvm_enable_analysis
- timing_enable_slew_variation
- timing_enable_constraint_variation

Feedback

timing_use_zero_slew_for_annotated_arcs

Allows disabling of the slew calculation to enhance performance in a pure SDF flow.

Data Types

list

Default auto

Description

This variable allows you to sacrifice slew calculation for performance in an SDF flow. You can set this variable to one of these values:

- *auto* (the default) - Allows automatic switching to the SDF flow if more than 95 percent of delay arcs on a design have annotated values.
- *always* - Uses a zero value for transition time on the load pins of fully annotated delay arcs. Fully annotated arcs have values for both rise and fall, either read from an SDF file, or set with the *set_annotated_delay* command. If blocks of arcs that are not annotated exist, delay is estimated using the best available slew at the inputs.
- *never* - Does not use the SDF flow.

See Also

- read_sdf

timing_voltage_swing_high_pulse_threshold

Specifies the threshold voltage for measuring high pulse voltage levels during voltage swing checking with advanced waveform propagation.

Data Types

float

Default 0.99

Description

This variable sets the threshold for voltage swing checking of high pulses (low-high-low) in the clock network by the *report_min_pulse_width* command with the *-voltage_swing* option.

The minimum required voltage reached by a high pulse is specified as a fraction of the rail voltage. For a more conservative check, set the threshold to a value closer to 1.0.

Feedback

This feature works only when waveform propagation is enabled by setting the *delay_calc_waveform_analysis_mode* variable to *full_design* and voltage swing checking is enabled by setting the *timing_enable_voltage_swing* variable to *true*.

See Also

- report_min_pulse_width
- delay_calc_waveform_analysis_mode
- timing_enable_voltage_swing
- timing_voltage_swing_low_pulse_threshold

---

timing_voltage_swing_low_pulse_threshold

Specifies the threshold voltage for measuring low pulse voltage levels during voltage swing checking with advanced waveform propagation.

Data Types

float

Default 0.01

Description

This variable sets the threshold for voltage swing checking of low pulses (high-low-high) in the clock network by the *report_min_pulse_width* command with the *-voltage_swing* option.

The maximum allowable low voltage reached by a low pulse is specified as a fraction of the rail voltage. For a more conservative check, set the threshold to a value closer to 0.0.

This feature works only when waveform propagation is enabled by setting the *delay_calc_waveform_analysis_mode* variable to *full_design* and voltage swing checking is enabled by setting the *timing_enable_voltage_swing* variable to *true*.

See Also

- report_min_pulse_width
- delay_calc_waveform_analysis_mode
- timing_enable_voltage_swing
- timing_voltage_swing_high_pulse_threshold

Feedback

timing_vt_mistracking_monte_carlo_sample_size

This is a synonym for the *vt_mistracking_monte_carlo_sample_size* variable.

Data Types

integer

Default 10000

See Also

- vt_mistracking_enable_path_analysis
- vt_mistracking_analysis_mode

training_data_directory

Enables machine learning for faster power recovery by the *fix_eco_power* command and specifies the directory for storing the training data.

Data Types

string

Default ""

Description

To enable machine learning for faster power recovery by the *fix_eco_power* command, set the *training_data_directory* variable to the name of a directory. The *fix_eco_power* command uses this directory to store training data during power recovery in the current session and to retrieve the training data from previous power recovery sessions. No other commands are needed to generate or use training data.

For distributed multi-scenario analysis (DMSA), set this variable in the manager.

To disable generation and usage of training data for power recovery, set the *training_data_directory* variable to an empty string (a pair of quotations marks: ""), which is the default setting.

Generally, you should allow the *fix_eco_power* command to read all of the files in the training directory, so that it can find the most suitable data to match the current design. However, if you want to read only a subset of the files in the directory, you can do so by using the *-training_data* option of the *fix_eco_power* command.

If you have created custom scripts to perform cell sizing for power optimization, you can direct the tool to learn from these strategies and apply them in future execution of

Feedback

the *fix_eco_power* command. To do this, use the *record_training_data*, *size_cell*, and *write_training_data* commands.

Note: Using this feature requires a PrimeTime-ADV-PLUS license.

See Also

- fix_eco_power
- record_training_data
- write_training_data

u

upf_allow_DD_primary_with_supply_sets

Enables use of a domain-dependent primary with an explicit supply set.

Data Types

Boolean

Default false

Description

Set this variable to *true* to enable coexisting of domain-dependent primary supply nets with explicit supply sets.

See Also

- load_upf

upf_allow_net_voltage_override

Enables the propagation of voltage from top supply_net to lower level nets.

Data Types

Boolean

Default false

Feedback

Description

When this variable is set to *true*, the top level net's voltage is propagated to lower level nets. In the case of a conflict with an existing voltage on a lower level net, the top level net's voltage overrides the lower level value.

See Also

- load_upf

---

upf_create_implicit_supply_sets

Enables creation of supply set handles for the power domains.

Data Types

Boolean

Default true

Description

Set this variable to *true* to enable creation of supply set handles. When this variable is *true*, the tool creates the *primary*, *default_isolation*, and *default_retention* supply set handles while creating the power domains. When this variable is *false*, supply set handles are not created for any power domains.

Note:

Set this variable before creating the power domains. After creating the power domains, this variable is considered read-only. The tool issues an error if you change the value of this variable after creating the power domain.

See Also

- create_power_domain

---

upf_enable_pst

Enables or disables UPF PST constructs in the tool.

Data Types

Boolean

Default false

Feedback

Description

This variable needs to be set to *true* before load_upf to enable UPF PST constructs. With UPF PST, PrimePower can perform PST based power analysis in averaged power analysis mode. User can then use *report_power -pst* to show power consumption per valid power state combination at the top level of the design.

See Also

- add_power_state
- report_power

---

upf_name_map

Specifies a list of {design_name name_map_file} to be used during golden upf reapplication.

Data Types

list

Default NULL

Description

This variable specifies a list in the format of {*design_name name_map_file*} to be used when golden UPF is reapplied to a post synthesis netlist. The *name_map_file* is the name map file for the *design_name* design. The specified *name_map_file* guides the golden UPF reapplication on the *design_name* design.

See Also

- load_upf
- enable_golden_upf

---

upf_power_switches_always_on

Controls if UPF power switches are considered to be always on when tracing pg connectivty.

Data Types

Boolean

Default false

Feedback

Description

Set this variable to *true* to enable tracing of the pg network through UPF power switches. As a result the supply_nets on both sides of the switch will be considered to be always at the same voltage. When this variable is set to *true* the *set_voltage* and *set_voltage_levels* will apply to all the connected suply_net segments on both sides of the switches. Also for SMVA analysis there will be no domain crossing inferred between supply_nets on the two sides of the switches. The behavior of the *upf_power_switches_always_on* variable is analogous to the connections inferred between supply nets declared as equivalent by the *set_equivalent* command.

Note:

Set this variable before assigning voltages or supplies. After creating the power domains, this variable is considered read-only. The tool issues an error if you change the value of this variable after creating power domains.

See Also

- set_voltage

---

upf_relax_association_checks

Allows the specification of an explicitly created supply_set with the -handle option.

Data Types

Boolean

Default false

Description

This variable affects the *associate_supply_set* command.

In UPF 2.0, only implicit supply sets can be specified with the *-handle* option. By default, PrimeTime errors out if an explicit *supply_set* is specified.

When this variable is set to *true*, you can specify an explicitly created *supply_set* with the *-handle* option. This enables association of two explicitly created *supply_sets*.

See Also

- load_upf

Feedback

upf_use_driver_receiver_for_io_voltages

Specifies whether input/output voltages are automatically derived from the driver, receiver, or pad cell.

Data Types

string

Default false

Description

You can set this variable to the following values:

- *false* (the default) - Uses the existing user-defined port voltages; does not derive input/output voltages from the driver, receiver, or pad cell. This setting can cause issues related to scaling library groups.
- *true* - Automatically derives input/output voltages from the driver, receiver, or pad cell and overwrites the existing user-defined port voltages.
- *pad* - Automatically derives input/output voltages from the pad cell (is_pad_cell=true) only and overwrites the existing user-defined port voltages.

upf_wscript_retain_object_name_scope

Specifies whether the *write_script* command writes out hierarchical scope names.

Data Types

Boolean

Default false

Description

If this variable is set to *false* (the default), the *write_script* command does not use hierarchical scope names for supply net names in *set_voltage* commands.

If you set this variable to *true*, the *write_script* command writes out hierarchical scope names.

See Also

- set_voltage
- write_script

Feedback

v

var_enable_bounding_mixed_vt_derate

This is a synonym for the *vt_mistracking_enable_bounding_analysis* variable.

Data Types

Boolean

Default false

See Also

- set_vt_mistracking_derate
- update_timing
- get_timing_paths
- vt_mistracking_enable_path_analysis

variation_derived_scalar_attribute_mode

Enables get_attribute command to return statistical timing attributes of timing_path and timing_point collections in variation-aware static timing analysis.

Data Types

string

Default quantile

Description

This variable is used to enable or disable statistical timing attributes as the return values of *get_attribute* command for timing_path and timing_point collections. If it is set to *mean* or *quantile*, the arrival, slack, required, or transition values returned by the *get_attribute* command are replaced with the mean or quantile values of corresponding variation_arrival, variation_slack, variation_required, or variation_transition attributes. The default is *quantile* in variation-aware analysis. Setting this variable to *none* disables statistical attributes, and *get_attribute* returns the corner values. If variation-aware analysis is turned off, the *get_attribute* command always returns the corner values regardless of the value of this variable.

The *sort_collection* and *filter_collection* are affected if this variable is set to *mean* or *quantile* while related timing attributes are used in the sorting or filtering schemes. This

Feedback

feature allows you to create custom timing_path collections that are sorted or filtered by statistical timing attributes.

Examples

The following example replaces the returned arrival value of get_attribute from corner value to mean of variation_arrival.

```
pt_shell> set variation_derived_scalar_attribute_mode none
none
pt_shell> get_attribute $path arrival
0.25
pt_shell> set variation_derived_scalar_attribute_mode mean
mean
pt_shell> get_attribute $path arrival
0.22
```

This example creates a new timing_path collection by sorting an existing collection by statistical quantile of variation_slack.

```
pt_shell> set path [get_timing_path -nworst 10]
_sel22
pt_shell> set variation_derived_scalar_attribute_mode quantile
quantile
pt_shell> set stat_path [sort_collection $path "arrival"]
_sel23
```

See Also

- filter_collection
- get_attribute
- sort_collection

---

variation_report_timing_increment_format

Controls the display of report timing increments for variation-aware timing paths.

Data Types

string

Default effective_delay

Description

This variable affects the display of paths which have been recalculated within the context of a variation-aware timing analysis. The transition times, delays, and arrival times associated with these paths are statistical in nature. This variable lets you configure the display of the delays appearing in the increment column (labeled Incr).

Feedback

Set the variable to one of the following values:

- *effective_delay* (the default) - Displays each increment equal to the scalar difference between the arrival values appearing in the path column.
- *delay_variation* - Displays the quantile value (or mean value) of the statistical increment, according to the *variation_derived_scalar_attribute_mode* variable.

The arrival times and transitions are also displayed using scalar representations of their underlying distributions. These also obey the *variation_derived_scalar_attribute_mode* variable.

See Also

- report_timing
- variation_derived_scalar_attribute_mode

---

vsa_non_dslg_cell_threshold

Specifies the maximum number of cells have no scaling library or are with -exact_match_only mode on the path.

Data Types

integer

Default 0

Description

This variable specifies the maximum number of cells without defined voltage-sacling library group (DSLG) or with -exact_match_only mode DSLG, in a path. If the number of such cells in the path exceeds this threshold, the tool skips voltage slack analysis. The default value of this variable is 0. This means that voltage slack analysis is skipped if there is any cell without valid DSLG or with exact_match_only mode DSLG, in the path.

---

vsa_voltage_slack_search_limit

Specifies the voltage slack search limitation in PrimeShield voltage slack analysis.

Data Types

float

Default 0.1

Feedback

Description

This variable specifies the lower and the upper limit of the voltage slack search range in voltage slack analysis. The default value of this variable is 0.1V. This means that, the minimum and maximum voltage slack search limits are -0.1V and 0.1V, respectively.

---

vt_mistracking_analysis_mode

Specifies the advanced Vt mistracking analysis mode.

Data Types

String

Default enumeration

Description

To specify the analysis mode, set this variable to one of the two values:

- *enumeration* - In this mode, each Vt class has a corner based derating model: min and max derating factors. The Vt mistracking impact is modeled by the enumerations of two extremes of all Vt classes. If there are N Vt classes, the number of enumerations will be two to the power of N. The path-based timing analysis will be performed for each enumeration by applying the same derating factor to the same Vt class in both launch and capture paths. The worst timing will be selected among all enumerations.
- *monte_carlo* - In this mode, each Vt class has a statistical derating model, and an efficient monte-carlo (MC) based approach is used to perform VT mistracking analysis. Partial correlation among different Vt classes can also be modeled in this mode. Partial correlation value can be prescribed by the variable *vt_mistracking_partial_correlation_factor*. The monte-carlo sample size can be specified by the variable *vt_mistracking_monte_carlo_sample_size*. The default sample size is 10,000.

See Also

- set_vt_mistracking_derate
- vt_mistracking_enable_path_analysis
- vt_mistracking_partial_correlation_factor
- vt_mistracking_monte_carlo_sample_size

---

vt_mistracking_enable_bounding_analysis

Enables or disables Vt mistracking bounding analysis.

Feedback

Data Types

Boolean

Default false

Description

When this variable is set to true, the tool can perform Vt mistracking bounding analysis. It supports both graph-based analysis and path-based analysis.

In bounding analysis, Vt mistracking impact is modeled by minimal and maximal derating factors. The two bounding factors can be specified by *set_vt_mistracking_derate* command for each Vt type, and then be consumed during *update_timing* and *get_timing_paths* commands for timing calculation. To generate the bounding slack, the minimal derating factor is applied in capture paths, and the maximal one is applied in launch paths. Hence, it results in the worst-case slack caused by Vt mistracking.

The Vt mistracking bounding analysis can guarantee sign-off safety, but the generate slack is more pessimistic than the actual one. To reduce pessimism, advanced Vt mistracking analysis can be enabled with the variable *vt_mistracking_enable_path_analysis* to generate more accurate Vt mistracking slack.

To run Vt mistracking bounding analysis, PrimeShield license is required.

Examples

An example of how to enable Vt mistracking bounding analysis is shown below.

```
prompt> set_vt_mistracking_derate -min 0.95 -max 1.05 -vt_type svt
 [get_lib_cell */*svt*]
prompt> set_vt_mistracking_derate -min 0.96 -max 1.04 -vt_type lvt
 [get_lib_cell */*lvt*]
prompt> set_vt_mistracking_derate -min 0.97 -max 1.03 -vt_type ulvt
 [get_lib_cell */*ulvt*]

prompt> set vt_mistracking_enable_bounding_analysis true
prompt> update_timing -full

prompt> set vt_paths \\
          [get_timing_paths \\
            -pba_mode path \\
            -path_type full_clock -nworst 10 \\
            -max_paths 2000000 -slack_lesser_than 0.0]
```

See Also

- set_vt_mistracking_derate
- update_timing

Feedback

- get_timing_paths
- vt_mistracking_enable_path_analysis

---

vt_mistracking_enable_path_analysis

Enables or disables path based advanced Vt mistracking analysis.

Data Types

Boolean

Default false

Description

When this variable is set to true, the tool will perform advanced Vt mistracking analysis, which is a single-path based timing analysis and performed as a part of *get_timing_paths* command.

The same Vt class is strongly correlated but different Vt classes are only partially correlated due to fabrication process. The tool effectively models such impact and performs physically realistic analysis on Vt mistracking to produce more accurate timing results than bounding analysis, significantly reducing pessimism.

To run advanced Vt mistracking analysis, two corner derating factors for each Vt type need to be specified with *set_vt_mistracking_derate* command. There are two modes in advanced analysis: one is enumeration mode, and the other is monte-carlo mode; they can be specified by the variable *vt_mistracking_analysis_mode*. Please refer to its man page for more details.

In advanced Vt mistracking analysis, bounding analysis should be always turned on before *update_timing* to guarantee a good coverage of Vt mistracking critical paths.

To run advanced Vt mistracking analysis, PrimeShield license is required.

Examples

An example of how to enable advanced Vt mistracking analysis is shown below.

```
prompt> set_vt_mistracking_derate -min 0.95 -max 1.05 -vt_type svt
 [get_lib_cell */*svt*]
prompt> set_vt_mistracking_derate -min 0.96 -max 1.04 -vt_type lvt
 [get_lib_cell */*lvt*]
prompt> set_vt_mistracking_derate -min 0.97 -max 1.03 -vt_type ulvt
 [get_lib_cell */*ulvt*]

prompt> set vt_mistracking_enable_bounding_analysis true
prompt> update_timing -full
```

Feedback

```
prompt> set vt_mistracking_enable_path_analysis true

prompt> set vt_paths \\
          [get_timing_paths \\
            -pba_mode path \\
            -path_type full_clock -nworst 10 \\
            -max_paths 2000000 -slack_lesser_than 0.0]
```

See Also

- set_vt_mistracking_derate
- update_timing
- get_timing_paths
- vt_mistracking_enable_bounding_analysis
- vt_mistracking_analysis_mode

---

vt_mistracking_monte_carlo_sample_size

Specifies the sample size for monte-carlo (MC) based Vt mistracking analysis.

Data Types

integer

Default 10000

Description

In MC based Vt mistracking analysis, the statistical derating model is built for each Vt class. Vt mistracking derating factors can be randomly sampled from statistical models. Here, the sample size refers to the number of sampling points obtained from statistical models. Vt mistracking analysis will be performed at each sampling point.

In order to make MC-based run converge with high confidence level, a large number of samples is needed. Hence, we choose a large sample size 10,000 as a default value. More runtime is expected with more samples.

See Also

- vt_mistracking_enable_path_analysis
- vt_mistracking_analysis_mode

Feedback

vt_mistracking_partial_correlation_factor

Specifies the correlation value among different Vt classes

Data Types

float

Default 0.0

Description

Different masks are used for different Vt classes during fabrication, which makes strong correlation within one Vt class, but less correlated among different Vt classes. In MC analysis mode, such partial correlation can be modeled, and this variable is used to prescribe the correlation value among different Vt classes.It's allowed to specify any value from 0 to 1.0 for this variable. The default value is 0, which indicates all Vt classes are independent. When a nonzero value is specified, the partial correlation among different Vt classes will be modeled. For example, a value 0.4 means the correlation among any pair of Vt classes is 0.4.

See Also

- vt_mistracking_enable_path_analysis
- vt_mistracking_analysis_mode

vt_skew_analysis_mode

Specifies the advanced Vt skew analysis mode.

Data Types

String

Default enumeration

Description

To specify the analysis mode, set this variable to one of the two values:

- *enumeration* - In this mode, each Vt class has a corner based derating model: min and max derating factors. The Vt skew impact is modeled by the enumerations of two extremes of all Vt classes. If there are N Vt classes, the number of enumerations will be two to the power of N. The path-based timing analysis will be performed for each enumeration by applying the same derating factor to the same Vt class in both launch and capture paths. The worst timing will be selected among all enumerations.

Feedback

- *monte_carlo* - In this mode, each Vt class has a statistical derating model, and an efficient monte-carlo (MC) based approach is used to perform VT skew analysis. Partial correlation among different Vt classes can also be modeled in this mode. Partial correlation value can be prescribed by the variable *vt_skew_partial_correlation_factor*. The monte-carlo sample size can be specified by the variable *vt_skew_monte_carlo_sample_size*. The default sample size is 10,000.

See Also

- set_vt_skew_derate
- vt_skew_enable_path_analysis
- vt_skew_partial_correlation_factor
- vt_skew_monte_carlo_sample_size

---

vt_skew_enable_bounding_analysis

Enables or disables Vt skew bounding analysis.

Data Types

Boolean

Default false

Description

When this variable is set to true, the tool can perform Vt skew bounding analysis. It supports both graph-based analysis and path-based analysis.

In bounding analysis, Vt skew impact is modeled by minimal and maximal derating factors. The two bounding factors can be specified by *set_vt_skew_derate* command for each Vt type, and then be consumed during *update_timing* and *get_timing_paths* commands for timing calculation. To generate the bounding slack, the minimal derating factor is applied in capture paths, and the maximal one is applied in launch paths. Hence, it results in the worst-case slack caused by Vt skew.

The Vt skew bounding analysis can guarantee sign-off safety, but the generate slack is more pessimistic than the actual one. To reduce pessimism, advanced Vt skew analysis can be enabled with the variable *vt_skew_enable_path_analysis* to generate more accurate Vt skew slack.

To run Vt skew bounding analysis, PrimeShield license is required.

Examples

An example of how to enable Vt skew bounding analysis is shown below.

Feedback

```
prompt> set_vt_skew_derate -min 0.95 -max 1.05 -vt_type svt [get_lib_cell
 */*svt*]
prompt> set_vt_skew_derate -min 0.96 -max 1.04 -vt_type lvt [get_lib_cell
 */*lvt*]
prompt> set_vt_skew_derate -min 0.97 -max 1.03 -vt_type ulvt
 [get_lib_cell */*ulvt*]

prompt> set vt_skew_enable_bounding_analysis true
prompt> update_timing -full

prompt> set vt_paths \\
          [get_timing_paths \\
            -pba_mode path \\
            -path_type full_clock -nworst 10 \\
            -max_paths 2000000 -slack_lesser_than 0.0]
```

See Also

- set_vt_skew_derate
- update_timing
- get_timing_paths
- vt_skew_enable_path_analysis

---

vt_skew_enable_path_analysis

Enables or disables path based advanced Vt skew analysis.

Data Types

Boolean

Default false

Description

When this variable is set to true, the tool will perform advanced Vt skew analysis, which is a single-path based timing analysis and performed as a part of *get_timing_paths* command.

The same Vt class is strongly correlated but different Vt classes are only partially correlated due to fabrication process. The tool effectively models such impact and performs physically realistic analysis on Vt skew to produce more accurate timing results than bounding analysis, significantly reducing pessimism.

To run advanced Vt skew analysis, two corner derating factors for each Vt type need to be specified with *set_vt_skew_derate* command. There are two modes in advanced analysis:

Feedback

one is enumeration mode, and the other is monte-carlo mode; they can be specified by the variable *vt_skew_analysis_mode*. Please refer to its man page for more details.

In advanced Vt skew analysis, bounding analysis should be always turned on before *update_timing* to guarantee a good coverage of Vt skew critical paths.

To run advanced Vt skew analysis, PrimeShield license is required.

Examples

An example of how to enable advanced Vt skew analysis is shown below.

```
prompt> set_vt_skew_derate -min 0.95 -max 1.05 -vt_type svt [get_lib_cell
 */*svt*]
prompt> set_vt_skew_derate -min 0.96 -max 1.04 -vt_type lvt [get_lib_cell
 */*lvt*]
prompt> set_vt_skew_derate -min 0.97 -max 1.03 -vt_type ulvt
 [get_lib_cell */*ulvt*]

prompt> set vt_skew_enable_bounding_analysis true
prompt> update_timing -full

prompt> set vt_skew_enable_path_analysis true

prompt> set vt_paths \\
          [get_timing_paths \\
            -pba_mode path \\
            -path_type full_clock -nworst 10 \\
            -max_paths 2000000 -slack_lesser_than 0.0]
```

See Also

- set_vt_skew_derate
- update_timing
- get_timing_paths
- vt_skew_enable_bounding_analysis
- vt_skew_analysis_mode

---

vt_skew_monte_carlo_sample_size

Specifies the sample size for monte-carlo (MC) based Vt skew analysis.

Data Types

integer

Default 10000

Feedback

Description

In MC based Vt skew analysis, the statistical derating model is built for each Vt class. Vt skew derating factors can be randomly sampled from statistical models. Here, the sample size refers to the number of sampling points obtained from statistical models. Vt skew analysis will be performed at each sampling point.

In order to make MC-based run converge with high confidence level, a large number of samples is needed. Hence, we choose a large sample size 10,000 as a default value. More runtime is expected with more samples.

See Also

- vt_skew_enable_path_analysis
- vt_skew_analysis_mode

---

vt_skew_partial_correlation_factor

Specifies the correlation value among different Vt classes

Data Types

float

Default 0.0

Description

Different masks are used for different Vt classes during fabrication, which makes strong correlation within one Vt class, but less correlated among different Vt classes. In MC analysis mode, such partial correlation can be modeled, and this variable is used to prescribe the correlation value among different Vt classes.It's allowed to specify any value from 0 to 1.0 for this variable. The default value is 0, which indicates all Vt classes are independent. When a nonzero value is specified, the partial correlation among different Vt classes will be modeled. For example, a value 0.4 means the correlation among any pair of Vt classes is 0.4.

See Also

- vt_skew_enable_path_analysis
- vt_skew_analysis_mode

Feedback

w

wildcards

Describes supported wildcard characters and ways in which they can be escaped.

Description

The following characters are supported as wildcards:

- Asterisks (*) - Substitute for a string of characters of any length.
- Question marks (?) - Substitute for a single character.

The following commands support wildcard characters:

```
get_cells
get_clocks
get_designs
get_lib_cells
get_lib_pins
get_libs
get_nets
get_pins
get_ports
list_libs
```

In addition to the commands listed, commands that perform an implicit get support wildcard characters.

Escaping Wildcards

Wildcard characters must be escaped using double backslashes (\\\\) to remove their special regular expression meaning. For more information, see the EXAMPLES section.

Escaping the Escape Character (\\\\)

This is similar to the escaping wildcard characters; however, the escaping escape character needs one escape character each to escape the escape character. For more information, see the EXAMPLES section.

Examples

The following examples show how to use wildcard characters.

Feedback

Using Wildcards

The following example gets all nets in the current design that are prefixed by *in* and followed by any two characters:

```
pt_shell> get_nets in??
{"in11", "in21"}
```

The following example gets all cells in the current design that are prefixed by *U* and followed by a string of characters of any length:

```
pt_shell> get_cells U*
{"U1", "U2", "U3", "U4"}
```

Escaping Wildcards

The following examples show how to use escaping wildcard characters.

The following example gets the *test?1* design in the system.

```
pt_shell> get_designs {test\\\\?1}
{"test?1"}
```

The same example can be used in a Tcl-based pt_shell using the *list* Tcl command. For example,

```
pt_shell> get_designs [list {test\\?1}]
{"test?1"}
```

If neither the curly braces nor the *list* command is used in the Tcl-based pt_shell, the syntax is as follows:

```
pt_shell> get_designs test\\\\\\\\?1
{"test?1"}
```

Escaping the Escape Character (\\\\)

The following examples show how to escape an escape character.

The following example gets the test\\1 design in the system.

```
pt_shell> get_designs {test\\\\\\\\1}
{"test\\1"}
```

The same example as above can be used in the Tcl-based pt_shell by using the *list* Tcl command. For example,

```
pt_shell> get_designs [list {test\\\\1}]
{"test\\1"}
```

Feedback

If neither curly braces nor the *list* command is used in the Tcl-based pt_shell, the syntax is as follows:

```
pt_shell> get_designs test\\\\\\\\\\\\\\\\1
{"test\\1"}
```

write_script_include_library_constraints

Controls whether constraints set on library objects are written to script output by the *write_script* and *write_sdc* commands.

Data Types

Boolean

Default true

Description

This variable controls whether constraints set on library objects are written to script output by the *write_script* and *write_sdc* commands. When this variable is set to *true* (the default), the tool writes the constraints that meet both of these conditions:

- Constraints that are attached to objects in libraries in use by the current design
- Constraints that were created with the *set_disable_timing* command

See Also

- set_disable_timing
- write_script
- write_sdc

write_script_output_lumped_net_annotation

Determines whether or not the *write_script* command outputs lumped network annotations.

Data Types

Boolean

Default false

Description

When this variable is set to *false* (the default), the *write_script* command does not output lumped network annotations because they are not valid equivalents of detailed network annotations made by the *read_parasitics* command. The lumped network annotations

Feedback

consist of total capacitance, set by the *set_load* command, and total resistance, set by the *set_resistance* command.

When you set the *write_script_output_lumped_net_annotation* variable to *true*, the *write_script* command outputs lumped network annotations.

If you want intentional *set_resistance* and *set_load* annotations, it is preferred to include them in a separate Tcl script rather than in a Synopsys Design Constraints (SDC) file.

See Also

- read_parasitics
- set_load
- set_resistance
- write_script

y

yield_enable_analysis

This is a synonym for the *ps_enable_analysis* variable.

Data Types

Boolean

Default false

See Also

- ps_enable_analysis
- sim_analyze_path
- get_timing_yield
- report_timing_yield
- report_yield_bottleneck
- report_cell_robustness
- report_voltage_robustness

Feedback

yield_enable_timing_analysis

This is a synonym for the *ps_enable_timing_analysis* variable.

See Also

- ps_enable_timing_analysis
- update_timing

Feedback

2

PrimeTime Suite Attributes

This document describes the attributes supported by the PrimeTime Suite tool.

C

category_node_attributes

Describes the predefined application attributes for category_node objects.

Description

name

Type: string

Returns the name of the category_node object.

object_class

Type: string

Returns the class of the object, which is a constant equal to category_node. You cannot set this attribute.

See Also

- get_attribute
- help_attributes
- list_attributes
- report_attribute
- attributes

category_tree_attributes

Describes the predefined application attributes for category_tree objects.

Feedback

Description

name

Type: string

Returns the name of the category_tree object.

object_class

Type: string

Returns the class of the object, which is a constant equal to category_tree. You cannot set this attribute.

See Also

- get_attribute
- help_attributes
- list_attributes
- report_attribute
- attributes

---

cell_attributes

Describes the predefined application attributes for cell objects.

Description

area

Type: float

Returns the area of the cell. If the cell is hierarchical, this includes net area.

base_name

Type: string

Returns the leaf name of the cell. For example, the base_name of cell U1/U2/U3 is U3.

block_config_name

Type: string

Returns the named session for saved HyperScale block data; applies to hierarchical cells for HyperScale top-level runs.

Feedback

block_config_path

Type: string

Returns the configuration path of the HyperScale block; applies to hierarchical cells for HyperScale top-level runs.

bottleneck_cost

Type: float

Returns the bottleneck cost computed by the report_bottleneck command; requires running this command before querying the attribute.

clock_pin_power

Type: double

Returns the internal power of the cell's clock pins that have the is_clock_pin library attribute set to true. If there are multiple clock pins on the cell, the attribute returns the sum of the internal power on the clock pins. The attribute value does not include power associated with the internal clock pins of the cell.

critical_path_max

Type: string

Returns a structured list that contains the name of the path group, levels of logic of the critical path, length, and setup slack for each path group present at the block level.

critical_path_min

Type: string

Returns a structured list containing the name of the path group, levels of logic of the critical path, length, and hold slack for each path group present at the block level.

disable_timing

Type: boolean

Returns true if the timing for the cell has been marked as disabled in set_disable_timing. You can set and unset the disable_timing attribute.

dont_touch

Type: boolean

Returns true if the cell is protected from change by the set_dont_touch command. When this attribute is set to true, the cell cannot be resized, removed, or replaced by the ECO fixing commands (fix_eco_drc, fix_eco_power, and fix_eco_timing). However, it can still

Feedback

be removed by the remove_cell command. This attribute exists for a cell only after the attribute is created by the set_dont_touch command.

drc_violations

Type: string

Returns a list containing, for each violated DRC check type: the check name, the number of violations, and the violation cost.

dynamic_power

Type: double

Returns the dynamic power of the cell in watts. It is the sum of the internal power and switching power.

early_fall_cell_check_derate_factor

Type: float

Returns the early timing derating factor, set by the set_timing_derate command, that applies to the cell.

early_fall_clk_cell_derate_factor

Type: float

Returns the early timing derating factor, set by the set_timing_derate command, that applies to the cell.

early_fall_clk_net_delta_derate_factor

Type: float

Returns the early timing derating factor, set by the set_timing_derate command, that applies to the cell.

early_fall_clk_net_derate_factor

Type: float

Returns the early timing derating factor, set by the set_timing_derate command, that applies to the cell.

early_fall_data_cell_derate_factor

Type: float

Returns the early timing derating factor, set by the set_timing_derate command, that applies to the cell.

Feedback

early_fall_data_net_delta_derate_factor

Type: float

Returns the early timing derating factor, set by the set_timing_derate command, that applies to the cell.

early_fall_data_net_derate_factor

Type: float

Returns the early timing derating factor, set by the set_timing_derate command, that applies to the cell.

early_rise_cell_check_derate_factor

Type: float

Returns the early timing derating factor, set by the set_timing_derate command, that applies to the cell.

early_rise_clk_cell_derate_factor

Type: float

Returns the early timing derating factor, set by the set_timing_derate command, that applies to the cell.

early_rise_clk_net_delta_derate_factor

Type: float

Returns the early timing derating factor, set by the set_timing_derate command, that applies to the cell.

early_rise_clk_net_derate_factor

Type: float

Returns the early timing derating factor, set by the set_timing_derate command, that applies to the cell.

early_rise_data_cell_derate_factor

Type: float

Returns the early timing derating factor, set by the set_timing_derate command, that applies to the cell.

early_rise_data_net_delta_derate_factor

Type: float

Feedback

Returns the early timing derating factor, set by the set_timing_derate command, that applies to the cell.

early_rise_data_net_derate_factor

Type: float

Returns the early timing derating factor, set by the set_timing_derate command, that applies to the cell.

escaped_full_name

Type: string

Returns the name of the cell. Any literal hierarchy characters are escaped with a backslash.

full_name

Type: string

Returns the complete name of the cell. For example, the full name cell U3 within cell U2 within cell U1 is U1/U2/U3. The full_name attribute is not affected by the current_instance setting.

gate_leakage_power

Type: double

Returns the gate leakage power of the cell in watts. Gate leakage power is the leakage power from the source to the gate or the gate to the drain.

glitch_power

Type: double

Returns the glitch power of the cell in watts. Glitch power is considered part of the dynamic power.

has_multi_ground_rails

Type: boolean

Returns true if the cell has multiple ground rails.

has_multi_power_rails

Type: boolean

Returns true if the cell has multiple power rails.

Feedback

has_rail_specific_power_tables

Type: boolean

Returns true if the cell has power tables attached to rails.

internal_power

Type: double

Returns the internal power of the cell in watts. Internal power is any dynamic power dissipated within the boundary of the cell.

internal_power_derate_factor

Type: float

Returns the power derating factor on the cell. To set this value, use the set_power_derate command.

intrinsic_leakage_power

Type: double

Returns the intrinsic leakage power of the cell in watts. Most intrinsic leakage power results from source-to-drain subthreshold leakage.

is_black_box

Type: boolean

Returns true if the cell's reference is not linked to a library cell or design. This attribute is read-only.

is_case_sequential_propagation

Type: boolean

Returns true if the cell is enabled for sequential case propagation by the set_case_sequential_propagation command.

is_clock_gating_check

Type: boolean

Returns true if the cell is a clock-gating check cell.

is_clock_network_cell

Type: boolean

Returns true if the cell is in the clock network of any clock.

Feedback

is_combinational

Type: boolean

Returns true if the corresponding library cell has no sequential timing arcs. This attribute is not valid for hierarchical and black-box cells. See also the is_hierarchical and is_black_box cell attributes.

is_design_mismatch

Type: boolean

Returns true if the cell has a mismatch between the block and the top-level design.

is_edited

Type: boolean

Returns true if the hierarchical cell has been uniquified as a result of a netlist editing (ECO) change. It is only defined for hierarchical cells.

is_fall_edge_triggered

Type: boolean

Returns true if the cell is used as a falling-edge-triggered flip-flop.

is_hierarchical

Type: boolean

Returns true for hierarchical and black-box cells, and false for leaf cells. Hierarchical cells represent instances of other designs, while leaf cells represent instances of library cells. Black-box cells are considered hierarchical.

is_hyperscale_block

Type: boolean

Returns true if the hierarchical cell is a HyperScale block.

is_ideal

Type: boolean

Returns true if the cell has been marked ideal using the set_ideal_network command.

is_integrated_clock_gating_cell

Type: boolean

Returns true if the cell is defined in the library as an integrated clock-gating cell.

Feedback

is_interface_logic_model

Type: boolean

Returns true if the cell is an interface logic model. PrimeTime automatically sets this attribute to true when you create an interface logic model with the create_ilm command. The attribute allows the check_timing command to identify interface logic models and suppress false reporting of unconnected clock pins.

is_macro_switch

Type: boolean

Returns true if the reference library cell is defined as a fine-grain switch cell in the library.

is_memory_cell

Type: boolean

Returns true if the cell is a memory cell. This attribute is inherited from the reference library cell. It is used by PrimePower to identify memory cells when reporting power or activity annotation for different cell types, such as sequential, combinational, and memory cell types.

is_mim_context_reference

Type: boolean

Returns true if the cell is used as the reference instance in the HyperScale multiply instantiated model (MIM) merging flow. It is used (1) to query the MIMs to determine the automatically chosen reference instance and (2) to provide confirmation of a user-specified reference instance.

is_mux

Type: boolean

Returns true if the cell is a multiplexer.

is_negative_level_sensitive

Type: boolean

Returns true if the cell is used as a negative level-sensitive latch.

is_pad_cell

Type: boolean

Returns true if the cell is a pad cell.

Feedback

is_positive_level_sensitive

Type: boolean

Returns true if the cell is used as a positive level-sensitive latch.

is_power_standby_cell

Type: boolean

Returns true if the cell is a power standby cell.

is_rise_edge_triggered

Type: boolean

Returns true if the cell is used as a rising-edge-triggered flip-flop.

is_sequential

Type: boolean

Returns true if the corresponding library cell has at least one sequential timing arc. This attribute is not valid for hierarchical and black-box cells. See also the is_hierarchical and is_black_box cell attributes.

is_three_state

Type: boolean

Returns true if the cell is a three-state device.

late_fall_cell_check_derate_factor

Type: float

Returns the late timing derating factor, set by the set_timing_derate command, that applies to the cell.

late_fall_clk_cell_derate_factor

Type: float

Returns the late timing derating factor, set by the set_timing_derate command, that applies to the cell.

late_fall_clk_net_delta_derate_factor

Type: float

Returns the late timing derating factor, set by the set_timing_derate command, that applies to the cell.

Feedback

late_fall_clk_net_derate_factor

Type: float

Returns the late timing derating factor, set by the set_timing_derate command, that applies to the cell.

late_fall_data_cell_derate_factor

Type: float

Returns the late timing derating factor, set by the set_timing_derate command, that applies to the cell.

late_fall_data_net_delta_derate_factor

Type: float

Returns the late timing derating factor, set by the set_timing_derate command, that applies to the cell.

late_fall_data_net_derate_factor

Type: float

Returns the late timing derating factor, set by the set_timing_derate command, that applies to the cell.

late_rise_cell_check_derate_factor

Type: float

Returns the late timing derating factor, set by the set_timing_derate command, that applies to the cell.

late_rise_clk_cell_derate_factor

Type: float

Returns the late timing derating factor, set by the set_timing_derate command, that applies to the cell.

late_rise_clk_net_delta_derate_factor

Type: float

Returns the late timing derating factor, set by the set_timing_derate command, that applies to the cell.

late_rise_clk_net_derate_factor

Type: float

Feedback

Returns the late timing derating factor, set by the set_timing_derate command, that applies to the cell.

late_rise_data_cell_derate_factor

Type: float

Returns the late timing derating factor, set by the set_timing_derate command, that applies to the cell.

late_rise_data_net_delta_derate_factor

Type: float

Returns the late timing derating factor, set by the set_timing_derate command, that applies to the cell.

late_rise_data_net_derate_factor

Type: float

Returns the late timing derating factor, set by the set_timing_derate command, that applies to the cell.

leaf_cell_count

Type: integer

Returns the cell count in the hierarchical block that was used to create the abstracted hierarchical instance.

leakage_power

Type: double

Returns the leakage power of the cell in watts. It is the power that the cell dissipates when it is not switching. Leakage power is the sum of the intrinsic leakage and gate leakage.

leakage_power_derate_factor

Type: float

Returns the power derating factors, specified using the set_power_derate command, that apply to the cell.

lib_cell

Type: collection

Returns a collection of library cells for this cell; this attribute is defined only on leaf cells.

Feedback

number_of_pins

Type: integer

Returns the number of pins on the cell. The number of pins can be different before and after linking. For example, if some pins were unconnected in a Verilog instance, after linking to the lower-level design, additional pins can be created on the cell.

object_class

Type: string

Returns the class of the object, which is "cell". You cannot set this attribute.

original_ref_name

Type: string

Returns the original reference name of hierarchical blocks within the HyperScale model. This attribute applies to hierarchical cells for HyperScale top-level runs. This attribute is needed to apply constraints and LEF files to non-HyperScale subdesigns within HyperScale models at the top level.

parasitics_corner_name

Type: string

Returns the "*// CORNER_NAME*" information from the SPEF or GPD detailed parasitics applied to the hierarchical cell.

parasitics_operating_temperature

Type: float

Returns the "*// OPERATING_TEMPERATURE*" information from the SPEF or GPD detailed parasitics applied to the hierarchical cell.

parasitics_program_name

Type: string

Returns the "**PROGRAM*" information from the SPEF or GPD detailed parasitics applied to the hierarchical cell.

parasitics_program_version

Type: string

Returns the "**VERSION*" information from the SPEF or GPD detailed parasitics applied to the hierarchical cell.

Feedback

parasitics_tech_file

Type: string

Returns the "*// TCAD_GRD_FILE*" information from the SPEF detailed parasitics applied to the hierarchical cell.

parent_cell

Type: collection

Returns the parent hierarchical cell of the cell.

peak_power

Type: double

Returns the peak power of the cell in watts. This is the largest power value for that cell in the simulation waveform.

peak_power_end_time

Type: double

Returns the end time of the time interval in which peak power is measured for that cell. The unit of measurement is nanoseconds.

peak_power_start_time

Type: double

Returns the start time of the time interval in which peak power is measured for that cell. The unit of measurement is nanoseconds.

pg_pin_info

Type: collection

Returns a collection of these pg_pin_info objects: pin_name, type, voltage_for_max_delay, voltage_for_min_delay, supply_connection.

pin_count

Type: integer

Returns the pin count in the hierarchical block that was used to create the abstracted hierarchical instance.

power_cell_type

Type: string

Feedback

Returns the predefined power group to which the cell belongs. Use the set_user_attribute command to set the attribute to one of these predefined power groups: clock_network, register, combinational, sequential, memory, io_pad, or black_box. You cannot use this attribute to assign a cell to a user-defined power group. The power_cell_type cell attribute overrides the power_cell_type library cell attribute. PrimePower uses this attribute in both averaged and time-based power analysis modes.

power_states

Type: double

Returns the number of power state changes for the cell. This is the sum of the number of transitions on all input and output pins of the cell.

ref_name

Type: string

Returns the name of the design or library cell of which the cell is (or will be) an instantiation; also known as the reference name. The linker looks for a design or library cell by this name to resolve the reference.

size_only

Type: boolean

Returns true if the cell is protected from removal by the set_size_only command. When this attribute is set to true, the ECO fixing commands (fix_eco_drc, fix_eco_power, and fix_eco_timing) can resize the cell but not remove it. The cell can still be removed by the remove_cell command.

switching_power

Type: double

Returns the switching power of the cell in watts. It is the power dissipated by the charging and discharging of the load capacitance at the output of the cell.

switching_power_derate_factor

Type: float

Returns the power derating factor, specified by the set_power_derate command, that applies to the cell.

temperature_max

Type: float

Returns the maximum (worst) case temperature for the cell specified by the operating condition or the set_temperature command.

Feedback

temperature_min

Type: float

Returns the minimum (best) case temperature for the cell specified by the operating condition or the set_temperature command.

timing_model_type

Type: string

Returns the timing model type of the cell. The possible values are default, abstracted, extracted, and qtm.

total_power

Type: double

Returns the total power of the cell in watts. It is the sum of dynamic power and leakage power.

upf_isolation_strategy

Type: string

Returns the strategy name (created by the set_isolation command) that was used to derive rail supply and ground nets of the cell.

upf_retention_strategy

Type: string

Returns the strategy name (created by the set_retention command) that was used to derive backup rail supply and ground nets.

wire_load_model_max

Type: string

Returns the name of the wire load model effective on a hierarchical cell for the maximum operating condition. You can set this attribute.

wire_load_model_min

Type: string

Returns the name of the wire load model effective on a hierarchical cell for the minimum operating condition (valid in on-chip variation analysis). You can set this attribute.

wire_load_selection_group_max

Type: string

Feedback

Returns the name of the wire load selection group on a hierarchical cell for the maximum operating condition. You can set this attribute.

wire_load_selection_group_min

Type: string

Returns the name of the wire load selection group on a hierarchical cell for the minimum operating condition. You can set this attribute.

x_coordinate_max

Type: float

Returns the maximum x-coordinate of the bounding box of the pin locations of the cell. Note that this bounding box can be smaller than the cell boundary.

x_coordinate_min

Type: float

Returns the minimum x-coordinate of the bounding box of the pin locations of the cell. Note that this bounding box can be smaller than the cell boundary.

x_transition_power

Type: double

Returns the dynamic power used by the cell during transitions from the unknown (X) state to a known state (0 or 1).

y_coordinate_max

Type: float

Returns the maximum y-coordinate of the bounding box of the pin locations of the cell. Note that this bounding box can be smaller than the cell boundary.

y_coordinate_min

Type: float

Returns the minimum y-coordinate of the bounding box of the pin locations of the cell. Note that this bounding box can be smaller than the cell boundary.

See Also

- get_attribute
- help_attributes
- list_attributes

Feedback

- report_attribute
- attributes

---

clock_attributes

Describes the predefined application attributes for clock objects.

Description

clock_latency_fall_max

Type: float

Returns the maximum fall latency (insertion delay) set by the set_clock_latency command.

clock_latency_fall_min

Type: float

Returns the minimum fall latency (insertion delay) set by the set_clock_latency command.

clock_latency_rise_max

Type: float

Returns the maximum rise latency (insertion delay) set by the set_clock_latency command.

clock_latency_rise_min

Type: float

Returns the minimum rise latency (insertion delay) set by the set_clock_latency command.

clock_network_pins

Type: collection

Returns a collection of pin and port objects in the propagation path of the clock.

clock_source_latency_early_fall_max

Type: float

Returns the maximum early falling source latency set by the set_clock_latency command.

clock_source_latency_early_fall_min

Type: float

Returns the minimum early falling source latency set by the set_clock_latency command.

Feedback

clock_source_latency_early_rise_max

Type: float

Returns the maximum early rising source latency set by the set_clock_latency command.

clock_source_latency_early_rise_min

Type: float

Returns the minimum early rising source latency set by the set_clock_latency command.

clock_source_latency_late_fall_max

Type: float

Returns the maximum late falling source latency set by the set_clock_latency command.

clock_source_latency_late_fall_min

Type: float

Returns the minimum late falling source latency set by the set_clock_latency command.

clock_source_latency_late_rise_max

Type: float

Returns the maximum late rising source latency set by the set_clock_latency command.

clock_source_latency_late_rise_min

Type: float

Returns the minimum late rising source latency set by the set_clock_latency command.

clock_source_latency_pins

Type: collection

Returns a collection of pins and ports in the source latency network of a generated clock. This attribute is undefined for master clocks.

full_name

Type: string

Returns the name of the clock. This is set with the create_clock command. It is either the name given with the -name option, or the name of the first object to which the clock is attached. After the name is set, this attribute is read-only.

generated_clocks

Type: collection

Feedback

Returns a collection of generated clock objects for which the clock is the source. This attribute is defined for any clock that is the parent source of one or more generated clocks.

hold_uncertainty

Type: float

Returns the clock uncertainty (skew) of the clock used for hold (and other minimum delay) timing checks, as set by the set_clock_uncertainty command.

is_active

Type: boolean

Returns true if the clock is active, the default state. To set clocks as active or inactive, use the set_active_clocks command.

is_generated

Type: boolean

Returns true for a generated clock. To create a generated clock, use the create_generated_clock command.

master_clock

Type: collection

Returns the master clock of a generated clock. This attribute is defined only for generated clocks.

master_pin

Type: collection

Returns the master source pin or port object used to determine the identity and polarity of the master clock. This corresponds to the pin or port specified by the -source option of the create_generated_clock command. This attribute is defined for generated clocks only.

max_capacitance_clock_path_fall

Type: float

Returns the upper limit for the falling maximum capacitance for all pins in the clock path, as set by the set_max_capacitance command.

max_capacitance_clock_path_rise

Type: float

Returns the upper limit for the rising maximum capacitance for all pins in the clock path, as set by the set_max_capacitance command.

Feedback

max_capacitance_data_path_fall

Type: float

Returns the upper limit for the falling maximum capacitance for all pins in the data path launched by the clock, as set by the set_max_capacitance command.

max_capacitance_data_path_rise

Type: float

Returns the upper limit for the rising maximum capacitance for all pins in the data path launched by the clock, as set by the set_max_capacitance command.

max_fall_delay

Type: float

Returns the maximum falling delay on ports, clocks, pins, cells, or on paths between such objects, as set by the set_max_delay command.

max_rise_delay

Type: float

Returns the maximum rising delay on ports, clocks, pins, cells, or on paths between such objects, as set by the set_max_delay command.

max_time_borrow

Type: float

Returns the upper limit for time borrowing; that is, it prevents the use of the entire pulse width for level-sensitive latches, as set by the set_max_time_borrow command. Units are those of the logic library.

max_transition_clock_path_fall

Type: float

Returns the upper limit for the falling maximum transition for all pins in the clock path, as set by the set_max_transition command.

max_transition_clock_path_rise

Type: float

Returns the upper limit for the rising maximum transition for all pins in the clock path, as set by the set_max_transition command.

max_transition_data_path_fall

Type: float

Feedback

Returns the upper limit for the falling maximum transition for all pins in the data path launched by the clock, as set by the set_max_transition command.

max_transition_data_path_rise

Type: float

Returns the upper limit for the rising maximum transition for all pins in the data path launched by the clock, as set by the set_max_transition command.

min_fall_delay

Type: float

Returns the minimum falling delay on ports, clocks, pins, cells, or paths between such objects, as set by the set_min_delay command.

min_rise_delay

Type: float

Returns the minimum rising delay on ports, clocks, pins, cells, or paths between such objects, as set by the set_min_delay command.

object_class

Type: string

Returns the class of the object, which is "clock". You cannot set this attribute.

period

Type: float

Returns the clock period (or cycle time), which is the shortest time during which the clock waveform repeats. For a simple waveform with one rising and one falling edge, the period is the difference between successive rising edges. It is set by the create_clock -period command.

propagated_clock

Type: boolean

Returns true if clock latency (insertion delay) is determined by propagating delays from the clock source to destination register clock pins, as set by the set_propagated_clock command. If this attribute is not present, ideal clocking is assumed.

setup_uncertainty

Type: float

Feedback

Returns the clock uncertainty (skew) of the clock used for setup (and other maximum delay) timing checks, as set by the set_clock_uncertainty command.

sources

Type: collection

Returns a collection of the source pins or ports of the clock, as defined by the create_clock command.

waveform

Type: string

Returns a string representation of the clock waveform, as defined by the create_clock command. For example, a clock rising at 2.5 and falling at 5.0 has a waveform attribute value of {2.5 5.0}.

See Also

- get_attribute
- help_attributes
- list_attributes
- report_attribute
- attributes

---

correlation_attributes

Describes the predefined application attributes for correlation objects.

Description

full_name

Type: string

Returns the full name of the correlation object.

object_class

Type: string

Returns the class of the object, which is a constant equal to correlation. You cannot set this attribute.

Feedback

See Also

- get_attribute
- help_attributes
- list_attributes
- report_attribute
- attributes

---

coupling_capacitor_attributes

Describes the predefined application attributes for coupling_capacitor objects.

Description

aggressor_layer_id

Type: integer

Returns the ITF (nxtgrd file) layer ID.

aggressor_layer_name

Type: string

Returns the ITF (nxtgrd file) layer name.

aggressor_net

Type: collection

Returns the aggressor net associated with the coupling capacitor.

aggressor_net_name

Type: string

Returns the aggressor node name in the format used in a SPEF output file.

aggressor_node_ground_capacitor

Type: collection

Returns the ground capacitor associated with the aggressor node of the coupling capacitor.

aggressor_node_index

Type: integer

Feedback

Returns the index value of the node at which the coupling capacitor connects to the net. Each node on a net has a unique index from 1 to N, where N is the total number of nodes on that net.

aggressor_node_name

Type: string

Returns the aggressor node name in the format used in a SPEF output file.

capacitance

Type: string

Returns a single-corner capacitance value in the format used in a SPEF output file.

capacitance_max

Type: string

Returns the maximum value of the list in the capacitance_multicorner attribute.

capacitance_min

Type: string

Returns the minimum value of the list in the capacitance_multicorner attribute.

capacitance_multicorner

Type: string

If data from multiple corners is retrieved, the attribute contains a list of the capacitances of the corners specified by the *-parasitic_corners* option, in that order.

layer_id

Type: integer

Returns the ITF (nxtgrd file) layer ID.

layer_name

Type: string

Returns the ITF (nxtgrd file) layer name.

net

Type: collection

Returns the victim net associated with the coupling capacitor.

Feedback

node_ground_capacitor

Type: collection

Returns the ground capacitor associated with the victim node of the coupling capacitor.

node_index

Type: integer

Returns the index value of the ground capacitor. Each ground capacitor on a net has a unique index from 1 to N, where N is the total number of nodes on that net.

node_name

Type: string

Returns the node name in the format used in a SPEF output file.

See Also

- get_attribute
- help_attributes
- list_attributes
- report_attribute
- attributes

d

design_attributes

Describes the predefined application attributes for design objects.

Description

analysis_type

Type: string

Returns the analysis type, either single or on_chip_variation.

area

Type: float

Returns the total area of the design. This is the sum of the areas of all leaf cells and nets.

Feedback

capacitance_unit_in_farad

Type: float

Returns the unit of capacitance in the main library in farads. This attribute is read-only.

config_name

Type: string

Returns the name of the session saved for the current HyperScale run.

config_path

Type: string

Returns the HyperScale configuration path of the current HyperScale run.

context_config_name

Type: string

Returns the named session of HyperScale top context applied in HyperScale block-level runs.

context_config_path

Type: string

Returns the configuration path of HyperScale top context applied in HyperScale block-level runs.

context_timestamp

Type: string

Returns the timestamp of the HyperScale top context applied in HyperScale block-level runs.

current_unit_in_amp

Type: float

Returns the unit of capacitance in the main library in amps. This attribute is read-only.

designWare

Type: boolean

Returns true for a DesignWare design.

dont_touch

Type: boolean

Feedback

Returns true if the design is excluded from changes. Values are undefined by default. Instantiations of designs with the dont_touch attribute set to true are not modified or replaced by ECO fixing commands (fix_eco_drc, fix_eco_power, and fix_eco_timing). This attribute is set by the set_dont_touch command and recognized by the ECO fixing commands (fix_eco_drc, fix_eco_power, and fix_eco_timing).

dynamic_power

Type: double

Returns the dynamic power of the design in watts. It is the sum of the dynamic power of all the cells of the design.

early_fall_cell_check_derate_factor

Type: float

Returns an early timing derating factor, specified by the set_timing_derate command, that applies to the design.

early_fall_clk_cell_derate_factor

Type: float

Returns an early timing derating factor, specified using the set_timing_derate command, that applies to the design.

early_fall_clk_net_delta_derate_factor

Type: float

Returns an early timing derating factor, specified using the set_timing_derate command, that applies to the design.

early_fall_clk_net_derate_factor

Type: float

Returns an early timing derating factor, specified using the set_timing_derate command, that applies to the design.

early_fall_data_cell_derate_factor

Type: float

Returns an early timing derating factor, specified using the set_timing_derate command, that applies to the design.

early_fall_data_net_delta_derate_factor

Type: float

Feedback

Returns an early timing derating factor, specified using the set_timing_derate command, that applies to the design.

early_fall_data_net_derate_factor

Type: float

Returns an early timing derating factor, specified using the set_timing_derate command, that applies to the design.

early_rise_cell_check_derate_factor

Type: float

Returns an early timing derating factor, specified using the set_timing_derate command, that applies to the design.

early_rise_clk_cell_derate_factor

Type: float

Returns an early timing derating factor, specified using the set_timing_derate command, that applies to the design.

early_rise_clk_net_delta_derate_factor

Type: float

Returns an early timing derating factor, specified using the set_timing_derate command, that applies to the design.

early_rise_clk_net_derate_factor

Type: float

Returns an early timing derating factor, specified using the set_timing_derate command, that applies to the design.

early_rise_data_cell_derate_factor

Type: float

Returns an early timing derating factor, specified using the set_timing_derate command, that applies to the design.

early_rise_data_net_delta_derate_factor

Type: float

Returns an early timing derating factor, specified using the set_timing_derate command, that applies to the design.

Feedback

early_rise_data_net_derate_factor

Type: float

Returns an early timing derating factor, specified using the set_timing_derate command, that applies to the design.

enable_bias

Type: boolean

Enables the UPF well bias mode when true. Set this attribute in the UPF command file with the following command:

```
set_design_attributes -elements {.} -attribute enable_bias true
```

extended_name

Type: string

Returns the complete, unambiguous name of the design. The extended_name of the design is the source_file_name attribute followed by a colon (:) followed by the full_name attribute. For example, the extended_name of design TOP read in from /u/user/simple.db is /u/user/simple.db:TOP.

full_name

Type: string

Returns the name of the design. For example, the full_name of design TOP read in from /abc/xyz/simple.db is TOP. This name can be ambiguous because several designs of the same name can be read in from different files.

gate_leakage_power

Type: double

Returns the gate leakage power of the design in watts. It is the sum of the gate leakage of all the cells of the design.

glitch_power

Type: double

Returns the glitch power of the design in watts. It is the sum of the glitch power of all the cells of the design.

internal_power

Type: double

Feedback

Returns the internal power of the design in watts. It is the sum of the internal power of all the cells of the design.

internal_power_derate_factor

Type: float

Returns the internal power derating factor specified by the set_power_derate command that applies to the design.

intrinsic_leakage_power

Type: double

Returns the intrinsic leakage power of the design in watts. It is the sum of the intrinsic leakage of all the cells of the design.

is_context_available

Type: boolean

Returns true if HyperScale top context is available to be applied. This attribute is applicable only for HyperScale block runs. The attribute returns true after link and after update_timing only if top-level context exists for each configuration.

is_context_loaded

Type: boolean

Returns true if HyperScale top context has been loaded and applied during update_timing. This attribute is applicable only for HyperScale block runs. The attribute returns true after update_timing only if top-level context data was loaded during update_timing, even if the context was not fully applied because of override controls. The attribute is never true before update_timing.

is_current

Type: boolean

Returns true for the current design. This attribute changes for all designs when you use the current_design command.

is_edited

Type: boolean

Returns true if the design has been uniquified as a result of a netlist editing (ECO) change.

is_hyperscale_block

Type: boolean

Feedback

Returns true if the current design is a HyperScale block.

is_hyperscale_top

Type: boolean

Returns true if the current design is a HyperScale top-level design. Mid-level HyperScale runs are considered to be both top and block.

late_fall_cell_check_derate_factor

Type: float

Returns a late timing derating factor, specified using the set_timing_derate command, that applies to the design.

late_fall_clk_cell_derate_factor

Type: float

Returns a late timing derating factor, specified using the set_timing_derate command, that applies to the design.

late_fall_clk_net_delta_derate_factor

Type: float

Returns a late timing derating factor, specified using the set_timing_derate command, that applies to the design.

late_fall_clk_net_derate_factor

Type: float

Returns a late timing derating factor, specified using the set_timing_derate command, that applies to the design.

late_fall_data_cell_derate_factor

Type: float

Returns a late timing derating factor, specified using the set_timing_derate command, that applies to the design.

late_fall_data_net_delta_derate_factor

Type: float

Returns a late timing derating factor, specified using the set_timing_derate command, that applies to the design.

Feedback

late_fall_data_net_derate_factor

Type: float

Returns a late timing derating factor, specified using the set_timing_derate command, that applies to the design.

late_rise_cell_check_derate_factor

Type: float

Returns a late timing derating factor, specified using the set_timing_derate command, that applies to the design.

late_rise_clk_cell_derate_factor

Type: float

Returns a late timing derating factor, specified using the set_timing_derate command, that applies to the design.

late_rise_clk_net_delta_derate_factor

Type: float

Returns a late timing derating factor, specified using the set_timing_derate command, that applies to the design.

late_rise_clk_net_derate_factor

Type: float

Returns a late timing derating factor, specified using the set_timing_derate command, that applies to the design.

late_rise_data_cell_derate_factor

Type: float

Returns a late timing derating factor, specified using the set_timing_derate command, that applies to the design.

late_rise_data_net_delta_derate_factor

Type: float

Returns a late timing derating factor, specified using the set_timing_derate command, that applies to the design.

late_rise_data_net_derate_factor

Type: float

Feedback

Returns a late timing derating factor, specified using the set_timing_derate command, that applies to the design.

leakage_power

Type: double

Returns the leakage power of the design in watts. It is the sum of the leakage power of all the cells of the design.

leakage_power_derate_factor

Type: float

Returns the leakage power derating factor specified by the set_power_derate command that applies to the design.

lower_domain_boundary

Type: boolean

Adds a lower-domain boundary in the boundary definition. You can set this attribute at any level of the hierarchy. To include lower-domain boundaries in the current scope, use the following command:

```
set_design_attributes -elements {.} -attribute lower_domain_boundary true
```

max_area

Type: float

Returns a floating-point number that represents the target area of the design. The units must be consistent with the units used from the logic library during optimization. The max_area value is set using the set_max_area command.

max_capacitance

Type: float

Returns the default maximum capacitance design rule limit for the design. The units must be consistent with those of the logic library used during optimization. It is set with the set_max_capacitance command.

max_fanout

Type: float

Returns the default maximum fanout design rule limit for the design. The units must be consistent with those of the logic library used during optimization. It is set with the set_max_fanout command.

Feedback

max_transition

Type: float

Returns the default maximum transition design rule limit for the design. The units must be consistent with those of the logic library used during optimization. It is set with the set_max_transition command.

min_capacitance

Type: float

Returns the default minimum capacitance design rule limit for the design. The units must be consistent with those of the logic library used during optimization. It is set with the set_min_capacitance command.

min_fanout

Type: float

Returns the default minimum fanout design rule limit for the design. The units must be consistent with those of the logic library used during optimization.

min_transition

Type: float

Returns the default minimum transition design rule limit for the design. The units must be consistent with those of the logic library used during optimization.

object_class

Type: string

Returns the class of the object, which is "design". You cannot set this attribute.

operating_condition_max

Type: string

Returns the name of the maximum or single operating condition for the design. It is set with the set_operating_conditions command.

operating_condition_min

Type: string

Returns the name of the minimum operating condition for the design. This attribute is not valid in single operating condition analysis.

parasitics_corner_name

Type: string

Feedback

Returns the "*// CORNER_NAME*" information from the SPEF or GPD detailed parasitics applied to the design.

parasitics_operating_temperature

Type: float

Returns the "*// OPERATING_TEMPERATURE*" information from the SPEF or GPD detailed parasitics applied to the design.

parasitics_program_name

Type: string

Returns the "**PROGRAM*" information from the SPEF or GPD detailed parasitics applied to the design.

parasitics_program_version

Type: string

Returns the "**VERSION*" information from the SPEF or GPD detailed parasitics applied to the design.

parasitics_tech_file

Type: string

Returns the "*// TCAD_GRD_FILE*" information from the SPEF detailed parasitics applied to the design.

peak_power

Type: double

Returns the peak power of the design in watts. Note that the peak power of the design is not the sum of the peak power of the cells of the design because the cell peaks can occur at different times.

peak_power_end_time

Type: double

Returns the end time of the time interval in which peak power is measured for the design. The unit is nanoseconds (ns).

peak_power_start_time

Type: double

Returns the start time of the time interval in which peak power is measured for the design. The unit is nanoseconds (ns).

Feedback

power_simulation_time

Type: float

Returns the total simulation time in averaged mode, in nanoseconds (ns).

power_states

Type: double

Returns the sum of the power_states of all the cells of the design.

process_max

Type: float

Returns the process value of the maximum or single operating condition for the design. The operating condition is defined in a library or by the create_operating_conditions command. To associate operating conditions with the design, use the set_operating_conditions command.

process_min

Type: float

Returns the process value of the minimum operating condition for the design. The operating condition is defined in a library or by the create_operating_conditions command. To associate operating conditions with the design, use the set_operating_conditions command.

rc_input_threshold_pct_fall

Type: float

Returns the characterization trip point (waveform measurement threshold) factor that the tool uses to calculate delays and transition times. Returns the value obtained from the main library (the first library in the link path).

rc_input_threshold_pct_rise

Type: float

Returns the characterization trip point (waveform measurement threshold) factor that the tool uses to calculate delays and transition times. Returns the value obtained from the main library (the first library in the link path).

rc_output_threshold_pct_fall

Type: float

Feedback

Returns the characterization trip point (waveform measurement threshold) factor that the tool uses to calculate delays and transition times. Returns the value obtained from the main library (the first library in the link path).

rc_output_threshold_pct_rise

Type: float

Returns the characterization trip point (waveform measurement threshold) factor that the tool uses to calculate delays and transition times. Specifies the value obtained from the main library (the first library in the link path).

rc_slew_derate_from_library

Type: float

Returns the slew derating factor that the tool uses to calculate delays and transition times. Specifies the value obtained from the main library (the first library in the link path).

rc_slew_lower_threshold_pct_fall

Type: float

Returns the characterization trip point (waveform measurement threshold) factor that the tool uses to calculate delays and transition times. Specifies the value obtained from the main library (the first library in the link path).

rc_slew_lower_threshold_pct_rise

Type: float

Returns the characterization trip point (waveform measurement threshold) factor that the tool uses to calculate delays and transition times. Specifies the value obtained from the main library (the first library in the link path).

rc_slew_upper_threshold_pct_fall

Type: float

Returns the characterization trip point (waveform measurement threshold) factor that the tool uses to calculate delays and transition times. Specifies the value obtained from the main library (the first library in the link path).

rc_slew_upper_threshold_pct_rise

Type: float

Returns the characterization trip point (waveform measurement threshold) factor that the tool uses to calculate delays and transition times. Specifies the value obtained from the main library (the first library in the link path).

Feedback

resistance_unit_in_ohm

Type: float

Returns the unit of resistance in the main library in Ohms. This attribute is read-only.

source_file_name

Type: string

Returns the name of the file from which the design was read. For example, the source_file_name of design TOP read in from /abc/xyz/simple.db is /abc/xyz/simple.db.

switching_power

Type: double

Returns the switching power of the design in watts. It is the sum of the switching power of all the cells of the design.

switching_power_derate_factor

Type: float

Returns the switching power derating factor specified by the set_power_derate command that applies to the design.

temperature_max

Type: float

Returns the ambient temperature value of the maximum or single operating condition for the design. The operating condition is defined in a library or by the create_operating_conditions command. To associate operating conditions with a design, use the set_operating_conditions command.

temperature_min

Type: float

Returns the ambient temperature value of the minimum operating condition for the design. The operating condition is defined in a library or by the create_operating_conditions command. To associate operating conditions with a design, use the set_operating_conditions command.

time_unit_in_second

Type: float

Returns the unit of time in the main library in seconds. This attribute is read-only.

Feedback

timestamp

Type: string

Returns the timestamp for the current HyperScale run. This attribute applies to top and block designs.

total_power

Type: double

Returns the total power of the design in watts. It is the sum of the total power of all the cells of the design.

tree_type_max

Type: string

Returns the tree_type value of the maximum or single operating condition for the design. The operating condition is defined in a library or by the create_operating_conditions command. You associate operating conditions with a design using the set_operating_conditions command. The tree_type value is used in prelayout interconnect delay estimation, and can have a value of best_case, balanced_case, balanced_resistance (cmos2 only), or worst_case.

tree_type_min

Type: string

Returns the tree_type value of the minimum operating condition for the design. The operating condition is defined in a library or by the create_operating_conditions command. You associate operating conditions with a design using the set_operating_conditions command. The tree_type value is used in prelayout interconnect delay estimation, and can have a value of best_case, balanced_case, balanced_resistance (cmos2 only), or worst_case.

violating_endpoints_max

Type: collection

Returns the timing path endpoints that have maximum delay violations, sorted in order of increasing slack.

violating_endpoints_min

Type: collection

Returns the timing path endpoints that have minimum delay violations, sorted in order of increasing slack.

Feedback

voltage_max

Type: float

Returns the voltage value of the maximum or single operating condition for the design. The operating condition is defined in a library or by the create_operating_conditions command. You associate operating conditions with a design using the set_operating_conditions command. This attribute represents the supply voltage value for the operating condition.

voltage_min

Type: float

Returns the voltage value of the minimum operating condition for the design. The operating condition is defined in a library or by the create_operating_conditions command. You associate operating conditions with a design using the set_operating_conditions command. This attribute represents the supply voltage value for the operating condition.

voltage_unit_in_volt

Type: float

Returns the unit of voltage in the main library in volts. This attribute is read-only.

wire_load_min_block_size

Type: float

Returns the smallest hierarchical cell that has automatic wire load selection by area applied. If an automatic wire load selection group is specified as the default in the main library, or through the set_wire_load_selection_group command, it is applied to all hierarchical cells larger than the specified minimum block size.

wire_load_mode

Type: string

Returns the wire load model used to compute wire capacitance, resistance, and area for nets in a hierarchical design that has different wire load models at different hierarchical levels. Allowed values:

- top (default) - Uses the wire load model at the top hierarchical level.
- enclosed - Uses the wire load model on the smallest design that encloses a net completely.
- segmented - Breaks the net into segments, one within each hierarchical level. In the segmented mode, each net segment is estimated using the wire load model on the design that encloses that segment. The segmented mode is not supported for wire load models on clusters.

Feedback

If no value is specified for this attribute, PrimeTime searches for a default in the first library in the link path. It is set with the set_wire_load_model command.

wire_load_model_max

Type: string

Returns the name of the design's wire load model for maximum conditions. It is set with set_wire_load_model.

wire_load_model_min

Type: string

Returns the name of the design's wire load model for minimum conditions. This attribute is not valid for single operating condition analysis. It is set with set_wire_load_model.

wire_load_selection_group_max

Type: string

Returns the name of the design's wire load selection group for maximum conditions. It is set with set_wire_load_selection_group.

wire_load_selection_group_min

Type: string

Returns the name of the design's wire load selection group for minimum conditions. It is set with set_wire_load_selection_group.

x_coordinate_max

Type: float

Returns the maximum x-coordinate of the area occupied by the design.

x_coordinate_min

Type: float

Returns the minimum x-coordinate of the area occupied by the design.

x_transition_power

Type: double

Returns the dynamic power used during transitions from the unknown (X) state to a known state (0 or 1).

y_coordinate_max

Type: float

Feedback

Returns the maximum y-coordinate of the area occupied by the design.

y_coordinate_min

Type: float

Returns the minimum y-coordinate of the area occupied by the design.

See Also

- get_attribute
- help_attributes
- list_attributes
- report_attribute
- attributes

g

ground_capacitor_attributes

Describes the predefined application attributes for ground_capacitor objects.

Description

capacitance

Type: string

Returns a single-corner capacitance value in the format used in a SPEF output file.

capacitance_max

Type: string

Returns the maximum value of the list in the capacitance_multicorner attribute.

capacitance_min

Type: string

Returns the minimum value of the list in the capacitance_multicorner attribute.

capacitance_multicorner

Type: string

Feedback

If data from multiple corners is retrieved, the attribute contains a list of the capacitances of the corners specified by the *-parasitic_corners* option, in that order.

layer_id

Type: integer

Returns the ITF (nxtgrd file) layer ID.

layer_name

Type: string

Returns the ITF (nxtgrd file) layer name.

net

Type: collection

Returns the net that contains the ground capacitor.

node_index

Type: integer

Returns the index value of the node at which the ground capacitor connects to the net. Each node on a net has a unique index from 1 to N, where N is the total number of nodes on that net.

node_name

Type: string

Returns the node name in the format used in a SPEF output file.

node_type

Type: string

Returns the node type: pin, port, or internal node.

x_coordinate_center

Type: float

Returns the x-coordinate (in microns) of the center of the capacitor bounding box.

x_coordinate_max

Type: float

Returns the upper-right x-coordinate (in microns) of the capacitor bounding box.

Feedback

x_coordinate_min

Type: float

Returns the lower-left x-coordinate (in microns) of the capacitor bounding box.

y_coordinate_center

Type: float

Returns the y-coordinate (in microns) of the center of the capacitor bounding box.

y_coordinate_max

Type: float

Returns the upper-right y-coordinate (in microns) of the capacitor bounding box.

y_coordinate_min

Type: float

Returns the lower-left y-coordinate (in microns) of the capacitor bounding box.

See Also

- get_attribute
- help_attributes
- list_attributes
- report_attribute
- attributes

---

gui_annotation_attributes

Description of the predefined attributes for gui_annotation.

Description

Objects of type gui_annotation are created by the *gui_add_annotation* command. See the manpage for that command for more details.

To determine the value of an attribute use the *get_attribute* command.

client_data

A string attribute that can be used by client code to store arbitrary information

Feedback

color

The color used to draw the annotation.

fill_pattern

The fill pattern used to fill the annotationshape

group

The group of the annotation.

info_tip

The tip text (or command) displayed when a user hovers over the annotation in the layout.

line_style

The line style to use when drawing the annotaiton

line_width

The width of the line in pixels.

object_class

The string attribute with the value "gui_annotation"

points

The points for the annotation

query_command

The command to use when the annotation is queried in the layout.

query_text

The text to display when the annotation is queried in the layout.

shape_type

The type of shape for this annotation. For example, rect, line, etc.

text

If the shape_type is text this attribute has the text to display

window

When set this annotation will only be drawn on the specified window.

Feedback

See Also

- gui_get_annotations
- gui_remove_all_annotations

gui_object_attributes

Describes the predefined application attributes for gui_object objects.

Description

full_name

Type: string

Returns the full name of the object.

name

Type: string

Returns the name of the object.

object_class

Type: string

Returns the class of the object, which is a constant equal to gui_object. You cannot set this attribute.

See Also

- get_attribute
- help_attributes
- list_attributes
- report_attribute
- attributes

l

lib_attributes

Describes the predefined application attributes for lib objects.

Feedback

Description

capacitance_unit_in_farad

Type: float

Returns the unit of capacitance in the main library in farads. This attribute is read-only; you cannot change the setting.

current_unit_in_amp

Type: float

Returns the unit of current in the main library in amps. This attribute is read-only; you cannot change the setting.

default_connection_class

Type: string

Returns the default connection class string for the connection class attribute of a pin or port.

default_max_capacitance

Type: float

Returns the library default maximum capacitance design rule limit.

default_max_fanout

Type: float

Returns the library default maximum fanout design rule limit.

default_max_transition

Type: float

Returns the library default maximum transition design rule limit.

default_min_capacitance

Type: float

Returns the library default minimum capacitance design rule limit.

default_min_fanout

Type: float

Returns the library default minimum fanout design rule limit.

Feedback

default_min_transition

Type: float

Returns the library default minimum transition design rule limit.

default_threshold_voltage_group

Type: string

Returns the default threshold voltage group of the cells in the library. This attribute can be defined by the library or by using the set_user_attribute command. The user-defined value takes precedence over the library-defined value. PrimePower uses this attribute to identify the threshold voltage group to which cells belong. The threshold_voltage_group library cell attribute takes precedence over the default_threshold_voltage_group library attribute if both are present when classifying cells per voltage threshold group.

extended_name

Type: string

Returns the complete, unambiguous name of the library. The extended_name of the library is the source_file_name attribute followed by a colon (:) followed by the full_name attribute. For example, the extended_name of library tech1 read in from /abc/xyz/lib1.db is /abc/xyz/lib1.db:tech1.

full_name

Type: string

Returns the name of the library. For example, the full_name of library tech1 read in from /abc/xyz/lib1.db is tech1. This name can be ambiguous because multiple libraries of the same name can be read in from different files.

has_sensitization_data

Type: boolean

Returns true if the library has sensitization data.

k_process_cell_fall

Type: float

This attribute is a nonlinear delay model (NLDM) scaling factor. For more information about this attribute, see the Library Compiler documentation.

k_process_cell_rise

Type: float

Feedback

This attribute is a nonlinear delay model (NLDM) scaling factor. For more information about this attribute, see the Library Compiler documentation.

k_process_fall_transition

Type: float

This attribute is a nonlinear delay model (NLDM) scaling factor. For more information about this attribute, see the Library Compiler documentation.

k_process_rise_transition

Type: float

This attribute is a nonlinear delay model (NLDM) scaling factor. For more information about this attribute, see the Library Compiler documentation.

k_temp_cell_fall

Type: float

This attribute is a nonlinear delay model (NLDM) scaling factor. For more information about this attribute, see the Library Compiler documentation.

k_temp_cell_rise

Type: float

This attribute is a nonlinear delay model (NLDM) scaling factor. For more information about this attribute, see the Library Compiler documentation.

k_temp_fall_transition

Type: float

This attribute is a nonlinear delay model (NLDM) scaling factor. For more information about this attribute, see the Library Compiler documentation.

k_temp_rise_transition

Type: float

This attribute is a nonlinear delay model (NLDM) scaling factor. For more information about this attribute, see the Library Compiler documentation.

k_volt_cell_fall

Type: float

This attribute is a nonlinear delay model (NLDM) scaling factor. For more information about this attribute, see the Library Compiler documentation.

Feedback

k_volt_cell_rise

Type: float

This attribute is a nonlinear delay model (NLDM) scaling factor. For more information about this attribute, see the Library Compiler documentation.

k_volt_fall_transition

Type: float

This attribute is a nonlinear delay model (NLDM) scaling factor. For more information about this attribute, see the Library Compiler documentation.

k_volt_rise_transition

Type: float

This attribute is a nonlinear delay model (NLDM) scaling factor. For more information about this attribute, see the Library Compiler documentation.

lib_scaling_group

Type: collection

Returns a collection of libraries in the scaling library group to which the library belongs, which is set with the define_scaling_lib_group command.

min_extended_name

Type: string

Returns the full name (filename:lib_name) of the minimum library associated with the given library. This attribute is read-only; you cannot change the setting.

min_source_file_name

Type: string

Returns the full name of the minimum library associated with the given library. This attribute is read-only; you cannot change the setting.

object_class

Type: string

Returns the class of the object, which is the string "lib". You cannot set this attribute.

resistance_unit_in_ohm

Type: float

Feedback

Returns the unit of resistance in the main library in ohms. This attribute is read-only; you cannot change the setting.

source_file_name

Type: string

Returns the name of the file from which the library was read. For example, the source_file_name of library tech1 read in from /abc/def/lib1.db is /abc/def/lib1.db.

time_unit_in_second

Type: float

Returns the unit of time in the main library in seconds. This attribute is read-only; you cannot change the setting.

voltage_unit_in_volt

Type: float

Returns the unit of voltage in the main library in volts. This attribute is read-only; you cannot change the setting.

See Also

- get_attribute
- help_attributes
- list_attributes
- report_attribute
- attributes

---

lib_cell_attributes

Describes the predefined application attributes for lib_cell objects.

Description

always_on

Type: boolean

Returns true if the library cell is a UPF always-on cells.

area

Type: float

Feedback

Returns the area of the library cell.

base_name

Type: string

Returns the name of the library cell. For example, the base_name of library cell tech1/AN2 is AN2.

disable_timing

Type: boolean

Returns true if the timing for the library cell has been specified to be disabled using the set_disable_timing command.

dont_touch

Type: boolean

Returns true if the library cell is excluded from changes. Values are undefined by default. Instances of library cells with the dont_touch attribute set to true are not modified or replaced by ECO fixing commands (fix_eco_drc, fix_eco_power, and fix_eco_timing). This attribute is set by the set_dont_touch command.

dont_use

Type: boolean

Returns true if the library cell is excluded from the target library during ECO fixing. This attribute is set by the set_dont_use command.

full_name

Type: string

Returns the fully qualified name of the library cell. This is the name of the library followed by the library cell name. For example, the full_name of library cell AN2 in library tech1 is tech1/AN2.

function_id

Type: string

Returns the name of the function that is created by Library Compiler.

has_multi_ground_rails

Type: boolean

Returns true if the library cell has multiple ground rails.

Feedback

has_multi_power_rails

Type: boolean

Returns true if the library cell has multiple power rails.

has_rail_specific_power_tables

Type: boolean

Returns true if the library cell has multiple tables attached to rails.

is_black_box

Type: boolean

Returns true if there is no reference library cell from any of the link libraries.

is_combinational

Type: boolean

Returns true if the library cell has no sequential timing arcs. See also the report_lib command.

is_case_sequential_propagation

Type: boolean

Returns true if the library cell is enabled for sequential case propagation by the set_case_sequential_propagation command.

is_fall_edge_triggered

Type: boolean

Returns true if the library cell is used as a falling-edge-triggered flip-flop.

is_instantiated

Type: boolean

Returns true if the library cell is instantiated in the current design.

is_integrated_clock_gating_cell

Type: boolean

Returns true if the cell's reference is not linked to a library cell or design. This attribute is read-only.

is_isolation

Type: boolean

Feedback

Returns true if the library cell is a UPF isolation cell.

is_level_shifter

Type: boolean

Returns true if the library cell is a UPF level shifter cell.

is_macro_switch

Type: boolean

Returns true if the library cell is defined as a fine-grain switch cell in the library.

is_memory_cell

Type: boolean

Returns true if the library cell is classified in the memory power group.

is_mux

Type: boolean

Returns true if the library cell is a multiplexer.

is_negative_level_sensitive

Type: boolean

Returns true if the library cell is used as a negative level-sensitive latch.

is_pad_cell

Type: boolean

Returns true if the library cell is a pad cell.

is_pll_cell

Type: boolean

Returns true if the library cell is used as a phase-locked loop (PLL) cell.

is_positive_level_sensitive

Type: boolean

Returns true if the library cell is used as a positive level-sensitive latch.

is_retention

Type: boolean

Returns true if the library cell is a UPF retention cell.

Feedback

is_rise_edge_triggered

Type: boolean

Returns true if the library cell is used as a rising-edge-triggered flip-flop.

is_sequential

Type: boolean

Returns true if the library cell has at least one sequential timing arc. See also the report_lib command.

is_three_state

Type: boolean

Returns true if the library cell is a three-state device.

lib_pg_pin_info

Type: collection

Returns a collection of lib_pg_pin_info objects with these attributes: pin_name, type, voltage.

mog_func_id

Type: string

Returns the name of the cell's function that is created by Library Compiler for multiple output gate (MOG) cells.

number_of_pins

Type: integer

Returns the number of pins on the library cell.

object_class

Type: string

Returns the class of the object, which is "lib_cell". You cannot set this attribute.

power_cell_type

Type: string

Returns the predefined power group to which the cell belongs. Use the set_user_attribute command to set the attribute to one of these predefined power groups: clock_network, register, combinational, sequential, memory, io_pad, or black_box. You cannot use this attribute to assign a cell to a user-defined power group. The power_cell_type cell attribute

Feedback

overrides the power_cell_type library cell attribute. PrimePower uses this attribute in both averaged and time-based power analysis modes.

threshold_voltage_group

Type: string

Returns the threshold voltage group of the library cell. The attribute can be defined by the library or by using the set_user_attribute command. The user-specified value takes precedence over the library-defined value. If the attribute is undefined, it returns "No Default Threshold".

timing_model_type

Type: string

Returns the timing model type of the library cell. The possible values default, abstracted, extracted, and qtm.

user_function_class

Type: string

Returns the functional behavior of the library cell.

See Also

- get_attribute
- help_attributes
- list_attributes
- report_attribute
- attributes

---

lib_pg_pin_info_attributes

Describes the predefined application attributes for lib_pg_pin_info objects.

Description

pin_name

Type: string

Returns the pin name.

Feedback

type

Type: string

Returns primary_power or primary_ground.

voltage

Type: float

Returns the pin voltage.

See Also

- get_attribute
- help_attributes
- list_attributes
- report_attribute
- attributes

---

lib_pin_attributes

Describes the predefined application attributes for lib_pin objects.

Description

base_name

Type: string

Returns the leaf name of the library cell pin. For example, the base_name of tech1/AN2/Z is Z.

clock

Type: boolean

Returns true when a clock attribute is attached to the library pin in the library definition.

connection_class

Type: string

Returns the connection class string for the pin.

direction

Type: string

Feedback

Returns the direction of the pin. Value can be in, out, inout, or internal.

disable_timing

Type: boolean

Returns true if the library pin has been specified to be disabled using the set_disable_timing command.

drive_resistance_fall

Type: float

Returns the linear drive resistance for falling delays of the library pin.

drive_resistance_rise

Type: float

Returns the linear drive resistance for rising delays of the library pin.

driver_waveform_fall

Type: string

Returns the type of driver waveform for the library pin, either ramp or standard.

driver_waveform_rise

Type: string

Returns the type of driver waveform for the library pin, either ramp or standard.

fanout_load

Type: float

Returns the fanout load value of the library pin. This value is used in computing max_fanout design rule cost.

full_name

Type: string

Returns the fully qualified name of a library cell pin. This is the name of the library followed by the library cell name followed by a pin name. For example, the full_name of pin Z on library cell AN2 in library tech1 is tech1/AN2/Z.

has_ccs_noise_above_high

Type: boolean

Returns true if CCS noise information is present for a pin in a library.

Feedback

has_ccs_noise_above_low

Type: boolean

Returns true if CCS noise information is present for a pin in a library.

has_ccs_noise_below_high

Type: boolean

Returns true if CCS noise information is present for a pin in a library.

has_ccs_noise_below_low

Type: boolean

Returns true if CCS noise information is present for a pin in a library.

has_ccs_receiver_fall

Type: boolean

Returns true if CCS information is present for a pin in a library, for receiver fall analysis.

has_ccs_receiver_rise

Type: boolean

Returns true if CCS information is present for a pin in a library, for receiver rise analysis.

is_async_pin

Type: boolean

Returns true if the library pin is an asynchronous preset/clear pin.

is_clear_pin

Type: boolean

Returns true if the library pin is an asynchronous clear pin.

is_clock_pin

Type: boolean

Returns true if at least one instance of that clock pin exists that has the is_clock_pin attribute equal to true.

is_data_pin

Type: boolean

Feedback

Returns true if at least one instance of that data pin exists that has the is_data_pin attribute equal to true.

is_fall_edge_triggered_clock_pin

Type: boolean

Returns true if the library pin is used as a falling-edge-triggered flip-flop clock pin.

is_fall_edge_triggered_data_pin

Type: boolean

Returns true if the library pin is used as a falling-edge-triggered flip-flop data pin.

is_mux_select_pin

Type: boolean

Returns true if the library pin is a select pin of a multiplexer device.

is_negative_level_sensitive_clock_pin

Type: boolean

Returns true if the library pin is used as a negative level-sensitive latch clock pin.

is_negative_level_sensitive_data_pin

Type: boolean

Returns true if the library pin is used as a negative level-sensitive latch data pin.

is_pad

Type: boolean

Returns true if the library pin is a pad. See the Library Compiler documentation.

is_pll_feedback_pin

Type: boolean

Returns true if the library pin is a feedback pin of a phase locked loop (PLL) cell.

is_pll_output_pin

Type: boolean

Returns true if the library pin is an output pin of a phase locked loop (PLL) cell.

is_pll_reference_pin

Type: boolean

Feedback

Returns true if the library pin is a reference pin of a phase locked loop (PLL) cell.

is_positive_level_sensitive_clock_pin

Type: boolean

Returns true if the library pin is used as a positive level-sensitive latch clock pin.

is_positive_level_sensitive_data_pin

Type: boolean

Returns true if the library pin is used as a positive level-sensitive latch data pin.

is_preset_pin

Type: boolean

Returns true if the library pin is an asynchronous preset pin.

is_rise_edge_triggered_clock_pin

Type: boolean

Returns true if the library pin is used as a rising-edge-triggered flip-flop clock pin.

is_rise_edge_triggered_data_pin

Type: boolean

Returns true if the library pin is used as a rising-edge-triggered flip-flop data pin.

is_three_state

Type: boolean

Returns true if the library pin is a three-state driver.

is_three_state_enable_pin

Type: boolean

Returns true if the library pin is an enable pin of a three-state device.

is_three_state_output_pin

Type: boolean

Returns true if the library pin could output a three-state signal.

is_unbuffered

Type: boolean

Returns true if the library pin is unbuffered. See the Library Compiler documentation.

Feedback

load_of_pin_capacitance

Type: float

Returns the capacitance as specified by the Liberty file by the capacitance attribute in the pin object class.

max_capacitance

Type: float

Returns the maximum capacitance design rule limit for the library pin.

max_fanout

Type: float

Returns the maximum fanout design rule limit for the library pin.

max_transition

Type: float

Returns the maximum transition time design rule limit for the library pin.

min_capacitance

Type: float

Returns the minimum capacitance design rule limit for the library pin.

min_fanout

Type: float

Returns the minimum fanout design rule limit for the library pin.

min_transition

Type: float

Returns the minimum transition time design rule limit for the library pin.

object_class

Type: string

Returns the class of the object, which is a constant equal to lib_pin. You cannot set this attribute.

original_pin

Type: string

Feedback

Returns the original pin name defined in the block netlist that is used to generate an extracted timing model (ETM).

pin_capacitance

Type: float

Returns the capacitance of the library pin.

pin_capacitance_max_fall

Type: float

Returns the maximum fall capacitance of the library pin. This attribute is read-only; you cannot change the setting.

pin_capacitance_max_rise

Type: float

Returns the maximum rise capacitance of the library pin. This attribute is read-only; you cannot change the setting.

pin_capacitance_min_fall

Type: float

Returns the minimum fall capacitance of the library pin. This attribute is read-only; you cannot change the setting.

pin_capacitance_min_rise

Type: float

Returns the minimum rise capacitance of the library pin. This attribute is read-only; you cannot change the setting.

pin_direction

Type: string

Returns the direction of a pin. Allowed values are in, out, inout, or unknown. This attribute is read-only; you cannot change the settings.

rc_input_threshold_pct_fall

Type: float

Returns the characterization trip point (waveform measurement threshold) that the tool uses to calculate delays and transition times. Specifies the value obtained from the library to which the pin belongs.

Feedback

rc_input_threshold_pct_rise

Type: float

Returns the characterization trip point (waveform measurement threshold) that the tool uses to calculate delays and transition times. Specifies the value obtained from the library to which the pin belongs.

rc_output_threshold_pct_fall

Type: float

Returns the characterization trip point (waveform measurement threshold) that the tool uses to calculate delays and transition times. Specifies the value obtained from the library to which the pin belongs.

rc_output_threshold_pct_rise

Type: float

Returns the characterization trip point (waveform measurement threshold) that the tool uses to calculate delays and transition times. Specifies the value obtained from the library to which the pin belongs.

rc_slew_derate_from_library

Type: float

Returns the slew derating factor that the tool uses to calculate delays and transition times. Specifies the value obtained from the library to which the pin belongs.

rc_slew_lower_threshold_pct_fall

Type: float

Returns the characterization trip point (waveform measurement threshold) that the tool uses to calculate delays and transition times. Specifies the value obtained from the library to which the pin belongs.

rc_slew_lower_threshold_pct_rise

Type: float

Returns the characterization trip point (waveform measurement threshold) that the tool uses to calculate delays and transition times. Specifies the value obtained from the library to which the pin belongs.

rc_slew_upper_threshold_pct_fall

Type: float

Feedback

Returns the characterization trip point (waveform measurement threshold) that the tool uses to calculate delays and transition times. Specifies the value obtained from the library to which the pin belongs.

rc_slew_upper_threshold_pct_rise

Type: float

Returns the characterization trip point (waveform measurement threshold) that the tool uses to calculate delays and transition times. Specifies the value obtained from the library to which the pin belongs.

si_has_immunity_above_high

Type: boolean

Returns true if NLDM noise immunity information is present for a pin in a library.

si_has_immunity_above_low

Type: boolean

Returns true if NLDM noise immunity information is present for a pin in a library.

si_has_immunity_below_high

Type: boolean

Returns true if NLDM noise immunity information is present for a pin in a library.

si_has_immunity_below_low

Type: boolean

Returns true if NLDM noise immunity information is present for a pin in a library.

See Also

- get_attribute
- help_attributes
- list_attributes
- report_attribute
- attributes

lib_timing_arc_attributes

Describes the predefined application attributes for lib_timing_arc objects.

Feedback

Description

from_lib_pin

Type: collection

Returns a collection containing the from library pin of the library timing arc.

has_ccs_driver_fall

Type: boolean

Returns true if the library timing arc has CCS timing driver information for fall analysis.

has_ccs_driver_rise

Type: boolean

Returns true if the library timing arc has CCS timing driver information for rise analysis.

has_ccs_noise_above_high

Type: boolean

Returns true if CCS noise information is present in the timing arc object in a library.

has_ccs_noise_above_low

Type: boolean

Returns true if CCS noise information is present in the timing arc object in a library.

has_ccs_noise_below_high

Type: boolean

Returns true if CCS noise information is present in the timing arc object in a library.

has_ccs_noise_below_low

Type: boolean

Returns true if CCS noise information is present in the timing arc object in a library.

has_ccs_receiver_fall

Type: boolean

Returns true if CCS receiver information is present in the timing arc object in a library, for fall analysis.

has_ccs_receiver_rise

Type: boolean

Feedback

Returns true if CCS receiver information is present in the timing arc object in a library, for rise analysis.

is_disabled

Type: boolean

Returns true if the library timing arc is disabled.

is_user_disabled

Type: boolean

Returns true if the library timing arc is disabled by the set_disable_timing command.

mode

Type: string

Returns the mode string of the library timing arc.

object_class

Type: string

Returns the class of the object, which is the string "lib_timing_arc". You cannot set this attribute.

sdf_cond

Type: string

Returns a string representing the SDF condition of the library timing arc.

sense

Type: string

Returns a string representing the sense of the library timing arc.

si_has_immunity_above_high

Type: boolean

Returns true if NLDM noise immunity information is present in the timing arc object in a library.

si_has_immunity_above_low

Type: boolean

Returns true if NLDM noise immunity information is present in the timing arc object in a library.

Feedback

si_has_immunity_below_high

Type: boolean

Returns true if NLDM noise immunity information is present in the timing arc object in a library.

si_has_immunity_below_low

Type: boolean

Returns true if NLDM noise immunity information is present in the timing arc object in a library.

si_has_iv_above_high

Type: boolean

Returns true if the library timing arc contains output steady-state information in the form of I/V relationships (polynomials or tables).

si_has_iv_above_low

Type: boolean

Returns true if the library timing arc contains output steady-state information in the form of I/V relationships (polynomials or tables).

si_has_iv_below_high

Type: boolean

Returns true if the library timing arc contains output steady-state information in the form of I/V relationships (polynomials or tables).

si_has_iv_below_low

Type: boolean

Returns true if the library timing arc contains output steady-state information in the form of I/V relationships (polynomials or tables).

si_has_propagation_above_high

Type: boolean

Returns true if the library timing arc contains information about how bumps present at the arc input are propagated across the arc to the output.

si_has_propagation_above_low

Type: boolean

Feedback

Returns true if the library timing arc contains information about how bumps present at the arc input are propagated across the arc to the output.

si_has_propagation_below_high

Type: boolean

Returns true if the library timing arc contains information about how bumps present at the arc input are propagated across the arc to the output.

si_has_propagation_below_low

Type: boolean

Returns true if the library timing arc contains information about how bumps present at the arc input are propagated across the arc to the output.

si_has_resistance_above_high

Type: boolean

Returns true if the library timing arc contains output steady-state information in the form of simple steady drive resistance.

si_has_resistance_above_low

Type: boolean

Returns true if the library timing arc contains output steady-state information in the form of simple steady drive resistance.

si_has_resistance_below_high

Type: boolean

Returns true if the library timing arc contains output steady-state information in the form of simple steady drive resistance.

si_has_resistance_below_low

Type: boolean

Returns true if the library timing arc contains output steady-state information in the form of simple steady drive resistance.

to_lib_pin

Type: collection

Returns a collection containing the to library pin of the library timing arc.

Feedback

when

Type: string

Returns a string representing the when string of the library timing arc.

See Also

- get_attribute
- help_attributes
- list_attributes
- report_attribute
- attributes

m

mode_attributes

Describes the predefined application attributes for mode objects.

Description

name

Type: string

Returns the name of the mode object.

See Also

- get_attribute
- help_attributes
- list_attributes
- report_attribute
- attributes

Feedback

n

net_attributes

Describes the predefined application attributes for net objects.

Description

activity_source

Type: string

Returns the source of switching activity information for the net: file, set_switching_activity, set_case_analysis, propagated, implied, default, or UNINITIALIZED.

aggressors

Type: string

Returns the aggressor nets that affect the victim net.

annotated_delay_delta_max

Type: float

Returns the maximum of the annotated_delay_delta_max attributes on all net arcs.

annotated_delay_delta_min

Type: float

Returns the minimum of the annotated_delay_delta_min attributes on all net arcs.

annotated_transition_delta_max

Type: float

Returns the maximum of the annotated_transition_delta_max attributes on all leaf pins and ports.

annotated_transition_delta_min

Type: float

Returns the minimum of the annotated_transition_delta_min attributes on all leaf pins and ports.

area

Type: float

Feedback

Returns the estimated area of the net, calculated using a wire load model.

ba_capacitance_max

Type: float

Returns the back-annotated capacitance on the net for maximum conditions, set with the set_load or read_parasitics command.

ba_capacitance_min

Type: float

Returns the back-annotated capacitance on the net for minimum conditions, set with the set_load or read_parasitics command.

ba_resistance_max

Type: float

Returns the back-annotated resistance on the net for maximum conditions, set with the set_resistance or read_parasitics command.

ba_resistance_min

Type: float

Returns the back-annotated resistance of the net for minimum conditions, set with the set_resistance or read_parasitics command.

base_name

Type: string

Returns the leaf name of the net. For example, the base name of net i1/i1z1 is i1z1. You cannot modify this attribute.

coupling_capacitors

Type: string

Lists the cross-coupling capacitance values in main library units. With this attribute the nets are explicitly identified. For example,

```
{u1a/A (n1) u2b/Z (n2) 0.40 not_filtered}
{n1:3 (n1) n2:5 (n2) 0.03 filtered_by_accum_noise_peak}
{in_port (n1) n3:7 (n3) 0.01 filtered_by_accum_noise_peak}
```

dominant_metal_layer

Type: string

Returns a string of metal layers names having maximum length.

Feedback

dont_touch

Type: boolean

Returns true if the net is protected from change by the set_dont_touch command. When this attribute is set to true, buffers cannot be inserted on the net by ECO fixing commands (fix_eco_drc, fix_eco_power, and fix_eco_timing). By default, driver and load cells connected to the net can still be modified or removed by ECO fixing commands. This attribute exists for a net only after the attribute is created by the set_dont_touch command.

early_fall_clk_net_delta_derate_factor

Type: float

Returns the derating factor specified by the set_timing_derate command that applies to the dynamic (crosstalk) component of net delay for early falling clock transitions on the net.

early_fall_clk_net_derate_factor

Type: float

Returns the derating factor specified by the set_timing_derate command that applies to the static component of net delay for early falling clock transitions on the net.

early_fall_data_net_delta_derate_factor

Type: float

Returns the derating factor specified by the set_timing_derate command that applies to the dynamic (crosstalk) component of net delay for early falling data transitions on the net.

early_fall_data_net_derate_factor

Type: float

Returns the derating factor specified by the set_timing_derate command that applies to the static component of net delay for early falling data transitions on the net.

early_rise_clk_net_delta_derate_factor

Type: float

Returns the derating factor specified by the set_timing_derate command that applies to the dynamic (crosstalk) component of net delay for early rising clock transitions on the net.

early_rise_clk_net_derate_factor

Type: float

Returns the derating factor specified by the set_timing_derate command that applies to the static component of net delay for early rising clock transitions on the net.

Feedback

early_rise_data_net_delta_derate_factor

Type: float

Returns the derating factor specified by the set_timing_derate command that applies to the dynamic (crosstalk) component of net delay for early rising data transitions on the net.

early_rise_data_net_derate_factor

Type: float

Returns the derating factor specified by the set_timing_derate command that applies to the static component of net delay for early rising data transitions on the net.

effective_aggressors

Type: string

Lists the cross-coupled aggressor nets that have an effect large enough to be analyzed on the net as a victim. Other cross-coupled aggressor nets are excluded from the list. For more information, see the description of the si_xtalk_bumps attribute.

effective_coupling_capacitors

Type: string

Lists the effective cross-coupling capacitance values in main library units. Only the capacitors that are not excluded are shown.

escaped_full_name

Type: string

Returns the full hierarchical name of the net with any literal hierarchy characters escaped with a backslash.

full_name

Type: string

Returns the full hierarchical name of the net. For example, the full name of net i1z1 within cell i1 is i1/i1z1. The full_name attribute is not affected by the current instance setting. The full_name attribute is read-only.

glitch_count

Type: float

Returns the number of glitch transitions on the net for the duration of power_simulation_time (a design attribute).

Feedback

glitch_rate

Type: double

Returns the rate at which glitch transitions occur on the net, equal to glitch_count/ power_simulation_time

has_detailed_parasitics

Type: boolean

Returns true if any part of the net has annotated detailed parasitics, even if only one segment of the net at a different level of hierarchy.

has_valid_parasitics

Type: boolean

Returns true if the net has an annotated pi model, or if all segments of the net are annotated with properly connected detailed parasitics that form a valid representation of physical interconnection between drivers and loads.

is_clock_network

Type: boolean

Returns true if the net is in the combinational fanout of a clock source, that is, the is_clock_network attribute is true on any of the leaf pins or ports of the net.

is_clock_source_network

Type: boolean

Returns true if the net is part of clock source latency network, that is, the is_clock_source_network attribute is true on any of its leaf pins or ports.

is_design_mismatch

Type: boolean

Returns true if the net has a mismatch between the block and the top-level design.

is_ideal

Type: boolean

Returns true if the net has been marked ideal using the set_ideal_network command.

is_power_control_signal_net

Type: boolean

Feedback

Returns true if the signal net is used to control any power rail or PrimePower rail mapping modes.

is_routed

Type: boolean

Returns true if the signal net is routed, or has valid netshapes.

late_fall_clk_net_delta_derate_factor

Type: float

Returns the derating factor specified by the set_timing_derate command that applies to the dynamic (crosstalk) component of net delay for late falling clock transitions on the net.

late_fall_clk_net_derate_factor

Type: float

Returns the derating factor specified by the set_timing_derate command that applies to the static component of net delay for late falling clock transitions on the net.

late_fall_data_net_delta_derate_factor

Type: float

Returns the derating factor specified by the set_timing_derate command that applies to the dynamic (crosstalk) component of net delay for late falling data transitions on the net.

late_fall_data_net_derate_factor

Type: float

Returns the derating factor specified by the set_timing_derate command that applies to the static component of net delay for late falling data transitions on the net.

late_rise_clk_net_delta_derate_factor

Type: float

Returns the derating factor specified by the set_timing_derate command that applies to the dynamic (crosstalk) component of net delay for late rising clock transitions on the net.

late_rise_clk_net_derate_factor

Type: float

Returns the derating factor specified by the set_timing_derate command that applies to the static component of net delay for late rising clock transitions on the net.

Feedback

late_rise_data_net_delta_derate_factor

Type: float

Returns the derating factor specified by the set_timing_derate command that applies to the dynamic (crosstalk) component of net delay for late rising data transitions on the net.

late_rise_data_net_derate_factor

Type: float

Returns the derating factor specified by the set_timing_derate command that applies to the static component of net delay for late rising data transitions on the net.

leaf_drivers

Type: collection

Returns a collection of leaf driver pins and ports of the net.

leaf_loads

Type: collection

Returns a collection of leaf load pins and ports of the net.

max_slack

Type: float

Returns the minimum of the max_slack of all leaf pins and ports. This attribute is computed only when the timing_save_pin_arrival_and_slack variable is set to true.

min_slack

Type: float

Returns the minimum of the min_slack attribute on all leaf pins and ports. This attribute is computed only when the timing_save_pin_arrival_and_slack variable is set to true.

net_resistance_max

Type: float

Returns the resistance of the net for maximum conditions. The value be computed from wire load models, set by the set_resistance command, or annotated by the read_parasitics command.

net_resistance_min

Type: float

Feedback

Returns the resistance of the net for minimum conditions. The value be computed from wire load models, set by the set_resistance command, or annotated by the read_parasitics command.

effective_aggressors

Type: string

Lists the cross-coupled aggressor nets that have an effect large enough to be analyzed on the net as a victim. Other cross-coupled aggressor nets are excluded from the list. For more information, see the description of the si_xtalk_bumps attribute.

effective_coupling_capacitors

Type: string

Lists the effective cross-coupling capacitance values in main library units. Only the capacitors that are not excluded are shown.

number_of_aggressors

Type: integer

Returns the total number of cross-coupled aggressor nets, including both effective and ignored aggressor nets.

number_of_coupling_capacitors

Type: integer

Returns the total number of coupling capacitors, including both effective and ignored capacitors.

number_of_effective_aggressors

Type: integer

Returns the number of cross-coupled aggressor nets that have an effect large enough to be analyzed on the net as a victim.

number_of_effective_coupling_capacitors

Type: integer

Returns the number of coupling capacitors on the net that have an effect large enough to be analyzed.

number_of_leaf_drivers

Type: integer

Feedback

Returns the number of driver leaf pins that are connected to the net. Because a leaf pin is connected to a leaf cell, this attribute does not include pins at hierarchical boundaries. For hierarchical nets, this attribute reflects the total number of driver pins connected to all net segments.

number_of_leaf_loads

Type: integer

Returns the number of load leaf pins that are connected to the net. Because a leaf pin is connected to a leaf cell, this attribute does not include pins at hierarchical boundaries. For hierarchical nets, this attribute reflects the total number of load pins connected to all net segments.

number_of_vias

Type: integer

Returns the total number of vias for a net.

object_class

Type: string

Returns the class of the object, which is "net". You cannot set this attribute.

parent_cell

Type: collection

Returns the parent hierarchical cell containing the net.

pin_capacitance_max

Type: float

Returns the sum of all pin capacitance values of the net for maximum conditions. You cannot modify this attribute.

pin_capacitance_max_fall

Type: float

Returns the maximum fall capacitance of all pins and ports for the net. You cannot modify this attribute.

pin_capacitance_max_rise

Type: float

Returns the maximum rise capacitance of all pins and ports for the net. You cannot modify this attribute.

Feedback

pin_capacitance_min

Type: float

Returns the sum of all pin capacitance values of the net for minimum conditions. You cannot modify this attribute.

pin_capacitance_min_fall

Type: float

Returns the minimum fall capacitance of all pins and ports for the net. You cannot modify this attribute.

pin_capacitance_min_rise

Type: float

Returns the minimum rise capacitance of all pins and ports for the net. You cannot modify this attribute.

power_base_clock

Type: string

Returns the name of the base clock associated with this net. If the net belongs to multiple clock domains, the attribute is set to the fastest of the clocks.

rc_annotated_segment

Type: boolean

Returns true if the specific net segment has annotated parasitics. For two segments at different levels of hierarchy (for example, n1 and h1/n1), the attribute values can differ.

rc_network

Type: string

Returns a string describing the parasitic data that has been back-annotated on the net, including resistor values (in KOhms) and capacitor values (in pF).

rc_network_with_sensitivity

Type: string

Returns the full RC network of the net with sensitivity values included. This attribute is read-only.

relative_toggle_rate

Type: double

Feedback

Returns the number of toggles per period of the power_base_clock. If the design does not have a defined clock, this attribute is undefined.

route_length

Type: double

Returns the total route length of the net.

si_double_switching_slack

Type: float

Returns the slack value if the net has double-switching slack on the victim net.

si_has_double_switching

Type: boolean

Returns true if the net has a double-switching violation. Double-switching analysis needs to be enabled.

si_xtalk_bumps

Type: string

Lists each aggressor net and the voltage bumps that rising and falling aggressor transitions induce on the victim net (worst of rising minimum or maximum bumps and worst of falling minimum or maximum bumps, each expressed as a decimal fraction of the rail-to-rail voltage), or gives the reason that an aggressor net has no effect on the victim net.

si_xtalk_bumps_max_fall

Type: string

Lists each aggressor net and the voltage induced on the victim net (expressed as a decimal fraction of the rail-to-rail voltage) for a maximum fall transition.

si_xtalk_bumps_max_rise

Type: string

Lists each aggressor net and the voltage induced on the victim net (expressed as a decimal fraction of the rail-to-rail voltage) for a maximum rise transition.

si_xtalk_bumps_min_fall

Type: string

Lists each aggressor net and the voltage induced on the victim net (expressed as a decimal fraction of the rail-to-rail voltage) for a minimum fall transition.

Feedback

si_xtalk_bumps_min_rise

Type: string

Lists each aggressor net and the voltage induced on the victim net (expressed as a decimal fraction of the rail-to-rail voltage) for a minimum rise transition.

si_xtalk_composite_aggr_max_fall

Type: collection

Returns a collection of aggressors in a potential composite aggressor group for a maximum fall transition.

si_xtalk_composite_aggr_max_rise

Type: collection

Returns a collection of aggressors in a potential composite aggressor group for a maximum rise transition.

si_xtalk_composite_aggr_min_fall

Type: collection

Returns a collection of aggressors in a potential composite aggressor group for a minimum fall transition.

si_xtalk_composite_aggr_min_rise

Type: collection

Returns a collection of aggressors in a potential composite aggressor group for a minimum rise transition.

si_xtalk_used_ccs_max_fall

Type: boolean

Returns true if the net was analyzed for crosstalk delay using CCS timing models for maximum fall timing constraints and transitions.

si_xtalk_used_ccs_max_rise

Type: boolean

Returns true if the net was analyzed for crosstalk delay using CCS timing models for maximum rise timing constraints and transitions.

si_xtalk_used_ccs_min_fall

Type: boolean

Feedback

Returns true if the net was analyzed for crosstalk delay using CCS timing models for minimum fall timing constraints and transitions.

si_xtalk_used_ccs_min_rise

Type: boolean

Returns true if the net was analyzed for crosstalk delay using CCS timing models for minimum rise timing constraints and transitions.

static_probability

Type: float

Returns the static probability of the net, which is the probability that the net has the logic value 1.

switching_power

Type: double

Returns the switching power of the net in watts, which is the power dissipated by the charging and discharging of the capacitance of the net.

toggle_count

Type: float

Returns the number of transitions of the net during the duration of the power_simulation_time attribute (a design attribute).

toggle_rate

Type: double

Returns the rate at which transitions occur on the net. It is equal to toggle_count/ power_simulation_time

total_capacitance_max

Type: float

Returns the sum of all pin capacitance values and the wire capacitance of the net for maximum conditions. This attribute is read-only.

total_capacitance_min

Type: float

Returns the sum of all pin capacitance values and the wire capacitance of the net for minimum conditions. This attribute is read-only.

Feedback

total_ccs_capacitance_max_fall

Type: float

Returns the total capacitance, including both wire capacitance and maximum of falling CCS receiver capacitance. For example, total_ccs_capacitance_max_fall is the sum of wire capacitance and maximum of fall_c1/fall_c2.

total_ccs_capacitance_max_rise

Type: float

Returns the total capacitance, including both wire capacitance and maximum of rising CCS receiver capacitance. For example, total_ccs_capacitance_max_rise is the sum of wire capacitance and maximum of rise_c1/rise_c2.

total_ccs_capacitance_min_fall

Type: float

Returns the total capacitance, including both wire capacitance and minimum of falling CCS receiver capacitance. For example, total_ccs_capacitance_min_fall is the sum of wire capacitance and minimum of fall_c1/fall_c2.

total_ccs_capacitance_min_rise

Type: float

Returns the total capacitance, including both wire capacitance and minimum of rising CCS receiver capacitance. For example, total_ccs_capacitance_min_rise is the sum of wire capacitance and minimum of rise_c1/rise_c2.

total_coupling_capacitance

Type: float

Returns the total cross-coupling capacitance (in main library units) of the net as a victim.

total_effective_coupling_capacitance

Type: float

Returns the total effective cross capacitance (in main library units) of the net as a victim, including only the capacitors that have a large enough effect to be considered for analysis.

user_global_coupling_separated

Type: boolean

Returns true if this net has been globally separated by the set_coupling_separation command.

Feedback

user_pairwise_coupling_separated

Type: collection

Returns a collection of other nets that have been pairwise-separated from the net by the set_coupling_separation command.

wire_capacitance_max

Type: float

Returns the wire capacitance of the net for maximum conditions. The value can be computed from wire load models, set by the set_load command, or annotated by the read_parasitics command.

wire_capacitance_min

Type: float

Returns the wire capacitance of the net for minimum conditions. The value can be computed from wire load models, set by the set_load command, or annotated by the read_parasitics command.

x_coordinate_max

Type: float

Returns the maximum x-coordinate of the area occupied by the net.

x_coordinate_min

Type: float

Returns the minimum x-coordinate of the area occupied by the net.

y_coordinate_max

Type: float

Returns the maximum y-coordinate of the area occupied by the net.

y_coordinate_min

Type: float

Returns the minimum y-coordinate of the area occupied by the net.

See Also

- get_attribute
- help_attributes

Feedback

- list_attributes
- report_attribute
- attributes

p

path_group_attributes

Describes the predefined application attributes for path_group objects.

Description

full_name

Type: string

Returns the name of the path group. Path groups are created by group_path or implicitly using create_clock. This attribute is read-only; you cannot change the setting.

object_class

Type: string

Returns the class of the object, which is a constant equal to path_group. You cannot set this attribute.

weight

Type: float

Returns the cost function weight assigned to this path group. The weight of a group specifies how much the group influences the total maximum delay cost and can be used to guide optimization. You can specify the weight with group_path.

See Also

- get_attribute
- help_attributes
- list_attributes
- report_attribute
- attributes

Feedback

pg_pin_info_attributes

Describes the predefined application attributes for pg_pin_info objects.

Description

full_name

Type: string

Returns the full supply pin name.

pin_name

Type: string

Returns the leaf supply pin name.

supply_connection

Type: string

Returns the supply connection.

type

Type: string

Returns the supply type: *primary_power* or *primary_ground*.

voltage_for_max_delay

Type: float

Returns the voltage used for max-delay analysis.

voltage_for_min_delay

Type: float

Returns the voltage used for min-delay analysis.

voltage_source

Type: string

Returns a keyword that indicates the source of the voltage value:

- *DCALM* - define_cell_alternative_lib_mapping specification
- *SET_VOLTAGE_ON_PG_PIN* - explicit set_voltage specification on pg_pin
- *CONNECTED_SUPPLY_NET* - connected supply net

Feedback

- *SUPPLY_GROUP* - derived from supply group
- *DESIGN_OC* - design operating condition
- *EXTENDED_OC* - cell operating condition
- *LIBRARY_VOLTAGE_MAP* - voltage map from library
- *NOMINAL_LIB_VOLTAGE* - default library voltages
- *BEST_MATCH* - best match library voltage

See Also

- get_attribute
- help_attributes
- list_attributes
- report_attribute
- attributes

---

pin_attributes

Describes the predefined application attributes for pin objects.

Description

actual_fall_transition_max

Type: float

Returns the largest falling transition time for the pin.

actual_fall_transition_min

Type: float

Returns the smallest falling transition time for the pin.

actual_min_clock_pulse_width_high

Type: string

Returns the per-clock actual minimum pulse width value at the pin (high pulse).

actual_min_clock_pulse_width_low

Type: string

Returns the per-clock actual minimum pulse width value at the pin (low pulse).

Feedback

actual_rise_transition_max

Type: float

Returns the largest rising transition time for the pin.

actual_rise_transition_min

Type: float

Returns the smallest rising transition time for the pin.

actual_transition_max

Type: float

Returns the maximum of the actual_rise_transition_max and actual_fall_transition_max attributes.

actual_transition_min

Type: float

Returns the minimum of the actual_rise_transition_min and actual_fall_transition_min attributes.

annotated_fall_transition_delta_max

Type: float

Returns the additional transition time added to the maximum falling transition time on the pin. The additional transition time is set by either the set_annotated_transition -delta_only command or by PrimeTime SI during crosstalk analysis.

annotated_fall_transition_delta_min

Type: float

Returns the additional transition time added to the minimum falling transition time on the pin. The additional transition time is set by either the set_annotated_transition -delta_only command or by PrimeTime SI during crosstalk analysis.

annotated_rise_transition_delta_max

Type: float

Returns the additional transition time added to the maximum rising transition time on the pin. The additional transition time is set by either the set_annotated_transition -delta_only command or by PrimeTime SI during crosstalk analysis.

annotated_rise_transition_delta_min

Type: float

Feedback

Returns the additional transition time added to the minimum rising transition time on the pin. The additional transition time is set by either the set_annotated_transition -delta_only command or by PrimeTime SI during crosstalk analysis.

annotated_transition_delta_max

Type: float

Returns the maximum of the annotated_rise_transition_delta_max and annotated_fall_transition_delta_max attributes.

annotated_transition_delta_min

Type: float

Returns the minimum of the annotated_rise_transition_delta_min and annotated_fall_transition_delta_min attributes.

arrival_window

Type: string

Returns the minimum and maximum arrivals for rise and fall transitions. To get the arrival_window attribute on pins that are not endpoints, set the timing_save_pin_arrival_and_slack variable to true.

cached_c1_max_fall

Type: float

Returns the C1 CCS receiver model for a maximum fall transition.

cached_c1_max_rise

Type: float

Returns the C1 CCS receiver model for a maximum rise transition.

cached_c1_min_fall

Type: float

Returns the C1 CCS receiver model for a minimum fall transition.

cached_c1_min_rise

Type: float

Returns the C1 CCS receiver model for a minimum rise transition.

cached_c2_max_fall

Type: float

Feedback

Returns the C2 CCS receiver model for a maximum fall transition.

cached_c2_max_rise

Type: float

Returns the C2 CCS receiver model for a maximum rise transition.

cached_c2_min_fall

Type: float

Returns the C2 CCS receiver model for a minimum fall transition.

cached_c2_min_rise

Type: float

Returns the C2 CCS receiver model for a minimum rise transition.

cached_ceff_max_fall

Type: float

Returns the worst effective capacitance of a net with detailed parasitics stored during the delay calculation that is connected to this output pin. This requires the rc_cache_min_max_rise_fall_ceff variable to be set to true before the timing update.

cached_ceff_max_rise

Type: float

Returns the worst effective capacitance of a net with detailed parasitics stored during the delay calculation that is connected to this output pin. This requires the rc_cache_min_max_rise_fall_ceff variable to be set to true before the timing update.

cached_ceff_min_fall

Type: float

Returns the worst effective capacitance of a net with detailed parasitics stored during the delay calculation that is connected to this output pin. This requires the rc_cache_min_max_rise_fall_ceff variable to be set to true before the timing update.

cached_ceff_min_rise

Type: float

Returns the worst effective capacitance of a net with detailed parasitics stored during the delay calculation that is connected to this output pin. This requires the rc_cache_min_max_rise_fall_ceff variable to be set to true before the timing update.

Feedback

case_value

Type: string

Returns the user-specified logic value of the pin or port propagated from a case analysis or logic constant. This attribute is computed only for leaf pins.

ceff_params_max

Type: string

Returns the parameters internally computed by PrimeTime to compute the effective capacitance of a driver pin of a cell, respectively for maximum operating conditions. The returned parameters are rd, t0, delta_t, and Ceff; they represent how a driver is modeled for computing the effective capacitance.

ceff_params_min

Type: string

Returns the parameters internally computed by PrimeTime to compute the effective capacitance of a driver pin of a cell, respectively for minimum operating conditions. The returned parameters are rd, t0, delta_t, and Ceff; they represent how a driver is modeled for computing the effective capacitance.

cell

Type: collection

Returns a collection that contains the cell that this pin belongs to.

clock_capture_arrival_dynamic

Type: string

Returns the arrival time on a clock capture path endpoint including the dynamic arrival time. The dynamic arrival time is modeled using the set_clock_latency -dynamic command. If the clock source latency is modeled including the dynamic latency (set_clock_latency -source -dynamic), the static arrival attributes exclude the dynamic value. To query this attribute, you need to enable PrimeTime SI analysis.

clock_capture_arrival_static

Type: string

Returns the static arrival time on a clock capture path endpoint. The tool calculates the static arrival time as the sum of the clock source latency (specified by the set_clock_latency -source command) and the clock network latency.

clock_latency_fall_max

Type: float

Feedback

Returns the user-specified maximum fall latency (insertion delay) of a pin in the clock network. Set with the set_clock_latency command.

clock_latency_fall_min

Type: float

Returns the user-specified minimum fall latency (insertion delay) of a pin in the clock network. Set with the set_clock_latency command.

clock_latency_rise_max

Type: float

Returns the user-specified maximum rise latency (insertion delay) of a pin in the clock network. Set with set_clock_latency.

clock_latency_rise_min

Type: float

Returns the user-specified minimum rise latency (insertion delay) of a pin in the clock network. Set with set_clock_latency.

clock_launch_arrival_dynamic

Type: string

Returns the arrival time on a clock launch path endpoint including the dynamic arrival time. The dynamic arrival time is modeled using the set_clock_latency -dynamic command. If the clock source latency is modeled including the dynamic latency (set_clock_latency -source -dynamic), the static arrival attributes exclude the dynamic value. To query this attribute, you need to enable PrimeTime SI analysis.

clock_launch_arrival_static

Type: string

Returns the static arrival time on a clock launch path endpoint. The tool calculates the static arrival time as the sum of the clock source latency (specified by the set_clock_latency -source command) and the clock network latency.

clock_source_latency_early_fall_max

Type: float

Returns the maximum early falling source latency. Set with the set_clock_latency command.

clock_source_latency_early_fall_min

Type: float

Feedback

Returns the minimum early falling source latency. Set with the set_clock_latency command.

clock_source_latency_early_rise_max

Type: float

Returns the maximum early rising source latency. Set with the set_clock_latency command.

clock_source_latency_early_rise_min

Type: float

Returns the minimum early rising source latency. Set with the set_clock_latency command.

clock_source_latency_late_fall_max

Type: float

Returns the maximum late falling source latency. Set with the set_clock_latency command.

clock_source_latency_late_fall_min

Type: float

Returns the minimum late falling source latency. Set with the set_clock_latency command.

clock_source_latency_late_rise_max

Type: float

Returns the maximum late rising source latency. Set with the set_clock_latency command.

clock_source_latency_late_rise_min

Type: float

Returns the minimum late rising source latency. Set with the set_clock_latency command.

clocks

Type: collection

Returns a collection of clock objects that propagate through the pin. It is undefined if no clocks are present.

constant_value

Type: string

Returns the logic value of a pin tied to logic constant zero or one in the netlist.

Feedback

constraining_max_transition

Type: float

Returns the most constraining maximum transition value from all sources at the pin.

drc_actual_max_capacitance

Type: float

Returns the actual capacitance value of the driven net at a driver (output or inout) pin. This is the value used by *report_constraint* for maximum capacitance DRC checking. This attribute is undefined for input pins.

drc_actual_max_transition

Type: float

Returns the actual transition value at the pin. This is the value used by *report_constraint* for maximum transition DRC checking. It is affected by the *si_xtalk_max_transition_mode variable and the timing_pocvm_max_transition_sigma variable (in a POCV analysis).*

drc_actual_min_capacitance

Type: float

Returns the actual capacitance value of the driven net at a driver (output or inout) pin. This is the value used by *report_constraint* for minimum capacitance DRC checking. This attribute is undefined for input pins.

drc_constraining_max_capacitance

Type: float

Returns the most constraining maximum capacitance value from all sources at a driver (output or inout) pin. This attribute is undefined for input pins.

drc_constraining_min_capacitance

Type: float

Returns the most constraining minimum capacitance value from all sources at a driver (output or inout) pin. This attribute is undefined for input pins.

drc_coupled_actual_max_transition

Type: float

This attribute is similar to the *drc_actual_max_transition* attribute, except that coupled delta delays are always included (equivalent to setting the *si_xtalk_max_transition_mode variable to reliability).*

Feedback

drc_max_capacitance_slack

Type: float

Returns the capacitance slack at the pin for maximum condition DRC checking purposes.

drc_max_transition_slack

Type: float

Returns the transition slack at the pin for maximum condition DRC checking purposes.

drc_min_capacitance_slack

Type: float

Returns the capacitance slack at the pin for minimum condition DRC checking purposes.

direction

Type: string

Returns the direction of the pin. Value can be in, out, inout, or internal. The pin_direction attribute is a synonym for direction. Directions can change as a result of linking a design, as references are resolved.

disable_timing

Type: boolean

Returns true for a disabled timing arc. This has the same effect on timing as not having the arc in the library. Set with the set_disable_timing command.

driver_model_scaling_libs_max

Type: collection

Returns a collection of library objects used for driver model scaling, where applicable, for libs maximum analysis.

driver_model_scaling_libs_min

Type: collection

Returns a collection of library objects used for driver model scaling, where applicable, for libs minimum analysis.

driver_model_type_max_fall

Type: string

Returns the driver model type, either basic (NLDM) or advanced (CCS timing), for maximum fall analysis.

Feedback

driver_model_type_max_rise

Type: string

Returns the driver model type, either basic (NLDM) or advanced (CCS timing), for maximum rise analysis.

driver_model_type_min_fall

Type: string

Returns the driver model type, either basic (NLDM) or advanced (CCS timing), for minimum fall analysis.

driver_model_type_min_rise

Type: string

Returns the driver model type, either basic (NLDM) or advanced (CCS timing), for minimum rise analysis.

effective_capacitance_max

Type: float

Returns the effective capacitance for maximum operating conditions. Valid only for driver pins attached to annotated RC networks.

effective_capacitance_min

Type: float

Returns the effective capacitance for minimum operating conditions. Valid only for driver pins attached to annotated RC networks.

escaped_full_name

Type: string

Returns the name of the cell. Any literal hierarchy characters are escaped with a backslash.

fanout_load

Type: float

Returns the fanout load value of a pin. This value is used in computing max_fanout design rule cost.

full_name

Type: string

Returns the complete name of the pin to the top of the hierarchy. For example, the full name of pin Z on cell U2 within cell U1 is U1/U2/Z. The setting of the current instance has no effect on the full name of a pin. See also the lib_pin_name attribute.

glitch_rate

Type: double

Returns the rate at which the glitch transitions occur on the pin within a specific time period.

has_model_mismatch_clock

Type: boolean

Returns true if at least one model arrival or required time refers to an unmapped clock. This HyperScale attribute applies to a side-input or stub pin.

has_valid_max_fall_transition

Type: boolean

Returns *true* if the max-fall transition is the startpoint for a valid delay calculation arc. Undriven pins and case analysis pins/ports (user-defined or constant-sourced) are initialized with placeholder zero transition values; pins/ports in these "placeholder transition" propagation regions also return *true*. This attribute returns *false* for pins/ports that receive an uninitialized transition value from a timing arc disabled by the *set_disable_timing command*, or that receive an uninitialized rise or fall transition value from a library cell with non-unate arcs that do not provide both rise and fall incoming transitions.

has_valid_max_rise_transition

Type: boolean

Returns *true* if the max-rise transition is the startpoint for a valid delay calculation arc. See the *has_valid_max_fall_transition* description for details.

has_valid_min_fall_transition

Type: boolean

Returns *true* if the min-fall transition is the startpoint for a valid delay calculation arc. See the *has_valid_max_fall_transition* description for details.

has_valid_min_rise_transition

Type: boolean

Returns *true* if the min-rise transition is the startpoint for a valid delay calculation arc. See the *has_valid_max_fall_transition* description for details.

Feedback

hold_uncertainty

Type: float

Returns the clock uncertainty (skew) of a clock used for hold (and other minimum delay) timing checks. Set with the set_clock_uncertainty command.

ideal_latency_max_fall

Type: float

Returns the ideal delay value annotated on a pin in an ideal network, using the set_ideal_latency command.

ideal_latency_max_rise

Type: float

Returns the ideal delay value annotated on a pin in an ideal network, using the set_ideal_latency command.

ideal_latency_min_fall

Type: float

Returns the ideal delay value annotated on a pin in an ideal network, using the set_ideal_latency command.

ideal_latency_min_rise

Type: float

Returns the ideal delay value annotated on a pin in an ideal network, using the set_ideal_latency command.

ideal_transition_max_fall

Type: float

Returns the ideal transition time annotated on a pin in an ideal network, using the set_ideal_transition command.

ideal_transition_max_rise

Type: float

Returns the ideal transition time annotated on a pin in an ideal network, using the set_ideal_transition command.

ideal_transition_min_fall

Type: float

Feedback

Returns the ideal transition time annotated on a pin in an ideal network, using the set_ideal_transition command.

ideal_transition_min_rise

Type: float

Returns the ideal transition time annotated on a pin in an ideal network, using the set_ideal_transition command.

is_abstracted

Type: boolean

Returns true if the port is abstracted in the HyperScale model. This attribute applies only to ports at the block level and corresponding pins at the top level. This attribute is available only after update_timing at the block level. The attribute returns true for set_port_abstraction -ignore. Ports with is_user_abstracted == true are a subset of ports with is_abstracted == true.

is_async_pin

Type: boolean

Returns true for an asynchronous preset/clear pin.

is_clear_pin

Type: boolean

Returns true for an asynchronous clear pin.

is_clock_gating_pin

Type: boolean

Returns true for a pin of a clock-gating cell.

is_clock_network

Type: boolean

Returns true if the pin is in the combinational fanout of a clock source.

is_clock_pin

Type: boolean

Returns true on a valid instance pin object and on an active clock pin that is reached by a clock signal and where that sequential cell is not disabled by disabled timing arcs or case analysis.

Feedback

is_clock_source

Type: boolean

Returns true if a design clock was declared with a source at the pin.

is_clock_source_network

Type: boolean

Returns true if the pin is part of a clock source latency network.

is_clock_used_as_clock

Type: boolean

Returns true if the clock through the pin acts as a clock.

is_clock_used_as_data

Type: boolean

Returns true if the clock through the pin acts as data.

is_data_check_constrained_pin

Type: boolean

Returns true if the pin is the constrained pin of a data-to-data check, the "to" pin of the *set_data_check* command.

is_data_check_related_pin

Type: boolean

Returns true if the pin is the related pin of a data-to-data check, the "from" pin of a *set_data_check* command.

is_data_pin

Type: boolean

Returns true if a pin is a data pin of a sequential cell or the "to" (constrained) pin of a data-to-data check. For instance pin objects, the pin must be a valid and active pin that is not disabled by disabled timing arcs, disabled/inactive clocks, or case analysis.

is_drc_data_path

Type: boolean

Returns true if the pin is considered as part of a data path during DRC checking.

Feedback

is_model_interface

Type: boolean

Returns true if the pin is considered as on the interface of the current design when HyperScale model is created for the current design. Pins on the interface will be written out in HyperScale model data and visible at the top level when the model is instantiated.

is_model_internal_aggressor

Type: boolean

Returns true if the pin is not on the interface of current design, but HyperScale model needs to consider its aggressor effects because the net connected with the pin is physically coupled to interface logic through parasitics data. The pin will be written in HyperScale model and visible at top level and contribute to SI analysis of the interface.

is_design_mismatch

Type: boolean

Returns true if the pin has a mismatch between the block and top-level design.

is_model_side_input

Type: boolean

Returns true if the pin is not considered as on the interface of the current design when HyperScale model is created for the current design. The cell containing the side input pin is on the interface since at least some pins of the cell are on the interface.

is_model_stub

Type: boolean

Returns true if the pin is not considered as on the interface of the current design when HyperScale model is created for the current design. The net is on the interface since the driver and some load pins of the same net are on the interface.

is_driver_scaled_max_fall

Type: boolean

Returns true if the pin uses scaling between libraries for the driver model for maximum fall analysis.

is_driver_scaled_max_rise

Type: boolean

Returns true if the pin uses scaling between libraries for the driver model for maximum rise analysis.

Feedback

is_driver_scaled_min_fall

Type: boolean

Returns true if the pin uses scaling between libraries for the driver model for minimum fall analysis.

is_driver_scaled_min_rise

Type: boolean

Returns true if the pin uses scaling between libraries for the driver model for minimum rise analysis.

is_excluded

Type: boolean

Returns true if the pin is excluded from timing analysis. The pin is inside the HyperScale block and visible at HyperScale top, retained for circuit completion.

is_fall_edge_triggered_clock_pin

Type: boolean

Returns true if the pin is used as a falling-edge-triggered flip-flop clock pin.

is_fall_edge_triggered_data_pin

Type: boolean

Returns true if the pin is used as a falling-edge-triggered flip-flop data pin.

is_hierarchical

Type: boolean

Returns true for a pin of a hierarchical cell (instantiation of a design or a black-box cell), or false for a pin of a leaf cell (an instantiation of a library cell).

is_ideal

Type: boolean

Returns true if the pin has been marked ideal using the set_ideal_network command.

is_interface

Type: boolean

Returns true if the pin is on an interface path inside a HyperScale block and can be seen from the HyperScale top.

Feedback

is_interface_logic_pin

Type: boolean

Returns true if the pin is in an interface logic model (ILM) of the design. This attribute is read-only; you cannot change the setting.

is_internal_aggressor

Type: boolean

Returns true if the pin is a dangling aggressor inside the HyperScale block.

is_latch_loop_breaker

Type: boolean

Returns true if the pin is the D pin of a loop-breaker latch.

is_mux_select_pin

Type: boolean

Returns true if the pin is the select pin of a multiplexer device.

is_negative_level_sensitive_clock_pin

Type: boolean

Returns true if the pin is used as a negative level-sensitive latch clock pin.

is_negative_level_sensitive_data_pin

Type: boolean

Returns true if the pin is used as a negative level-sensitive latch data pin.

is_port

Type: boolean

Returns true for a pin or a port. Pins or ports are accessible only from a timing_point object.

is_positive_level_sensitive_clock_pin

Type: boolean

Returns true if the pin is used as a positive level-sensitive latch clock pin.

is_positive_level_sensitive_data_pin

Type: boolean

Feedback

Returns true if the pin is used as a positive level-sensitive latch data pin.

is_preset_pin

Type: boolean

Returns true if the pin is an asynchronous preset pin.

is_receiver_scaled_max_fall

Type: boolean

Returns true if the pin uses scaling between libraries for the receiver model for maximum fall analysis.

is_receiver_scaled_max_rise

Type: boolean

Returns true if the pin uses scaling between libraries for the receiver model for maximum rise analysis.

is_receiver_scaled_min_fall

Type: boolean

Returns true if the pin uses scaling between libraries for the receiver model for minimum fall analysis.

is_receiver_scaled_min_rise

Type: boolean

Returns true if the pin uses scaling between libraries for the receiver model for minimum rise analysis.

is_rise_edge_triggered_clock_pin

Type: boolean

Returns true if the pin is used as a rising-edge-triggered flip-flop clock pin.

is_rise_edge_triggered_data_pin

Type: boolean

Returns true if the pin is used as a rising-edge-triggered flip-flop data pin.

is_side_input

Type: boolean

Feedback

Returns true if the pin is inside the HyperScale block and visible at HyperScale top, where the entire fanin (starting from internal registers) is removed, and valid slews, arrivals, or case values are annotated.

is_stub

Type: boolean

Returns true if the pin is inside the HyperScale block and visible at HyperScale top, where the entire fanout (ending at internal registers) is removed, and valid required times are annotated for accurate slack computation.

is_three_state

Type: boolean

Returns true if the pin is a three-state driver.

is_three_state_enable_pin

Type: boolean

Returns true if the pin is an enable pin of a three-state device.

is_three_state_output_pin

Type: boolean

Returns true if the pin could output a three-state signal.

is_user_abstracted

Type: boolean

Returns true if the pin is abstracted because of user-specified settings (set_port_abstraction -ignore) in the HyperScale model. This attribute applies only to ports at the block level and corresponding pins at the top level. This attribute is available only after update_timing at the block level. Ports with is_user_abstracted == true also have is_abstracted == true.

launch_clocks

Type: collection

Returns a collection of the clocks that launch signals reaching the pin.

lib_pin

Type: collection

Returns a collection of library pins for this pin; this attribute is defined only on pins of leaf cells.

Feedback

lib_pin_name

Type: string

Returns the leaf pin name. For example, the lib_pin_name of pin U2/U1/Z is Z. This attribute is read-only.

max_arrival

Type: float

Returns the maximum of max_fall_arrival and max_rise_arrival.

max_capacitance

Type: float

Returns the maximum capacitance design rule limit defined directly at an output pin. This attribute returns the most restrictive requirement defined directly at the leaf pin or its underlying library pin (user-defined or library-defined). It does not consider requirements inherited from clock-specific or design-specific *set_max_capacitance* specifications. To also consider inherited requirements, use the *drc_constraining_max_capacitance* attribute instead.

max_fall_arrival

Type: float

Returns the arrival time for the longest path with a falling transition on a pin. In best-case, worst-case mode, this value is for the worst-case operating condition.

max_fall_delay

Type: float

Returns the maximum falling delay on ports, clocks, pins, cells, or on paths between such objects. Set with the set_max_delay command.

max_fall_local_slack

Type: float

For a pin of a transparent latch, the local slack is the timing slack considering the data arrival at the local pin and setup constraints at the local pin, without considering constraints downstream from the pin. If there are no constraints at the pin, the attribute is undefined. The normal slack (max_fall_slack or max_slack) considers the constraints downstream from the pin.

max_fall_slack

Type: float

Feedback

Returns the worst slack at a pin for falling maximum path delays. This attribute is valid only after timing has been updated. If the *timing_save_pin_arrival_and_slack* variable is *false* (the default), the attribute is valid only for path endpoints (register data pins and primary outputs). Switching the variable to *true* allows the attribute to be queried at all pins; in this case, the worst slack among all paths traversing a pin is returned.

max_fanout

Type: float

Returns the maximum fanout design rule limit for a pin.

max_rise_arrival

Type: float

Returns the arrival time for the longest path with a rising transition on a pin. In best-case, worst-case mode, this value is for the worst-case operating condition.

max_rise_delay

Type: float

Returns the maximum rising delay on ports, clocks, pins, cells, or on paths between such objects. Set with the set_max_delay command.

max_rise_local_slack

Type: float

For a pin of a transparent latch, the local slack is the timing slack considering the data arrival at the local pin and setup constraints at the local pin, without considering constraints downstream from the pin. If there are no constraints at the pin, the attribute is undefined. The normal slack (max_rise_slack or max_slack) considers the constraints downstream from the pin.

max_rise_slack

Type: float

Returns the worst slack at a pin for rising maximum path delays. This attribute is valid only after timing has been updated. If the *timing_save_pin_arrival_and_slack* variable is *false* (the default), the attribute is valid only for path endpoints (register data pins and primary outputs). Switching the variable to *true* allows the attribute to be queried at all pins; in this case, the worst slack among all paths traversing a pin is returned.

max_slack

Type: float

Returns the minimum of the max_rise_slack and max_fall_slack attributes.

Feedback

max_time_borrow

Type: float

Returns a floating-point number that establishes an upper limit for time borrowing; that is, it prevents the use of the entire pulse width for level-sensitive latches. Units are those used in the logic library. Set with the set_max_time_borrow command.

max_transition

Type: float

Returns the maximum transition time design rule limit defined directly at a pin. This attribute returns the most restrictive requirement defined directly at the leaf pin or its underlying library pin (user-defined or library-defined). It does not consider requirements inherited from clock-specific or design-specific *set_max_transition* specifications. To also consider inherited requirements, use the *constraining_max_transition* attribute instead.

min_arrival

Type: float

Returns the minimum of min_fall_arrival and min_rise_arrival.

min_capacitance

Type: float

Returns the minimum capacitance design rule limit for a pin.

min_fall_arrival

Type: float

Returns the arrival time for the shortest path with a falling transition on a pin. In best-case worst-case mode, this value is for the best-case mode.

min_fall_delay

Type: float

Returns the minimum falling delay on clocks, pins, cells, or on paths between such objects. Set with the set_min_delay command.

min_fall_slack

Type: float

Returns the worst slack at a pin for falling minimum path delays. This attribute is valid only after timing has been updated. If the *timing_save_pin_arrival_and_slack* variable is *false* (the default), the attribute is valid only for path endpoints (register data pins and primary

Feedback

outputs). Switching the variable to *true* allows the attribute to be queried at all pins; in this case, the worst slack among all paths traversing a pin is returned.

min_fanout

Type: float

Returns the minimum fanout design rule limit for a pin.

min_launch_clock_period

Type: float

Returns the minimum period among the clocks that launch signals reaching the pin, i.e. among the clocks of the ‘launch_clocks’ pin attribute.

min_rise_arrival

Type: float

Returns the worst hold slack of all paths passing through a pin with a rising transition at a pin. In best-case, worst-case operating conditions, this value is for the best-case condition. If all such paths are unconstrained, the value is infinity.

min_rise_delay

Type: float

Returns the minimum rising delay on ports, clocks, pins, cells, or on paths between such objects. Set with the set_min_delay command.

min_rise_slack

Type: float

Returns the worst slack at a pin for rising minimum path delays. This attribute is valid only after timing has been updated. If the *timing_save_pin_arrival_and_slack* variable is *false* (the default), the attribute is valid only for path endpoints (register data pins and primary outputs). Switching the variable to *true* allows the attribute to be queried at all pins; in this case, the worst slack among all paths traversing a pin is returned.

min_slack

Type: float

Returns the minimum of the min_rise_slack and min_fall_slack attributes.

min_transition

Type: float

Returns the minimum transition time design rule limit for a pin.

Feedback

net

Type: collection

Returns a collection that contains the net connected to this pin; this attribute is defined only if the pin is connected to a net.

object_class

Type: string

Returns the class of the object, which is "pin". You cannot set this attribute.

pg_level

Type: string

Returns PG connectivity status of the pin. A return value '1' means the pin has a connection to a power PG net or logic constant 1, whereas a return value '0' means the pin has a connection to a ground PG net or logic constant 0.

pin_capacitance_max

Type: float

Returns the capacitance of a pin for maximum conditions.

pin_capacitance_max_fall

Type: float

Returns the maximum fall capacitance of the pin. This attribute is read-only; you cannot change the setting.

pin_capacitance_max_rise

Type: float

Returns the maximum rise capacitance of the pin. This attribute is read-only; you cannot change the setting.

pin_capacitance_min

Type: float

Returns the capacitance of a pin for minimum conditions.

pin_capacitance_min_fall

Type: float

Returns the minimum fall capacitance of the pin. This attribute is read-only; you cannot change the setting.

Feedback

pin_capacitance_min_rise

Type: float

Returns the minimum rise capacitance of the pin. This attribute is read-only; you cannot change the setting.

pin_direction

Type: string

Returns the direction of the pin. Value can be in, out, inout, or internal. This attribute exists for backward compatibility with dc_shell. See the "direction" attribute.

power_base_clock

Type: string

Returns the name of the base clock associated with this pin. If the pin belongs to multiple clock domains, the power_base_clock attribute is set to the fastest of the clocks.

power_rail_voltage_bidir_input_max

Type: float

Returns the input voltage of a bidirectional pin.

power_rail_voltage_bidir_input_min

Type: float

Returns the input voltage of a bidirectional pin.

power_rail_voltage_max

Type: float

Returns the maximum power rail voltage set on the pin. In the case of a bidirectional pin, the output voltage is returned by default. To return the input voltage of a bidirectional pin, query the power_rail_voltage_bidir_input_max attribute.

power_rail_voltage_min

Type: float

Returns the minimum power rail voltage set on the pin. In the case of a bidirectional pin, the output voltage is returned by default. To return the input voltage of a bidirectional pin, query the power_rail_voltage_bidir_input_min attribute.

propagated_clock

Type: boolean

Feedback

Returns true if clock edge times are delayed by propagating the values through the clock network. If this attribute is not present, ideal clocking is assumed. Set with set_propagated_clock.

rc_input_threshold_pct_fall_max

Type: float

Returns the characterization trip point (waveform measurement threshold) that the tool uses to calculate delays and transition times. This attribute on an instance pin returns the library threshold value obtained from the maximum library in a min-max analysis.

rc_input_threshold_pct_fall_min

Type: float

Returns the characterization trip point (waveform measurement threshold) that the tool uses to calculate delays and transition times. This attribute on an instance pin returns the library threshold value obtained from the maximum library in a min-max analysis.

rc_input_threshold_pct_rise_max

Type: float

Returns the characterization trip point (waveform measurement threshold) that the tool uses to calculate delays and transition times. This attribute on an instance pin returns the library threshold value obtained from the maximum library in a min-max analysis.

rc_input_threshold_pct_rise_min

Type: float

Returns the characterization trip point (waveform measurement threshold) that the tool uses to calculate delays and transition times. This attribute on an instance pin returns the library threshold value obtained from the maximum library in a min-max analysis.

rc_output_threshold_pct_fall_max

Type: float

Returns the characterization trip point (waveform measurement threshold) that the tool uses to calculate delays and transition times. This attribute on an instance pin returns the library threshold value obtained from the maximum library in a min-max analysis.

rc_output_threshold_pct_fall_min

Type: float

Returns the characterization trip point (waveform measurement threshold) that the tool uses to calculate delays and transition times. This attribute on an instance pin returns the library threshold value obtained from the maximum library in a min-max analysis.

Feedback

rc_output_threshold_pct_rise_max

Type: float

Returns the characterization trip point (waveform measurement threshold) that the tool uses to calculate delays and transition times. This attribute on an instance pin returns the library threshold value obtained from the maximum library in a min-max analysis.

rc_output_threshold_pct_rise_min

Type: float

Returns the characterization trip point (waveform measurement threshold) that the tool uses to calculate delays and transition times. This attribute on an instance pin returns the library threshold value obtained from the maximum library in a min-max analysis.

rc_slew_derate_from_library_max

Type: float

Returns the slew derating factor that the tool uses to calculate delays and transition times. This attribute on an instance pin returns the library threshold value obtained from the maximum library in a min-max analysis.

rc_slew_derate_from_library_min

Type: float

Returns the slew derating factor that the tool uses to calculate delays and transition times. This attribute on an instance pin returns the library threshold value obtained from the maximum library in a min-max analysis.

rc_slew_lower_threshold_pct_fall_max

Type: float

Returns the characterization trip point (waveform measurement threshold) that the tool uses to calculate delays and transition times. This attribute on an instance pin returns the library threshold value obtained from the maximum library in a min-max analysis.

rc_slew_lower_threshold_pct_fall_min

Type: float

Returns the characterization trip point (waveform measurement threshold) that the tool uses to calculate delays and transition times. This attribute on an instance pin returns the library threshold value obtained from the maximum library in a min-max analysis.

rc_slew_lower_threshold_pct_rise_max

Type: float

Feedback

Returns the characterization trip point (waveform measurement threshold) that the tool uses to calculate delays and transition times. This attribute on an instance pin returns the library threshold value obtained from the maximum library in a min-max analysis.

rc_slew_lower_threshold_pct_rise_min

Type: float

Returns the characterization trip point (waveform measurement threshold) that the tool uses to calculate delays and transition times. This attribute on an instance pin returns the library threshold value obtained from the maximum library in a min-max analysis.

rc_slew_upper_threshold_pct_fall_max

Type: float

Returns the characterization trip point (waveform measurement threshold) that the tool uses to calculate delays and transition times. This attribute on an instance pin returns the library threshold value obtained from the maximum library in a min-max analysis.

rc_slew_upper_threshold_pct_fall_min

Type: float

Returns the characterization trip point (waveform measurement threshold) that the tool uses to calculate delays and transition times. This attribute on an instance pin returns the library threshold value obtained from the maximum library in a min-max analysis.

rc_slew_upper_threshold_pct_rise_max

Type: float

Returns the characterization trip point (waveform measurement threshold) that the tool uses to calculate delays and transition times. This attribute on an instance pin returns the library threshold value obtained from the maximum library in a min-max analysis.

rc_slew_upper_threshold_pct_rise_min

Type: float

Returns the characterization trip point (waveform measurement threshold) that the tool uses to calculate delays and transition times. This attribute on an instance pin returns the library threshold value obtained from the maximum library in a min-max analysis.

receiver_model_scaling_libs_max

Type: collection

Returns a collection of library objects used for receiver model scaling, where applicable, for libs maximum analysis.

Feedback

receiver_model_scaling_libs_min

Type: collection

Returns a collection of library objects used for receiver model scaling, where applicable, for libs minimum analysis.

receiver_model_type_max_fall

Type: string

Returns the receiver model type, basic (NLDM) or advanced (CCS timing), for maximum fall analysis.

receiver_model_type_max_rise

Type: string

Returns the receiver model type, basic (NLDM) or advanced (CCS timing), for maximum rise analysis.

receiver_model_type_min_fall

Type: string

Returns the receiver model type, basic (NLDM) or advanced (CCS timing), for minimum fall analysis.

receiver_model_type_min_rise

Type: string

Returns the receiver model type, basic (NLDM) or advanced (CCS timing), for minimum rise analysis.

relative_toggle_rate

Type: double

Returns the number of toggles per period of the power_base_clock; if the design does not have a defined clock, this attribute is undefined.

setup_uncertainty

Type: float

Returns the clock uncertainty (skew) of a clock used for setup (and other maximum delay) timing checks. Set with set_clock_uncertainty.

si_noise_active_aggressors_above_high

Type: collection

Feedback

Returns a collection of a subset of effective aggressors that contributed to the worst case alignment to have largest effect on noise bump height.

si_noise_active_aggressors_above_low

Type: collection

Returns a collection of a subset of effective aggressors that contributed to the worst case alignment to have largest effect on noise bump height.

si_noise_active_aggressors_below_high

Type: collection

Returns a collection of a subset of effective aggressors that contributed to the worst case alignment to have largest effect on noise bump height.

si_noise_active_aggressors_below_low

Type: collection

Returns a collection of a subset of effective aggressors that contributed to the worst case alignment to have largest effect on noise bump height.

si_noise_bumps_above_high

Type: string

Returns a string that lists the aggressor nets for the input pin and their corresponding coupled bump heights and widths, considering noise bumps in the above-high region. The format of the string is:

```
{{aggr1_name height width}
 {aggr2_name height width}
  ... }
```

Height is in volts and width is in library time units.

si_noise_bumps_above_low

Type: string

Returns a string that lists the aggressor nets for the input pin and their corresponding coupled bump heights and widths, considering noise bumps in the above-low region. The format of the string is:

```
{{aggr1_name height width}
 {aggr2_name height width}
  ... }
```

Height is in volts and width is in library time units.

Feedback

si_noise_bumps_below_high

Type: string

Returns a string that lists the aggressor nets for the input pin and their corresponding coupled bump heights and widths, considering noise bumps in the below-high region. The format of the string is:

```
{{aggr1_name height width}
 {aggr2_name height width}
  ... }
```

Height is in volts and width is in library time units.

si_noise_bumps_below_low

Type: string

Returns a string that lists the aggressor nets for the input pin and their corresponding coupled bump heights and widths, considering noise bumps in the below-low region. The format of the string is:

```
{{aggr1_name height width}
 {aggr2_name height width}
  ... }
```

Height is in volts and width is in library time units.

si_noise_height_factor_above_high

Type: float

Returns the noise height derating factor for the pin in the above-high region, as set by the *set_noise_derate* command.

si_noise_height_factor_above_low

Type: float

Returns the noise height derating factor for the pin in the above-low region.

si_noise_height_factor_below_high

Type: float

Returns the noise height derating factor for the pin in the below-high region.

si_noise_height_factor_below_low

Type: float

Returns the noise height derating factor for the pin in the below-low region.

Feedback

si_noise_height_offset_above_high

Type: float

Returns the noise height offset for the pin in the above-high region, as set by the *set_noise_derate* command.

si_noise_height_offset_above_low

Type: float

Returns the noise height offset for the pin in the above-low region.

si_noise_height_offset_below_high

Type: float

Returns the noise height offset for the pin in the below-high region.

si_noise_height_offset_below_low

Type: float

Returns the noise height offset for the pin in the below-low region.

si_noise_lib_pin_name

Type: string

Returns the name of the library pin of equivalent library cell specified for noise analysis.

si_noise_prop_bumps_above_high

Type: string

Returns a string that shows the bump height and width at the input pin caused by noise propagation in the above-high region. The format of the string is: {*height width*}. Height is in volts and width is in library time units.

si_noise_prop_bumps_above_low

Type: string

Returns a string that shows the bump height and width at the input pin caused by noise propagation in the above-low region. The format of the string is: {*height width*}. Height is in volts and width is in library time units.

si_noise_prop_bumps_below_high

Type: string

Returns a string that shows the bump height and width at the input pin caused by noise propagation in the below-high region. The format of the string is: {*height width*}. Height is in volts and width is in library time units.

si_noise_prop_bumps_below_low

Type: string

Returns a string that shows the bump height and width at the input pin caused by noise propagation in the below-low region. The format of the string is: {*height width*}. Height is in volts and width is in library time units.

si_noise_slack_above_high

Type: float

Returns the amount of noise slack for the pin in the above-high region.

si_noise_slack_above_low

Type: float

Returns the amount of noise slack for the pin in the above-low region.

si_noise_slack_below_high

Type: float

Returns the amount of noise slack for the pin in the below-high region.

si_noise_slack_below_low

Type: float

Returns the amount of noise slack for the pin in the below-low region.

si_noise_total_bump_above_high

Type: string

Returns a string that shows the total bump height and width at the input pin caused by crosstalk and noise propagation in the above-high region. The format of the string is: {*height width*}. Height is in volts and width is in library time units.

si_noise_total_bump_above_low

Type: string

Returns a string that shows the total bump height and width at the input pin caused by crosstalk and noise propagation in the above-low region. The format of the string is: {*height width*}. Height is in volts and width is in library time units.

Feedback

si_noise_total_bump_below_high

Type: string

Returns a string that shows the total bump height and width at the input pin caused by crosstalk and noise propagation in the below-high region. The format of the string is: {*height width*}. Height is in volts and width is in library time units.

si_noise_total_bump_below_low

Type: string

Returns a string that shows the total bump height and width at the input pin caused by crosstalk and noise propagation in the below-low region. The format of the string is: {*height width*}. Height is in volts and width is in library time units.

si_noise_width_factor_above_high

Type: float

Returns the noise width derating factor for the pin in the above-high region.

si_noise_width_factor_above_low

Type: float

Returns the noise width derating factor for the pin in the above-low region.

si_noise_width_factor_below_high

Type: float

Returns the noise width derating factor for the pin in the below-high region.

si_noise_width_factor_below_low

Type: float

Returns the noise width derating factor for the pin in the below-low region.

si_noise_worst_prop_arc_above_high

Type: collection

Returns the cell arc that corresponds to the worst noise propagation to an output pin of a cell.

si_noise_worst_prop_arc_above_low

Type: collection

Returns the cell arc that corresponds to the worst noise propagation to an output pin of a cell.

Feedback

si_noise_worst_prop_arc_below_high

Type: collection

Returns the cell arc that corresponds to the worst noise propagation to an output pin of a cell.

si_noise_worst_prop_arc_below_low

Type: collection

Returns the cell arc that corresponds to the worst noise propagation to an output pin of a cell.

static_probability

Type: float

Returns the static probability that the pin has the logic value 1.

temperature_max

Type: float

Returns the maximum temperature for the cell. This value is set by the operating condition specification or the set_temperature command.

temperature_min

Type: float

Returns the minimum temperature for the cell. This value is set by the operating condition specification or the set_temperature command.

toggle_rate

Type: double

Returns the rate at which transitions occur on the pin within a time period.

user_case_value

Type: string

Returns the user-specified logic value of a pin or port.

voltage_source

Type: string

Feedback

Returns a keyword that indicates the source of the pin's supply voltage value:

- *DCALM* - define_cell_alternative_lib_mapping specification
- *SET_VOLTAGE* - explicit set_voltage specification on pg_pin
- *CONNECTED_SUPPLY_NET* - connected supply net
- *CONNECTED_SUPPLY_GROUP* - derived from supply group
- *DESIGN_OC* - design operating condition
- *EXTENDED_OC* - cell operating condition
- *VOLTAGE_MAP* - voltage map from library
- *NOMINAL_LIB_VOLTAGE* - default library voltages
- *BEST_MATCH* - best match library voltage

worst_max_time_borrow_fall

Type: float

Returns the maximum time that could be potentially borrowed across all incoming paths to the latch and across all capturing clocks at the latch. The value is the most restrictive of the following values:

- Closing edge
- Value specified by the *set_max_time_borrow* command

The attribute returns

- Zero if the endpoint is not a transparent latch D pin
- Path corner of the distribution of the actual variation quantity if the design uses parametric on-chip variation (POCV)

worst_max_time_borrow_rise

Type: float

Returns the maximum time that could be potentially borrowed across all incoming paths to the latch and across all capturing clocks at the latch. The value is the most restrictive of the following values:

- Closing edge
- Value specified by the *set_max_time_borrow* command

The attribute returns

Feedback

- Zero if the endpoint is not a transparent latch D pin
- Path corner of the distribution of the actual variation quantity if the design uses parametric on-chip variation (POCV)

x_coordinate

Type: float

Returns the x-coordinate of the pin.

y_coordinate

Type: float

Returns the y-coordinate of the pin.

See Also

- get_attribute
- help_attributes
- list_attributes
- report_attribute
- attributes

---

port_attributes

Describes the predefined application attributes for port objects.

Description

actual_fall_transition_max

Type: float

Returns the largest falling transition time for the port. You cannot set this attribute.

actual_fall_transition_min

Type: float

Returns the smallest falling transition time for the port. You cannot set this attribute.

actual_min_clock_pulse_width_high

Type: string

Feedback

Returns a string containing a per-clock actual minimum pulse width value at the port (high pulse).

actual_min_clock_pulse_width_low

Type: string

Returns a string containing a per-clock actual minimum pulse width value at the port (low pulse).

actual_rise_transition_max

Type: float

Returns the largest rising transition time for the port. You cannot set this attribute.

actual_rise_transition_min

Type: float

Returns the smallest rising transition time for the port. You cannot set this attribute.

actual_transition_max

Type: float

Returns the maximum of the actual_rise_transition_max and actual_fall_transition_max attributes.

actual_transition_min

Type: float

Returns the minimum of the actual_rise_transition_min and actual_fall_transition_min attributes.

annotated_fall_transition_delta_max

Type: float

Returns the additional transition time added to the maximum falling transition time on the port. The additional transition time is set by either the set_annotated_transition -delta_only command or by PrimeTime SI during crosstalk analysis.

annotated_fall_transition_delta_min

Type: float

Returns the additional transition time added to the minimum falling transition time on the port. The additional transition time is set by either the set_annotated_transition -delta_only command or by PrimeTime SI during crosstalk analysis.

Feedback

annotated_rise_transition_delta_max

Type: float

Returns the additional transition time added to the maximum rising transition time on the port. The additional transition time is set by either the set_annotated_transition -delta_only command or by PrimeTime SI during crosstalk analysis.

annotated_rise_transition_delta_min

Type: float

Returns the additional transition time added to the minimum rising transition time on the port. The additional transition time is set by either the set_annotated_transition -delta_only command or by PrimeTime SI during crosstalk analysis.

annotated_transition_delta_max

Type: float

Returns the maximum of the annotated_rise_transition_delta_max and annotated_fall_transition_delta_max attributes.

annotated_transition_delta_min

Type: float

Returns the minimum of the annotated_rise_transition_delta_min and annotated_fall_transition_delta_min attributes.

arrival_window

Type: string

Returns the minimum and maximum arrivals for rising and falling transitions.

cached_c1_max_fall

Type: float

Returns the value of C1 (first capacitance of CCS receiver model) stored during the delay calculation.

cached_c1_max_rise

Type: float

Returns the value of C1 (first capacitance of CCS receiver model) stored during the delay calculation.

cached_c1_min_fall

Type: float

Feedback

Returns the value of C1 (first capacitance of CCS receiver model) stored during the delay calculation.

cached_c1_min_rise

Type: float

Returns the value of C1 (first capacitance of CCS receiver model) stored during the delay calculation.

cached_c2_max_fall

Type: float

Returns the value of C2 (second capacitance of CCS receiver model) stored during the delay calculation.

cached_c2_max_rise

Type: float

Returns the value of C2 (second capacitance of CCS receiver model) stored during the delay calculation.

cached_c2_min_fall

Type: float

Returns the value of C2 (second capacitance of CCS receiver model) stored during the delay calculation.

cached_c2_min_rise

Type: float

Returns the value of C2 (second capacitance of CCS receiver model) stored during the delay calculation.

cached_ceff_max_fall

Type: float

Worst effective capacitance of a net with detailed parasitics stored during the delay calculation that is connected to this input port. This requires the rc_cache_min_max_rise_fall_ceff variable to be set to true before the timing update.

cached_ceff_max_rise

Type: float

Feedback

Worst effective capacitance of a net with detailed parasitics stored during the delay calculation that is connected to this input port. This requires the rc_cache_min_max_rise_fall_ceff variable to be set to true before the timing update.

cached_ceff_min_fall

Type: float

Worst effective capacitance of a net with detailed parasitics stored during the delay calculation that is connected to this input port. This requires the rc_cache_min_max_rise_fall_ceff variable to be set to true before the timing update.

cached_ceff_min_rise

Type: float

Worst effective capacitance of a net with detailed parasitics stored during the delay calculation that is connected to this input port. This requires the rc_cache_min_max_rise_fall_ceff variable to be set to true before the timing update.

case_value

Type: string

Returns the user-specified logic value of a pin or port propagated from a case analysis or logic constant.

ceff_params_max

Type: string

Returns the parameters used to find the maximum effective capacitance for each timing arc feeding a driver pin attached to an annotated RC network. The parameters specify a linear driver model (rd = drive resistance, t0 = start of voltage ramp, delta_t = duration of voltage ramp, ceff = effective capacitance).

ceff_params_min

Type: string

Returns the parameters used to find the minimum effective capacitance for each timing arc feeding a driver pin attached to an annotated RC network. The parameters specify a linear driver model (rd = drive resistance, t0 = start of voltage ramp, delta_t = duration of voltage ramp, ceff = effective capacitance).

clock_capture_arrival_dynamic

Type: string

Returns the arrival time on a clock capture path endpoint including the dynamic arrival time. The dynamic arrival time is modeled using the set_clock_latency -dynamic

Feedback

command. If the clock source latency is modeled including the dynamic latency (set_clock_latency -source -dynamic), the static arrival attributes exclude the dynamic value. To query this attribute, you need to enable PrimeTime SI analysis.

clock_capture_arrival_static

Type: string

Returns the static arrival time on a clock capture path endpoint. The tool calculates the static arrival time as the sum of the clock source latency (specified by the set_clock_latency -source command) and the clock network latency.

clock_latency_fall_max

Type: float

Returns the user-specified maximum fall latency (insertion delay) for clock networks through the port. Set with the set_clock_latency command.

clock_latency_fall_min

Type: float

Returns the user-specified minimum fall latency (insertion delay) for clock networks through the port. Set with the set_clock_latency command.

clock_latency_rise_max

Type: float

Returns the user-specified maximum rise latency (insertion delay) for clock networks through the port. Set with the set_clock_latency command.

clock_latency_rise_min

Type: float

Returns the user-specified minimum rise latency (insertion delay) for clock networks through the port. Set with the set_clock_latency command.

clock_launch_arrival_dynamic

Type: string

Returns the arrival time on a clock launch path endpoint including the dynamic arrival time. The dynamic arrival time is modeled using the set_clock_latency -dynamic command. If the clock source latency is modeled including the dynamic latency (set_clock_latency -source -dynamic), the static arrival attributes exclude the dynamic value. To query this attribute, you need to enable PrimeTime SI analysis.

Feedback

clock_launch_arrival_static

Type: string

Returns the static arrival time on a clock launch path endpoint. The tool calculates the static arrival time as the sum of the clock source latency (specified by the set_clock_latency -source command) and the clock network latency.

clock_source_latency_early_fall_max

Type: float

Returns the maximum early falling source latency. Set with the set_clock_latency command.

clock_source_latency_early_fall_min

Type: float

Returns the minimum early falling source latency. Set with the set_clock_latency command.

clock_source_latency_early_rise_max

Type: float

Returns the maximum early rising source latency. Set with the set_clock_latency command.

clock_source_latency_early_rise_min

Type: float

Returns the minimum early rising source latency. Set with the set_clock_latency command.

clock_source_latency_late_fall_max

Type: float

Returns the maximum late falling source latency. Set with the set_clock_latency command.

clock_source_latency_late_fall_min

Type: float

Returns the minimum late falling source latency. Set with the set_clock_latency command.

clock_source_latency_late_rise_max

Type: float

Returns the maximum late rising source latency. Set with the set_clock_latency command.

Feedback

clock_source_latency_late_rise_min

Type: float

Returns the minimum late rising source latency. Set with the set_clock_latency command.

clocks

Type: collection

Returns a collection of clock objects which propagate through this port. It is undefined if no clocks are present.

connection_class

Type: string

Returns the connection class string for the port.

constant_value

Type: string

Returns the logic value of the port tied to logic constant zero or one in the netlist.

constraining_max_transition

Type: float

Returns the most constraining maximum transition value from all sources at the port.

drc_actual_max_capacitance

Type: float

Returns the actual capacitance value of the driven net at a driver (input or inout) port. This is the value used by *report_constraint* for maximum capacitance DRC checking. This attribute is undefined for output ports.

drc_actual_max_transition

Type: float

Returns the actual transition value at the pin. This is the value used by *report_constraint* for maximum transition DRC checking. It is affected by the *si_xtalk_max_transition_mode variable and the timing_pocvm_max_transition_sigma variable (in a POCV analysis).*

drc_actual_min_capacitance

Type: float

Feedback

Returns the actual capacitance value of the driven net at a driver (input or inout) port. This is the value used by *report_constraint* for minimum capacitance DRC checking. This attribute is undefined for output ports.

drc_constraining_max_capacitance

Type: float

Returns the most constraining maximum capacitance value from all sources at a driver (input or inout) port. This attribute is undefined for output ports.

drc_constraining_min_capacitance

Type: float

Returns the most constraining minimum capacitance value from all sources at a driver (input or inout) port. This attribute is undefined for output ports.

drc_coupled_actual_max_transition

Type: float

This attribute is similar to the *drc_actual_max_transition* attribute, except that coupled delta delays are always included (equivalent to setting the *si_xtalk_max_transition_mode variable to reliability).*

drc_max_capacitance_slack

Type: float

Returns the capacitance slack at the pin for maximum condition DRC checking purposes.

drc_max_transition_slack

Type: float

Returns the transition slack at the pin for maximum condition DRC checking purposes.

drc_min_capacitance_slack

Type: float

Returns the capacitance slack at the pin for minimum condition DRC checking purposes.

direction

Type: string

Returns the direction of the port. Value can be in, out, inout, or internal. The port_direction attribute is a synonym for direction. You cannot set this attribute.

Feedback

disable_timing

Type: boolean

Returns true if the timing for the port has been marked as disabled with the set_disable_timing command.

drive_resistance_fall_max

Type: float

Returns the linear drive resistance for falling delays and maximum conditions, associated with an input or inout port. Set with the set_drive command.

drive_resistance_fall_min

Type: float

Returns the linear drive resistance for falling delays and minimum conditions, associated with an input or inout port. Set with the set_drive command.

drive_resistance_rise_max

Type: float

Returns the linear drive resistance for rising delays and maximum conditions, associated with an input or inout port. Set with the set_drive command.

drive_resistance_rise_min

Type: float

Returns the linear drive resistance for rising delays and minimum conditions, associated with an input or inout port. Set with the set_drive command.

driver_model_scaling_libs_max

Type: collection

Returns a collection of library objects used for driver model scaling, where applicable, for maximum analysis.

driver_model_scaling_libs_min

Type: collection

Returns a collection of library objects used for driver model scaling, where applicable, for minimum analysis.

driver_model_type_max_fall

Type: string

Feedback

Returns the driver model type, basic (NLDM) or advanced (CCS timing), for maximum fall analysis.

driver_model_type_max_rise

Type: string

Returns the driver model type, basic (NLDM) or advanced (CCS timing), for maximum rise analysis.

driver_model_type_min_fall

Type: string

Returns the driver model type, basic (NLDM) or advanced (CCS timing), for minimum fall analysis.

driver_model_type_min_rise

Type: string

Returns the driver model type, basic (NLDM) or advanced (CCS timing), for minimum rise analysis.

driving_cell_fall_max

Type: string

Returns a library cell from which to copy maximum fall drive capability to be used in fall transition calculation for the port. Set with the set_driving_cell command.

driving_cell_fall_min

Type: string

Returns a library cell from which to copy the minimum fall drive capability to be used in fall transition calculation for the port. Set with the set_driving_cell command.

driving_cell_from_pin_fall_max

Type: string

Returns the driving_cell_fall_max input pin to be used to find timing arc maximum fall drive capability. Set with the set_driving_cell command.

driving_cell_from_pin_fall_min

Type: string

Returns the driving_cell_fall_min input pin to be used to find timing arc minimum fall drive capability. Set with the set_driving_cell command.

Feedback

driving_cell_from_pin_rise_max

Type: string

Returns the driving_cell_rise_max input pin to be used to find timing arc rise drive capability. Set with the set_driving_cell command.

driving_cell_from_pin_rise_min

Type: string

Returns the driving_cell_rise_min input pin to be used to find timing arc rise drive capability. Set with the set_driving_cell command.

driving_cell_library_fall_max

Type: string

Returns the library in which to find the driving_cell_fall_max. Set with the set_driving_cell command.

driving_cell_library_fall_min

Type: string

Returns the library in which to find the driving_cell_fall_min. Set with the set_driving_cell command.

driving_cell_library_rise_max

Type: string

Returns the library in which to find the driving_cell_rise_max. Set with the set_driving_cell command.

driving_cell_library_rise_min

Type: string

Returns the library in which to find the driving_cell_rise_min. Set with the set_driving_cell command.

driving_cell_max_fall_itrans_fall

Type: float

Returns the value of the maximum input transition for the driving cell that was associated with the port by the set_driving_cell command. This attribute represents the falling transition at the from_pin of the driving cell that is used to compute the falling transition value at a pin that drives the port.

Feedback

driving_cell_max_fall_itrans_rise

Type: float

Returns the value of the maximum input transition for the driving cell that was associated with a port by the set_driving_cell command. This attribute represents the falling transition value at the from_pin of the driving cell that is used to compute the rising transition value at the pin that drives the port.

driving_cell_max_rise_itrans_fall

Type: float

Returns the value of the maximum input transition for the driving cell that was associated with a port by the set_driving_cell command. This attribute represents the rising transition value at the from_pin of the driving cell that is used to compute falling transition value at the pin that drives the port.

driving_cell_max_rise_itrans_rise

Type: float

Returns the value of the maximum input transition for the driving cell that was associated with a port by the set_driving_cell command. This attribute represents the rising transition value at the from_pin of the driving cell that is used to compute the rising transition value at the pin that drives the port.

driving_cell_min_fall_itrans_fall

Type: float

Returns the value of the minimum input transition for the driving cell that was associated with a port by the set_driving_cell command. This attribute represents the minimum falling transition value at the from_pin of the driving cell that is used to compute the falling transition value at the pin that drives the port.

driving_cell_min_fall_itrans_rise

Type: float

Returns the value of the minimum input transition for the driving cell that was associated with a port by the set_driving_cell command. This attribute represents the minimum falling transition value at the from_pin of the driving cell that is used to compute the rising transition value at the pin that drives the port.

driving_cell_min_rise_itrans_fall

Type: float

Returns the value of the minimum input transition for the driving cell that was associated with a port by the set_driving_cell command. This attribute represents the minimum

Feedback

rise transition value at the from_pin of the driving cell that is used to compute the falling transition value at the pin that drives the port.

driving_cell_min_rise_itrans_rise

Type: float

Returns the value of the minimum input transition for the driving cell that was associated with a port by the set_driving_cell command. This attribute represents the minimum rise transition value at the from_pin of the driving cell that is used to compute the rising transition value at the pin that drives the port.

driving_cell_no_design_rule

Type: boolean

Returns true if driving cell information has been set on a port with set_driving_cell -no_design_rule. If true, the driving cell's design rule limits (max_capacitance and so forth) are not used for the port.

driving_cell_pin_fall_max

Type: string

Returns the driving_cell_fall_max output pin to be used to find timing arc fall drive capability. Set with the set_driving_cell command.

driving_cell_pin_fall_min

Type: string

Returns the driving_cell_fall_min output pin to be used to find timing arc fall drive capability. Set with the set_driving_cell command.

driving_cell_pin_rise_max

Type: string

Returns the driving_cell_rise_max output pin to be used to find timing arc rise drive capability. Set with the set_driving_cell command.

driving_cell_pin_rise_min

Type: string

Returns the driving_cell_rise_min output pin to be used to find timing arc rise drive capability. Set with the set_driving_cell command.

driving_cell_rise_max

Type: string

Feedback

Returns a library cell from which to copy maximum rise drive capability to be used in rise transition calculation for the port. Set with the set_driving_cell command.

driving_cell_rise_min

Type: string

Returns a library cell from which to copy the minimum rise drive capability to be used in rise transition calculation for the port. Set with the set_driving_cell command.

effective_capacitance_max

Type: float

Returns the effective capacitance for maximum operating conditions. Valid only for driver pins attached to annotated RC networks.

effective_capacitance_min

Type: float

Returns the effective capacitance for minimum operating conditions. Valid only for driver pins attached to annotated RC networks.

escaped_full_name

Type: string

Returns the name of the cell. Any literal hierarchy characters are escaped with a backslash.

fanout_load

Type: float

Returns the fanout load on output ports. Set with the set_fanout_load command.

full_name

Type: string

Returns the name of a port. You cannot set this attribute.

has_valid_max_fall_transition

Type: boolean

Returns *true* if the max-fall transition is the startpoint for a valid delay calculation arc. Undriven pins and case analysis pins/ports (user-defined or constant-sourced) are initialized with placeholder zero transition values; pins/ports in these "placeholder transition" propagation regions also return *true*. This attribute returns *false* for pins/ports that receive an uninitialized transition value from a timing arc disabled by the

Feedback

set_disable_timing command, or that receive an uninitialized rise or fall transition value from a library cell with non-unate arcs that do not provide both rise and fall incoming transitions.

has_valid_max_rise_transition

Type: boolean

Returns *true* if the max-rise transition is the startpoint for a valid delay calculation arc. See the *has_valid_max_fall_transition* description for details.

has_valid_min_fall_transition

Type: boolean

Returns *true* if the min-fall transition is the startpoint for a valid delay calculation arc. See the *has_valid_max_fall_transition* description for details.

has_valid_min_rise_transition

Type: boolean

Returns *true* if the min-rise transition is the startpoint for a valid delay calculation arc. See the *has_valid_max_fall_transition* description for details.

hold_uncertainty

Type: float

Returns the clock uncertainty (skew) of a clock used for hold (and other minimum delay) timing checks. Set with the set_clock_uncertainty command.

ideal_latency_max_fall

Type: float

Returns the ideal delay value annotated on a port in an ideal network. To set this value, use the set_ideal_latency command.

ideal_latency_max_rise

Type: float

Returns the ideal delay value annotated on a port in an ideal network. To set this value, use the set_ideal_latency command.

ideal_latency_min_fall

Type: float

Returns the ideal delay value annotated on a port in an ideal network. To set this value, use the set_ideal_latency command.

Feedback

ideal_latency_min_rise

Type: float

Returns the ideal delay value annotated on a port in an ideal network. To set this value, use the set_ideal_latency command.

ideal_transition_max_fall

Type: float

Returns the ideal transition time annotated on a port in an ideal network. To set this value, use the set_ideal_transition command.

ideal_transition_max_rise

Type: float

Returns the ideal transition time annotated on a port in an ideal network. To set this value, use the set_ideal_transition command.

ideal_transition_min_fall

Type: float

Returns the ideal transition time annotated on a port in an ideal network. To set this value, use the set_ideal_transition command.

ideal_transition_min_rise

Type: float

Returns the ideal transition time annotated on a port in an ideal network. To set this value, use the set_ideal_transition command.

input_transition_fall_max

Type: float

Returns the fixed transition time for falling delays, maximum conditions associated with an input or inout port. Set with the set_input_transition command.

input_transition_fall_min

Type: float

Returns the fixed transition time for falling delays and minimum conditions associated with an input or inout port. Set with the set_input_transition command.

input_transition_rise_max

Type: float

Returns the fixed transition time for rising delays and maximum conditions associated with an input or inout port. Set with the set_input_transition command.

input_transition_rise_min

Type: float

Returns the fixed transition time for rising delays and minimum conditions associated with an input or inout port. Set with the set_input_transition command.

is_abstracted

Type: boolean

Returns true if the port is abstracted in the HyperScale model. This attribute applies only to ports at the block level and corresponding pins at the top level. This attribute is available only after update_timing at the block level. The attribute returns true for set_port_abstraction -ignore. Ports with is_user_abstracted == true are a subset of ports with is_abstracted == true.

is_clock_dont_override

Type: boolean

Returns true if context override is not applied because of context referring to unmapped clocks. This attribute applies only to ports at the block level.

is_clock_network

Type: boolean

Returns true if the port is a clock source or is in the combinational fanout of a clock source.

is_clock_source

Type: boolean

Returns true if a design clock was declared with a source at the port.

is_clock_source_network

Type: boolean

Returns true if the port is part of a clock source latency network.

is_clock_used_as_clock

Type: boolean

Returns true if the clock through the port acts as a clock.

Feedback

is_clock_used_as_data

Type: boolean

Returns true if the clock through the port acts as data.

is_drc_data_path

Type: boolean

Returns true if the port is considered as part of a data path during DRC checking.

is_driver_scaled_max_fall

Type: boolean

Returns true if the port uses scaling between libraries for the driver model for maximum fall analysis.

is_driver_scaled_max_rise

Type: boolean

Returns true if the port uses scaling between libraries for the driver model for maximum rise analysis.

is_driver_scaled_min_fall

Type: boolean

Returns true if the port uses scaling between libraries for the driver model for minimum fall analysis.

is_driver_scaled_min_rise

Type: boolean

Returns true if the port uses scaling between libraries for the driver model for minimum rise analysis.

is_ideal

Type: boolean

Returns true if the port has been marked ideal using the set_ideal_network command.

is_netlist_dont_override

Type: boolean

Returns true if context override is not applied because of anchor leaf pin change on a port net.

Feedback

is_receiver_scaled_max_fall

Type: boolean

Returns true if the port uses scaling between libraries for the receiver model for maximum fall analysis.

is_receiver_scaled_max_rise

Type: boolean

Returns true if the port uses scaling between libraries for the receiver model for maximum rise analysis.

is_receiver_scaled_min_fall

Type: boolean

Returns true if the port uses scaling between libraries for the receiver model for minimum fall analysis.

is_receiver_scaled_min_rise

Type: boolean

Returns true if the port uses scaling between libraries for the receiver model for minimum rise analysis.

is_user_abstracted

Type: boolean

Returns true if the port is abstracted because of user-specified settings (set_port_abstraction -ignore) in the HyperScale model. This attribute applies only to ports at the block level and corresponding pins at the top level. This attribute is available only update_timing at the block level. Ports with is_user_abstracted == true also have is_abstracted == true.

is_user_dont_override

Type: boolean

Returns true if context override is not applied because of user-specified settings. This attribute applies only to ports at the block level.

launch_clocks

Type: collection

Returns a collection of the clocks that launch signals reaching the port.

Feedback

max_capacitance

Type: float

Returns the maximum capacitance design rule limit defined directly at a driver (input or inout) port. This attribute considers only port-specific *set_max_capacitance* specifications. It does not consider requirements inherited from clock-specific or design-specific *set_max_capacitance* specifications. To also consider inherited requirements, use the *drc_constraining_max_capacitance* attribute instead.

max_fall_delay

Type: float

Returns the maximum falling delay on ports, clocks, pins, cells, or on paths between such objects. Set with the set_max_delay command.

max_fall_local_slack

Type: float

For a port of a transparent latch, the local slack is the timing slack considering the data arrival at the local port and setup constraints at the local port, without considering constraints downstream from the port. If there are no constraints at the port, the attribute is undefined. The normal slack (max_fall_slack or max_slack) considers the constraints downstream from the port.

max_fall_slack

Type: float

Returns the worst slack at a port for falling maximum path delays. This attribute is valid for path endpoints (register data pins and primary outputs) after timing has been updated. You cannot set this attribute.

max_fanout

Type: float

Returns the maximum fanout load for the net connected to this port. PrimeTime ensures that the fanout load on this port is less than the specified value. Set with the set_max_fanout command

max_rise_delay

Type: float

Returns the maximum rising delay on ports, clocks, pins, cells, or on paths between such objects. Set with the set_max_delay command.

Feedback

max_rise_local_slack

Type: float

For a port of a transparent latch, the local slack is the timing slack considering the data arrival at the local port and setup constraints at the local port, without considering constraints downstream from the port. If there are no constraints at the port, the attribute is undefined. The normal slack (max_rise_slack or max_slack) considers the constraints downstream from the port.

max_rise_slack

Type: float

Returns the worst slack at a port for rising maximum path delays. This attribute is valid for path endpoints (register data pins and primary outputs) after timing is updated. You cannot set this attribute.

max_slack

Type: float

Returns the minimum of the max_rise_slack and max_fall_slack attributes.

max_transition

Type: float

Returns the maximum transition time design rule limit defined directly at the port. This attribute considers only port-specific *set_max_transition* specifications. It does not consider requirements inherited from clock-specific or design-specific *set_max_transition* specifications. To also consider inherited requirements, use the *constraining_max_transition* attribute instead.

min_capacitance

Type: float

Returns the minimum capacitance value for input and bidirectional ports. The units must be consistent with those of the logic library used. Set with the set_min_capacitance command.

min_fall_delay

Type: float

Returns the minimum falling delay on ports, clocks, pins, cells, or on paths between such objects. Set with the set_min_delay command.

min_fall_slack

Type: float

Returns the worst slack at a port for falling minimum path delays. This attribute is valid for path endpoints (register data pins and primary outputs) after timing is updated. You cannot set this attribute.

min_fanout

Type: float

Returns the minimum fanout design rule limit for a port.

min_rise_delay

Type: float

Returns the minimum rising delay on ports, clocks, pins, cells, or on paths between such objects. Set with the set_min_delay command.

min_rise_slack

Type: float

Returns the worst slack at a port for rising minimum path delays. This attribute is valid for path endpoints (register data pins and primary outputs) after timing is updated. You cannot set this attribute.

min_slack

Type: float

Returns the minimum of the min_rise_slack and min_fall_slack attributes.

min_transition

Type: float

Returns the minimum transition time design rule limit for a port.

net

Type: collection

Returns a collection that contains the net connected to this port; this attribute is defined only if the port is connected to a net.

object_class

Type: string

Returns the class of the object, which is "port". You cannot set this attribute.

pg_level

Type: string

Feedback

Returns PG connectivity status of the port. A return value '1' means the port has a connection to a power PG net or logic constant 1, whereas a return value '0' means the port has a connection to a ground PG net or logic constant 0.

pg_pin_info

Type: collection

Returns a collection of pg_pin_info objects with these attributes: pin_name, type, voltage_for_max_delay, voltage_for_min_delay, supply_connection.

pin_capacitance_max

Type: float

Returns the pin capacitance of a port for maximum conditions (wire capacitance is not included). Set with the set_load command.

pin_capacitance_max_fall

Type: float

Returns the maximum fall capacitance of the port. This attribute is read-only; you cannot change the setting.

pin_capacitance_max_rise

Type: float

Returns the maximum rise capacitance of the port. This attribute is read-only; you cannot change the setting.

pin_capacitance_min

Type: float

Returns the pin capacitance of a port for minimum conditions (wire capacitance is not included). Set with the set_load command.

pin_capacitance_min_fall

Type: float

Returns the minimum fall capacitance of the port. This attribute is read-only; you cannot change the setting.

pin_capacitance_min_rise

Type: float

Returns the minimum rise capacitance of the port. This attribute is read-only; you cannot change the setting.

Feedback

port_direction

Type: string

Returns the direction of a port. Value can be in, out, or inout. This attribute exists for backward compatibility with dc_shell. See the "direction" attribute. You cannot set this attribute.

power_base_clock

Type: string

Returns the name of the base clock associated with this port. If the port belongs to multiple clock domains, the power_base_clock attribute is set to the fastest of the clocks.

power_rail_voltage_max

Type: float

Returns the voltage value on port or pin object for maximum condition.

power_rail_voltage_min

Type: float

Returns the voltage value on port or pin object for minimum condition.

propagated_clock

Type: boolean

Returns true if clock edge times are delayed by propagating the values through the clock network. Affects all sequential cells in the transitive fanout of this port. If this attribute is not present, PrimeTime assumes ideal clocking. Set with set_propagated_clock command.

receiver_model_scaling_libs_max

Type: collection

Returns a collection of library objects used for receiver model scaling, where applicable, for libs maximum analysis.

receiver_model_scaling_libs_min

Type: collection

Returns a collection of library objects used for receiver model scaling, where applicable, for libs minimum analysis.

receiver_model_type_max_fall

Type: string

Feedback

Returns the receiver model type, either basic (NLDM) or advanced (CCS timing), for type maximum fall analysis.

receiver_model_type_max_rise

Type: string

Returns the receiver model type, either basic (NLDM) or advanced (CCS timing), for type maximum rise analysis.

receiver_model_type_min_fall

Type: string

Returns the receiver model type, either basic (NLDM) or advanced (CCS timing), for type minimum fall analysis.

receiver_model_type_min_rise

Type: string

Returns the receiver model type, either basic (NLDM) or advanced (CCS timing), for type minimum rise analysis.

setup_uncertainty

Type: float

Returns the clock uncertainty (skew) of a clock used for setup (and other maximum delay) timing checks. Set with the set_clock_uncertainty command.

si_noise_active_aggressors_above_high

Type: collection

Returns a collection of active aggressor nets for the input port, considering crosstalk noise bumps in the above-high region. An active aggressor is an aggressor that contributes to the worst-case noise bump on the victim net.

si_noise_active_aggressors_above_low

Type: collection

Returns a collection of active aggressor nets for the input port, considering crosstalk noise bumps in the above-low region. An active aggressor is an aggressor that contributes to the worst-case noise bump on the victim net.

si_noise_active_aggressors_below_high

Type: collection

Feedback

Returns a collection of active aggressor nets for the input port, considering crosstalk noise bumps in the below-high region. An active aggressor is an aggressor that contributes to the worst-case noise bump on the victim net.

si_noise_active_aggressors_below_low

Type: collection

Returns a collection of active aggressor nets for the input port, considering crosstalk noise bumps in the below-low region. An active aggressor is an aggressor that contributes to the worst-case noise bump on the victim net.

si_noise_bumps_above_high

Type: string

Returns a string that lists the aggressor nets for the input port and their corresponding coupled bump heights and widths, considering noise bumps in the above-high region. The format of the string is:

```
{{aggr1_name height width}
 {aggr2_name height width}
  ... }
```

Height is in volts and width is in library time units.

si_noise_bumps_above_low

Type: string

Returns a string that lists the aggressor nets for the input port and their corresponding coupled bump heights and widths, considering noise bumps in the above-low region. The format of the string is:

```
{{aggr1_name height width}
 {aggr2_name height width}
  ... }
```

Height is in volts and width is in library time units.

si_noise_bumps_below_high

Type: string

Returns a string that lists the aggressor nets for the input port and their corresponding coupled bump heights and widths, considering noise bumps in the below-high region. The format of the string is:

```
{{aggr1_name height width}
 {aggr2_name height width}
  ... }
```

Feedback

Height is in volts and width is in library time units.

si_noise_bumps_below_low

Type: string

Returns a string that lists the aggressor nets for the input port and their corresponding coupled bump heights and widths, considering noise bumps in the below-low region. The format of the string is:

```
{{aggr1_name height width}
 {aggr2_name height width}
  ... }
```

Height is in volts and width is in library time units.

si_noise_height_factor_above_high

Type: float

Returns the noise height derating factor for the port, in the above-high region, as set by the *set_noise_derate* command.

si_noise_height_factor_above_low

Type: float

Returns the noise height derating factor for the port, in the above-low region.

si_noise_height_factor_below_high

Type: float

Returns the noise height derating factor for the port, in the below-high region.

si_noise_height_factor_below_low

Type: float

Returns the noise height derating factor for the port, in the below-low region.

si_noise_height_offset_above_high

Type: float

Returns the noise height offset for the port, in the above-high region, as set by the *set_noise_derate* command.

si_noise_height_offset_above_low

Type: float

Returns the noise height offset for the port, in the above-low region.

Feedback

si_noise_height_offset_below_high

Type: float

Returns the noise height offset for the port, in the below-high region.

si_noise_height_offset_below_low

Type: float

Returns the noise height offset for the port, in the below-low region.

si_noise_prop_bumps_above_high

Type: string

Returns a string that shows the bump height and width at the input port caused by noise propagation, in the above-high region. The format of the string is: {*height width*}. Height is in volts and width is in library time units.

si_noise_prop_bumps_above_low

Type: string

Returns a string that shows the bump height and width at the input port caused by noise propagation, in the above-low region. The format of the string is: {*height width*}. Height is in volts and width is in library time units.

si_noise_prop_bumps_below_high

Type: string

Returns a string that shows the bump height and width at the input port caused by noise propagation, in the below-high region. The format of the string is: {*height width*}. Height is in volts and width is in library time units.

si_noise_prop_bumps_below_low

Type: string

Returns a string that shows the bump height and width at the input port caused by noise propagation, in the below-low region. The format of the string is: {*height width*}. Height is in volts and width is in library time units.

si_noise_slack_above_high

Type: float

Returns the amount of noise slack for the port in the above-high region.

si_noise_slack_above_low

Type: float

Feedback

Returns the amount of noise slack for the port in the above-low region.

si_noise_slack_below_high

Type: float

Returns the amount of noise slack for the port in the below-high region.

si_noise_slack_below_low

Type: float

Returns the amount of noise slack for the port in the below-low region.

si_noise_total_bump_above_high

Type: string

Returns a string that shows the total bump height and width at the input port caused by crosstalk and noise propagation, in the above-high region. The format of the string is: {*height width*}. Height is in volts and width is in library time units.

si_noise_total_bump_above_low

Type: string

Returns a string that shows the total bump height and width at the input port caused by crosstalk and noise propagation, in the above-low region. The format of the string is: {*height width*}. Height is in volts and width is in library time units.

si_noise_total_bump_below_high

Type: string

Returns a string that shows the total bump height and width at the input port caused by crosstalk and noise propagation, in the below-high region. The format of the string is: {*height width*}. Height is in volts and width is in library time units.

si_noise_total_bump_below_low

Type: string

Returns a string that shows the total bump height and width at the input port caused by crosstalk and noise propagation, in the below-low region. The format of the string is: {*height width*}. Height is in volts and width is in library time units.

si_noise_width_factor_above_high

Type: float

Returns the noise width derating factor for the port, in the above-high region.

Feedback

si_noise_width_factor_above_low

Type: float

Returns the noise width derating factor for the port, in the above-low region.

si_noise_width_factor_below_high

Type: float

Returns the noise width derating factor for the port, in the below-high region.

si_noise_width_factor_below_low

Type: float

Returns the noise width derating factor for the port, in the below-low region.

si_noise_worst_prop_arc_above_high

Type: collection

Returns the cell arc that corresponds to the worst noise propagation to an input port with a driving cell.

si_noise_worst_prop_arc_above_low

Type: collection

Returns the cell arc that corresponds to the worst noise propagation to an input port with a driving cell.

si_noise_worst_prop_arc_below_high

Type: collection

Returns the cell arc that corresponds to the worst noise propagation to an input port with a driving cell.

si_noise_worst_prop_arc_below_low

Type: collection

Returns the cell arc that corresponds to the worst noise propagation to an input port with a driving cell.

temperature_max

Type: float

Returns the maximum temperature specified for the cell through the operating condition specification or application of the set_temperature command.

Feedback

temperature_min

Type: float

Returns the minimum temperature specified for the cell through the operating condition specification or application of the set_temperature command.

user_case_value

Type: string

Returns the user-specified logic value of a pin or port.

voltage_source

Type: string

Returns a keyword that indicates the source of the port's supply voltage value:

- *CONNECTED_PAD_PIN* - pad pin connected to port
- *DRIVING_CELL* - set_driving_cell specification
- *DEFAULT_FROM_DOMAIN* - default of top-level power domain

wire_capacitance_max

Type: float

Returns the wire capacitance of the port for maximum conditions (pin capacitance is not included). Set with the set_load command.

wire_capacitance_max_fall

Type: float

Returns the maximum fall wire capacitance of the net. This attribute is read-only; you cannot change the setting.

wire_capacitance_max_rise

Type: float

Returns the maximum rise wire capacitance of the net. This attribute is read-only; you cannot change the setting.

wire_capacitance_min

Type: float

Returns the wire capacitance of the port for minimum conditions (pin capacitance is not included). Set with the set_load command.

Feedback

wire_capacitance_min_fall

Type: float

Returns the minimum fall wire capacitance of the net. This attribute is read-only; you cannot change the setting.

wire_capacitance_min_rise

Type: float

Returns the minimum rise wire capacitance of the net. This attribute is read-only; you cannot change the setting.

wire_load_model_max

Type: string

Returns the name of a wire load model (for maximum conditions) that can be used for prelayout wire load estimation of the net connected to a port. Set with the set_wire_load_model command.

wire_load_model_min

Type: string

Returns the name of a wire load model (for minimum conditions) that can be used for prelayout wire load estimation of the net connected to a port. Set with the set_wire_load_model command.

x_coordinate

Type: float

Returns the x-coordinate of the port.

y_coordinate

Type: float

Returns the y-coordinate of the port.

See Also

- get_attribute
- help_attributes
- list_attributes
- report_attribute
- attributes

Feedback

r

resistor_attributes

Describes the predefined application attributes for parasitic resistor objects.

Description

area

Type: float

Returns the via area in square microns (does not apply to metal resistors); populated only if the *is_via* attribute is true.

is_short

Type: Boolean

Returns true if the resistor is a shorting resistor.

is_via

Type: Boolean

Returns true if the resistor is part of a via.

is_via_array

Type: Boolean

Returns true if the resistor is part of a via array.

is_via_ladder_em

Type: Boolean

Returns true if the resistor is associated with a via ladder in an NDM format IC Compiler II database that has the *is_electromigration* attribute.

is_via_ladder_high_performance

Type: Boolean

Returns true if the resistor is associated with a via ladder in an NDM format IC Compiler II database that has the *is_high_performance* attribute.

layer_id

Type: integer

Feedback

Returns the ITF (nxtgrd file) layer ID. If resistor detail is not available in the GPD, the layer_id and layer_name attributes are estimated using the associated ground capacitance layers.

layer_name

Type: integer

Returns the ITF (nxtgrd file) layer name. If resistor detail is not available in the GPD, the layer_id and layer_name attributes are estimated using the associated ground capacitance layers.

length

Type: float

Returns the metal resistor length in microns (does not apply to vias); populated only when *report_gpd_properties* reports the GPD property "has_resistor_detail Yes."

net

Type: collection

Returns the net that contains the parasitic resistor.

node1_ground_capacitor

Type: collection

Returns the ground capacitor associated with node1 of the resistor.

node1_index

Type: integer

Returns the index value of node1, one of two nodes at which the parasitic resistor connects to the net. Each node on a net has a unique index from 1 to N, where N is the total number of nodes on that net.

node1_name

Type: string

Returns the name of node1 in the format used in a SPEF output file.

node2_ground_capacitor

Type: collection

Returns the ground capacitor associated with node1 of the resistor.

Feedback

node2_index

Type: integer

Returns the index value of node2, one of two nodes at which the parasitic resistor connects to the net. Each node on a net has a unique index from 1 to N, where N is the total number of nodes on that net.

node2_name

Type: string

Returns the name of node2 in the format used in a SPEF output file.

resistance

Type: string

Returns a single-corner resistance value in the format used in a SPEF output file.

resistance_max

Type: string

Returns the maximum value of the list in the resistance attribute.

resistance_min

Type: string

Returns the minimum value of the list in the resistance attribute.

resistance_multicorner

Type: string

If data from multiple corners is retrieved, the attribute contains a list of the resistances of the corners specified by the *-parasitic_corners* option, in that order.

via_array_nx

Type: integer

Returns the number of vias in the x-direction for a via array. Populated only if the is_via_array attribute is true.

via_array_ny

Type: integer

Returns the number of vias in the y-direction for a via array. Populated only if the is_via_array attribute is true.

Feedback

via_array_perimeter

Type: float

Returns the perimeter of a via array in microns. Populated only if the is_via_array attribute is true.

width

Type: float

Returns the metal resistor width in microns (does not apply to vias); populated only when *report_gpd_properties* reports the GPD property "has_resistor_detail Yes."

x_coordinate_min

Type: float

Returns the lower-left x-coordinate (in microns) of the resistor bounding box, if the bounding box information is available in the GPD.

x_coordinate_max

Type: float

Returns the upper-right x-coordinate (in microns) of the resistor bounding box, if the bounding box information is available in the GPD.

y_coordinate_min

Type: float

Returns the lower-left y-coordinate (in microns) of the resistor bounding box, if the bounding box information is available in the GPD.

y_coordinate_max

Type: float

Returns the upper-right y-coordinate (in microns) of the resistor bounding box, if the bounding box information is available in the GPD.

See Also

- get_attribute
- help_attributes
- list_attributes
- report_attribute
- attributes

Feedback

s

scenario_attributes

Describes the predefined application attributes for scenario objects.

Description

name

Type: string

Returns the name of the scenario object.

See Also

- get_attribute
- help_attributes
- list_attributes
- report_attribute
- attributes

t

timing_arc_attributes

Describes the predefined application attributes for timing_arc objects.

Description

annotated_delay_delta_max

Type: float

Returns the maximum of the annotated_delay_delta_max_rise and annotated_delay_delta_max_fall attributes.

annotated_delay_delta_max_fall

Type: float

Feedback

Returns the delay that is added to the maximum falling delay of the timing arc. The additional delay is set by either the set_annotated_delay -delta_only command or by PrimeTime SI during crosstalk analysis.

annotated_delay_delta_max_rise

Type: float

Returns the delay that is added to the maximum rising delay of the timing arc. The additional delay is set by either the set_annotated_delay -delta_only command or by PrimeTime SI during crosstalk analysis.

annotated_delay_delta_min

Type: float

Returns the minimum of the annotated_delay_delta_min_rise and annotated_delay_delta_min_fall attributes.

annotated_delay_delta_min_fall

Type: float

Returns the delay that is added to the minimum falling delay of the timing arc. The additional delay is set by either the set_annotated_delay -delta_only command or by PrimeTime SI during crosstalk analysis.

annotated_delay_delta_min_rise

Type: float

Returns the delay that is added to the minimum rising delay of the timing arc. The additional delay is set by either the set_annotated_delay -delta_only command or by PrimeTime SI during crosstalk analysis.

delay_max

Type: float

Returns the maximum of the delay_max_rise and delay_max_fall attributes.

delay_max_fall

Type: float

Returns the maximum falling delay of the timing arc.

delay_max_rise

Type: float

Returns the maximum rising delay of the timing arc.

Feedback

delay_min

Type: float

Returns the minimum of the delay_min_rise and delay_min_fall attributes.

delay_min_fall

Type: float

Returns the minimum falling delay of the timing arc.

delay_min_rise

Type: float

Returns the minimum rising delay of the timing arc.

from_pin

Type: collection

Returns a collection containing the from pin of the timing arc.

is_annotated_fall_max

Type: boolean

Returns true if the maximum falling delay of the timing arc is back-annotated.

is_annotated_fall_min

Type: boolean

Returns true if the minimum falling delay of the timing arc is back-annotated.

is_annotated_rise_max

Type: boolean

Returns true if the maximum rising delay of the timing arc is back-annotated.

is_annotated_rise_min

Type: boolean

Returns true if the minimum rising delay of the timing arc is back-annotated.

is_cellarc

Type: boolean

Returns true if the timing arc is a cell arc, and false for net arcs.

Feedback

is_constraint_scaled_max_fall

Type: boolean

Returns true if the constraint timing arc uses scaling between libraries for the driver model for maximum fall analysis.

is_constraint_scaled_max_rise

Type: boolean

Returns true if the constraint timing arc uses scaling between libraries for the driver model for maximum rise analysis.

is_constraint_scaled_min_fall

Type: boolean

Returns true if the constraint timing arc uses scaling between libraries for the driver model for minimum fall analysis.

is_constraint_scaled_min_rise

Type: boolean

Returns true if the constraint timing arc uses scaling between libraries for the driver model for minimum rise analysis.

is_db_inherited_disabled

Type: boolean

Returns true if the arc is a .db database-inherited disabled arc. Such an arc has been disabled for loop breaking by upstream tools. To be consistent, the PrimeTime tool disables these same arcs if the timing_keep_loop_breaking_disabled_arcs variable is set to true.

is_disabled

Type: boolean

Returns true if the timing arc is disabled.

is_user_disabled

Type: boolean

Returns true if the timing arc is disabled by using the set_disable_timing command.

mode

Type: string

Feedback

Returns the mode string of the timing arc.

object_class

Type: string

Returns the class of the object, which is the string "timing_arc". You cannot set this attribute.

sdf_cond

Type: string

Returns the SDF condition of the timing arc.

sdf_cond_end

Type: string

Returns the SDF condition at the endpoint of the timing arc. Variable sdf_enable_cond_start_end must be true.

sdf_cond_start

Type: string

Returns the SDF condition at the startpoint of the timing arc. Variable sdf_enable_cond_start_end must be true.

sense

Type: string

Returns the sense of the timing arc.

to_pin

Type: collection

Returns a collection containing the "to" pin of the timing arc.

when

Type: string

Returns the "when" string of the timing arc.

See Also

- get_attribute
- help_attributes

Feedback

- list_attributes
- report_attribute
- attributes

---

timing_path_attributes

Describes the predefined application attributes for *timing_path* objects.

Description

arrival

Type: float

Returns the arrival time at the endpoint of the timing path; excludes the *startpoint_clock_open_edge_value* value by default unless the *timing_path_arrival_required_attribute_include_clock_edge* variable is set to *true*.

capture_clock_paths

Type: collection

Returns timing path collections for the capture clock. The paths correspond to the output of the *report_timing -path_type full_clock_expanded* command. If the capturing register is clocked by a regular clock, the attribute returns only one path in the collection. If it is a generated clock, the first path in the collection is the master clock path, followed by each dependent generated clock, until the register's clock pin is reached.

clock_jitter

Type: float

Returns the clock jitter of the timing path. Clock jitter can be defined by the *set_clock_jitter* command. The type of jitter returned (full-cycle or duty-cycle) depends on which type applies to the path according to its clock edge polarities (same or opposite).

clock_uncertainty

Type: float

Returns the clock uncertainty of the timing path. The uncertainty can be defined with the *set_clock_uncertainty* command.

close_edge_adjustment

Type: float

Returns the sum of the recovery amounts along the path.

Feedback

common_path_pessimism

Type: float

Returns the clock reconvergence common path pessimism. This attribute is defined only if you are using OCV analysis and the *timing_remove_clock_reconvergence_pessimism* variable is set to *true* to enable the clock reconvergence pessimism removal (CRPR) feature.

crpr_common_point

Type: collection

Returns the clock network pin that is the common pin used for clock reconvergence pessimism calculation for the path. If the launch or capture clock is ideal, the clock source is returned as the common point. If the launch clock is different from the capture clock, this attribute does not exist on the path.

depth_cell_capture

Type: float

Returns the advanced on-chip variation (AOCV) calculated depth for cells in a capture path.

depth_cell_launch

Type: float

Returns the advanced on-chip variation (AOCV) calculated depth for cells in a launch path.

depth_net_capture

Type: float

Returns the advanced on-chip variation (AOCV) calculated depth for nets in a capture path.

depth_net_launch

Type: float

Returns the advanced on-chip variation (AOCV) calculated depth for nets in a launch path.

distance_cell

Type: float

Returns the AOCV calculated distance for cells in a path.

distance_net

Type: float

Feedback

Returns the AOCV calculated distance for nets in a path.

dominant_exception

Type: string

Returns the type of the dominant exception for the path: *false_path*, *multicycle_path*, or *min_max_delay*. This attribute exists only if the path has at least one timing exception. For more information, see the description of the *-exceptions* option of the *report_timing* command.

dominant_metal_layer

Type: string

Returns a string of metal layers names having maximum length for the timing path.

endpoint

Type: collection

Returns the endpoint of the timing path, corresponding to the endpoint in the header of a timing report.

endpoint_clock

Type: collection

Returns the name of the clock at the path endpoint.

endpoint_clock_close_edge_type

Type: string

Returns the type of clock edge (rise or fall) that closes (latches) the data.

endpoint_clock_close_edge_value

Type: float

Returns the clock edge time for the endpoint.

endpoint_clock_is_inverted

Type: boolean

Returns *true* if the endpoint clock has been inverted.

endpoint_clock_is_propagated

Type: boolean

Returns *true* if the endpoint clock is a propagated clock, or *false* if it is an ideal clock. You can set a clock as propagated by using the *set_propagated_clock* command.

Feedback

endpoint_clock_latency

Type: float

Returns the capture clock arrival of the timing path, excluding the endpoint clock edge time (*endpoint_clock_close_edge_value* attribute).

endpoint_clock_open_edge_type

Type: string

Returns the type of clock edge (*rise* or *fall*) that opens the endpoint latch. If the endpoint is edge-triggered, the open and close edges are the same.

endpoint_clock_open_edge_value

Type: float

Returns the opening clock edge time of the endpoint clock.

endpoint_clock_pin

Type: collection

Returns the complete path name of the endpoint clock pin, for example, U23/U_reg/out_reg[2]/CP.

endpoint_hold_time_value

Type: float

Returns the value of the register hold time at the timing endpoint. For example, for a flip-flop, this is the library hold time for the flip-flop cell.

endpoint_is_level_sensitive

Type: boolean

Returns *true* if the endpoint is a level-sensitive device such as a latch, or *false* if the endpoint is edge-triggered.

endpoint_output_delay_value

Type: float

Returns the value of the output delay of the timing endpoint. You can set the output delay value with the *set_output_delay* command.

endpoint_recovery_time_value

Type: float

Feedback

Returns the value of the recovery time at the timing endpoint. Recovery and removal times are often defined for the asynchronous set/clear pins of registers.

endpoint_removal_time_value

Type: float

Returns the value of the removal time at the timing endpoint. Recovery and removal times are often defined for the asynchronous set/clear pins of registers.

endpoint_setup_time_value

Type: float

Returns the value of the register setup time at the timing endpoint. For example, for a flip-flop, this is the library setup time for the flip-flop cell.

endpoint_unconstrained_reason

Type: string

Returns the reason for an unconstrained endpoint: *no_capture_clock*, *dangling_end_point*, *fanin_of_disabled*, *no_max_check*, *no_min_check*, or *endpoint_unconstrained_reason*. This attribute exists only if the path endpoint is unconstrained.

exception_delay

Type: float

Returns the value of the minimum or maximum delay timing exception that applies to the path, if any. If the path is not constrained by either a *set_min_delay* or *set_max_delay* command, the return value is *UNINIT*.

exception_shift

Type: float

Returns the value of the multicycle path timing exception that applies to the path, if any. If the path is not constrained by a *set_multicycle_path* command, the return value is UNINIT.

ideal_path_ratio

Type: double

Returns the ratio of the actual path length to the ideal path length. The ideal path length is calculated as the Manhattan distance between the start pin and the end pin.

ideal_route_ratio

Type: double

Feedback

Returns the ratio of the actual route length to the ideal route length. The ideal route length is calculated as the Manhattan pin-to-pin distance along the timing path.

is_recalculated

Type: boolean

Returns *true* if the timing information for the path comes from path-based (recalculated) timing analysis.

is_recovered

Type: boolean

Returns *true* if one of the latches in the timing path arrived after the closing edge, and there is a violation at the latch.

launch_clock_paths

Type: collection

Returns timing path collections for the launch clock. The paths correspond to the output of the *report_timing -path_type full_clock_expanded* command. If the launching register is clocked by a regular clock, the attribute returns only one path in the collection. If it is a generated clock, the first path in the collection is the master clock path, followed by each dependent generated clock, until the register's clock pin is reached.

max_time_borrow_from_endpoint

Type: float

Returns the maximum time that could be potentially borrowed at the endpoint of the path, which is the most restrictive of the following values:

- Closing edge
- Value specified by the *set_max_time_borrow* command

The attribute returns

- Zero if the endpoint does not end in a latch
- Corner value of the quantity if the design uses parametric on-chip variation (POCV)

mim_pessimism

Type: float

Returns the value of the multiply instantiated module (MIM) pessimism. This attribute is defined only if you applied a merged HyperScale binary context with MIM information in the current analysis.

Feedback

netshapes

Type: collection

Returns the netshapes belonging to the timing path.

normalized_slack

Type: float

Returns the normalized slack of the timing path.

normalized_slack_no_close_edge_adjustment

Type: float

Returns a value equal to *slack_no_close_edge_adjustment* divided by the allowed propagation delay for the path. This attribute is available only if the *timing_enable_normalized_slack* variable is set to *true*.

number_of_vias

Type: integer

Returns the total number of vias for a timing path.

object_class

Type: string

Returns the class of the object, which is the string "*timing_path*".

path_group

Type: collection

Returns the path group of the timing path.

path_type

Type: string

Returns the type of timing path, either maximum or minimum. For a setup check, it is *maximum*. For a hold check, it is *minimum*.

points

Type: collection

Returns a collection of the timing points that comprise the timing path, corresponding to the timing points in the left column of a report generated by the *report_timing* command. A single timing path can contain many timing points. You can iterate through the point collection by using the *foreach_in_collection* command.

Feedback

required

Type: float

Returns the required time at the endpoint of the timing path; excludes the *endpoint_clock_close_edge_value* value by default unless the *timing_path_arrival_required_attribute_include_clock_edge* variable is set to *true*.

route_length

Type: double

Returns the total route length of the timing path.

session_name

Type: string

Returns the name of the saved session that the timing path belongs to. This attribute is applicable only to the interactive multi-scenario analysis (IMSA) flow.

signature

Type: string

Returns the signature of the timing path.

slack

Type: float

Returns the slack of the timing path, corresponding to the slack of a timing report. A negative value indicates a path with a timing violation.

slack_no_close_edge_adjustment

Type: float

Returns the endpoint slack of the path, without considering the recoveries along the path. The value matches the slack attribute when there are no recoveries on the path.

startpoint

Type: collection

Returns the startpoint of the timing path, corresponding to the startpoint in the header of a timing report.

startpoint_clock

Type: collection

Returns the name of the clock at the path startpoint.

Feedback

startpoint_clock_is_inverted

Type: boolean

Returns *true* if the startpoint clock is inverted.

startpoint_clock_is_propagated

Type: boolean

Returns *true* if the startpoint clock is a propagated clock, or *false* if it is an ideal clock. You can set a clock as propagated by using the *set_propagated_clock* command.

startpoint_clock_latency

Type: float

Returns the launch clock arrival of the timing path, excluding the startpoint clock edge time (*startpoint_clock_open_edge_value* attribute).

startpoint_clock_open_edge_type

Type: string

Returns the type of clock edge (*rise* or *fall*) that launches the data.

startpoint_clock_open_edge_value

Type: float

Returns the clock edge time for the startpoint.

startpoint_input_delay_value

Type: float

Returns the value of the startpoint input delay.

startpoint_is_level_sensitive

Type: boolean

Returns *true* if the startpoint is a level-sensitive device such as a latch, or *false* if the startpoint is edge-triggered.

startpoint_unconstrained_reason

Type: string

Returns the reason for an unconstrained startpoint: *no_capture_clock*, *dangling_end_point*, *fanin_of_disabled*, *no_max_check*, *no_min_check*, or *endpoint_unconstrained_reason*. This attribute exists only if the path startpoint is unconstrained.

Feedback

statistical_adjustment

Type: float

Returns the statistical adjustment of the path. In POCV paths where both the arrival and required time include statistical variation, the slack corner of the path will not be the difference between required corner and arrival corner; this follows from the fact that standard deviations of statistical quantities do not combine as a simple sum, but instead as a square root of sum of squares. The statistical adjustment of the path is the difference between the slack corner of the path on the one hand and the difference between required corner and arrival corner on the other.

time_borrowed_from_endpoint

Type: float

Returns the amount of time borrowed from the timing endpoint. Time borrowing occurs in paths with level-sensitive devices.

time_lent_to_startpoint

Type: float

Returns the amount of time lent to the timing startpoint. Time borrowing occurs in paths with level-sensitive devices.

transparent_latch_paths

Type: collection

Returns the chain of "upstream" borrowing paths that lead up to the borrowing startpoint for paths with a transparent latch D-pin startpoint. To use the attribute, you must gather the path using the *-trace_latch_borrow* option.

variation_arrival

Type: collection

Returns the arrival time variation of the path.

variation_common_path_pessimism

Type: collection

Returns the variation of the clock reconvergence common path pessimism value.

variation_endpoint_clock_latency

Type: collection

Returns the capture-clock arrival time variation.

Feedback

variation_endpoint_hold_time_value

Type: collection

Returns the variation of the register hold time at the timing endpoint.

variation_endpoint_recovery_time_value

Type: collection

Returns the variation of the recovery time at the timing endpoint.

variation_endpoint_removal_time_value

Type: collection

Returns the variation of the removal time at the timing endpoint.

variation_endpoint_setup_time_value

Type: collection

Returns the variation of the register setup time at the timing endpoint.

variation_required

Type: collection

Returns the required time variation of the path.

variation_slack

Type: collection

Returns the slack variation of the path.

variation_startpoint_clock_latency

Type: collection

Returns the launch clock arrival time variation.

vias

Type: collection

Returns the vias belonging to the timing path.

See Also

- get_attribute
- help_attributes

Feedback

- list_attributes
- report_attribute
- attributes

---

timing_point_attributes

Describes the predefined application attributes for timing_point objects.

Description

annotated_delay_delta

Type: float

Returns the delta delay in the timing point.

annotated_delta_transition

Type: float

Returns the delta transition in the timing point.

aocvm_coefficient

Type: float

Returns the AOCV coefficient calculated for the arc to the timing point.

applied_derate

Type: float

Returns the applied derating calculated for the arc to the timing point.

arrival

Type: float

Returns the arrival time at the timing point without accounting for the following:

- Clock latency to the startpoint clock
- Time lent to the startpoint (due to latch borrowing)
- Input delay
- The *startpoint_clock_open_edge_value* attribute

The *timing_point_arrival_attribute_compatibility* variable controls whether the arrival value cumulatively includes the delay of previous path segments. For details, see the man page.

Feedback

depth

Type: float

Returns the depth for the pin or port of the timing point. This value is scaled by AOCV coefficients if they exist.

derate_factor_depth_distance

Type: float

Returns the derating factor as a function of depth and distance calculated for the arc to the timing point.

distance

Type: float

Returns the distance for the pin or port of the timing point.

guardband

Type: float

Returns the guard band calculated for the arc to the timing point.

incremental

Type: float

Returns the incremental value calculated for the arc to the timing point.

object

Type: collection

Returns the object at this point in the timing path.

object_class

Type: string

Returns the class of the object, which is the string "timing_point". You cannot set this attribute.

path_phase

Type: string

Returns the type of path segment containing the point: *clock_launch*, *clock_capture*, or *data*.

Feedback

rise_fall

Type: string

Returns "rise" if the timing point is a rising-edge delay or "fall" if it is a falling-edge delay.

si_xtalk_bumps

Type: string

Returns the crosstalk bump at the timing point, listing each aggressor net and the voltage bumps that rising and falling aggressor transitions induce on the victim net (worst of rising and falling minimum or maximum bumps, each expressed as a decimal fraction of the rail-to-rail voltage); or gives a reason why the aggressor net has no effect on the victim net.

slack

Type: float

Returns the slack value at the timing point.

timing_arc

Type: collection

Returns the timing arc at the timing point.

transition

Type: float

Returns the transition value at the timing point.

variation_arrival

Type: collection

Returns the arrival time variation of the timing point.

variation_increment

Type: collection

Returns the incremental variation of the timing point.

variation_slack

Type: collection

Returns the slack time variation of the timing point.

variation_transition

Type: collection

Feedback

Returns the transition time variation of the timing point.

voltage

Type: float

Returns the voltage level of the timing point.

x_coordinate

Type: float

Returns the x coordinate of the physical location of the timing point.

y_coordinate

Type: float

Returns the y coordinate of the physical location of the timing point.

See Also

- get_attribute
- help_attributes
- list_attributes
- report_attribute
- attributes

u

upf_attributes

Describes the predefined application attributes for IEEE 1801, also known as Unified Power Format (UPF).

Description

UPF-related attributes exist on the following object classes:

- upf_power_domain
- upf_power_switch
- upf_supply_net

Feedback

- upf_supply_port
- upf_supply_set

See Also

- get_attribute
- help_attributes
- list_attributes
- report_attribute
- attributes
- upf_power_domain_attributes
- upf_power_switch_attributes
- upf_supply_net_attributes
- upf_supply_port_attributes
- upf_supply_set_attributes

---

upf_power_domain_attributes

Describes the predefined application attributes for upf_power_domain objects.

Description

default_isolation_supply

Type: collection

Returns the UPF default isolation supply set handle associated with the power domain.

default_retention_supply

Type: collection

Returns the UPF default retention supply set handle associated with the power domain.

full_name

Type: string

Returns the hierarchical name of the UPF power domain.

name

Type: string

Feedback

Returns the name of the UPF power domain.

primary_supply

Type: collection

Returns the UPF primary supply set handle associated with the power domain.

scope

Type: string

Returns the scope of the power domain, which is the level of logic hierarchy designated as the root of the domain.

supplies

Type: string

Returns the function keyword and supply set name for each supply of the power domain, for example, {primary TOP.primary} {default_retention TOP.default_retention} {default_isolation TOP.default_isolation}

See Also

- get_attribute
- help_attributes
- list_attributes
- report_attribute
- attributes
- upf_attributes

---

upf_power_switch_attributes

Describes the predefined application attributes for upf_power_switch objects.

Description

control_net

Type: collection

Returns the control net.

control_port_name

Type: string

Feedback

Returns the control port name.

domain

Type: collection

Returns the domain.

full_name

Type: string

Returns the hierarchical name of the UPF power switch.

input_supply_net

Type: collection

Returns the input supply net.

input_supply_port_name

Type: string

Returns the name of the input supply port.

name

Type: string

Returns the name of the UPF power switch.

on_state

Type: string

Returns a list in the following format: state_name input_supply_port boolean function.

output_supply_net

Type: collection

Returns the output supply net.

output_supply_port_name

Type: string

Returns the name of the output supply port.

Feedback

See Also

- get_attribute
- help_attributes
- list_attributes
- report_attribute
- attributes
- upf_attributes

---

upf_supply_net_attributes

Describes the predefined application attributes for upf_supply_net objects.

Description

domains

Type: collection

Returns the domains of the supply net.

full_name

Type: string

Returns the hierarchical name of the supply net.

name

Type: string

Returns the simple name of the supply net.

resolve

Type: string

Returns the resolution when the UPF supply net is driven by multiple power switches: unresolved, parallel, one_hot, or parallel_one_hot.

static_prob

Type: float

Returns the probability of logic 1 (on) for power analysis.

Feedback

supply_group

Type: collection

Returns the supply group to which the supply net belongs.

voltage_max

Type: float

Returns the maximum-delay operating voltage of the supply net.

voltage_min

Type: float

Returns the minimum-delay operating voltage of the supply net.

See Also

- get_attribute
- help_attributes
- list_attributes
- report_attribute
- attributes
- upf_attributes

---

upf_supply_port_attributes

Describes the predefined application attributes for upf_supply_port objects.

Description

direction

Type: string

Returns the port direction.

domain

Type: collection

Returns the domain.

full_name

Type: string

Feedback

Returns the hierarchical name of the UPF supply port.

name

Type: string

Returns the simple name of the UPF supply port.

scope

Type: string

Returns the scope of the UPF supply port, which is the level of logic hierarchy designated as the root of the supply port's domain.

See Also

- get_attribute
- help_attributes
- list_attributes
- report_attribute
- attributes
- upf_attributes

---

upf_supply_set_attributes

Describes the predefined application attributes for upf_supply_set objects.

Description

full_name

Type: string

Returns the hierarchical name of the UPF supply set.

ground

Type: collection

Returns the ground net of the supply set.

name

Type: string

Returns the name of the UPF supply set.

Feedback

power

Type: collection

Returns the power net of the supply set.

See Also

- get_attribute
- help_attributes
- list_attributes
- report_attribute
- attributes
- upf_attributes

V

---

variation_attributes

Describes the predefined application attributes for variation objects.

Description

full_name

Type: string

Returns the full name of the variation object.

mean

Type: float

Returns the mean of the distribution of the variation.

object_class

Type: string

Returns the class of the object, which is a constant equal to variation. You cannot set this attribute.

skewness

Type: float

Feedback

Returns the skewness of the distribution of the variation.

std_dev

Type: float

Returns the standard deviation of the distribution of the variation.

summary

Type: string

Returns information about the distribution of the variation, including mean, standard deviation, and skewness.

variance

Type: float

Returns the variance of the distribution of the variation.

See Also

- get_attribute
- help_attributes
- list_attributes
- report_attribute
- attributes